XCPC Template Library

Morning Flower
and Evening Oath

Tongji University

目录

第一部分　搜索 **1**

第一章　Dancing Links **2**
1.1　精确覆盖（Dancing Links） . . . 2
1.2　最少行重复覆盖（Dancing Links） . . . 6

第二部分　动态规划 **10**

第二章　插头 DP **11**
2.1　插头 DP：网格单闭环计数 . . . 11
2.2　插头 DP：恰好 K 个不嵌套回路 . . . 16

第三章　动态 DP **20**
3.1　动态 DP：树上最大权独立集带修改 . . . 20

第四章　DP 优化 **24**
4.1　斜率优化 . . . 24
4.1.1　单调斜率凸包：最小值与决策 . . . 24
4.2　四边形不等式优化 . . . 27
4.2.1　定义与性质 . . . 27
4.2.2　分组 DP 的分治优化 . . . 27
4.2.3　分治决策单调性：一层分组 DP . . . 28
4.2.4　区间 DP 的 Knuth 优化 . . . 29
4.2.5　Knuth 优化：区间合并 DP . . . 31
4.3　WQS 二分 . . . 33
4.3.1　WQS 二分：至多 k 个不相邻位置 . . . 33

第三部分　字符串 **34**

第五章　字符串哈希 **35**
5.1　子串与反串双哈希 . . . 35

第六章　前缀函数与 KMP 算法 **38**
6.1　前缀函数 . . . 38
6.2　KMP 匹配 . . . 39

第七章　Z 函数（扩展 KMP） **40**
7.1　Z 函数 . . . 40
7.2　两串扩展 KMP . . . 41

第八章　AC 自动机 **43**
8.1　AC 自动机出现次数 . . . 43

第九章 后缀数组 (SA) **47**
9.1 后缀数组简介 . . . 47
9.1.1 线性后缀数组（SA-IS） . . . 47
9.1.2 倍增后缀数组与 LCP . . . 52
9.1.3 后缀 LCP 与子串比较 . . . 54
9.1.4 整数后缀数组的边界与 SAM 次数 . . . 55

第十章 后缀自动机 (SAM) **56**
10.1 后缀自动机 . . . 56
10.2 SAM 字典序子串查询 . . . 58
10.3 SAM 最长公共子串 . . . 61

第十一章 广义后缀自动机 **62**
11.1 广义后缀自动机：Trie BFS 构建 . . . 62
11.2 SAM 文档频率与阈值后缀 . . . 65
11.3 广义 SAM 点数与后缀语言 . . . 68

第十二章 Manacher **69**
12.1 Manacher 回文半径 . . . 69
12.2 回文半径与在线回文计数 . . . 70

第十三章 回文树 **71**
13.1 回文树 PAM：回文后缀与出现次数 . . . 71

第十四章 最小表示法 **74**
14.1 最小循环表示 . . . 74

第十五章 Lyndon 分解 **76**
15.1 Lyndon 分解：Duval . . . 76

第四部分 数学 **78**

第十六章 高精度计算 **79**
16.1 高精度整数：有符号加减乘与小整数除法 . . . 79

第十七章 快速幂 **85**
17.1 无符号 64 位模乘与模幂 . . . 85

第十八章 数论 **86**
18.1 模算术简介 . . . 86
18.1.1 动态模整数与合数模逆元 . . . 86
18.1.2 静态模整数 . . . 90
18.1.3 模运算的适用条件 . . . 93
18.1.3.1 除法与解同余式不是一回事 . . . 93
18.1.3.2 知道整数可整除时，先升模数再整除 . . . 93
18.1.3.3 组合数的整数性不等于分母可逆 . . . 93
18.1.3.4 大指数要同时保留余数和阈值信息 . . . 93
18.2 素数 . . . 94
18.2.1 确定性 64 位 Miller–Rabin 判素 . . . 94
18.2.2 64 位判素与质因数分解 . . . 95

18.3 欧拉函数 . . . 96
18.3.1 单个数的欧拉函数 . . . 96
18.3.2 单个数的欧拉函数 . . . 96
18.4 筛法 . . . 97
18.4.1 区间筛素数 . . . 97
18.4.2 线性筛、欧拉函数与 Möbius 函数 . . . 98
18.4.3 线性筛：三张表的约定 . . . 98
18.4.4 区间筛：只保存目标区间 . . . 99
18.5 分解质因数 . . . 99
18.5.1 Pollard–Rho 因数分解 . . . 99
18.6 裴蜀定理 & 一次不定方程 . . . 101
18.6.1 扩展欧几里得与 Bézout 系数 . . . 101
18.6.2 一次不定方程：特解与正整数解范围 . . . 102
18.6.2.1 有符号矩形范围：把两维约束化成一个参数区间 . . . 102
18.7 费马小定理 & 欧拉定理 . . . 103
18.7.1 十进制大指数：扩展欧拉降幂 . . . 103
18.7.2 Carmichael：最小公共指数 . . . 104
18.8 模逆元 . . . 104
18.8.1 任意正模数的单个逆元 . . . 104
18.8.2 任意数组的批量逆元 . . . 105
18.8.3 连续整数的线性逆元表 . . . 107
18.8.4 任意数组的批量逆元 . . . 107
18.8.5 连续整数的线性逆元表 . . . 107
18.9 线性同余方程 . . . 108
18.9.1 线性同余方程 . . . 108
18.9.2 线性同余：最小解与最小周期 . . . 108
18.10 中国剩余定理 . . . 109
18.10.1 广义 CRT：逐项合并同余 . . . 109
18.10.2 Garner：大周期 CRT 与混合进制 . . . 111
18.10.3 扩展 CRT：逐式合并 . . . 113
18.10.4 Garner：总周期可以超过整数范围 . . . 113
18.11 升幂引理 . . . 114
18.11.1 素因子估值与升幂：先约去公共因子 . . . 114
18.11.1.1 基本运算与公共因子 . . . 114
18.11.1.2 奇素数：差式与和式的条件不同 . . . 114
18.11.1.3 素数 2：先分指数奇偶 . . . 114
18.11.1.4 把大整数整除转成指数条件 . . . 114
18.12 阶乘取模 . . . 115
18.12.1 Legendre 赋值与 Kummer 进位判据 . . . 115
18.12.1.1 组合数：逐位相加时进了几次位 . . . 115
18.12.1.2 素数幂为零与 Lucas 的边界 . . . 115
18.12.2 阶乘表：组合数与排列数 . . . 115
18.13 卢卡斯定理 . . . 116
18.13.1 Lucas：小素数模的大下标组合数 . . . 116
18.13.2 合数模数组合数（扩展 Lucas） . . . 117
18.13.3 Lucas：按素数进制逐位计算 . . . 120
18.13.4 合数模数组合数与插值 . . . 120

18.14 二次剩余 . . . 121
18.14.1 素数模平方根（Cipolla） . . . 121
18.14.2 素数模平方根 . . . 123
18.15 阶 & 原根 . . . 123
18.15.1 乘法阶与原根 . . . 123
18.15.2 乘法阶与原根 . . . 126
18.15.3 自动准备模数分解与原根 . . . 126
18.16 离散对数 . . . 127
18.16.1 扩展 BSGS . . . 127
18.17 高次剩余 & 单位根 . . . 129
18.17.1 素数模高次剩余：全部根的紧凑表示 . . . 129
18.17.2 素数模高次剩余与单位根 . . . 129
18.17.3 素数幂模高次剩余与根的提升 . . . 130
18.17.4 一般合数模高次剩余：CRT 组合 . . . 130
18.18 数论分块 . . . 130
18.18.1 整除分块：前缀和来源决定算法 . . . 130
18.18.1.1 按数据与资源选前缀和 . . . 130
18.19 狄利克雷卷积 . . . 131
18.19.1 幂的约数和：不除以 $p-1$ . . . 131
18.20 莫比乌斯反演 . . . 131
18.20.1 整除与反演：先辨认求和方向 . . . 131
18.20.1.1 约数和与倍数和不能混用 . . . 131
18.20.1.2 GCD 权值先反演，再交换求和 . . . 131
18.20.2 GCD 计数：缩放、边界与对称性 . . . 131
18.20.2.1 恰好等于 k：先缩放整除对象，再做矩形容斥 . . . 131
18.20.2.2 去掉顺序之前，先检查交换是否仍在定义域内 . . . 131
18.20.3 矩形 GCD 计数：缩放后容斥 . . . 132
18.21 狄利克雷双曲线法 & 杜教筛 . . . 132
18.21.1 杜教筛 . . . 132
18.21.2 杜教筛：预筛与缓存都不可省略 . . . 133
18.22 Min_25 筛 . . . 134
18.22.1 Min_25 积性函数求和 . . . 134
18.23 类欧几里德算法 . . . 138
18.23.1 有符号整除和：类欧几里德 . . . 138
18.23.2 整除和、带权和与平方和 . . . 140
18.23.3 类欧几里德：有符号整除求和 . . . 142
18.23.4 类欧几里德的三个求和量 . . . 143

第十九章 多项式与生成函数 144

19.1 快速傅里叶变换 . . . 144
19.1.1 复数 FFT（可复用单位根） . . . 144
19.1.2 拆系数 FFT 任意模卷积 . . . 146
19.1.3 CDQ 分治卷积递推 . . . 149
19.1.4 浮点 FFT：舍入前先约束最坏误差 . . . 150
19.2 快速数论变换 . . . 151
19.2.1 可指定素数与原根的 NTT 卷积 . . . 151
19.2.2 精确有符号整数卷积 . . . 154
19.2.3 参数化 NTT 与模数适配 . . . 155

19.3 快速沃尔什变换 . . . 156
19.3.1 按位 OR、AND、XOR 卷积 . . . 156
19.3.2 不相交子集卷积 . . . 158
19.3.3 集合变换与不相交子集卷积 . . . 159
19.3.4 集合变换：逆变换对模数的要求 . . . 160
19.4 Chirp Z 变换 . . . 160
19.4.1 Chirp Z：等比点求值 . . . 160
19.5 多项式牛顿迭代 . . . 162
19.5.1 形式幂级数求逆 . . . 162
19.5.2 多项式带余除法 . . . 164
19.5.3 多项式带余除法：先反转再求逆 . . . 165
19.6 多项式多点求值 | 快速插值 . . . 166
19.6.1 任意点多项式多点求值 . . . 166
19.6.2 任意点多项式快速插值 . . . 169
19.7 多项式初等函数 . . . 171
19.7.1 形式幂级数开根（含前导零与无解） . . . 171
19.7.2 形式幂级数微积分、对数与指数 . . . 174
19.7.3 形式幂级数快速幂（十进制大指数） . . . 176
19.7.4 级数开根：前导零和自由尾项 . . . 178
19.7.5 ln 与 exp：只处理所需前缀 . . . 178
19.7.6 幂函数的大指数：保留三份指数信息 . . . 178
19.8 常系数齐次线性递推 . . . 179
19.8.1 特征多项式求线性递推第 k 项 . . . 179
19.8.2 快速递推与有理生成函数单项 . . . 180
19.8.3 线性递推：系数方向与外推条件 . . . 182
19.8.4 Bostan–Mori 与快速递推单项 . . . 183
19.9 多项式平移 | 连续点值平移 . . . 184
19.9.1 多项式系数平移 . . . 184
19.9.2 Taylor 平移与等比点求值 . . . 185
19.10 符号化方法 . . . 186
19.10.1 生成函数建模到模板：三个可核验例子 . . . 186
19.10.1.1 例一：不同盒子、相同球、每盒有上下界（OGF 乘积） . . . 186
19.10.1.2 例二：步长 1、2 的有序序列（OGF 求逆） . . . 186
19.10.1.3 例三：不同元素分成单点与无序对（EGF 指数） . . . 186
19.11 Lagrange 反演 . . . 187
19.11.1 隐式生成函数与 Lagrange 反演 . . . 187
19.11.1.1 适用条件与取系数公式 . . . 187
19.11.1.2 模型一按内部结点计数的满有序多叉树 . . . 187
19.11.1.3 模型二带标号有根树的 EGF . . . 187
19.11.1.4 落到现有接口及模数边界 . . . 187
19.12 普通生成函数 . . . 188
19.12.1 生成函数与递推 . . . 188

第二十章 组合数学 189
20.1 排列组合 . . . 189
20.1.1 阶乘组合数 . . . 189
20.1.2 计数建模：先确认哪些对象被区分 . . . 190
20.1.2.1 OGF、EGF 的选择来自乘法，而非题目用词 . . . 190

20.1.3 球盒问题：八种情形 . . . 190
20.1.4 组合数选型：先看模数，再看下标与预算 . . . 191
20.2 容斥原理 . . . 192
20.2.1 容斥与二项式反演：恰好与至少 . . . 192
20.2.1.1 从交集和恢复恰好数量 . . . 192
20.2.1.2 至少数量与变换方向 . . . 192
20.2.1.3 例：有标号球放入有标号盒 . . . 192
20.2.1.4 直接使用现有组合数与模整数 . . . 192
20.2.2 组合计数 . . . 193
20.3 错位排列 . . . 193
20.3.1 错位排列 . . . 193
20.4 卡特兰数 . . . 193
20.4.1 Catalan 与投票问题：把前缀限制变成反射双射 . . . 193
20.4.1.1 弱前缀条件：减去首次走到负一的坏路径 . . . 193
20.4.1.2 严格领先：先取走必需的第一步 . . . 193
20.4.1.3 初始有余额：反射起点，而非只改终点 . . . 194
20.4.1.4 取模时优先使用差式，除法条件不能省 . . . 194
20.5 斯特林数与斯特林反演 . . . 195
20.5.1 定义与性质 . . . 195
20.5.1.1 行列快速求解：推导与选择 . . . 195
20.5.2 无符号第一类斯特林数：同一行 . . . 196
20.5.3 无符号第一类斯特林数：同一列 . . . 198
20.5.4 第二类斯特林数：同一行前缀 . . . 199
20.5.5 第二类斯特林数：同一列 . . . 201
20.5.6 斯特林反演与应用 . . . 203
20.6 贝尔数 . . . 204
20.6.1 贝尔数 . . . 204
20.7 伯努利数 . . . 204
20.7.1 伯努利数批量计算 . . . 204
20.7.2 伯努利数与自然数幂和 . . . 204
20.8 Eulerian Number . . . 205
20.8.1 欧拉数（Eulerian number） . . . 205
20.9 分拆数 . . . 206
20.9.1 整数分拆与重数上限 . . . 206
20.9.2 至多 m 个部分的分拆表 . . . 207
20.9.3 至多 m 个部分的分拆表（生成函数） . . . 208
20.9.4 整数分拆：重数限制的差一问题 . . . 209
20.9.4.1 三个容易混淆的限制 . . . 209
20.9.4.2 重数限制与禁止倍数部分 . . . 209
20.10 范德蒙德卷积 . . . 209
20.10.1 范德蒙德卷积及常用变形 . . . 209
20.11 Pólya 计数 . . . 210
20.11.1 Burnside 平均：任意模数下的精确除法 . . . 210
20.11.2 置换的循环长度分解 . . . 211
20.11.3 Pólya 环染色：旋转与翻转 . . . 212
20.11.4 群作用计数：先确定什么算同一个方案 . . . 213
20.11.5 项链与手链：从置换循环推公式 . . . 213

20.11.6 指定颜色库存：给每个循环分配整份颜色 . . . 213
20.11.7 群阶不可逆：先提升模数，再作整数除法 . . . 214

第二十一章 线性代数 215
21.1 矩阵 . . . 215
21.1.1 模矩阵乘法与幂 . . . 215
21.1.2 素数模矩阵求逆 . . . 217
21.1.3 Max-plus 矩阵幂 . . . 219
21.1.4 线性代数与生成树 . . . 219
21.2 行列式 . . . 220
21.2.1 任意正模数行列式：欧几里得消元 . . . 220
21.2.2 任意正模数行列式 . . . 221
21.3 线性基 . . . 222
21.3.1 64 位异或线性基 . . . 222
21.3.2 带位置限制的异或线性基 . . . 224
21.3.3 异或线性空间求交 . . . 227
21.3.4 Zassenhaus：同时求和空间与交空间 . . . 229
21.4 特征多项式 . . . 231
21.4.1 有限状态计数到线性递推：先证明阶数，再用 BM . . . 231
21.4.1.1 先把状态、方向和输出写清楚 . . . 231
21.4.1.2 Cayley–Hamilton 给出阶数至多 d . . . 231
21.4.1.3 为什么前 2d 项足以恢复这个标量序列 . . . 231
21.4.1.4 模型：不含连续子串 101 的二进制串 . . . 231
21.4.1.5 按目标选算法，并把采样成本算进去 . . . 231

第二十二章 线性规划 233
22.1 单纯形法 . . . 233
22.1.1 线性规划：两阶段单纯形 . . . 233

第二十三章 概率论 239
23.1 随机变量的数字特征 . . . 239
23.1.1 期望选型：计数拆指标，等待拆尾和 . . . 239
23.1.2 条件期望：先定义状态，再解自环 . . . 239
23.1.3 模概率：可逆的不只是输入分母 . . . 239

第二十四章 博弈论 240
24.1 公平组合游戏 . . . 240
24.1.1 SG 的前提、递推与异或策略 . . . 240
24.1.2 减法游戏小例与三类失效边界 . . . 240
24.1.3 巴什博弈 . . . 240
24.1.4 Nim 游戏 . . . 240
24.1.5 威佐夫博弈 . . . 240
24.1.6 斐波那契博弈 . . . 241
24.1.7 阶梯博弈 . . . 241
24.1.8 反 Nim（取走最后一颗者负） . . . 241
24.1.9 Nim-k（每次至多动 k 堆） . . . 241
24.1.10 树上删边（Green Hackenbush） . . . 241
24.1.11 二分图博弈（不重复走点） . . . 242
24.1.12 对称策略：环形取石子 . . . 242

24.1.13 策略窃取：Chomp . . . 242

第二十五章 数值算法 243
25.1 插值 . . . 243
25.1.1 拉格朗日插值与连续点优化 . . . 243
25.1.2 横坐标连续的 O(n) 拉格朗日插值 . . . 246
25.2 数值积分 . . . 247
25.2.1 有工作上限的自适应 Simpson 积分 . . . 247
25.2.2 数值积分：局部估计、误差预算与采样盲区 . . . 250
25.2.3 数值算法与几何 . . . 250
25.3 高斯消元 . . . 250
25.3.1 压位模 2 高斯消元与全部解 . . . 250
25.3.2 素数模高斯消元与全部解 . . . 252
25.3.3 实数高斯消元与数值解空间 . . . 255
25.3.4 模线性方程组：工厂工期 . . . 257
25.3.5 模 2 压位消元与完整解空间 . . . 258
25.3.6 模 2 消元后的最少开关用法 . . . 258

第二十六章 Berlekamp–Massey 算法 259
26.1 Berlekamp–Massey：最短线性递推 . . . 259

第五部分 数据结构 261

第二十七章 哈希表 262
27.1 GNU gp_hash_table 与 cc_hash_table . . . 262

第二十八章 并查集 264
28.1 并查集 . . . 264
28.1.1 并查集 . . . 264
28.1.2 可撤销并查集 . . . 266
28.1.3 带势并查集与差值约束 . . . 268
28.1.4 时间线段树分治：离线动态连通性 . . . 270
28.1.5 并查集模板题的接口范围 . . . 273

第二十九章 堆 274
29.1 配对堆 . . . 274
29.1.1 PBDS 可修改、可合并优先队列 . . . 274
29.2 左偏树 . . . 276
29.2.1 左偏树可并小根堆 . . . 276

第三十章 ST 表 278
30.1 位掩码静态区间最值与位置 . . . 278

第三十一章 树状数组 282
31.1 四矩二维树状数组：矩形加与矩形和 . . . 282
31.2 树状数组第 k 小与组合依赖 . . . 284
31.3 离线静态矩形权值和 . . . 285

第三十二章 线段树 **289**
32.1 线段树基础 . . . 289
32.1.1 通用懒标记线段树与边界搜索 . . . 289
32.1.2 通用线段树与左右边界搜索 . . . 295
32.1.3 线段树维护单调栈：区间加与接雨水 . . . 300
32.1.4 ACL 的组合约定 . . . 302
32.1.5 线段树维护单调栈：双向扫描与接雨水 . . . 303
32.2 线段树合并 & 分裂 . . . 303
32.2.1 线段树合并与分裂 . . . 303
32.3 李超线段树 . . . 312
32.3.1 精确线段李超树：最大值编号 . . . 312
32.3.2 离散李超树与最优直线编号 . . . 317
32.4 区间最值操作 & 区间历史最值 . . . 320
32.4.1 Segment Tree Beats：区间截断、加法与和 . . . 320

第三十三章 二叉搜索树 & 平衡树 **326**
33.1 二叉搜索树 & 平衡树 . . . 326
33.1.1 PBDS 有序树与顺序统计 . . . 326
33.1.2 GNU rope 与版本共享 . . . 328
33.2 Treap . . . 329
33.2.1 FHQ Treap：可重集合与顺序统计 . . . 329
33.2.2 隐式 Treap：顺序反转、位翻转、加值与区间和 . . . 333
33.2.3 隐式 Treap 位翻转与顺序反转 . . . 338
33.3 Splay 树 . . . 339
33.3.1 Splay：可重集合与顺序统计 . . . 339
33.4 替罪羊树 . . . 345
33.4.1 替罪羊树：重复计数与墓碑清理 . . . 345
33.5 笛卡尔树 . . . 346
33.5.1 笛卡尔树与相等值优先级 . . . 346

第三十四章 可持久化数据结构 **348**
34.1 可持久化线段树 . . . 348
34.1.1 主席树：静态区间第 k 小 . . . 348
34.1.2 可持久化区间加与版本拼接 . . . 350
34.1.3 可持久化并查集 . . . 353
34.1.4 可持久化区间拼接的懒标记 . . . 356
34.1.5 杭州 2023 K：跨版本拼接应用 . . . 356
34.2 可持久化平衡树 . . . 357
34.2.1 可持久化有序 Treap：分叉版本与顺序统计 . . . 357
34.3 可持久化字典树 . . . 363
34.3.1 前缀可持久化 01 字典树 . . . 363

第三十五章 树套树 **365**
35.1 树状数组套权值线段树 . . . 365
35.1.1 动态区间第 k 小（树状数组套权值线段树） . . . 365

第三十六章 K-D Tree **369**
36.1 动态二维 K-D Tree：单点累加与矩形和 . . . 369
36.2 静态 K-D Tree 最近与最远点 . . . 372
36.3 动态 K-D Tree：矩形和与重建边界 . . . 374

第三十七章 动态树 **375**
37.1 Link Cut Tree . . . 375
37.1.1 Link-cut tree：动态森林与路径异或 . . . 375
37.1.2 动态树路径乘积：虚子树与双向信息 . . . 380

第六部分 图论 **381**

第三十八章 图论相关概念 **382**
38.1 函数图的入环链与到达查询 . . . 382
38.2 杭州 2023 H：随机事件顺序与函数图 . . . 383

第三十九章 树上问题 **384**
39.1 树的直径 . . . 384
39.1.1 边权修改动态树直径 . . . 384
39.1.2 树上点集直径摘要 . . . 388
39.1.3 非负权树的点集直径摘要 . . . 390
39.2 最近公共祖先 . . . 391
39.2.1 两条树路径的交集 . . . 391
39.2.2 倍增 LCA、祖先与最大边权 . . . 392
39.2.3 树路径交：投影与单点交 . . . 395
39.3 树链剖分 . . . 395
39.3.1 长链剖分与常数时间 K 级祖先 . . . 395
39.3.2 重链剖分与 LCA . . . 399
39.4 树上启发式合并 . . . 404
39.4.1 树上启发式合并：子树众数颜色和 . . . 404
39.5 虚树 . . . 407
39.5.1 虚树：关键点压缩与路径信息 . . . 407
39.6 树分治 . . . 410
39.6.1 点分治：树上距离点对计数 . . . 410
39.6.2 点分治、启发式合并与虚树用法 . . . 414
39.7 动态树分治 . . . 414
39.7.1 动态点分树：点权赋值与距离球求和 . . . 414
39.7.2 动态点分树：有符号边权活动点直径与端点 . . . 419
39.8 树哈希 . . . 424
39.8.1 确定性有根与无根树同构 . . . 424
39.8.2 双 64 位随机树哈希（Mix13） . . . 428
39.8.3 树哈希 O(n) 换根 . . . 430

第四十章 最短路问题 **431**
40.1 最短路 . . . 431
40.1.1 非负权最短路 . . . 431
40.1.2 Bellman–Ford：负环影响与路径方案 . . . 432
40.1.3 SPFA：队列松弛、负环判定与有限路径 . . . 435

40.1.4 Floyd 全源最短路与原边方案 . . . 437
40.1.5 杭州 2023 G：开放时间的单位边最短路 . . . 439
40.2 k 短路 . . . 440
40.2.1 非负权严格次短行走 . . . 440
40.2.2 正权前 K 短行走 . . . 441

第四十一章 生成树问题 444
41.1 最小生成树 . . . 444
41.1.1 Kruskal 重构森林与阈值连通块 . . . 444
41.1.2 曼哈顿最小生成树 . . . 448
41.1.3 严格与非严格次小生成树、换边方案 . . . 450
41.1.4 Kruskal 最小生成树与生成森林 . . . 455
41.1.5 稠密图 Prim 最小生成森林 . . . 458
41.1.6 曼哈顿最小生成树：闭锥扫描与平局割证明 . . . 460
41.1.7 严格次小生成树与换边 . . . 460
41.2 最小树形图 . . . 461
41.2.1 有向最小树形图 . . . 461

第四十二章 斯坦纳树 465
42.1 最小斯坦纳树与原边方案 . . . 465

第四十三章 连通性相关 471
43.1 强连通分量 . . . 471
43.1.1 函数图分解、跳转与到达 . . . 471
43.1.2 Tarjan 强连通分量与缩点 . . . 475
43.1.3 SCC 模板题与速度榜口径 . . . 477
43.2 双连通分量 . . . 478
43.2.1 点双与边双：共享 lowlink 核心 . . . 478
43.2.2 边双缩点森林（保留桥编号） . . . 482
43.3 圆方树 . . . 482
43.3.1 静态仙人掌森林最短路 . . . 482
43.3.2 由点双构造圆方森林 . . . 487

第四十四章 环计数问题 489
44.1 简单无向图四元环计数 . . . 489
44.2 无向简单图三元环枚举 . . . 491

第四十五章 最小环 493
45.1 有向图最小平均权环：线性空间 Karp . . . 493
45.2 非负权无向图最小环与原边方案 . . . 495

第四十六章 2-SAT 500
46.1 2-SAT 与任意可行方案 . . . 500
46.2 2-SAT 编号方向与双连通定义 . . . 501

第四十七章 欧拉图 502
47.1 有向图字典序最小欧拉路 . . . 502
47.2 无向图欧拉路与边方向 . . . 506
47.3 欧拉回路与指定端点定向 . . . 509
47.4 任意起终点的混合欧拉路径 . . . 511

第四十八章 网络流 **512**
48.1 最大流 . . . 512
48.1.1 HLPP 最高标号预流推进与流方案 . . . 512
48.1.2 最大流与方案 . . . 518
48.1.3 排列约束与最大流单位边分类 . . . 522
48.2 最小割 . . . 523
48.2.1 最大权闭合子图与选点方案 . . . 523
48.2.2 正容量最小割的可行边与必选边 . . . 524
48.2.3 Gomory–Hu 最小割树 . . . 525
48.2.4 闭合集、树形图与全局割的完整用法 . . . 526
48.3 费用流 . . . 527
48.3.1 势能最小费用最大流 . . . 527
48.3.2 允许负环的最小费用最大流 . . . 530
48.3.3 SPFA 费用流与残量方案 . . . 533
48.4 上下界网络流 . . . 534
48.4.1 上下界可行循环流 . . . 534
48.4.2 有源汇上下界最值流与边流量 . . . 535
48.5 Stoer–Wagner 算法 . . . 538
48.5.1 无向全局最小割 . . . 538

第四十九章 图的匹配 **541**
49.1 二分图最大匹配 . . . 541
49.1.1 正则二分图完美匹配 . . . 541
49.1.2 Hopcroft–Karp 最大匹配与最小点覆盖 . . . 544
49.1.3 二分图最大匹配的可行边与必选边 . . . 547
49.1.4 二分图最大独立集方案 . . . 549
49.2 二分图最大权匹配 . . . 549
49.2.1 矩形二分图最小权匹配 . . . 549
49.3 一般图最大匹配 . . . 551
49.3.1 一般图最大基数匹配：带花树 . . . 551
49.4 一般图最大权匹配 . . . 555
49.4.1 一般图最大权匹配：带权带花树 . . . 555

第五十章 Prüfer 序列 **566**
50.1 Prüfer 双射与有标号树计数 . . . 566
50.1.1 双射为何成立 . . . 566
50.1.2 Cayley、指定度数与叶子数 . . . 566
50.1.3 加权 Cayley 与连接已有连通块 . . . 567

第五十一章 矩阵树定理 **568**
51.1 Tutte 多项式求值 . . . 568
51.2 任意正模数带权矩阵树 . . . 573
51.3 带权矩阵树定理与方向约定 . . . 575
51.4 任意正模数的矩阵树计数 . . . 576

第五十二章 LGV 引理 **577**
52.1 LGV：DAG 路径权值行列式 . . . 577

第五十三章 支配树 **579**
53.1 支配树：Lengauer–Tarjan 579

第七部分 计算几何 583

第五十四章 二维计算几何基础 **584**
54.1 浮点点向量、圆与交点结果 584
54.2 点在直线上的投影 586
54.3 浮点两直线交点 587
54.4 浮点直线与圆交点 589
54.5 浮点两圆交点 591
54.6 两圆交面积 594
54.7 凸多边形裁剪：保留有向直线左侧 595
54.8 简单多边形面积重心 597
54.9 三角形内切圆：边长加权内心 598
54.10 有限二进制浮点极角精确排序 599
54.11 整数点向量与线段判定 602
54.12 整数多边形有向面积 605
54.13 点在整数简单多边形内 606
54.14 整数输入直线与圆交点 607
54.15 整数输入两圆交点 609
54.16 圆与简单多边形相交面积 611
54.17 两圆公切线与过点切线 613
54.18 一般多边形面积并：整数精确判定 615
54.19 圆面积并：未覆盖圆弧积分 622
54.20 圆覆盖重数面积：恰好覆盖 k 次 627

第五十五章 三维计算几何基础 **632**
55.1 三维凸包：精确方向与退化处理 632
55.2 整数三维共线、共面与方向 643
55.3 实数三维向量与夹角 644
55.4 三维直线投影、线段距离与绕轴旋转 648
55.5 三维平面投影与分类求交 651

第五十六章 距离 **654**
56.1 点到闭线段的距离 654

第五十七章 Pick 定理 **656**
57.1 Pick 定理：格点、剖分与孔洞 656

第五十八章 凸包 **657**
58.1 凸多边形外点的两条切线 657
58.2 点在整数凸包内（二分） 660
58.3 整数凸包：单调链 661
58.4 凸多边形 Minkowski 和 662

第五十九章 扫描线 **665**
59.1 整数轴对齐矩形面积并 665

第六十章 旋转卡壳 **667**
60.1 整数凸包直径：旋转卡壳 . . . 667
60.2 最小面积包围矩形：旋转卡壳与四角 . . . 669

第六十一章 半平面交 **672**
61.1 整数半平面的有界多边形交 . . . 672

第六十二章 平面最近点对 **676**
62.1 浮点平面最近点对 . . . 676

第六十三章 随机增量法 **679**
63.1 随机增量最小覆盖圆 . . . 679

第六十四章 计算几何：精确判定与适用范围 **682**
64.1 定点凸包、直径和极角排序 . . . 682
64.2 整数系数的有界半平面交 . . . 682
64.3 最近点对、Minkowski 和与三维整数判定 . . . 682
64.4 浮点最近点对 . . . 682
64.5 公切线与过点切线 . . . 682
64.6 随机增量最小覆盖圆 . . . 683
64.7 圆与简单多边形相交面积 . . . 683
64.8 最近点对编号与多边形闭集查询 . . . 683
64.9 整数圆交点：精确判定与浮点构造 . . . 683
64.10 公切线：切点次序也需要精确判定 . . . 683
64.11 圆交点：容差分类与坐标误差 . . . 684
64.12 三维投影、旋转与平面求交的数值约定 . . . 684
64.12.1 原三维片段中需要避开的错误 . . . 684

第八部分 杂项 **685**

第六十五章 离线算法 **686**
65.1 CDQ 分治 . . . 686
65.1.1 静态三维偏序计数 . . . 686
65.1.2 CDQ 卷积递推：贡献只跨越一次 . . . 689
65.2 整体二分 . . . 689
65.2.1 整体二分：动态区间第 k 小 . . . 689
65.3 莫队算法 . . . 693
65.3.1 普通莫队算法 . . . 693
65.3.1.1 曼哈顿 MST 的递归 DFS 查询顺序 . . . 693
65.3.1.2 普通莫队：静态区间统计 . . . 693
65.3.2 带修改莫队 . . . 696
65.3.2.1 带修改的莫队 . . . 696
65.3.2.2 带修莫队：时间轴与区间状态 . . . 699
65.3.3 树上莫队 . . . 699
65.3.3.1 树上莫队：点权与边权 . . . 699
65.3.4 回滚莫队 . . . 701
65.3.4.1 回滚莫队 . . . 701
65.3.5 莫队二次离线 . . . 706
65.3.5.1 莫队二次离线：异或位数配对 . . . 706

65.3.5.2 莫队二次离线：前缀事件与自身扣除 710

第六十六章 分数规划 **711**
66.1 0/1 分数规划：Dinkelbach 711
66.2 0/1 分数规划 711

第六十七章 表达式求值 **712**
67.1 整数表达式求值 712

第六十八章 珂朵莉树/颜色段均摊 **715**
68.1 珂朵莉树（颜色段均摊） 715

第一部分

搜索

第一章　Dancing Links

1.1　精确覆盖（Dancing Links）

问题描述：给定 m 列和若干行（每行是一组列号，即稀疏 0/1 矩阵），选出若干行，使每一列**恰好**被一个选中行覆盖；求任意一组解的行号，或判定无解。

核心思路：Knuth 的 X 算法加舞蹈链：所有 1 连成十字循环链表，0 号节点是列头链表的表头，`s[c]` 是列 c 当前剩余的候选行数。`cover(c)` 把列 c 从列头链表摘除，并把含有列 c 的每一行的其余节点从各自列中纵向摘除（这些行与之后选的行冲突）；`uncover` 按相反顺序原样接回。`dfs` 每次选候选行最少的列，依次尝试其中每一行：记录行号，覆盖该行涉及的其余列后递归，失败则恢复并换下一行。列头链表为空即找到解；成功返回前同样恢复链表，所以可以重复求解。

接口契约：ExactCover(m,rows)：列 0..m-1，各稀疏行内列号互异、无需排序；行可重复或为空，保留原 0-based 行号。solve() 返回 optional<vector<int»，无解为 nullopt，m=0 为有值的空解。只求任意首解，不保证最少行数。成功/失败均恢复链与列计数，可重复调用及复制；实现字段不可外改，移后源对象仅销毁或重新赋值。m 和行数 <INT_MAX，1+m+ 非零数 ≤INT_MAX。构造 O(行数 +m+ 非零数)，空间 O(m+ 非零数)，递归深度 ≤m，搜索最坏指数。

```
struct ExactCover
{
    struct Node { int l, r, u, d, c, row; };
    int m;
    vector<Node> a;
    vector<int> s, ans;

    ExactCover(int columns, const vector<vector<int>> &rows) : m(columns)
    {
        assert(0 <= m && m < INT_MAX && rows.size() < INT_MAX);
        size_t z = size_t(m) + 1;
        for (const auto &r : rows)
        {
            assert(r.size() <= size_t(INT_MAX) - z);
            z += r.size();
        }
        a.reserve(z);
        s.assign(m + 1, 0);
        for (int c = 0; c <= m; c++)
            a.push_back({c ? c - 1 : m, c == m ? 0 : c + 1, c, c, c, -1});
```

稀疏行建链与删除（接上页同一结构体）：

```
        vector<int> seen(m, -1);
        for (int i = 0; i < int(rows.size()); i++)
        {
            int h = -1;
            for (int col : rows[i])
            {
                assert(0 <= col && col < m && seen[col] != i);
                seen[col] = i;
                int c = col + 1, x = a.size(), u = a[c].u;
                a.push_back({x, x, u, c, c, i});
                a[u].d = a[c].u = x;
                s[c]++;
                if (h == -1) h = x;
                else
                {
                    int l = a[h].l;
                    a[x].l = l; a[x].r = h;
                    a[l].r = a[h].l = x;
                }
            }
        }
    }

    void cover(int c)
    {
        a[a[c].l].r = a[c].r;
        a[a[c].r].l = a[c].l;
        for (int i = a[c].d; i != c; i = a[i].d)
            for (int j = a[i].r; j != i; j = a[j].r)
            {
                a[a[j].u].d = a[j].d;
                a[a[j].d].u = a[j].u;
                s[a[j].c]--;
            }
    }
```

逆序恢复与搜索（接上页同一结构体）：

```
    void uncover(int c)
    {
        for (int i = a[c].u; i != c; i = a[i].u)
            for (int j = a[i].l; j != i; j = a[j].l)
            {
                s[a[j].c]++;
                a[a[j].u].d = a[a[j].d].u = j;
            }
        a[a[c].l].r = a[a[c].r].l = c;
    }

    bool dfs()
    {
        int c = a[0].r;
        if (!c) return true;
        for (int j = a[c].r; j; j = a[j].r)
            if (s[j] < s[c]) c = j;
        cover(c);
        for (int i = a[c].d; i != c; i = a[i].d)
        {
            ans.push_back(a[i].row);
            for (int j = a[i].r; j != i; j = a[j].r) cover(a[j].c);
            bool ok = dfs();
            for (int j = a[i].l; j != i; j = a[j].l) uncover(a[j].c);
            if (ok)
            {
                uncover(c);
                return true;
            }
            ans.pop_back();
        }
        uncover(c);
        return false;
    }

    optional<vector<int>> solve()
    {
        ans.clear();
        ans.reserve(m);
        if (!dfs()) return nullopt;
        return ans;
    }
};
```

模板题使用：Luogu P4929 / 精确覆盖

题意：读入 N×M 的 01 矩阵，N,M≤500 且非零数 ≤5000；每列恰被选择行覆盖一次。稀疏建模后输出原 1-based 行号，任一可行解均可；无解输出 No Solution!。

所需模板：ExactCover（第 2 页，1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4929

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<vector<int>> rows(n);
    for (int i = 0; i < n; i++)
        for (int j = 0, x; j < m; j++)
        {
            cin >> x;
            if (x) rows[i].push_back(j);
        }
    ExactCover dlx(m, rows);
    auto ans = dlx.solve();
    if (!ans) cout << "No Solution!\n";
    else
    {
        for (int i : *ans) cout << i + 1 << ' ';
        cout << '\n';
    }
}
```

1.2 最少行重复覆盖（Dancing Links）

问题描述：给定 m 列和若干行（每行是一组列号），选出**最少**的行，使每一列至少被一个选中行覆盖（允许重复覆盖），返回一组最优解的行号或判定无解。

核心思路：舞蹈链上的分支限界搜索。重复覆盖时行永远不会作废，只需横向摘除：`remove(x)` 把 `x` 所在列的其他节点（含列头）从各自的行链表中摘下，这一列就从待覆盖列表中消失，而各列的候选行数 `s` 保持不变。`dfs` 选候选最少的未覆盖列，枚举其中每一行：摘除该行覆盖的所有列，递归后逆序 `restore`。剪枝用 `lower_bound`：贪心选出若干两两不能被同一行同时覆盖的未覆盖列，其个数是至少还需的行数；当前深度加此下界不小于已知最优 `best` 时剪掉。`best` 初值为 $m+1$，到达全部覆盖时更新答案。

接口契约：MinimumCover(m, rows)，列号 0..m-1，每行内部互异，允许空行/重复行。solve() 返回最少行数的一组原 0-based 行号；nullopt 无解，有值空向量表示 m=0。每列至少一次，允许重叠；不含权值、覆盖次数或全解枚举。0≤m<INT_MAX，行数 <INT_MAX，m+1+ 非零数 ≤INT_MAX；前提由调用方保证。构造 O(行数 +m+ 非零数)，空间 O(m+ 非零数)；递归搜索最坏指数、深度 ≤m，每状态 O(m+ 非零数)。成功失败均恢复链，可重复求解或复制；重建用重新赋值。字段及辅助方法不作为可修改接口，移动后源对象只销毁或赋值。下界选取不能被同一行共同覆盖的列，至少还需同样多行；按最少候选列搜索，逆序恢复只改左右链。

```
struct MinimumCover
{
    struct Node
    {
        int l, r, u, d, c, row;
    };
    int m, best = 0;
    vector<Node> a;
    vector<int> s, path, ans;
    vector<unsigned char> mark;

    MinimumCover(int columns, const vector<vector<int>> &rows) : m(columns)
    {
        assert(0 <= m && m < INT_MAX && rows.size() < INT_MAX);
        size_t z = size_t(m) + 1;
        for (const auto &r : rows)
        {
            assert(r.size() <= size_t(INT_MAX) - z);
            z += r.size();
        }
        a.reserve(z);
        s.assign(m + 1, 0);
        mark.resize(m + 1);
        path.reserve(m);
        ans.reserve(m);
        for (int c = 0; c <= m; c++)
            a.push_back({c ? c - 1 : m, c == m ? 0 : c + 1, c, c, -1});
```

稀疏建链与删列（接上页同一结构体）：

```
        vector<int> seen(m, -1);
        for (int i = 0; i < int(rows.size()); i++)
        {
            int h = -1;
            for (int col : rows[i])
            {
                assert(0 <= col && col < m && seen[col] != i);
                seen[col] = i;
                int c = col + 1, x = a.size(), u = a[c].u;
                a.push_back({x, x, u, c, c, i});
                a[u].d = x;
                a[c].u = x;
                s[c]++;
                if (h == -1) h = x;
                else
                {
                    int l = a[h].l;
                    a[x].l = l;
                    a[x].r = h;
                    a[l].r = x;
                    a[h].l = x;
                }
            }
        }
    }

    // x is a selected row node, not a column header.
    void remove(int x)
    {
        for (int i = a[x].d; i != x; i = a[i].d)
        {
            a[a[i].l].r = a[i].r;
            a[a[i].r].l = a[i].l;
        }
    }
```

逆序恢复与可采纳下界（接上页同一结构体）：

```
    void restore(int x)
    {
        for (int i = a[x].u; i != x; i = a[i].u)
        {
            a[a[i].l].r = i;
            a[a[i].r].l = i;
        }
    }

    int lower_bound()
    {
        fill(mark.begin(), mark.end(), 0);
        for (int c = a[0].r; c; c = a[c].r) mark[c] = 1;
        int bound = 0;
        for (int c = a[0].r; c; c = a[c].r)
        {
            if (!mark[c]) continue;
            bound++;
            mark[c] = 0;
            for (int i = a[c].d; i != c; i = a[i].d)
                for (int j = a[i].r; j != i; j = a[j].r)
                    mark[a[j].c] = 0;
        }
        return bound;
    }
```

最优搜索与求解入口（接上页同一结构体）：

```
    void dfs()
    {
        int depth = path.size(), c = a[0].r;
        if (!c)
        {
            if (depth < best)
            {
                best = depth;
                ans = path;
            }
            return;
        }
        if (lower_bound() >= best - depth) return;
        for (int j = a[c].r; j; j = a[j].r)
            if (s[j] < s[c]) c = j;
        for (int i = a[c].d; i != c; i = a[i].d)
        {
            path.push_back(a[i].row);
            remove(i);
            for (int j = a[i].r; j != i; j = a[j].r) remove(j);
            dfs();
            for (int j = a[i].l; j != i; j = a[j].l) restore(j);
            restore(i);
            path.pop_back();
        }
    }

    optional<vector<int>> solve()
    {
        path.clear();
        ans.clear();
        best = m + 1;
        dfs();
        if (best > m) return nullopt;
        return ans;
    }
};
```

第二部分

动态规划

第二章　插头 DP

2.1　插头 DP：网格单闭环计数

问题描述：给定由 . 和 * 组成的矩形网格，. 是必须经过的格子，* 不能经过，只能在上下左右相邻的格子间移动。求有多少种方法，用**恰好一条**闭合回路经过所有 .，且每个 . 恰好经过一次；回路不区分起点和行走方向。例如四个格子都是 . 的 2×2 网格，答案为 1。

核心思路：按行逐格扫描。已处理部分是若干条尚未闭合的路径，只需记录它们穿过当前扫描轮廓（已处理格的下边与当前格左侧）的位置，以及哪两个端点属于同一条路径：轮廓上第 j 个槽用 4 位编号，同编号的两个槽是同一路径的两端，0 表示无插头。处理格 (i, j) 时，槽 j、$j+1$ 分别是来自左侧、上方的插头，转移后变为向下、向右的插头。每个 . 度数恰为 2，根据左、上插头决定新建、延伸、合并路径或闭环；目标只有一个覆盖全部格子的环，所以只在最后一个 . 且没有其他插头时允许闭环计数。每行开始时整体左移一槽，右边界插头恒为 0。

思路中的情况	代码中的处理	行
取出左、上插头	解码状态，`x`、`y` 为左、上插头编号，并把两槽清零	44
遇到 *	不能有来自左、上的插头，原样保留状态，跳过该格	61
左、上都没有插头	下、右两格都可用时新建一条路径，向下、向右各一个端点（临时编号 15）	65
恰有一个插头	把这条路径向下或向右延伸（两种选择分别计数）	74
两个插头属于不同路径	合并两条路径：把编号 `y` 全部改成 `x`	88
两个插头属于同一路径	形成闭环；只有在最后一个 . 且其余槽全为 0 时计入答案	94
不同编号表示相同连接关系	`add` 按首次出现重新编号后再存入哈希表，避免重复状态	48
换行	所有状态左移一槽（4 位），对齐下一行的轮廓	33

接口契约：grid_cycle<Count>(a) 统计覆盖所有可用格子的单个无向简单环；矩形网格中. 必须经过、* 不能经过，仅上下左右相邻。空网格/全障碍返回 0。自动转置到较窄方向，宽度 m≤15，尺寸须可用 int 表示。每个未闭合连通块是一条路径，在轮廓上恰有两个同编号端点；宽度 ≤15 时最多 8 条路径，临时编号 15 不与规范编号冲突。每个边集唯一对应逐格选择，不按起点或方向重复计数。

Count 支持 0/1 构造和 +=；固定整数必须保证所有中间计数不溢出，可改模整数或 cpp_int。S 为最多可达状态数，固定宽度计数的期望时间 O(n*m^2*S)、状态表 O(S)，另有网格 O(nm)。S 随宽度指数增长；cpp_int 还需计入位复杂度。

```cpp
template <class Count>
Count grid_cycle(vector<string> a)
{
    int n = a.size();
    if (n == 0) return Count(0);
    int m = a[0].size();
    for (auto &row : a)
    {
        assert((int)row.size() == m);
        for (char c : row) assert(c == '.' || c == '*');
    }
    if (m == 0) return Count(0);
    if (n < m)
    {
        vector<string> b(m, string(n, '*'));
        for (int i = 0; i < n; i++)
            for (int j = 0; j < m; j++) b[j][i] = a[i][j];
        a.swap(b);
        swap(n, m);
    }
    assert(m <= 15);
    pair<int, int> last = {-1, -1};
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            if (a[i][j] == '.') last = {i, j};
    using Key = unsigned long long;
    unordered_map<Key, Count> dp, next;
    dp[0] = Count(1);
    Count answer = 0;
    for (int i = 0; i < n; i++)
    {
        next.clear();
        for (auto &[s, ways] : dp) next[s << 4] += ways;
        dp.swap(next);
```

轮廓转移：解码左、上插头，重编号，障碍格与新建路径（接上页同一函数）：

```
        for (int j = 0; j < m; j++)
        {
            next.clear();
            bool down = i + 1 < n && a[i + 1][j] == '.';
            bool right = j + 1 < m && a[i][j + 1] == '.';
            for (auto &[s, ways] : dp)
            {
                vector<int> p(m + 1);
                for (int k = 0; k <= m; k++) p[k] = s >> (4 * k) & 15;
                int x = p[j];
                int y = p[j + 1];
                p[j] = 0;
                p[j + 1] = 0;
                auto add = [&]()
                {
                    array<int, 16> id{};
                    int count = 0;
                    Key key = 0;
                    for (int k = 0; k <= m; k++)
                    {
                        int v = p[k];
                        if (v && !id[v]) id[v] = ++count;
                        key |= Key(id[v]) << (4 * k);
                    }
                    next[key] += ways;
                };
                if (a[i][j] == '*')
                {
                    if (!x && !y) add();
                }
                else if (!x && !y)
                {
                    if (down && right)
                    {
                        p[j] = 15;
                        p[j + 1] = 15;
                        add();
                    }
                }
```

延伸、合并与最终闭环（接上页同一函数）：

```
                else if (!x || !y)
                {
                    if (down)
                    {
                        p[j] = x + y;
                        add();
                        p[j] = 0;
                    }
                    if (right)
                    {
                        p[j + 1] = x + y;
                        add();
                    }
                }
                else if (x != y)
                {
                    for (int &v : p)
                        if (v == y) v = x;
                    add();
                }
                else if (make_pair(i, j) == last)
                {
                    if (all_of(p.begin(), p.end(), [](int v)
                    {
                        return v == 0;
                    })) answer += ways;
                }
            }
            dp.swap(next);
        }
    }
    return answer;
}
```

数值与验证 固定整数要保证所有中间计数可表示，不能只检查最后答案。P5056 仅内部 (n-1)(m-1) 个格子可能二选一，其余转移至多一个。所有中间计数不超过 2 的 121 次方，因此下面用 128 位整数仍能保证精确计数。已穷举 74954 个小障碍网格并与独立递归环搜索核对；完整 2 至 14 阶偶数方格与 OEIS A003763 核对。输入超过题目范围时状态数仍可能很大。

模板题使用：P5056 插头 DP

题意：2≤n,m≤12，所有点格必须形成唯一闭环。仅同时存在下、右邻格时可能二选一，至多 $(12\text{-}1)^2=121$ 次，因此全部中间计数 ≤2^121，用 ___int128 安全；十进制输出。

所需模板：grid_cycle（第 11 页，2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5056

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<string> a(n);
    for (auto &row : a) cin >> row;
    __int128 answer = grid_cycle<__int128>(a);
    string out;
    do
    {
        out += char('0' + answer % 10);
        answer /= 10;
    } while (answer);
    reverse(out.begin(), out.end());
    cout << out << '\n';
}
```

2.2 插头 DP：恰好 K 个不嵌套回路

问题描述：给定由 . 和 * 组成的网格，只能在上下左右相邻格之间走，求用**恰好** k **个**闭合回路经过所有 . 且每格恰好一次的方案数（回路不区分起点和方向）；默认要求任何回路不包含另一回路，也可允许嵌套。

核心思路：与单回路插头 DP 一样逐格扫描（宽大于高时先转置），轮廓每槽 4 位存路径编号，同编号两槽为同一条未闭合路径的两端，0 表示无插头；状态再按已闭合的环数 c 分开存于 `dp[c]`。每格取出左插头 `x`、上插头 `y` 分情况转移，`add` 规范化编号后存入目标环数的哈希表。左、上为同一路径的两端时闭合成环，环数加一；不允许嵌套时要求当前格左边的插头数为偶数，否则新环落在另一条路径内部。扫完所有格后取 `dp[k]` 中的空状态。

思路中的情况	代码中的处理	行
换行	所有状态左移一槽（4 位），对齐下一行的轮廓	37
取出左、上插头	解码状态，`x`、`y` 为左、上插头编号，并把两槽清零	51
遇到 *	不能有插头进入，原样保留状态	68
左、上都没有插头	下、右两格都可用时新建一条路径（临时编号 15）	72
恰有一个插头	把这条路径向下或向右延伸，两种选择分别计数	81
两个插头属于不同路径	合并两条路径：把编号 `y` 全部改成 `x`	95
两个插头属于同一路径	闭合成环，环数加一；禁止嵌套时检查左侧插头数的奇偶	101

接口契约：grid_cycles<Count>(a,k,nested=false) 统计覆盖全部. 格子的恰好 k 个无向简单环；* 为障碍，只有上下左右相邻。默认禁止任意环包含另一个环；nested=true 允许嵌套。无可用格且 k=0 时返回 1，其余无法覆盖时返回 0。k 为非负 int 且 k<INT_MAX；矩形尺寸可用 int 表示，自动转置后宽度 m≤15。

同编号的两个插头表示同一路径端点，规范化编号。闭环时环数加 1；禁止嵌套时检查当前格左侧插头数为偶数。每个环在其最后扫描顶点闭合，该奇偶性是环的嵌套深度奇偶；若存在嵌套，必有深度 1 的环被拒绝。只在扫完全部格子、无剩余插头时取答案。

Count 需支持 0/1 构造、默认构造为 0、复制和 +=；所有中间计数均须可表示，可用模整数或 cpp_int。S 为固定环数下最多轮廓状态数，固定宽度计数下期望时间 O(n*m^2*(k+1)*(S+1))，两层状态表 O((k+1)*(S+1))，网格 O(nm)。S 随宽度指数增长；高精度计数另计位复杂度。

```cpp
template <class Count>
Count grid_cycles(vector<string> a, int k, bool nested = false)
{
    assert(k >= 0);
    int n = a.size();
    if (n == 0) return Count(k == 0);
    int m = a[0].size();
    long long cells = 0;
    for (auto &row : a)
    {
        assert((int)row.size() == m);
        for (char c : row)
        {
            assert(c == '.' || c == '*');
            cells += c == '.';
        }
    }
    if (cells == 0) return Count(k == 0);
    if (k == 0 || k > cells / 4) return Count(0);
    if (n < m)
    {
        vector<string> b(m, string(n, '*'));
        for (int i = 0; i < n; i++)
            for (int j = 0; j < m; j++) b[j][i] = a[i][j];
        a.swap(b);
        swap(n, m);
    }
    assert(m <= 15);
    using Key = unsigned long long;
    vector<unordered_map<Key, Count>> dp(k + 1), next(k + 1);
    dp[0][0] = Count(1);
```

行移位、规范化与创建路径（接上页同一函数）：

```
    for (int i = 0; i < n; i++)
    {
        for (int c = 0; c <= k; c++)
        {
            next[c].clear();
            for (auto &[s, ways] : dp[c]) next[c][s << 4] += ways;
        }
        dp.swap(next);
        for (int j = 0; j < m; j++)
        {
            for (auto &t : next) t.clear();
            bool down = i + 1 < n && a[i + 1][j] == '.';
            bool right = j + 1 < m && a[i][j + 1] == '.';
            for (int c = 0; c <= k; c++)
            {
                for (auto &[s, ways] : dp[c])
                {
                    array<int, 16> p{};
                    for (int t = 0; t <= m; t++) p[t] = s >> (4 * t) & 15;
                    int x = p[j];
                    int y = p[j + 1];
                    p[j] = 0;
                    p[j + 1] = 0;
                    auto add = [&](int count)
                    {
                        array<int, 16> id{};
                        int top = 0;
                        Key key = 0;
                        for (int t = 0; t <= m; t++)
                        {
                            int v = p[t];
                            if (v && !id[v]) id[v] = ++top;
                            key |= Key(id[v]) << (4 * t);
                        }
                        next[count][key] += ways;
                    };
                    if (a[i][j] == '*')
                    {
                        if (!x && !y) add(c);
                    }
                    else if (!x && !y)
                    {
                        if (down && right)
                        {
                            p[j] = 15;
                            p[j + 1] = 15;
                            add(c);
                        }
                    }
```

延伸、合并与闭环计数（接上页同一函数）：

```
                    else if (!x || !y)
                    {
                        if (down)
                        {
                            p[j] = x + y;
                            add(c);
                            p[j] = 0;
                        }
                        if (right)
                        {
                            p[j + 1] = x + y;
                            add(c);
                        }
                    }
                    else if (x != y)
                    {
                        for (int &v : p)
                            if (v == y) v = x;
                        add(c);
                    }
                    else if (c < k)
                    {
                        int parity = 0;
                        for (int t = 0; t < j; t++) parity ^= p[t] != 0;
                        if (nested || parity == 0) add(c + 1);
                    }
                }
            }
            dp.swap(next);
        }
    }
    auto it = dp[k].find(0);
    return it == dp[k].end() ? Count(0) : it->second;
}
```

第三章　动态 DP

3.1　动态 DP：树上最大权独立集带修改

问题描述：给定点带权的树，支持修改单点权值，每次修改后求全树的最大权独立集（所选点两两不相邻，可为空集）。

核心思路：设 f_0, f_1 为不选、选 u 时子树的最优值，只把轻儿子的贡献汇总为 $g_0 = \sum \max(f_0, f_1)$、$g_1 = w_u + \sum f_0$，则 u 的 (f_0, f_1) 等于 $(\max, +)$ 矩阵 $[[g_0, g_0], [g_1, -\infty]]$ 乘重儿子的向量。线段树按 `dfn` 存各点矩阵，把重链从链顶到链底连乘，再作用于 $(0, -\infty)$ 就得到链顶的 DP 值（`chain`）。修改时更新本点矩阵并沿重链上跳：每到一条链，求出链顶新旧 DP 值，用其差修正链顶父亲的 g_0, g_1，再更新父亲的矩阵；根链链顶的 $\max(f_0, f_1)$ 即答案。

代码依赖：HLD（第 399 页，39.3.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：DynamicDP(h,w) 接收已 build 的 HLD h（点号 1..n，根任意，须比本对象活得久）与点权 w[1..n]；set(u,val) 修改点权，answer() 返回全树最大权独立集（允许空集）。每个点维护轻儿子贡献 g0=∑max(f0,f1)、g1=w+∑f0，对应 (max,+) 矩阵 [[g0,g0],[g1,-inf]]；线段树按 dfn 维护矩阵积，重链自顶向下连乘，再乘 (0,-inf) 得链顶 (f0,f1)。修改时沿重链上跳，用链顶新旧值之差更新父亲的 g0、g1。构造 O(n)，每次修改 O($\log^2$ n)，查询 O(log n)；HLD 的 DFS 为递归。换成其他树形 DP：把转移写成 f_u 等于 M_u 乘 f_ 重儿子的半环矩阵乘法，矩阵只依赖 u 自身与轻儿子信息即可套用。

```
// Dynamic DP: max-weight independent set of a tree under point weight changes (P4719).
// Vertices 1..n, built HLD (any root). f0/f1 = best in subtree with u excluded/included.
struct DynamicDP
{
    using ll = long long;
    static constexpr ll NEG = LLONG_MIN / 4;
    struct M
    {
        ll a[2][2];
    };

    // (max,+) product; matrices along a heavy chain multiply from top to bottom.
    static M mul(const M &x, const M &y)
    {
        M r;
        for (int i = 0; i < 2; i++)
            for (int j = 0; j < 2; j++) r.a[i][j] = max(x.a[i][0] + y.a[0][j], x.a[i][1] + y.a[1][j]);
        return r;
    }

    const HLD &h;
    int n;
    vector<ll> w, g0, g1; // g0 = sum max(f0,f1), g1 = w + sum f0, over light children only
    vector<int> ed;       // ed[top] = dfn of the chain's last vertex
    vector<M> t;

    M leaf(int u) const { return {{{g0[u], g0[u]}, {g1[u], NEG}}}; }

    void build(int p, int l, int r)
    {
        if (l == r)
```

```
    {
        t[p] = leaf(h.rk[l]);
        return;
    }
    int m = (l + r) / 2;
    build(2 * p, l, m), build(2 * p + 1, m + 1, r);
    t[p] = mul(t[2 * p], t[2 * p + 1]);
}

void update(int p, int l, int r, int x)
{
    if (l == r)
    {
        t[p] = leaf(h.rk[l]);
        return;
    }
    int m = (l + r) / 2;
    x <= m ? update(2 * p, l, m, x) : update(2 * p + 1, m + 1, r, x);
    t[p] = mul(t[2 * p], t[2 * p + 1]);
}

M query(int p, int l, int r, int ql, int qr) const
{
    if (ql <= l && r <= qr) return t[p];
    int m = (l + r) / 2;
    if (qr <= m) return query(2 * p, l, m, ql, qr);
    if (ql > m) return query(2 * p + 1, m + 1, r, ql, qr);
    return mul(query(2 * p, l, m, ql, qr), query(2 * p + 1, m + 1, r, ql, qr));
}

// (f0, f1) of a chain top: chain product applied to the empty vector (0, -inf).
pair<ll, ll> chain(int top) const
{
    M p = query(1, 1, n, h.dfn[top], ed[top]);
    return {p.a[0][0], p.a[1][0]};
}

DynamicDP(const HLD &h, const vector<ll> &weight) : h(h), n(h.n), w(weight), g0(n + 1), g1(n + 1), ed(n +
 1), t(4 * n + 4)
{
    vector<ll> f0(n + 1), f1(n + 1);
    for (int i = n; i >= 1; i--)
    {
        int u = h.rk[i];
        f1[u] += w[u], g1[u] += w[u];
        for (int v : h.g[u])
            if (v != h.fa[u])
            {
                f0[u] += max(f0[v], f1[v]), f1[u] += f0[v];
                if (v != h.son[u]) g0[u] += max(f0[v], f1[v]), g1[u] += f0[v];
            }
    }
    for (int i = 1; i <= n; i++) ed[h.top[h.rk[i]]] = i;
    build(1, 1, n);
}

```

```
    // Set w[u] = val in O(log^2 n).
    void set(int u, ll val)
    {
        g1[u] += val - w[u], w[u] = val;
        while (true)
        {
            int top = h.top[u];
            auto old = chain(top);
            update(1, 1, n, h.dfn[u]);
            auto now = chain(top);
            if (h.fa[top] == top) break; // root chain
            int p = h.fa[top];
            g0[p] += max(now.first, now.second) - max(old.first, old.second);
            g1[p] += now.first - old.first;
            u = p;
        }
    }

    // Maximum weight of an independent set in the whole tree.
    ll answer() const
    {
        auto [a, b] = chain(h.rk[1]);
        return max(a, b);
    }
};
```

模板题使用：Luogu P4719 【模板】动态 DP

题意：n,m≤10^5 的树，每次把一个点的权改为 y，输出全树最大权独立集。以 1 为根建 HLD 后构造 DynamicDP，每次 set 后输出 answer。

所需模板：HLD（第 399 页，39.3.2 节）、DynamicDP（第 20 页，3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4719

```
int main()
{
    int n, m;
    scanf("%d%d", &n, &m);
    vector<long long> w(n + 1);
    for (int i = 1; i <= n; i++) scanf("%lld", &w[i]);
    HLD h(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        scanf("%d%d", &u, &v);
        h.add(u, v);
    }
    h.build(1);
    DynamicDP d(h, w);
    while (m--)
    {
        int x;
        long long y;
        scanf("%d%lld", &x, &y);
        d.set(x, y);
        printf("%lld\n", d.answer());
    }
}
```

第四章　DP 优化

4.1　斜率优化

4.1.1　单调斜率凸包：最小值与决策

功能：维护直线集合 $y = mx + b$：直线按斜率非增加入、查询点 x 非减时，求所有直线在 x 处的最小值及取到它的直线编号，用于斜率优化 DP。

核心思路：下凸壳存于数组 `q`，有效部分从 `head` 开始，斜率依次递减，每条直线在一段连续的 x 上最优。加线时同斜率只保留截距更小者；若队尾直线 b 与前一条 a 的交点不在 b 与新线 c 的交点左侧，b 永远不会最优，弹出（交点比较用 int128 交叉相乘，不做除法）。查询 x 单调增大，最优直线只会往斜率更小的一侧移动，`query` 在下一条直线严格更优时推进 `head`，被越过的直线以后也不会再最优。

接口契约：默认空；add(m,b,id) 按斜率非增加入直线 mx+b，query(x) 按 x 非减查询，返回 (pair<int128,int>)(最小值, 编号)。首次查询前至少加一条线；可交替加线与查询。并列保留最早加入的直线，不是最小 id；同斜率只留最小截距，截距也同则留旧线。q/head 仅供只读；查询会永久淘汰旧候选，不支持回退 x 或任意斜率。要求 |m|,|b|,|x|<=10^18，总加线数 <INT_MAX。先提升 int128 再做差与交叉乘法，范围内精确无浮点；返回值勿未经证明窄化为 long long。总 M 次加线、Q 次查询 O(M+Q)，空间 O(M)，被查询淘汰的前缀仍留在 vector 中。

```
struct MonotoneHull
{
    using ll = long long;
    using I = __int128_t;
    struct Line
    {
        ll m, b;
        int id;

        I value(ll x) const
        {
            return I(m) * x + b;
        }
    };
    vector<Line> q;
    int head = 0;
    ll last_x = 0;
    bool queried = false;

    void add(ll m, ll b, int id)
    {
        assert(q.empty() || m <= q.back().m);
        if (int(q.size()) > head && q.back().m == m)
        {
            if (q.back().b <= b) return;
            q.pop_back();
        }
        Line c{m, b, id};
        while (int(q.size()) - head >= 2)
        {
            auto a = q[q.size() - 2];
            auto b = q.back();
            if ((I(b.b) - a.b) * (b.m - I(c.m)) <
                (I(c.b) - b.b) * (a.m - I(b.m))) break;
            q.pop_back();
        }
        q.push_back(c);
    }
```

查询（接上页同一结构体）：

```
    pair<I, int> query(ll x)
    {
        assert(int(q.size()) > head);
        assert(!queried || last_x <= x);
        queried = true;
        last_x = x;
        while (head + 1 < int(q.size()) && q[head + 1].value(x) < q[head].value(x))
        {
            head++;
        }
        return {q[head].value(x), q[head].id};
    }
};
```

4.2 四边形不等式优化

4.2.1 定义与性质

本节代价 $w(l,r)$ 只对闭区间 $1 \le l \le r \le n$ 定义，DP 求**最小值**；求最大值时把下列不等号全部反向。

四边形不等式 对所有 $a \le b \le c \le d$，

$$w(a,c) + w(b,d) \le w(a,d) + w(b,c),$$

即“两段交叉区间的代价和”不超过“外层区间加内层区间的代价和”。把 $M_{l,r} = w(l,r)$ 看作上三角矩阵，这正是 **Monge 性质**：任取两行 $a < b$、两列 $c < d$（且 $b \le c$），主对角线之和不超过副对角线之和。**只需验证相邻格**：若对所有 $i+1 \le j$ 有 $w(i,j) + w(i+1,j+1) \le w(i,j+1) + w(i+1,j)$，把 $a \le i < b$、$c \le j < d$ 的这些不等式全部相加，中间项相消即得一般形式（$b \le c$ 保证每格都有定义）。相邻形式等价于“右端扩一格的增量 $w(l,r+1) - w(l,r)$ 随左端 l 右移而不增”。

区间包含单调性 对所有 $a \le b \le c \le d$，$w(b,c) \le w(a,d)$，即大区间代价不小于其子区间。它与四边形不等式**互不蕴含**：$w(l,r) = -[l=1]$ 使四边形不等式取等号，却有 $w(2,3) = 0 > w(1,4) = -1$；$w(l,r) = [l<r]$ 满足包含单调性，却有 $w(1,2) + w(2,3) = 2 > w(1,3) + w(2,2) = 1$。

最优决策单调性 它是 **DP 的最优转移点**的性质，不是代价函数的性质：某一转移把 i 的最优决策记为 $opt(i)$，若 $i < i'$ 时 $opt(i) \le opt(i')$，称决策单调。多个决策并列最优时，opt 必须按统一规则取（本节一律取**最小**的最优决策），否则单调性可能不成立，代码也不能按单调性剪枝。以下两种 DP 形式分别说明：四边形不等式（及 Knuth 情形下的包含单调性）**推出**哪一种决策单调性；没有这些条件时决策也可能单调，但必须另行证明。

常见满足条件的代价 非负权的区间和 $w = \sum_{i=l}^{r} x_i$：四边形不等式取等号且包含单调。$w = g(S_r - S_{l-1})$，其中 S 为非降前缀和、g 凸：满足四边形不等式；g 再非降则包含单调。区间内所有无序对的非负权和 $w = \sum_{l \le i < j \le r} e_{ij}$：右减左恰为 $[a, b-1]$ 与 $[c+1, d]$ 之间的点对权，两条件都满足。满足条件的代价的非负线性组合仍满足相应条件。

4.2.2 分组 DP 的分治优化

转移形式 把 $1..n$ 分成恰好 K 个非空连续段，段 $[l,r]$ 的代价为 $w(l,r)$，最小化总代价。令 $f_g(i)$ 表示前 i 个元素恰分成 g 段的最小代价，j 为前一段的结束位置：

$$f_g(i) = \min_{g-1 \le j < i} \big\{ f_{g-1}(j) + w(j+1, i) \big\}, \qquad g \le i \le n.$$

初值 $f_0(0) = 0$，$f_0(i) = +\infty$ $(i > 0)$；$i < g$ 时 $f_g(i) = +\infty$（不可达），答案 $f_K(n)$。

条件与结论 若 w 满足四边形不等式，则对**任意**给定的上一层 f_{g-1}，取最小最优决策 $opt_g(i)$ 时有 $opt_g(i) \le opt_g(i+1)$。不需要包含单调性。

证明. 设 $i < i'$，$j = opt_g(i)$，若 $j' = opt_g(i') < j$。则 j' 对 i 合法、j 对 i' 合法（$j < i < i'$），且两者的 f_{g-1} 有限。取 $(a,b,c,d) = (j'+1, j+1, i, i')$，四边形不等式给出

$$\big[f_{g-1}(j) + w(j+1,i')\big] - \big[f_{g-1}(j') + w(j'+1,i')\big] \le \big[f_{g-1}(j) + w(j+1,i)\big] - \big[f_{g-1}(j') + w(j'+1,i)\big] < 0,$$

最后的严格号来自“j 是 i 的**最小**最优决策而 $j' < j$”。于是 j 对 i' 严格优于 j'，矛盾。□

分治 求 f_g 在 $[l,r]$ 上的值且已知决策落在 $[lo, hi]$ 时，取中点 mid，枚举 $j \in [lo, \min(hi, mid-1)]$ 求出 $opt(mid)$（严格小于才更新，得最小最优决策）；左半递归用 $[lo, opt(mid)]$，右半用 $[opt(mid), hi]$。由上述结论，左右两半的最小最优决策确实落在这两个范围内。同一递归深度各区间的枚举范围首尾相接，总量 $O(n)$，深度 $O(\log n)$，所以一层 $O(n \log n)$、K 层 $O(Kn \log n)$（w 须 $O(1)$ 求值）。

对应接口 `monotone_dp_layer(prev,first,cost,opt)` 计算一层：`prev` 即 $f_{g-1}(0..n)$，`first`= g，`cost(j+1,i)` 即 $w(j+1,i)$，返回 $f_g(0..n)$，`opt[i]` 即 $opt_g(i)$；$i < g$ 时返回 `inf`、`opt` 为 -1。`prev[first-1]`= $f_{g-1}(g-1)$ 必须有限，保证每个 $i \geq g$ 至少有决策 $j = g-1$。分治要求整层 `prev` 事先算完，**不能**直接用于依赖同层未计算状态的转移，例如不限段数的 $f(i) = \min_j\{f(j) + w(j+1,i)\}$：中点处的 $f(j)$ 尚未求出。这类转移需改用 CDQ 分治套分治、单调队列加二分等其他写法。

4.2.3 分治决策单调性：一层分组 DP

问题描述：把 $1..n$ 分成恰好 K 个非空连续段、段代价 $w(l,r)$ 满足四边形不等式时，求最小总代价；本函数完成其中一层转移 $f_g(i) = \min_j\{f_{g-1}(j) + w(j+1,i)\}$。

核心思路：上节已证明最小最优决策 $opt_g(i)$ 随 i 非减。先算中点的最优决策，左半只在不超过它的决策中找、右半只在不小于它的决策中找，每层递归总枚举 $O(n)$。

接口契约：monotone_dp_layer(prev,first,cost,opt) 返回 cur，prev 下标 0..n，1<=first<=n<INT_MAX。对 i=first..n 求 min(prev[j]+cost(j+1,i))，j 在 first-1..i-1，cost 是闭区间代价；用于第 g 组时 first=g。inf=LLONG_MAX/4；prev[first-1] 必须可达，其他 inf 跳过，cost 对全部合法区间有限。所有有限 prev、cost 及转移和须严格在 (-inf,inf)，保证 long long 加法安全。cur[0..first-1]=inf，opt 对应为-1；其余 opt 返回最小最优 j。调用方须证明该并列规则下 opt[i] 非减，本函数不会检查，任意 DP 不能套用。cost 须无副作用且 O(1) 才能得 O(n log(n+1)) 单层时间；递归深度 O(log n)，输出与辅助 O(n)。

```
template<class Cost>
vector<long long> monotone_dp_layer(const vector<long long> &prev, int first,
                                    Cost cost, vector<int> &opt)
{
    using ll = long long;
    constexpr ll inf = LLONG_MAX / 4;
    int n = int(prev.size()) - 1;
    assert(1 <= first && first <= n && prev[first - 1] != inf);
    vector<ll> cur(n + 1, inf);
    opt.assign(n + 1, -1);
    auto solve = [&](auto &&self, int l, int r, int lo, int hi) -> void
    {
        if (l > r) return;
        int mid = l + (r - l) / 2;
        for (int j = lo; j <= min(hi, mid - 1); j++)
        {
            if (prev[j] == inf) continue;
            ll value = prev[j] + cost(j + 1, mid);
            if (value < cur[mid])
            {
                cur[mid] = value;
                opt[mid] = j;
            }
        }
        assert(opt[mid] != -1);
        self(self, l, mid - 1, lo, opt[mid]);
        self(self, mid + 1, r, opt[mid], hi);
    };
    solve(solve, first, n, first - 1, n - 1);
    return cur;
}
```

模板题使用：P4767 [IOI 2000] 邮局加强版

题意：n≤3000 个村庄、k≤300 个邮局，坐标 1..10000，最小化各村到最近邮局距离和。排序后每个邮局服务一段连续村庄，邮局放在段中位数；f_g(i)=min f_{g-1}(j)+w(j+1,i)，w 由前缀和 O(1) 求。左端每左移一格的增量 x[(a+r)/2 下取整]-x[a-1] 随 r 增大不减，故 w 满足四边形不等式，逐层调用 monotone_dp_layer，第 g 层 first=g；总 O(kn log n)。

所需模板：monotone_dp_layer（第 28 页，4.2.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4767

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, k;
    cin >> n >> k;
    vector<long long> x(n + 1), sum(n + 1);
    for (int i = 1; i <= n; i++) cin >> x[i];
    sort(x.begin() + 1, x.end());
    for (int i = 1; i <= n; i++) sum[i] = sum[i - 1] + x[i];
    auto cost = [&](int l, int r) -> long long
    {
        int mid = (l + r) / 2;
        return x[mid] * (mid - l + 1) - (sum[mid] - sum[l - 1]) +
               (sum[r] - sum[mid]) - x[mid] * (r - mid);
    };
    vector<long long> dp(n + 1, LLONG_MAX / 4);
    vector<int> opt;
    dp[0] = 0;
    for (int g = 1; g <= k; g++)
    {
        dp = monotone_dp_layer(dp, g, cost, opt);
    }
    cout << dp[n] << '\n';
}
```

4.2.4 区间 DP 的 Knuth 优化

转移形式 典型如石子合并：

$$f(l,r) = w(l,r) + \min_{l\le k<r}\{f(l,k)+f(k+1,r)\}\ (l<r), \qquad f(i,i)=0.$$

$w(l,r)$ 只在 $l<r$ 时使用，$w(i,i)$ 无意义。若题目给叶子代价 $f(i,i)=e_i$，每个叶子恰出现一次，可先按 0 计算再加 $\sum_{i=l}^{r} e_i$。记 $opt(l,r)$ 为最小最优分割点，约定 $opt(i,i)=i$。

充分条件 对长度至少为 2 的区间，w 同时满足**四边形不等式**与**区间包含单调性**：

$$w(a,c)+w(b,d)\le w(a,d)+w(b,c), \qquad w(b,c)\le w(a,d), \qquad a\le b\le c\le d.$$

结论：$opt(l,r-1)\le opt(l,r)\le opt(l+1,r)$（$r-l\ge 2$），于是只枚举 $k\in[opt(l,r-1),opt(l+1,r)]$。

引理（f 也满足四边形不等式） 对 $d-a$ 归纳证 $f(a,c)+f(b,d)\le f(a,d)+f(b,c)$。$a=b$ 或 $c=d$ 时两边相同。否则 $a<b$、$c<d$，令 $z=opt(a,d)$。若 $b=c$：当 $z<c$ 时，$f(a,c)\le w(a,c)+f(a,z)+f(z+1,c)$，对较短的 $[z+1,d]$ 用归纳得 $f(z+1,c)+f(c,d)\le f(z+1,d)+f(c,c)$，再用 $w(a,c)\le w(a,d)$ 即得 $f(a,c)+f(c,d)\le f(a,d)+f(c,c)$；

$z \geq c$ 对称（拆 $f(c,d)$，对 $[a,z]$ 归纳，用 $w(c,d) \leq w(a,d)$）。若 $b < c$：令 $y = opt(b,c)$。当 $z \leq y$ 时，用 z 拆 $f(a,c)$、用 y 拆 $f(b,d)$，由 w 的四边形不等式与归纳式 $f(z+1,c)+f(y+1,d) \leq f(z+1,d)+f(y+1,c)$ 相加即得；当 $z > y$ 时，用 y 拆 $f(a,c)$、用 z 拆 $f(b,d)$，配合归纳式 $f(a,y)+f(b,z) \leq f(a,z)+f(b,y)$ 同理。□（包含单调性只在 $b=c$ 一步使用，前面的反例 $w=-[l=1]$ 正好在这里失效。）

决策范围的推导 记 $g_k(l,r) = f(l,k)+f(k+1,r)$，比较两个分割点时 $w(l,r)$ 抵消。*左界*：对 $k < k' \leq r-2$，引理用于 $(k+1,k'+1,r-1,r)$ 得 $g_k(l,r)-g_{k'}(l,r) \geq g_k(l,r-1)-g_{k'}(l,r-1)$。取 $k' = K = opt(l,r-1)$，对任意 $k < K$ 右边 > 0（K 是最小最优），故 k 不是 (l,r) 的最优，$opt(l,r) \geq K$。*右界*：令 $K = opt(l+1,r) \geq l+1$，对 $k' > K$，引理用于 $(l,l+1,K,k')$ 得 $g_K(l,r)-g_{k'}(l,r) \leq g_K(l+1,r)-g_{k'}(l+1,r) \leq 0$，所以 K 不劣于更大的分割点，最小最优决策 $\leq K$。左界用到严格不等号、右界只用到非严格不等号，两者在“取最小最优决策”下同时成立；若改为都取最大最优决策也成立，但不能混用。

复杂度 按区间长度递增计算。固定长度 L 时，各 l 的枚举量 $\sum_l \big(opt(l+1,r)-opt(l,r-1)+1\big)$ 相邻项相消，不超过 $2n$；共 n 种长度，时间 $O(n^2)$，表 $O(n^2)$，比朴素 $O(n^3)$ 少一个 n。长度为 2 时 $[opt(l,l), opt(l+1,l+1)] = [l,l+1]$，须截到 $r-1=l$。

与分组 DP 分治优化的区别

	分组 DP 分治优化	区间 DP Knuth 优化
转移	$f_g(i)$ 只依赖已算完的上一层	$f(l,r)$ 依赖同一张表的更短区间
代价条件	仅四边形不等式	四边形不等式 + 区间包含单调性
决策关系	单侧：$opt_g(i) \leq opt_g(i+1)$	双侧：$opt(l,r-1) \leq opt(l,r) \leq opt(l+1,r)$
证明对象	只用 w，与上一层取值无关	先证 f 满足四边形不等式
实现与复杂度	中点分治，每层 $O(n \log n)$	按长度递推，总 $O(n^2)$

反例 $n=4$、$w(l,r) = -[l=1]$：四边形不等式取等号但不满足包含单调性。真值 $f(1,4) = -3$（$k=3$），而 $opt(1,3)=2$、$opt(2,4)=2$，Knuth 只枚举 $k=2$ 得 -2。环形问题（如环形石子合并）复制成长 $2n-1$ 的链，答案取 $\min_i f(i,i+n-1)$。

4.2.5 Knuth 优化：区间合并 DP

问题描述： 计算区间合并型 DP $f(l,r)=w(l,r)+\min_{l\le k<r}\{f(l,k)+f(k+1,r)\}$，$f(i,i)=0$，其中 w 满足四边形不等式与区间包含单调性（如石子合并）。

核心思路： 按区间长度递增计算；由上节的双侧决策关系，(l,r) 的最小最优分割点只在 $[opt(l,r-1),opt(l+1,r)]$ 内枚举，同一长度的枚举量相消为 $O(n)$，总 $O(n^2)$。

接口契约： knuth_interval_dp(n,cost,opt) 返回 f，下标 1..n：f[i][i]=0，l<r 时 f[l][r]=cost(l,r)+min(f[l][k]+f[k+1][r])，k 取 l..r-1；其余位置为 inf=LLONG_MAX/4。opt[i][i]=i，l<r 时 opt[l][r] 为最小最优 k，只在 [opt[l][r-1],opt[l+1][r]] 内枚举。调用方须证明对长度 ≥2 的区间 cost 满足四边形不等式与区间包含单调性（见上文），本函数不检查。1≤n，n+1 个 vector 各 n+1 项须可分配；cost 只对 l<r 调用、无副作用；所有 f、cost 及候选和须严格在 (-inf,inf) 内。时间 O(n^2) 次 cost 与转移，空间 O(n^2)（f 为 long long，opt 为 int）。

```
template<class Cost>
vector<vector<long long>> knuth_interval_dp(int n, Cost cost,
                                            vector<vector<int>> &opt)
{
    using ll = long long;
    constexpr ll inf = LLONG_MAX / 4;
    assert(n >= 1);
    vector<vector<ll>> f(n + 1, vector<ll>(n + 1, inf));
    opt.assign(n + 1, vector<int>(n + 1, -1));
    for (int i = 1; i <= n; i++)
    {
        f[i][i] = 0;
        opt[i][i] = i;
    }
    for (int len = 2; len <= n; len++)
        for (int l = 1, r = len; r <= n; l++, r++)
        {
            ll w = cost(l, r);
            int hi = min(opt[l + 1][r], r - 1);
            for (int k = opt[l][r - 1]; k <= hi; k++)
            {
                ll value = f[l][k] + f[k + 1][r] + w;
                if (value < f[l][r])
                {
                    f[l][r] = value;
                    opt[l][r] = k;
                }
            }
        }
    return f;
}
```

模板题使用：AtCoder AWC0100 O Joining of Circular Tablets（环形石板合并）

题意：2≤N≤3000 块石板成环，重量 1≤A_i≤10^9；每次合并相邻两块，代价为新块重量，求合成一块的最小总代价。复制成长 2N-1 的链，f(l,r)=w(l,r)+min f(l,k)+f(k+1,r)，w 为区间和：四边形不等式取等、区间越大和越大，满足 Knuth 条件。答案取 min f(i,i+N-1)；O(N^2) 时间，表约 432MB（限 1024MB），答案 <10^16。

所需模板：knuth_interval_dp（第 31 页，4.2.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://atcoder.jp/contests/awc0100/tasks/awc0100_o

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<long long> a(n), sum(2 * n);
    for (auto &x : a) cin >> x;
    // Break the circle: positions 1..2n-1 repeat a, every window of n is a line.
    for (int i = 1; i < 2 * n; i++) sum[i] = sum[i - 1] + a[(i - 1) % n];
    auto cost = [&](int l, int r) { return sum[r] - sum[l - 1]; };
    vector<vector<int>> opt;
    auto f = knuth_interval_dp(2 * n - 1, cost, opt);
    long long answer = f[1][n];
    for (int i = 2; i <= n; i++) answer = min(answer, f[i][i + n - 1]);
    cout << answer << '\n';
}
```

4.3 WQS 二分

4.3.1 WQS 二分：至多 k 个不相邻位置

问题描述：给定数组 a 和 k，选出至多 k 个两两不相邻的位置，使所选元素之和最大。

核心思路：WQS 二分：恰选 j 个时的最优值关于 j 是凹的，于是给每个选中位置扣罚 λ，去掉个数限制做线性 DP：`solve` 中 `cur`、`prev` 分别是前 i 个、前 $i-1$ 个位置的最优（权和，个数），权和相同时取个数多者。罚 0 时最优个数已不超过 k 就直接返回；否则二分最大的非负整数 λ，使最优个数仍至少为 k，此时恰选 k 个也是该罚值下的最优解之一，答案为惩罚后最优值加 λk。

接口契约：wqs_independent_set(a,k) 返回 0-based 数组中至多选 k 个两两不相邻位置的最大权和；允许空选、空数组、负权，k>=0。n=a.size()<INT_MAX，W=max|a[i]|，要求 n*W<=LLONG_MAX/4（以足够宽类型核验），因此不含 LLONG_MIN。只返回最优值，不返回方案。路径独立集等价于路径边匹配，定基数最优值离散凹；每次选择扣 penalty，pair 比较先权和、并列取更多位置。若罚 0 的最优数量 <=k 直接返回；否则二分仍能选至少 k 个的最大非负整数罚值，答案为惩罚后最优值 +penalty*k。不是任意 DP 的 WQS 通用封装。O(n log(W+2)) 时间、O(1) 额外空间。

```
long long wqs_independent_set(const vector<long long> &a, int k)
{
    using ll = long long;
    assert(k >= 0);
    if (k == 0 || a.empty()) return 0;
    auto solve = [&](ll penalty) -> pair<ll, int>
    {
        pair<ll, int> prev{0, 0}, cur{0, 0};
        for (ll w : a)
        {
            pair<ll, int> take{prev.first + w - penalty, prev.second + 1};
            prev = cur;
            cur = max(cur, take);
        }
        return cur;
    };
    auto free = solve(0);
    if (free.second <= k) return free.first;
    ll lo = 0, hi = max(0LL, *max_element(a.begin(), a.end()));
    while (lo < hi)
    {
        ll mid = lo + (hi - lo + 1) / 2;
        if (solve(mid).second >= k) lo = mid;
        else hi = mid - 1;
    }
    return solve(lo).first + lo * k;
}
```

第三部分

字符串

第五章　字符串哈希

5.1　子串与反串双哈希

功能：对字符串预处理双模多项式哈希，常数时间取任意子串或其反串的哈希值，并能拼接两段哈希；用于子串判等、回文判断等。

核心思路：字符编码为字节值加一，在两个模数下分别维护正串前缀哈希 `pre`、反串前缀哈希 `rev` 与底数幂 `pw`。子串 $[l,r)$ 的哈希为 $\mathrm{pre}_r-\mathrm{pre}_l\cdot B^{r-l}$；取反串时把区间映射为反串上的 $[n-r,n-l)$。`join` 计算 $h_a\cdot B^{\mathrm{len}_b}+h_b$，即串 a 后接串 b 的哈希。判等时须同时比较长度。

接口契约：StringHash(s, base)，n<INT_MAX，两个底数分别在 [257,mod-2]，所有比较对象共享同组底数。get(l, r, rev=false) 取 0-based 半开区间；rev=true 反转原区间，空区间返回 {0,0}。join(a, b, len_b) 拼接，要求规范余数、同底数及 0≤len_b≤n，可拼出长度超过 n 的哈希。构造 O(n)，查询/拼接 O(1)，约 24(n+1) 字节；按 unsigned char 加 1 编码。判等必须先比长度；双哈希仍可能碰撞。理想独立均匀底数下，固定同长不同串的碰撞上界为两个 min(1,(L-1)/(mod[j]-258)) 之积，多次比较用联合界；可预测种子和适应性输入不享有该保证。需确定性时用直接比较或 SuffixArray + SuffixLCP；回文可用 Manacher。成员只读。

```
struct StringHash
{
    using H = array<int, 2>;
    static constexpr H mod = {1000000007, 1000000009};
    int n;
    H base;
    vector<H> pw, pre, rev;

    StringHash(const string &s, H b)
        : n(s.size()), base(b), pw(n + 1), pre(n + 1), rev(n + 1)
    {
        pw[0] = {1, 1};
        for (int j = 0; j < 2; j++)
        {
            assert(257 <= base[j] && base[j] <= mod[j] - 2);
            for (int i = 0; i < n; i++)
            {
                int x = (unsigned char)s[i] + 1;
                int y = (unsigned char)s[n - 1 - i] + 1;
                pw[i + 1][j] = 1LL * pw[i][j] * base[j] % mod[j];
                pre[i + 1][j] = (1LL * pre[i][j] * base[j] + x) % mod[j];
                rev[i + 1][j] = (1LL * rev[i][j] * base[j] + y) % mod[j];
            }
        }
    }
```

子串、反串与拼接（接上页同一结构体）：

```
    H get(int l, int r, bool reversed = false) const
    {
        assert(0 <= l && l <= r && r <= n);
        if (reversed)
        {
            int x = n - r;
            r = n - l;
            l = x;
        }
        const auto &h = reversed ? rev : pre;
        H ans;
        for (int j = 0; j < 2; j++)
        {
            long long x = h[r][j] - 1LL * h[l][j] * pw[r - l][j] % mod[j];
            ans[j] = (x + mod[j]) % mod[j];
        }
        return ans;
    }

    H join(H a, H b, int len_b) const
    {
        assert(0 <= len_b && len_b <= n);
        for (int j = 0; j < 2; j++)
        {
            a[j] = (1LL * a[j] * pw[len_b][j] + b[j]) % mod[j];
        }
        return a;
    }
};
```

模板题使用：P3370 【模板】字符串哈希

题意：输入 N 个非空字母数字串，输出不同串数。随机底数全局共享，按长度与双哈希去重；概率判等，AC 不证明零碰撞。

所需模板：StringHash（第 35 页，5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3370

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    mt19937 rng(random_device{}());
    StringHash::H base;
    for (int j = 0; j < 2; j++)
    {
        base[j] = uniform_int_distribution<int>(257, StringHash::mod[j] - 2)(rng);
    }
    int n;
    cin >> n;
    set<pair<int, StringHash::H>> seen;
    while (n--)
    {
        string s;
        cin >> s;
        StringHash h(s, base);
        seen.insert({int(s.size()), h.get(0, s.size())});
    }
    cout << seen.size() << '\n';
}
```

第六章　前缀函数与 KMP 算法

6.1　前缀函数

问题描述：对字符串 s 求前缀函数：π_i 为前缀 $s[0..i]$ 的最长真 border（既是真前缀又是后缀的串）的长度。例如 `abab` 的前缀函数为 $0, 0, 1, 2$。

核心思路：$s[0..i]$ 的 border 去掉末字符后必是 $s[0..i-1]$ 的 border，而后者的全部 border 长度依次为 $\pi_{i-1}, \pi_{\pi_{i-1}-1}, \ldots$。代码从 `j = p[i-1]` 开始，失配就跳到 `p[j-1]`，直到 $s_i = s_j$ 或 `j` 为 0；匹配则长度为 $j+1$。每步 `j` 至多加一，跳转总次数线性。

接口契约：prefix_function(s) 返回每个前缀的最长真 border 长度，0-based；空串返回空数组。O(n)，无状态，直接抄写函数。

```
vector<int> prefix_function(const string &s)
{
    int n = (int)s.size();
    vector<int> p(n);
    for (int i = 1; i < n; i++)
    {
        int j = p[i - 1];
        while (j && s[i] != s[j]) j = p[j - 1];
        p[i] = j + (s[i] == s[j]);
    }
    return p;
}
```

联合使用示例见第 39 页（Luogu P3375），其中直接调用本模板。

6.2 KMP 匹配

问题描述：在文本 s 中找出模式串 t 的全部出现（允许重叠），返回各次出现的起点。

核心思路：先求 t 的前缀函数 `p`。扫描文本时 `j` 是当前已匹配的 t 的前缀长度：失配时沿 `j = p[j-1]` 退到更短的 border 再比较，字符相等则 `j` 加一。`j` 等于 $|t|$ 时记录起点 $i-|t|+1$，再退到 `p[j-1]` 继续匹配，因此不会漏掉重叠的出现。

代码依赖：prefix_function（第 38 页，6.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：kmp_match(s,t) 返回非空模式串 t 在 s 中的所有起点（0-based），允许重叠。需要同章前缀函数。O(|s|+|t|)。

```
vector<int> kmp_match(const string &s, const string &t)
{
    assert(!t.empty());
    auto p = prefix_function(t);
    vector<int> ans;
    for (int i = 0, j = 0; i < (int)s.size(); i++)
    {
        while (j && s[i] != t[j]) j = p[j - 1];
        j += (s[i] == t[j]);
        if (j == (int)t.size())
        {
            ans.push_back(i - j + 1);
            j = p[j - 1];
        }
    }
    return ans;
}
```

模板题使用：Luogu P3375

题意：求模式串 t 在文本 s 中所有出现的位置，包含重叠匹配；模板返回 0-based 位置，输出加 1。最后一行直接输出 prefix_function(t)，它是模式串各前缀的最长真 border 长度。题目输入为非空大写串，函数按字符相等比较，不限于小写。

所需模板：prefix_function（第 38 页，6.1 节）、kmp_match（第 39 页，6.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3375

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s, t;
    cin >> s >> t;
    for (int p : kmp_match(s, t)) cout << p + 1 << '\n';
    for (int x : prefix_function(t)) cout << x << ' ';
    cout << '\n';
}
```

第七章　Z 函数（扩展 KMP）

7.1　Z 函数

问题描述：对字符串 s 求 Z 数组：z_i 为后缀 $s[i..]$ 与 s 本身的最长公共前缀长度。例如 aaba 的 Z 数组为 $4, 1, 0, 1$。

核心思路：维护右端最远的已知匹配段 $[l, r)$，满足 $s[l..r) = s[0..r-l)$。对 $i < r$，$s[i..r)$ 与 $s[i-l..r-l)$ 相同，所以 z_i 至少为 $\min(r-i, z_{i-l})$；之后逐字符向右比较延长，若越过 r 就把匹配段更新为 $[i, i+z_i)$。r 只增不减，总比较次数线性。

接口契约：z_function(s) 返回后缀 s[i..] 与原串的 LCP，约定 z[0]=n，空串返回空数组。O(n)。

```
vector<int> z_function(const string &s)
{
    int n = (int)s.size();
    vector<int> a(n);
    if (n) a[0] = n;
    for (int i = 1, l = 0, r = 0; i < n; i++)
    {
        if (i < r) a[i] = min(r - i, a[i - l]);
        while (i + a[i] < n && s[a[i]] == s[i + a[i]]) ++a[i];
        if (i + a[i] > r)
        {
            l = i;
            r = i + a[i];
        }
    }
    return a;
}
```

模板题使用：Library Checker zalgorithm

题意：给非空字符串，对每个起点求其后缀与原串的最长公共前缀长度。直接输出 Z 数组，z[0] 等于串长。

所需模板：z_function（第 40 页，7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/zalgorithm

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    auto z = z_function(s);
    for (int i = 0; i < (int)z.size(); i++)
        cout << z[i] << (i + 1 == (int)z.size() ? '\n' : ' ');
}
```

7.2 两串扩展 KMP

问题描述：给定文本 s 和模式 t，求 t 的 Z 数组，以及 s 的每个后缀 $s[i..]$ 与 t 的最长公共前缀长度 p_i。

核心思路：先用 z_function 求 t 的 Z 数组。再像 Z 函数一样维护右端最远的匹配段 $[l, r)$，满足 $s[l..r) = t[0..r-l)$。对 $i < r$，$s[i..r)$ 等于 $t[i-l..r-l)$，而 $t[i-l..]$ 与 t 的 LCP 为 z_{i-l}，故 p_i 的初值取 $\min(r-i, z_{i-l})$，然后逐字符向右扩展，越过 r 时更新匹配段。两串分开处理，不需要拼接分隔符。

代码依赖：z_function（第 40 页，7.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：exkmp(s,t) 返回 {z,p}：z 为模式 t 的 Z 数组，z[0]=|t|；p[i] 是文本后缀 s[i..] 与 t 的 LCP 长度，均 0-based。空串合法，t 空时 p 全 0，s 空时 p 为空。任意字节相等比较，含零字节与高位字节；不拼接分隔符、不拷贝输入。需要 z_function。两串长度均须 <INT_MAX；O(|s|+|t|) 时间，两个输出 vector 共 O(|s|+|t|) 空间，返回时移动缓冲区。维护右端最远的匹配区间 [l,r)，区间内利用模式自身 Z 值再向右扩展。P5410 的 1-based 权值乘法应先提升到 long long。

```
// Return {Z(t), LCP(s[i..], t) for every i}; empty strings are allowed.
pair<vector<int>, vector<int>> exkmp(const string &s, const string &t)
{
    int n = s.size(), m = t.size();
    auto z = z_function(t);
    vector<int> p(n);
    for (int i = 0, l = 0, r = 0; i < n; i++)
    {
        if (i < r) p[i] = min(r - i, z[i - l]);
        while (p[i] < m && p[i] < n - i && s[i + p[i]] == t[p[i]])
            p[i]++;
        if (i + p[i] > r)
        {
            l = i;
            r = i + p[i];
        }
    }
    return {move(z), move(p)};
}
```

模板题使用：Luogu P5410 / 扩展 KMP

题意：输入小写串 a、b，长度均 1..20000000。求 b 的 Z 数组与 b 对 a 各后缀的 LCP 数组，各输出 xor((i+1)*(长度 +1))。z[0]=|b|；乘法先提升 long long。直接求两串 LCP，不拼接整串。

所需模板：z_function（第 40 页，7.1 节）、exkmp（第 41 页，7.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5410

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string a, b;
    cin >> a >> b;
    auto [z, p] = exkmp(a, b);
    long long x = 0, y = 0;
    for (int i = 0; i < (int)z.size(); i++)
        x ^= 1LL * (i + 1) * (z[i] + 1);
    for (int i = 0; i < (int)p.size(); i++)
        y ^= 1LL * (i + 1) * (p[i] + 1);
    cout << x << '\n' << y << '\n';
}
```

第八章　AC 自动机

8.1　AC 自动机出现次数

功能：多模式串匹配：插入若干模式串建成 AC 自动机后，对给定文本一次求出每个 Trie 节点所代表的串（含每个模式）在文本中的出现次数，允许重叠。

核心思路：`add` 把模式插入 Trie，返回终止节点。`build` 按 BFS 序求失配指针 `fail`（当前串在 Trie 中的最长真后缀节点），同时把缺失的转移补成 `fail` 节点的同字符转移，得到完整的自动机，BFS 序存入 `order`。`count` 让文本在自动机上走，每到一个状态计数加一，表示该位置以此状态的串结尾（根计入空前缀一次）；再按 BFS 逆序把计数沿 `fail` 累加给后缀节点，每个节点的计数就成为其串的出现次数。

接口契约：add(s)/count(s) 默认小写 string；add(s,offset)/count(s,offset) 接受 string 或 vector<int>，每个元素减 offset 须在 0..25，同一自动机使用一致编码。字符串字面量请先转 string 再传双参数入口。add 返回终态，支持空模式返回根 0；count 返回逐状态重叠出现次数，根恒为文本长度 +1。全部插入后 build 一次，随后只读且可重复查询；重复模式共享状态。a[u].len 为 Trie 深度，go/fail 与 order 可读，order 不含根。size 用 a.size()，重置赋新对象。构建 O(26V+L)，查询 O(|text|+V)，空间 O(26V)。状态数、深度须在 int 内。非空旧用法不变；旧根计数语义需减 1。出现过的模式编号数仍按每个终点分别判断 cnt>0。

```
struct AhoCorasick
{
    struct Node
    {
        array<int, 26> go{};
        int fail = 0, len = 0;
    };

    vector<Node> a{Node{}};
    vector<int> order;
    bool built = false;

    template <class S>
    int add(const S &s, int offset)
    {
        assert(!built);
        int u = 0;
        for (auto ch : s)
        {
            long long c = (long long)ch - offset;
            assert(0 <= c && c < 26);
            if (!a[u].go[c])
            {
                int v = (int)a.size();
                a[u].go[c] = v;
                a.push_back(Node{});
                a[v].len = a[u].len + 1;
            }
            u = a[u].go[c];
        }
        return u;
    }

    int add(const string &s)
    {
        return add(s, 'a');
    }
```

构建失败链接与统计出现次数（接上页同一结构体）：

```
    void build()
    {
        assert(!built);
        built = true;
        queue<int> q;
        for (int v : a[0].go)
            if (v) q.push(v);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            order.push_back(u);
            for (int c = 0; c < 26; c++)
            {
                int v = a[u].go[c];
                if (v)
                {
                    a[v].fail = a[a[u].fail].go[c];
                    q.push(v);
                }
                else
                    a[u].go[c] = a[a[u].fail].go[c];
            }
        }
    }

    template <class S>
    vector<long long> count(const S &s, int offset) const
    {
        assert(built);
        vector<long long> ans(a.size());
        ans[0] = 1;
        int u = 0;
        for (auto ch : s)
        {
            long long c = (long long)ch - offset;
            assert(0 <= c && c < 26);
            u = a[u].go[c];
            ++ans[u];
        }
        for (int i = (int)order.size() - 1; i >= 0; i--)
            ans[a[order[i]].fail] += ans[order[i]];
        return ans;
    }

    vector<long long> count(const string &s) const
    {
        return count(s, 'a');
    }
};
```

模板题使用：Luogu P5357

题意：给多个小写字母模式和文本，输出每个模式的出现次数，允许重叠。先保存 add 返回的终止状态，全部插入后 build，再 count 文本并按状态取值；重复模式共享状态，仍按输入顺序输出。

所需模板：AhoCorasick（第 43 页，8.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5357

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    AhoCorasick ac;
    vector<int> terminal(n);
    for (int &id : terminal)
    {
        string pattern;
        cin >> pattern;
        id = ac.add(pattern);
    }
    ac.build();
    string text;
    cin >> text;
    auto count = ac.count(text);
    for (int id : terminal) cout << count[id] << '\n';
}
```

第九章　后缀数组 (SA)

9.1　后缀数组简介

9.1.1　线性后缀数组（SA-IS）

功能：对字节串或整数序列线性构造后缀数组：`sa` 为按字典序排列的后缀起点，`rk` 为其逆排列，`lcp` 为排名相邻两后缀的最长公共前缀。

核心思路：诱导排序。从右往左给位置定类型：后缀小于下一个后缀为 S 型（`type` 为真），否则为 L 型，末位视为 L 型，因而不需要哨兵；左邻为 L 型的 S 型位置称为 LMS。每个字符桶分为前部 L 区与后部 S 区。`induce` 先把给定顺序的 LMS 放入各桶 S 区，以末位后缀为种子，从左往右由已放好的后缀 v 把 L 型的 $v-1$ 放到桶的 L 区，再从右往左把 S 型的 $v-1$ 放到桶尾。第一次诱导后 LMS 子串已有序，相邻比较得到名字和缩减串，对缩减串递归排序，再按得到的 LMS 顺序第二次诱导即为最终结果。`lcp` 用 Kasai 算法按原串位置顺序计算。

思路中的情况	代码中的处理	行
划分 L/S 类型	自右向左递推，相等字符继承右邻的类型	46
计算桶边界	`lo[c]` 为桶 c 起点，`hi[c]` 为桶内 S 区起点	52
诱导：放种子	LMS 已按给定顺序放入 S 区，再把末位后缀放到其桶的 L 区开头	61
诱导：L 型、S 型	正向扫描放 L 型，反向扫描从桶尾放 S 型	71
给 LMS 子串命名	按第一次诱导的顺序比较相邻 LMS 子串（长度与逐字符），不同则名字加一	94
递归与第二次诱导	对缩减串递归求后缀数组，映回 LMS 位置后再诱导一次	103
求 LCP	Kasai：相邻原串位置的 LCP 至多减一	31

接口契约：SAIS(s,alphabet)：整数符号 0..alphabet-1，1≤alphabet<INT_MAX，长度 n<INT_MAX；SAIS(string) 按 unsigned byte 处理，支持零字节。sa/rk/lcp 均长 n，0-based；lcp[0]=0，其余为相邻后缀 LCP。无需哨兵，空串结果为空。诱导排序时间/空间 O(n+alphabet)，递归深度 O(log n)；任意值离散化另需 O(n log n)。保留倍增 SuffixArray；SuffixLCP 不直接接受本类型。

```
struct SAIS
{
    vector<int> sa, rk, lcp;

    SAIS(const string &s)
    {
        assert(s.size() < INT_MAX);
        vector<int> a;
        a.reserve(s.size());
        for (unsigned char c : s) a.push_back(c);
        *this = SAIS(a, 256);
    }

    SAIS(const vector<int> &s, int alphabet)
    {
        assert(s.size() < INT_MAX && 1 <= alphabet && alphabet < INT_MAX);
        for (int c : s) assert(0 <= c && c < alphabet);
        int n = s.size();
        sa = sort_suffixes(s, alphabet);
        rk.resize(n);
        lcp.assign(n, 0);
        for (int i = 0; i < n; i++) rk[sa[i]] = i;
        for (int i = 0, k = 0; i < n; i++)
        {
            if (!rk[i])
            {
                k = 0;
                continue;
            }
            int j = sa[rk[i] - 1];
            while (k < n - i && k < n - j && s[i + k] == s[j + k]) k++;
            lcp[rk[i]] = k;
            if (k) k--;
        }
    }
```

类型与桶内诱导（接上页同一结构体）：

```
private:
    static vector<int> sort_suffixes(const vector<int> &s, int alphabet)
    {
        int n = s.size();
        if (!n) return {};
        if (n == 1) return {0};
        vector<int> sa(n), lo(alphabet + 1), hi(alphabet + 1);
        vector<bool> type(n);
        for (int i = n - 2; i >= 0; i--)
            type[i] = s[i] == s[i + 1] ? type[i + 1] : s[i] < s[i + 1];
        for (int i = 0; i < n; i++)
            if (type[i]) lo[s[i] + 1]++;
            else hi[s[i]]++;
        for (int c = 0; c < alphabet; c++)
        {
            hi[c] += lo[c];
            lo[c + 1] += hi[c];
        }
        auto induce = [&](const vector<int> &lms)
        {
            fill(sa.begin(), sa.end(), -1);
            vector<int> pos = hi;
            for (int i : lms) sa[pos[s[i]]++] = i;
            pos = lo;
            sa[pos[s[n - 1]]++] = n - 1;
            for (int i = 0; i < n; i++)
            {
                int v = sa[i];
                if (v > 0 && !type[v - 1]) sa[pos[s[v - 1]]++] = v - 1;
            }
            pos = lo;
            for (int i = n - 1; i >= 0; i--)
            {
                int v = sa[i];
                if (v > 0 && type[v - 1]) sa[--pos[s[v - 1] + 1]] = v - 1;
            }
        };
```

LMS 命名与递归（接上页同一结构体）：

```
        vector<int> id(n, -1), lms;
        for (int i = 1; i < n; i++)
            if (!type[i - 1] && type[i])
            {
                id[i] = lms.size();
                lms.push_back(i);
            }
        induce(lms);
        int m = lms.size();
        if (m)
        {
            vector<int> order, reduced(m);
            for (int v : sa)
                if (id[v] >= 0) order.push_back(v);
            int names = 1;
            for (int i = 1; i < m; i++)
            {
                int a = order[i - 1], b = order[i];
                int x = id[a] + 1 < m ? lms[id[a] + 1] : n;
                int y = id[b] + 1 < m ? lms[id[b] + 1] : n;
                bool same = x - a == y - b;
                if (same)
                {
                    while (a < x && s[a] == s[b]) a++, b++;
                    same = a < n && b < n && s[a] == s[b];
                }
                if (!same) names++;
                reduced[id[order[i]]] = names - 1;
            }
            order = sort_suffixes(reduced, names);
            for (int &i : order) i = lms[i];
            induce(order);
        }
        return sa;
    }
};
```

模板题使用：Library Checker suffixarray

题意：输入长度 1..500000 的小写字符串，按字典序输出所有非空后缀的 0-based 起点。独立 SA-IS 实现；无需附加哨兵，不继承倍增版本的线上记录。

所需模板：SAIS（第 47 页，9.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/suffixarray

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    SAIS suffix(s);
    for (int i = 0; i < (int)s.size(); i++)
        cout << suffix.sa[i] << (i + 1 == (int)s.size() ? '\n' : ' ');
}
```

9.1.2 倍增后缀数组与 LCP

功能：对字节串或整数序列构造后缀数组：`sa[k]` 为字典序第 k 小的后缀起点，`rk` 为其逆排列，`lcp[k]` 为排名相邻两后缀的最长公共前缀。

核心思路：倍增：已知每个后缀前 k 个字符的排名 `rk`，按二元组 $(\mathrm{rk}_i, \mathrm{rk}_{i+k})$ 排序即得前 $2k$ 个字符的顺序，越界的第二关键字记为 0（最小）。第二关键字的顺序无需排序：先放 $i \geq n-k$ 的位置，再按上一轮 `sa` 的顺序放 $i-k$，得到 `order`；随后按第一关键字稳定计数排序并重新编号，排名两两不同时提前结束。最后用 Kasai 算法按原串位置顺序求 `lcp`：相邻起点的 LCP 至多减一，总计线性。

接口契约：构造 SuffixArray(s) 支持任意字节串，按 unsigned char 的 0..255 数值比较，包含内嵌零字节；也可构造 SuffixArray(a,alphabet)，整数序列要求 0≤a[i]<alphabet、1≤alphabet<INT_MAX。重新赋值 SuffixArray(a,alphabet) 可重建。长度须小于 INT_MAX。sa[k] 为第 k 个非空后缀起点，rk[i] 为排名，全部 0-based；lcp[k] 为 sa[k-1] 与 sa[k] 的最长公共前缀，lcp[0]=0。空序列合法，三数组均为空。无需手动追加哨兵，也不输出空后缀。每轮按第二关键字已有顺序、第一关键字稳定计数排序，倍增 O(n log n+alphabet)，空间 O(n+alphabet)；字节串 alphabet=256。大整数先自行保序离散化，避免按最大原值开桶。Kasai 求 lcp 为 O(n)。不同起点 x,y 的 LCP 为 lcp[min(rk[x],rk[y])+1..max(rk[x],rk[y])] 的最小值；本结构不内建 RMQ，需要频繁查询时另接静态 RMQ。同一起点的 LCP 为 n-x。覆盖倍增构造，不替代 DC3/SA-IS。

```
struct SuffixArray
{
    vector<int> sa, rk, lcp;

    SuffixArray(const string &s)
    {
        vector<int> a;
        a.reserve(s.size());
        for (unsigned char c : s) a.push_back(c);
        *this = SuffixArray(a, 256);
    }

    SuffixArray(const vector<int> &s, int alphabet)
    {
        assert(s.size() < INT_MAX);
        assert(1 <= alphabet && alphabet < INT_MAX);
        int n = (int)s.size();
        sa.resize(n);
        rk.resize(n);
        lcp.assign(n, 0);
        if (!n) return;
        vector<int> order(n), old(n), cnt(max(n + 1, alphabet + 1));
        int m = alphabet;
        for (int i = 0; i < n; i++)
        {
            assert(0 <= s[i] && s[i] < alphabet);
            rk[i] = s[i] + 1;
            cnt[rk[i]]++;
        }
        for (int i = 1; i <= m; i++) cnt[i] += cnt[i - 1];
        for (int i = n - 1; i >= 0; i--) sa[--cnt[rk[i]]] = i;
```

```
        for (int k = 1; k < n; k *= 2)
        {
            int p = 0;
            for (int i = n - k; i < n; i++) order[p++] = i;
            for (int i : sa)
                if (i >= k) order[p++] = i - k;
            fill(cnt.begin(), cnt.begin() + m + 1, 0);
            for (int x : rk) cnt[x]++;
            for (int i = 1; i <= m; i++) cnt[i] += cnt[i - 1];
            for (int i = n - 1; i >= 0; i--) sa[--cnt[rk[order[i]]]] = order[i];
            old.swap(rk);
            auto second = [&](int i)
            {
                return i < n - k ? old[i + k] : 0;
            };
            m = 1;
            rk[sa[0]] = m;
            for (int i = 1; i < n; i++)
            {
                int a = sa[i - 1], b = sa[i];
                if (old[a] != old[b] || second(a) != second(b)) m++;
                rk[b] = m;
            }
            if (m == n) break;
        }
        for (int i = 0; i < n; i++) rk[sa[i]] = i;
        for (int i = 0, k = 0; i < n; i++)
        {
            if (!rk[i])
            {
                k = 0;
                continue;
            }
            int j = sa[rk[i] - 1];
            while (k < n - i && k < n - j && s[i + k] == s[j + k]) ++k;
            lcp[rk[i]] = k;
            if (k) --k;
        }
    }
};
```

模板题使用：Luogu P3809

题意：把所有非空后缀按字典序排序，输出各后缀在原串中的起点。模板 sa 为 0-based，题目输出为 1-based，因此加 1。

所需模板：SuffixArray（第 52 页，9.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3809

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    std::string s;
    std::cin >> s;
    SuffixArray suffix(s);
    for (int i = 0; i < (int)s.size(); i++)
        std::cout << suffix.sa[i] + 1 << (i + 1 == (int)s.size() ? '\n' : ' ');
}
```

9.1.3 后缀 LCP 与子串比较

功能：在已建好的后缀数组上快速回答任意两个后缀的最长公共前缀，并比较任意两个子串的字典序。

核心思路：后缀 x, y 的 LCP 等于 `lcp` 数组在排名区间 $(\mathrm{rk}_x, \mathrm{rk}_y]$ 上的最小值，因此对 `lcp` 建 ST 表 `st`，`query` 用两段重叠区间取最小；同起点或空后缀直接返回剩余长度。`compare` 先求两子串起点处后缀的 LCP：若它覆盖较短的子串，结果由长度决定；否则第一个不同字符落在两子串内部，直接比较两后缀的排名。

代码依赖：SuffixArray（第 52 页，9.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：先构造 SuffixArray，再构造 SuffixLCP(sa)。重新赋值 SuffixLCP(sa) 可建新表。复制所需排名与 LCP 数据，不保存原字符串或源对象引用，源对象可销毁或重建。query(x,y) 返回 s[x..n) 与 s[y..n) 的最长公共前缀，0≤x,y≤n，允许空后缀；同起点返回 n-x。compare(l1,r1,l2,r2) 比较两个 0-based 半开子串，允许空串，返回 -1/0/1。若公共前缀覆盖较短子串，按长度比较，否则原后缀排名给出首个不同字符的顺序。字节排序及整数字母表约束继承 SuffixArray。预处理稀疏表 O(n log n) 时间、空间，每次查询 O(1)，无需哈希和原串。n、rk、lg、st 可读，禁止调用方破坏。前缀的最长公共后缀：在反串上构造 SuffixLCP rev，s[0..x) 与 s[0..y) 的最长公共后缀为 rev.query(n-x,n-y)。重复子串（AA 型）计数：枚举半长 len 与锚点 i,i+len，用正反两个 SuffixLCP 求向右、向左可延伸长度，再按区间差分累加。

```
struct SuffixLCP
{
    int n;
    vector<int> rk, lg;
    vector<vector<int>> st;

    SuffixLCP(const SuffixArray &suffix)
    {
        n = suffix.sa.size();
        rk = suffix.rk;
        lg.assign(n + 1, 0);
        for (int i = 2; i <= n; i++) lg[i] = lg[i / 2] + 1;
        st.assign(lg[n] + 1, {});
```

```
        st[0] = suffix.lcp;
        for (int k = 1; k <= lg[n]; k++)
        {
            int len = 1 << k;
            st[k].resize(n - len + 1);
            for (int i = 0; i <= n - len; i++)
                st[k][i] = min(st[k - 1][i], st[k - 1][i + len / 2]);
        }
    }

    int query(int x, int y) const
    {
        assert(0 <= x && x <= n && 0 <= y && y <= n);
        if (x == y || x == n || y == n) return n - max(x, y);
        int a = rk[x], b = rk[y];
        if (a > b) swap(a, b);
        int k = lg[b - a];
        return min(st[k][a + 1], st[k][b - (1 << k) + 1]);
    }

    // Compare [l1,r1) and [l2,r2); return -1, 0 or 1.
    int compare(int l1, int r1, int l2, int r2) const
    {
        assert(0 <= l1 && l1 <= r1 && r1 <= n);
        assert(0 <= l2 && l2 <= r2 && r2 <= n);
        int a = r1 - l1, b = r2 - l2;
        if (query(l1, l2) >= min(a, b)) return (a > b) - (a < b);
        return rk[l1] < rk[l2] ? -1 : 1;
    }
};
```

9.1.4 整数后缀数组的边界与 SAM 次数

SuffixArray 整数入口要求 $0 \le a[i] < alphabet$，大整数先保序离散化。sa/rk/lcp 从零编号，lcp[k] 比较 sa[k-1] 与 sa[k]，首项为零。LCP 需遇分隔符停止时，令 remain[i] 为 i 到下一分隔符前的普通字符数：

$$height[k] = \min\{lcp[k], remain[sa[k-1]], remain[sa[k]]\}.$$

两串拼接求最长公共子串只需检查异来源的相邻后缀。

SAM 状态 u 代表长度 $(len[link[u]], len[u]]$ 内同一 endpos 的子串。occ 种子真实状态为一、克隆为零，按 len 倒序传播（不能按创建编号）。根计数为 n，非空子串只看非根状态；次数乘长度用 long long。

第十章　后缀自动机 (SAM)

10.1　后缀自动机

功能：逐字符在线构建字符串的后缀自动机，可判定子串，并求不同非空子串数（`distinct`）与每个状态中子串的出现次数（`counts`）。

核心思路：每个状态是一个 endpos 等价类，`len` 为类中最长串长，`link` 指向不同类的最长后缀所在状态。`extend(c)` 新建整串状态 `cur`，从 `last` 沿后缀链接为缺 c 转移的状态补边到 `cur`；遇到 c 转移指向 q 的 p 时，若 q 的长度恰为 p 的长度加一就以 q 为后缀链接，否则克隆一个这样长度的 q 并把链上指向 q 的转移改指克隆。`counts` 以 `occ`（真实状态 1、克隆 0）为初值，按 `len` 从大到小沿 `link` 累加；不同子串数为各状态与其 `link` 的 `len` 之差的和。

接口契约：extend(c) 接收整数 0..25，每次追加一个小写字符；last 为整串状态，a[0] 为根，根 link=-1。a[u].len 为状态最大长度，非根代表 (len[link[u]],len[u]] 且具有同一 endpos 集合。occ 是原始种子：真实扩展为 1、克隆为 0，不能直接当出现次数。counts() 按长度倒序汇总后返回副本，不改结构，重复查询及查询后继续追加安全；根计数为 n，非空子串只看 u>0。distinct() 为不同非空子串数。重赋值 SuffixAutomaton() 清空；固定 26 字母表构建 O(n)，counts/distinct 为 O(n)，空间 O(26n)。如需前缀位置对应状态，在每次 extend 后保存 last；本结构不输出广义 SAM 或最小 DFA 状态数。

```
struct SuffixAutomaton
{
    struct Node
    {
        array<int, 26> go{};
        int link = -1, len = 0;
        long long occ = 0;
    };

    vector<Node> a{Node{}};
    int last = 0;
```

接上页同一结构体：

```
    void extend(int c)
    {
        assert(0 <= c && c < 26);
        int cur = (int)a.size();
        a.push_back(Node{});
        a[cur].len = a[last].len + 1;
        a[cur].occ = 1;
        int p = last;
        while (p != -1 && !a[p].go[c])
        {
            a[p].go[c] = cur;
            p = a[p].link;
        }
        if (p == -1)
            a[cur].link = 0;
        else
        {
            int q = a[p].go[c];
            if (a[p].len + 1 == a[q].len)
                a[cur].link = q;
            else
            {
                int clone = (int)a.size();
                Node copy = a[q];
                a.push_back(copy);
                a[clone].len = a[p].len + 1;
                a[clone].occ = 0;
                while (p != -1 && a[p].go[c] == q)
                {
                    a[p].go[c] = clone;
                    p = a[p].link;
                }
                a[q].link = a[cur].link = clone;
            }
        }
        last = cur;
    }
```

接上页同一结构体：

```
    vector<long long> counts() const
    {
        vector<int> bucket(a[last].len + 1), order(a.size());
        vector<long long> ans(a.size());
        for (auto &v : a) ++bucket[v.len];
        partial_sum(bucket.begin(), bucket.end(), bucket.begin());
        for (int i = 0; i < (int)a.size(); i++)
        {
            order[--bucket[a[i].len]] = i;
            ans[i] = a[i].occ;
        }
        for (int i = (int)order.size() - 1; i > 0; i--)
            ans[a[order[i]].link] += ans[order[i]];
        return ans;
    }

    long long distinct() const
    {
        long long ans = 0;
        for (int i = 1; i < (int)a.size(); i++) ans += a[i].len - a[a[i].link].len;
        return ans;
    }
};
```

模板题使用：Library Checker number_of_substrings

题意：给小写字母串，求不同非空子串的个数。extend 接收 0..25 的整数，故字符减去 a；所有字符追加后调用 distinct，结果使用 long long。

所需模板：SuffixAutomaton（第 56 页，10.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/number_of_substrings

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    SuffixAutomaton sam;
    for (char c : s) sam.extend(c - 'a');
    cout << sam.distinct() << '\n';
}
```

10.2 SAM 字典序子串查询

功能：在已建好的（广义）后缀自动机上查询字典序第 k 小的子串：默认每个不同子串计一次；传入各状态出现次数作权重时，相同子串按出现次数重复计入；还可指定字母顺序。

核心思路：从根出发的每条路径对应一个子串，到达状态 u 的每个串计 `weight[u]` 份（根为 0）。`sum[u]` 为 u 自身权重加上从 u 出发所有路径的权重，按 `len` 桶排序后逆序计算（转移总使 `len` 增大，这就是拓扑逆序），加法在 `LLONG_MAX` 处饱和。`kth` 从根开始：若 k 不超过当前串自身的权重就停止；否则减去它，按字母顺序枚举出边，k 大于 `sum[v]` 就减去并跳过，否则沿该边走一步并记下字符。

接口契约： SAMLex(sam,w={}) 引用已建好的 SuffixAutomaton 或已 build 的 GeneralSAM；索引使用期间 sam 必须存活且保持不变。默认每个非根状态权重 1，得到不同子串；传单串 sam.counts() 按出现次数计数。自定义 w 长度为状态数、非负 long long，每条到达该状态的串有 w[u] 份，根权重强制 0。sum[u] 为本状态及后续路径权重和，超过 LLONG_MAX 截断，仍正确支持所有 1≤k≤LLONG_MAX。kth(k,alphabet) 返回 optional<string>，不足或 k≤0 为 nullopt；默认 a..z，alphabet 必须是 26 字母的排列。短串先于其扩展，支持重复查询与自定义字母序。构建 O(26V+L)，空间 O(V+L)，L 为最长串长；单次查询 O(26× 答案长度)，额外空间为答案。weight/sum 只读，不支持扩展 SAM 后沿用旧索引。

```
// Keep sam alive and unchanged while this index is used.
template <class S>
struct SAMLex
{
    const S &sam;
    vector<long long> weight, sum;

    SAMLex(const S &sam, vector<long long> w = {}) : sam(sam), weight(move(w))
    {
        int n = sam.a.size(), length = 0;
        if (weight.empty()) weight.assign(n, 1);
        assert((int)weight.size() == n);
        weight[0] = 0;
        sum = weight;
        for (auto &v : sam.a) length = max(length, v.len);
        vector<int> bucket(length + 1), order(n);
        for (auto &v : sam.a) bucket[v.len]++;
        for (int i = 1; i <= length; i++) bucket[i] += bucket[i - 1];
        for (int i = 0; i < n; i++) order[--bucket[sam.a[i].len]] = i;
        for (int i = n - 1; i >= 0; i--)
        {
            int u = order[i];
            for (int v : sam.a[u].go)
                if (v) sum[u] += min(sum[v], LLONG_MAX - sum[u]);
        }
    }
```

字典序查询（接上页同一结构体）：

```
    // alphabet must be a permutation of a..z; k is 1-based.
    optional<string> kth(long long k, const string &alphabet = "abcdefghijklmnopqrstuvwxyz") const
    {
        if (k <= 0 || k > sum[0]) return nullopt;
        int u = 0;
        string ans;
        while (k > weight[u])
        {
            k -= weight[u];
            for (char c : alphabet)
            {
                int v = sam.a[u].go[c - 'a'];
                if (!v) continue;
                if (k > sum[v]) k -= sum[v];
                else
                {
                    ans += c;
                    u = v;
                    break;
                }
            }
        }
        return ans;
    }
};
```

10.3 SAM 最长公共子串

问题描述：给定一个串（或一组串）的后缀自动机和文本 t，求 t 与自动机中任一串的最长公共子串，返回它在 t 中的起点和长度。

核心思路：让 t 在自动机上匹配，维护状态 `p` 与长度 `length`，使 t 中以位置 i 结尾、长 `length` 的串是能被识别的最长者。读入字符 c 时，若 `p` 没有 c 转移就沿 `link` 回退并把长度截为回退后状态的 `len`，直到有转移或回到根；有转移则走过去、长度加一，否则长度归零。过程中长度的最大值即答案，只在严格更大时更新，所以取最早起点。

接口契约：sam_lcs(sam,t) 接收已有单串 SuffixAutomaton 或已 build 的 GeneralSAM 及小写文本 t，返回 t 中的 {0-based 起点, 长度}；多解取 t 中最早起点，无公共非空串返回 {0,0}。广义 SAM 时求 t 与任一已加入文档的最长公共子串，不是所有文档公共串。t 可空，长度 <INT_MAX；构建 SAM 的成本另计，查询 O(|t|)、额外空间 O(1)，输入与 SAM 不变。沿后缀链接回退并同步匹配长度，不返回源文档坐标；需实际串时用 t.substr(start,length)。

```cpp
// Return {start, length} in t; ties choose the earliest start.
template <class S>
pair<int, int> sam_lcs(const S &sam, const string &t)
{
    int p = 0, length = 0;
    pair<int, int> ans{0, 0};
    for (int i = 0; i < (int)t.size(); i++)
    {
        int c = t[i] - 'a';
        assert(0 <= c && c < 26);
        while (p && !sam.a[p].go[c])
        {
            p = sam.a[p].link;
            length = sam.a[p].len;
        }
        if (sam.a[p].go[c])
        {
            p = sam.a[p].go[c];
            length++;
        }
        else length = 0;
        if (length > ans.second) ans = {i - length + 1, length};
    }
    return ans;
}
```

第十一章　广义后缀自动机

11.1　广义后缀自动机：Trie BFS 构建

功能：把多个小写串（或调用方逐边给出的 Trie）建成广义后缀自动机，识别所有输入串的全部子串；`distinct()` 给出所有串中不同非空子串的总数。

核心思路：先用 `add` 建 Trie，Trie 节点直接作为 SAM 状态。`build` 按 BFS 顺序处理每条 Trie 边：以父节点为 last、以已存在的子节点 `cur` 为新状态，执行与单串 SAM 相同的扩展——沿后缀链接补上缺失的 c 转移，遇到已有转移的 q 时直接挂链或克隆。`len` 为 0 的非根节点表示尚未处理的 Trie 节点，一个节点处理完后把它这样的子节点入队；克隆时把指向未处理节点的转移清零，避免把尚未处理的 Trie 边当作 SAM 转移。不同子串数为 $\sum_v(\text{len}_v - \text{len}_{\text{link}(v)})$。

接口契约：先 add(s) 登记若干小写字符串，再 build() 原地将 Trie 变为广义 SAM；build 后禁止继续 add，可重复 build，重新收集串需新建对象。空串与重复串允许，distinct() 返回所有输入串中不同非空子串的总数，不包含跨串拼接。根为 0，a[v].go[c] 为转移、0 表示无边，link 为后缀链接、len 为最长代表串长度；从根沿字符转移可判定子串是否存在。a.size() 为按文档 endpos 等价类构造的状态数，不应直接称为后缀并集语言的最小 DFA 点数；ab、b 是两者不同的例子。P6139 官方样例一要求 endpos 点数 10，后缀语言最小化仅 7 点；该题用 a.size()，勿再最小化。固定 26 字符，设累计输入长度 L、Trie 点数 K，登记总时间 O(L)，build 时间与空间 O(26K)，状态少于 2K；累计输入长度及 2K 须可由 int 表示。克隆只复制已处理转移，不能带入尚未 BFS 处理的 Trie 子边。此版不提供出现次数或各串所属标记。新增 add(p,c) 在 build 前按 Trie 边登记，0<=c<26，p 为已有 Trie 节点；返回子节点号，重复边复用。单次均摊 O(1)，避免把输入树展开成多条长串；add(s) 复用此入口。原 Trie 编号在 build 后仍代表原根路径，但禁止继续 add。

```
struct GeneralSAM
{
    struct Node
    {
        array<int, 26> go{};
        int link = -1, len = 0;
    };

    vector<Node> a{Node{}};
    bool ready = false;

    // Trie step before build; repeated edges return the same node.
    int add(int p, int c)
    {
        assert(!ready && 0 <= p && p < (int)a.size());
        assert(0 <= c && c < 26);
        if (!a[p].go[c])
        {
            int v = a.size();
            a.push_back(Node{});
            a[p].go[c] = v;
        }
        return a[p].go[c];
    }

    void add(const string &s)
    {
        assert(!ready);
        int p = 0;
        for (char ch : s) p = add(p, ch - 'a');
    }
```

BFS 构建（接上页同一结构体）：

```
    void build()
    {
        if (ready) return;
        a.reserve(2 * a.size());
        queue<pair<int, int>> todo;
        for (int c = 0; c < 26; c++)
            if (a[0].go[c]) todo.push({0, c});
        while (!todo.empty())
        {
            auto [parent, c] = todo.front();
            todo.pop();
            int cur = a[parent].go[c], p = a[parent].link;
            a[cur].len = a[parent].len + 1;
            while (p != -1 && !a[p].go[c])
            {
                a[p].go[c] = cur;
                p = a[p].link;
            }
            if (p == -1)
                a[cur].link = 0;
            else
            {
                int q = a[p].go[c];
                if (a[p].len + 1 == a[q].len)
                    a[cur].link = q;
                else
                {
                    int clone = a.size();
                    Node copy = a[q];
                    copy.len = a[p].len + 1;
                    for (int &v : copy.go)
                        if (!a[v].len) v = 0;
                    a.push_back(copy);
                    while (p != -1 && a[p].go[c] == q)
                    {
                        a[p].go[c] = clone;
                        p = a[p].link;
                    }
                    a[q].link = a[cur].link = clone;
                }
            }
            for (int ch = 0; ch < 26; ch++)
                if (int v = a[cur].go[ch]; v && !a[v].len) todo.push({cur, ch});
        }
        ready = true;
    }
```

不同子串计数（接上页同一结构体）：

```
    long long distinct() const
    {
        assert(ready);
        long long ans = 0;
        for (int v = 1; v < (int)a.size(); v++) ans += a[v].len - a[a[v].link].len;
        return ans;
    }
};
```

模板题使用：Library Checker number_of_substrings

题意：本题只有一个字符串，输出不同非空子串数。广义模板可在 build 前多次 add；本例验证其单串特例，多串行为另有独立对拍。若只处理单串，原 SuffixAutomaton 更合适。此题不验证广义 SAM 状态数或最小 DFA 状态数。

所需模板：GeneralSAM（第 62 页，11.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/number_of_substrings

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    GeneralSAM sam;
    sam.add(s);
    sam.build();
    cout << sam.distinct() << '\n';
}
```

11.2 SAM 文档频率与阈值后缀

功能：对一组文档建好的广义后缀自动机，求每个状态的代表串出现在多少篇不同文档中（cnt）；对给定 k，求每个状态最长代表串的、至少出现在 k 篇文档中的最长后缀长度。

核心思路：每篇文档从根逐字符走，经过的状态恰为它各前缀所在的状态，把文档号标记在这些状态上。某子串出现在一篇文档中，当且仅当后缀链接树上其状态的子树内有该文档的标记，所以 cnt[u] 是子树内不同文档号的个数。DFS 按访问顺序给标记编时间戳，树状数组中每篇文档只保留最近一次时间戳；离开 u 时，子树内出现过的文档的最近时间戳都落在 u 的时间区间内，区间求和即得。lengths 自根向下传递：cnt 沿树向下不增，答案是最深的满足 cnt $\geq k$ 的祖先（含自身）的 len。

代码依赖：Fenwick（第 284 页，31.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：SAMDocuments(sam,s) 接收已完成构建的 GeneralSAM 及同一批小写文档 s；也兼容按该文档集完成的 OnlineSAM 或单文档 SuffixAutomaton。允许空串和重复串，重复输入按不同文档计数。索引期间 sam 须存活且不变。cnt[u] 是该状态任一非空代表串出现的文档数，cnt[0]=s.size() 约定空串出现在每篇文档；不能把单串出现次数当文档数。lengths(k) 要求 k>=1，返回每个状态最长代表串中、出现于至少 k 篇文档的最长后缀长度，无非空解为 0；k 大于文档数全 0。查询任意非空模式串时，从根转移成功后读取 cnt，缺边返回 0。L 为文档总长，V 为状态数，D 为文档数：构造 O((L+V)log(L+2)+D)，空间 O(L+V+D)，每次 lengths 为 O(V)。递归后缀链接树深度不超过最长文档长度加 1；保留递归，不模拟开栈。L、D、V 须小于 INT_MAX，数组可分配；a/g/cnt 视为只读。

```
template <class S>
struct SAMDocuments
{
    const S &sam;
    vector<vector<int>> g;
    vector<int> cnt;
```

文档标记与子树去重（接上页同一结构体）：

```
    SAMDocuments(const S &sam, const vector<string> &s)
        : sam(sam), g(sam.a.size()), cnt(sam.a.size())
    {
        int n = sam.a.size(), size = 0;
        vector<vector<int>> tag(n);
        for (int u = 1; u < n; u++) g[sam.a[u].link].push_back(u);
        for (int id = 0; id < (int)s.size(); id++)
        {
            int p = 0, len = 0;
            for (char c : s[id])
            {
                assert('a' <= c && c <= 'z');
                p = sam.a[p].go[c - 'a'];
                len++;
                assert(p && sam.a[p].len == len);
                tag[p].push_back(id);
                size++;
            }
        }
        Fenwick<int> bit(size);
        vector<int> last(s.size());
        int timer = 0;
        auto dfs = [&](auto &&self, int u) -> void
        {
            int l = timer + 1;
            for (int id : tag[u])
            {
                if (last[id]) bit.add(last[id], -1);
                last[id] = ++timer;
                bit.add(timer, 1);
            }
            for (int v : g[u]) self(self, v);
            cnt[u] = bit.query(l, timer);
        };
        dfs(dfs, 0);
        cnt[0] = s.size();
    }
```

阈值后缀长度（接上页同一结构体）：

```
    // Longest suffix length occurring in at least k documents, for each state.
    vector<int> lengths(int k) const
    {
        assert(k >= 1);
        vector<int> best(cnt.size());
        auto dfs = [&](auto &&self, int u) -> void
        {
            if (cnt[u] >= k) best[u] = sam.a[u].len;
            for (int v : g[u])
            {
                best[v] = best[u];
                self(self, v);
            }
        };
        dfs(dfs, 0);
        return best;
    }
};
```

11.3 广义 SAM 点数与后缀语言

GeneralSAM 的状态按（文档编号，结束位置）的 endpos 集合等价。先 add 全部小写串，再 build；根为零，distinct 为不同非空子串数，a.size() 含根。构造后不再 add。多串 endpos 自动机不是后缀并集的最小 DFA，点数按 endpos 口径输出 a.size()。

第十二章　Manacher

12.1　Manacher 回文半径

问题描述：对序列求每个位置的最长回文半径：`odd[i]` 为以 i 为中心的最长奇回文的半径（含中心），`even[i]` 为中心在 $i-1$ 与 i 之间的最长偶回文的半长。由此可得最长回文子串与回文子串总数。

核心思路：奇、偶两遍独立扫描，各维护右端最远的已知回文 $[l,r]$。若 $i \le r$，i 关于该回文中心的镜像位置（奇回文为 $l+r-i$，偶回文为 $l+r-i+1$）的半径就是初值，但不超过 $r-i+1$；否则奇回文从 1、偶回文从 0 开始。然后带边界检查地向两侧逐字符扩展，若扩展到 r 之外就更新 $[l,r]$。r 单调右移，总时间线性，不需要插入分隔符或哨兵。

接口契约：manacher(s) 返回 odd/even，下标从 0 开始。odd[i] 包含中心，闭区间 [i-odd[i]+1,i+odd[i]-1]，长度 2*odd[i]-1；even[i] 的中心在 i 前，区间 [i-even[i],i+even[i]-1]，长度 2*even[i]，零半径为空。空串合法，支持任意字节（含零），按字节而非 Unicode 字素比较；int 索引要求 n<2^30。最长回文取上述长度最大值；全部 odd/even 以 long long 求和得到回文出现总数。O(n) 时间和空间，边界显式检查，无需选择哨兵字符。输入为 string、vector<int> 或 vector<long long> 对象，支持空序列和任意整数；泛型 S 需 size()、随机下标及 O(1) 相等比较。字符串字面量请显式写 string("abc")。输入不拷贝，输出移动返回。

```
// odd[i] includes the center; even[i] is centered before i.
template <class S>
pair<vector<int>, vector<int>> manacher(const S &s)
{
    int n = (int)s.size();
    vector<int> odd(n), even(n);
    for (int i = 0, l = 0, r = -1; i < n; i++)
    {
        int k = i > r ? 1 : min(odd[l + r - i], r - i + 1);
        while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) ++k;
        odd[i] = k--;
        if (i + k > r)
        {
            l = i - k;
            r = i + k;
        }
    }
    for (int i = 0, l = 0, r = -1; i < n; i++)
    {
        int k = i > r ? 0 : min(even[l + r - i + 1], r - i + 1);
        while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k]) ++k;
        even[i] = k--;
        if (i + k > r)
        {
            l = i - k - 1;
            r = i + k;
        }
    }
    return {move(odd), move(even)};
}
```

模板题使用：Library Checker enumerate_palindromes

题意：给非空字符串，从左到右输出每个字符及相邻字符间隙处的最长回文长度，共 2n-1 项。odd 含中心，even[i] 的中心在 i 前面，因此分别转成 2*odd[i]-1 和 2*even[i]。

所需模板：manacher（第 69 页，12.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/enumerate_palindromes

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    int n = s.size();
    auto [odd, even] = manacher(s);
    vector<int> answer(2 * n - 1);
    for (int i = 0; i < n; i++)
    {
        answer[2 * i] = 2 * odd[i] - 1;
        if (i) answer[2 * i - 1] = 2 * even[i];
    }
    for (int i = 0; i < (int)answer.size(); i++)
        cout << answer[i] << (i + 1 == (int)answer.size() ? '\n' : ' ');
}
```

12.2 回文半径与在线回文计数

Manacher 的 odd[i] 包含中心，最长奇回文为 $[i - odd[i] + 1, i + odd[i] - 1]$；even[i] 以 i 前间隙为中心，最长偶回文为 $[i - even[i], i + even[i] - 1]$。odd/even 之和是回文出现总次数（long long），不是不同回文串数。按字节比较，空串合法，int 索引要求 $n < 2^{30}$。

PAM 的 suffixes 等于 fail 节点的 suffixes 加一；hits 只计最长后缀命中，完整次数用 occurrences()。total 最多 $n(n+1)/2$，distinct 才是不同非空回文串数。PAM 固定小写字母。

第十三章　回文树

13.1　回文树 PAM：回文后缀与出现次数

功能：逐字符追加构建回文树，每个节点是一个本质不同的回文子串；可得以当前位置结尾的回文串个数、不同回文子串数及每个回文串的出现次数。

核心思路：0 号根长 0、1 号根长 -1，`fail` 指向最长真回文后缀。追加字符 c 时，`get_fail` 从上一位置的最长回文后缀 `last` 沿 `fail` 跳到第一个前一位字符等于 c 的节点 u，两端加 c 即新的最长回文后缀；若该节点不存在就新建，长为 $\text{len}_u + 2$，其 `fail` 从 `fail[u]` 继续跳并取 c 转移（先求 `fail` 再挂边，长 1 的节点才不会指向自己）。`suffixes` 为 fail 链上的回文个数，即以该位置结尾的回文数。`hits` 记每个节点作为最长回文后缀的次数，`occurrences` 按编号倒序沿 `fail` 累加得出现次数。

接口契约：add(ch) 追加小写英文字母，返回最长回文后缀节点，亦保存于 last。节点 0 是长度 0 根，1 是长度 -1 根，实际回文节点从 2 开始；distinct() 为不同非空回文串数。t[u].len 为长度，fail 为最长真回文后缀链接，suffixes 为该节点的非空回文后缀数量；追加后的 t[last].suffixes 就是本位置结尾的回文串数。total 为当前全部回文子串的出现次数之和。end 为首次出现时的 0-based 右端点，可恢复原串子段；hits 仅统计作为最长后缀的次数，不能当出现次数。occurrences() 返回按节点编号的完整出现次数副本，根置零，O(n)；重复调用以及查询后继续追加均安全。重新赋值 PalindromicTree() 清空重建。固定 26 字符集下建树 O(n)，空间 O(26n)。

```
struct PalindromicTree
{
    struct Node
    {
        int next[26]{};
        int len = 0, fail = 0, suffixes = 0, end = -1;
        long long hits = 0;
    };

    vector<Node> t;
    vector<int> s;
    int last = 0;
    long long total = 0;

    PalindromicTree() : t(2), s(1, -1)
    {
        t[0].fail = 1;
        t[1].len = -1;
    }

    int get_fail(int u) const
    {
        int pos = s.size() - 1;
        while (s[pos - t[u].len - 1] != s[pos]) u = t[u].fail;
        return u;
    }
```

接上页同一结构体：

```
    // Append a lowercase letter; return the longest palindromic suffix node.
    int add(char ch)
    {
        int c = ch - 'a';
        assert(0 <= c && c < 26);
        s.push_back(c);
        int u = get_fail(last);
        if (!t[u].next[c])
        {
            int v = t.size();
            t.push_back(Node{});
            t[v].len = t[u].len + 2;
            t[v].fail = t[get_fail(t[u].fail)].next[c];
            t[v].suffixes = t[t[v].fail].suffixes + 1;
            t[v].end = (int)s.size() - 2;
            t[u].next[c] = v;
        }
        last = t[u].next[c];
        t[last].hits++;
        total += t[last].suffixes;
        return last;
    }

    int distinct() const { return (int)t.size() - 2; }

    // Copy-and-propagate: repeated calls and later appends remain valid.
    vector<long long> occurrences() const
    {
        vector<long long> count(t.size());
        for (int u = 2; u < (int)t.size(); u++) count[u] = t[u].hits;
        for (int u = (int)t.size() - 1; u >= 2; u--) count[t[u].fail] += count[u];
        count[0] = 0;
        count[1] = 0;
        return count;
    }
};
```

模板题使用：Luogu P5496

题意：按上一次答案把当前字符循环偏移后追加，输出以当前位置结尾的回文串个数。add 返回最长回文后缀节点，读取其 suffixes。

所需模板：PalindromicTree（第 71 页，13.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5496

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    std::string s;
    std::cin >> s;
    PalindromicTree pam;
    int answer = 0;
    for (int i = 0; i < (int)s.size(); i++)
    {
        char ch = 'a' + (s[i] - 'a' + answer) % 26;
        int u = pam.add(ch);
        answer = pam.t[u].suffixes;
        std::cout << answer << (i + 1 == (int)s.size() ? '\n' : ' ');
    }
}
```

第十四章　最小表示法

14.1　最小循环表示

问题描述：求字符串所有循环位移中字典序最小者的起点（最小表示法）。例如 bca 的最小循环表示为 abc，起点为 2。

核心思路：双指针：i、j 是两个候选起点，k 是二者从头起已相等的长度。第 k 个字符相等则 k 加一；若从 i 起的较大，那么起点 $i, i+1, \ldots, i+k$ 分别不优于 $j, \ldots, j+k$，全部淘汰，i 跳到 $i+k+1$（与 j 重合时再加一），反之同理移动 j，然后 k 清零。有指针越过末尾或 $k = n$ 时停止，较小的指针即为答案。

接口契约：minimum_rotation(s) 返回字典序最小循环表示的起点 (0-based)，空串返回 0；按无符号字节比较，与 string 字典序一致。O(n)。

```
int minimum_rotation(const string &s)
{
    int n = (int)s.size(), i = 0, j = 1, k = 0;
    if (!n) return 0;
    while (i < n && j < n && k < n)
    {
        unsigned char a = s[(i + k) % n], b = s[(j + k) % n];
        if (a == b)
        {
            ++k;
            continue;
        }
        if (a > b)
        {
            i += k + 1;
            if (i == j) ++i;
        }
        else
        {
            j += k + 1;
            if (i == j) ++j;
        }
        k = 0;
    }
    return min(i, j);
}
```

模板题使用：Luogu P13270【模板】最小表示法

题意：给定长度 1..10^7 的小写字母字符串，输出所有循环位移中字典序最小的字符串。minimum_rotation 返回 0-based 起点，使用 rotate 原地重排后输出；O(n) 时间，算法本体 O(1) 额外空间。

所需模板：minimum_rotation（第 74 页，14.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P13270

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    string s;
    if (!(cin >> n >> s)) return 0;
    int p = minimum_rotation(s);
    rotate(s.begin(), s.begin() + p, s.end());
    cout << s << '\n';
}
```

第十五章　Lyndon 分解

15.1　Lyndon 分解：Duval

问题描述：把字符串唯一地分解为字典序非递增的 Lyndon 串之积 $s = w_1 w_2 \cdots w_k$（Lyndon 串：严格小于自己所有非空真后缀的串），返回各因子的右端点。例如 `bab` 分解为 `b`、`ab`，返回 $1, 3$。

核心思路：Duval 算法。`i` 是尚未输出部分的起点，从 `i` 开始已读入的部分始终形如“若干个相同的 Lyndon 串 w，再接 w 的一个真前缀”；`j` 是正在读入的字符，`k` 是它在周期中对应的位置，$|w| = j - k$。若 $s_k = s_j$，周期延续，`k` 加一；若 $s_k < s_j$，已读部分整体成为一个更长的 Lyndon 串，`k` 退回 `i`；若 $s_k > s_j$ 则停止，把完整的若干个 w 依次输出（每个长 $j - k$），剩下的真前缀从新的 `i` 重新扫描。

接口契约：lyndon(s) 返回严格递增的因子右端点 ends；按 0-based 半开区间 [0,ends[0])、[ends[i-1],ends[i]) 取因子，数值恰好也是 1-based 闭区间右端点。各因子严格小于自己的所有非空真后缀，因子序列按字典序非递增，分解唯一。输入按 unsigned char 的 0..255 字节序比较，支持空串、零字节与高位字节；不是 Unicode 字符顺序。空串返回空 vector，s 不变，长度须 ≤INT_MAX。Duval 时间 O(n)，预留 n 个 int 的输出容量，空间 O(n)，除返回数组外只需 O(1) 工作空间；不复制子串。P6114 直接异或 ends，无需再加 1；全部相同字符会拆成 n 个单字符，不能合并相等因子。

```
vector<int> lyndon(const string &s)
{
    assert(s.size() <= INT_MAX);
    int n = s.size();
    vector<int> ends;
    ends.reserve(n);
    int i = 0;
    while (i < n)
    {
        int j = i + 1, k = i;
        while (j < n && (unsigned char)s[k] <= (unsigned char)s[j])
        {
            if (s[k] == s[j])
                k++;
            else
                k = i;
            j++;
        }
        int len = j - k;
        while (i <= k)
        {
            i += len;
            ends.push_back(i);
        }
    }
    return ends;
}
```

模板题使用：P6114 【模板】Lyndon 分解

题意：输入小写字符串，输出非递增 Lyndon 分解的全部 1-based 右端点异或和；n≤5000001。

所需模板：lyndon（第 76 页，15.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6114

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    int ans = 0;
    for (int r : lyndon(s))
        ans ^= r;
    cout << ans << '\n';
}
```

第四部分

数学

第十六章　高精度计算

16.1　高精度整数：有符号加减乘与小整数除法

功能：有符号任意精度整数：可由 `long long` 或十进制字符串构造并转回字符串，支持比较、取负、加、减、乘，除以一个 `int` 得商和余数，以及非负整数次幂。

核心思路：绝对值按 10^9 进制、低位在前存于 `d`，符号单独存于 `sign`；零为空数组且符号为正，运算后用 `trim` 去掉高位零保持规范。同号加法逐节进位；异号加法化为绝对值大者减小者，结果取绝对值较大者的符号；减法即加相反数。乘法是逐节竖式，用 `long long` 携带进位。除以小整数从高节往低节做长除法，商向零截断，余数随被除数的符号。幂用二进制快速幂。

思路中的情况	代码中的处理	行
由字符串构造	从末尾起每 9 位数字一组转成一节	30
转为字符串	最高节原样输出，其余节补足 9 位前导零	51
异号相加	若本数绝对值较小则交换两数，再逐节借位相减	81
同号相加	逐节相加并进位	100
乘法	第 i 节乘以 b 的每一节，累加到第 $i+j$ 节并进位	126
除以小整数	高节到低节长除法，`rem` 为当前余数	145

接口契约：BigInteger(x) 接收 long long，BigInteger(s) 接收可带正负号及前导零的非空十进制串；str() 返回规范化十进制，cmp(b) 返回 $-1/0/1$。支持一元负号、加减乘，divmod(int b) 返回商和余数，b≠0；商向零截断，余数与被除数同号或为 0。pow(unsigned e) 计算非负整数幂，约定 0 的 0 次方为 1。

d 是低位在前的 10^9 进制绝对值，sign 为 ±1；零为空 d 且 sign=1，勿直接修改。n,m 为分节数：加减比较 O(n+m)，乘法 O(nm+n+m)，小整数除法 O(n)，结果空间 O(n+m)；幂执行 O(log(e+1)) 次乘法，位数增长另计。字符串长度、所有中间分节数及两操作数长度之和须小于 INT_MAX 并留进位空间。不含大整数除大整数；不用浮点或 Boost。

```
struct BigInteger
{
    static constexpr int base = 1000000000;
    vector<int> d;
    int sign = 1;
    void trim()
    {
        while (!d.empty() && d.back() == 0)
            d.pop_back();
        if (d.empty())
            sign = 1;
    }
    BigInteger(long long x = 0)
    {
        sign = x < 0 ? -1 : 1;
        auto v = x < 0 ? 0ULL - (unsigned long long)x : (unsigned long long)x;
        while (v)
        {
            d.push_back(v % base);
            v /= base;
        }
        trim();
    }
```

十进制构造、输出与比较（接上页结构体）：

```
    explicit BigInteger(const string &s)
    {
        assert(!s.empty());
        int first = s[0] == '-' || s[0] == '+';
        sign = s[0] == '-' ? -1 : 1;
        assert(first < (int)s.size());
        for (int r = int(s.size()); r > first; r -= 9)
        {
            int v = 0;
            for (int i = max(first, r - 9); i < r; i++)
            {
                assert('0' <= s[i] && s[i] <= '9');
                v = v * 10 + s[i] - '0';
            }
            d.push_back(v);
        }
        trim();
    }
    string str() const
    {
        if (d.empty())
            return "0";
        string s = sign < 0 ? "-" : "";
        s += to_string(d.back());
        for (int i = int(d.size()) - 2; i >= 0; i--)
        {
            string t = to_string(d[i]);
            s += string(9 - t.size(), '0') + t;
        }
        return s;
    }
    int abs_cmp(const BigInteger &b) const
    {
        if (d.size() != b.d.size())
            return d.size() < b.d.size() ? -1 : 1;
        for (int i = int(d.size()) - 1; i >= 0; i--)
            if (d[i] != b.d[i])
                return d[i] < b.d[i] ? -1 : 1;
        return 0;
    }
    int cmp(const BigInteger &b) const
    {
        if (sign != b.sign)
            return sign < b.sign ? -1 : 1;
        return sign * abs_cmp(b);
    }
```

取负与加减法（接上页结构体）：

```
    BigInteger operator-() const
    {
        BigInteger c = *this;
        if (!c.d.empty())
            c.sign = -c.sign;
        return c;
    }
    BigInteger operator+(const BigInteger &b) const
    {
        if (sign != b.sign)
        {
            if (abs_cmp(b) < 0)
                return b + *this;
            BigInteger c = *this;
            int borrow = 0;
            for (int i = 0; i < (int)d.size(); i++)
            {
                int v = d[i] - borrow;
                if (i < (int)b.d.size())
                    v -= b.d[i];
                borrow = v < 0;
                c.d[i] = v + borrow * base;
            }
            c.trim();
            return c;
        }
        BigInteger c;
        c.sign = sign;
        int carry = 0;
        int n = max(d.size(), b.d.size());
        for (int i = 0; i < n || carry; i++)
        {
            int v = carry;
            if (i < (int)d.size())
                v += d[i];
            if (i < (int)b.d.size())
                v += b.d[i];
            c.d.push_back(v % base);
            carry = v / base;
        }
        return c;
    }
    BigInteger operator-(const BigInteger &b) const
    {
        return *this + (-b);
    }
```

乘法、小整数除法与整数幂（接上页结构体）：

```
    BigInteger operator*(const BigInteger &b) const
    {
        BigInteger c;
        c.sign = sign * b.sign;
        c.d.resize(d.size() + b.d.size());
        for (int i = 0; i < (int)d.size(); i++)
        {
            long long carry = 0;
            for (int j = 0; j < (int)b.d.size(); j++)
            {
                long long v = 1LL * d[i] * b.d[j] + c.d[i + j] + carry;
                c.d[i + j] = v % base;
                carry = v / base;
            }
            c.d[i + b.d.size()] = carry;
        }
        c.trim();
        return c;
    }
    pair<BigInteger, int> divmod(int b) const
    {
        assert(b != 0);
        long long v = b < 0 ? -1LL * b : b;
        long long rem = 0;
        BigInteger q;
        q.sign = sign * (b < 0 ? -1 : 1);
        q.d.resize(d.size());
        for (int i = int(d.size()) - 1; i >= 0; i--)
        {
            rem = rem * base + d[i];
            q.d[i] = rem / v;
            rem %= v;
        }
        q.trim();
        return {q, int(sign * rem)};
    }
    BigInteger pow(unsigned int e) const
    {
        BigInteger a = *this;
        BigInteger r = 1;
        while (e)
        {
            if (e & 1)
                r = r * a;
            e >>= 1;
            if (e)
                a = a * a;
        }
        return r;
    }
};
```

模板题使用：P1480 高精度除法 I

题意：0≤a≤10^5000，1≤b≤10^9；输出整数商。divmod 同时返回余数，这里只打印 first。完整协议本地通过，在线状态见验证记录。

所需模板：BigInteger（第 79 页，16.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1480

```
int main()
{
    string s;
    int b;
    cin >> s >> b;
    BigInteger a(s);
    cout << a.divmod(b).first.str() << '\n';
}
```

第十七章　快速幂

17.1　无符号 64 位模乘与模幂

功能： 64 位无符号整数的模乘 `mul` 与快速幂 `power`，供判素、分解等数论模板使用。

核心思路： `mul` 把乘数提升到 `__uint128_t` 再取模，乘积不会溢出且结果精确；`power` 是普通二进制快速幂，从低位到高位扫描指数，底数逐次平方。

接口契约： Mod64::mul(a,b,m) 与 power(a,e,m)，a、b、e 支持完整 unsigned long long，m 为非零无符号 64 位模数。乘积用 uint128 后取模，不通过浮点近似；m=1 返回 0，e=0 返回 1 mod m。mul 为 O(1)，power 为 O(log(e+1)) 次模乘、O(1) 空间。它也是 64 位判素的底层组件。

```
struct Mod64
{
    using ull = unsigned long long;
    using u128 = __uint128_t;

    static ull mul(ull a, ull b, ull m) { return u128(a) * b % m; }

    static ull power(ull a, ull b, ull m)
    {
        assert(m);
        ull r = 1 % m;
        a %= m;
        for (; b; b >>= 1, a = mul(a, a, m))
            if (b & 1) r = mul(r, a, m);
        return r;
    }
};
```

第十八章　数论

18.1　模算术简介

18.1.1　动态模整数与合数模逆元

功能：运行时设定模数的取模整数，模数不要求是素数。支持加、减、乘、快速幂；与模数互素时可求逆、做除法，不可逆时 try_inv 返回空。不同 tag 各自持有独立的模数。

核心思路：成员 v 始终是 $[0,p)$ 内的代表，加减只做一次条件校正。乘法用 Barrett 约减代替 64 位除法：set_mod 预存 im$\approx\lceil 2^{64}/p\rceil$，对乘积 z 用 $\lfloor z\cdot \mathrm{im}/2^{64}\rfloor$ 估计商 q，q 至多偏大 1，因此 $z-qp$ 为负时补一个 p。求逆对 (v,p) 做迭代扩展欧几里得并维护 v 的系数，最后余数为 1 才可逆，系数即逆元。

接口契约：mint<tag> 按 tag 独立保存运行时模数，默认 998244353。先 set_mod(p)，1≤p≤INT_MAX，再构造该模数下的所有对象；更换同 tag 的模数后，旧对象、容器元素和缓存全部失效，必须重建，不能跨模混算。构造支持 long long 全范围，v 为规范余数；加减乘及复合赋值、相等比较为 O(1)，加减只作一次区间校正，乘法使用 Barrett32，不依赖素数。pow(k) 的 k 为 uint64_t 非负指数，0 次幂为 1 mod p。try_inv() 以扩展欧几里得求逆，gcd(v,p)≠1 返回 nullopt；inv() 与除法要求可逆，否则违反断言前提。p=1 时唯一余数为 0，乘法单位元也为 0，try_inv 返回 0。幂和求逆为 O(log p) 或 O(log k)，除法包含求逆。可与 Fenwick 区间和组合，不能用于模数和的 kth。mod、reducer 只读，必须通过 set_mod 同步切换；Barrett32 本身支持完整 uint32，但此包装仍受 int 上界保护；不支持超过 INT_MAX 的模数，也不提供线程间安全切换模数。

```
template <int tag = 0> struct mint
{
    static inline int mod = 998244353;
    static inline unsigned long long im = -1ULL / 998244353 + 1; // Barrett reciprocal
    int v;

    static void set_mod(int p)
    {
        assert(p >= 1);
        mod = p;
        im = -1ULL / p + 1;
    }

    mint(long long x = 0) : v(x % mod)
    {
        if (v < 0)
            v += mod;
    }

    mint operator+(mint b) const
    {
        mint r;
        long long sum = (long long)v + b.v;
        r.v = sum >= mod ? sum - mod : sum;
        return r;
    }
```

模整数（接上页同一结构体）：

```
    mint operator-(mint b) const
    {
        mint r;
        r.v = v - b.v;
        if (r.v < 0)
            r.v += mod;
        return r;
    }

    mint operator*(mint b) const
    {
        mint r;
        unsigned long long z = 1ULL * v * b.v, y = (unsigned long long)((__uint128_t(z) * im) >> 64) * mod;
        r.v = int(z - y + (z < y ? mod : 0));
        return r;
    }

    mint &operator+=(mint b)
    {
        return *this = *this + b;
    }

    mint &operator-=(mint b)
    {
        return *this = *this - b;
    }

    mint &operator*=(mint b)
    {
        return *this = *this * b;
    }

    mint pow(uint64_t k) const
    {
        mint a = *this, r = 1;
        while (k)
        {
            if (k & 1)
                r *= a;
            a *= a;
            k >>= 1;
        }
        return r;
    }
```

模整数（接上页同一结构体）：

```
    optional<mint> try_inv() const
    {
        long long a = v, b = mod, x = 1, y = 0;
        while (b)
        {
            long long q = a / b;
            tie(a, b) = pair{b, a - q * b};
            tie(x, y) = pair{y, x - q * y};
        }
        if (a != 1)
            return nullopt;
        return mint(x);
    }

    mint inv() const
    {
        auto x = try_inv();
        assert(x.has_value());
        return *x;
    }

    mint operator/(mint b) const
    {
        return *this * b.inv();
    }

    mint &operator/=(mint b)
    {
        return *this = *this / b;
    }

    bool operator==(const mint &) const = default;
};
```

模板题使用：Luogu P5431 【模板】模意义下的乘法逆元 2

题意：输入 1≤n≤5000000、2≤k<p≤10^9、1≤a_i<p 且 p 为素数，求 ∑k^i/a_i 模 p。先设置 mint 的模数 p，再构造数组；求前缀积后只对总积求一次逆（try_inv），倒序乘出各项逆元。权重从 1 开始，每项先乘 k 再累加。read 与 main 一起抄。更换同 tag 模数后旧对象失效，计算期间不能切换。核心支持合数单位，本题只覆盖素数；动态模数、构造、乘法及累加、v 输出在此直接使用，合数求逆接口另由核心测试核验。

所需模板：mint（第 86 页，18.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5431

```
int read()
{
    int c = getchar_unlocked();
    while (c < '0' || c > '9')
    {
        if (c == EOF) return 0;
        c = getchar_unlocked();
    }
    int value = 0;
    while ('0' <= c && c <= '9')
    {
        value = value * 10 + c - '0';
        c = getchar_unlocked();
    }
    return value;
}

int main()
{
    int n = read(), p = read(), k = read();
    mint<>::set_mod(p);
    vector<mint<>> a(n);
    for (auto &x : a) x = read();
    // Batch inverse: one inversion of the prefix product.
    vector<mint<>> prefix(n + 1, 1);
    for (int i = 0; i < n; i++) prefix[i + 1] = prefix[i] * a[i];
    auto total = prefix[n].try_inv();
    if (!total) return 1;
    mint<> suffix = *total, answer = 0;
    vector<mint<>> inverse(n);
    for (int i = n - 1; i >= 0; i--)
    {
        inverse[i] = suffix * prefix[i];
        suffix *= a[i];
    }
    mint<> weight = 1;
    for (auto x : inverse)
    {
        weight *= mint<>(k);
        answer += weight * x;
    }
    printf("%d\n", answer.v);
    return 0;
}
```

18.1.2 静态模整数

功能：编译期固定模数的取模整数（模数可为合数），支持加、减、乘、快速幂；与模数互素时可求逆、做除法，不可逆时 `try_inv` 返回空。

核心思路：成员 `v` 始终保存 $[0, \mathrm{mod})$ 内的规范代表，构造时把负余数调正。加减各做一次条件校正，乘法用 `long long` 乘积取模，快速幂按二进制拆分指数。求逆不依赖费马小定理：对 (v, mod) 做迭代扩展欧几里得并维护 v 的系数 `x`，结束时 gcd 为 1 则该系数就是逆元。

接口契约：固定 int 模数 1≤mod≤INT_MAX，可为合数。构造接受 signed64 并正规化，v 保持 0..mod-1；不要手动破坏正规化。加减乘 O(1)，pow 非负 signed64 指数、时间 O(log(e+1))。try_inv 用欧几里得求逆：gcd(v,mod)=1 时返回 optional 值，否则 nullopt；mod=1 返回代表 0。inv、/、/= 要求分母可逆；断言开启时诊断违反前提，关闭断言不承诺拒绝非法除法。逆元 O(log(mod+1))、额外空间 O(1)，只用 long long，不新增辅助模板依赖。+=、-=、*=、/= 返回自身引用，支持自赋值与链式运算，可作为 Fenwick 区间和值类型。类型的扩展不放宽 Binomial、NTT、高斯或 FPS 自身的素数等前提。

```
template <int mod> struct ModInt
{
    static_assert(mod >= 1);
    int v;

    ModInt(long long x = 0)
    {
        v = x % mod;
        if (v < 0)
            v += mod;
    }

    ModInt operator+(ModInt b) const
    {
        long long x = (long long)v + b.v;
        if (x >= mod)
            x -= mod;
        ModInt r;
        r.v = x;
        return r;
    }

    ModInt operator-(ModInt b) const
    {
        int x = v - b.v;
        if (x < 0)
            x += mod;
        ModInt r;
        r.v = x;
        return r;
    }

    ModInt operator*(ModInt b) const
    {
        ModInt r;
        r.v = 1LL * v * b.v % mod;
        return r;
    }
```

模整数（接上页同一结构体）：

```
    ModInt pow(long long e) const
    {
        assert(e >= 0);
        ModInt a = *this, r = 1;
        while (e)
        {
            if (e & 1)
                r = r * a;
            a = a * a;
            e >>= 1;
        }
        return r;
    }

    optional<ModInt> try_inv() const
    {
        long long a = v, b = mod, x = 1, y = 0;
        while (b)
        {
            long long q = a / b;
            a -= q * b;
            swap(a, b);
            x -= q * y;
            swap(x, y);
        }
        if (a != 1)
            return nullopt;
        return ModInt(x);
    }

    ModInt inv() const
    {
        auto r = try_inv();
        assert(r);
        return *r;
    } // requires an invertible residue
```

模整数（接上页同一结构体）：

```
77      ModInt operator/(ModInt b) const
78      {
79          return *this * b.inv();
80      }
81
82      ModInt &operator+=(ModInt b)
83      {
84          return *this = *this + b;
85      }
86
87      ModInt &operator-=(ModInt b)
88      {
89          return *this = *this - b;
90      }
91
92      ModInt &operator*=(ModInt b)
93      {
94          return *this = *this * b;
95      }
96
97      ModInt &operator/=(ModInt b)
98      {
99          return *this = *this / b;
100     }
101 };
```

18.1.3 模运算的适用条件

模逆元存在当且仅当 $\gcd(a,m)=1$；费马逆元 a^{p-2} 仅适用于素数模数且 $a\not\equiv 0$。$x\equiv a\pmod m$、$x\equiv b\pmod n$ 可合并当且仅当 $\gcd(m,n)\mid(b-a)$，新模数 $\operatorname{lcm}(m,n)$。Lucas 仅限素数模；素数模阶乘表上限须小于模数。

18.1.3.1 除法与解同余式不是一回事

$g=\gcd(a,m)$：$ax+b\equiv 0\pmod m$ 在 $g\nmid b$ 时无解，否则

$$\frac{a}{g}x+\frac{b}{g}\equiv 0\pmod{m/g}$$

有唯一解 x_0，原模下全部解为 $x_0+t(m/g)$，$0\le t<g$。约去 g 时模数也要除以 g。`linear_congruence(a,b,m)` 解 $ax+b\equiv 0$，返回（最小非负解，最小正周期），无解 $(-1,-1)$；$ax\equiv c$ 须先把 c 规范到 $[0,m)$ 再取负传入。见第 108 页。`mod_inverse` 不可逆返回 -1，`ModInt::try_inv` 返回 `nullopt`；`inv` 与除法以可逆为前提。见第 104 页、第 90 页。$m=1$ 时代表与逆元均为 0、周期 1；域上算法（高斯、NTT、FPS、组合数）不推广到模 1。

18.1.3.2 知道整数可整除时，先升模数再整除

已知 $d\mid X$，求 $(X/d)\bmod m$（$\gcd(d,m)$ 任意）：取规范剩余 $r=X\bmod(md)\in[0,md)$，则

$$\frac{X}{d}\equiv\frac{r}{d}\pmod m.$$

须能计算 md 及其下的乘模（必要时多精度）；新模数可能是合数，不能套素数模模板；生成 X 的过程含除法时需重验。应用见第 20.11.7 节（第 214 页）。

18.1.3.3 组合数的整数性不等于分母可逆

`Binomial` 要求素数 p 且 $N<p$；合数模下 $N<m$ 不够。大下标素数模用 Lucas；合数模用素数幂分解、单位部分与 CRT（exLucas）。见第 189 页。插值节点差、级数积分分母、模概率分母都须各自检查可逆。

18.1.3.4 大指数要同时保留余数和阈值信息

$a\ge 0$，$b\ge 0$，$m\ge 1$，$\phi=\varphi(m)$：

$$a^b\equiv a^e\pmod m,\qquad e=\begin{cases}b, & b<\phi,\\ b\bmod\phi+\phi, & b\ge\phi.\end{cases}$$

零次幂为 1，$m=1$ 返回 0；$\gcd(a,m)=1$ 时可直接对 ϕ 取模。十进制大指数逐位维护 $b\bmod\phi$ 与 $\min(b,\phi)$。`euler_power(a,b,m,phi)` 要求传入真实 $\varphi(m)$，基数与模数为 unsigned64（负数先规范化），降幂指数可达 65 位，内部用 128 位。见第 103 页。

18.2 素数

18.2.1 确定性 64 位 Miller–Rabin 判素

问题描述：判断一个 64 位无符号整数 n 是否为素数，结果是确定的而不是概率性的。

核心思路：先用 $2 \sim 37$ 的小素数试除，整除时只有 n 恰为该素数才是素数。否则写 $n-1=d\cdot 2^s$（d 为奇数），对 7 个固定底数做 Miller–Rabin：$x=a^d$ 为 1 或 $n-1$ 则该底数通过；否则再平方至多 $s-1$ 次，途中出现 $n-1$ 才通过，否则 n 是合数。这组底数在 64 位范围内没有反例；底数是 n 的倍数时跳过。

代码依赖：Mod64（第 85 页，17.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：Prime64::prime(n) 接收完整 unsigned long long，n<2 返回 false。先试除小素数，再使用 2、325、9375、28178、450775、9780504、1795265022 七个底数，结果在 64 位输入范围内确定。继承 Mod64，模乘用 uint128；O(log n) 次模乘，O(1) 空间，不包含因数分解。

```
struct Prime64 : Mod64
{
    static bool prime(ull n)
    {
        if (n < 2) return false;
        for (ull p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37})
            if (n % p == 0) return n == p;
        ull d = n - 1;
        int s = 0;
        while (!(d & 1))
        {
            d >>= 1;
            ++s;
        }
        for (ull a :
             {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL, 1795265022ULL})
        {
            if (a % n == 0) continue;
            ull x = power(a, d, n);
            if (x == 1 || x == n - 1) continue;
            bool ok = false;
            for (int j = 1; j < s; j++)
            {
                x = mul(x, x, n);
                if (x == n - 1)
                {
                    ok = true;
                    break;
                }
            }
            if (!ok) return false;
        }
        return true;
    }
};
```

模板题使用：Library Checker primality_test

题意：逐个判断给定正整数是否为素数，输出 Yes 或 No。此题输入不超过 10^{18}；模板支持的完整范围另见接口约定。

所需模板：Mod64（第 85 页，17.1 节）、Prime64（第 94 页，18.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/primality_test

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    while (q--)
    {
        unsigned long long n;
        cin >> n;
        cout << (Prime64::prime(n) ? "Yes" : "No") << '\n';
    }
}
```

18.2.2 64 位判素与质因数分解

Miller–Rabin 用七个固定底数，覆盖无符号 64 位整数：

$$2,\ 325,\ 9375,\ 28178,\ 450775,\ 9780504,\ 1795265022.$$

$n-1=d2^s$，检查 a^d 及平方链；$n<2$ 非素数；底数模 n 为 0 时跳过。模乘用无符号 128 位。

Pollard–Rho 迭代 $f(x)=x^2+c \pmod n$，取 $\gcd(|x-y|,n)$；只接受 $1<d<n$，否则换参数重试，无确定最坏上界。`factor` 返回排序且保留重复的质因子，`factor(1)` 为空，取首尾前先判空。

18.3 欧拉函数

18.3.1 单个数的欧拉函数

问题描述：求单个正整数 n 的欧拉函数 $\varphi(n)$。提供两个入口：试除法，以及借助 PollardRho 分解后计算（适合含大质因子的 n）。

核心思路：公式 $\varphi(n)=n\prod_{p\mid n}(1-1/p)$。试除版从 $p=2$ 枚举到 $\sqrt{n}$，遇到质因子就把答案改为 answer/p·($p-1$) 并把 n 中的 p 除尽，最后剩下的 $n>1$ 是唯一的大质因子。分解版对升序质因子列表去重后做同样的乘法。先除后乘，中间值不超过原数。

代码依赖：PollardRho（第 99 页，18.5.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：euler_phi(n) 使用试除，O($\sqrt{n}$) 时间、O(1) 额外空间；euler_phi(n,solver) 复用 PollardRho，时间由随机分解主导，分解结果与递归占 O(log n) 空间。1≤n≤ULLONG_MAX，phi(1)=1。大素数或平衡大半素数应选分解器入口，不要用试除穷扫平方根；批量求 1..N 用 LinearSieve。先除质因子再乘 p-1，结果及中间乘积均不超过原 n。只抄试除函数时不需要 Prime64/PollardRho。

```
unsigned long long euler_phi(unsigned long long n)
{
    assert(n >= 1);
    unsigned long long answer = n;
    for (unsigned long long p = 2; p <= n / p; p++)
        if (n % p == 0)
        {
            answer = answer / p * (p - 1);
            while (n % p == 0) n /= p;
        }
    if (n > 1) answer = answer / n * (n - 1);
    return answer;
}

unsigned long long euler_phi(unsigned long long n, PollardRho &solver)
{
    assert(n >= 1);
    auto factors = solver.factor(n);
    unsigned long long answer = n;
    unsigned long long previous = 0;
    for (auto p : factors)
        if (p != previous)
        {
            answer = answer / p * (p - 1);
            previous = p;
        }
    return answer;
}
```

联合使用示例见第 104 页（Luogu P5091），其中直接调用本模板。

18.3.2 单个数的欧拉函数

实现见第 96 页。对 $n\geq 1$，

$$\varphi(n)=n\prod_{p\mid n}\left(1-\frac{1}{p}\right),$$

按 ans ← (ans/p)($p-1$) 先除后乘，不会溢出。试除入口用 $p\leq n/p$，$O(\sqrt{n})$，不适合 64 位大素数；传入 PollardRho 对象时先分解再去重更新。$\varphi(1)=1$，0 不在定义域内。

18.4 筛法

18.4.1 区间筛素数

问题描述：给定区间 $[l,r]$，列出其中全部素数；r 可以很大，只要区间长度可承受。

核心思路：区间内的合数必有不超过 $\sqrt{r}$ 的质因子。先用埃氏筛（`composite`）求出 $[2,\lfloor\sqrt{r}\rfloor]$ 的素数，同时对每个素数 p，从 $\max(p^2,\lceil l/p\rceil p)$ 开始，在区间数组 `removed` 上按步长 p 标记倍数（下标偏移 $j-l$）。最后从 $\max(l,2)$ 起收集未被标记的数。

接口契约：segmented_primes(l,r) 返回闭区间内递增的全部素数，0≤l≤r≤INT_MAX；区间长度须能分配内存。独立函数，每次重建底筛；不依赖预存素数表。令 W=r-l+1，时间 O((√r+W)log log(r+3))，空间 O(√r+W)。倍数起点、递增变量均用 long long，避免 INT_MAX 端点溢出；0 和 1 不进入结果。

```
vector<int> segmented_primes(int l, int r)
{
    assert(0 <= l && l <= r);
    int limit = 0;
    while (1LL * (limit + 1) * (limit + 1) <= r) limit++;
    vector<char> composite(limit + 1);
    vector<char> removed(1LL * r - l + 1);
    for (int p = 2; p <= limit; p++)
        if (!composite[p])
        {
            for (long long j = 1LL * p * p; j <= limit; j += p) composite[j] = true;
            long long start = max(1LL * p * p, (l + 1LL * p - 1) / p * p);
            for (long long j = start; j <= r; j += p) removed[j - l] = true;
        }
    vector<int> primes;
    for (long long x = max(l, 2); x <= r; x++)
        if (!removed[x - l]) primes.push_back(x);
    return primes;
}
```

18.4.2 线性筛、欧拉函数与 Möbius 函数

功能：一次预处理 $0..n$，得到递增素数表 `prime`、最小质因子 `lp`、欧拉函数 `phi` 和 Möbius 函数 `mu`；反复除以 `lp` 可快速分解表内的数。

核心思路：欧拉筛：每个合数 $j = i \cdot p$ 只被它的最小质因子 p 筛一次。外层枚举 i，内层从小到大枚举已有素数 p 并令 `lp[j]`$= p$；一旦 $p \mid i$，更大的素数就不再是 $i \cdot p$ 的最小质因子，立即 `break`。积性函数同步转移：$p \mid i$ 时 $\varphi(ip) = \varphi(i)\,p$、$\mu(ip) = 0$；否则 $\varphi(ip) = \varphi(i)(p-1)$、$\mu(ip) = -\mu(i)$。

接口契约：构造参数 0≤n<INT_MAX，且数组须能分配内存。prime 为递增质数表；lp/phi/mu 为 0..n 数组。x≥2 时 lp[x] 是最小质因子，lp[x]==x 等价于 x 为素数；反复除以 lp[x] 可分解质因数。lp[0]=lp[1]=0，不能直接用 lp[x]==x 判断 0。n≥1 时 phi[1]=mu[1]=1；第 0 项均为占位 0。O(n) 时间和空间，分解单个表内正整数耗时 O(log x)。n=0 不可访问下标 1。

```
struct LinearSieve
{
    vector<int> prime, lp, phi, mu;

    LinearSieve(int n) : lp(n + 1), phi(n + 1), mu(n + 1)
    {
        if (n) phi[1] = mu[1] = 1;
        for (int i = 2; i <= n; i++)
        {
            if (!lp[i])
            {
                lp[i] = i;
                prime.push_back(i);
                phi[i] = i - 1;
                mu[i] = -1;
            }
            for (int p : prime)
            {
                if (p > n / i) break;
                int j = i * p;
                lp[j] = p;
                if (i % p == 0)
                {
                    phi[j] = phi[i] * p;
                    mu[j] = 0;
                    break;
                }
                phi[j] = phi[i] * (p - 1);
                mu[j] = -mu[i];
            }
        }
    }
};
```

18.4.3 线性筛：三张表的约定

`LinearSieve` 构造最小质因子、φ、μ，每个合数只由 lp 生成一次，$O(n)$。若 $p \mid i$，$\varphi(ip) = p\varphi(i)$、$\mu(ip) = 0$；否则为 $(p-1)\varphi(i)$ 与 $-\mu(i)$。判素用 $x \geq 2$ 且 $\mathrm{lp}(x) = x$；反复除 $\mathrm{lp}(x)$ 分解，0 不可分解。要求 $0 \leq n <$ `INT_MAX`，乘积前检查 $p \leq n/i$。相关恒等式：$\sum_{d|x} \varphi(d) = x$，$\sum_{d|x} \mu(d) = [x = 1]$，$\varphi(n) = \sum_{d|n} \mu(d)(n/d)$。

18.4.4 区间筛：只保存目标区间

实现见第 97 页。设 $0 \le L \le R \le$ INT_MAX，筛出不超过 $\lfloor\sqrt{R}\rfloor$ 的素数，对每个 p 从

$$\max\bigl(p^2, \lceil L/p\rceil p\bigr)$$

开始标记区间倍数，输出跳过 0 和 1。令 $W = R - L + 1$，空间 $O(\sqrt{R} + W)$，时间 $O((\sqrt{R} + W)\log\log(R + 3))$；每次调用重建底筛。$p^2$、取整加法与遍历变量用 long long，否则 $R =$ INT_MAX 时溢出。

18.5 分解质因数

18.5.1 Pollard–Rho 因数分解

问题描述：把 64 位正整数 n 分解质因数，返回升序且含重数的质因子列表。

核心思路：split 递归：n 为素数（Prime64）就记为一个因子，否则用 rho 找一个非平凡因子 d，分别递归 d 与 n/d，最后排序。rho 对偶数直接返回 2；否则随机取 c 和起点，迭代 $f(v) = v^2 + c \bmod n$，用 Floyd 判圈（x 走一步、y 走两步），每步计算 $\gcd(|x - y|, n)$。得到 $1 < d < n$ 即返回；得到 n 或轮数用尽，就换一组随机参数重试。

代码依赖：Mod64（第 85 页，17.1 节）、Prime64（第 94 页，18.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：factor(n) 返回升序质因子（含重数），1≤n≤ULLONG_MAX。n=1 返回空数组；n≥2 时 size()==1 等价于 n 为素数，否则 front()/back() 分别取最小/最大质因子。依赖 Mod64 与 Prime64。公开使用 factor；rho/split 为内部辅助，rho 只接受大于 1 的合数，不能直接传素数。Las Vegas 随机运行时间，固定默认种子方便复现，可由构造参数更换；无确定性最坏时间保证。每次 rho 尝试最多 200000 轮，失败后换随机参数重试，不将失败当作素数。递归仅沿真因子拆分，深度 O(log n)。

```
struct PollardRho
{
    using ull = unsigned long long;
    using u128 = __uint128_t;
    mt19937_64 rng;

    PollardRho(ull seed = 712367821) : rng(seed) {}

    ull rho(ull n)
    {
        if (n % 2 == 0) return 2;
        for (;;)
        {
            ull c = rng() % (n - 1) + 1, x = rng() % n, y = x, d = 1;
            auto f = [&](ull v)
            {
                return (Mod64::mul(v, v, n) + u128(c)) % n;
            };
            // Retry bounded attempts; Las Vegas: only exact divisors returned.
            for (int i = 0; i < 200000 && d == 1; i++)
            {
                x = f(x);
                y = f(f(y));
                d = gcd(x > y ? x - y : y - x, n);
            }
            if (1 < d && d < n) return d;
        }
    }
```

```
29
30      void split(ull n, vector<ull> &a)
31      {
32          if (n == 1) return;
33          if (Prime64::prime(n))
34          {
35              a.push_back(n);
36              return;
37          }
38          ull d = rho(n);
39          split(d, a);
40          split(n / d, a);
41      }
42
43      vector<ull> factor(ull n)
44      {
45          assert(n >= 1);
46          vector<ull> a;
47          split(n, a);
48          sort(a.begin(), a.end());
49          return a;
50      }
51  };
```

模板题使用：Luogu P4718

题意：逐个判断 n 是否为素数，是则输出 Prime，否则输出最大质因子。题目保证 n≥2，factor 返回升序含重数的质因子，因此可取 back；若用于 n=1，须先处理空结果。

所需模板：Mod64（第 85 页，17.1 节）、Prime64（第 94 页，18.2.1 节）、PollardRho（第 99 页，18.5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4718

```
1   int main()
2   {
3       int t;
4       scanf("%d", &t);
5       PollardRho solver;
6       while (t--)
7       {
8           unsigned long long n;
9           scanf("%llu", &n);
10          auto factors = solver.factor(n);
11          if (factors.size() == 1)
12              puts("Prime");
13          else
14              printf("%llu\n", factors.back());
15      }
16      return 0;
17  }
```

18.6 裴蜀定理 & 一次不定方程

18.6.1 扩展欧几里得与 Bézout 系数

问题描述：给定非负整数 a, b，求 $g = \gcd(a, b)$ 以及一组整数 x, y 使 $ax + by = g$（Bézout 系数），据此可解一般的一次不定方程。

核心思路：按欧几里得算法递归：$b = 0$ 时取 $(x, y) = (1, 0)$。否则对 $(b, a \bmod b)$ 递归，并把 `y`、`x` 交换位置传入，得到 $bx' + (a \bmod b)y' = g$，此时 `x` 中已是 y'、`y` 中是 x'。由 $a \bmod b = a - \lfloor a/b \rfloor b$ 得 $x = y'$、$y = x' - \lfloor a/b \rfloor y'$，只需再执行 `y -= a / b * x`。

接口契约：extended_gcd(a,b,x,y) 约定 0≤a,b≤2^63，参数和输出系数为 int128，返回非负 gcd 并满足 ax+by=gcd(a,b)。a=b=0 返回 0、x=1、y=0；递归实现，中间乘加为 int128；O(log(min(a,b)+1)) 时间与递归空间。解 ax+by=c：令 g=gcd(a,b)，c 不是 g 的倍数时无解；否则 x0=x · c/g、y0=y · c/g，通解 x=x0+k · b/g、y=y0-k · a/g；最小非负 x 为 x0 对 b/g 取非负余数。乘 c/g 时注意 int128 范围。

```
__int128_t extended_gcd(__int128_t a, __int128_t b, __int128_t &x, __int128_t &y)
{
    using i128 = __int128_t;
    if (!b)
    {
        x = 1;
        y = 0;
        return a;
    }
    i128 d = extended_gcd(b, a % b, y, x);
    y -= a / b * x;
    return d;
}
```

模板题使用：AOJ NTL_1_E Extended Euclid Algorithm

题意：给出正整数 1≤a,b≤10^9，求 ax+by=gcd(a,b)，先最小化 |x|+|y|，仅在并列最小时取 x≤y。直接调用 extended_gcd；该正整数范围内递归所得解已满足最小性，a=b 时返回 (0,1)。系数用 ___int128_t 接收，输出转 long long 安全；不应对所有答案强行要求 x≤y。时间、递归空间均 O(log(min(a,b)+1))。

所需模板：extended_gcd（第 101 页，18.6.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=NTL_1_E

```
int main()
{
    long long a, b;
    scanf("%lld%lld", &a, &b);
    __int128_t x, y;
    extended_gcd(a, b, x, y);
    printf("%lld %lld\n", (long long)x, (long long)y);
    return 0;
}
```

18.6.2 一次不定方程：特解与正整数解范围

底层接口见第 18.6.1 节（第 101 页）。a,b 不全为零时令 $g=\gcd(|a|,|b|)>0$；当且仅当 $g\mid c$ 有整数解。由 $|a|u+|b|v=g$ 乘 c/g 并恢复符号得特解 $ax_0+by_0=c$，全部解为

$$x=x_0+\frac{b}{g}t,\qquad y=y_0-\frac{a}{g}t,\qquad t\in\mathbb{Z}.$$

$a=b=0$ 时 $c=0$ 全体整数对为解，否则无解。`exgcd` 约定非负系数、上限 2^{63}，返回非负 g；取负、缩放后的特解都用 128 位保存。

正系数 a,b,c 求正整数解：$d_x=b/g, d_y=a/g$，把 x_0 调到 $x_{\min}\in[1,d_x]$，$y_{\max}=(c-ax_{\min})/b$。若 $y_{\max}>0$，

$$N=\left\lfloor\frac{y_{\max}-1}{d_y}\right\rfloor+1,\quad x_{\max}=x_{\min}+(N-1)d_x,\quad y_{\min}=y_{\max}-(N-1)d_y.$$

否则无正解，x,y 的最小正值分别按周期归一化（通常不是同一组解）；正值归一化用 `(v % d + d - 1) % d + 1`。

18.6.2.1 有符号矩形范围：把两维约束化成一个参数区间

求 $L_x\le x\le R_x$、$L_y\le y\le R_y$ 内的解数（a,b,c 可负可零）。空区间答案为 0；$a=b=0$ 时 $c\neq0$ 无解，$c=0$ 答案 $(R_x-L_x+1)(R_y-L_y+1)$。否则取特解与带符号步长 $d_x=b/g$、$d_y=-a/g$，对 $z=z_0+dt$：

$d>0$	$\lceil(L-z_0)/d\rceil\le t\le\lfloor(R-z_0)/d\rfloor$
$d<0$, $k=-d>0$	$\lceil(z_0-R)/k\rceil\le t\le\lfloor(z_0-L)/k\rfloor$
$d=0$	$L\le z_0\le R$ 时不限制t，否则无解

两区间取交 $[T_l,T_r]$，解数 $\max(0,T_r-T_l+1)$。floor/ceil 为数学取整：正分母 D、C++ 商余 $q=n/D, r=n\%D$ 时

$$\lfloor n/D\rfloor=q-[r<0],\qquad\lceil n/D\rceil=q+[r>0].$$

差值、区间长度可能超出 64 位（全零系数覆盖全部 64 位坐标时答案为 2^{128}）；不能证明范围时用任意精度整数。求特解后只需常数次运算。

18.7 费马小定理 & 欧拉定理

18.7.1 十进制大指数：扩展欧拉降幂

问题描述：给定底数 a、模数 m 及 $\varphi(m)$，以及以十进制字符串给出的超大指数 b，求 $a^b \bmod m$；a 与 m 不必互素。

核心思路：扩展欧拉定理：$b < \varphi(m)$ 时指数保持 b；$b \geq \varphi(m)$ 时 $a^b \equiv a^{(b \bmod \varphi(m))+\varphi(m)} \pmod m$。逐位扫描字符串，同时维护 `rem`$= b \bmod \varphi$ 和饱和值 `cap`$= \min(b, \varphi)$；扫描结束时 `cap` 等于 φ 就说明 $b \geq \varphi$，指数加上 φ。降幂后的指数小于 2φ，再用 `Mod64::mul` 做快速幂。

代码依赖：Mod64（第 85 页，17.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：euler_power(a,b,m,phi) 求 a^b mod m，a、m 为 unsigned long long，m>0；b 为非空十进制数字串，可为 0、可含前导零，约定 0^0=1。调用者必须传入正确的 phi=φ(m)，包括 $\varphi(1)=1$；函数只检查数值范围，不验证 phi 正确性。指数小于 phi 时保持原值，否则使用 b mod phi + phi，非互质底数也适用。相加和十进制更新都用 int128，支持完整 unsigned64 模数。O(|b|+log m) 时间、O(1) 额外空间，不含求 phi 的成本；依赖 Mod64::mul。

```
unsigned long long euler_power(unsigned long long a,
                               string_view b,
                               unsigned long long m,
                               unsigned long long phi)
{
    using U = unsigned long long;
    using I = __uint128_t;
    assert(m && phi && phi <= m && !b.empty());
    U rem = 0, cap = 0;
    for (char c : b)
    {
        assert('0' <= c && c <= '9');
        int d = c - '0';
        rem = (I(rem) * 10 + d) % phi;
        cap = min(I(phi), I(cap) * 10 + d);
    }
    I e = I(rem) + (cap == phi ? I(phi) : 0);
    U ans = 1 % m;
    a %= m;
    for (; e; e >>= 1, a = Mod64::mul(a, a, m))
        if (e & 1) ans = Mod64::mul(ans, a, m);
    return ans;
}
```

模板题使用：Luogu P5091

题意：输入顺序为底数 a、模数 m、指数 b；b 最多 20000001 位，以字符串读入。先用试除法计算 $\varphi(m)$，再做扩展欧拉降幂；题目 m≤10^8，不能把输入顺序写成 a,b,m。

所需模板：Mod64（第 85 页，17.1 节）、Prime64（第 94 页，18.2.1 节）、PollardRho（第 99 页，18.5.1 节）、euler_phi（第 96 页，18.3.1 节）、euler_power（第 103 页，18.7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5091

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    unsigned long long a, m;
    string b;
    cin >> a >> m >> b;
    auto phi = euler_phi(m);
    cout << euler_power(a, b, m, phi) << '\n';
}
```

18.7.2 Carmichael：最小公共指数

$\lambda(n)$ 为所有可逆元阶的最小公倍数，$\lambda(1)=1$，$\lambda(n)\mid\varphi(n)$。

$$\lambda(n)=\operatorname{lcm}_i \lambda(p_i^{e_i}),\qquad \lambda(p^e)=\begin{cases}2^{e-2}, & p=2,\ e\ge 3,\\ p^{e-1}(p-1), & \text{其他情形.}\end{cases}$$

仅可逆底数可把指数对 $\lambda(n)$ 取模。合数 n 是 Carmichael 数当且仅当 $\lambda(n)\mid(n-1)$，等价于无平方因子且每个素因子 p 满足 $(p-1)\mid(n-1)$（须排除素数）；Fermat 检验不能代替素性判定。

18.8 模逆元

18.8.1 任意正模数的单个逆元

问题描述：给定整数 a 和正整数 m（可为合数），求 a 模 m 的乘法逆元；$\gcd(a,m)\neq 1$ 时逆元不存在。

核心思路：逆元 x 满足 $ax+my=1$。先把 a 规范到 $[0,m)$，调用 `extended_gcd` 求 Bézout 系数；gcd 不为 1 则不可逆，否则把系数 x 调整到 $[0,m)$ 返回。不使用费马小定理，因此不要求 m 为素数。

代码依赖：extended_gcd（第 101 页，18.6.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：mod_inverse(a,m) 支持完整 long long a 和正 long long m，将 a 规范化后用 extended_gcd 求逆。不可逆返回-1，可逆返回 0..m-1 内的代表；模数 1 返回 0。允许合数模数，不使用费马小定理。O(log m) 时间与递归空间。

```
long long mod_inverse(long long a, long long m)
{
    using i128 = __int128_t;
    assert(m > 0);
    a %= m;
    if (a < 0) a += m;
    i128 x, y;
    if (extended_gcd(a, m, x, y) != 1) return -1;
    return (x % m + m) % m;
}
```

18.8.2 任意数组的批量逆元

问题描述：给定 n 个整数和模数（可为合数），一次求出它们各自的模逆元；只要有一个不可逆就整体判定失败。

核心思路：前缀积技巧：`prefix[i]` 为前 i 个数之积。只对总积 `prefix[n]` 调用一次 `mod_inverse`（总积不可逆当且仅当某一项不可逆）。再从后往前扫：当前 `inverse` 是前 $i+1$ 项积的逆，乘 `prefix[i]` 得到 a_i^{-1}，乘 a_i 得到前 i 项积的逆。结果原地写回数组。

代码依赖：mod_inverse（第 104 页，18.8.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：batch_inverse(a,mod) 返回与 a 同序的逆元向量。要求 mod 为正 long long，每个输入可为完整 long long；只要某一项与 mod 不互素，整个结果返回 nullopt，不输出部分逆元。空输入返回有效的空向量；模数 1 约定返回全零，与单个逆元接口一致。模数可以是合数。函数按值接收数组并归一化；不再需要原数组时传 move(a) 可避免输入副本。依赖 mod_inverse 和 extended_gcd。仅求一次逆元，时间 O(n+log mod)，空间 O(n)；128 位中间乘法防止溢出。

```
optional<vector<long long>> batch_inverse(vector<long long> a, long long mod)
{
    assert(mod > 0);
    int n = a.size();
    vector<long long> prefix(n + 1, 1 % mod);
    for (int i = 0; i < n; i++)
    {
        a[i] %= mod;
        if (a[i] < 0) a[i] += mod;
        prefix[i + 1] = __int128_t(prefix[i]) * a[i] % mod;
    }
    long long inverse = mod_inverse(prefix[n], mod);
    if (inverse == -1) return nullopt;
    for (int i = n - 1; i >= 0; i--)
    {
        long long value = a[i];
        a[i] = __int128_t(inverse) * prefix[i] % mod;
        inverse = __int128_t(inverse) * value % mod;
    }
    return a;
}
```

模板题使用：Luogu P5431 【模板】模意义下的乘法逆元 2

题意：给定 1≤n≤5000000、2≤k<p≤10^9、1≤a_i<p，p 为素数，输出 ∑(i=1..n)k^i/a_i 模 p。先批量求全部逆元再累乘权重，注意首项权重是 k。完整抄写 read 与 main；read 仅适用于本题的非负十进制输入。move(a) 避免输入副本，此后不再读取 a。本题所有元素可逆；一般模数或零元素须检查 nullopt。乘积在 long long 范围内，批量核心内部用 int128 保护一般模数。

所需模板：extended_gcd（第 101 页，18.6.1 节）、mod_inverse（第 104 页，18.8.1 节）、batch_inverse（第 105 页，18.8.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5431

```
int read()
{
    int c = getchar_unlocked();
    while (c < '0' || c > '9')
    {
        if (c == EOF) return 0;
        c = getchar_unlocked();
    }
    int value = 0;
    while ('0' <= c && c <= '9')
    {
        value = value * 10 + c - '0';
        c = getchar_unlocked();
    }
    return value;
}

int main()
{
    int n = read(), p = read(), k = read();
    vector<long long> a(n);
    for (auto &x : a) x = read();
    auto inverse = batch_inverse(move(a), p);
    if (!inverse) return 1;
    long long answer = 0, weight = 1;
    for (auto x : *inverse)
    {
        weight = weight * k % p;
        answer = (answer + weight * x) % p;
    }
    printf("%lld\n", answer);
    return 0;
}
```

18.8.3 连续整数的线性逆元表

问题描述：给定素数 p 和 $n < p$，线性时间求出 $1, 2, \ldots, n$ 模 p 的逆元表。

核心思路：设 $p = qi + r$，$q = \lfloor p/i \rfloor$，$r = p \bmod i < i$。由 $qi + r \equiv 0 \pmod p$，两边乘 $i^{-1}r^{-1}$ 得 $i^{-1} \equiv -q \cdot r^{-1} \pmod p$。因为 $r < i$，r^{-1} 已经算出，按 i 递增一次递推即可；代码写成 $p - (q \cdot r^{-1} \bmod p)$，使结果落在 $[1, p)$。**接口契约：**inverse_table(n,p) 返回长度 n+1 的 int 向量，位置 i 保存 i 模 p 的逆元，1≤i≤n。位置 0 固定为 0，仅为占位，不表示零有逆元；n=0 返回单个占位项。要求 p 为素数、2≤p≤INT_MAX 且 0≤n<p，不检测素性。时间和空间均 O(n)，中间乘法用 long long；独立可抄，不依赖其他模板。不能将同一递推直接用于合数模数，即使当前 i 与模数互素；一般模数使用扩展欧几里得或任意数组批量逆元。

```
vector<int> inverse_table(int n, int p)
{
    assert(p >= 2 && 0 <= n && n < p);
    vector<int> inverse(n + 1);
    if (n) inverse[1] = 1;
    for (int i = 2; i <= n; i++) inverse[i] = p - 1LL * (p / i) * inverse[p % i] % p;
    return inverse;
}
```

模板题使用：Luogu P3811 【模板】模意义下的乘法逆元

题意：输入 1≤n≤3000000、n<p<20000528，p 保证为素数，输出 1..n 各自的逆元，每项一行。inverse_table 返回 n+1 项，0 是占位且不输出。该递推要求素数，不能照搬到合数；一般数组使用 batch_inverse。

所需模板：inverse_table（第 107 页，18.8.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3811

```
int main()
{
    int n, p;
    scanf("%d%d", &n, &p);
    auto inverse = inverse_table(n, p);
    for (int i = 1; i <= n; i++) printf("%d\n", inverse[i]);
    return 0;
}
```

18.8.4 任意数组的批量逆元

设 $s_0 = 1$，$s_i = \prod_{j<i} a_j \pmod m$，只对 s_n 求一次逆元，再从右向左：若 $t = s_{i+1}^{-1}$，则

$$a_i^{-1} = t s_i \pmod m, \qquad s_i^{-1} = t a_i \pmod m.$$

先保存原 a_i 再覆盖。总乘积与 m 互素当且仅当每项互素，模数无需为素数。实现见第 105 页：无解返回空的可选值，空输入返回空向量，模数 1 返回全零。

18.8.5 连续整数的线性逆元表

素数 p、$1 < i < p$ 时

$$i^{-1} \equiv -\lfloor p/i \rfloor \, (p \bmod i)^{-1} \pmod p,$$

从 $1^{-1} = 1$ 递推，$O(N)$。实现见第 107 页，第 0 项占位，允许 $N = 0$；要求 $N < p$。合数模数不能照搬（$p \bmod i$ 可能不可逆），单个逆元用扩展欧几里得（$\gcd(a, m) = 1$ 才有解）或素数下的 a^{p-2}。

18.9 线性同余方程

18.9.1 线性同余方程

问题描述：求解线性同余方程 $ax + b \equiv 0 \pmod m$：给出最小非负解与解的周期，或判断无解。

核心思路：方程等价于 $ax \equiv -b \pmod m$。规范化 a 后用 `extended_gcd` 求 $g = \gcd(a, m)$ 及满足 $ax_0 \equiv g \pmod m$ 的 x_0。$g \nmid b$ 时无解；否则 $x \equiv x_0 \cdot (-b/g) \pmod{m/g}$，周期为 m/g，取模后调成非负。

代码依赖：extended_gcd（第 101 页，18.6.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：linear_congruence(a,b,m) 求 a*x+b≡0 (mod m)，返回 {最小非负解, 最小正周期}，无解返回 {-1,-1}。m 为正 long long，a、b 支持完整 long long。若 g=gcd(a,m) 整除 b，周期为 m/g；包括 a=0 和 m=1，所有整数均满足时返回 {0,1}。先归一化 a，并在 int128 中对 b 取相反数和乘法。依赖 extended_gcd；递归时间、空间 O(log(m+1))。注意本接口的常数项是 +b，若题目写 a*x≡c，应传 -c；c 为 LLONG_MIN 时须避免在 64 位中取负。

```
pair<long long, long long> linear_congruence(long long a, long long b, long long m)
{
    assert(m > 0);
    using i128 = __int128_t;
    a %= m;
    if (a < 0) a += m;
    i128 x, y;
    i128 g = extended_gcd(a, m, x, y);
    if (i128(b) % g) return {-1, -1};
    i128 period = m / g;
    x = x * (-i128(b) / g) % period;
    if (x < 0) x += period;
    return {(long long)x, (long long)period};
}
```

18.9.2 线性同余：最小解与最小周期

实现见第 108 页，解 $ax + b \equiv 0 \pmod m$。a 归一化后令 $g = \gcd(a, m)$；有解当且仅当 $g \mid b$，若 $au + mv = g$，则 $x_0 = u(-b/g)$，全部解为 $x_0 + (m/g)t$，返回 $[0, m/g)$ 内最小解与最小正周期 m/g。$a = 0$ 且 $m \mid b$ 或 $m = 1$ 时返回 $(0, 1)$。b 可为 `LLONG_MIN`（内部用 `-i128(b)`）；题目为 $ax \equiv c$ 时先把 c 归一化再取负传入。exgcd 见第 102 页。追及问题 $(m - n)t + (x - y) \equiv 0 \pmod L$ 直接传速度差与起点差，不取绝对值。

18.10 中国剩余定理

18.10.1 广义 CRT：逐项合并同余

问题描述：求同时满足若干同余式 $x \equiv r_i \pmod{m_i}$ 的最小非负整数，模数不必两两互素，或判断无解。函数每次把一个新同余式合并进当前解 $x \equiv r \pmod{m}$。

核心思路：合并 $x \equiv r \pmod{m}$ 与 $x \equiv b \pmod{n}$：设 $x = r + mk$，需 $mk \equiv b - r \pmod{n}$。令 $g = \gcd(m, n)$，由 `extended_gcd` 得 $mx_0 + ny_0 = g$；$g \nmid (b - r)$ 时无解，否则 $k \equiv \frac{b-r}{g} x_0 \pmod{n/g}$，取最小非负值。新模数为 $\mathrm{lcm} = m \cdot n/g$，新余数 $r + mk$ 对它取模，原地写回 `r`、`m`。

代码依赖：extended_gcd（第 101 页，18.6.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：crt_merge(r,m,b,n) 合并 x≡r (mod m) 与 x≡b (mod n)，m,n>0，r、m 须为不同变量。从 r=0、m=1 开始逐项合并；成功后 r 为最小非负解、m 为最小正周期。余数支持完整 long long。无解返回 false；有解但新周期超过 LLONG_MAX 抛 overflow_error。两种失败都会先规范化 r，m 不变，失败后不能视为完整解。依赖 extended_gcd，中间量用 int128；O(log min(m,n)) 时间与递归空间。

```
bool crt_merge(long long &r, long long &m, long long b, long long n)
{
    using ll = long long;
    using i128 = __int128_t;
    assert(m > 0 && n > 0);
    r %= m;
    if (r < 0) r += m;
    b %= n;
    if (b < 0) b += n;
    i128 x, y;
    ll g = (ll)extended_gcd(m, n, x, y);
    i128 diff = i128(b) - r;
    if (diff % g) return false;
    i128 q = n / g, k = (diff / g * x % q + q) % q, mod = i128(m) * q;
    if (mod > LLONG_MAX) throw overflow_error("CRT modulus");
    r = (r + i128(m) * k) % mod;
    m = (ll)mod;
    return true;
}
```

模板题使用：Luogu P4777

题意：求同时满足多个同余式的最小非负整数，模数可以不互质。每行按模数、余数输入，逐式合并；合并后模数须在 long long 范围内。

所需模板：extended_gcd（第 101 页，18.6.1 节）、crt_merge（第 109 页，18.10.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4777

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int count;
    cin >> count;
    long long r = 0, modulus = 1;
    while (count--)
    {
        long long m, b;
        cin >> m >> b;
        if (!crt_merge(r, modulus, b, m))
        {
            cout << -1 << '\n';
            return 0;
        }
    }
    cout << r << '\n';
    return 0;
}
```

18.10.2 Garner：大周期 CRT 与混合进制

问题描述：给定两两互素的模数 $m_0, \ldots, m_{k-1}$ 与余数 b_i，求满足全部同余式的最小非负整数 x（可能远超 64 位）对任意目标模数的余数，或求出 x 的混合进制各位。常用于三模数 NTT 的结果合并。

核心思路：把 x 写成混合进制 $x = d_0 + d_1 m_0 + d_2 m_0 m_1 + \cdots$，$0 \le d_i < m_i$，按顺序求各位。`value[j]` 维护已确定各位之和模 m_j，`coefficient[j]` 维护 $m_0 \cdots m_{j-1} \bmod m_j$。第 i 位须满足 $\text{value}_i + \text{coef}_i \cdot d_i \equiv b_i \pmod{m_i}$，故 $d_i = (b_i - \text{value}_i) \cdot \text{coef}_i^{-1} \bmod m_i$，系数不可逆说明模数不互素；随后更新后面各 j 的两个数组。`garner` 再从高位到低位用秦九韶法在目标模数下求出 x。

代码依赖：mod_inverse（第 104 页，18.8.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：garner_digits(b,moduli) 返回按输入顺序排列的混合进制各位；garner(b,moduli,target) 返回最小非负 CRT 解对 target 的余数。两个输入向量等长，长度可为 0；模数为正且两两互素，允许模数 1，余数支持完整 long long 范围。target 为正 long long，与输入模数无需互素。输入不互素时抛 invalid_argument，即使方程有解也不适用；此时用扩展 CRT。空输入的规范解取 0。算法从不构造模数乘积，中间运算使用 int128。k 个模数耗时 $O(k^2+k \log M)$，空间 O(k)；M 为最大输入模数。依赖 mod_inverse 和 extended_gcd。

```
// Pairwise coprime positive moduli. Digits use the input order.
vector<long long> garner_digits(const vector<long long> &b,
                                const vector<long long> &moduli)
{
    using ll = long long;
    using i128 = __int128_t;
    int n = b.size();
    assert(moduli.size() == b.size());
    vector<ll> value(n), coefficient(n, 1), digit(n);
    for (ll m : moduli) assert(m > 0);
    for (int i = 0; i < n; i++)
    {
        ll m = moduli[i];
        ll inverse = mod_inverse(coefficient[i], m);
        if (inverse == -1) throw invalid_argument("Garner moduli are not coprime");
        i128 delta = (i128(b[i]) - value[i]) % m;
        if (delta < 0) delta += m;
        digit[i] = delta * inverse % m;
        for (int j = i + 1; j < n; j++)
        {
            value[j] = (value[j] + i128(coefficient[j]) * digit[i]) % moduli[j];
            coefficient[j] = i128(coefficient[j]) * m % moduli[j];
        }
    }
    return digit;
}
```

规范解对目标数取模

```
// Return the least nonnegative CRT solution modulo target, not all solutions.
long long
garner(const vector<long long> &b, const vector<long long> &moduli, long long target)
{
    assert(target > 0);
    auto digit = garner_digits(b, moduli);
    long long answer = 0;
    for (int i = (int)digit.size() - 1; i >= 0; i--)
        answer = (__int128_t(answer) * moduli[i] + digit[i]) % target;
    return answer;
}
```

模板题使用：Library Checker Convolution (Mod 1,000,000,007)

题意：给出长度 n、m 的升幂系数，输出模 1000000007 的卷积，共 n+m−1 项。两数组长度均为 1 到 524288；输入系数非负且小于目标模数。分别在 167772161、469762049、1224736769 下做 NTT，再逐项调用既有 garner。真实非负系数不超过 524288 乘以 1000000006 的平方，严格小于三素数乘积。因此 CRT 最小非负代表就是原整数系数，最后对目标模数取余；仅有模数互素并不足够。输入通过 ModInt 构造器归一化。三模数公共变换容量为 2^24，本题至多需 2^20；无系数界时不能直接套用。

所需模板：extended_gcd（第 101 页，18.6.1 节）、mod_inverse（第 104 页，18.8.1 节）、garner（第 111 页，18.10.2 节）、ModInt（第 90 页，18.1.2 节）、NttConvolution（第 151 页，19.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/convolution_mod_1000000007

```
template <int p>
auto conv(const vector<int> &a, const vector<int> &b)
{
    using N = NttConvolution<p>;
    typename N::Poly x(a.begin(), a.end()), y(b.begin(), b.end());
    return N::multiply(move(x), move(y));
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<int> a(n), b(m);
    for (auto &x : a) cin >> x;
    for (auto &x : b) cin >> x;
    auto x = conv<167772161>(a, b);
    auto y = conv<469762049>(a, b);
    auto z = conv<1224736769>(a, b);
    vector<long long> r(3), p = {167772161, 469762049, 1224736769};
    for (int i = 0; i < n + m - 1; i++)
    {
        r[0] = x[i].v;
        r[1] = y[i].v;
        r[2] = z[i].v;
        cout << garner(r, p, 1000000007) << (i + 1 == n + m - 1 ? '\n' : ' ');
    }
```

```
}
```

18.10.3 扩展 CRT：逐式合并

实现见第 109 页的 `crt_merge`：从 $r=0, m=1$ 起依次合并 $x \equiv b \pmod{n}$，一旦无解即停止。设 $g=\gcd(m,n)$，$mu+nv=g$；若 $g \nmid (b-r)$ 无解，否则

$$q=n/g, \qquad t \equiv \frac{b-r}{g}u \pmod{q}, \qquad 0 \le t < q,$$

新状态为 $((r+mt) \bmod mq,\ mq)$。模数不必互素。要求正模数且最终周期不超过有符号 64 位；中间量用 128 位。无解返回假，周期越界抛异常。时间 $O(k \log M)$，递归栈 $O(\log M)$。

18.10.4 Garner：总周期可以超过整数范围

模数 m_i 两两互素，$P=\prod m_i$，把解写成混合进制

$$x = d_0 + d_1 m_0 + d_2 m_0 m_1 + \cdots, \qquad 0 \le d_i < m_i.$$

对每个后续模数维护已定部分余数 v_j 与位权 $c_j = \prod_{t<i} m_t \bmod m_j$，$d_i = (b_i - v_i)c_i^{-1} \bmod m_i$，然后 $v_j \leftarrow v_j + c_j d_i$，$c_j \leftarrow c_j m_i$。所得 $x \in [0,P)$ 为最小非负解；模数 1 可直接参与。实现见第 111 页。求 $x \bmod T$ 时从高位 Horner，T 无需与 m_i 互素；结果只是规范解的余数，多模 NTT 还原整数时须保证真实值在 $[0,P)$ 内。

18.11 升幂引理

18.11.1 素因子估值与升幂：先约去公共因子

p 为素数，a,b,n 为正整数，$v_p(x)$ 为非零整数 x 中 p 的重数。$v_p(0)$ **无定义**：差式先判 $a=b$。

18.11.1.1 基本运算与公共因子

$v_p(xy)=v_p(x)+v_p(y)$，$v_p(x^n)=nv_p(x)$；$v_p(x+y)\ge\min(v_p(x),v_p(y))$，估值不同才取等。记 $\alpha=v_p(a)$、$\beta=v_p(b)$：若 $\alpha\ne\beta$，$v_p(a^n\pm b^n)=n\min(\alpha,\beta)$；若 $\alpha=\beta=t$，写 $a=p^tA,b=p^tB$，则 $v_p(a^n\pm b^n)=nt+v_p(A^n\pm B^n)$，再对 A,B 用 LTE。

18.11.1.2 奇素数：差式与和式的条件不同

p 为奇素数且 $p\nmid ab$：

$$\begin{aligned} p\mid a-b,\ a\ne b &\implies v_p(a^n-b^n)=v_p(a-b)+v_p(n),\\ p\mid a+b,\ n\text{ 为奇数} &\implies v_p(a^n+b^n)=v_p(a+b)+v_p(n). \end{aligned}$$

差式不要求 n 奇，和式必须 n 奇；不要求 $\gcd(a,b)=1$。

18.11.1.3 素数 2：先分指数奇偶

a,b 均为奇数，差式要求 $a\ne b$：

$$v_2(a^n-b^n)=\begin{cases} v_2(a-b), & n\text{ 奇},\\ v_2(a-b)+v_2(a+b)+v_2(n)-1, & n\text{ 偶}, \end{cases} \qquad v_2(a^n+b^n)=\begin{cases} v_2(a+b), & n\text{ 奇},\\ 1, & n\text{ 偶}. \end{cases}$$

若 $4\mid a-b$，差式可写 $v_2(a-b)+v_2(n)$。奇偶不同则和差均奇；都偶先约去公共 2 因子。

18.11.1.4 把大整数整除转成指数条件

$M=\prod p_i^{e_i}\mid X$（$X\ne0$）等价于逐项 $v_{p_i}(X)\ge e_i$，无需构造大幂。LTE 只处理满足条件的局部因子，不能代替乘法阶或离散对数。$a+b$、$|a-b|$、nt 须防溢出。阶乘估值用 $v_p(N!)=\sum_{j\ge1}\lfloor N/p^j\rfloor$（ExLucas，第 117 页），不套 LTE；素数幂开根还须检查单位部分有根，估值条件不充分。

18.12 阶乘取模

18.12.1 Legendre 赋值与 Kummer 进位判据

p 为素数，$s_p(n)$ 为 p 进制数位和，$n \geq 0$：

$$v_p(n!) = \sum_{j \geq 1} \left\lfloor \frac{n}{p^j} \right\rfloor = \frac{n - s_p(n)}{p - 1}.$$

反复 $n \leftarrow \lfloor n/p \rfloor$ 累加，$O(1 + \log_p(n+1))$，不要计算 p^j（会溢出）。`ExLucas::valuation(n,p)` 见第 117 页：n 为 unsigned64，p 为正 int 素数，不验证素性。

18.12.1.1 组合数：逐位相加时进了几次位

$0 \leq k \leq n$，$h = n - k$：

$$v_p\binom{n}{k} = \frac{s_p(k) + s_p(h) - s_p(n)}{p - 1}$$

等于 p 进制下 $k + h$ 的进位次数（含传播进位），即 Kummer 定理。

18.12.1.2 素数幂为零与 Lucas 的边界

$\binom{n}{k} \equiv 0 \pmod{p^a} \iff v_p\binom{n}{k} \geq a$；合数 m 须每个素数幂分量都满足，模 1 恒为 0。零进位 $\iff$ 每位 $k_i \leq n_i$，即 Lucas 非零（第 116 页）。赋值只判整除，不给出非零余数。$k > n$ 先返回 0。LTE 选型见第 18.11.1 节（第 114 页）。

18.12.2 阶乘表：组合数与排列数

素数 p，$0 \leq k \leq n < p$：

$$C(n,k) = n!\,(k!)^{-1}\,((n-k)!)^{-1}, \qquad P(n,k) = n!\,((n-k)!)^{-1} \pmod{p}.$$

扩表到 N：正向补阶乘，只对 $N!$ 求一次逆，再 $(i-1)!^{-1} = i\,(i!)^{-1}$ 反向补。查询前显式扩表，查询 $O(1)$；扩表后旧元素引用失效。见第 189 页。

18.13 卢卡斯定理

18.13.1 Lucas：小素数模的大下标组合数

问题描述：给定较小的素数 p 和很大的 n, k，求 $\binom{n}{k} \bmod p$。

核心思路：Lucas 定理：把 n, k 写成 p 进制，$\binom{n}{k} \equiv \prod_i \binom{n_i}{k_i} \pmod p$，某一位 $k_i > n_i$ 时整体为 0。构造时预处理 $0..p-1$ 的阶乘 `fac`，用费马小定理求 $(p-1)!$ 的逆，再倒推逆阶乘 `ifac`。`choose` 每轮取最低位 $a = n \bmod p$、$b = k \bmod p$，乘上 $\binom{a}{b}$，再把 n, k 除以 p。

接口契约：Lucas(p) 预处理素数模数 p，choose(n,k) 返回 C(n,k) 模 p；n、k 为完整 unsigned long long 范围，k>n 返回 0，C(0,0)=1。重新赋值 Lucas(p) 可更换模数并重新预处理；不检测素性，要求调用者保证 p 为素数且 p≥2。预处理时间 O(p+log p)，动态版空间 O(p)；单次查询 O(1+log_p(n+1))。适合小素数模数，通常 p≤10^6；大素数不能因查询下标小就忽略这里整段 O(p) 的预处理。fac、ifac 下标为 0..p-1，模数和表只读；中间乘法用 long long。合数模数请使用 exLucas。

```
struct Lucas
{
    int mod;
    vector<int> fac, ifac;

    Lucas(int p)
    {
        assert(p >= 2);
        mod = p;
        fac.resize(p);
        ifac.resize(p);
        fac[0] = 1;
        for (int i = 1; i < p; i++) fac[i] = 1LL * fac[i - 1] * i % p;
        long long a = fac[p - 1], r = 1;
        for (int e = p - 2; e; e >>= 1)
        {
            if (e & 1) r = r * a % p;
            a = a * a % p;
        }
        ifac[p - 1] = r;
        for (int i = p - 1; i; i--) ifac[i - 1] = 1LL * ifac[i] * i % p;
    }
```

```
    int choose(unsigned long long n, unsigned long long k) const
    {
        if (k > n) return 0;
        long long answer = 1;
        while (n || k)
        {
            int a = n % mod, b = k % mod;
            if (b > a) return 0;
            answer = answer * fac[a] % mod;
            answer = answer * ifac[b] % mod;
            answer = answer * ifac[a - b] % mod;
            n /= mod;
            k /= mod;
        }
        return answer;
    }
};
```

模板题使用：Luogu P3807

题意：多组查询 C(n+m,n) 模 p，题目保证 p 为素数且不超过 10^5。每组构造 Lucas(p)，choose 的参数为 n+m、n；此题范围内 n+m 不会溢出。

所需模板：Lucas（第 116 页，18.13.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3807

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while (t--)
    {
        unsigned long long n, m;
        int p;
        cin >> n >> m >> p;
        Lucas solver(p);
        cout << solver.choose(n + m, n) << '\n';
    }
    return 0;
}
```

18.13.2 合数模数组合数（扩展 Lucas）

问题描述：给定任意正整数模数 M（可为合数，规模约 10^6）和很大的 n, k，求 $\binom{n}{k} \bmod M$。

核心思路：把 M 分解为互素的素数幂 $q = p^e$，分别求 $\binom{n}{k} \bmod q$ 后用 CRT 合并。模 q 时把 $n!$ 写成 $p^{v(n)}u(n)$，$u(n)$ 与 p 互素：v 用 Legendre 公式；$n!$ 中不被 p 整除的因子模 q 以 q 为周期，故 $u(n) = (\text{整段积})^{\lfloor n/q \rfloor} \cdot (\text{零头积}) \cdot u(\lfloor n/p \rfloor)$。于是 $\binom{n}{k} \equiv p^v \, u(n)\, u(k)^{-1} u(n-k)^{-1} \pmod q$，其中 $v = v(n) - v(k) - v(n-k)$，$v \geq e$ 时该分量为 0。

思路中的情况	代码中的处理	行
分解模数	试除得到每个素数幂 $q = p^e$	41
段内前缀积	`fac` 中该因子的一段：$0..q-1$ 内不被 p 整除的数之积，末尾多存一份整段积	50
CRT 系数	$c_q = (M/q) \cdot ((M/q)^{-1} \bmod q)$，模 q 为 1、模其他素数幂为 0	53
$n!$ 中 p 的指数	Legendre 公式 $\sum_{i\geq 1}\lfloor n/p^i \rfloor$	63
去掉 p 后的 $n!$	整段积的 $\lfloor n/q \rfloor$ 次幂乘零头，再对 $\lfloor n/p \rfloor$ 迭代	73
p 的指数不少于 e	该分量为 0，跳过	87
合并分量	分母单位部分用欧拉定理求逆，乘 p^v 后按 CRT 系数累加	92

接口契约： ExLucas(mod) 预处理固定模数，choose(n,k) 返回 C(n,k) 模 mod；n、k 为 unsigned long long，k>n 返回 0，mod=1 返回 0。适合 mod≤10^6 的合数或素数模数，不能用于 10^9 级素数的整段预处理。分解 mod 为互素素数幂 q=p^e，预处理每段内不含 p 因子的前缀积；unit(n,f) 去除 n! 中全部 p 因子，valuation 统计指数。分子分母指数之差不少于 e 时该项为零，否则只对互素单位部分求逆，最后 CRT 合并。预处理 O(√mod+∑q)，空间 O(∑(q+1))；每次查询 O(∑(log_p(n+1) · log(n+1)+log q))，求和遍历不同素因子。所有乘法用 long long；同一对象可重复查询，factors、fac 与辅助函数视为内部数据。

```
struct ExLucas
{
    using ull = unsigned long long;
    using ll = long long;

    struct Factor
    {
        int p, q, exponent, start, crt;
    };

    int mod;
    vector<int> fac;
    vector<Factor> factors;

    static ll power(ll a, ull b, int m)
    {
        ll result = 1;
        while (b)
        {
            if (b & 1) result = result * a % m;
            a = a * a % m;
            b >>= 1;
        }
        return result;
    }

    ExLucas(int m) : mod(m)
    {
        assert(m >= 1);
        int x = m;
        for (int p = 2; p <= x / p; p++)
        {
            if (x % p) continue;
            int q = 1, e = 0;
            do
            {
                x /= p;
                q *= p;
```

```
            e++;
        } while (x % p == 0);
        add_factor(p, q, e);
    }
    if (x > 1) add_factor(x, x, 1);
}

void add_factor(int p, int q, int e)
{
    int start = fac.size();
    fac.push_back(1);
    for (int i = 1; i < q; i++) fac.push_back(ll(fac.back()) * (i % p ? i : 1) % q);
    fac.push_back(fac.back());
    int phi = q - q / p;
    int crt = ll(mod / q) * power(mod / q % q, phi - 1, q) % mod;
    factors.push_back({p, q, e, start, crt});
}

static ull valuation(ull n, int p)
{
    ull result = 0;
    while (n)
    {
        n /= p;
        result += n;
    }
    return result;
}

ll unit(ull n, const Factor &f) const
{
    ll result = 1;
    while (n)
    {
        result = result * power(fac[f.start + f.q], n / f.q, f.q) % f.q;
        result = result * fac[f.start + n % f.q] % f.q;
        n /= f.p;
    }
    return result;
}
```

```
    int choose(ull n, ull k) const
    {
        if (k > n || mod == 1) return 0;
        ll answer = 0;
        for (const auto &f : factors)
        {
            ull e = valuation(n, f.p) - valuation(k, f.p) - valuation(n - k, f.p);
            if (e >= ull(f.exponent)) continue;
            ll denominator = unit(k, f) * unit(n - k, f) % f.q;
            ll residue =
                unit(n, f) * power(denominator, f.q - f.q / f.p - 1, f.q) % f.q;
            residue = residue * power(f.p, e, f.q) % f.q;
            answer = (answer + residue * f.crt) % mod;
        }
        return answer;
    }
};
```

模板题使用：Luogu P4720

题意：求 C(n,m) 模 p，p 可以为合数且不超过 10^6。构造 ExLucas(p) 后 choose(n,m)；预处理与模数规模有关，不能直接用于 10^9 级模数。

所需模板：ExLucas（第 117 页，18.13.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4720

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    unsigned long long n, m;
    int p;
    std::cin >> n >> m >> p;
    ExLucas comb(p);
    std::cout << comb.choose(n, m) << '\n';
}
```

18.13.3 Lucas：按素数进制逐位计算

p 为素数，n_i, k_i 为 p 进制位：$\binom{n}{k} \equiv \prod_i \binom{n_i}{k_i} \pmod p$，某位 $k_i > n_i$ 则为零。只预处理 $0!, \ldots, (p-1)!$ 及逆元；每位常数次乘法。实现见第 116 页，参数为 n, k；自行求和时检查 64 位溢出。

18.13.4 合数模数组合数与插值

$q = p^e$，$v_p(n!) = \sum_{j \ge 1} \lfloor n/p^j \rfloor$，$U_p(n) = n!/p^{v_p(n!)}$，$t = v_p(n!) - v_p(k!) - v_p((n-k)!)$。$t \ge e$ 时为零，否则

$$\binom{n}{k} \equiv U_p(n)\, U_p(k)^{-1}\, U_p(n-k)^{-1}\, p^t \pmod q,$$

各素数幂结果 CRT 合并，见第 117 页。

素数域中 n 个两两不同的 x_i，$w_i = y_i / \prod_{j \ne i}(x_i - x_j)$：

$$f(k) = \sum_{i=0}^{n-1} w_i \prod_{j<i}(k - x_j) \prod_{j>i}(k - x_j),$$

查询点可等于样本点。$x_i = i$ 时分母为 $i!\,(n-1-i)!\,(-1)^{n-1-i}$，要求 $n \le p$。见第 243 页。$S_d(k) = \sum_{i \le k} i^d$ 在 $0 \le d \le p-2$ 时取 $0, \ldots, d+1$ 共 $d+2$ 个样本。

18.14 二次剩余

18.14.1 素数模平方根（Cipolla）

问题描述：给定素数 p 与整数 n，求方程 $x^2 \equiv n \pmod p$ 的全部解。

核心思路：$n \equiv 0$ 或 $p = 2$ 时唯一解就是 n。否则用 Euler 判别法，$n^{(p-1)/2} \neq 1$ 时无解。$p \equiv 3 \pmod 4$ 时 $n^{(p+1)/4}$ 直接是一个根。其余情况用 Cipolla：随机找 a 使 $w = a^2 - n$ 为非二次剩余，在扩域 $\mathbb{F}_p[\sqrt{w}]$ 中计算 $(a + \sqrt{w})^{(p+1)/2}$，元素用（实部，虚部）表示，乘法中 $(\sqrt{w})^2 = w$，结果的实部就是一个根。另一个根为 $p - x$，排序后一并返回。

思路中的情况	代码中的处理	行
$n \equiv 0$ 或 $p = 2$	只有一个根 n	6
Euler 判别	n 不是二次剩余则返回空	18
$p \equiv 3 \pmod 4$	根为 $n^{(p+1)/4}$	21
找非剩余 w	随机 a 直到 $w^{(p-1)/2} = p - 1$	30
扩域乘法	$(x_1 + y_1\sqrt{w})(x_2 + y_2\sqrt{w})$ 的实部与虚部	32
两个根	x 与 $p - x$ 按大小排序返回	48

接口契约：mod_sqrt(n,p) 返回 vector<long long>，按升序列出全部不同根；无解返回空向量，零或模数 2 返回单根。要求 p 为素数，2≤p≤INT_MAX，n 支持完整 long long 范围并在入口归一化。先用 Euler 判别；p≡3 (mod 4) 时直接快速幂，其余使用 Cipolla 二次扩域。函数内保存固定种子的伪随机生成器；随机性影响寻找非剩余的尝试次数，不影响结果，通常期望 O(log p) 时间、O(1) 辅助空间。乘法用 long long，扩域实部的三因子乘积中间先取模；指数中的 p+1 先转 long long，避免 INT_MAX 溢出。此版本不处理合数模数及 64 位大素数，也不用于浮点开方。

```
vector<long long> mod_sqrt(long long n, int p)
{
    using ll = long long;
    assert(p >= 2);
    n = (n % p + p) % p;
    if (n == 0 || p == 2) return {n};
    auto power = [p](ll a, ll b)
    {
        ll result = 1;
        while (b)
        {
            if (b & 1) result = result * a % p;
            a = a * a % p;
            b >>= 1;
        }
        return result;
    };
    if (power(n, (p - 1) / 2) != 1) return {};
    ll answer;
    if (p % 4 == 3)
        answer = power(n, (ll(p) + 1) / 4);
    else
    {
        static mt19937_64 rng(712367);
        ll a, w;
```

```
        do
        {
            a = rng() % p;
            w = (a * a % p - n + p) % p;
        } while (power(w, (p - 1) / 2) != p - 1);
        using P = pair<ll, ll>;
        auto mul = [p, w](P x, P y)
        {
            ll real = (x.first * y.first + x.second * y.second % p * w) % p;
            ll imag = (x.first * y.second + x.second * y.first) % p;
            return P{real, imag};
        };
        P result{1, 0}, base{a, 1};
        ll exponent = (ll(p) + 1) / 2;
        while (exponent)
        {
            if (exponent & 1) result = mul(result, base);
            base = mul(base, base);
            exponent >>= 1;
        }
        answer = result.first;
    }
    ll other = p - answer;
    if (answer > other) swap(answer, other);
    return {answer, other};
}
```

模板题使用：Luogu P5491

题意：多组求 x*x 与 N 模 p 同余的全部解，题目保证 p 为奇素数。mod_sqrt 返回升序且去重的根，空数组表示无解并输出 Hola!。

所需模板：mod_sqrt（第 121 页，18.14.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5491

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int tests;
    std::cin >> tests;
    while (tests--)
    {
        long long n;
        int p;
        std::cin >> n >> p;
        auto roots = mod_sqrt(n, p);
        if (roots.empty())
            std::cout << "Hola!\n";
        else
        {
            for (int i = 0; i < (int)roots.size(); i++)
                std::cout << roots[i] << (i + 1 == (int)roots.size() ? '\n' : ' ');
        }
    }
}
```

18.14.2 素数模平方根

奇素数 p、$n \not\equiv 0$：有平方根当且仅当 $n^{(p-1)/2} \equiv 1 \pmod p$。$p \equiv 3 \pmod 4$ 时 $r = n^{(p+1)/4}$，另一根 $p - r$。Cipolla：取 a 使 $w = a^2 - n$ 非平方，$\omega^2 = w$，根为 $(a+\omega)^{(p+1)/2}$。实现见第 121 页；零和模 2 单独处理；合数模不适用。

18.15 阶 & 原根

18.15.1 乘法阶与原根

问题描述：给定模数 n，求剩余类 a 的乘法阶，判断 a 是否为原根，求最小原根，或升序列出全部原根。

核心思路：构造时试除分解求 $\varphi(n)$ 及其不同素因子 `factors`，并按“n 为 $2, 4, p^k, 2p^k$（p 为奇素数）”判断原根是否存在。阶：从 $\varphi(n)$ 出发，对每个素因子 p，只要 $a^{\text{result}/p} \equiv 1$ 就把 `result` 除以 p。原根判定：a 与 n 互素，且对每个 p 都有 $a^{\varphi/p} \not\equiv 1$。最小原根从 1 起逐个试；全部原根是 g^k（$\gcd(k, \varphi) = 1$），按 k 累乘枚举后排序。

代码依赖：Mod64（第 85 页，17.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：构造 PrimitiveRoot(n)，要求 2≤n≤INT_MAX；重新赋值 PrimitiveRoot(n) 可重建。order(a) 返回 a 模 n 的乘法阶，a 不可逆时返回 0；is_root(a) 判断该剩余类是否为原根，a 可为完整 long long。minimum() 返回 1..n-1 内最小原根，无原根返回 -1；all() 返回升序的全部原根，无原根返回空向量。mod、phi、exists 只读；factors 为 phi 的不同素因子，辅助 factor 为试除分解。依赖 Mod64::power。初始化 O($\sqrt{n}$)，求阶 O($\log^2 n$)，原根判定 O((1+ω(phi))log n)。若最小原根为 G，顺序枚举最坏检查 G 个候选，不能假定 G 总是常数。枚举全部另需 O(phi log phi+c log c) 时间、O(c) 空间，c 为原根数；大模数只求最小原根或阶时，不应调用 all。

```
struct PrimitiveRoot
```

```
{
    int mod, phi;
    bool exists;
    vector<int> factors;

    static vector<int> factor(int n)
    {
        vector<int> result;
        for (int p = 2; p <= n / p; p++)
            if (n % p == 0)
            {
                result.push_back(p);
                while (n % p == 0) n /= p;
            }
        if (n > 1) result.push_back(n);
        return result;
    }
```

```
    PrimitiveRoot(int n)
    {
        assert(n >= 2);
        mod = n;
        phi = n;
        for (int p : factor(n)) phi -= phi / p;
        int odd = n;
        if (odd % 2 == 0) odd /= 2;
        exists = n == 2 || n == 4 || (odd % 2 && factor(odd).size() == 1);
        factors = factor(phi);
    }

    int order(long long a) const
    {
        a %= mod;
        if (a < 0) a += mod;
        if (gcd(a, (long long)mod) != 1) return 0;
        int result = phi;
        for (int p : factors)
            while (result % p == 0 && Mod64::power(a, result / p, mod) == 1)
                result /= p;
        return result;
    }
```

```
    bool is_root(long long a) const
    {
        if (!exists) return false;
        a %= mod;
        if (a < 0) a += mod;
        if (gcd(a, (long long)mod) != 1) return false;
        for (int p : factors)
            if (Mod64::power(a, phi / p, mod) == 1) return false;
        return true;
    }

    int minimum() const
    {
        if (!exists) return -1;
        for (int g = 1; g < mod; g++)
            if (is_root(g)) return g;
        return -1;
    }
```

```
    vector<int> all() const
    {
        int g = minimum();
        vector<int> roots;
        if (g == -1) return roots;
        long long value = 1;
        for (int k = 1; k <= phi; k++)
        {
            value = value * g % mod;
            if (gcd(k, phi) == 1) roots.push_back(value);
        }
        sort(roots.begin(), roots.end());
        return roots;
    }
};
```

模板题使用：Luogu P6091

题意：多组枚举模 n 的全部原根，先输出个数，再输出升序原根的第 d、2d、3d 等项。all 返回升序数组，下标从 d-1 开始；即使没有取到任何原根，第二行也必须输出空行。

所需模板：Mod64（第 85 页，17.1 节）、PrimitiveRoot（第 123 页，18.15.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6091

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while (t--)
    {
        int n, d;
        cin >> n >> d;
        auto roots = PrimitiveRoot(n).all();
        cout << roots.size() << '\n';
        bool first = true;
        for (int i = d - 1; i < (int)roots.size(); i += d)
        {
            if (!first) cout << ' ';
            cout << roots[i];
            first = false;
        }
        cout << '\n';
    }
    return 0;
}
```

18.15.2 乘法阶与原根

$\gcd(a,n)=1$ 时阶 $d \mid \varphi(n)$：从 $d=\varphi(n)$ 起，对每个素因子 q，$a^{d/q}\equiv 1$ 就除去 q。不可逆 a 接口返回 0。a 为原根当且仅当 $\gcd(a,n)=1$ 且对 $\varphi(n)$ 每个素因子 q 有 $a^{\varphi(n)/q}\not\equiv 1$。$n\ge 2$ 时原根仅存在于 $2,4,p^k,2p^k$（p 奇素数）。$\operatorname{ord}(g^t)=\varphi(n)/\gcd(t,\varphi(n))$，全部原根为 $\gcd(t,\varphi(n))=1$ 的 g^t，共 $\varphi(\varphi(n))$ 个，按大小输出需排序。实现见第 123 页。$\operatorname{ord}(a^k)=d/\gcd(d,k)$；两阶互素时乘积之阶为两阶之积。

18.15.3 自动准备模数分解与原根

奇素数幂 $m=p^e$ 有原根，设 $\varphi=m/p\cdot(p-1)$，候选 g 为原根当且仅当

$$p\nmid g,\qquad g^{\varphi/q}\not\equiv 1 \pmod m \quad \text{对每个不同素因子} q\mid\varphi.$$

须检查模 p^e 的阶，不能只取模 p 的原根。准备成本含 M 与各 φ 的分解及逐个候选的模幂；同一模数多个方程应复用分解结果。

18.16 离散对数

18.16.1 扩展 BSGS

问题描述：给定 a, b, m（a 与 m 不必互质），求最小的非负整数 x 使 $a^x \equiv b \pmod m$，无解返回 -1。

核心思路：把方程看成 $k \cdot a^{x-\text{offset}} \equiv b$ 逐步化简：只要 $g = \gcd(a, m) > 1$，先检查当前指数 `offset` 是否已是解（即 $k \equiv b$），否则要求 $g \mid b$，把 b 和模数同除以 g，系数 k 乘上 a/g，偏移加一。a 与 m 互质后求 $a^y \equiv bk^{-1}$：baby step 表存 a^j（$0 \le j < s$，同值保留最小 j），giant step 从目标值出发每次乘 a^{-s}，第 i 步命中 a^j 即得 $y = is + j$。i 递增、j 取最小，首个命中就是最小解。

思路中的情况	代码中的处理	行
平凡情形	$m = 1$ 或 $b = 1$ 时 $x = 0$；底数为 1 而 $b \neq 1$ 时无解	11
$\gcd(a, m) > 1$	循环剥离公因子	14
较小指数恰为解	$k \equiv b$ 时返回当前 `offset`	16
化为互质问题	目标值乘 k^{-1}	23
baby step	记录 a^j 的最小指数	30
giant step 命中	返回 offset $+ is + j$	38

代码依赖：Mod64（第 85 页，17.1 节）、mod_inverse（第 104 页，18.8.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：依赖 number_theory.hpp 的求逆与快速幂。solve(a,b,m) 返回最小 x≥0 使 a^x≡b (mod m)，无解返回 -1；1≤m≤10^{12}，a、b 可为完整 long long 范围，入口归一化，约定 0^0=1。先剥离 gcd 并逐次检查较小指数，再在互质剩余问题上 BSGS。baby 表对同余幂保留最小指数 j，按递增大步 i 检查区间，因此首个命中即最小解。单次期望时间 O(√m+log m)，空间 O(√m)，哈希表先 reserve；基于 unordered_map 的界为期望界。每次调用独立建表；底数 1 及零指数解直接返回。

```
struct DiscreteLog
{
    using ll = long long;

    // 1<=m<=1e12; Expected O(sqrt(m)) time, O(sqrt(m)) storage. Smallest x>=0, or -1.
    static ll solve(ll a, ll b, ll m)
    {
        assert(m >= 1 && m <= 1000000000000LL);
        a = (a % m + m) % m;
        b = (b % m + m) % m;
        if (m == 1 || b == 1) return 0;
        if (a == 1) return -1;
        ll offset = 0, k = 1;
        for (ll g; (g = gcd(a, m)) > 1;)
        {
            if (b == k) return offset;
            if (b % g) return -1;
            b /= g;
            m /= g;
            k = (__int128)k * (a / g) % m;
            ++offset;
        }
        ll target = (__int128)b * mod_inverse(k, m) % m;
        ll step = sqrtl(m) + 1;
        unordered_map<ll, ll> baby;
        baby.reserve(step);
        ll cur = 1 % m;
        for (ll j = 0; j < step; j++)
```

```
        {
            if (!baby.count(cur)) baby[cur] = j;
            cur = (__int128)cur * a % m;
        }
        ll inv = mod_inverse(Mod64::power(a, step, m), m);
        cur = target;
        for (ll i = 0; i <= step; i++)
        {
            auto it = baby.find(cur);
            if (it != baby.end()) return offset + i * step + it->second;
            cur = (__int128)cur * inv % m;
        }
        return -1;
    }
};
```

模板题使用：Luogu P4195

题意：多组求 a^x 与 b 模 p 同余的最小非负整数 x，a 与 p 不必互质。输入顺序为 a、p、b，调用顺序为 solve(a,b,p)；返回 -1 时输出 No Solution，三个 0 结束输入。

所需模板：Mod64（第 85 页，17.1 节）、extended_gcd（第 101 页，18.6.1 节）、mod_inverse（第 104 页，18.8.1 节）、DiscreteLog（第 127 页，18.16.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4195

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    long long a, p, b;
    while (std::cin >> a >> p >> b)
    {
        if (a == 0 && p == 0 && b == 0) break;
        long long answer = DiscreteLog::solve(a, b, p);
        if (answer == -1)
            std::cout << "No Solution\n";
        else
            std::cout << answer << '\n';
    }
}
```

18.17 高次剩余 & 单位根

18.17.1 素数模高次剩余：全部根的紧凑表示

问题描述：给定素数 p 及其原根 g、正整数 k 与整数 a，求同余方程 $x^k \equiv a \pmod p$ 的全部解。解集以首项 `first`、公比 `ratio`、个数 `count` 的等比数列形式返回，不必展开即可得知解数。

核心思路：令 $x = g^y$，并用 `DiscreteLog` 求出 $a = g^L$ 中的 L，原方程化为指数上的线性同余 $ky \equiv L \pmod{p-1}$。`linear_congruence` 给出最小解 y_0 与周期 $T = (p-1)/\gcd(k, p-1)$，同余无解则原方程无解。于是全部根为 $g^{y_0}(g^T)^i$，$0 \le i < \gcd(k, p-1)$，即 `first` 取 g^{y_0}、`ratio` 取 g^T、`count` 取 $(p-1)/T$。$a \equiv 0$ 时唯一根为 0，单独返回。

代码依赖：DiscreteLog（第 127 页，18.16.1 节）、Mod64（第 85 页，17.1 节）、linear_congruence（第 108 页，18.9.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：solve(a,k,p,g) 求 x^k≡a (mod p)；p 必须是 2 到 1e12 的素数，g 是模 p 原根，k 为正 unsigned long long，a 支持完整 signed long long。无解返回 nullopt；否则得到 first、ratio、count，全部解为 first*ratio^i mod p，0≤i<count，无重复；first 不保证最小。a≡0 时仅有根 0，返回 {0,1,1}。枚举时每次乘 ratio，乘法先转 int128；无需展开全部根即可读取解数。给定原根后，期望 O(√p) 时间与空间，依赖 DiscreteLog、linear_congruence 及模快速幂。不验证 p 的素性或 g 的阶；合数模数不在此接口范围内。

```
struct KthResidue
{
    using ll = long long;
    ll first, ratio, count;

    // Prime p <= 1e12; g is a primitive root of p; k > 0.
    // All roots: first * ratio^i mod p, 0 <= i < count.
    static optional<KthResidue> solve(ll a, unsigned long long k, ll p, ll g)
    {
        assert(p >= 2 && p <= 1000000000000LL && g >= 1 && g < p && k > 0);
        a %= p;
        if (a < 0) a += p;
        if (a == 0) return KthResidue{0, 1, 1};
        ll logarithm = DiscreteLog::solve(g, a, p);
        if (logarithm < 0) return nullopt;
        auto [exponent, period] = linear_congruence(k % (p - 1), -logarithm, p - 1);
        if (exponent < 0) return nullopt;
        ll first = Mod64::power(g, exponent, p);
        ll ratio = Mod64::power(g, period, p);
        return KthResidue{first, ratio, (p - 1) / period};
    }
};
```

18.17.2 素数模高次剩余与单位根

实现见第 129 页；依赖离散对数、线性同余和快速幂。设 p 为素数，g 是原根，$k > 0$。$a = 0$ 时根只有零；否则 $a = g^b$、$x = g^y$，

$$ky \equiv b \pmod{p-1}.$$

令 $d = \gcd(k, p-1)$，$d \nmid b$ 时无解；否则 $y = y_0 + i(p-1)/d$，$0 \le i < d$，存为

$$\texttt{first} = g^{y_0}, \qquad \texttt{ratio} = g^{(p-1)/d}, \qquad \texttt{count} = d.$$

依次乘 ratio 枚举，不保证递增。$p \le 10^{12}$，调用者提供已验证的素数与原根；k 为 unsigned long long，a 先正规化。空 optional 表示无解，`{0,1,1}` 表示唯一零根。期望时间、空间 $O(\sqrt{p})$，枚举另需 $O(d)$；模乘须扩展到 128 位。

18.17.3 素数幂模高次剩余与根的提升

令 $M = p^e$。$a = 0$ 时根为 $p^{\lceil e/k \rceil}$ 的倍数。否则 $t = v_p(a) < e$，须 $k \mid t$，否则无解；取 $s = t/k$，$q = p^{e-t}$，$x = p^s y$，解

$$y^k \equiv a/p^t \pmod{q},$$

每个单位根 y 对应 $x = p^s y + \ell p^s q$，$0 \le \ell < p^{t-s}$。奇素数用模 p^e 的原根转线性同余；$q = 2$ 时单位根只有 1；$q \ge 4$ 为二的幂时单位写成 $(-1)^z 5^y$，z 模 2、y 模 $q/4$（$q = 4$ 只有符号分支），分别解指数同余。二的高次幂没有原根。时间由 BSGS 决定，期望 $O(\sqrt{q})$。

18.17.4 一般合数模高次剩余：CRT 组合

设 $M = \prod_t m_t$，$m_t = p_t^{e_t}$ 两两互素。分别求局部根族，任一局部无解则整体无解；整体根数为 $\prod_t c_t$。预处理

$$E_t = \frac{M}{m_t}\left(\left(\frac{M}{m_t}\right)^{-1} \bmod m_t\right) \bmod M, \qquad x = \sum_t r_t E_t \bmod M.$$

按混合进制索引可逐个取根，不需建出全部根；CRT 乘积扩展到 128 位。$M = 1$ 时唯一代表为 0。

18.18 数论分块

18.18.1 整除分块：前缀和来源决定算法

$H(t) = \sum_{i \le t} h(i)$，$H(0) = 0$，$q_A = \lfloor A/l \rfloor, q_B = \lfloor B/l \rfloor$，$l \le \min(A, B)$：

$$r = \min(\lfloor A/q_A \rfloor, \lfloor B/q_B \rfloor), \quad \text{整块贡献}(H(r) - H(l-1))q_A q_B.$$

两边都要参与取 r。段数 $O(\sqrt{A} + \sqrt{B})$（且 $\le \min(A, B)$），不是 $O(\sqrt{\min(A, B)})$。

18.18.1.1 按数据与资源选前缀和

上界可预筛，询问多： 第 98 页 `LinearSieve s(L)` 的 `s.mu`、`s.phi` 建前缀和，$O(L)$ 时间空间，查表 $O(1)$；筛上限覆盖短边。μ 前缀可负，φ 前缀勿用 `int`。

只求少量巨大前缀： 第 132 页 `DuJiao s(B)`：`s.mertens(n)`（有符号 64 位）、`s.totient_sum(n)`（128 位），$0 \le n \le 10^{11}$，$1 \le B <$ `INT_MAX`，B 是预筛上限。单次时间 $O(B + N/\sqrt{B})$，$B \approx N^{2/3}$ 得 $O(N^{2/3})$（哈希期望）。

内存：`DuJiao` 两张 64 位前缀表约 $16(B+1)$ 字节，构造时另有 `LinearSieve` 表；$N = 10^{11}$ 取 $B = N^{2/3}$ 约 345 MB，先看内存限制。不同查询增缓存；勿每次重建筛对象。正方形互质对直接 `2 * s.totient_sum(n) - 1`（$n \ge 1$）。一般矩形用 `mertens` 分块时，内层递归使总复杂度不止外层段数。

$$M(n) = 1 - \sum_{i=2}^{n} M(\lfloor n/i \rfloor), \qquad \Phi(n) = \frac{n(n+1)}{2} - \sum_{i=2}^{n} \Phi(\lfloor n/i \rfloor),$$

$M(n) = \sum_{i \le n} \mu(i)$，按等商段乘段长。先提升类型再乘；模下负和要归一化；最后除以 2 时先整数除再取模，合数模下不擅自求 2 的逆元。

18.19 狄利克雷卷积

18.19.1 幂的约数和：不除以 $p-1$

若 $a=\prod p_i^{e_i}$，则 $\sigma(a^b)=\prod_i(1+p_i+\cdots+p_i^{e_ib})$。合数模数或 $p\equiv 1$ 时分母不可逆，不能用几何级数除法公式。令 $G(a,n)=(a^n,\sum_{i=0}^{n-1}a^i)$，$G(a,0)=(1,0)$；由 $h=\lfloor n/2\rfloor$ 的 (P,S)，偶数项得 $(P^2,S(1+P))$，奇数项得 $(P^2a,S(1+P)+P^2)$。时间 $O(\log(n+1))$，任意正模数。参数是项数，约数和须传 e_ib+1；e_ib 可能超过 64 位，模乘在 128 位中做。

18.20 莫比乌斯反演

18.20.1 整除与反演：先辨认求和方向

数论函数定义在**正整数**上；下标 0 只是前缀和占位。

$$(f*g)(n)=\sum_{d\mid n}f(d)g(n/d),\qquad \varepsilon(n)=[n=1],\quad \mathbf{1}(n)=1,\quad \mathrm{id}(n)=n,\qquad \mu*\mathbf{1}=\varepsilon,\quad \varphi*\mathbf{1}=\mathrm{id}\,.$$

18.20.1.1 约数和与倍数和不能混用

约数方向。$F(n)=\sum_{d\mid n}f(d)\Rightarrow f(n)=\sum_{d\mid n}\mu(d)F(n/d)$，不要求 f 积性。
倍数方向。$E(k)$ 仅在 $1\le k\le V$ 非零，$G(d)=\sum_{j\ge 1,\,dj\le V}E(dj)$，则

$$E(k)=\sum_{t=1}^{\lfloor V/k\rfloor}\mu(t)G(kt),$$

或从大到小 $E(k)=G(k)-\sum_{j\ge 2,\,kj\le V}E(kj)$。须有限支持。互质下标对：$c_d=\#\{i:d\mid a_i\}$，答案 $\sum_{d=1}^{V}\mu(d)\binom{c_d}{2}$（重复值按下标区分）。枚举倍数求 c_d，时间 $O(V\log V)$、空间 $O(V)$；V 太大时须改为枚举实际约数。

18.20.1.2 GCD 权值先反演，再交换求和

令 $h=\mu*w$（$w=h*\mathbf{1}$），则

$$\sum_{x=1}^{A}\sum_{y=1}^{B}w(\gcd(x,y))=\sum_{d=1}^{\min(A,B)}h(d)\left\lfloor\frac{A}{d}\right\rfloor\left\lfloor\frac{B}{d}\right\rfloor.$$

互质计数 $h=\mu$，GCD 总和 $h=\varphi$；系数是卷积 $h(d)$，一般不是 $\mu(d)w(d)$。已知 h 的前缀和后按第 18.18.1 节分块。

18.20.2 GCD 计数：缩放、边界与对称性

记 $C(A,B)$ 为 $[1,A]\times[1,B]$ 中的**有序**互质数对数，任一边为 0 时为 0；不含坐标轴，不去掉 $x=y$。

18.20.2.1 恰好等于 k：先缩放整除对象，再做矩形容斥

$k\ge 1$，$a,c\ge 1$，$a\le x\le b$、$c\le y\le d$ 中 $\gcd=k$ 的对数为 $C(A,B)-C(A_0,B)-C(A,B_0)+C(A_0,B_0)$，其中

$$A=\lfloor b/k\rfloor,\quad A_0=\lfloor(a-1)/k\rfloor,\qquad B=\lfloor d/k\rfloor,\quad B_0=\lfloor(c-1)/k\rfloor.$$

先减 1 再除；$k>\min(b,d)$ 时自动为 0。多测预筛到所有查询的 $\max\min(b/k,d/k)$。

18.20.2.2 去掉顺序之前，先检查交换是否仍在定义域内

同一正整数区间 I，T 为 $I\times I$ 中 GCD 为 k 的有序对数，$\delta=[k\in I]$：$\#\{x<y\}=(T-\delta)/2$，$\#\{x\le y\}=(T+\delta)/2$。仅当定义域交换对称时可用；一般矩形不能直接除以 2。不同下标计数回到频率反演。正方形：$C(n,n)=2\Phi(n)-1$，$\Phi(n)=\sum_{i\le n}\varphi(i)$，$n\ge 1$；$n=0$ 为 0。含 0：$\gcd(0,y)=|y|$，轴上仅 $(0,k),(k,0)$ 另行判断；$(0,0)$ 按题面约定；负数处理绝对值。

18.20.3 矩形 GCD 计数：缩放后容斥

前缀矩形互质数对数

$$F(A,B)=\sum_{i=1}^{\min(A,B)}\mu(i)\left\lfloor\frac{A}{i}\right\rfloor\left\lfloor\frac{B}{i}\right\rfloor,$$

按两个商同时分块，右端点 $r=\min(A/\lfloor A/l\rfloor,B/\lfloor B/l\rfloor)$，用 μ 前缀和；只需筛到 $\min(A,B)$。$A=0$ 或 $B=0$ 时为 0。矩形 $[a,b]\times[c,d]$ 内 $\gcd(x,y)=k$ 的对数为

$$\begin{aligned}&F(\lfloor b/k\rfloor,\lfloor d/k\rfloor)-F(\lfloor(a-1)/k\rfloor,\lfloor d/k\rfloor)\\&-F(\lfloor b/k\rfloor,\lfloor(c-1)/k\rfloor)+F(\lfloor(a-1)/k\rfloor,\lfloor(c-1)/k\rfloor),\end{aligned}$$

必须先减 1 再除以 k。答案不超过面积，可存有符号 64 位；分块累加用 128 位。

18.21 狄利克雷双曲线法 & 杜教筛

18.21.1 杜教筛

问题描述：对很大的 n（约 10^{11}），求莫比乌斯函数前缀和 $M(n)=\sum_{i\le n}\mu(i)$ 与欧拉函数前缀和 $\Phi(n)=\sum_{i\le n}\varphi(i)$，支持多次查询。

核心思路：利用狄利克雷卷积 $\mu*1=\varepsilon$、$\varphi*1=\mathrm{id}$ 得 $\sum_{d=1}^{n}M(\lfloor n/d\rfloor)=1$、$\sum_{d=1}^{n}\Phi(\lfloor n/d\rfloor)=n(n+1)/2$。把 $d=1$ 项移到一边，其余 $d\ge 2$ 按 $\lfloor n/d\rfloor$ 相同的区间 $[l,r]$ 整除分块，递归求更小的值。不超过 `limit` 的值直接查线性筛预处理的前缀和 `pmu`、`pphi`，更大的值记忆化在哈希表 `mmu`、`mphi` 中。

代码依赖：LinearSieve（第 98 页，18.4.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：构造预筛上限 1≤B<INT_MAX，内存须可用；mertens(n) 返回 μ 的前缀和，totient_sum(n) 返回 φ 的前缀和（int128）。约定 0≤n≤10^{11}，n=0 返回 0。按整除分块递归并写入 mmu/mphi 缓存；重复查询复用结果，清空缓存可重算，不可修改 pmu/pphi/limit。单次最大参数 N 的常用均摊分析为 O(B+N/$\sqrt{}$B)（哈希按期望常数计算），B 约 N^(2/3) 时为 O(N^(2/3))；小 B 不构成大输入性能保证。多次不同 N 可能新增缓存，空间随实际缓存增长；递归深度 O(log N)。

```
struct DuJiao
{
    using ll = long long;
    using I = __int128_t;
    int limit;
    vector<ll> pmu, pphi;
    unordered_map<ll, ll> mmu;
    unordered_map<ll, I> mphi;

    DuJiao(int limit) : limit(limit), pmu(limit + 1), pphi(limit + 1)
    {
        assert(limit >= 1);
        LinearSieve s(limit);
        for (int i = 1; i <= limit; i++)
        {
            pmu[i] = pmu[i - 1] + s.mu[i];
            pphi[i] = pphi[i - 1] + s.phi[i];
        }
    }

    ll mertens(ll n)
    {
        if (n <= limit) return pmu[n];
        if (mmu.count(n)) return mmu[n];
        ll ans = 1;
```

```
        for (ll l = 2, r; l <= n; l = r + 1)
        {
            r = n / (n / l);
            ans -= (r - l + 1) * mertens(n / l);
        }
        return mmu[n] = ans;
    }

    I totient_sum(ll n)
    {
        if (n <= limit) return pphi[n];
        if (mphi.count(n)) return mphi[n];
        I ans = I(n) * (n + 1) / 2;
        for (ll l = 2, r; l <= n; l = r + 1)
        {
            r = n / (n / l);
            ans -= I(r - l + 1) * totient_sum(n / l);
        }
        return mphi[n] = ans;
    }
};
```

模板题使用：Luogu P4213

题意：多次查询 φ 与 μ 的前缀和，先输出 φ，再输出 μ。先读入全部 n，预筛到 min(最大 n,2000000)，同一结构复用缓存。题目 n<2^31，所以 φ 的总和可安全转为 long long；模板更大范围的 int128 返回值不能照搬此转换。

所需模板：LinearSieve（第 98 页，18.4.2 节）、DuJiao（第 132 页，18.21.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4213

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    vector<long long> a(t);
    for (auto &n : a) cin >> n;
    int b = min(2000000LL, *max_element(a.begin(), a.end()));
    DuJiao d(b);
    for (long long n : a)
        cout << (long long)d.totient_sum(n) << ' ' << d.mertens(n) << '\n';
}
```

18.21.2 杜教筛：预筛与缓存都不可省略

整除反演建模（第 18.20.1 节，第 131 页）的两条卷积恒等式给出 DuJiao 的 mertens 与 totient_sum 递推：商相同的一段合并，系数为区间长度；超过预筛上限的结果必须写回缓存。最大查询 N、预筛上限 B 时单次 $O(B+N/\sqrt{B})$，取 $B\approx N^{2/3}$ 得 $O(N^{2/3})$；多个不同查询会继续增加缓存。范围 $0\le n\le 10^{11}$，欧拉函数前缀和用 128 位。取模时负值先求余，仅余数为负时加模数。

18.22 Min_25 筛

18.22.1 Min_25 积性函数求和

问题描述：给定积性函数 f：$f(p)$ 是素数 p 的至多二次多项式，$f(p^e)$ 由回调给出。求 $\sum_{i=1}^{n} f(i)$ 模素数，n 可达 10^{12}。

核心思路：所有 $\lfloor n/l \rfloor$ 只有 $O(\sqrt{n})$ 种，降序存于 `w`，经 `id` 用 `small`、`large` 两张表定位。第一阶段对每个 $x \in w$ 求素数幂和 $g(x,d)=\sum_{p\le x} p^d$（$d=0,1,2$）：初值为 $\sum_{i=2}^{x} i^d$，依次对素数 $p \le \sqrt{n}$ 去掉最小质因子为 p 的合数。第二阶段 `dfs(x,k)` 求 $2..x$ 中最小质因子不小于第 k 个素数的数的 f 之和：素数部分由 g 的线性组合减去前 k 个素数的和，合数部分枚举最小质因子及其指数递归。

思路中的情况	代码中的处理	行
枚举所有整除商	$x=\lfloor n/l \rfloor$ 入 `w`，按 $x \le s$ 与否记入 `small[x]` 或 `large[n/x]`	21
g 的初值	把 $2..x$ 都当作素数：$\sum_{i=2}^{x} i^0, i^1, i^2$	32
判断 p 是否为素数	此时 $g(p,0)$ 已是精确的素数个数，与 $g(p-1,0)$ 相等说明 p 是合数	38
筛去最小质因子为 p 的合数	对 $x \ge p^2$：$g(x)$ $-=$ $p^d\,(g(x/p)-\sum_{q<p} q^d)$，`prefix` 是较小素数的幂和	45
`dfs` 的素数部分	$c_0g_0+c_1g_1+c_2g_2$ 减去前 k 个素数的 $f(p)$ 之和	65
`dfs` 的合数部分	枚举 p^e：加 $f(p^e)\cdot$`dfs`$(x/p^e, i+1)$ 与 $f(p^{e+1})$	70
加上 $f(1)$	`dfs` 只统计 ≥ 2 的数，最后补 1	75

代码依赖：ModInt（第 90 页，18.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：构造 Min25<mod>(n)，1≤n≤10^12，mod 为大于 3 的素数。sum(c, power) 求 ∑(1≤i≤n) f(i)，要求 f 为积性函数、f(1)=1，f(p)=c[0]+c[1]p+c[2]p²；power(p,e,pe) 返回 f(p^e)，其中 pe=p^e、e≥1，回调必须与一次幂系数一致。支持同一对象多次 sum；不改筛表。预处理 O(n^0.75 / log n)，回调 O(1) 时递归求和保守 O(n) 上界；O(√n) 空间、O(log n) 递归栈。w 降序存不同整除商，g[id(x)][d] 是模意义下 ∑(p≤x) p^d；id 仅允许查询这些整除商（包含 1..√n），各字段只读。依赖 ModInt；普通递归版，未承诺全部官方时限达标。

```
template <int mod> struct Min25
{
    using ll = long long;
    using Z = ModInt<mod>;
    ll n;
    int s;
    vector<ll> w;
    vector<int> small, large, primes;
    vector<array<Z, 3>> g;

    Min25(ll n) : n(n)
    {
        static_assert(mod > 3);
        assert(1 <= n && n <= 1000000000000LL);
        s = sqrtl(n);
        while (1LL * (s + 1) * (s + 1) <= n) s++;
        while (1LL * s * s > n) s--;
        small.resize(s + 1);
        large.resize(s + 1);
        Z half = Z(2).inv(), sixth = Z(6).inv();
        for (ll l = 1, r; l <= n; l = r + 1)
        {
            ll x = n / l;
            r = n / x;
            int k = w.size();
            if (x <= s)
                small[x] = k;
            else
                large[n / x] = k;
            w.push_back(x);
            Z v = x;
            g.push_back(
                {v - 1, v * (v + 1) * half - 1, v * (v + 1) * (v * 2 + 1) * sixth - 1});
        }
```

```
        array<Z, 3> prefix{};
        for (int p = 2; p <= s; p++)
        {
            if (g[id(p)][0].v == g[id(p - 1)][0].v) continue;
            primes.push_back(p);
            array<Z, 3> weight{1, p, Z(p) * p};
            for (int j = 0; j < (int)w.size() && 1LL * p * p <= w[j]; j++)
            {
                int k = id(w[j] / p);
                for (int d = 0; d < 3; d++)
                    g[j][d] = g[j][d] - weight[d] * (g[k][d] - prefix[d]);
            }
            for (int d = 0; d < 3; d++) prefix[d] = prefix[d] + weight[d];
        }
    }

    int id(ll x) const { return x <= s ? small[x] : large[n / x]; }

    template <class F> Z sum(array<Z, 3> c, F power) const
    {
        vector<Z> prefix(primes.size() + 1);
        for (int i = 0; i < (int)primes.size(); i++)
        {
            Z p = primes[i];
            prefix[i + 1] = prefix[i] + c[0] + c[1] * p + c[2] * p * p;
        }
        auto dfs = [&](auto &&self, ll x, int k) -> Z
        {
            if (x < 2 || (k && x <= primes[k - 1])) return 0;
            auto a = g[id(x)];
            Z ans = c[0] * a[0] + c[1] * a[1] + c[2] * a[2] - prefix[k];
            for (int i = k; i < (int)primes.size() && primes[i] <= x / primes[i]; i++)
            {
                ll p = primes[i], pe = p;
                for (int e = 1; pe <= x / p; e++, pe *= p)
                    ans = ans + power(p, e, pe) * self(self, x / pe, i + 1) +
                          power(p, e + 1, pe * p);
            }
            return ans;
        };
        return dfs(dfs, n, 0) + 1;
    }
};
```

模板题使用：Luogu P5325

题意：求积性函数前缀和，f(p^e)=p^e(p^e-1)，模 1000000007。一次幂为 p^2-p，故系数传 {0,-1,1}；回调的 pe 已是整数 p^e，无需再快速幂。

所需模板：ModInt（第 90 页，18.1.2 节）、Min25（第 134 页，18.22.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5325

```
int main()
{
    using Z = ModInt<1000000007>;
    long long n;
    cin >> n;
    Min25<1000000007> sieve(n);
    auto power = [](long long p, int e, long long pe)
    {
        Z x = pe;
        return x * (x - 1);
    };
    cout << sieve.sum({0, -1, 1}, power).v << '\n';
}
```

18.23 类欧几里德算法

18.23.1 有符号整除和：类欧几里德

问题描述：给定 n, m, a, b（a, b 可以为负），求 $\sum_{i=0}^{n-1} \lfloor (ai+b)/m \rfloor$。

核心思路：先把 a, b 按数学向下取整规范到 $[0, m)$，分离出的商 q_a, q_b 贡献 $q_a \cdot n(n-1)/2 + q_b \cdot n$。之后循环（类欧几里得）：每轮先把 a, b 中不小于 m 的整商部分直接累加并取余；若 $y = an + b < m$，剩下各项全为 0，结束。否则把和看成直线 $(ax+b)/m$ 下方的格点数并交换坐标轴，问题化为项数 $\lfloor y/m \rfloor$、常数 $y \bmod m$、并交换 a 与 m 的同形和式，(a, m) 像欧几里得算法一样迅速缩小。

接口契约：floor_sum(n,m,a,b) 返回 i=0..n-1 的 floor((ai+b)/m) 之和，结果为 int128。要求 0≤n≤10^9、1≤m≤10^9，a、b 支持完整 long long；n=0 返回 0。先按数学向下取整规范化负系数，再迭代交换分母和余数；不能把 C++ 负数截零除法当作 floor。O(log m) 时间、O(1) 空间，不依赖完整数论模块。

```
__int128_t floor_sum(long long n, long long m, long long a, long long b)
{
    using ll = long long;
    using i128 = __int128_t;
    assert(n >= 0 && m > 0);
    i128 ans = 0;
    auto norm = [&](ll &v)
    {
        ll q = v / m;
        if (v % m < 0) --q;
        v = (ll)(i128(v) - i128(q) * m);
        return q;
    };
    ans += i128(norm(a)) * n * (n - 1) / 2;
    ans += i128(norm(b)) * n;
    while (true)
    {
        ans += i128(n) * (n - 1) / 2 * (a / m);
        a %= m;
        ans += i128(n) * (b / m);
        b %= m;
        i128 y = i128(a) * n + b;
        if (y < m) break;
        n = (ll)(y / m);
        b = (ll)(y % m);
        swap(a, m);
    }
    return ans;
}
```

模板题使用：Library Checker sum_of_floor_of_linear

题意：求 i=0..n-1 的 floor((a*i+b)/m) 之和。此题有 0≤a,b<m，因此结果可转为 long long 输出；更一般的有符号输入不能照搬该转换。

所需模板：floor_sum（第 138 页，18.23.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/sum_of_floor_of_linear

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while (t--)
    {
        long long n, m, a, b;
        cin >> n >> m >> a >> b;
        // This task has 0 <= a,b < m, so the sum is at most n*(n-1)/2.
        cout << (long long)floor_sum(n, m, a, b) << '\n';
    }
}
```

18.23.2 整除和、带权和与平方和

问题描述： 对 $i=0..n-1$ 令 $y_i=\lfloor(ai+b)/m\rfloor$，同时求 $\sum y_i$、$\sum i\,y_i$、$\sum y_i^2$（模 998244353）。

核心思路： 先把 a,b 规范到 $[0,m)$，商为 x,y，则 $y_i=xi+y+z_i$，展开后只需 z_i 的三种和 f,g,h 及 $\sum i$、$\sum i^2$（s1、s2）。求 f,g,h 时令 t 为最大值 height，交换求和次序：$z_i\ge j$ 当且仅当 $i\ge c_j$，而 $c_{j+1}=\lfloor(mj+m+a-b-1)/a\rfloor$ 仍是同形的整除式，于是以参数 $(t,a,m,m+a-b-1)$ 递归（a 与 m 互换，类欧几里得）得到 $u=\sum c$、$v=\sum j\,c$、$w=\sum c^2$，再由 $f=nt-u$、$g=t\,s_1-(w-u)/2$、$h=nt^2-2v-u$ 还原。

代码依赖：ModInt（第 90 页，18.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约： floor_moments(n,m,a,b) 对 i=0..n-1，令 y_i=floor((a*i+b)/m)，返回 {∑y_i, ∑i*y_i, ∑y_i²}，元素为 ModInt<998244353>，取整数用.v。0≤n≤1000000001，1≤m≤10^9；a、b 支持完整 long long。n=0 返回全零。时间与递归空间 O(log(m+1))；模数固定，499122177 与 166374059 分别是 2、6 的逆元。P5170 输入上限为闭区间，驱动传 n+1，输出索引顺序 0、2、1。

```
array<ModInt<998244353>, 3>
floor_moments(long long n, long long m, long long a, long long b)
{
    using Z = ModInt<998244353>;
    assert(n >= 0 && n <= 1000000001 && m >= 1 && m <= 1000000000);
    if (!n) return {0, 0, 0};
    auto normalize = [&](long long &v)
    {
        long long q = v / m;
        if (v % m < 0) q--;
        v = (__int128_t(v) - __int128_t(q) * m);
        return Z(q);
    };
    Z x = normalize(a);
    Z y = normalize(b);
    Z count = n;
    Z s1 = count * Z(n - 1) * Z(499122177);
    Z s2 = count * Z(n - 1) * Z(2 * n - 1) * Z(166374059);
    Z f = 0, g = 0, h = 0;
    if (a)
    {
        long long height = (a * (n - 1) + b) / m;
        auto [u, v, w] = floor_moments(height, a, m, m + a - b - 1);
        Z t = height;
        f = count * t - u;
        g = t * s1 - (w - u) * Z(499122177);
        h = count * t * t - Z(2) * v - u;
    }
    return {f + x * s1 + y * count,
            g + x * s2 + y * s1,
            h + x * x * s2 + y * y * count + Z(2) * x * y * s1 + Z(2) * x * g +
                Z(2) * y * f};
}
```

模板题使用：Luogu P5170

题意：求 i=0..n 的整除和、整除值平方和、i 乘整除值之和，模 998244353。t 至 10^5，n/a/b/c 至 10^9 且 c>0。核心参数顺序 (n+1,c,a,b)，返回顺序是和/带权和/平方和；本题输出索引 0、2、1，不能只核对第一项。

所需模板：ModInt（第 90 页，18.1.2 节）、floor_moments（第 140 页，18.23.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5170

```
int main()
{
    int t;
    scanf("%d", &t);
    while (t--)
    {
        long long n, a, b, c;
        scanf("%lld%lld%lld%lld", &n, &a, &b, &c);
        auto result = floor_moments(n + 1, c, a, b);
        printf("%d %d %d\n", result[0].v, result[2].v, result[1].v);
    }
    return 0;
}
```

18.23.3 类欧几里德：有符号整除求和

接口见第 18.23.1 节（第 138 页）。`floor_sum(n,m,a,b)` 返回

$$F(n,m,a,b)=\sum_{i=0}^{n-1}\left\lfloor\frac{ai+b}{m}\right\rfloor.$$

要求 $0\le n\le 10^9$、$1\le m\le 10^9$，a,b 为完整有符号 64 位，返回 128 位。n 是项数，下标 $0\dots N$ 时传 $N+1$。令 $a=q_am+r_a$、$b=q_bm+r_b$（余数在 $[0,m)$，负数商须向下取整），先加 $q_an(n-1)/2+q_bn$。对 $0\le a,b<m$，设 $y=an+b$；$y<m$ 时余下为 0，否则

$$F(n,m,a,b)=F(\lfloor y/m\rfloor,a,m,y \bmod m).$$

时间 $O(\log(m+1))$，额外空间 $O(1)$。

18.23.4 类欧几里德的三个求和量

实现见第 18.23.2 节（第 140 页）。令 $y_i=\lfloor(ai+b)/m\rfloor$，$0\le i<n$，求 $f=\sum y_i$、$g=\sum iy_i$、$h=\sum y_i^2$。范围 $0\le n\le 1000000001$、$1\le m\le 10^9$，a,b 为完整有符号 64 位；$n=0$ 返回 $(0,0,0)$。结果固定模 998244353，换模数须同时换 2、6 的逆元。

归一化 $a=xm+a'$、$b=ym+b'$（向下取整），规范化问题返回 (f_0,g_0,h_0)，$S_1=\sum i$、$S_2=\sum i^2$：

$$
\begin{aligned}
f &= f_0 + xS_1 + yn,\\
g &= g_0 + xS_2 + yS_1,\\
h &= h_0 + x^2S_2 + y^2n + 2xyS_1 + 2xg_0 + 2yf_0.
\end{aligned}
$$

$a'=0$ 时剩余量为 0。$0<a'<m$ 时令 $t=\lfloor(a'(n-1)+b')/m\rfloor$，递归 $(t,a',m,m+a'-b'-1)$ 得 (u,v,w)：

$$
f_0=nt-u,\qquad g_0=tS_1-(w-u)/2,\qquad h_0=nt^2-2v-u.
$$

递归 $O(\log(m+1))$ 层。P5170 上界含 n，传 $n+1$，输出顺序为 f,h,g。

第十九章　多项式与生成函数

19.1　快速傅里叶变换

19.1.1　复数 FFT（可复用单位根）

功能：对长度为 n（二次幂）的复数数组做原地离散傅里叶变换及逆变换；单位根表在构造时一次算好，可供多次变换复用，是浮点多项式乘法、拆系数任意模卷积等的底层工具。

核心思路：构造时直接按角度用 $\cos,\sin$ 算出 $\omega^i = e^{2\pi \mathrm{i}\, i/n}$（$i < n/2$），不靠连乘，避免误差累积。`transform` 先做位逆序重排（`j` 是 `i` 的二进制反转，增量维护），再自底向上合并：两段长度为 `half` 的子结果用单位根 `roots[j*step]` 做蝴蝶 $u \pm \omega v$。逆变换把单位根取共轭，最后整体除以 n。

接口契约：ComplexFFT(n) 预计算单位根，n 为 1..2^22 的二次幂。transform(a,inverse=false) 原地正变换，正向指数为正；inverse=true 用共轭根并除以 n。a 长度须等于 n；C=complex<long double>。根直接用角度计算，不沿整层连乘。单次变换 O(n log(n+1))，根表 O(n)，可复用。浮点环境须二进制至少 53 位、最近舍入、禁用 fast-math；整数卷积还要求单位根绝对误差 $\le 8\times 2^{-53}$，本机全根已用百位参考核验。变换本身是近似值。

```
struct ComplexFFT
{
    using Real = long double;
    using C = complex<Real>;
    static constexpr int max_size = 1 << 22;
    int n;
    vector<C> roots;

    explicit ComplexFFT(int size) : n(size)
    {
        static_assert(numeric_limits<Real>::radix == 2);
        static_assert(numeric_limits<Real>::digits >= 53);
        assert(n > 0 && n <= max_size && (n & (n - 1)) == 0);
        roots.resize(n / 2);
        Real pi = acosl(-1);
        for (int i = 0; i < n / 2; i++)
        {
            Real angle = pi * (2.0L * i / n);
            roots[i] = C(cosl(angle), sinl(angle));
        }
    }
```

正逆变换（接上页同一结构体）：

```
    void transform(vector<C> &a, bool inverse = false) const
    {
        assert(a.size() == (unsigned)n);
        for (int i = 1, j = 0; i < n; i++)
        {
            int bit = n >> 1;
            for (; j & bit; bit >>= 1) j ^= bit;
            j ^= bit;
            if (i < j) swap(a[i], a[j]);
        }
        for (int half = 1; half < n; half *= 2)
        {
            int step = n / (2 * half);
            for (int i = 0; i < n; i += 2 * half)
            {
                for (int j = 0; j < half; j++)
                {
                    C w = roots[j * step];
                    if (inverse) w = conj(w);
                    C u = a[i + j], v = a[i + j + half] * w;
                    a[i + j] = u + v;
                    a[i + j + half] = u - v;
                }
            }
        }
        if (inverse)
            for (auto &x : a) x /= n;
    }
};
```

模板题使用：Luogu P3803 【模板】多项式乘法（FFT）—直接变换接口

题意：给出次数 0≤n,m≤1000000、系数均为 0..9 整数的两个多项式，按升幂读入并输出乘积的 n+m+1 项。补零到不小于 n+m+1 的最小二次幂 L（至多 2^21），显式构造 ComplexFFT，分别正变换、逐点相乘、逆变换后 llroundl 取整。保留本节的二进制至少 53 位、最近舍入、单位根误差界及禁用 fast-math 前提；本题范数乘积至多 81000081。时间 O(L log L)，空间 O(L)。这是底层变换接口的完整用法，卷积包装器和精确 CRT 用法另列。

所需模板：ComplexFFT（第 144 页，19.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3803

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    int size = 1;
    while (size < n + m + 1) size *= 2;
    auto fft = ComplexFFT(size);
    vector<ComplexFFT::C> a(size), b(size);
    for (int i = 0, x; i <= n; i++) cin >> x, a[i] = x;
    for (int i = 0, x; i <= m; i++) cin >> x, b[i] = x;
    fft.transform(a);
    fft.transform(b);
    for (int i = 0; i < size; i++) a[i] *= b[i];
    fft.transform(a, true);
    for (int i = 0; i <= n + m; i++) cout << llroundl(a[i].real()) << ' ';
    cout << '\n';
}
```

19.1.2 拆系数 FFT 任意模卷积

功能：计算两个整数多项式乘积的系数模任意给定整数 M，即 $c_k = \sum_{i+j=k} a_i b_j \bmod M$；$M$ 可以是合数或不支持 NTT 的素数。

核心思路：每个系数先归一化到 $[0, M)$，再拆成三个 11 位块 $v = v_0 + v_1 B + v_2 B^2$（$B = 2^{11}$），这样任意两块卷积的真实系数都足够小，浮点 FFT 后四舍五入即可精确还原。对 a, b 的三个块各做一次正变换（共六次），再对每对块 (i, j) 逐点相乘、逆变换、`llroundl` 取整，乘上 $B^{i+j} \bmod M$（预存于 `weight`）累加进结果，共九次逆变换。

代码依赖：ComplexFFT（第 144 页，19.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：convolution_mod_fft(a,b,mod) 接受 vector<int>，返回 [0,mod) 系数；1≤mod≤INT_MAX，允许合数与负输入，mod=1 返回零。空输入返回空，非空结果长度 ≤2^19。依赖 ComplexFFT 及其浮点前提；归一化后拆成三个 11 位块，六次正变换和九次逆变换，分别舍入后模合并。每路误差界小于 0.079；不依赖更宽的 long double。时间 O(N log N)，空间 O(N)，常数高于打包版本以保持数值界与代码清楚；覆盖 P4245。

```
// Three 11-bit limbs; same floating-point assumptions as convolution_fft.
vector<int> convolution_mod_fft(const vector<int> &a, const vector<int> &b, int mod)
{
    assert(mod >= 1);
    if (a.empty() || b.empty()) return {};
    assert(a.size() + b.size() - 1 <= (1 << 19));
    int size = a.size() + b.size() - 1, n = 1;
    while (n < size) n *= 2;
    vector<int> result(size);
    if (mod == 1) return result;
```

```
    ComplexFFT fft(n);
    using C = ComplexFFT::C;
    const int base = 1 << 11;
    vector<vector<C>> x(3, vector<C>(n)), y = x;
    for (int i = 0; i < (int)a.size(); i++)
    {
        unsigned v = (a[i] % mod + (long long)mod) % mod;
        for (int j = 0; j < 3; j++) x[j][i] = (v >> (11 * j)) & (base - 1);
    }
    for (int i = 0; i < (int)b.size(); i++)
    {
        unsigned v = (b[i] % mod + (long long)mod) % mod;
        for (int j = 0; j < 3; j++) y[j][i] = (v >> (11 * j)) & (base - 1);
    }
    for (int j = 0; j < 3; j++)
    {
        fft.transform(x[j]);
        fft.transform(y[j]);
    }
    long long weight[5] = {1};
    for (int i = 1; i < 5; i++) weight[i] = weight[i - 1] * base % mod;
    vector<C> z(n);
    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 3; j++)
        {
            for (int k = 0; k < n; k++) z[k] = x[i][k] * y[j][k];
            fft.transform(z, true);
            for (int k = 0; k < size; k++)
            {
                long long v = llroundl(z[k].real()) % mod;
                result[k] = (result[k] + v * weight[i + j]) % mod;
            }
        }
    }
    return result;
}
```

模板题使用：Luogu P4245

题意：输入两个多项式的次数、任意模数与升幂系数，输出模乘积。题目次数不超过十万、模数最多 1000000009；函数会先归一化再拆系数，不要求模数支持 NTT。这里使用三块浮点 FFT，并遵守其单位根与舍入前提。

所需模板：ComplexFFT（第 144 页，19.1.1 节）、convolution_mod_fft（第 146 页，19.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4245

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, mod;
    cin >> n >> m >> mod;
    vector<int> a(n + 1), b(m + 1);
    for (int &x : a) cin >> x;
    for (int &x : b) cin >> x;
    auto c = convolution_mod_fft(a, b, mod);
    for (int x : c) cout << x << ' ';
    cout << '\n';
}
```

相关应用：CDQ 分治配合 NTT 的卷积递推，附件标题为分治 FFT；算法不做浮点变换。

19.1.3 CDQ 分治卷积递推

问题描述：求满足半在线卷积递推 $f_i = \text{init}_i + \sum_{j<i} f_j\, g_{i-j}$ 的前 n 项（核满足 $g_0 = 0$，模 NTT 素数）。f_i 依赖自身更早的项，不能直接做一次卷积。

核心思路：在区间 $[l, r)$ 上分治：先递归解出左半 $[l, m)$，此时左半的 f 已全部确定；把它与核 $g_0, \ldots, g_{r-l-1}$ 做一次 NTT 乘法，乘积第 $i - l$ 项就是左半对右半位置 $i \in [m, r)$ 的贡献，累加进 `initial[i]`；再递归右半。区间长度为 1 时该位置的所有贡献都已加完（$g_0 = 0$，无自身项），即为最终值。数组 `initial` 原地存放结果。

代码依赖：ModInt（第 90 页，18.1.2 节）、NttConvolution（第 151 页，19.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：cdq_convolution<mod=998244353,primitive=3>(kernel,initial) 求 f[i]=initial[i]+∑(j<i)f[j]kernel[i-j]，返回前 n 项。两数组为等长 vector<ModInt<mod»，kernel[0] 必须为 0；空输入返回空。模数须为 int 范围素数，primitive 是原根；依赖 NttConvolution。非空时要求 n+floor(n/2)-1≤NTT::max_size，保证每层完整卷积可变换。初始项可任意，并非只有 f[0]=1；左半区解定后，将其到右半区的卷积贡献加上，再递归右半区。O(n $\log^2$(n+1)) 时间、O(n) 额外空间，递归深度 O(log n)。本函数是静态线性递推，不提供流式追加或非线性叶子回调。P4721 使用 initial[0]=1、其他项 0。

```
// f[i] = initial[i] + sum_{j<i} f[j] * kernel[i-j]; kernel[0] must be zero.
template <int mod = 998244353, int primitive = 3>
vector<ModInt<mod>> cdq_convolution(const vector<ModInt<mod>> &kernel,
                                    vector<ModInt<mod>> initial)
{
    using Ntt = NttConvolution<mod, primitive>;
    using Poly = typename Ntt::Poly;
    assert(kernel.size() == initial.size());
    if (initial.empty()) return {};
    assert(kernel[0].v == 0);
    assert(initial.size() + initial.size() / 2 - 1 <= Ntt::max_size);
    int n = initial.size();
    auto solve = [&](auto &&self, int l, int r) -> void
    {
        if (r - l == 1) return;
        int m = (l + r) / 2;
        self(self, l, m);
        Poly left(initial.begin() + l, initial.begin() + m);
        Poly right(kernel.begin(), kernel.begin() + r - l);
        Poly product = Ntt::multiply(move(left), move(right));
        for (int i = m; i < r; i++) initial[i] += product[i - l];
        self(self, m, r);
    };
    solve(solve, 0, n);
    return initial;
}
```

模板题使用：Luogu P4721 分治 FFT

题意：输入 g[1..n-1]，计算 f[0]=1、f[i]=∑(j=1..i)f[i-j]g[j]，模 998244353。kernel[0] 留零，initial 仅第零项为 1；调用的是真正 CDQ 分治卷积，依赖 NTT 与模整数。

所需模板：ModInt（第 90 页，18.1.2 节）、NttConvolution（第 151 页，19.2.1 节）、cdq_convolution（第 149 页，19.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4721

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    using Z = ModInt<998244353>;
    vector<Z> g(n), b(n);
    for (int i = 1; i < n; i++)
    {
        int x;
        cin >> x;
        g[i] = x;
    }
    b[0] = 1;
    auto f = cdq_convolution(g, b);
    for (int i = 0; i < n; i++) cout << f[i].v << " \n"[i + 1 == n];
}
```

19.1.4 浮点 FFT：舍入前先约束最坏误差

前提：53 位有效精度、最近舍入、禁用 fast-math，单位根绝对误差不超过 $8u$（$u = 2^{-53}$）。长度 $N = 2^{\ell}$、两输入欧氏范数积 S 时卷积误差界

$$S\left((1+u)^{3\ell}(1+\sqrt{5}u)^{3\ell+1}(1+8u)^{3\ell}-1\right).$$

整数入口要求 $N \le 2^{22}$、$\max|a_i|\max|b_i|\sqrt{|a||b|} \le 2\cdot 10^{12}$，误差 < 0.166，用 llroundl 舍入；超限不保证正确。任意模卷积拆三个 11 位块，结果长度 $\le 2^{19}$，误差 < 0.079；模数可为 1 或合数，空输入返回空。

19.2 快速数论变换

19.2.1 可指定素数与原根的 NTT 卷积

功能：给定两个模素数 p 的多项式（系数数组）a, b，求乘积 $c_k = \sum_{i+j=k} a_i b_j \bmod p$；同时提供原地数论变换 `ntt` 供其他多项式算法复用。

核心思路：取 $p-1$ 的二次幂因子中的长度 n 与 n 次单位根 $\omega = g^{(p-1)/n}$，把系数表示变成 n 个点值：位逆序重排后自底向上做蝴蝶合并。两个点值逐项相乘后做逆变换（同一正变换后翻转下标 $1..n-1$ 并乘 n^{-1}）还原系数。单位根表按层缓存，各长度复用。

代码依赖：ModInt（第 90 页，18.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：声明 NttConvolution<mod, primitive>，primitive 默认 3。mod 必须为 int 范围内的素数，primitive 是模 mod 原根；不在接口内验证素性或阶。max_size 是 mod-1 的最大二次幂因子。ntt(a,invert) 原地变换，长度须为正二次幂且不超过 max_size；invert=true 为逆变换。multiply(a,b) 返回普通卷积，任一为空则为空；非空时长度和减一不超过 max_size。元素类型 Z，数组类型 Poly。O(n log n) 时间；每种 mod/primitive 实例缓存分层单位根，空间 O(历次最大变换长度)，同一实例调用时不并发扩容。输入输出均为自然顺序，逆变换复用正变换后翻转下标 1..n-1 并乘 n 的逆元。998244353 和 167772161 均可用原根 3。

```
template <int mod, int primitive = 3> struct NttConvolution
{
    static_assert(mod >= 2 && primitive >= 1 && primitive < mod);
    static constexpr int max_size = (mod - 1) & -(mod - 1);
    using Z = ModInt<mod>;
    using Poly = vector<Z>;

    // mod is prime, primitive is a primitive root modulo mod.
    static void ntt(Poly &a, bool invert = false)
    {
        int n = a.size();
        assert(n > 0 && (n & (n - 1)) == 0 && n <= max_size);
        static Poly roots{0, 1};
        for (int half = roots.size(); half < n; half *= 2)
        {
            Z step = Z(primitive).pow((mod - 1) / (2 * half));
            roots.resize(2 * half);
            for (int i = half / 2; i < half; i++)
            {
                roots[2 * i] = roots[i];
                roots[2 * i + 1] = roots[i] * step;
            }
        }
        for (int i = 1, j = 0; i < n; i++)
        {
            int bit = n >> 1;
            for (; j & bit; bit >>= 1)
                j ^= bit;
            j ^= bit;
            if (i < j)
                swap(a[i], a[j]);
        }
```

蝶形变换与卷积（接上页同一结构体与函数）：

```
        for (int half = 1; half < n; half *= 2)
        {
            for (int i = 0; i < n; i += 2 * half)
            {
                for (int j = 0; j < half; j++)
                {
                    Z u = a[i + j];
                    Z v = a[i + j + half] * roots[half + j];
                    a[i + j] = u + v;
                    a[i + j + half] = u - v;
                }
            }
        }
        if (invert)
        {
            reverse(a.begin() + 1, a.end());
            Z inverse = Z(n).inv();
            for (auto &x : a)
                x *= inverse;
        }
    }

    static Poly multiply(Poly a, Poly b)
    {
        if (a.empty() || b.empty())
            return {};
        assert(a.size() + b.size() - 1 <= max_size);
        int size = a.size() + b.size() - 1, n = 1;
        while (n < size)
            n *= 2;
        a.resize(n);
        b.resize(n);
        ntt(a);
        ntt(b);
        for (int i = 0; i < n; i++)
            a[i] *= b[i];
        ntt(a, true);
        a.resize(size);
        return a;
    }
};
```

模板题使用：Library Checker convolution_mod

题意：求两个多项式的卷积，模 998244353。输入为各自系数个数和升幂系数；输出 n+m-1 个系数。题目保证输入系数已经取模。

所需模板：ModInt（第 90 页，18.1.2 节）、NttConvolution（第 151 页，19.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/convolution_mod

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using C = NttConvolution<998244353>;
    int n, m;
    cin >> n >> m;
    C::Poly a(n), b(m);
    for (auto &x : a) cin >> x.v;
    for (auto &x : b) cin >> x.v;
    auto c = C::multiply(move(a), move(b));
    for (int i = 0; i < (int)c.size(); i++)
        cout << c[i].v << (i + 1 == (int)c.size() ? '\n' : ' ');
}
```

19.2.2 精确有符号整数卷积

功能：计算两个 `long long` 整数系数（可为负）多项式乘积的精确系数，要求结果系数本身落在有符号 64 位范围内。

核心思路：用三个 NTT 友好素数 p, q, r（乘积约 2^{86}）分别做模卷积（`convolution_residue`），再用 Garner 算法合并：先由模 p、q 的余数求出 $v \equiv c \pmod{pq}$，再求 u 使 $v + pq\,u \equiv c \pmod{r}$，得到 $[0, pqr)$ 中的唯一代表。它超过 $pqr/2$ 时减去 pqr 取中心代表，从而还原负系数。Garner 所需的两个逆元只计算一次。

代码依赖：ModInt（第 90 页，18.1.2 节）、NttConvolution（第 151 页，19.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：convolution_i64(a,b) 返回 signed64 整数卷积，空输入返回空。非空结果长度 ≤2^24，所有真实系数必须在 signed64 范围内。依赖 NttConvolution，三个素数模卷积后用 int128 CRT 重构中心代表。断言不能检测任意越界结果的模数别名。O(L log L) 时间、O(L) 空间，L 为补齐的二次幂长度。负数与极值已通过 cpp_int 参考验证；P3803 最大次数驱动本地通过，线上性能待验证。

```
template <int p>
vector<int> convolution_residue(const vector<long long> &a, const vector<long long> &b)
{
    using N = NttConvolution<p>;
    typename N::Poly x(a.begin(), a.end()), y(b.begin(), b.end());
    auto c = N::multiply(move(x), move(y));
    vector<int> r(c.size());
    for (int i = 0; i < (int)c.size(); i++) r[i] = c[i].v;
    return r;
}

vector<long long> convolution_i64(const vector<long long> &a,
                                  const vector<long long> &b)
{
    if (a.empty() || b.empty()) return {};
    assert(a.size() <= (1 << 24) && b.size() <= (1 << 24));
    assert(a.size() + b.size() - 1 <= (1 << 24));
    const long long p = 167772161, q = 469762049, r = 1224736769;
    const long long pq = p * q;
    const __int128 mod = (__int128)pq * r;
    static const long long ip = ModInt<q>(p).inv().v;
    static const long long ipq = ModInt<r>(pq).inv().v;
    auto x = convolution_residue<p>(a, b);
    auto y = convolution_residue<q>(a, b);
    auto z = convolution_residue<r>(a, b);
    vector<long long> c(x.size());
    for (int i = 0; i < (int)c.size(); i++)
    {
        long long t = (y[i] - (long long)x[i] + q) % q * ip % q;
        long long v = x[i] + p * t;
        long long u = (z[i] - v % r + r) % r * ipq % r;
        __int128 value = v + (__int128)pq * u;
        if (value > mod / 2) value -= mod;
        assert(LLONG_MIN <= value && value <= LLONG_MAX);
        c[i] = (long long)value;
    }
    return c;
}
```

模板题使用：Luogu P3803

题意：求两个整数多项式的乘积，系数按升幂输入。首行 n、m 是次数，系数个数分别为 n+1、m+1；输出精确整数，不能取模。本题系数在 0..9，满足 signed64 结果范围。

所需模板：ModInt（第 90 页，18.1.2 节）、NttConvolution（第 151 页，19.2.1 节）、convolution_i64（第 154 页，19.2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3803

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<long long> a(n + 1), b(m + 1);
    for (auto &x : a) cin >> x;
    for (auto &x : b) cin >> x;
    auto c = convolution_i64(a, b);
    for (int i = 0; i < (int)c.size(); i++)
        cout << c[i] << (i + 1 == (int)c.size() ? '\n' : ' ');
}
```

19.2.3 参数化 NTT 与模数适配

实现见第 151 页。传入素数 p 与原根 g；长度 n 为二次幂且整除 $p-1$，上限 $L = 2^{v_2(p-1)}$。普通卷积补齐到不小于长度和减一的二次幂。998244353 的 $L = 2^{23}$，167772161 的 $L = 2^{25}$，原根均为 3。接口不检查素性或原根的阶；模数限 int 范围内的素数，乘积用 64 位。逆变换用逆单位根并乘 n^{-1}。

19.3 快速沃尔什变换

19.3.1 按位 OR、AND、XOR 卷积

功能：给定两个长度为 2^m 的数组，求按下标位运算合并的卷积 $c_s = \sum_{i \circ j = s} a_i b_j$，其中 $\circ$ 为按位或、与、异或之一（模 `mod`）；也单独提供原地变换 `transform`。

核心思路：三种运算都能逐位独立处理：每层把下标在该位上为 0、1 的一对元素 u, v 做变换。OR 做子集和 $v \leftarrow v + u$，AND 做超集和 $u \leftarrow u + v$，逆变换改为减法；XOR 做 Walsh 变换 $(u, v) \leftarrow (u + v, u - v)$，逆变换每层再乘 $1/2$（取 $(\mathrm{mod} + 1)/2$，故只要求模数为奇数）。`multiply` 对两数组正变换、逐点相乘，再逆变换。

代码依赖：ModInt（第 90 页，18.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：SetConvolution<mod=998244353>；Z 为 ModInt<mod>，Poly 为 vector<Z>。数组长度 N 必须是正的 2 次幂且 ≤INT_MAX，表示固定二进制位数的全集，长度 1 合法。transform(a,op,invert=false) 原地变换，op 只能为 |、&、^；| 为子集和，& 为超集和，^ 为 Walsh 变换。multiply(a,b,op) 返回 $\sum$ a[i]b[j]（满足 i op j=s），a、b 长度必须相同。mod 可为 ≥2 的 int 范围合数；XOR 要求 mod 为奇数，逆 2 用 (mod+1)/2，不使用费马逆元。O(N log N) 时间，原地变换 O(1) 额外空间，卷积 O(N) 空间。

```
template <int mod = 998244353> struct SetConvolution
{
    static_assert(mod >= 2);
    using Z = ModInt<mod>;
    using Poly = vector<Z>;

    static void transform(Poly &a, char op, bool invert = false)
    {
        assert(!a.empty() && a.size() <= INT_MAX);
        int n = a.size();
        assert((n & (n - 1)) == 0);
        assert(op == '|' || op == '&' || op == '^');
        assert(op != '^' || mod % 2 == 1);
        Z half = (mod + 1LL) / 2;
        for (int len = 1; len < n; len *= 2)
            for (int i = 0; i < n; i += len * 2)
                for (int j = 0; j < len; j++)
                {
                    Z &u = a[i + j], &v = a[i + j + len];
                    if (op == '|')
                        v = invert ? v - u : v + u;
                    else if (op == '&')
                        u = invert ? u - v : u + v;
                    else
                    {
                        Z x = u, y = v;
                        u = x + y;
                        v = x - y;
                        if (invert)
                        {
                            u = u * half;
                            v = v * half;
                        }
                    }
                }
    }
```

点乘并逆变换（接上页同一结构体）：

```
    static Poly multiply(Poly a, Poly b, char op)
    {
        assert(a.size() == b.size());
        transform(a, op);
        transform(b, op);
        for (int i = 0; i < int(a.size()); i++) a[i] = a[i] * b[i];
        transform(a, op, true);
        return a;
    }
};
```

模板题使用：Luogu P4717

题意：给两个长度为 2^m 的数组，分别求按下标 OR、AND、XOR 合并的卷积，模 998244353，按此顺序输出三行。multiply 的最后参数选择运算。

所需模板：ModInt（第 90 页，18.1.2 节）、SetConvolution（第 156 页，19.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4717

```
int main()
{
    int m;
    scanf("%d", &m);
    int n = 1 << m;
    using F = SetConvolution<998244353>;
    F::Poly a(n), b(n);
    for (auto &x : a)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    for (auto &x : b)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    for (char op : string("|&^"))
    {
        auto c = F::multiply(a, b, op);
        for (int s = 0; s < n; s++) printf("%d%c", c[s].v, s + 1 == n ? '\n' : ' ');
    }
}
```

19.3.2 不相交子集卷积

功能：把下标视为集合，求不相交子集卷积 $c_S = \sum_{T \subseteq S} a_T\, b_{S \setminus T}$，即只统计 $i \cup j = S$ 且 $i \cap j = \varnothing$ 的乘积对（模 mod）。

核心思路：按集合大小（popcount，存于 rank）分层：f[k]、g[k] 只保留大小为 k 的下标，各自做 OR 变换（子集和）。$i \cup j = S$ 且 $|i| + |j| = |S|$ 恰等价于 i, j 不相交，所以对每个 k 在变换域逐点累加 $\sum_t f_t \cdot g_{k-t}$，逆 OR 变换后只取 $|S| = k$ 的位置写入答案。

代码依赖：ModInt（第 90 页，18.1.2 节）、SetConvolution（第 156 页，19.3.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：subset_convolution<mod=998244353>(a,b) 返回 c[s]=∑ a[t]b[s xor t]，求和枚举 t⊆s，等价于 i|j=s 且 i&j=0。依赖 SetConvolution，只使用 OR 变换；mod 为 ≥2 的 int 范围整数，允许偶数与合数。两个数组长度相同、非空，N=2^m≤INT_MAX；长度 1 的结果为 a[0]b[0]。按 popcount 分层，层间普通卷积再逆 OR 变换，保留总层数等于目标集合大小的项。O(m²2^m) 时间、O(m2^m) 空间；两组分层数组约占 8(m+1)2^m 字节，另有线性辅助数组。m=20 时仅分层数组约 168 MiB，使用前核对内存。

```
// c[s] = sum_{t subset s} a[t] * b[s xor t].
template <int mod = 998244353>
vector<ModInt<mod>> subset_convolution(const vector<ModInt<mod>> &a,
                                       const vector<ModInt<mod>> &b)
{
    using F = SetConvolution<mod>;
    using Poly = typename F::Poly;
    assert(!a.empty() && a.size() == b.size() && a.size() <= INT_MAX);
    int n = a.size(), m = 0;
    assert((n & (n - 1)) == 0);
    while ((1 << m) < n) m++;
    vector<Poly> f(m + 1, Poly(n)), g(m + 1, Poly(n));
    vector<int> rank(n);
    for (int s = 0; s < n; s++)
    {
        rank[s] = __builtin_popcount(unsigned(s));
        f[rank[s]][s] = a[s];
        g[rank[s]][s] = b[s];
    }
    for (int k = 0; k <= m; k++)
    {
        F::transform(f[k], '|');
        F::transform(g[k], '|');
    }
    Poly answer(n), h(n);
    for (int k = 0; k <= m; k++)
    {
        fill(h.begin(), h.end(), 0);
        for (int i = 0; i <= k; i++)
            for (int s = 0; s < n; s++) h[s] = h[s] + f[i][s] * g[k - i][s];
        F::transform(h, '|', true);
        for (int s = 0; s < n; s++)
            if (rank[s] == k) answer[s] = h[s];
    }
    return answer;
}
```

模板题使用：Luogu P6097

题意：把下标看作集合，求 c[s]=∑ a[t]*b[s⁀]，t 遍历 s 的子集，两部分不相交。输入首项为集合大小 m，数组长度 2^m；本题模数为 1000000009。

所需模板：ModInt（第 90 页，18.1.2 节）、SetConvolution（第 156 页，19.3.1 节）、subset_convolution（第 158 页，19.3.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6097

```
int main()
{
    int m;
    scanf("%d", &m);
    int n = 1 << m;
    using F = SetConvolution<1000000009>;
    F::Poly a(n), b(n);
    for (auto &x : a)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    for (auto &x : b)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    auto c = subset_convolution<1000000009>(a, b);
    for (int s = 0; s < n; s++) printf("%d%c", c[s].v, s + 1 == n ? '\n' : ' ');
}
```

19.3.3 集合变换与不相交子集卷积

代码见第 156 页与第 158 页。长度 $N = 2^m$，第 s 项对应集合 s。OR 变换 $\widehat{a}_s = \sum_{t \subseteq s} a_t$，AND 变换求超集和，逆变换改加为减。XOR 变换系数 $(-1)^{\mathrm{popcount}(s\&t)}$，蝶形 $(u, v) \to (u + v, u - v)$，逆变换每层除以 2。OR、AND 可用任意模数；XOR 需 2 可逆，本实现限奇数模数并用 $(\mathrm{mod}+1)/2$。三类卷积均为变换、逐项相乘、逆变换。

$$c_s = \sum_{t \subseteq s} a_t b_{s \setminus t} \qquad \text{（两集合不相交）}$$

按大小分层 $A_{i,s} = [|s| = i] a_s$，各层 OR 变换后 $H_{k,s} = \sum_{i=0}^{k} \widehat{A}_{i,s} \widehat{B}_{k-i,s}$，逆变换后取 $k = |s|$。时间 $O(m^2 2^m)$，空间 $O(m 2^m)$；$m = 20$ 时两组分层数组占 168 MiB。

19.3.4 集合变换：逆变换对模数的要求

OR（子集和）单层 $(u,v)\mapsto(u,u+v)$，AND（超集和）单层 $(u+v,v)$，只需加减，任意模数。XOR 单层 $(u+v,u-v)$，两次等于乘 2，要求奇数模数，逆 2 取 $(p+1)/2$。长度为正的二次幂；不同于子集卷积。

19.4 Chirp Z 变换

19.4.1 Chirp Z：等比点求值

功能：给定多项式 f 与 a,r，一次求出 f 在等比点 $a,ar,ar^2,\dots,ar^{m-1}$ 处的 m 个值（模 NTT 素数），比逐点秦九韶求值快。

核心思路：关键恒等式 $ij=\binom{i+j}{2}-\binom{i}{2}-\binom{j}{2}$，于是 $f(ar^i)=r^{-\binom{i}{2}}\sum_j\left(f_ja^jr^{-\binom{j}{2}}\right)r^{\binom{i+j}{2}}$，成为一次差卷积。`inverse[t]` 存 $r^{-\binom{t}{2}}$，三角数指数由连乘递推，不计算平方级指数。把 $f_ja^jr^{-\binom{j}{2}}$ 逆序放入 `x`，$r^{\binom{t}{2}}$ 放入 `y`，循环卷积后第 $n-1+i$ 项乘 $r^{-\binom{i}{2}}$ 即为答案；循环回绕只落在下标小于 $n-1$ 处，不影响结果。$a=0$、$r=0$ 或 $r=1$ 时点值退化为 $f(a)$ 与 $f(0)$，用秦九韶直接填充。

代码依赖：ModInt（第 90 页，18.1.2 节）、NttConvolution（第 151 页，19.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：chirp_z(f,a,r,m) 返回 f(a*r^i)，i=0..m-1；f 为升幂系数数组，a、r 为 ModInt<mod>。m≥0、f.size()≤INT_MAX；空 f 表示零多项式，m=0 返回空。约定 r^0=1，因此 r=0 时首项为 f(a)、其余为 f(0)；a=0 或 r=1 也单独处理。普通分支依赖 NTT 素数及原根，非空长度 n 要求 n+m-1≤NTT::max_size。利用三角数指数与循环卷积，避免平方指数的整数溢出；O((n+m) log(n+m)) 时间、O(n+m) 空间，特殊分支 O(n+m)。默认模板参数 mod=998244353、primitive=3。输出顺序允许重复点。

```
template <int mod = 998244353, int primitive = 3>
vector<ModInt<mod>>
chirp_z(const vector<ModInt<mod>> &f, ModInt<mod> a, ModInt<mod> r, int m)
{
    using N = NttConvolution<mod, primitive>;
    using Z = typename N::Z;
    using Poly = typename N::Poly;
    assert(m >= 0 && f.size() <= INT_MAX);
    if (m == 0 || f.empty()) return Poly(m);
    if (a.v == 0 || r.v == 0 || r.v == 1)
    {
        Poly answer(m, f[0]);
        Z value = 0;
        for (int i = int(f.size()) - 1; i >= 0; i--) value = value * a + f[i];
        answer[0] = value;
        if (r.v == 1) fill(answer.begin(), answer.end(), value);
        return answer;
    }
    assert(f.size() + m - 1 <= N::max_size);
    int n = f.size(), size = 1;
    while (size < n + m - 1) size *= 2;
    Poly x(size), y(size), inverse(max(n, m));
    Z power = 1, triangle = 1, ri = r.inv();
    for (auto &v : inverse)
    {
        v = triangle;
        triangle = triangle * power;
        power = power * ri;
    }
```

构造循环卷积并取点值（接上页同一函数）：

```
    power = 1;
    for (int j = 0; j < n; j++)
    {
        x[n - 1 - j] = f[j] * power * inverse[j];
        power = power * a;
    }
    power = 1;
    triangle = 1;
    for (int j = 0; j < n + m - 1; j++)
    {
        y[j] = triangle;
        triangle = triangle * power;
        power = power * r;
    }
    N::ntt(x);
    N::ntt(y);
    for (int j = 0; j < size; j++) x[j] = x[j] * y[j];
    N::ntt(x, true);
    Poly answer(m);
    for (int i = 0; i < m; i++) answer[i] = x[n - 1 + i] * inverse[i];
    return answer;
}
```

模板题使用：Library Checker multipoint_evaluation_on_geometric_sequence

题意：给多项式 f 的升幂系数及 a、r，依次输出 f(a*r^i)，i=0..m-1，模 998244353。r=0 时首项仍为 f(a)，后续各项为 f(0)。

所需模板：ModInt（第 90 页，18.1.2 节）、NttConvolution（第 151 页，19.2.1 节）、chirp_z（第 160 页，19.4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/multipoint_evaluation_on_geometric_sequence

```
int main()
{
    int n, m, a, r;
    scanf("%d%d%d%d", &n, &m, &a, &r);
    using Z = ModInt<998244353>;
    vector<Z> f(n);
    for (auto &x : f)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    auto answer = chirp_z(f, Z(a), Z(r), m);
    for (int i = 0; i < m; i++) printf("%d%c", answer[i].v, i + 1 == m ? '\n' : ' ');
}
```

19.5 多项式牛顿迭代

19.5.1 形式幂级数求逆

功能：给定常数项非零的形式幂级数 $a(x)$，求 $b(x)$ 使 $a(x)b(x) \equiv 1 \pmod{x^n}$，返回前 n 项（模 998244353）。

核心思路：牛顿迭代：已知 $b \bmod x^k$ 时，$b \leftarrow b - b(ab-1)$ 把精度翻倍到 $2k$，初值为 a_0^{-1}。实现只计算新增的高半段：取 a 的前 $2k$ 项 `f` 与当前 b 做长度 $2k$ 的循环卷积，真实乘积低 k 项为 $1, 0, \ldots, 0$，而回绕只污染低 k 项，于是清零低半段后剩下的就是 $ab-1$ 的有效部分；再乘一次 b（复用其频域值 `g`），第 k 到 $m-1$ 项取负即为 b 新增的系数。

代码依赖：NttConvolution（第 151 页，19.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：FpsInverse::inverse(a,n) 返回 a 的逆级数前 n 项，模数固定 998244353。要求 $1 \le n \le 2^{23}$、a 非空且 a[0]≠0；a.size() 在 int 范围内，不足项视为零，多余项只取需要前缀。继承 NttConvolution<998244353> 的 Poly、Z、multiply。Newton 公式 b←b(2-ab) 只更新新增高半段：长度 2k 循环卷积的回绕只污染低 k 项，清除低半段后再乘 b，高半段即修正项。每轮五次长度 2k 变换，复用 b 的频域值。O(n log n) 时间、O(n) 工作空间；单位根缓存另按 NTT 约定。

```
struct FpsInverse : NttConvolution<998244353>
{
    static constexpr int mod = 998244353;

    static Poly inverse(const Poly &a, int n)
    {
        assert(n > 0 && n <= max_size && !a.empty() && a[0].v);
        Poly b{a[0].inv()};
        while ((int)b.size() < n)
        {
            int k = b.size(), len = 2 * k;
            int m = min(n, len);
            Poly f(a.begin(), a.begin() + min(m, (int)a.size()));
            Poly g = b;
            f.resize(len);
            g.resize(len);
            ntt(f);
            ntt(g);
            for (int i = 0; i < len; i++)
                f[i] *= g[i];
            ntt(f, true);
            // Only the high half of a*b is free of cyclic wraparound.
            fill(f.begin(), f.begin() + k, Z(0));
            ntt(f);
            for (int i = 0; i < len; i++)
                f[i] *= g[i];
            ntt(f, true);
            b.resize(m);
            for (int i = k; i < m; i++)
                b[i] = Z(0) - f[i];
        }
        return b;
    }
};
```

模板题使用：Library Checker inv_of_formal_power_series

题意：给出常数项非零的形式幂级数，求其乘法逆的前 n 项，模 998244353。系数按升幂输入，题目保证已经取模。

所需模板：ModInt（第 90 页，18.1.2 节）、NttConvolution（第 151 页，19.2.1 节）、FpsInverse（第 162 页，19.5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/inv_of_formal_power_series

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using F = FpsInverse;
    int n;
    cin >> n;
    F::Poly a(n);
    for (auto &x : a) cin >> x.v;
    auto b = F::inverse(a, n);
    for (int i = 0; i < n; i++) cout << b[i].v << (i + 1 == n ? '\n' : ' ');
}
```

19.5.2 多项式带余除法

功能：给定多项式 a, b（$b \neq 0$），求商 q 与余式 r，使 $a = qb + r$ 且 $\deg r < \deg b$（模 998244353）；另附去掉高次零系数的 `trim`。

核心思路：设 $\deg a = n$、$\deg b = m$，则商的次数为 $n - m$。记系数反转 $a^R(x) = x^n a(1/x)$，有 $a^R = q^R b^R + x^{n-m+1}(\cdots)$，故 $q^R \equiv a^R (b^R)^{-1} \pmod{x^{n-m+1}}$。代码取 a 反转后的前 $k = n - m + 1$ 项，与 b^R 的逆级数（`inverse`）相乘、截断、再反转得到 q；最后 $r = a - qb$ 只保留低 m 项并去高次零。

代码依赖：FpsInverse（第 162 页，19.5.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：PolynomialDivision::divide(a,b) 返回 pair<Poly,Poly>{商, 余式}，模数固定 998244353，系数从低次到高次。输入按值传入；先删除高次端零系数，零多项式统一返回空 vector；b 不可为零。支持零被除式、常数除数及 deg(a)<deg(b)。返回 a=q*b+r，deg(r)<deg(b)，商和余式都去掉高次零。依赖 FpsInverse 与 NttConvolution；反转后求逆得到商，再计算余式。输入各长度 $\leq 2^{22}$；令 N=max(a.size(),b.size())，时间 O(N log(N+1))，额外空间 O(N)。trim(a) 是可直接使用的去高次零工具。P4512 的固定长度输出需在使用处 resize 补零。

```
struct PolynomialDivision : FpsInverse
{
    static void trim(Poly &a)
    {
        while (!a.empty() && a.back().v == 0) a.pop_back();
    }

    static pair<Poly, Poly> divide(Poly a, Poly b)
    {
        assert(a.size() <= max_size / 2 && b.size() <= max_size / 2);
        trim(a);
        trim(b);
        assert(!b.empty());
        if (a.size() < b.size()) return {{}, a};
        int k = a.size() - b.size() + 1;
        Poly ar(a.rbegin(), a.rbegin() + k);
        Poly br(b.rbegin(), b.rend());
        br.resize(min(k, (int)br.size()));
        auto q = multiply(ar, inverse(br, k));
        q.resize(k);
        reverse(q.begin(), q.end());
        auto product = multiply(q, b);
        for (int i = 0; i < (int)a.size(); i++) a[i] -= product[i];
        a.resize(b.size() - 1);
        trim(a);
        return {q, a};
    }
};
```

模板题使用：Luogu P4512 多项式除法

题意：输入两个多项式的次数与系数，输出商的 n-m+1 项和余式的 m 项；模板返回的商、余式均已去高次零，打印前按题目要求补零。

所需模板：ModInt（第 90 页，18.1.2 节）、NttConvolution（第 151 页，19.2.1 节）、FpsInverse（第 162 页，19.5.1 节）、PolynomialDivision（第 164 页，19.5.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4512

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    using P = PolynomialDivision;
    P::Poly a(n + 1), b(m + 1);
    for (auto &x : a)
    {
        int value;
        cin >> value;
        x = value;
    }
    for (auto &x : b)
    {
        int value;
        cin >> value;
        x = value;
    }
    auto [q, r] = P::divide(a, b);
    q.resize(n - m + 1);
    r.resize(m);
    for (auto x : q) cout << x.v << ' ';
    cout << '\n';
    for (auto x : r) cout << x.v << ' ';
    cout << '\n';
}
```

19.5.3 多项式带余除法：先反转再求逆

`PolynomialDivision::divide(a,b)` 返回规范化的商、余式，零多项式为空向量；除数去高次零后须非零。设 $A=\deg a$、$B=\deg b$、$k=A-B+1$：

$$\operatorname{rev}(q)=\operatorname{rev}(a)\,\operatorname{rev}(b)^{-1}\pmod{x^{k}},\qquad r=a-qb.$$

定长输出需自行补零。各长度 $\le 2^{22}$，$O(N\log(N+1))$ 时间、$O(N)$ 空间。

19.6 多项式多点求值 | 快速插值

19.6.1 任意点多项式多点求值

问题描述：给定模 998244353 的多项式 f 和 m 个任意点 $x_0,\dots,x_{m-1}$（可以重复），求全部 $f(x_i)$。点集固定后，同一棵树可对多个不同的 f 反复求值。

核心思路：利用 $f(x_i)=f \bmod (X-x_i)$。构造时在线段树形的乘积树上自底向上建出 p[u]$=\prod_{l\le i<r}(X-x_i)$：叶子为 $X-x_l$，内部结点用 multiply 合并左右儿子。eval 自顶向下传递余式：进入结点时，若当前多项式次数不低于 p[u] 就对它取模，再分别递归左右儿子。余式已为零多项式时直接返回（答案数组初值为 0）；只剩常数时把它填进整段点的答案，不再下探。

代码依赖：PolynomialDivision（第 164 页，19.5.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：固定模数 998244353，Poly 为升幂系数 vector<Z>。MultipointEvaluation(x) 建立点集乘积树，evaluate(f) 按输入点顺序返回 f(x[i])；允许重复点、零点、空点集、空或尾零多项式，返回长度始终等于点数。构造复制点值，查询按值接收 f，不改变树或调用者的左值，可多次复用；不再使用 f 时可 move 传入。系数个数 n≤2^22，点数 m<2^22，且内存可用；本地最大规模证据按报告列出，不表示已实测此绝对上限。每结点存首一多项式，查询将余式下传，叶值就是答案；仅求值，不包含快速插值。建树 O(m $\log^2$(m+1))，查询 O(n log(n+1)+m $\log^2$(m+1))，常驻空间 O(m log(m+1))、含临时数组 O(n+m log(m+1))，递归深度 O(log(m+1))。依赖 PolynomialDivision/FpsInverse/NTT 及模整数。

```
struct MultipointEvaluation : PolynomialDivision
{
    int n;
    vector<Poly> p;

    MultipointEvaluation(const Poly &x)
    {
        assert(x.size() < max_size / 2);
        n = x.size();
        p.resize(4 * max(1, n));
        if (n) build(1, 0, n, x);
    }

    void build(int u, int l, int r, const Poly &x)
    {
        if (r - l == 1)
        {
            p[u] = {Z(0) - x[l], 1};
            return;
        }
        int m = (l + r) / 2;
        build(2 * u, l, m, x);
        build(2 * u + 1, m, r, x);
        p[u] = multiply(p[2 * u], p[2 * u + 1]);
    }
```

余式下传与求值入口（接上页同一结构体）：

```
    void eval(int u, int l, int r, Poly f, Poly &a) const
    {
        if (f.size() >= p[u].size()) f = divide(move(f), p[u]).second;
        if (f.empty()) return;
        if (f.size() == 1)
        {
            fill(a.begin() + l, a.begin() + r, f[0]);
            return;
        }
        int m = (l + r) / 2;
        eval(2 * u, l, m, f, a);
        eval(2 * u + 1, m, r, move(f), a);
    }

    Poly evaluate(Poly f) const
    {
        assert(f.size() <= max_size / 2);
        trim(f);
        Poly a(n);
        if (n) eval(1, 0, n, move(f), a);
        return a;
    }
};
```

模板题使用：Library Checker Multipoint Evaluation

题意：给出 n 项升幂系数和 m 个任意横坐标，按输入顺序输出全部 f(x[i])，模 998244353。1≤n,m≤131072，系数和点均已取模，最高次系数非零；横坐标可以重复。构造一次乘积树后调用 evaluate；同一树可以继续求其他多项式的点值。

所需模板：ModInt（第 90 页，18.1.2 节）、NttConvolution（第 151 页，19.2.1 节）、FpsInverse（第 162 页，19.5.1 节）、PolynomialDivision（第 164 页，19.5.2 节）、MultipointEvaluation（第 166 页，19.6.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/multipoint_evaluation

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    using P = MultipointEvaluation;
    P::Poly f(n), x(m);
    for (auto &a : f)
    {
        int v;
        cin >> v;
        a = v;
    }
    for (auto &a : x)
    {
        int v;
        cin >> v;
        a = v;
    }
    auto a = P(x).evaluate(f);
    for (int i = 0; i < m; i++) cout << a[i].v << (i + 1 == m ? '\n' : ' ');
}
```

19.6.2 任意点多项式快速插值

问题描述：给定 n 个横坐标互异的点 (x_i, y_i)，在模 998244353 下求次数小于 n 的唯一多项式 f，使 $f(x_i) = y_i$，返回其系数。

核心思路：Lagrange 插值写成

$$f(X) = \sum_i \frac{y_i}{P'(x_i)} \cdot \frac{P(X)}{X - x_i},$$

其中 $P(X) = \prod_i (X - x_i)$。先用 `MultipointEvaluation` 建乘积树，根 `tree.p[1]` 就是 P；逐项求导得 P'，多点求值得到各 $P'(x_i)$（出现 0 说明横坐标重复），权重 $w_i = y_i / P'(x_i)$。然后在同一棵树上自底向上合并：叶子返回常数 w_l，内部结点返回 “左半结果乘右儿子乘积，加右半结果乘左儿子乘积”，根处即为 f。

代码依赖：MultipointEvaluation（第 166 页，19.6.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：polynomial_interpolation(x,y) 返回模 998244353 下次数 <m 的唯一插值多项式，升幂 vector 恰好 m 项并保留高次零。optional 为空表示长度不符或归一化横坐标重复；两个空数组返回有值的空 vector，不改输入。复用 MultipointEvaluation 乘积树及导数求值，按 y/P'(x) 递归合并；重复点只在独立求值接口合法。m≤$2^{22}-1$ 是既有除法/NTT 的统一充分界，官方最大实测 131072。时间 O(m $\log^2$(m+1)+m log p)，p=998244353；单点逆元沿用模幂，不省略 m log p 项。总空间 O(m log(m+1)+1)，递归 O(log(m+1))。

```
optional<MultipointEvaluation::Poly>
polynomial_interpolation(const MultipointEvaluation::Poly &x,
                         const MultipointEvaluation::Poly &y)
{
    using P = MultipointEvaluation;
    using Poly = P::Poly;
    if (x.size() != y.size()) return nullopt;
    assert(x.size() < P::max_size / 2);
    int n = x.size();
    if (!n) return Poly{};
    const P tree(x);
    Poly d(n);
    for (int i = 1; i <= n; i++) d[i - 1] = tree.p[1][i] * i;
    auto w = tree.evaluate(move(d));
    for (auto v : w)
        if (!v.v) return nullopt;
    for (int i = 0; i < n; i++) w[i] = y[i] * w[i].inv();
    auto combine = [&](auto &&self, int u, int l, int r) -> Poly
    {
        if (r - l == 1) return {w[l]};
        int m = (l + r) / 2;
        auto a = self(self, 2 * u, l, m);
        auto b = self(self, 2 * u + 1, m, r);
        a = P::multiply(move(a), tree.p[2 * u + 1]);
        b = P::multiply(move(b), tree.p[2 * u]);
        for (int i = 0; i < r - l; i++) a[i] += b[i];
        return a;
    };
    return combine(combine, 1, 0, n);
}
```

模板题使用：Library Checker Polynomial Interpolation

题意：给出 n 个互异横坐标及对应点值，输出唯一的次数小于 n 的多项式的 n 项升幂系数，模 998244353。1≤n≤131072，坐标与点值均已取模；先以乘积根多项式的导数求权重，再递归合并。接口返回 optional，长度不符或模意义重复横坐标返回空 optional，空输入返回有值的空向量；本题保证输入合法。

所需模板：ModInt（第 90 页，18.1.2 节）、NttConvolution（第 151 页，19.2.1 节）、FpsInverse（第 162 页，19.5.1 节）、PolynomialDivision（第 164 页，19.5.2 节）、MultipointEvaluation（第 166 页，19.6.1 节）、polynomial_interpolation（第 169 页，19.6.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/polynomial_interpolation

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    using Poly = MultipointEvaluation::Poly;
    Poly x(n), y(n);
    for (auto &a : x)
    {
        int v;
        cin >> v;
        a = v;
    }
    for (auto &a : y)
    {
        int v;
        cin >> v;
        a = v;
    }
    auto f = polynomial_interpolation(x, y);
    assert(f);
    for (int i = 0; i < n; i++) cout << (*f)[i].v << (i + 1 == n ? '\n' : ' ');
}
```

19.7 多项式初等函数

19.7.1 形式幂级数开根（含前导零与无解）

功能： 给定形式幂级数 $a(x)$，求 $b(x)$ 使 $b^2 \equiv a \pmod{x^n}$ 的前 n 项（模 998244353），无解时报告；`inverse_sqrt` 求 $a^{-1/2}$ 的前 n 项。

核心思路： 先求逆平方根：常数项用 `mod_sqrt` 取一个模平方根再求逆，之后按牛顿迭代 $b \leftarrow (3b - ab^3)/2$ 倍增。已有 k 项时，在长度 $4k$ 下一次算出 ab^3，回绕只落到低 k 项；新 b 的低 k 项不变，高半段等于 $-\frac{1}{2}(ab^3)$ 的对应项。`sqrt` 先找首个非零项次数 k：k 为奇数或该系数不是二次剩余则无解；否则除掉 x^k 得 `f`，用 $\sqrt{f} = f \cdot f^{-1/2}$ 求根，最后在前面补 $k/2$ 个零。

代码依赖：FpsInverse（第 162 页，19.5.1 节）、mod_sqrt（第 121 页，18.14.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约： FpsSqrt::sqrt(a,n) 返回 optional<Poly>，模 998244353；无解为 nullopt，有解恰好 n 项。$0 \le n \le 2^{22}$、a.size() 在 int 范围内，仅看前 n 项，不足补零；n=0 返回有值空 vector。全零返回全零，首个非零项次数 k 为奇数或系数非二次剩余时无解。否则提出 x^k，首项选较小模平方根，末尾未确定的 k/2 项取零。inverse_sqrt(a,n) 直接求所选根的逆：n>0 时要求常数项非零且为二次剩余，否则 nullopt；n=0 仍返回空。按 $b \leftarrow (3b\text{-}ab^3)/2$ 倍增，长度 4k 的循环卷积只回绕到低 k 项，保留旧低半段、更新高半段；sqrt 再乘原级数并移位。不枚举全部根。依赖 mod_sqrt、FpsInverse、NTT；O(n log(n+1)) 时间加一次标量开根的期望 O(log mod)，O(n) 工作空间，单位根缓存另按 NTT 约定。P5205 的 a[0]=1 得到首项 1。

```
struct FpsSqrt : FpsInverse
{
    static optional<Poly> inverse_sqrt(const Poly &a, int n)
    {
        assert(n >= 0 && n <= max_size / 2);
        if (n == 0)
            return Poly{};
        if (a.empty() || a[0].v == 0)
            return nullopt;
        auto roots = mod_sqrt(a[0].v, mod);
        if (roots.empty())
            return nullopt;
        Poly b{Z(roots[0]).inv()};
        Z half = Z(2).inv();
        while ((int)b.size() < n)
        {
            int k = b.size(), len = 4 * k;
            int m = min(n, 2 * k);
            Poly f(a.begin(), a.begin() + min(m, (int)a.size()));
            Poly g = b;
            f.resize(len);
            g.resize(len);
            ntt(f);
            ntt(g);
            for (int i = 0; i < len; i++)
                f[i] *= g[i] * g[i] * g[i];
            ntt(f, true);
            // b <- (3*b - a*b^3)/2; cyclic wrap only reaches below k.
            b.resize(m);
            for (int i = k; i < m; i++)
                b[i] = Z(0) - f[i] * half;
        }
        return b;
    }
```

平方根与前导零处理（接上页同一结构体）：

```
    static optional<Poly> sqrt(const Poly &a, int n)
    {
        assert(n >= 0 && n <= max_size / 2);
        int k = 0;
        while (k < n && (k >= (int)a.size() || a[k].v == 0))
            k++;
        if (k == n)
            return Poly(n);
        if (k & 1)
            return nullopt;
        int target = n - k;
        Poly f(a.begin() + k, a.begin() + min(n, (int)a.size()));
        auto inverse = inverse_sqrt(f, target);
        if (!inverse)
            return nullopt;
        Poly b = multiply(f, *inverse);
        b.resize(target);
        b.insert(b.begin(), k / 2, Z(0));
        b.resize(n);
        return b;
    }
};
```

模板题使用：Luogu P5205 多项式开根

题意：输入 n 项、保证 a[0]=1，输出平方根的前 n 项，首项取较小的 1。通用接口也可用于无解输入，须先检查 optional。

所需模板：ModInt（第 90 页，18.1.2 节）、NttConvolution（第 151 页，19.2.1 节）、FpsInverse（第 162 页，19.5.1 节）、mod_sqrt（第 121 页，18.14.1 节）、FpsSqrt（第 171 页，19.7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5205

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    FpsSqrt::Poly a(n);
    for (auto &x : a)
    {
        int value;
        cin >> value;
        x = value;
    }
    auto b = FpsSqrt::sqrt(a, n);
    if (!b)
    {
        cout << -1 << '\n';
        return 0;
    }
    for (auto x : *b) cout << x.v << ' ';
    cout << '\n';
}
```

19.7.2 形式幂级数微积分、对数与指数

功能： 形式幂级数的基本运算：逐项求导 `derivative`、常数项为零的积分 `integral`、常数项为 1 时的对数 $\ln a$，以及常数项为 0 时的指数 $\exp a$，后两者返回前 n 项（模 998244353）。

核心思路： 积分所需的 $1/i$ 用线性递推 $i^{-1} = -\lfloor p/i \rfloor \cdot (p \bmod i)^{-1}$ 求出。$\ln a = \int a'/a$：截取前 n 项，导数乘逆级数（`inverse`）后积分。exp 用牛顿迭代 $b \leftarrow b(1 - \ln b + a)$，从 $b = 1$ 开始每轮精度翻倍：求 $\ln b$ 的前 m 项，构造 $a - \ln b + 1$，再与 b 相乘并截断到 m 项。

代码依赖：FpsInverse（第 162 页，19.5.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约： FpsFunctions 继承 FpsInverse，复用同一 NTT 和求逆。derivative(a) 形式求导；integral(a) 返回常数项为零的积分，要求 a.size()<998244353。log(a,n) 要求 n>0、a 非空、a[0]=1；exp(a,n) 要求 n>0、a 为空或 a[0]=0。返回前 n 项，不足输入项视为零，超过 n 的尾项不参与计算。内部每次卷积结果长度须 $\le 2^{23}$；$n \le 2^{22}$ 是统一充分界，输入数组长度须在 int 范围内。微积分 O(a.size())；log/exp 为 O(n log(n+1)) 时间和 O(n) 辅助空间。

```
struct FpsFunctions : FpsInverse
{
    static Poly derivative(const Poly &a)
    {
        Poly b;
        for (int i = 1; i < (int)a.size(); i++) b.push_back(a[i] * i);
        return b;
    }

    static Poly integral(const Poly &a)
    {
        assert(a.size() < mod);
        Poly b(a.size() + 1);
        vector<Z> inv(a.size() + 1);
        if (!a.empty()) inv[1] = 1;
        for (int i = 2; i <= (int)a.size(); i++)
            inv[i] = Z(mod - mod / i) * inv[mod % i];
        for (int i = 0; i < (int)a.size(); i++) b[i + 1] = a[i] * inv[i + 1];
        return b;
    }

    static Poly log(const Poly &a, int n)
    {
        assert(n > 0 && !a.empty() && a[0].v == 1);
        Poly f(a.begin(), a.begin() + min(n, (int)a.size()));
        auto b = multiply(derivative(f), inverse(f, n));
        b.resize(n - 1);
        return integral(b);
    }
```

```
    static Poly exp(const Poly &a, int n)
    {
        assert(n > 0 && (a.empty() || a[0].v == 0));
        Poly b{1};
        while ((int)b.size() < n)
        {
            int m = min(n, 2 * (int)b.size());
            auto t = log(b, m);
            for (int i = 0; i < m; i++) t[i] = (i < (int)a.size() ? a[i] : Z(0)) - t[i];
            t[0] = t[0] + 1;
            b = multiply(b, t);
            b.resize(m);
        }
        return b;
    }
};
```

模板题使用：Library Checker log_of_formal_power_series

题意：给定常数项为 1 的形式幂级数，求其形式对数的前 n 项，模 998244353。系数按升幂输入且已取模，结果常数项为 0。

所需模板：ModInt（第 90 页，18.1.2 节）、NttConvolution（第 151 页，19.2.1 节）、FpsInverse（第 162 页，19.5.1 节）、FpsFunctions（第 174 页，19.7.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/log_of_formal_power_series

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using F = FpsFunctions;
    int n;
    cin >> n;
    F::Poly a(n);
    for (auto &x : a) cin >> x.v;
    auto b = F::log(a, n);
    for (int i = 0; i < n; i++) cout << b[i].v << (i + 1 == n ? '\n' : ' ');
}
```

19.7.3 形式幂级数快速幂（十进制大指数）

功能：给定形式幂级数 $a(x)$ 与以十进制字符串给出的非负整数 k（可能极大），求 $a(x)^k \bmod x^n$ 的前 n 项（模 998244353）。

核心思路：设首个非零项为 $c\,x^s$，则 $a = c\,x^s\,b$ 且 b 的常数项为 1，$a^k = c^k x^{sk} \exp(k \ln b)$。读指数时同时保留 $k \bmod p$（乘进 $\ln b$）、$k \bmod (p-1)$（`phi`，由费马小定理用于 c^k）以及截到 n 的真实值 `cap`（判断位移 sk 是否越界）。若 $sk \geq n$ 结果全零；否则只需 b 的前 $n - sk$ 项，做 log、乘 k、exp、乘 c^k 后平移 sk 位。$k = 0$ 时直接返回 1。

思路中的情况	代码中的处理	行
读入大指数	逐位维护 $k \bmod p$、$k \bmod (p-1)$ 及饱和于 n 的 `cap`	14
指数为 0	返回常数 1	18
位移 $sk \geq n$ 或级数全零	直接返回全零	25
提出 $c\,x^s$	截取 b 并整体乘 c^{-1}，使常数项为 1	29
计算 $\exp(k \ln b)$	对数乘 $k \bmod p$ 后取指数	32
乘回 $c^k x^{sk}$	乘 c^{phi} 并写到第 `shift` 项之后	34

代码依赖：FpsFunctions（第 174 页，19.7.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：FpsPower::power(a,exponent,n) 返回 a 的非负整数次幂模 x^n 的前 n 项，固定模数 998244353。exponent 为非空十进制数字 string_view，允许前导零；整数指数可传 to_string(k)。$0 \leq n \leq 2^{22}$，a.size() 须在 int 范围内；短输入补零、长输入只看前 n 项。n=0 返回空，零次幂（含零多项式的零次幂）返回 1。对首个非零项次数 s，若真实 s*k≥n 则返回全零；否则提取常数 c，计算 x^(s*k)*c^k*exp(k*log(a/(c*x^s)))。指数分别保留模 p、模 p-1 和截到 n 的值；不可仅用 k mod p 判零或算位移。依赖 FpsFunctions、FpsInverse、NTT。指数位数 L，时间 O(L+n log(n+1))、额外空间 O(n)，不支持负指数或小数指数。

```
struct FpsPower : FpsFunctions
{
    static Poly power(const Poly &a, string_view exponent, int n)
    {
        assert(n >= 0 && n <= max_size / 2 && !exponent.empty());
        long long k = 0, phi = 0;
        int cap = 0;
        for (char c : exponent)
        {
            assert('0' <= c && c <= '9');
            int digit = c - '0';
            k = (k * 10 + digit) % mod;
            phi = (phi * 10 + digit) % (mod - 1);
            cap = min(n, cap * 10 + digit);
        }
        Poly result(n);
        if (n == 0) return result;
        if (cap == 0)
        {
            result[0] = 1;
            return result;
        }
        int first = 0;
        while (first < n && (first >= (int)a.size() || a[first].v == 0)) first++;
        if (first == n || (first && cap > (n - 1) / first)) return result;
        int shift = first * cap, m = n - shift;
        Poly b(a.begin() + first, a.begin() + min(first + m, (int)a.size()));
        Z leading = a[first], inverse_leading = leading.inv();
        for (auto &x : b) x *= inverse_leading;
        auto f = log(b, m);
        for (auto &x : f) x *= Z(k);
        auto t = exp(f, m);
        Z scale = leading.pow(phi);
        for (int i = 0; i < m; i++) result[i + shift] = t[i] * scale;
        return result;
    }
};
```

模板题使用：Luogu P5273 多项式幂函数（加强版）

题意：输入 n 和十进制大指数 k，输出 a^k 的前 n 项；k 必须用 string 读入，不能只读 k mod p。允许前导零和 k=0。本程序同样适用于 P5245（a[0]=1）及 Library Checker Pow of Formal Power Series，三个入口均已本地验证。

所需模板：ModInt（第 90 页，18.1.2 节）、NttConvolution（第 151 页，19.2.1 节）、FpsInverse（第 162 页，19.5.1 节）、FpsFunctions（第 174 页，19.7.2 节）、FpsPower（第 176 页，19.7.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5273

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    string k;
    cin >> n >> k;
    FpsPower::Poly a(n);
    for (auto &x : a)
    {
        int value;
        cin >> value;
        x = value;
    }
    auto b = FpsPower::power(a, k, n);
    for (auto x : b) cout << x.v << ' ';
    cout << '\n';
}
```

19.7.4 级数开根：前导零和自由尾项

`FpsSqrt::sqrt(a,n)` 只用前 n 项。全零返回全零；首个非零项次数 k 为奇数或其系数非二次剩余则无解。否则 $a = x^k h$，迭代 $b \leftarrow (b + h/b)/2$ 求 h 的根前 $n-k$ 项后左移 $k/2$；末尾 $k/2$ 个自由系数取零，故解一般不止两个。返回 optional，$n = 0$ 返回空向量。$n \le 2^{22}$，$O(n \log(n+1))$ 加一次模开根，空间 $O(n)$。

19.7.5 ln 与 exp：只处理所需前缀

log 要求 $a_0 = 1$，$(\log a)' = a'/a$；exp 要求 $a_0 = 0$，$b \leftarrow b(1 + a - \log b)$ 倍增。对数输入先截到 n 项。积分要求 $|a| < 998244353$。$n \le 2^{22}$，$O(n \log(n+1))$ 时间、$O(n)$ 空间。

19.7.6 幂函数的大指数：保留三份指数信息

`FpsPower::power(a,exponent,n)` 的指数为十进制字符串，求模 x^n 的非负整数次幂；$n = 0$ 返回空，零次幂返回 1（含 0^0）。$k > 0$ 时设 $a = x^s ch$、$h(0) = 1$、$c \neq 0$；全零或 $sk \ge n$ 时全零，否则

$$a^k = x^{sk} c^k \exp(k \log h) \pmod{x^n}.$$

$k \bmod p$ 用于对数系数，$k \bmod (p-1)$ 用于 c^k，$\min(k, n)$ 用于零次幂与位移（要求 $n \le 2^{22} < p$）。判 $sk \ge n$ 用 $k > (n-1)/s$ 防溢出。指数位数 L，$O(L + n \log(n+1))$ 时间、$O(n)$ 空间。

19.8 常系数齐次线性递推

19.8.1 特征多项式求线性递推第 k 项

问题描述：已知 k 阶常系数齐次线性递推 $a_i = \sum_{j=1}^{k} c_{j-1} a_{i-j}$ 及初值 $a_0, \ldots, a_{k-1}$，求第 n 项（n 很大），模数可为合数。

核心思路：设特征多项式 $P(x) = x^k - \sum_{j=1}^{k} c_{j-1} x^{k-j}$。把 a_i 看作单项式 x^i 的线性函数，递推说明 x^i 可以按 P 降次而值不变，故若 $x^n \equiv \sum_{i<k} r_i x^i \pmod P$，则 $a_n = \sum_i r_i a_i$。`multiply` 做朴素多项式乘法后，从最高次往下用 $x^i = \sum_j c_{j-1} x^{i-j}$ 消去次数不小于 k 的项；以此快速幂求 $x^n \bmod P$（$k = 1$ 时 x 已降为常数 c_0），最后与初值点乘。

接口契约：recurrence_nth(init,c,n,p) 使用 0-based 下标，n 为 unsigned long long。init 与 c 长度均为阶数 k；a[i]=∑ c[j-1]a[i-j]。2≤p≤INT_MAX，数值均在 [0,p)，本函数允许合数模数。空系数约定全零序列。计算 x^n 对特征多项式取余再与初值点乘，O(k² log(n+1)) 时间、O(k) 空间；不依赖 BM 或其他模板。较大阶数可使用本库 BostanMori::recurrence，须满足其 NTT 素数模数与容量条件；本函数允许合数模数。

```
// init has exactly k entries; c[j-1] multiplies a[n-j]. Modulus need not be prime.
int recurrence_nth(const vector<int> &init,
                   const vector<int> &c,
                   unsigned long long n,
                   int p = 998244353)
{
    assert(p >= 2 && init.size() == c.size());
    int k = c.size();
    if (!k) return 0;
    for (int x : init) assert(0 <= x && x < p);
    for (int x : c) assert(0 <= x && x < p);
    if (n < (unsigned)k) return init[n];
    auto multiply = [&](const vector<int> &a, const vector<int> &b)
    {
        vector<int> t(2 * k - 1);
        for (int i = 0; i < k; i++)
        {
            for (int j = 0; j < k; j++) t[i + j] = (t[i + j] + 1LL * a[i] * b[j]) % p;
        }
        for (int i = 2 * k - 2; i >= k; i--)
        {
            for (int j = 1; j <= k; j++)
                t[i - j] = (t[i - j] + 1LL * t[i] * c[j - 1]) % p;
        }
        t.resize(k);
        return t;
    };
    vector<int> answer(k), x(k);
    answer[0] = 1;
    if (k == 1)
        x[0] = c[0];
    else
        x[1] = 1;
    while (n)
```

```
    {
        if (n & 1) answer = multiply(answer, x);
        x = multiply(x, x);
        n >>= 1;
    }
    long long value = 0;
    for (int i = 0; i < k; i++) value = (value + 1LL * answer[i] * init[i]) % p;
    return value;
}
```

模板题使用：Luogu P5487

题意：给出 n 个初值，恢复唯一的最短递推并求第 m 项，n≤10000、n<m≤10^9、阶数 ≤5000，模 998244353。BM 返回 c[j-1] 乘 a[i-j]，首行仅输出系数、不带阶数；裁取恰好 c.size() 个初值后调用 recurrence_nth。全零序列空递推仍输出空首行和第二行 0。复杂度 O(nk+k^2log m)，O(n+k) 空间。

所需模板：berlekamp_massey（第 259 页，26.1 节）、recurrence_nth（第 179 页，19.8.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5487

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    unsigned long long m;
    cin >> n >> m;
    vector<int> s(n);
    for (auto &x : s)
    {
        cin >> x;
        x %= 998244353;
        if (x < 0) x += 998244353;
    }
    auto c = berlekamp_massey(s);
    for (int i = 0; i < (int)c.size(); i++)
    {
        if (i) cout << ' ';
        cout << c[i];
    }
    cout << '\n';
    vector<int> init(s.begin(), s.begin() + c.size());
    cout << recurrence_nth(init, c, m) << '\n';
}
```

19.8.2 快速递推与有理生成函数单项

问题描述：求有理生成函数的单项系数 $[x^n]\frac{p(x)}{q(x)}$；以及给定 k 阶常系数齐次线性递推与初值，求第 n 项（模 NTT 素数）。

核心思路：分子分母同乘 $q(-x)$：$\frac{p(x)}{q(x)}=\frac{p(x)q(-x)}{q(x)q(-x)}$，新分母只含偶次项，记为 $V(x^2)$，所以只需分子中次数与 n 同奇偶的项，记为 $x^{n \bmod 2}U(x^2)$，于是 $[x^n]\frac{p}{q}=[x^{\lfloor n/2\rfloor}]\frac{U}{V}$。`coefficient` 每轮两次 NTT 乘法后抽取奇偶项、$n$ 减半，直到 $n=0$ 时答案为 p_0/q_0。`recurrence` 取 $q=1-\sum_i c_i x^{i+1}$、$p=(\text{init}\cdot q) \bmod x^k$，转化为上述问题。

代码依赖：NttConvolution（第 151 页，19.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约： BostanMori<mod=998244353,primitive=3>，依赖参数化 NTT；素数模数与原根条件同 NttConvolution。Poly 为升幂系数 vector<Z>。coefficient(p,q,n) 返回 [x^n]p(x)/q(x)，q 非空且 q[0]≠0，p 可空或次数不小于 q；令 d=max(p.size(),q.size())，要求 2d-1≤NTT::max_size。recurrence(init,c,n) 返回 a[n]，从下标 k 起 a[i]=c[0]a[i-1]+⋯+c[k-1]a[i-k]，init、c 长度均为 k；k=0 约定全零。非初值查询要求 2k+1≤NTT::max_size。n 支持完整 unsigned long long。每轮两次卷积后抽取奇偶项，O(d log d log(n+1)) 时间、O(d) 空间；递推接口 d=k+1。

```
template <int mod = 998244353, int primitive = 3> struct BostanMori
{
    using N = NttConvolution<mod, primitive>;
    using Z = typename N::Z;
    using Poly = typename N::Poly;
    using U = unsigned long long;

    // Coefficient of x^n in p(x) / q(x), with q[0] != 0.
    static Z coefficient(Poly p, Poly q, U n)
    {
        assert(!q.empty() && q[0].v != 0);
        assert(max(p.size(), q.size()) <= (N::max_size + 1ULL) / 2);
        while (n && !p.empty())
        {
            Poly r = q;
            for (int i = 1; i < int(r.size()); i += 2) r[i] = Z(0) - r[i];
            Poly a = N::multiply(p, r);
            Poly b = N::multiply(q, r);
            p.clear();
            q.clear();
            for (int i = n & 1; i < int(a.size()); i += 2) p.push_back(a[i]);
            for (int i = 0; i < int(b.size()); i += 2) q.push_back(b[i]);
            n >>= 1;
        }
        return p.empty() ? Z(0) : p[0] / q[0];
    }
```

从初值构造递推生成函数（接上页同一结构体）：

```
    // a[i] = c[0]*a[i-1] + ... + c[k-1]*a[i-k], for i >= k.
    static Z recurrence(const Poly &init, const Poly &c, U n)
    {
        assert(init.size() == c.size());
        int k = c.size();
        if (k == 0) return 0;
        if (n < init.size()) return init[n];
        assert(k + 1ULL <= (N::max_size + 1ULL) / 2);
        Poly q(k + 1);
        q[0] = 1;
        for (int i = 0; i < k; i++) q[i + 1] = Z(0) - c[i];
        Poly p = N::multiply(init, q);
        p.resize(k);
        return coefficient(p, q, n);
    }
};
```

模板题使用：Luogu P4723

题意：给 k 个递推系数和 a[0..k-1]，求 a[n] 模 998244353。c[0] 乘前一项，c[k-1] 乘前 k 项；输入先系数后初值，recurrence 参数先初值后系数。通过 ModInt 赋值归一化负数。

所需模板：ModInt（第 90 页，18.1.2 节）、NttConvolution（第 151 页，19.2.1 节）、BostanMori（第 180 页，19.8.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4723

```
int main()
{
    unsigned long long n;
    int k;
    scanf("%llu%d", &n, &k);
    using B = BostanMori<>;
    B::Poly c(k), init(k);
    for (auto &x : c)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    for (auto &x : init)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    printf("%d\n", B::recurrence(init, c, n).v);
}
```

19.8.3 线性递推：系数方向与外推条件

约定 $a_i = \sum_{j=1}^{k} c_{j-1} a_{i-j}$，初值 $a_0, \ldots, a_{k-1}$，特征多项式

$$F(x) = x^k - c_0 x^{k-1} - c_1 x^{k-2} - \cdots - c_{k-1}.$$

若 $x^n \bmod F(x) = \sum_{i=0}^{k-1} r_i x^i$，则 $a_n = \sum_{i=0}^{k-1} r_i a_i$。BM 要求素数模数；已知阶数至多 d 时至少给出前 $2d$ 项。系数末尾的零不能删除，否则改变递推开始适用的位置。BM 加特征多项式求余求第 m 项：时间 $O(nk + k^2 \log m)$，空间 $O(n + k)$。

19.8.4 Bostan–Mori 与快速递推单项

代码见第 180 页。对 $a_i = \sum_{j=1}^{k} c_{j-1} a_{i-j}$，令 $Q(x) = 1 - \sum_{j=1}^{k} c_{j-1} x^j$，$P(x) = (Q(x) \sum_{i=0}^{k-1} a_i x^i) \bmod x^k$，则 $a_n = [x^n] P/Q$。系数按前一项、前两项顺序给出，初值从 a_0 开始。

$$\frac{P(x)}{Q(x)} = \frac{P(x)Q(-x)}{Q(x)Q(-x)} = \frac{U_0(x^2) + xU_1(x^2)}{V(x^2)},$$

保留与 n 同奇偶的分子项、分母偶数项，n 右移一位；$n = 0$ 时答案为 $P(0)/Q(0)$。分母常数项须非零；不要求真分式，空分子返回零。初始最大长度 d 须满足 $2d - 1$ 不超过 NTT 容量。时间 $O(d \log d \log(n + 1))$，空间 $O(d)$；下标可达完整无符号 64 位。

19.9 多项式平移 | 连续点值平移

19.9.1 多项式系数平移

功能：给定多项式 $f(x)$ 的升幂系数和常数 c，求 $f(x+c)$ 的升幂系数（模 NTT 素数）。

核心思路：二项式展开得 $[x^j]f(x+c)=\frac{1}{j!}\sum_{i\geq j} f_i\, i!\cdot\frac{c^{i-j}}{(i-j)!}$，右边是关于差 $i-j$ 的卷积。把 $f_i\, i!$ 逆序放进 a（存在 a[n-1-i]），把 $c^t/t!$ 放进 b，一次 NTT 乘法后第 $n-1-j$ 项正是上式的求和部分，再乘 $1/j!$ 即得答案。阶乘与逆阶乘先线性预处理。

代码依赖：ModInt（第 90 页，18.1.2 节）、NttConvolution（第 151 页，19.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：polynomial_shift<mod=998244353,primitive=3>(f,c) 返回 f(x+c) 的升幂系数，结果长度始终与 f 相同，保留末尾零；空输入返回空。参数 c 使用 ModInt<mod>。依赖参数化 NTT，mod 为素数且 primitive 为原根；非空长度 n 要求 2n-1≤NTT::max_size，因此阶乘均可逆。逆序的 f[i]i! 与 c^i/i! 卷积后提取对应项，O(n log n) 时间、O(n) 空间。c=0 合法。

```
template <int mod = 998244353, int primitive = 3>
vector<ModInt<mod>> polynomial_shift(const vector<ModInt<mod>> &f, ModInt<mod> c)
{
    using N = NttConvolution<mod, primitive>;
    using Z = typename N::Z;
    using Poly = typename N::Poly;
    if (f.empty()) return {};
    assert(f.size() <= (N::max_size + 1ULL) / 2);
    int n = f.size();
    Poly fact(n), inv(n), a(n), b(n);
    fact[0] = 1;
    for (int i = 1; i < n; i++) fact[i] = fact[i - 1] * i;
    inv[n - 1] = fact[n - 1].inv();
    for (int i = n - 1; i > 0; i--) inv[i - 1] = inv[i] * i;
    Z power = 1;
    for (int i = 0; i < n; i++)
    {
        a[n - 1 - i] = f[i] * fact[i];
        b[i] = power * inv[i];
        power = power * c;
    }
    auto product = N::multiply(a, b);
    for (int i = 0; i < n; i++) a[i] = product[n - 1 - i] * inv[i];
    return a;
}
```

模板题使用：Library Checker polynomial_taylor_shift

题意：给多项式 f 的 n 个升幂系数和常数 c，输出 f(x+c) 的 n 个升幂系数，模 998244353。平移方向为加 c，c=0 时原样返回。

所需模板：ModInt（第 90 页，18.1.2 节）、NttConvolution（第 151 页，19.2.1 节）、polynomial_shift（第 184 页，19.9.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/polynomial_taylor_shift

```
int main()
{
    int n, c;
    scanf("%d%d", &n, &c);
    using Z = ModInt<998244353>;
    vector<Z> f(n);
    for (auto &x : f)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    auto answer = polynomial_shift(f, Z(c));
    for (int i = 0; i < n; i++) printf("%d%c", answer[i].v, i + 1 == n ? '\n' : ' ');
}
```

19.9.2 Taylor 平移与等比点求值

代码见第 184 页和第 160 页。升幂系数，长度 n 表示次数小于 n。

$$[x^i]f(x+c)=\frac{1}{i!}\sum_{j=i}^{n-1}f_j j!\frac{c^{j-i}}{(j-i)!}.$$

倒序 $f_j j!$ 与 $c^j/j!$ 卷积，取第 $n-1-i$ 项乘 $1/i!$。要求 NTT 素数且 $2n-1$ 不超过变换容量；空数组返回空数组。时间 $O(n\log n)$、空间 $O(n)$。这不是点值平移。

Chirp Z 返回 $f(ar^i)$，$0\le i<m$。设 $T_j=j(j-1)/2$，

$$f(ar^i)=r^{-T_i}\sum_{j=0}^{n-1}f_j a^j r^{-T_j}r^{T_{i+j}}.$$

倒序第一项数组与 r^{T_j} 卷积，取第 $n-1+i$ 项；循环卷积长度取不小于 $n+m-1$ 的二次幂即可。要求 $n+m-1$ 不超过 NTT 容量，时间 $O((n+m)\log(n+m))$、空间 $O(n+m)$。$r=0$（约定 $r^0=1$）、$a=0$、$r=1$ 单独处理；$m=0$ 返回空，空输入返回 m 个零。

19.10 符号化方法

19.10.1 生成函数建模到模板：三个可核验例子

记 N 为所求最大次数、$L=N+1$、$M(L)=O(L\log L)$ 为 NTT 乘法复杂度。第 151 页 `NttConvolution<p,g>` 可指定素数与原根；`FpsInverse`（第 162 页）、`FpsFunctions`（第 174 页）、`FpsPower`（第 176 页）固定 $p=998244353$。FPS 长度参数是项数，求 x^N 须传 $N+1$。NTT 检查完整乘积长度 $a.size()+b.size()-1$，固定模数最大 2^{23}，FPS 取 $L\le 2^{22}$。

19.10.1.1 例一：不同盒子、相同球、每盒有上下界（OGF 乘积）

$l_i\le a_i\le u_i$、$\sum a_i=N$。令 $c_i=u_i-l_i$、$R=N-\sum l_i$；若 $u_i<l_i$ 或 $R<0$ 答案为 0，否则

$$[x^R]\prod_{i=1}^{k}(1+x+\cdots+x^{c_i}).$$

只保留 $0,\dots,R$ 次：c_i 截至 R，乘后超过 $R+1$ 才截短，不要把中间多项式补零。小规模用滑动窗口

$$d_i(s)=d_i(s-1)+d_{i-1}(s)-[s>c_i]d_{i-1}(s-c_i-1),$$

$d_0(0)=1$、负下标为 0，$O(k(R+1))$ 时间、$O(R+1)$ 空间，任意模数可用。分治 NTT 为 $O(k+M(D+1)\log(k+1))$，$D=\sum\min(c_i,R)$。所有 $c_i=c$ 时可用 `FpsPower::power(P,to_string(k),R+1)`，$P=1+\cdots+x^{\min(c,R)}$，$O(M(R+1)+\log(k+1))$；或在 $k\ge 1$ 时容斥

$$\sum_{j=0}^{\min(k,\lfloor R/(c+1)\rfloor)}(-1)^j\binom{k}{j}\binom{R-j(c+1)+k-1}{k-1},$$

组合数接口见第 191 页。$k=0$ 单独返回 $[N=0]$。

19.10.1.2 例二：步长 1、2 的有序序列（OGF 求逆）

单步 OGF $B(x)=\sum b_sx^s$（$s\ge 1$），有序序列 $F=(1-B)^{-1}\bmod x^{N+1}$，$O(M(N+1))$ 时间、$O(N+1)$ 空间。步长 1、2 时调用：

```
FpsInverse::inverse({1,-1,-1},N+1)
```

要求 $B(0)=0$；只区分各长度使用次数时应改为 $\prod_s(1-x^s)^{-1}$。N 巨大且步长有界时改用远项算法。

19.10.1.3 例三：不同元素分成单点与无序对（EGF 指数）

块的 EGF 为

$$B(x)=x+\frac{x^2}{2!},\qquad F(x)=\exp B(x),\qquad a_n=n![x^n]F(x).$$

在 $p=998244353$、$N<p$ 下用 `FpsFunctions::exp(b,N+1)`，再乘 `Binomial` 的 `fac[N]`；$O(M(N+1))$。也有 $a_0=a_1=1$、$a_n=a_{n-1}+(n-1)a_{n-2}$。恰好 k 块为 $n![x^n]B(x)^k/k!$：用 `FpsPower::power(b,to_string(k),n+1)` 取第 n 项再乘 `fac[n]`、`ifac[k]`，表预处理至 $\max(n,k)<p$；先判 $k\le n\le 2k$，$(n,k)=(0,0)$ 为 1。盒子带标号时不除以 $k!$；允许空盒时单盒因子添 1。

19.11 Lagrange 反演

19.11.1 隐式生成函数与 Lagrange 反演

用于 $T = x\Phi(T)$ 只问 T 或 $H(T)$ 某一项的情形；这里是**复合逆**，不同于第 243 页的点值插值。

19.11.1.1 适用条件与取系数公式

特征零域上形式幂级数，$\Phi(0) \neq 0$（一般 $\mathbb{Q}$-代数中要求 $\Phi(0)$ 可逆），$T(x) = x\Phi(T(x))$、$T(0) = 0$ 唯一确定 T。对 $n \geq 1$：

$$\boxed{[x^n]H(T(x)) = \frac{1}{n}[t^{n-1}]H'(t)\Phi(t)^n,} \qquad \boxed{[x^n]T(x)^k = \frac{k}{n}[t^{n-k}]\Phi(t)^n \ (n \geq k \geq 1).}$$

边界：$[x^0]H(T) = H(0)$；$[x^n]T^0 = [n = 0]$；$k \geq 1$、$n < k$ 时为 0。不要求解析收敛。

19.11.1.2 模型一按内部结点计数的满有序多叉树

每个结点为叶子或恰有 $d \geq 2$ 个有序孩子，按内部结点计数：$A = 1 + xA^d$，$a_0 = 1$，$n \geq 1$ 时

$$a_n = \frac{1}{n}\binom{dn}{n-1} = \frac{1}{(d-1)n+1}\binom{dn}{n}.$$

孩子无序时不能用 A^d。用 `Binomial<p> c(d*n)` 须在宽类型中算 dn 并保证 $dn < p$，否则按第 191 页选组合数算法。

19.11.1.3 模型二带标号有根树的 EGF

根的孩子无序：$T = x\exp T$，$r_n = n![x^n]T = n^{n-1}$（$n \geq 1$）；指定根时为 n^{n-2}（$n \geq 2$）。EGF 系数须乘回 $n!$。

19.11.1.4 落到现有接口及模数边界

单个幂系数。第 176 页的 `FpsPower::power` 固定 $p = 998244353$；在 $1 \leq k \leq n < p$、$L = n - k + 1 \leq 2^{22}$ 下：

```
using F = FpsPower;
using Z = F::Z;
int l = n - k + 1;
auto q = F::power(phi, to_string(n), l);
Z answer = Z(k) * Z(n).inv() * q[n - k];
```

长度是项数（传 $n-k+1$），指数传完整十进制串。单次 $O(L\log(L+1)+\log(n+1))$、空间 $O(L)$；对所有 $n \leq N$ 逐个求为 $O(N^2\log(N+1))$。一般 H：先算 $q = \Phi^n \bmod t^n$，答案为 $n^{-1}\sum_{i=1}^{n} iH_i q_{n-i}$。第 174 页的 `log`（首项 1）、`exp`（首项 0）可构造已知表达式，不能解任意隐式方程；`Binomial<p> c(N)` 要求 $N < p$。第 162 页的 `inverse` 是乘法逆，要求常数项非零，不能求 $f = t/\Phi(t)$ 的复合逆。

模数。在 $\mathbb{F}_p$ 中使用 $1/n$ 公式要求 $p \nmid n$，EGF 的 $1/j!$ 不能跨过 $j = p$；否则改用无除法递推或提升精度后再整除。

19.12 普通生成函数

19.12.1 生成函数与递推

OGF 乘法对应普通卷积；EGF $\sum a_n x^n/n!$ 乘法对应 $c_n = \sum_k \binom{n}{k} a_k b_{n-k}$。求逆要求常数项可逆；$\log A$ 要求 $A(0) = 1$，$\exp A$ 要求 $A(0) = 0$；有限域积分要求 i 不被特征整除。

$$B_{\text{new}} = B(2 - AB) \pmod{x^{2m}}, \qquad (\log A)' = A'/A.$$

BM 外推未来项前须另知序列有不超过相应阶数的线性递推（如由矩阵转移、有限状态得到阶数上界）。

第二十章　组合数学

20.1　排列组合

20.1.1　阶乘组合数

功能：在素数模数下预处理阶乘与逆阶乘，快速回答组合数 $\binom{n}{k}$ 与排列数 $n!/(n-k)!$；表的上限可以按需扩大。

核心思路：`fac[i]`$= i!$，`ifac[i]`$= (i!)^{-1}$。`init` 扩表时先顺推新增的阶乘，只对最大的 `fac[n]` 求一次逆元，再用 $((i-1)!)^{-1} = (i!)^{-1} \cdot i$ 倒推回旧上限。查询时 $\binom{n}{k} = n!\,(k!)^{-1}((n-k)!)^{-1}$，$k$ 越界返回 0。

代码依赖：ModInt（第 90 页，18.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：Binomial<prime>(n=0) 预处理 0..n，要求 prime 为素数且 0≤n<prime。init(n) 只扩大当前上限，较小或相同的 n 不改变表；choose(n,k) 返回 C(n,k)，permute(n,k) 返回 n!/(n-k)!。查询前显式预处理到 n，查询本身不扩表。n<0、k<0 或 k>n 返回 0，C(0,0)=P(0,0)=1。fac、ifac 为可直接读取的阶乘与逆阶乘表，增长可能使旧引用失效。单次查询 O(1)；从 old 扩到 n 新增 O(n-old+log prime) 次域运算，重分配还可能复制旧表。首次预处理 O(n+log prime)，空间 O(n)。超过模数不能使用阶乘除法：小素数用 Lucas，小合数用 exLucas。

```
template <int mod> struct Binomial
{
    using Z = ModInt<mod>;
    vector<Z> fac{Z(1)}, ifac{Z(1)};

    Binomial(int n = 0) { init(n); }

    void init(int n)
    {
        assert(0 <= n && n < mod);
        int old = (int)fac.size() - 1;
        if (n <= old) return;
        fac.resize(n + 1);
        ifac.resize(n + 1);
        for (int i = old + 1; i <= n; i++) fac[i] = fac[i - 1] * i;
        ifac[n] = fac[n].inv();
        for (int i = n; i > old; i--) ifac[i - 1] = ifac[i] * i;
    }
```

```
    Z choose(int n, int k) const
    {
        if (k < 0 || k > n) return 0;
        assert(n < (int)fac.size());
        return fac[n] * ifac[k] * ifac[n - k];
    }

    Z permute(int n, int k) const
    {
        if (k < 0 || k > n) return 0;
        assert(n < (int)fac.size());
        return fac[n] * ifac[n - k];
    }
};
```

20.1.2 计数建模：先确认哪些对象被区分

先说明**一个方案是什么，什么变化会产生新方案**，再选公式。

1. **物品可区分吗？**相同球只记录数量，不能再乘 $n!$。
2. **容器可区分吗？**无标号盒子记录集合划分，空盒不成块。
3. **次序在哪里？**固定 s 个标签组成非空块：集合 1 种，线性排列 $s!$ 种，圆排列 $(s-1)!$ 种。
4. **哪些数量固定？**“恰好 k 个非空组”与“至多 k 个组”不同；统计块数时每块贡献一个 y。
5. **空对象是否是一种方案？**空序列、空集合各 1 种，常数项为 1；“至少一个块”减去 1。无物品且无盒计 1，非空物品放入零盒计 0，题面另有定义时以题面为准。

仅当恰好 k 个非空块且只区分盒子标号时，才可除以 $k!$。

20.1.2.1 OGF、EGF 的选择来自乘法，而非题目用词

两部分合并恰有 $a_i b_j$ 种时用 OGF 普通卷积；若还需从 $i+j$ 个标签中分配，多出 $\binom{i+j}{i}$，用 EGF。

$$\text{OGF 数组：}A_i = a_i, \qquad \text{EGF 数组：}A_i = a_i/i!, \qquad a_n = n!A_n.$$

卷积模板始终计算普通数组卷积，不会补上 EGF 的阶乘。非空带标号块的 EGF 为 $B(x)$ 时，有序块序列是 $1/(1-B(x))$，无序块集合是 $\exp B(x)$；二者都要求 $B(0)=0$。

20.1.3 球盒问题：八种情形

n 个球放入 m 个盒子，$S(n,k)$ 为第二类斯特林数，$f_m(s)$ 为把 s 拆成至多 m 个部分的分拆数（第 207 页）。

球	盒	允许空盒	不允许空盒
不同	不同	m^n	$m!\,S(n,m)$
不同	相同	$\sum_{k=0}^{m} S(n,k)$	$S(n,m)$
相同	不同	$\binom{n+m-1}{m-1}$	$\binom{n-1}{m-1}$
相同	相同	$f_m(n)$	$f_m(n-m)$ $(n \ge m)$

$S(n,0..m)$ 一次取第 199 页整行；$m \ge n$ 时第二行左列即贝尔数。组合数用第 189 页 `Binomial`，负下标返回 0 正好给出 $n < m$ 时的 0。边界：$m=0$ 时八种都为 $[n=0]$；$n=0$、$m \ge 1$ 时允许空盒为 1、不允许空盒为 0。f_m 小数据用 $O(nm)$ 的 `parts_at_most`，模 998244353 且规模大时用 $O(n \log n)$ 的 `parts_at_most_fps`。

20.1.4 组合数选型：先看模数，再看下标与预算

先用有符号或更宽类型处理负下标、$k > n$ 与 $n + k$ 溢出；`Lucas::choose`、`ExLucas::choose` 接收无符号参数，不要传入容斥中的负数。

素数 p，所有查询 $n \le N < p$： 第 189 页 `Binomial<p> c(N)`，`c.choose(n,k)`、`c.permute(n,k)`。建表 $O(N + \log p)$ 时间、$O(N)$ 空间，查询 $O(1)$。`c.init(M)` 每次扩表都求一次逆，尽量一次扩至最大上界。p 为编译期模数，由调用者保证是素数。

素数 p，n 很大且 p 足够小： 第 116 页 `Lucas c(p)`。预处理 $O(p + \log p)$ 时间、$O(p)$ 空间，单次 $O(1 + \log_p n)$。$p = 10^9 + 7$ 时两张表约 8 GB，不适用。

合数 m，所有素数幂因子的表可以放下： 第 117 页 `ExLucas c(m)`，按 $m = \prod q_i$、$q_i = p_i^{e_i}$ 建表。空间 $O(\sum q_i)$，建表 $O(\sqrt{m} + \sum q_i + \sum \log q_i)$；单次 $O(\sum_i (1 + \log_{p_i}(n+1)) \log(n+2) + \sum_i \log q_i)$。模数 1 返回 0；某个 q_i 很大时不适用。

三者都不合适时： n 小、模数任意用 Pascal 递推，$O(n^2)$ 时间、$O(n)$ 滚动空间（只需 $k \le K$ 列时 $O(nK)$）；素数 p 很大、$k' = \min(k, n-k) < p$ 很小时，连乘 k' 个分子再乘 $(k'!)^{-1}$，$O(k' + \log p)$，$n \ge p$ 也可用。合数模数下分母未必可逆，不能把整数除法直接换成模逆元。

20.2 容斥原理

20.2.1 容斥与二项式反演：恰好与至少

交集和带重数，不能直接当作“至少”的数量；性质之间不要求独立。

20.2.1.1 从交集和恢复恰好数量

全集 Ω，性质集合 $A_1,\dots,A_m$，$q(x)=|\{i:x\in A_i\}|$，

$$N_t=|\{x\in\Omega:q(x)=t\}|,\qquad S_j=\sum_{|J|=j}\left|\bigcap_{i\in J}A_i\right|,\quad S_0=|\Omega|.$$

$$\boxed{S_j=\sum_{t=j}^{m}\binom{t}{j}N_t,\qquad N_r=\sum_{j=r}^{m}(-1)^{j-r}\binom{j}{r}S_j\quad(0\le r\le m).}$$

并集 $|\bigcup_i A_i|=\sum_{j=1}^{m}(-1)^{j-1}S_j$。

20.2.1.2 至少数量与变换方向

对 $1\le r\le m$：

$$\boxed{|\{x:q(x)\ge r\}|=\sum_{j=r}^{m}(-1)^{j-r}\binom{j-1}{r-1}S_j.}$$

$r=0$ 返回 S_0，$r>m$ 返回 0。另一方向（下三角）的二项式反演：

$$b_n=\sum_{k=0}^{n}\binom{n}{k}a_k\quad\Longleftrightarrow\quad a_n=\sum_{k=0}^{n}(-1)^{n-k}\binom{n}{k}b_k.$$

只含整数乘加减，对任意正模数成立。一般 S_j 需枚举 2^m 个子集；仅当交集大小只依赖 $|J|$ 时才可写 $S_j=\binom{m}{j}\cdot$ 代表值。

20.2.1.3 例：有标号球放入有标号盒

n 个互异球放入 m 个互异盒，恰好 r 个非空盒：

$$\boxed{\binom{m}{r}\sum_{j=0}^{r}(-1)^j\binom{r}{j}(r-j)^n=\binom{m}{r}r!\left\{{n\atop r}\right\}.}$$

以“盒为空”为性质时 $S_j=\binom{m}{j}(m-j)^n$，对应 N_{m-r}。$r<0$、$r>m$ 或 $r>n$ 时为 0；$n=0$ 时仅 $r=0$ 为 1；$0^0=1$。

20.2.1.4 直接使用现有组合数与模整数

固定 $p=998244353$，要求 $m<p$；用第 18.1.2 节（第 90 页）和第 20.1.1 节（第 189 页）：

```
using Z = ModInt<998244353>;
Z answer = 0;
if (0 <= r && r <= m && r <= n)
{
    Binomial<998244353> c(m);
    for (int j = 0; j <= r; j++)
    {
        Z term = c.choose(r, j) * Z(r - j).pow(n);
        if (j & 1) answer -= term;
        else answer += term;
    }
```

```
    answer *= c.choose(m, r);
}
```

n 须为非负 signed long long（`pow` 指数类型）。时间 $O(m+\log p+(r+1)\log(n+2))$、空间 $O(m)$。素数限制来自 `Binomial` 的费马逆元；合数模或 $m \geq p$ 改用 Pascal、Lucas 或扩展 Lucas。

20.2.2 组合计数

容斥：$|\bigcup A_i| = \sum_{\emptyset \neq S}(-1)^{|S|+1}|\bigcap_{i\in S} A_i|$。二项式反演：若 $b_n = \sum_{k=0}^{n}\binom{n}{k}a_k$，则 $a_n = \sum_{k=0}^{n}(-1)^{n-k}\binom{n}{k}b_k$。

$$\sum_k \binom{a}{k}\binom{b}{n-k} = \binom{a+b}{n}, \qquad C_n = \binom{2n}{n} - \binom{2n}{n+1}.$$

差分形式的 Catalan 公式避免在模数下除以 $n+1$。斯特林数见第 195 页。

球与盒	空盒允许	空盒禁止
不同球，不同盒	k^n	$k!S(n,k)$
相同球，不同盒	$\binom{n+k-1}{k-1}$	$\binom{n-1}{k-1}$
不同球，相同盒	$\sum_{j=0}^{k} S(n,j)$	$S(n,k)$

上表默认 $k \geq 1$；$n=0$、$k=0$ 按原问题单独处理。

20.3 错位排列

20.3.1 错位排列

D_n 为没有不动点的 n 元排列数：

$$D_0 = 1,\ D_1 = 0, \quad D_n = (n-1)(D_{n-1}+D_{n-2}), \qquad D_n = n!\sum_{k=0}^{n}\frac{(-1)^k}{k!} = nD_{n-1}+(-1)^n.$$

恰有 k 个不动点：$\binom{n}{k}D_{n-k}$。$n \geq 1$ 时 D_n 是最接近 $n!/e$ 的整数，只用于估计，取模时用递推。$O(n)$ 打表。

20.4 卡特兰数

20.4.1 Catalan 与投票问题：把前缀限制变成反射双射

一种操作记 $+1$、另一种记 -1，高度为两者之差；p 正 q 负共 $\binom{p+q}{q}$ 个次序。约定 $\binom{N}{k}=0$ 当 $k<0$ 或 $k>N$。

20.4.1.1 弱前缀条件：减去首次走到负一的坏路径

每个前缀高度 ≥ 0，$p \geq q \geq 0$：

$$W(p,q) = \binom{p+q}{q} - \binom{p+q}{q-1}.$$

$p<q$ 时为 0（不能用差式），$p=q=0$ 计 1。$p=q=n$ 得

$$C_n = \binom{2n}{n} - \binom{2n}{n-1} = \frac{1}{n+1}\binom{2n}{n}, \qquad C_0 = 1.$$

20.4.1.2 严格领先：先取走必需的第一步

每个非空前缀高度 >0，$p>q$：

$$S(p,q) = \binom{p+q-1}{q} - \binom{p+q-1}{q-1} = \frac{p-q}{p+q}\binom{p+q}{q}.$$

$p+q>0$ 且 $p \leq q$ 时为 0；空路径单独计 1。

20.4.1.3 初始有余额：反射起点，而非只改终点

从 $h \geq 0$ 走 L 步到 $e \geq 0$、全程不低于 0：须 $|e-h| \leq L$ 且 $L+e-h$ 为偶数，令 $k=(L+e-h)/2$，

$$B(L,h,e)=\binom{L}{k}-\binom{L}{k+h+1}.$$

下界为 b 时起终点先减 b；严格大于 b 取下界 $b+1$。只适用单个常数下界与单位 ± 1 步；有上界、禁位、非单位步等需 DP 或重复反射。

20.4.1.4 取模时优先使用差式，除法条件不能省

差式不要求 $n+1$、$p+q$ 可逆；除法形式在分母不可逆时失效。选择见第 191 页（20.1.4 节）：素数模数且最大上标小于模数用 Binomial（第 189 页，20.1.1 节）；跨过素数模数用 Lucas（第 116 页，18.13.1 节）；合数模数考虑 ExLucas（第 117 页，18.13.2 节）。先用有符号类型检查负下标与 $p+q, 2n, L+e-h, k+h+1$ 的溢出，最后规范化两模值之差。

20.5 斯特林数与斯特林反演

20.5.1 定义与性质

设 $n,k\ge 0$。**无符号第一类** $c(n,k)$：n 个带标号元素的排列中，恰有 k 个循环的方案数；循环之间无序，循环内只忽略起点。**第二类** $S(n,k)$：把 n 个带标号元素划分为 k 个无标号非空集合的方案数。**有符号第一类** $s(n,k)=(-1)^{n-k}c(n,k)$。例：$c(3,1)=2$、$c(3,2)=3$；$S(3,1)=1$、$S(3,2)=3$。

递推与边界 考虑第 n 个元素：第一类可自成一个循环，或插在已有 $n-1$ 个元素之一的后面；第二类可自成一块，或加入已有 k 块之一。故

$$c(n,k)=c(n-1,k-1)+(n-1)c(n-1,k),\qquad S(n,k)=S(n-1,k-1)+kS(n-1,k),$$

$$s(n,k)=s(n-1,k-1)-(n-1)s(n-1,k).$$

三者 $(0,0)$ 项为 1；$n>0$ 时第 0 列为 0；$k>n$ 或下标为负时为 0。$c(n,n)=S(n,n)=1$；$n\ge 1$ 时 $c(n,1)=(n-1)!$、$S(n,1)=1$；$\sum_k c(n,k)=n!$，$\sum_k S(n,k)$ 为 Bell 数。

三种幂的转换 记下降幂 $x^{\underline{k}}=x(x-1)\cdots(x-k+1)$、上升幂 $x^{\overline{k}}=x(x+1)\cdots(x+k-1)$，零次幂为 1，且 $x^{\underline{n}}=(-1)^n(-x)^{\overline{n}}$。普通幂 x^n 与两种阶乘幂互相展开：

$$x^{\overline{n}}=\sum_{k=0}^{n}c(n,k)x^k,\qquad x^{\underline{n}}=\sum_{k=0}^{n}s(n,k)x^k,\qquad x^n=\sum_{k=0}^{n}S(n,k)x^{\underline{k}}=\sum_{k=0}^{n}(-1)^{n-k}S(n,k)x^{\overline{k}}.$$

第一式按“每个因子 $x+i$ 取 x 还是取 i”对应排列的循环结构；第三式在 x 为正整数时数 n 元素到 x 种颜色的映射，按同色类划分后给 k 块分配互异颜色。其余两式由 $x\to -x$ 得到。所以 c、s 分别把上升幂、下降幂展开成普通幂；S 反过来把普通幂展开成下降幂，带符号 $(-1)^{n-k}S$ 展开成上升幂。

生成函数 固定列的指数生成函数（按循环/非空块的标号集合构造）为

$$\sum_{n\ge k}c(n,k)\frac{x^n}{n!}=\frac{(-\log(1-x))^k}{k!},\qquad \sum_{n\ge k}s(n,k)\frac{x^n}{n!}=\frac{(\log(1+x))^k}{k!},$$

$$\sum_{n\ge k}S(n,k)\frac{x^n}{n!}=\frac{(e^x-1)^k}{k!}.$$

第二类的普通生成函数与两类的双变量指数生成函数为

$$\sum_{n\ge k}S(n,k)x^n=\frac{x^k}{\prod_{j=1}^{k}(1-jx)},\qquad \sum_{n,k}c(n,k)y^k\frac{x^n}{n!}=(1-x)^{-y},\qquad \sum_{n,k}S(n,k)y^k\frac{x^n}{n!}=e^{y(e^x-1)}.$$

模素数 p 算到 N 次需 $N<p$，使出现的整数与阶乘可逆；快速实现另受 NTT 长度限制。

20.5.1.1 行列快速求解：推导与选择

需求	函数	方法	复杂度
第一类整行 $c(n,0..n)$	`stirling_first_row`	倍增 + 泰勒平移	$O(n\log n)$
第一类整列 $c(0..N,k)$	`stirling_first_column`	$\exp(k\log A)$	$O(N+m\log m)$
第二类整行 $S(n,0..m)$	`stirling_second_row`	容斥一次卷积	$O(m\log m+m\log p)$
第二类整列 $S(0..N,k)$	`stirling_second_column`	$\exp(k\log A)$	$O(N+m\log m)$

第一类整行。$F_n(x)=x^{\overline{n}}$ 的系数即所求。若已知 F_m，则 $F_{2m}(x)=F_m(x)\,F_m(x+m)$，奇数长度再乘 $x+2m$。$F_m(x+m)$ 用泰勒平移：$[x^i]F(x+m)=\frac{1}{i!}\sum_{j\ge i}(j!f_j)\frac{m^{j-i}}{(j-i)!}$，为一次卷积。$T(n)=T(\lfloor n/2\rfloor)+O(n\log n)$，故 $O(n\log n)$ 时间、$O(n)$ 空间。

第二类整行。由容斥（i^n 为 n 元到 i 色的映射数，再要求每色非空并除以 $k!$），$S(n,k)=\sum_{i=0}^{k}\frac{i^n}{i!}\cdot\frac{(-1)^{k-i}}{(k-i)!}$，一次卷积得 $k=0..m$；$0^0=1$ 单独处理，正底数的指数才对 $p-1$ 取模。

两种整列。列生成函数 $(-\log(1-x))^k$、$(e^x-1)^k$ 都以 x^k 开头，常数项为 0，不能直接取对数。先提出 x^k：第一类 $A(x)=-\log(1-x)/x=\sum_{j\ge 0}\frac{x^j}{j+1}$，第二类 $A(x)=(e^x-1)/x=\sum_{j\ge 0}\frac{x^j}{(j+1)!}$，二者常数项都是 1，于是

$$c(n,k)\text{ 或 }S(n,k)=\frac{n!}{k!}[x^{n-k}]A(x)^k,\qquad A^k=\exp(k\log A).$$

只需 A 的前 $m=N-k+1$ 项；倍率 $n!/k!$ 从 $n=k$ 时的 1 起逐次乘 $n+1$。$k=0$ 时列为 $(1,0,\dots,0)$，$k>N$ 时全零，均直接返回。有符号列再乘 $(-1)^{n-k}$。整行模板可传素数模参数（如 167772161）；整列依赖固定 998244353 的 `FpsFunctions`，不能只改调用处换模数。

20.5.2 无符号第一类斯特林数：同一行

问题描述：固定 n，求无符号第一类斯特林数 $c(n,0),\dots,c(n,n)$，即 $x^{\overline{n}}=x(x+1)\cdots(x+n-1)$ 的全部系数。

核心思路：倍增：$x^{\overline{2m}}=x^{\overline{m}}\cdot(x+m)^{\overline{m}}$，后者由前者泰勒平移一次卷积得到，奇数再乘一个一次因子（推导见上文）。

代码依赖：ModInt（第 90 页，18.1.2 节）、NttConvolution（第 151 页，19.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：stirling_first_row<mod,primitive>(n) 返回无符号 c(n,0..n)，默认模 998244353、原根 3。c(n,k) 为 n 元排列恰有 k 个循环的个数。n≥0，n+1≤NTT 的 max_size；n=0 返回 {1}。倍增配合多项式平移计算 x(x+1)…(x+n-1)，O(n log n) 时间、O(n) 空间。带符号版本再乘 (-1)^(n-k)。依赖 NttConvolution；模 167772161 可用 <167772161>。

```
template <int mod = 998244353, int primitive = 3>
vector<ModInt<mod>> stirling_first_row(int n)
{
    using N = NttConvolution<mod, primitive>;
    assert(n >= 0 && n < N::max_size);
    using Z = typename N::Z;
    using Poly = typename N::Poly;
    Poly fact(n + 1), inv(n + 1);
    fact[0] = 1;
    for (int i = 1; i <= n; i++) fact[i] = fact[i - 1] * i;
    inv[n] = fact[n].inv();
    for (int i = n; i > 0; i--) inv[i - 1] = inv[i] * i;
    function<Poly(int)> solve = [&](int len) -> Poly
    {
        if (len == 0) return {1};
        int m = len / 2;
        Poly f = solve(m);
        Poly a(m + 1), b(m + 1);
        Z power = 1;
        for (int i = 0; i <= m; i++)
        {
            a[m - i] = f[i] * fact[i];
            b[i] = power * inv[i];
            power = power * m;
        }
        auto c = N::multiply(a, b);
        for (int i = 0; i <= m; i++) a[i] = c[m - i] * inv[i];
        f = N::multiply(f, a);
        if (len % 2)
```

```
        {
            f.push_back(0);
            for (int i = len; i > 0; i--) f[i] = f[i - 1] + f[i] * (len - 1);
            f[0] = f[0] * (len - 1);
        }
        return f;
    };
    return solve(n);
}
```

模板题使用：Luogu P5408

题意：输出无符号第一类斯特林数 c(n,0..n)，即 n 元排列按循环个数计数。本题模数为 167772161，须显式传模板参数；无需乘交替符号。

所需模板：ModInt（第 90 页，18.1.2 节）、NttConvolution（第 151 页，19.2.1 节）、stirling_first_row（第 196 页，20.5.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5408

```
int main()
{
    int n;
    scanf("%d", &n);
    auto row = stirling_first_row<167772161>(n);
    for (int i = 0; i <= n; i++) printf("%d%c", row[i].v, i == n ? '\n' : ' ');
}
```

20.5.3 无符号第一类斯特林数：同一列

问题描述：固定 k，求无符号第一类斯特林数 $c(0,k),\ldots,c(N,k)$。

核心思路：列的指数生成函数为 $(-\log(1-x))^k/k!$；提出 x^k 后对常数项为 1 的 $A(x)$ 用 $\exp(k\log A)$ 求 k 次幂，再乘 $n!/k!$（推导见上文）。

代码依赖：FpsFunctions（第 174 页，19.7.2 节）、ModInt（第 90 页，18.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：stirling_first_column(N,k) 返回无符号 c(0..N,k)，固定模 998244353。0≤N<2^22，k≥0；k>N 返回全零，k=0 仅第零项为 1。令 A=-log(1-x)/x=∑x^j/(j+1)（常数项 1），则 c(n,k)=n!/k! · [x^(n-k)]A^k；用 exp(k · log A) 求前 m=N-k+1 项，再从 n=k 起逐项乘 n+1 得 n!/k!。有符号 s(n,k) 另乘 (-1)^(n-k)。时间 O(N+m log m)，空间 O(N)。依赖 FpsFunctions（及 FpsInverse、NttConvolution、ModInt）；分母都小于模数，不能直接改模 167772161。

```
vector<ModInt<998244353>> stirling_first_column(int n, int k)
{
    using F = FpsFunctions;
    using Z = F::Z;
    assert(n >= 0 && n < F::max_size / 2 && k >= 0);
    vector<Z> answer(n + 1);
    if (k > n) return answer;
    if (k == 0)
    {
        answer[0] = 1;
        return answer;
    }
    // A = -log(1-x)/x, c(n,k) = n!/k! [x^(n-k)] A^k.
    int m = n - k + 1;
    F::Poly a(m), inv(m + 1);
    inv[1] = 1;
    for (int i = 2; i <= m; i++) inv[i] = Z(F::mod - F::mod / i) * inv[F::mod % i];
    for (int i = 0; i < m; i++) a[i] = inv[i + 1];
    auto f = F::log(a, m);
    for (auto &x : f) x *= k;
    f = F::exp(f, m);
    Z scale = 1;
    for (int i = k; i <= n; i++)
    {
        answer[i] = f[i - k] * scale;
        scale *= i + 1;
    }
    return answer;
}
```

模板题使用：Library Checker Stirling Number of the First Kind (Fixed K)

题意：输入 N K，0≤K≤N≤500000，模 998244353；只输出 n=K..N 的有符号 s(n,K)。库函数返回无符号 c(n,K)，逐项乘 (-1)^(n-K)；不输出 n<K 的前导零。

所需模板：ModInt（第 90 页，18.1.2 节）、NttConvolution（第 151 页，19.2.1 节）、FpsInverse（第 162 页，19.5.1 节）、FpsFunctions（第 174 页，19.7.2 节）、stirling_first_column（第 198 页，20.5.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/stirling_number_of_the_first_kind_fixed_k

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, k;
    cin >> n >> k;
    auto column = stirling_first_column(n, k);
    for (int i = k; i <= n; i++)
    {
        auto value = column[i];
        if ((i - k) % 2)
            value = decltype(value)(0) - value;
        cout << value.v << (i == n ? '\n' : ' ');
    }
}
```

20.5.4 第二类斯特林数：同一行前缀

问题描述：固定 n（可达 $2^{64}-1$），求第二类斯特林数 $S(n,0),\dots,S(n,m)$。

核心思路：容斥式 $S(n,k)=\sum_i \frac{i^n}{i!}\cdot\frac{(-1)^{k-i}}{(k-i)!}$ 是一次卷积：两个序列分别取 $i^n/i!$ 与 $(-1)^i/i!$。

代码依赖：ModInt（第 90 页，18.1.2 节）、NttConvolution（第 151 页，19.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：stirling_second_row<mod,primitive>(n,m) 返回 S(n,0..m)，默认 mod=998244353、primitive=3。模数必须是素数，primitive 是原根。n 支持完整 unsigned long long；2m+1 不能超过 NTT 的 max_size。S(n,k) 为无标号非空集合划分数；n=0 时仅第零项为 1，k>n 为零。一次卷积配合逆阶乘，O(m log m + m log mod) 时间、O(m) 空间。依赖 NttConvolution；模 167772161 可直接使用模板参数 <167772161>。

```
template <int mod = 998244353, int primitive = 3>
vector<ModInt<mod>> stirling_second_row(unsigned long long n, int m)
{
    using N = NttConvolution<mod, primitive>;
    assert(m >= 0 && m <= (N::max_size - 1) / 2);
    using Z = typename N::Z;
    typename N::Poly a(m + 1), b(m + 1);
    b[0] = 1;
    for (int i = 1; i <= m; i++) b[i] = b[i - 1] * i;
    b[m] = b[m].inv();
    for (int i = m; i > 0; i--) b[i - 1] = b[i] * i;
    for (int i = 0; i <= m; i++)
```

```
    {
        Z value = i ? Z(i).pow(n % (mod - 1)) : Z(n == 0);
        a[i] = value * b[i];
        if (i % 2) b[i] = Z(0) - b[i];
    }
    auto answer = N::multiply(a, b);
    answer.resize(m + 1);
    return answer;
}
```

模板题使用：Luogu P5395

题意：输出第二类斯特林数 S(n,0..n)，即把 n 个不同元素分成各个数目的非空无标号集合的方案数。本题模数为 167772161，须显式传模板参数。

所需模板：ModInt（第 90 页，18.1.2 节）、NttConvolution（第 151 页，19.2.1 节）、stirling_second_row（第 199 页，20.5.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5395

```
int main()
{
    int n;
    scanf("%d", &n);
    auto row = stirling_second_row<167772161>(n, n);
    for (int i = 0; i <= n; i++) printf("%d%c", row[i].v, i == n ? '\n' : ' ');
}
```

20.5.5 第二类斯特林数：同一列

问题描述：固定 k，求第二类斯特林数 $S(0,k),\ldots,S(N,k)$。

核心思路：列的指数生成函数为 $(e^x-1)^k/k!$；提出 x^k 后对 $A(x)=(e^x-1)/x$ 用 $\exp(k\log A)$ 求幂，再乘 $n!/k!$（推导见上文）。

代码依赖：FpsFunctions（第 174 页，19.7.2 节）、ModInt（第 90 页，18.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：stirling_second_column(N,k) 返回 S(0..N,k)，固定模 998244353。0≤N<2^22，k≥0；k>N 返回全零，k=0 仅第零项为 1。令 A=(e^x-1)/x=∑x^j/(j+1)!（常数项 1），则 S(n,k)=n!/k! · [x^(n-k)]A^k；用 exp(k · log A) 求前 m=N-k+1 项，再逐项乘 n!/k!。时间 O(N+m log m)，空间 O(N)。依赖 FpsFunctions（及 FpsInverse、NttConvolution、ModInt）；不能直接改模 167772161。

```
vector<ModInt<998244353>> stirling_second_column(int n, int k)
{
    using F = FpsFunctions;
    using Z = F::Z;
    assert(n >= 0 && n < F::max_size / 2 && k >= 0);
    vector<Z> answer(n + 1);
    if (k > n) return answer;
    if (k == 0)
    {
        answer[0] = 1;
        return answer;
    }
    // A = (e^x-1)/x, S(n,k) = n!/k! [x^(n-k)] A^k.
    int m = n - k + 1;
    F::Poly a(m), inv_fact(m + 1);
    inv_fact[0] = 1;
    for (int i = 1; i <= m; i++) inv_fact[i] = inv_fact[i - 1] * i;
    inv_fact[m] = inv_fact[m].inv();
    for (int i = m; i > 0; i--) inv_fact[i - 1] = inv_fact[i] * i;
    for (int i = 0; i < m; i++) a[i] = inv_fact[i + 1];
    auto f = F::log(a, m);
    for (auto &x : f) x *= k;
    f = F::exp(f, m);
    Z scale = 1;
    for (int i = k; i <= n; i++)
    {
        answer[i] = f[i - k] * scale;
        scale *= i + 1;
    }
    return answer;
}
```

模板题使用：Library Checker Stirling Number of the Second Kind (Fixed K)

题意：输入 N K，0≤K≤N≤500000，模 998244353；只输出 n=K..N 的 S(n,K)。K=0 时输出首项 1、其余 0。

所需模板：ModInt（第 90 页，18.1.2 节）、NttConvolution（第 151 页，19.2.1 节）、FpsInverse（第 162 页，19.5.1 节）、FpsFunctions（第 174 页，19.7.2 节）、stirling_second_column（第 201 页，20.5.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/stirling_number_of_the_second_kind_fixed_k

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, k;
    cin >> n >> k;
    auto column = stirling_second_column(n, k);
    for (int i = k; i <= n; i++)
    {
        auto value = column[i];
        cout << value.v << (i == n ? '\n' : ' ');
    }
}
```

20.5.6 斯特林反演与应用

公式 对下标从 0 开始的任意序列 $a_0, a_1, \dots$ 与 $b_0, b_1, \dots$（求和均为 $0 \le k \le n$）：

$$b_n = \sum_{k=0}^{n} S(n,k) a_k \quad \Longleftrightarrow \quad a_n = \sum_{k=0}^{n} s(n,k) b_k = \sum_{k=0}^{n} (-1)^{n-k} c(n,k) b_k;$$

$$b_n = \sum_{k=0}^{n} c(n,k) a_k \quad \Longleftrightarrow \quad a_n = \sum_{k=0}^{n} (-1)^{n-k} S(n,k) b_k.$$

即“第二类正变换 $\leftrightarrow$ 有符号第一类逆变换”“无符号第一类正变换 $\leftrightarrow$ 带 $(-1)^{n-k}$ 的第二类逆变换”。符号不能省略，$S(n,k)$ 的行标 n 与列标 k 不能对调；两边都必须对 k 从 0 求和（$n \ge 1$ 时 $k=0$ 项系数为 0，可省）。

推导 把 $x^n = \sum_k S(n,k) x^{\underline{k}}$ 中的 $x^{\underline{k}}$ 再按 $x^{\underline{k}} = \sum_j s(k,j) x^j$ 展开，比较 x^j 的系数得 $\sum_{k=j}^{n} S(n,k) s(k,j) = [n=j]$。矩阵 $\mathbf{S} = (S(n,k))$ 与 $\mathbf{s} = (s(n,k))$ 都是对角线为 1 的下三角矩阵，右逆即左逆，故 $\mathbf{sS} = \mathbf{Ss} = I$，$\mathbf{b} = \mathbf{Sa} \iff \mathbf{a} = \mathbf{sb}$，得到第一对。令对角矩阵 $D = \mathrm{diag}((-1)^n)$，则 $\mathbf{s} = D\mathbf{c}D$，$\mathbf{c}^{-1} = D\mathbf{S}D$，得到第二对。两对关系是整数恒等式，取任意模数仍成立；只有快速行列算法需要素数模和逆元。在指数生成函数层面（$A = \sum a_n x^n/n!$，$B = \sum b_n x^n/n!$），第一对等价于 $B(x) = A(e^x - 1)$ 与 $A(x) = B(\log(1+x))$，是形式复合，不是逐项乘积或普通卷积。

手算检查 取 $a = (1,2,3,4)$，第二类正变换得 $b_0 = 1, b_1 = 2, b_2 = S(2,1)\cdot 2 + S(2,2)\cdot 3 = 5, b_3 = 1\cdot 2 + 3\cdot 3 + 1\cdot 4 = 15$。逆变换用 $s(2,\cdot) = (0,-1,1)$、$s(3,\cdot) = (0,2,-3,1)$：$a_2 = -b_1 + b_2 = 3$，$a_3 = 2b_1 - 3b_2 + b_3 = 4$，恢复原序列。

建模：“按块数赋权” 若对 n 个带标号元素的每种集合划分，给含 k 块的划分赋权 a_k，则总权 $b_n = \sum_k S(n,k) a_k$；若 b_n 容易直接计数，就用有符号第一类恢复“恰好 k 块”的量 a_n。例如 n 个元素各染 q 色之一共 q^n 种，按同色类划分后各块颜色互异，有 $q^{\underline{k}}$ 种，即 $b_n = q^n$、$a_k = q^{\underline{k}}$；反演得 $q^{\underline{n}} = \sum_k s(n,k) q^k$，正是“下降幂 $\to$ 普通幂”的转换。$a_k \equiv 1$ 时 b_n 为 Bell 数，逆变换应恢复全 1。排列按循环数赋权时同理使用第二对。只需少数几个 a_n 时，用整行模板求 $s(n,\cdot)$ 后做点积；四种行列模板不提供任意序列的整段快速斯特林变换。

应用：自然数幂和（普通幂 $\to$ 下降幂） 因为 $\sum_{i=0}^{n} i^{\underline{j}} = \frac{(n+1)^{\underline{j+1}}}{j+1}$，

$$\sum_{i=0}^{n} i^k = \sum_{j=0}^{k} S(k,j) \frac{(n+1)^{\underline{j+1}}}{j+1}.$$

例：$n=3, k=2$ 时 $S(2,1) = S(2,2) = 1$，得 $\frac{4\cdot 3}{2} + \frac{4\cdot 3\cdot 2}{3} = 14 = 0+1+4+9$。模 998244353 时，用一次整行 $O(k \log k)$ 求出系数，n 可达 10^{18}（需 $k+1 < p$ 以除以 $j+1$）：

```
using Z = ModInt<998244353>;
Z power_sum(unsigned long long n, int k) // sum of i^k for i = 0..n
{
    auto S = stirling_second_row(k, k);
    Z answer = 0, falling = Z((n + 1) % 998244353);
    for (int j = 0; j <= k; j++)
    {
        answer += S[j] * falling / Z(j + 1);
        falling *= Z(n % 998244353) - Z(j);
    }
    return answer;
}
```

`falling` 依次为 $(n+1)^{\underline{j+1}}$；$n < j$ 时乘到因子 0 后保持为 0，与真实值一致。

20.6 贝尔数

20.6.1 贝尔数

B_n 为 n 元集合的划分数，$B_0 = 1$：

$$B_{n+1} = \sum_{k=0}^{n} \binom{n}{k} B_k, \qquad B_n = \sum_{k=0}^{n} S(n,k), \qquad \sum_{n\geq 0} B_n \frac{x^n}{n!} = e^{e^x-1}.$$

小范围用递推 $O(n^2)$，或贝尔三角：首列 $a_{n,0} = a_{n-1,n-1}$，$a_{n,k} = a_{n,k-1} + a_{n-1,k-1}$，$B_n = a_{n,0}$。批量模 998244353 用第 174 页 `FpsFunctions::exp` 对 $e^x - 1$ 取 exp，$O(n \log n)$；单个 B_n 也可由第 199 页的斯特林数整行求和。素数 p 下有 Touchard 同余 $B_{n+p} \equiv B_n + B_{n+1} \pmod p$。

20.7 伯努利数

20.7.1 伯努利数批量计算

问题描述： 批量求伯努利数 $B_0, B_1, \ldots, B_n$ 模 998244353，取 $B_1 = -\frac{1}{2}$ 的约定，常用于自然数幂和公式。

核心思路： 伯努利数的指数生成函数为 $\frac{x}{e^x-1} = \sum_k B_k \frac{x^k}{k!}$，它是 $\frac{e^x-1}{x} = \sum_k \frac{x^k}{(k+1)!}$ 的逆级数。代码预处理阶乘与逆阶乘，构造后者的前 $n+1$ 项 `d`，用 `FpsInverse` 求逆，再把第 k 项乘 $k!$ 得到 B_k。

代码依赖：FpsInverse（第 162 页，19.5.1 节）、ModInt（第 90 页，18.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约： bernoulli(n) 返回 B_0..B_n 模 998244353，约定 B_1=-1/2（生成函数 x/(e^x-1)）。对 (e^x-1)/x 的前 n+1 项求逆再乘阶乘，O(n log n) 时间、O(n) 空间，要求 n+2≤2^23；依赖 FpsInverse。自然数幂和公式见知识节。

```
// B_0..B_n mod 998244353 from x/(e^x-1) (B_1 = -1/2). n + 1 <= 2^23. O(n log n).
vector<ModInt<998244353>> bernoulli(int n)
{
    using F = FpsInverse;
    assert(n >= 0 && n + 2 <= F::max_size);
    vector<F::Z> fac(n + 2, 1), ifac(n + 2);
    for (int i = 1; i <= n + 1; i++) fac[i] = fac[i - 1] * F::Z(i);
    ifac[n + 1] = fac[n + 1].inv();
    for (int i = n + 1; i > 0; i--) ifac[i - 1] = ifac[i] * F::Z(i);
    F::Poly d(n + 1); // (e^x - 1)/x = sum x^k/(k+1)!
    for (int k = 0; k <= n; k++) d[k] = ifac[k + 1];
    auto b = F::inverse(d, n + 1);
    for (int k = 0; k <= n; k++) b[k] = b[k] * fac[k];
    return b;
}
```

20.7.2 伯努利数与自然数幂和

约定 $B_1 = -\frac{1}{2}$：$\sum_{k\geq 0} B_k \frac{x^k}{k!} = \frac{x}{e^x - 1}$，等价于 $B_0 = 1$，$\sum_{j=0}^{m} \binom{m+1}{j} B_j = 0$（$m \geq 1$）；$k \geq 3$ 为奇数时 $B_k = 0$。

$$\sum_{i=0}^{n-1} i^k = \frac{1}{k+1} \sum_{j=0}^{k} \binom{k+1}{j} B_j \, n^{k+1-j}.$$

批量 $B_0..B_n$（模 998244353）用第 204 页 `bernoulli(n)`，$O(n \log n)$。只求一个 k 的幂和时，用第 246 页连续点拉格朗日插值 $O(k)$ 更简单；需要对很多 k 求、或要多项式系数时用伯努利数。模数需大于 $k+1$ 以便除法。

20.8 Eulerian Number

20.8.1 欧拉数（Eulerian number）

$\left\langle {n \atop k} \right\rangle$ 为恰有 k 个下降（$p_i > p_{i+1}$）的 n 元排列数，$\left\langle {0 \atop 0} \right\rangle = 1$：

$$\left\langle {n \atop k} \right\rangle = (k+1)\left\langle {n-1 \atop k} \right\rangle + (n-k)\left\langle {n-1 \atop k-1} \right\rangle = \sum_{i=0}^{k} (-1)^i \binom{n+1}{i} (k+1-i)^n.$$

Worpitzky 恒等式：$x^n = \sum_k \left\langle {n \atop k} \right\rangle \binom{x+k}{n}$。按行和为 $n!$，且关于 $k \leftrightarrow n-1-k$ 对称。递推 $O(n^2)$，单项用求和式 $O(k \log n)$。

20.9 分拆数

20.9.1 整数分拆与重数上限

问题描述：求 0..N 每个数的无序整数分拆数 $p(n)$（模任意正整数），并能查询“每个正整数至多出现 k 次”的分拆个数。

核心思路：五边形数定理 $\prod_{i\geq 1}(1-x^i)=\sum_j(-1)^j x^{j(3j\mp 1)/2}$ 给出递推 $p(i)=\sum_{j\geq 1}(-1)^{j+1}\left[p(i-\frac{j(3j-1)}{2})+p(i-\frac{j(3j+1)}{2})\right]$，每个 i 只有 $O(\sqrt{i})$ 项。重数不超过 k 的生成函数为 $\prod_i \frac{1-x^{(k+1)i}}{1-x^i}=P(x)\prod_i(1-x^{(k+1)i})$，后者就是把五边形数级数中的 x 换成 x^{k+1}，因此 `limited` 只需在 $p(n)$ 上加减 $p(n-(k+1)\cdot$五边形数$)$。

接口契约：Partitions(N,mod) 求无序正整数分拆，0≤N<INT_MAX、1≤mod≤INT_MAX，p[n] 为不限制重数的分拆数；p[0]=1%mod。预处理 O(N$\sqrt{}$N) 时间、O(N) 空间；模数不要求素数。limited(n,k) 限制每个正整数最多出现 k 次，0≤n≤N、0≤k≤INT_MAX；k=0 时仅空分拆存在，k=1 为互异分拆。单次 O(1+$\sqrt{}$(n/(k+1)))，额外空间 O(1)。p/mod 只读；中间五边形数与累加采用 long long。

```
struct Partitions
{
    int mod;
    vector<int> p;

    Partitions(int n, int mod) : mod(mod), p(n + 1)
    {
        assert(n >= 0 && mod >= 1);
        p[0] = 1 % mod;
        for (int i = 1; i <= n; i++)
        {
            long long value = 0;
            for (long long j = 1; j * (3 * j - 1) / 2 <= i; j++)
            {
                long long a = j * (3 * j - 1) / 2;
                long long b = j * (3 * j + 1) / 2;
                int sign = j % 2 ? 1 : -1;
                value += 1LL * sign * p[i - a];
                if (b <= i) value += 1LL * sign * p[i - b];
            }
            p[i] = (value % mod + mod) % mod;
        }
    }

    int limited(int n, int k) const
    {
        assert(0 <= n && n < (int)p.size() && k >= 0);
        long long step = 1LL + k;
        long long value = p[n];
        for (long long j = 1; j * (3 * j - 1) / 2 <= n / step; j++)
        {
            long long a = step * (j * (3 * j - 1) / 2);
            long long b = step * (j * (3 * j + 1) / 2);
            int sign = j % 2 ? -1 : 1;
            value += 1LL * sign * p[n - a];
            if (b <= n) value += 1LL * sign * p[n - b];
        }
        return (value % mod + mod) % mod;
    }
};
```

模板题使用：Library Checker partition_function

题意：输出 0..N 的全部整数分拆数，模 998244353，N≤500000。构造 Partitions 后直接读 p；p[0]=1 表示空分拆。五边形数递推 O(N$\sqrt{N}$) 时间、O(N) 空间，不是有序拆分。此用法不调用 limited。

所需模板：Partitions（第 206 页，20.9.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/partition_function

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    Partitions part(n, 998244353);
    for (int i = 0; i <= n; i++) cout << part.p[i] << (i == n ? '\n' : ' ');
}
```

20.9.2 至多 m 个部分的分拆表

问题描述：求 0..n 每个数分拆成至多 m 个正整数之和（不计顺序）的方案数，模任意正整数。

核心思路：由 Ferrers 图共轭，“至多 m 个部分”等价于“每部分不超过 m”。于是做完全背包：物品是大小 $1..\min(m,n)$ 的部分，每种可取任意多个，按大小逐个加入，正序遍历 s 执行 `f[s] += f[s - i]`。

接口契约：parts_at_most(n,m,mod) 返回 f[0..n]，f[s] 为把 s 拆成至多 m 个正整数之和（不计顺序）的方案数，等价于每部分不超过 m（共轭）。0≤n、0≤m、1≤mod≤INT_MAX，模数不要求素数；f[0]=1%mod，m=0 时仅 f[0] 非零。恰好 m 个部分：n≥m 时为 f[n-m]，否则为 0（m=0 时为 [n=0]）。完全背包逐个加入部分大小 1..min(m,n)，O(n · min(m,n)) 时间、O(n) 空间；n · m 大时用 parts_at_most_fps。

```
// f[s] = partitions of s into at most m parts (= parts no larger than m), 0<=s<=n.
vector<int> parts_at_most(int n, int m, int mod)
{
    assert(n >= 0 && m >= 0 && mod >= 1);
    vector<int> f(n + 1);
    f[0] = 1 % mod;
    for (int i = 1; i <= min(m, n); i++)
        for (int s = i; s <= n; s++) f[s] = (0LL + f[s] + f[s - i]) % mod;
    return f;
}
```

20.9.3 至多 m 个部分的分拆表（生成函数）

问题描述：与 `parts_at_most` 相同，求 $0..n$ 每个数分拆成至多 m 个部分的方案数，但模数固定为 998244353，适合 n, m 都很大的情形。

核心思路：所求生成函数为 $\prod_{i=1}^{m} \frac{1}{1-x^i}$。取对数：$\ln \frac{1}{1-x^i} = \sum_{j\geq 1} x^{ij}/j$，所以对每个 $i \leq \min(m,n)$ 枚举 j，把 $1/j$ 加到 `g[i*j]`，总共调和级数次加法；$1/j$ 由线性逆元递推表 `inv` 给出。最后调用 `FpsFunctions::exp` 求出前 $n+1$ 项。

代码依赖：FpsFunctions（第 174 页，19.7.2 节）、ModInt（第 90 页，18.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：parts_at_most_fps(n,m) 返回与 parts_at_most 相同的表，模 998244353。生成函数为 1/(1-x)(1-x^2)⋯(1-x^m)，其对数等于对所有 i≤m、j≥1 累加 x^(ij)/j，再用 FpsFunctions::exp 求前 n+1 项。要求 0≤n、n+1≤2^22、m≥0；m≥n 时即无限制分拆数。构造对数 O(n log n)，exp 为 O(n log n) 时间、O(n) 空间；依赖 FpsFunctions。

```
// Same table modulo 998244353: exp(sum_{i<=m} sum_j x^{ij}/j), n+1 <= 2^22.
vector<ModInt<998244353>> parts_at_most_fps(int n, int m)
{
    using F = FpsFunctions;
    assert(n >= 0 && m >= 0 && n + 1 <= (1 << 22));
    F::Poly g(n + 1);
    vector<F::Z> inv(n + 1, 1);
    for (int j = 2; j <= n; j++) inv[j] = F::Z(F::mod - F::mod / j) * inv[F::mod % j];
    for (int i = 1; i <= min(m, n); i++)
        for (int j = 1; i * j <= n; j++) g[i * j] = g[i * j] + inv[j];
    return F::exp(g, n + 1);
}
```

20.9.4 整数分拆：重数限制的差一问题

实现见第 206 页。$P(z)=\prod_{i\ge1}(1-z^i)^{-1}$，$p(0)=1$，五边形数定理

$$\prod_{i\ge1}(1-z^i)=1+\sum_{j\ge1}(-1)^j\left(z^{j(3j-1)/2}+z^{j(3j+1)/2}\right)$$

给出 $p(n)$ 递推，符号 $+,+,-,-,\ldots$，每个 n 有 $O(\sqrt{n})$ 项；无需逆元，任意正模数（含 1）。每个值至多出现 k 次：

$$\prod_{i\ge1}(1+z^i+\cdots+z^{ki})=P(z)\prod_{i\ge1}(1-z^{(k+1)i}),$$

偏移放大 $k+1$ 倍，符号 $-,-,+,+,\ldots$。`limited(n,k)` 包含 k；$k=0$ 时非零 n 无解，$k=1$ 为互异分拆。“每种少于 k 次”对应 `limited(n,k-1)`（$k\ge1$）。

20.9.4.1 三个容易混淆的限制

$A_k(n)$：部分均不超过 k 的分拆数，等于至多 k 个部分的分拆数，生成函数 $\prod_{i=1}^{k}(1-z^i)^{-1}$，用面值 $1,\ldots,k$ 完全背包。恰好 k 部分为 $A_k(n-k)$；恰好 k 个互异部分为 $A_k\left(n-\frac{k(k+1)}{2}\right)$（用 `1LL*k*(k+1LL)/2`）。负目标为 0；$A_0(0)=1$，$A_0(n)=0$（$n>0$）。这些都不同于限制每个值出现次数的 `limited`。

20.9.4.2 重数限制与禁止倍数部分

$d=k+1\ge1$ 时

$$\prod_{i\ge1}\frac{1-z^{di}}{1-z^i}=\prod_{\substack{i\ge1\\ d\nmid i}}\frac{1}{1-z^i},$$

即每值至多 k 次等价于禁用 d 的倍数，可用完全背包核对；$k=1$ 时互异分拆数等于奇数部分分拆数；$k\ge n$ 时无限制。

20.10 范德蒙德卷积

20.10.1 范德蒙德卷积及常用变形

$$\sum_i\binom{n}{i}\binom{m}{k-i}=\binom{n+m}{k},\qquad\sum_i\binom{n}{i}^2=\binom{2n}{n},\qquad\sum_i\binom{n}{i}\binom{m}{i}=\binom{n+m}{n},$$

$$\sum_{i=0}^{n}\binom{i}{a}\binom{n-i}{b}=\binom{n+1}{a+b+1}.$$

组合解释：从两组中共选 k 个，按第一组选了几个分类；最后一式按第 $a+1$ 个被选元素的位置分类。遇到“对某个分界点求和的两个组合数之积”时先尝试化成一个组合数，可把 $O(n)$ 的求和变成 $O(1)$。

20.11 Pólya 计数

20.11.1 Burnside 平均：任意模数下的精确除法

功能：用 Burnside 引理求轨道数 $\frac{1}{|G|}\sum_{\sigma\in G}|\mathrm{Fix}(\sigma)|$ 时的累加器：逐个加入不动点数（可带重数），最后得到除以群阶后的结果模 p，即使群阶在模 p 下不可逆也正确。

核心思路：记群阶为 N，真实总和 S 必被 N 整除，设 $S = NT$。在模 Np 下累加，得到 $S \bmod Np = N\,(T \bmod p)$，直接整除 N 就是 $T \bmod p$，无需逆元。因此所有不动点数（如颜色数 循环数）都必须在模 mod= Np 下计算，结构内附带同模的快速幂 power；乘法用 __int128 防溢出。

接口契约：BurnsideAverage(g,p)：群阶 g>0，目标模数 p>0，要求 g*p<=LLONG_MAX/2（先用 int128 检查）。mod=g*p 供不动点计数使用；add(fixed,multiplicity=1) 累加非负 long long 计数与重数，内部 int128 乘法。power(a,k) 求 a^k 模 mod，a,k 非负，0^0=1。result() 返回 sum/g，支持 p=1 与 g 不可逆；sum 必须是对全部群元素的不动点计数之和模 mod，不是先模 p 后的计数。调用方保证确为群作用并完整枚举群元素或共轭类型，重数合计 g；不会生成闭包或验证群公理。字段只读，add 后可继续累计，只有完整合法总和才能 result。每次 add/result 为 O(1)，power 为 O(log(k+1))，空间 O(1)。

```
struct BurnsideAverage
{
    using ll = long long;
    ll order, mod, sum = 0;

    BurnsideAverage(ll g, ll p) : order(g)
    {
        assert(g > 0 && p > 0 && __int128(g) * p <= LLONG_MAX / 2);
        mod = g * p;
    }

    ll power(ll a, long long k) const
    {
        assert(a >= 0 && k >= 0);
        a %= mod;
        ll ans = 1 % mod;
        while (k)
        {
            if (k & 1) ans = __int128(ans) * a % mod;
            a = __int128(a) * a % mod;
            k >>= 1;
        }
        return ans;
    }

    void add(ll fixed, ll multiplicity = 1)
    {
        assert(fixed >= 0 && multiplicity >= 0);
        sum = (sum + __int128(fixed) * multiplicity) % mod;
    }

    ll result() const
    {
        assert(sum % order == 0);
        return sum / order;
    }
};
```

20.11.2 置换的循环长度分解

功能：给定 0-based 置换 p，求它分解成的各个不交循环的长度。Pólya 计数中，一个置换下的不动染色数为颜色数 ^{循环数}。

核心思路：从每个未访问的位置 i 出发，沿 $u \leftarrow p_u$ 一直走到回到 i，途中标记 `seen` 并计数，得到包含 i 的循环长度。每个位置恰被访问一次。

接口契约：permutation_cycles(p) 返回各不交循环长度，按循环最小位置升序。p 是 0-based 的合法置换，n<INT_MAX；空置换返回空 vector。本函数不校验输入，非置换不允许。O(n) 时间与空间；环上循环迭代不是 DFS。固定染色在每个循环内同色，命名颜色不限数量时为 colors^ 循环数；限定各色用量时以循环长度做背包。

```
vector<int> permutation_cycles(const vector<int> &p)
{
    int n = p.size();
    vector<int> seen(n), lengths;
    for (int i = 0; i < n; i++)
    {
        if (seen[i]) continue;
        int u = i, len = 0;
        do
        {
            seen[u] = 1;
            len++;
            u = p[u];
        } while (u != i);
        lengths.push_back(len);
    }
    return lengths;
}
```

20.11.3 Pólya 环染色：旋转与翻转

问题描述： n 个珠子排成环，每颗用 c 种颜色之一染色，旋转（可选再加翻转）后相同的方案视为同一种，求本质不同的方案数模给定模数。例如 $n=4$、$c=2$、只计旋转时答案为 6。

核心思路： Burnside 引理。旋转 s 格的置换有 $\gcd(s,n)$ 个循环；阶为 d（$d \mid n$）的旋转恰有 $\varphi(d)$ 个，各有 n/d 个循环，故累加 $\varphi(d)\,c^{n/d}$。代码先分解 n 的质因数，再枚举约数 d 并用这些质因子求 $\varphi(d)$。计翻转时：n 为奇数有 n 条对称轴，各有 $(n+1)/2$ 个循环；n 为偶数时，过顶点的轴与过边中点的轴各 $n/2$ 条，循环数分别为 $n/2+1$ 与 $n/2$。总和交给 `BurnsideAverage` 精确除以群阶 n 或 $2n$。

代码依赖：BurnsideAverage（第 210 页，20.11.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约： necklace_colorings(n,colors,mod,reflection=false)：n>=1 的环，每位置可选 colors>=0 种命名颜色，返回本质不同染色数模 mod。n 为 int；mod>0，群阶 g 为 n 或 2n，须 g*mod<=LLONG_MAX/2。默认只视旋转相同；reflection=true 也视翻转相同。允许颜色不用完，不能用于相邻异色、固定颜色用量或颜色重命名模型。n=1/2 仍适用，二面体抽象群作用可不忠实。用 BurnsideAverage 模 g*mod 后整除，无需模逆元；因子分解与枚举约数 O(sqrt(n))，每约数 O(omega(n)+log(n))，额外 O(omega(n))。

```
long long necklace_colorings(int n, long long colors, long long mod,
                             bool reflection = false)
{
    assert(n >= 1 && colors >= 0);
    BurnsideAverage avg((reflection ? 2LL : 1LL) * n, mod);
    vector<int> primes;
    int x = n;
    for (int p = 2; p <= x / p; p++)
    {
        if (x % p != 0) continue;
        primes.push_back(p);
        while (x % p == 0) x /= p;
    }
    if (x > 1) primes.push_back(x);
    auto add = [&](int d)
    {
        long long phi = d;
        for (int p : primes)
        {
            if (d % p == 0) phi = phi / p * (p - 1);
        }
        avg.add(avg.power(colors, n / d), phi);
    };
    for (int d = 1; d <= n / d; d++)
    {
        if (n % d != 0) continue;
        add(d);
        if (d != n / d) add(n / d);
    }
    if (reflection)
    {
        if (n & 1) avg.add(avg.power(colors, n / 2 + 1), n);
        else
        {
            avg.add(avg.power(colors, n / 2), n / 2);
            avg.add(avg.power(colors, n / 2 + 1), n / 2);
        }
    }
    return avg.result();
}
```

模板题使用：P4980 【模板】Pólya 定理

题意：t<=1000，1<=n<=10^9；n 个点用 n 种命名颜色，仅旋转等价，模 10^9+7。直接调用旋转环计数。

所需模板：BurnsideAverage（第 210 页，20.11.1 节）、necklace_colorings（第 212 页，20.11.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4980

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while (t--)
    {
        int n;
        cin >> n;
        cout << necklace_colorings(n, n, 1000000007) << '\n';
    }
}
```

20.11.4 群作用计数：先确定什么算同一个方案

合法对象集合 X 有限，等价变换组成有限群 G，且作用把 X 映回 X；一个轨道就是一个方案。项链商去 C_n；手链（允许翻面，$n \geq 3$）商去阶 $2n$ 的二面体群，反射是否等价由题意决定。颜色可重命名、指定位置或障碍等会改变作用，只有保持整个合法集合的变换才能用 Burnside：

$$|Gx| = \frac{|G|}{|G_x|}, \qquad |X/G| = \frac{1}{|G|}\sum_{g\in G} |\operatorname{Fix}_X(g)|.$$

仅当所有稳定子平凡时才可用 $|X|/|G|$。

20.11.5 项链与手链：从置换循环推公式

$n \geq 1$、$k \geq 1$。置换 g 有 $c(g)$ 个循环时不动染色数为 $k^{c(g)}$。旋转 t 步有 $\gcd(n,t)$ 个循环（$\gcd(n,0)=n$），故

$$R(n,k) = \sum_{t=0}^{n-1} k^{\gcd(n,t)} = \sum_{d\mid n} \varphi(d) k^{n/d}, \qquad N(n,k) = R(n,k)/n.$$

反射：n 奇时 n 个，各 $k^{(n+1)/2}$；n 偶时穿顶点 $n/2$ 个各 $k^{n/2+1}$，穿边中点 $n/2$ 个各 $k^{n/2}$。

$$B(n,k) = \frac{R(n,k)+F(n,k)}{2n}, \qquad F(n,k) = \begin{cases} nk^{(n+1)/2}, & n \text{ 奇数}, \\ \dfrac{n}{2}\left(k^{n/2+1}+k^{n/2}\right), & n \text{ 偶数}. \end{cases}$$

$n = 1, 2$ 时公式仍成立（$2n$ 个形式元素连同重复置换保留）；若对置换去重，分母同步改为不同置换数。不定义 $n = 0$；$k = 0$ 时答案为 0。

20.11.6 指定颜色库存：给每个循环分配整份颜色

颜色数量 $a_1, \ldots, a_k$，和为 n。若 g 的循环长度为 $\ell_1, \ldots, \ell_c$，

$$|\operatorname{Fix}_{\boldsymbol{a}}(g)| = [x_1^{a_1}\cdots x_k^{a_k}] \prod_{j=1}^{c} \left(x_1^{\ell_j} + \cdots + x_k^{\ell_j}\right),$$

相加后除以 $|G|$（同长循环不再除阶乘）。全部循环长 d 时：若有 $d \nmid a_i$ 则为 0，否则为 $\frac{(n/d)!}{\prod_i (a_i/d)!}$，可依次累乘 $\binom{r}{a_i/d}$，素数模数下用第 189 页的 `Binomial<p>`；若在合数模数 $M|G|$ 下计算，按第 191 页选组合数算法或用下

面的 DP。二色恰 r 个红：取 $[x^r]\prod_j(1+x^{\ell_j})$，$f(0)=1$ 起每个循环做 $f(s)\leftarrow f(s)+[s\ge\ell]f(s-\ell)$，$s$ 从 r 降到 ℓ，时间 $O(c(r+1))$、空间 $O(r+1)$，只用加法，任意正模数。多色状态数至多 $\prod_{i<k}(a_i+1)$。“相邻异色” 等不变约束须把循环缩点后在约束图上数不动点；不对群封闭的约束（如指定位置颜色）不能直接平均。

20.11.7 群阶不可逆：先提升模数，再作整数除法

求 $(S/h) \bmod M$，$h=|G|$。仅当 $\gcd(h,M)=1$ 才能乘逆元；否则在模 $L=Mh$ 下求 $s=S \bmod L$，则

$$h \mid s, \qquad S/h \equiv s/h \pmod{M}.$$

须先合计全部项再除。用第 85 页的 `Mod64::mul`、`Mod64::power` 时要求 $Mh \le 2^{64}-1$（先转宽类型再乘，求和也用 128 位）；超出则需多精度。无库存旋转/反射计数时间 $O(n\log(n+1))$、额外空间 $O(1)$。

第二十一章　线性代数

21.1　矩阵

21.1.1　模矩阵乘法与幂

功能：模 mod 下的矩阵乘法 multiply 与方阵快速幂 power，常用于加速常系数线性递推或计数转移。

核心思路：乘法按 i,t,j 的顺序三重循环累加 $c_{ij}\mathrel{+}=a_{it}b_{tj}$，内层连续访问行，缓存友好。快速幂从单位阵 r 出发按二进制拆分指数：当前位为 1 时乘上 a，每轮把 a 平方。只用加法和乘法，所以模数不必是素数。

代码依赖：ModInt（第 90 页，18.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：ModMatrix<p>::multiply(a,b) 计算 n×k 与 k×m 的矩阵积，要求非空且每行尺寸一致。power(a,e) 接收非空 n×n 方阵和 unsigned long long 指数；e=0 返回单位阵。只使用加乘，p 为 ≥2 的 int 模数，允许合数。矩阵乘法 O(nkm)，矩阵幂 O(n^3 log(e+1))，额外空间 O(n^2)；复用 ModInt。

```
template <int mod> struct ModMatrix
{
    using Z = ModInt<mod>;
    using Matrix = vector<vector<Z>>;

    static Matrix multiply(const Matrix &a, const Matrix &b)
    {
        assert(!a.empty() && !b.empty());
        int n = a.size(), m = b[0].size(), k = b.size();
        assert((int)a[0].size() == k);
        Matrix c(n, vector<Z>(m));
        for (int i = 0; i < n; i++)
            for (int t = 0; t < k; t++)
                for (int j = 0; j < m; j++) c[i][j] = c[i][j] + a[i][t] * b[t][j];
        return c;
    }

    static Matrix power(Matrix a, unsigned long long e)
    {
        int n = a.size();
        assert(n > 0 && (int)a[0].size() == n);
        Matrix r(n, vector<Z>(n));
        for (int i = 0; i < n; i++) r[i][i] = 1;
        for (; e; e >>= 1, a = multiply(a, a))
            if (e & 1) r = multiply(r, a);
        return r;
    }
};
```

模板题使用：Library Checker matrix_product

题意：计算 n×m 矩阵与 m×k 矩阵的乘积，模 998244353。输入元素已取模，按行输出 n×k 个结果。

所需模板：ModInt（第 90 页，18.1.2 节）、ModMatrix（第 215 页，21.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/matrix_product

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using M = ModMatrix<998244353>;
    int n, m, k;
    cin >> n >> m >> k;
    M::Matrix a(n, vector<M::Z>(m)), b(m, vector<M::Z>(k));
    for (auto &row : a)
        for (auto &x : row) cin >> x.v;
    for (auto &row : b)
        for (auto &x : row) cin >> x.v;
    auto c = M::multiply(a, b);
    for (auto &row : c)
        for (int j = 0; j < k; j++) cout << row[j].v << (j + 1 == k ? '\n' : ' ');
}
```

21.1.2 素数模矩阵求逆

功能：给定模素数的 n 阶方阵 A，判断是否可逆，可逆时求 A^{-1}。

核心思路：对增广矩阵 $[A \mid I]$ 做高斯—若尔当消元：第 i 步从第 i 行往下找第 i 列的非零主元，找不到说明 A 奇异；否则两边同时换行、主元行乘逆元归一，并用它消去其他所有行的第 i 列。左边化为单位阵时，右边的 b 就是 A^{-1}。

代码依赖：ModInt（第 90 页，18.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：matrix_inverse<mod>(a) 返回 optional 方阵；a 为 n×n 的 ModInt<mod> 矩阵，mod 必须是素数。可逆时返回逆矩阵，奇异时返回 nullopt；按值传入不改原矩阵，空方阵返回有值的空矩阵。每次同时对原矩阵和单位阵进行换行、倍乘、消元；必须同步交换两边的行。依赖 ModInt；$O(n^3 + n \log mod)$ 时间、$O(n^2)$ 额外空间。

```
template <int mod>
optional<vector<vector<ModInt<mod>>>> matrix_inverse(vector<vector<ModInt<mod>>> a)
{
    using Z = ModInt<mod>;
    int n = a.size();
    vector<vector<Z>> b(n, vector<Z>(n));
    for (int i = 0; i < n; i++)
    {
        assert((int)a[i].size() == n);
        b[i][i] = 1;
    }
    for (int i = 0; i < n; i++)
    {
        int p = i;
        while (p < n && !a[p][i].v) p++;
        if (p == n) return nullopt;
        swap(a[p], a[i]);
        swap(b[p], b[i]);
        Z inv = a[i][i].inv();
        for (int k = i; k < n; k++) a[i][k] = a[i][k] * inv;
        for (int k = 0; k < n; k++) b[i][k] = b[i][k] * inv;
        for (int j = 0; j < n; j++)
            if (j != i && a[j][i].v)
            {
                Z f = a[j][i];
                for (int k = i; k < n; k++) a[j][k] = a[j][k] - f * a[i][k];
                for (int k = 0; k < n; k++) b[j][k] = b[j][k] - f * b[i][k];
            }
    }
    return b;
}
```

模板题使用：Luogu P4783

题意：给定 n 阶方阵，求模 1000000007 的逆矩阵；不可逆输出 No Solution。输入按行读取，输出按行给出逆矩阵，不对原矩阵进行覆盖。

所需模板：ModInt（第 90 页，18.1.2 节）、matrix_inverse（第 217 页，21.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4783

```
int main()
{
    using Z = ModInt<1000000007>;
    int n;
    scanf("%d", &n);
    vector<vector<Z>> a(n, vector<Z>(n));
    for (auto &row : a)
        for (auto &x : row)
        {
            int v;
            scanf("%d", &v);
            x = v;
        }
    auto b = matrix_inverse<1000000007>(a);
    if (!b)
        puts("No Solution");
    else
        for (auto &row : *b)
        {
            for (auto x : row) printf("%d ", x.v);
            puts("");
        }
}
```

21.1.3 Max-plus 矩阵幂

功能： $(\max,+)$ 半环上的 $n\times n$ 矩阵乘法与快速幂：a_{ij} 是从状态 i 一步转移到 j 的收益（无此转移为 `neg`），`power(k)` 的 (i,j) 元即从 i 出发恰好走 k 步到达 j 的最大总收益。

核心思路： 把普通矩阵乘法中的乘换成加、加换成取最大：$c_{ij}=\max_t(a_{it}+b_{tj})$。该运算满足结合律，故可按二进制拆分指数做快速幂。`neg` 充当 $-\infty$，乘法中跳过等于 `neg` 的项，避免把不可达状态加成有限值；单位阵对角为 0、其余为 `neg`。数值用 `__int128` 存储。

接口契约： a[i][j] 为转移收益，不存在的边为 neg；power(k) 求恰好 k 步的最优转移，单位矩阵对角为 0。所有有限中间值绝对值小于 2^119。O(n^3 log k)；2025 成都 B 的转移类型，具体题目的建模仍需调用者完成。

```
struct MaxPlusMatrix
{
    using I = __int128_t;
    static constexpr I neg = -(I(1) << 120);
    int n;
    vector<vector<I>> a;

    MaxPlusMatrix(int n, bool identity = false) : n(n), a(n, vector<I>(n, neg))
    {
        if (identity)
            for (int i = 0; i < n; i++) a[i][i] = 0;
    }

    MaxPlusMatrix operator*(const MaxPlusMatrix &b) const
    {
        assert(n == b.n);
        MaxPlusMatrix c(n);
        for (int i = 0; i < n; i++)
            for (int k = 0; k < n; k++)
                if (a[i][k] != neg)
                    for (int j = 0; j < n; j++)
                        if (b.a[k][j] != neg)
                            c.a[i][j] = max(c.a[i][j], a[i][k] + b.a[k][j]);
        return c;
    }

    MaxPlusMatrix power(unsigned long long e) const
    {
        MaxPlusMatrix b = *this, r(n, true);
        for (; e; e >>= 1, b = b * b)
            if (e & 1) r = r * b;
        return r;
    }
};
```

21.1.4 线性代数与生成树

域上一致方程组的解集为特解加零空间，零空间维数为未知数个数减秩。浮点秩依赖阈值；模素数消元精确；合数模数下不可随意除以非零元。无向图 Laplacian 对角为度数、非对角为负的边重数，删去同一行列的行列式为生成树数；自环不计，重边累加，单点为 1。有向图须另定入度/出度 Laplacian 与树的方向。

21.2 行列式

21.2.1 任意正模数行列式：欧几里得消元

问题描述：给定整数方阵 A 与任意正整数模数 M（可为合数），求 $\det A \bmod M$。

核心思路：合数模下主元未必可逆，因此用辗转相除代替除法：处理第 i 列时，对下方每一行 j，只要 $a_{ji} \neq 0$，就让第 i 行减去 $\lfloor a_{ii}/a_{ji} \rfloor$ 倍的第 j 行，再交换两行，相当于对 (a_{ii}, a_{ji}) 做欧几里得算法，直到 $a_{ji} = 0$。行相减不改变行列式，每次交换把答案取负。一列消完后若 $a_{ii} = 0$ 则行列式为 0，否则乘进答案，最终得到上三角矩阵对角元的乘积。

接口契约：determinant_mod(a,mod) 返回 det(a) 的最小非负余数。a 为 long long 方阵，元素支持完整 signed 64 位；1≤mod≤LLONG_MAX，可以是合数。不使用模逆元，允许不可逆主元。矩阵按值传入，调用后原矩阵不变；空矩阵返回 1%mod，mod=1 时返回 0。换行改变符号，最终乘上三角矩阵的对角元素。所有乘法先转 int128。$O(n^3 \log \text{mod})$ 时间、$O(n^2)$ 空间；素数模通常优先用已有高斯行列式。

```
long long determinant_mod(vector<vector<long long>> a, long long mod)
{
    assert(mod >= 1);
    int n = a.size();
    for (auto &row : a)
    {
        assert((int)row.size() == n);
        for (auto &x : row)
        {
            x %= mod;
            if (x < 0) x += mod;
        }
    }
    long long answer = 1 % mod;
    for (int i = 0; i < n; i++)
    {
        for (int j = i + 1; j < n; j++)
            while (a[j][i])
            {
                long long quotient = a[i][i] / a[j][i];
                for (int k = i; k < n; k++)
                {
                    __int128 value = (__int128)a[i][k] - (__int128)quotient * a[j][k];
                    value %= mod;
                    if (value < 0) value += mod;
                    a[i][k] = (long long)value;
                }
                swap(a[i], a[j]);
                answer = answer ? mod - answer : 0;
            }
        if (a[i][i] == 0) return 0;
        answer = (__int128)answer * a[i][i] % mod;
    }
    return answer;
}
```

模板题使用：Library Checker matrix_det_arbitrary_mod

题意：给方阵和模数，求行列式的最小非负余数，模数可以为合数。函数无需模逆元，按值接收矩阵，示例用 move 交出数据。0 阶方阵的行列式为 1，再对模数取余。

所需模板：determinant_mod（第 220 页，21.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/matrix_det_arbitrary_mod

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    long long mod;
    cin >> n >> mod;
    vector<vector<long long>> a(n, vector<long long>(n));
    for (auto &row : a)
        for (auto &x : row) cin >> x;
    cout << determinant_mod(move(a), mod) << '\n';
}
```

21.2.2 任意正模数行列式

实现见第 220 页。合数模不能除以任意非零主元；只用倍加行与交换行：对第 i 列两元素 a, b，$R_i \leftarrow R_i - \lfloor a/b \rfloor R_j$ 后交换，按欧几里得终止；交换改变符号。消成上三角后乘对角元与符号；某列剩余全零直接返回零。支持任意正 signed 64 位模数与输入，中间量用有符号 128 位。空矩阵返回 $1 \bmod m$；模 1 返回零。时间 $O(n^3 \log m)$，空间 $O(n^2)$。合数模的矩阵树使用第 51.2 节（第 573 页）的接口。

21.3 线性基

21.3.1 64 位异或线性基

功能：维护一组 64 位无符号整数的异或线性基，支持插入、判断某数能否由若干元素异或得到、求 x 与任意子集异或的最大值、第 k 小的异或值与最小值，以及合并两组线性基。

核心思路：`a[i]` 保存最高位为 i 的基向量。插入时从高位到低位，该位已有基就异或消去，遇到空位就落户；消成 0 说明线性相关，置 `dependent`（存在异或为 0 的非空子集）。最大值从高到低贪心取 $\max(x, x \oplus a_i)$。第 k 小先由 `rebuild` 化成简化阶梯形（每个主元位只在自己的向量中为 1），按主元升序存入 `p`，则把序号的各二进制位对应的 `p[i]` 异或起来就是答案；能得到 0 时 0 占第 1 名，序号先减一。

接口契约：值类型 U 为 unsigned long long，完整支持 64 位。默认空基；重新赋值 XorBasis() 清空。insert/Insert 返回是否增秩；contains/Contains 判断可表示性（允许空子集，所以 0 总可表示）。query/Query(x=0) 返回 x 与所张成空间异或后的最大无符号值。kth/Kth(k,nonempty=true) 返回第 k 小的不同异或值，1-based，参数 k 为 ___uint128_t，非法序号返回 nullopt。默认只允许非空输入子集，0 仅在 dependent 为真时出现；传 false 则包括空子集所得的 0。minimum/Minimum 同此约定，空输入的非空最小值不存在。merge/Merge 将另一基所代表的输入多重集并入；重复元素和自合并可新增异或为 0 的非空方案。rank 是维数，dependent 表示存在非空零异或子集。插入、成员判断、最大值为 O(64)；合并 O(64^2)。kth 会在需要时 rebuild/Rebuild 规范化缓存，首次有效查询 O(64^2)，后续 O(64)；仅新增独立向量使缓存失效。所有成员仅供只读，不要直接改数组或标志。

```
struct XorBasis
{
    using U = unsigned long long;
    using Wide = __uint128_t;
    U a[64]{};
    vector<U> p;
    int rank = 0;
    bool dependent = false, dirty = false;

    bool insert(U x)
    {
        for (int i = 63; i >= 0; i--)
            if (x >> i & 1)
            {
                if (!a[i])
                {
                    a[i] = x;
                    rank++;
                    dirty = true;
                    return true;
                }
                x ^= a[i];
            }
        dependent = true;
        return false;
    }
```

```
28      bool contains(U x) const
29      {
30          for (int i = 63; i >= 0; i--)
31              if (x >> i & 1) x ^= a[i];
32          return x == 0;
33      }
34
35      U query(U x = 0) const
36      {
37          for (int i = 63; i >= 0; i--) x = max(x, x ^ a[i]);
38          return x;
39      }
40
41      void rebuild()
42      {
43          vector<U> b(a, a + 64);
44          for (int i = 0; i < 64; i++)
45              if (b[i])
46                  for (int j = i + 1; j < 64; j++)
47                      if (b[j] >> i & 1) b[j] ^= b[i];
48          p.clear();
49          for (U x : b)
50              if (x) p.push_back(x);
51          dirty = false;
52      }
```

```
    // k is 1-based; nonempty excludes the empty input subset, not value zero.
    optional<U> kth(Wide k, bool nonempty = true)
    {
        if (!k) return nullopt;
        Wide index = k;
        if (!nonempty || dependent) index--;
        if (index >= (Wide(1) << rank)) return nullopt;
        if (dirty) rebuild();
        U answer = 0;
        for (int i = 0; i < rank; i++)
            if (index >> i & 1) answer ^= p[i];
        return answer;
    }

    optional<U> minimum(bool nonempty = true) { return kth(1, nonempty); }

    void merge(const XorBasis &other)
    {
        dependent |= other.dependent;
        for (U x : other.a)
            if (x) insert(x);
    }
};
```

模板题使用：Luogu P3812

题意：给定一组可能重复的非负整数，选任意子集使异或和最大。逐个 insert 后 query()；使用 unsigned long long 读取和输出。

所需模板：XorBasis（第 222 页，21.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3812

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n;
    std::cin >> n;
    XorBasis basis;
    for (int i = 0; i < n; i++)
    {
        unsigned long long x;
        std::cin >> x;
        basis.insert(x);
    }
    std::cout << basis.query() << '\n';
}
```

21.3.2 带位置限制的异或线性基

功能：按标签（如数组下标）插入 64 位整数；查询时给定下界 `lower`，只允许使用标签不小于它的元素，判断能否异或出 x、求最大异或值或张成空间的维数。常用于按右端点扫描，回答区间内选数异或的问题。

核心思路：仍是按最高位存放的线性基，另用 `pos[i]` 记录 `a[i]` 的标签。插入时若当前向量的标签大于该位已有基的标签，就交换二者，让标签大的留在该位，换出来的旧向量继续往低位消元。这样每一位都尽量保留标签大的向量，标签不小于 `lower` 的那些基恰好张成标签不小于 `lower` 的输入所张成的空间；查询时跳过 `pos[i]` 小于下界的位，其余与普通线性基相同。

接口契约：默认构造为空，重新赋值 PositionBasis() 可清空。insert/Insert(x,at) 插入 unsigned long long 值及非负 int 标签，支持完整 64 位，标签可重复、乱序。contains/Contains(x,lower=0) 判断 x 能否由标签至少为 lower 的输入异或得到；query/Query(lower=0,x=0) 返回 x 与这些输入子集异或后的最大值；rank/Rank(lower=0) 返回对应空间维数。lower 为非负 long long，可大于所有 int 标签，空集仍可产生 0。全部操作 O(64)，空间 O(64)。消元时同一主元优先保留标签较大的向量，较旧向量继续消元；按阈值筛选后仍是正确线性基。标签是组合可用范围的界，不是选点方案；数组仅供只读。区间 [l,r] 应按右端点排序询问，扫描并仅插入到 r，再查询 lower=l；接口本身没有上界，插入超过 r 后不能只用 lower 恢复旧区间。若需保留历史，可复制每个前缀的对象，但空间为 O(64n)。不提供非空零异或判定或第 k 小；普通线性基的 dependent 不能直接移植到所有阈值。

```
struct PositionBasis
{
    using U = unsigned long long;
    U a[64]{};
    int pos[64];

    PositionBasis() { fill(pos, pos + 64, -1); }
```

```
    void insert(U x, int at)
    {
        assert(at >= 0);
        for (int i = 63; i >= 0; i--)
            if (x >> i & 1)
            {
                if (!a[i])
                {
                    a[i] = x;
                    pos[i] = at;
                    return;
                }
                if (at > pos[i])
                {
                    swap(a[i], x);
                    swap(pos[i], at);
                }
                x ^= a[i];
            }
    }

    bool contains(U x, long long lower = 0) const
    {
        assert(lower >= 0);
        for (int i = 63; i >= 0; i--)
            if ((x >> i & 1) && pos[i] >= lower) x ^= a[i];
        return x == 0;
    }

    U query(long long lower = 0, U x = 0) const
    {
        assert(lower >= 0);
        for (int i = 63; i >= 0; i--)
            if (pos[i] >= lower) x = max(x, x ^ a[i]);
        return x;
    }

    int rank(long long lower = 0) const
    {
        assert(lower >= 0);
        int answer = 0;
        for (int i = 0; i < 64; i++) answer += pos[i] >= lower;
        return answer;
    }
};
```

21.3.3 异或线性空间求交

问题描述：给定两个 $\mathbb{F}_2^{64}$ 子空间 A, B 的线性基（XorBasis），求交空间 $A \cap B$ 的一组基。

核心思路：以 A 的基作为初始主元表 pivot，逐个插入 B 的基向量。每个主元附带 from_b，记录它来自 B 的部分（A 的主元为 0）；当前向量 x 异或 pivot[i] 时，part 同步异或 from_b[i]，始终保持 part $\in B$ 且 $x \oplus \text{part} \in A$。若 x 被消成 0，则 part 同属 A 和 B，插入结果；否则 x 成为新主元，并记下它的 part。

代码依赖：XorBasis（第 222 页，21.3.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：basis_intersection(a,b) 返回 span(a) 与 span(b) 的交空间的一组独立生成元，结果类型 XorBasis。支持完整 64 位无符号值，不修改输入，也不要求先 rebuild。使用第一个空间的主元，消去第二个空间的向量，同时记录来自第二个空间的部分；消元为零时，该部分属于交空间。时间 $O(64^2)$，额外空间 O(64)。输出按独立生成元解释：dependent 为 false，输入的重复次数及非空零异或关系不传播；表示的是子空间，不是两组原始输入元素的集合交。枚举整个交空间应调用 result.kth(k,false)，包含零；零空间返回空基，rank=0、contains(0)=true。

```
XorBasis basis_intersection(const XorBasis &a, const XorBasis &b)
{
    using U = unsigned long long;
    U pivot[64], from_b[64]{};
    copy(a.a, a.a + 64, pivot);
    XorBasis result;
    for (U value : b.a)
    {
        U x = value, part = value;
        for (int i = 63; i >= 0 && x; i--)
            if (x >> i & 1)
            {
                if (!pivot[i])
                {
                    pivot[i] = x;
                    from_b[i] = part;
                    break;
                }
                x ^= pivot[i];
                part ^= from_b[i];
            }
        if (!x && part) result.insert(part);
    }
    return result;
}
```

模板题使用：LC Intersection of F2 Vector Spaces

题意：多组给两个 F2 向量空间的基，输出交空间的一组基。T≤100000，n,m≤30，输入独立且小于 2^30。把输入插入两个 XorBasis，调用求交，输出 rank 和 a 中的非零主元；答案不唯一，不能逐字比较不同实现输出。模板本身支持 64 位和相关生成元，属于题外扩展。零维输出 0。

所需模板：XorBasis（第 222 页，21.3.1 节）、basis_intersection（第 227 页，21.3.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/intersection_of_f2_vector_spaces

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int tests;
    std::cin >> tests;
    while (tests--)
    {
        XorBasis a, b;
        int n, m;
        std::cin >> n;
        for (int i = 0; i < n; i++)
        {
            unsigned long long x;
            std::cin >> x;
            a.insert(x);
        }
        std::cin >> m;
        for (int i = 0; i < m; i++)
        {
            unsigned long long x;
            std::cin >> x;
            b.insert(x);
        }
        auto result = basis_intersection(a, b);
        std::cout << result.rank;
        for (auto x : result.a)
            if (x) std::cout << ' ' << x;
        std::cout << '\n';
    }
}
```

21.3.4 Zassenhaus：同时求和空间与交空间

问题描述：给定两个 $\mathbb{F}_2^{64}$ 子空间 A, B 的线性基，一次求出和空间 $A+B$ 与交空间 $A\cap B$ 各自的一组基。

核心思路：Zassenhaus 算法：A 的基向量 x 写成 128 位的 (x,x)，B 的基向量 y 写成 $(y,0)$（高 64 位在前），全部插入一个按最高位存放的 128 位线性基。任一行形如 $(a\oplus b,a)$，$a\in A$、$b\in B$。主元在高半部的行，其高半部组成 $A+B$ 的基；主元在低 64 位的行高半部为 0，即 $a=b$，其低半部同属两个空间，组成 $A\cap B$ 的基。

代码依赖：XorBasis（第 222 页，21.3.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：basis_sum_intersection(a,b) 返回和空间基、交空间基这一对结果。和空间由两组基共同张成，一般不是两个空间的集合并；两结果都是独立生成元，其 dependent 均为 false。将 a 的向量 x 编成 (x,x)，b 的向量 y 编成 (y,0)，用 ___uint128_t 储存高低各 64 位并按高位到低位消元。高半部非零的行投影构成和空间的基，高半部为零的行取低半部构成交空间的基。必须先提升到 128 位再左移 64；不能用 64 位变量移位 64。时间 $O(128^2)$，额外空间 O(128)。结果满足 dim(A+B)+dim(A 交 B)=dim(A)+dim(B)，但维数恒等式只是检查条件，不能单独证明返回向量正确。包含空空间、相同空间和输入别名。

```cpp
// Return independent generators of (sum space, intersection space).
pair<XorBasis, XorBasis> basis_sum_intersection(const XorBasis &a, const XorBasis &b)
{
    using W = __uint128_t;
    W pivot[128]{};
    auto insert = [&](W x)
    {
        for (int i = 127; i >= 0 && x; i--)
            if (x >> i & 1)
            {
                if (!pivot[i])
                {
                    pivot[i] = x;
                    return;
                }
                x ^= pivot[i];
            }
    };
    for (auto x : a.a) insert((W(x) << 64) | x);
    for (auto x : b.a) insert(W(x) << 64);
    XorBasis sum, intersection;
    for (int i = 0; i < 128; i++)
        if (pivot[i])
        {
            if (i < 64)
                intersection.insert((unsigned long long)pivot[i]);
            else
                sum.insert((unsigned long long)(pivot[i] >> 64));
        }
    return {sum, intersection};
}
```

模板题使用：LC Intersection / Zassenhaus

题意：同一线性空间求交题，约束与前例相同。Zassenhaus 返回 {和空间, 交空间}，本题仅输出 second 的 rank 和非零主元，不能据此声称 first 也获线上验证。和空间由独立本地测试检查。两部分均为独立生成元；无需对输入枚举全部异或值。

所需模板：XorBasis（第 222 页，21.3.1 节）、basis_sum_intersection（第 229 页，21.3.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/intersection_of_f2_vector_spaces

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int tests;
    std::cin >> tests;
    while (tests--)
    {
        XorBasis a, b;
        int n, m;
        std::cin >> n;
        for (int i = 0; i < n; i++)
        {
            unsigned long long x;
            std::cin >> x;
            a.insert(x);
        }
        std::cin >> m;
        for (int i = 0; i < m; i++)
        {
            unsigned long long x;
            std::cin >> x;
            b.insert(x);
        }
        auto result = basis_sum_intersection(a, b).second;
        std::cout << result.rank;
        for (auto x : result.a)
            if (x) std::cout << ' ' << x;
        std::cout << '\n';
    }
}
```

21.4 特征多项式

21.4.1 有限状态计数到线性递推：先证明阶数，再用 BM

只问极大长度 N 的一个计数时，用有限状态 DP 生成少量项，再用 BM 求标量递推外推；须先有阶数上界。

21.4.1.1 先把状态、方向和输出写清楚

$v_n \in R^d$ 为长度 n 时各状态计数的列向量，A_{ij} 为从状态 j 一步到状态 i 的方案数（重边须累加）：

$$v_{n+1} = Av_n, \qquad v_n = A^n v_0, \qquad a_n = u^{\mathsf{T}} v_n.$$

R 可为整数环或模 m 的交换环；按行向量建模时整体转置。**前提：**状态维数 d 有限且固定，转移矩阵固定，更新与输出均为线性（min-plus 不算）。固定周期可加相位状态，仿射项可加恒为 1 的坐标，阶数按扩充后的维数计。

21.4.1.2 Cayley–Hamilton 给出阶数至多 d

$\chi_A(t) = \det(tI - A) = t^d + q_{d-1}t^{d-1} + \cdots + q_0$，$\chi_A(A) = 0$ 对交换环成立，故

$$\boxed{a_{n+d} = -\sum_{i=0}^{d-1} q_i a_{n+i} \quad (n \geq 0).}$$

按 $a_n = \sum_{j=1}^{k} c_{j-1} a_{n-j}$ 约定取 $k = d$、$c_j = -q_{d-1-j}$。调用 BM 只需上界，不必算出特征多项式。

21.4.1.3 为什么前 2d 项足以恢复这个标量序列

在素数域 $\mathbb{F}_p$ 中，精确的前 $2d$ 项 $a_0, \ldots, a_{2d-1}$ 经 BM 得到的最短递推（阶 $k \leq d$）对整个序列成立；$2d-1$ 项不够。恢复的只是该 u, v_0 下的标量递推，不是矩阵恒等式。BM 返回的末尾零系数不能删除。额外项交叉检查不代替阶数证明。

21.4.1.4 模型：不含连续子串 101 的二进制串

状态为与禁串前缀的最长匹配 $\varepsilon, 1, 10$：

$$A = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 0 \end{pmatrix}, \quad v_0 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad u = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}, \qquad a_n = 2a_{n-1} - a_{n-2} + a_{n-3}.$$

调用链：`c=berlekamp_massey(s,p)`，`init` 取前 `c.size()` 项，`recurrence_nth(init,c,N,p)`，N 为 `unsigned long long` 零基下标。样本与系数须先正规化到 $[0, p)$ 并在目标素数域内恢复；模一个素数得到的递推不保证在别的模数下有效。

21.4.1.5 按目标选算法，并把采样成本算进去

e 为非零转移数：采样前 $2d$ 项 $O(d(e+d))$ 时间、$O(e+d)$ 空间（稠密 $O(d^3)$）；BM $O(d^2 + d\log p)$ 时间、$O(d)$ 空间。

- 直接 DP：$O(N(e+d))$，适合 N 不大或要整个前缀。
- 矩阵幂：第 215 页 `ModMatrix<p>::power(A,N)`，$O(d^3 \log(N+1))$ 时间、$O(d^2)$ 空间，可复用于不同初值/输出。
- 小阶：第 179 页 `recurrence_nth(init,c,N,p)`，$O(k^2 \log(N+1))$ 时间、$O(k)$ 空间。
- 大阶：第 180 页 `BostanMori<p,g>::recurrence(init,c,N)`，$O(k \log(k+1) \log(N+1))$ 时间、$O(k)$ 空间；参数转换为其 `Poly`，p 素数、g 原根，非初值查询要求 $2k+1 \leq$ `NttConvolution<p,g>::max_size`。

BM 见第 259 页。两种远项接口要求初值**恰好** k 项，下标为 `unsigned long long`；$k=0$ 返回 0，$N<k$ 返回初值。

合数模数： BM 用 x^{p-2} 求逆，只支持素数模数。已知正确递推时，`recurrence_nth` 支持 $2 \le m \le$ `INT_MAX` 的合数模数（初值和系数在 $[0,m)$）；`ModMatrix` 乘法与非负次幂也无需逆元。求逆矩阵与 NTT 版 Bostan–Mori 不适用。

第二十二章　线性规划

22.1　单纯形法

22.1.1　线性规划：两阶段单纯形

问题描述：线性规划：在约束 $Ax \le b$、$x \ge 0$ 下最大化 $c^{\mathsf{T}}x$，判断有最优解、不可行或无界；有最优解时给出最优值与一组最优解。

核心思路：单纯形表 `d` 的前 m 行是约束（初始以松弛变量为基，最后一列为右端），第 m 行是目标 $-c$，第 $m+1$ 行是第一阶段目标；第 n 列是唯一的人工变量（编号 $n+m$），系数为 -1。若有 $b_i < 0$，先把人工变量与右端最小的行换基，使所有右端非负，再让人工变量尽量取 0：做不到就不可行，否则若它仍在基中则换出。第二阶段在原目标行上 `improve`。`improve` 选编号最小的负检验数列入基（Bland 规则），按最小比值、平局取基变量编号最小的行出基，找不到出基行即无界。

思路中的情况	代码中的处理	行
建表	约束行写入 A、人工变量列 -1 与右端 b	26
存在负右端	人工变量与右端最小的行换基	86
第一阶段结束	人工变量无法降为 0，判不可行	90
人工变量仍在基中	在该行找非零列换出	93
第二阶段	优化原目标，失败即无界	101
选入基列	负检验数中取变量编号最小者	60
选出基行	最小比值，平局取基变量编号最小	74

接口契约：Simplex(A,b,c) 求 max c · x，约束 Ax≤b、x≥0；A 为 m×n，b 长 m，c 长 n，变量/约束均 0 基。构造后只调用一次 solve()，返回 status、value、x；optimal 时 value 与 x 有效，infeasible/unbounded 时只读 status。允许负 b、冗余约束及零维；内部表与 pivot/improve 仅供实现使用。

long double、绝对 eps=1e-12。两阶段人工变量法；按变量编号选入基、最小比值并按基变量编号解平局。精确运算下 Bland 规则避免循环，浮点容差下不保证任意病态输入的正确分类或终止；勿把 AC 视为精确 LP 保证。先缩放数量级，求解后复核原约束和目标值。每次换基 O((m+1)(n+1))，空间同阶；总换基次数最坏指数级。

```
struct Simplex
{
    using Real = long double;
    static constexpr Real eps = 1e-12L;
    enum Status { optimal, infeasible, unbounded };
    struct Result
    {
        Status status;
        Real value;
        vector<Real> x;
    };
    int n, m;
    vector<int> basic, nonbasic;
    vector<vector<Real>> d;

    Simplex(const vector<vector<Real>> &a, const vector<Real> &b,
            const vector<Real> &c) : n(c.size()), m(b.size()),
            basic(m), nonbasic(n + 1), d(m + 2, vector<Real>(n + 2))
    {
        assert((int)a.size() == m);
        for (int i = 0; i < m; i++)
        {
            assert((int)a[i].size() == n);
            for (int j = 0; j < n; j++) d[i][j] = a[i][j];
            basic[i] = n + i;
            d[i][n] = -1;
            d[i][n + 1] = b[i];
        }
        iota(nonbasic.begin(), nonbasic.end(), 0);
        nonbasic[n] = n + m;
        for (int j = 0; j < n; j++) d[m][j] = -c[j];
        d[m + 1][n] = 1;
    }
```

换基（接上页结构体）：

```
    void pivot(int r, int s)
    {
        Real inv = 1 / d[r][s];
        for (int i = 0; i < m + 2; i++)
        {
            if (i == r) continue;
            Real scale = d[i][s] * inv;
            for (int j = 0; j < n + 2; j++)
                if (j != s) d[i][j] -= scale * d[r][j];
        }
        for (int j = 0; j < n + 2; j++)
            if (j != s) d[r][j] *= inv;
        for (int i = 0; i < m + 2; i++)
            if (i != r) d[i][s] *= -inv;
        d[r][s] = inv;
        swap(basic[r], nonbasic[s]);
    }
```

表的含义 第 i 行表示基本变量等于右端值减去非基本变量的线性组合。目标行同样表示当前目标常数减去约化系数乘变量。负约化系数表示增加该变量可改善目标；仅系数为正的约束限制增加量，故用右端除系数的最小值选择离基。若没有限制行则沿该方向无界。

人工变量与两阶段 对每条约束引入同一个人工变量 t，系数为-1。从最负的右端行让 t 入基即得到可行起点，第一阶段最大化-t。最优值小于零说明原问题无解；为零时把仍在基内的人工变量换出，零冗余行可以保留。原目标行始终参加换基，第二阶段禁止人工变量入基，直接优化原目标。

选择入基与出基变量（接上页结构体）：

```
bool improve(int objective)
{
    while (true)
    {
        int s = -1;
        for (int j = 0; j <= n; j++)
        {
            if (nonbasic[j] == n + m) continue;
            if (d[objective][j] >= -eps) continue;
            if (s == -1 || nonbasic[j] < nonbasic[s]) s = j;
        }
        if (s == -1) return true;
        int r = -1;
        for (int i = 0; i < m; i++)
        {
            if (d[i][s] <= eps) continue;
            if (r == -1)
            {
                r = i;
                continue;
            }
            Real x = d[i][n + 1] / d[i][s];
            Real y = d[r][n + 1] / d[r][s];
            if (x < y - eps || (abs(x - y) <= eps && basic[i] < basic[r]))
                r = i;
        }
        if (r == -1) return false;
        pivot(r, s);
    }
}
```

两阶段求解与方案恢复（接上页结构体）：

```
    Result solve()
    {
        int r = -1;
        for (int i = 0; i < m; i++)
            if (r == -1 || d[i][n + 1] < d[r][n + 1]) r = i;
        if (r != -1 && d[r][n + 1] < -eps)
        {
            pivot(r, n);
            improve(m + 1);
            if (d[m + 1][n + 1] < -eps) return {infeasible, 0, {}};
            for (int i = 0; i < m; i++)
            {
                if (basic[i] != n + m) continue;
                int s = -1;
                for (int j = 0; j <= n; j++)
                    if (abs(d[i][j]) > eps &&
                        (s == -1 || nonbasic[j] < nonbasic[s])) s = j;
                if (s != -1) pivot(i, s);
            }
        }
        if (!improve(m)) return {unbounded, 0, {}};
        vector<Real> x(n);
        for (int i = 0; i < m; i++)
            if (basic[i] < n) x[basic[i]] = d[i][n + 1];
        return {optimal, d[m][n + 1], x};
    }
};
```

退化与数值 在精确运算下，入基选最小变量编号，最小比值并列时选最小基本变量编号，是 Bland 防循环规则。这里按 eps 判断约化系数和比值并列，所以不承诺浮点下严格防循环。极小系数、严重消去或尺度悬殊都可能改变分类；先对模型缩放，并用原输入复核返回方案。需要精确判定时应使用有理数算法，不能仅缩小 eps。

模板题使用：P13337 线性规划

题意：n,m≤100，整数系数绝对值 ≤100，题目保证随机数据。最大化 c · x，Ax≤b、x≥0。输出最优值及任一最优解，或 Infeasible/Unbounded；允许绝对或相对误差 1e-6。

所需模板：Simplex（第 233 页，22.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P13337

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<long double> c(n), b(m);
    vector<vector<long double>> a(m, vector<long double>(n));
    for (auto &v : c) cin >> v;
    for (int i = 0; i < m; i++)
    {
        for (auto &v : a[i]) cin >> v;
        cin >> b[i];
    }
    Simplex lp(a, b, c);
    auto result = lp.solve();
    if (result.status == Simplex::infeasible) cout << "Infeasible\n";
    else if (result.status == Simplex::unbounded) cout << "Unbounded\n";
    else
    {
        cout << fixed << setprecision(12) << result.value << '\n';
        for (auto x : result.x) cout << x << ' ';
        cout << '\n';
    }
}
```

第二十三章　概率论

23.1　随机变量的数字特征

23.1.1　期望选型：计数拆指标，等待拆尾和

先确定随机对象：均匀随机排列、逐步等概率转移和等概率选择最终方案是不同模型，不能用路径条数代替路径概率。

计数型： $X=\sum_i I_i$，$I_i=[A_i]$，则 $\mathbb{E}X=\sum_i \Pr(A_i)$，不要求独立。但一般 $\mathbb{E}(XY)\neq\mathbb{E}X\,\mathbb{E}Y$；方差为

$$\operatorname{Var}\left(\sum_i X_i\right)=\sum_i \operatorname{Var}(X_i)+2\sum_{i<j}\operatorname{Cov}(X_i,X_j),$$

两两独立才可消去协方差。

等待时间或最大值：非负整数随机变量有

$$\mathbb{E}X=\sum_{k\geq 1}\Pr(X\geq k)=\sum_{k\geq 0}\Pr(X>k).$$

覆盖过程：均匀抽 n 种券，已有 i 种时等待期望 $n/(n-i)$，收齐期望 $n\sum_{j=1}^{n}1/j$；非均匀抽券时“已见种类数”不是充分状态。

23.1.2　条件期望：先定义状态，再解自环

状态须保存决定未来分布的信息。终点 $E_t=0$，第一步分析：

$$E_i=\sum_j p_{ij}(c_{ij}+E_j)=b_i+\sum_j p_{ij}E_j,\qquad b_i=\sum_j p_{ij}c_{ij}.$$

命中概率：好终点 1、坏终点 0，$h_i=\sum_j p_{ij}h_j$；可能不终止时取最小非负解。只有自环且 $p_{ii}<1$ 时 $E_i=\big(b_i+\sum_{j\neq i}p_{ij}E_j\big)/(1-p_{ii})$，其余按依赖序 DP；存在 $i\to j\to i$ 时不是 DAG。成功率 $q>0$ 的重复试验期望 $1/q$。

有环时：取起点可达的非终点状态，转移矩阵 Q，解 $(I-Q)E=b$，稠密消元 $O(s^3)$ 时间、$O(s^2)$ 空间；DAG、链、带状或可按 SCC 分块时先用结构。实数接口见GaussReal（第 255 页，25.3.3 节），秩按阈值判定，概率极小时可能病态。有限状态、固定转移概率下，期望有限的条件：可达区域（仅正概率边）中没有封闭的非吸收强连通分量。无限状态时“几乎必然终止”不推出期望有限。

23.1.3　模概率：可逆的不只是输入分母

既约分数 a/b 仅当 $p\nmid b$ 时映射为 $a\,b^{-1}\bmod p$；模值不能比较大小，模值为零不表示概率为零。直接转模方程的充分条件：所有转移概率与费用的分母可逆，且 $I-Q$（含自环的 $1-p_{ii}$）模 p 可逆。实数下可逆不保证模下可逆；未约分表达式应先在有理数层面约分。合数模 m 要求 $\gcd(b,m)=1$，不能用费马小定理求逆。

接口：第 18.1.2 节（第 90 页）`ModInt<p>` 的 `try_inv()` 用欧几里得，不可逆返回空值；`inv()` 与除法要求可逆。第 25.3.2 节（第 252 页）`GaussMod<p>::solve(a,n)` 要求 p 为素数，传 s 行 $s+1$ 列增广矩阵 $[I-Q\mid b]$ 及变量数 s；先查 `consistent`，再查 `rank==s`，才读 `particular`。

第二十四章　博弈论

24.1　公平组合游戏

24.1.1　SG 的前提、递推与异或策略

前提：双人轮流、无偏（同一状态双方合法操作相同）、无随机与隐藏信息、不能行动者输、状态图为有限 DAG。

$$g(u) = \operatorname{mex}\{g(v) : u \to v\}, \qquad \operatorname{mex} \varnothing = 0.$$

按逆拓扑序计算；d 个后继时 $g(u) \le d$，用时间戳标记，$O(|V| + |E|)$ 时间与空间。$g(u) = 0$ 当且仅当先手必败；SG 值不是胜率或步数。

不交和（一步只动一个子游戏且互不影响）：$G = g_1 \oplus \cdots \oplus g_k$，$G = 0$ 先手必败。$G \neq 0$ 时取 G 最高位上为 1 的 g_i，在该子游戏中找 SG 值为 $g_i \oplus G$ 的后继。可同时改多堆或相互禁止操作时不是不交和。

24.1.2　减法游戏小例与三类失效边界

一堆 n 颗，每次取 S 中数量：$g(0) = 0$，

$$g(n) = \operatorname{mex}\{g(n - s) : s \in S,\ s \le n\}.$$

$g(n) \le |S|$；长度为 $\max S$ 的整个窗口重复后才保证最终周期，几项相同不是周期证明。

- **反常玩法（misère）**：普通 SG 终点规则与异或结论不可直接搬用。
- **有环游戏（loopy）**：mex 可能无解且可能有和局；须按题目处理重复局面或加入步数使之成为 DAG。
- **有偏游戏（partizan）**：按完整状态（含行动方）做胜负搜索，一般不能异或普通 SG 值。

24.1.3　巴什博弈

内容：一堆 n 颗，每次取 $1 \sim k$ 颗，取走最后一颗者胜。
结论：$(k+1) \mid n$ 时先手必败，否则先手取 $n \bmod (k+1)$ 颗必胜。
思路：对方取 t 颗，自己就取 $k+1-t$ 颗，使每轮共取 $k+1$ 颗，局面始终保持 $(k+1) \mid n$。
适用：单堆、每次取数量为连续区间 $[1, k]$；若允许集合不是区间，按减法游戏求 SG（第 240 页）。

24.1.4　Nim 游戏

内容：若干堆，每次从一堆取任意正数颗，取走最后一颗者胜。
结论：各堆异或为 0 时先手必败。
思路：异或非零时，取异或最高位为 1 的一堆 a_i，改成 $a_i \oplus X < a_i$ 即可使异或变 0；异或为 0 时任何一步都会使其非零。
适用：多个独立子游戏且每个子游戏的 SG 值等于堆大小；一般的不交和先求各自 SG 再异或（第 240 页）。

24.1.5　威佐夫博弈

内容：两堆 $a \le b$，每次从一堆取任意正数颗，或从两堆取相同正数颗，取走最后一颗者胜。
结论：先手必败当且仅当 $a = \lfloor (b-a)\varphi \rfloor$，$\varphi = \frac{1+\sqrt{5}}{2}$。整数计算用 $\lfloor d\varphi \rfloor = \left(d + \lfloor \sqrt{5d^2} \rfloor\right)/2$（整除），$\lfloor \sqrt{\cdot} \rfloor$ 用整数开方，避免浮点误差。
思路：必败态 $(a_d, a_d + d)$ 每个差 d 恰有一个，且所有 a_d 与 $a_d + d$ 恰好不重不漏地覆盖全体正整数（Beatty 序

列），所以从必败态出发的单堆或双堆操作都到达不了另一个必败态。
适用：两堆且允许“同时等量取”；三堆及以上或其他组合操作没有此闭式，需打表或求 SG。

24.1.6 斐波那契博弈

内容：一堆 $n \geq 2$ 颗，先手第一次取 $1 \sim n-1$ 颗，之后每次至少取 1 颗、至多取上一次对手所取数量的 2 倍，取走最后一颗者胜。
结论：n 是斐波那契数时先手必败。否则把 n 写成不相邻斐波那契数之和（Zeckendorf 表示），先手取其中最小的一项必胜。
思路：最小项 F_i 与次小项至少差一个下标，有 $2F_i < F_{i+2}$，对手取不完下一项。剩余部分可以看成若干段独立的斐波那契数局面，于是总能把“剩下斐波那契数”的局面留给对手。
适用：每次可取量依赖上一次取量的倍数限制；限制换成其他倍数时没有此结论，按状态 $(n, \text{上限})$ 记忆化搜索。

24.1.7 阶梯博弈

内容：第 $1..k$ 级台阶上各有若干石子，每次把某一级（$i \geq 1$）的任意正数颗移到第 $i-1$ 级，移到第 0 级（地面）后不再参与，不能操作者负。
结论：等价于只看奇数级台阶的 Nim，即各奇数级石子数的异或为 0 时先手必败。
思路：从奇数级往下移，相当于在 Nim 中取石子。对手若把偶数级的石子移到奇数级，自己就把同样数量再往下移一级，使奇数级恢复原状。
适用：石子只能单向移动、且终点不计分的模型，如树上把子结点的石子移给父亲（按深度奇偶看）、数轴上的硬币只能左移（相邻间隔就是“台阶”）。

24.1.8 反 Nim（取走最后一颗者负）

内容：Nim 规则不变，但取走最后一颗者负。
结论：先手必胜当且仅当以下二者之一成立：所有堆都 ≤ 1 且异或为 0（即 1 的堆数为偶数）；或至少有一堆 > 1 且异或不为 0。
思路：全为 1 时只看奇偶。存在大于 1 的堆时，先手按普通 Nim 取胜，只在最后一个大于 1 的堆处改为留 0 或 1，使剩下的 1 的堆数为奇数。
适用：反常规则的 Nim；一般反常游戏不能套 SG 异或，需要 SJ 定理的前提或直接搜索（第 240 页）。

24.1.9 Nim-k（每次至多动 k 堆）

内容：多堆，每次选 $1 \sim k$ 堆，每堆各取任意正数颗，取走最后一颗者胜。
结论：把各堆写成二进制，若每一位上为 1 的堆数都是 $k+1$ 的倍数，则先手必败。$k = 1$ 即 Nim。
思路：与 Nim 相同，只是把“按位异或”换成“按位求和模 $k+1$”：从这种局面一步最多改动 k 堆，不可能让每一位都保持模 $k+1$ 为 0；反之总能从高位到低位调整至多 k 堆，恢复为 0。
适用：一次可同时操作多堆、且各堆操作相同的模型。

24.1.10 树上删边（Green Hackenbush）

内容：有根树，每次删一条边，与根不连通的部分一起删除，不能操作者负。
结论：$g(v) = \bigoplus_{c \in \text{son}(v)} \big(g(c)+1\big)$，叶子为 0，$g(\text{根}) = 0$ 时先手必败。多棵树把各根的值异或。
思路：一条长为 ℓ 的链等价于大小为 ℓ 的 Nim 堆。同一结点下的几条分支互不影响，可以合并成一条长度为各自 SG 异或的链（Colon 原理）；在子树上方加一条边，SG 值加 1。
适用：删边或删子树的树上博弈。图中有环时需要“融合原理”，把环缩成链或自环后再算，不能直接套用此公式。

24.1.11 二分图博弈（不重复走点）

内容：无向图中一枚棋子放在 s，双方轮流把它移到一个未访问过的相邻点（s 视为已访问），不能移动者负。
结论：先手必胜当且仅当 s 在每一个最大匹配中都被匹配，即删去 s 后最大匹配变小。对一般图也成立：二分图用第 544 页 `BipartiteMatching`，一般图用第 551 页 `Blossom`，求一次最大匹配，删去 s 后再求一次比较。
思路：若 s 在每个最大匹配中，任取一个最大匹配 M，先手总沿 M 中当前点的匹配边走。后手若走到 M 的未匹配点，从 s 到该点是一条交错路，沿它翻转得到同样大小、却不含 s 的最大匹配，矛盾；所以后手每走一步，先手都还有匹配边可走。反之若有不含 s 的最大匹配 M，后手总沿 M 的匹配边应对；先手若走到 M 的未匹配点，从 s 到该点两端都未匹配，是增广路，与 M 最大矛盾，所以先手先无路可走。
适用：棋盘上走马、单词接龙等“走过的格子不能再走”的两人移动博弈；多个起点或询问时，逐个判断“该点是否一定被匹配”。

24.1.12 对称策略：环形取石子

内容：n 枚硬币围成一圈，每次取 1 枚或相邻 2 枚，取走最后一枚者胜。
结论：$n \le 2$ 时先手胜，$n \ge 3$ 时后手胜。
思路：先手第一步把环断成一条链，后手在链的正中取 1 或 2 枚，把链分成长度相等的两段，之后先手在一段中怎么取，后手就在另一段的对称位置照做。
适用：局面可以被对手的一步变成两个完全对称的部分（棋盘中心对称、环上断成两条等长链等）；对称位置必须总是可走，且自己的一步不能同时破坏两边。

24.1.13 策略窃取：Chomp

内容：$r \times c$ 的巧克力，左上角一格有毒。每次选一块，吃掉它以及右下方所有剩余块，吃到有毒那块者负。
结论：只要 $rc > 1$，先手必胜（不给出具体走法）。
思路：设先手先吃右下角一块。若后手对此有必胜应对，这个应对局面先手第一步就能直接走出来，因为吃掉的部分包含右下角。于是先手可以窃取后手的策略，所以后手不可能有必胜策略；有限且无和局时，先手必胜。
适用：只判断谁胜、且存在“先走一步不会更坏”的空步论证（如棋盘对称、多走一步无害）时；需要具体方案时仍需搜索。

第二十五章　数值算法

25.1　插值

25.1.1　拉格朗日插值与连续点优化

问题描述：已知 n 个横坐标两两不同的点 (x_i, y_i)，求唯一的次数小于 n 的多项式 f 在给定 k 处的值（模素数 p）。预处理一次后可多次查询；横坐标恰为 $0, 1, \ldots, n-1$ 时有更快的专用构造。

核心思路：拉格朗日公式写成 $f(k) = \sum_i w_i \prod_{j \neq i}(k - x_j)$，其中 $w_i = y_i / \prod_{j \neq i}(x_i - x_j)$ 只与样本有关，构造时存入 `w`：任意横坐标逐对相乘后求一次逆；横坐标连续时分母为 $i!\,(n-1-i)!$ 并带符号 $(-1)^{n-1-i}$，用逆阶乘表直接得到。`query` 先求前缀积 `pre`，再从右往左维护后缀积 `suffix`，两者相乘即去掉第 i 个因子的乘积。全程不对 $k - x_i$ 求逆，所以 k 恰为样本点时也正确。

接口契约：Lagrange(xs,ys,p) 使用任意横坐标，xs、ys 为等长 0-based vector<long long>；Lagrange(ys,p) 专用于横坐标 0..n-1；单次查询直接用 lagrange_consecutive。重新构造赋值可更换数据。要求 p 为素数，2≤p≤INT_MAX，点数 n≤p，任意横坐标取模后两两不同；不是仅要求原整数不同。query(k) 返回次数小于 n 的插值多项式在 k 处的值，允许 k、xs、ys 为完整 long long 范围，内部先归一化；空数据约定零多项式。通用预处理 O(n^2+n log p)，连续点预处理 O(n+log p)，每次查询 O(n)，空间 O(n)。查询利用前缀积与滚动后缀积，无需求逆，因此 k 与样本点模 p 相等时也正确。若求 1^d+⋯+k^d，要求 0≤d≤p-2，先计算 0..d+1 的前缀和作为 ys，再构造 Lagrange(ys,p)；用快速幂生成这些样本另花 O(d log(d+1))。仅处理素数域中的精确插值，不适用于浮点曲线拟合或重复节点的导数约束。

```
struct Lagrange
{
    using ll = long long;
    int n, mod;
    vector<ll> x, w;

    ll norm(ll a) const
    {
        a %= mod;
        if (a < 0) a += mod;
        return a;
    }

    ll power(ll a, int b) const
    {
        ll result = 1;
        while (b)
        {
            if (b & 1) result = result * a % mod;
            a = a * a % mod;
            b >>= 1;
        }
        return result;
    }

    Lagrange(const vector<ll> &xs, const vector<ll> &ys, int p)
    {
        n = ys.size();
        mod = p;
```

```
        assert(p >= 2 && n <= p && xs.size() == ys.size());
        x.resize(n);
        w.resize(n);
        for (int i = 0; i < n; i++) x[i] = norm(xs[i]);
        for (int i = 0; i < n; i++)
        {
            ll denominator = 1;
            for (int j = 0; j < n; j++)
                if (i != j) denominator = denominator * norm(x[i] - x[j]) % mod;
            assert(denominator != 0);
            w[i] = norm(ys[i]) * power(denominator, mod - 2) % mod;
        }
    }

    Lagrange(const vector<ll> &ys, int p)
    {
        n = ys.size();
        mod = p;
        assert(p >= 2 && n <= p);
        x.resize(n);
        w.resize(n);
        if (!n) return;
        vector<ll> invfac(n, 1);
        ll factorial = 1;
        for (int i = 1; i < n; i++) factorial = factorial * i % mod;
        invfac[n - 1] = power(factorial, mod - 2);
        for (int i = n - 1; i > 0; i--) invfac[i - 1] = invfac[i] * i % mod;
        for (int i = 0; i < n; i++)
        {
            x[i] = i;
            w[i] = norm(ys[i]) * invfac[i] % mod * invfac[n - 1 - i] % mod;
            if ((n - 1 - i) & 1) w[i] = norm(-w[i]);
        }
    }

    ll query(ll k) const
    {
        k = norm(k);
        vector<ll> pre(n + 1, 1);
        for (int i = 0; i < n; i++) pre[i + 1] = pre[i] * norm(k - x[i]) % mod;
        ll answer = 0, suffix = 1;
        for (int i = n - 1; i >= 0; i--)
        {
            answer = (answer + w[i] * pre[i] % mod * suffix) % mod;
            suffix = suffix * norm(k - x[i]) % mod;
        }
        return answer;
    }
};
```

模板题使用：Luogu P4781

题意：给出 n 个横坐标模素数后互不相同的点，求次数小于 n 的插值多项式在 k 处的值，模 998244353。使用任意点构造函数。

所需模板：Lagrange（第 243 页，25.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4781

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n;
    long long k;
    std::cin >> n >> k;
    std::vector<long long> x(n), y(n);
    for (int i = 0; i < n; i++) std::cin >> x[i] >> y[i];
    Lagrange poly(x, y, 998244353);
    std::cout << poly.query(k) << '\n';
}
```

25.1.2 横坐标连续的 O(n) 拉格朗日插值

问题描述：已知次数小于 n 的多项式 f 在 $0,1,\ldots,n-1$ 处的值，求单点值 $f(k) \bmod p$。常用于 $\sum_{i=1}^{k} i^d$ 这类关于 k 是多项式的量：先暴力算出前若干项，再插值到很大的 k。

核心思路：$f(k)=\sum_i y_i \prod_{j\neq i} \frac{k-j}{i-j}$。分子用前缀积 `pre` 与后缀积 `suf` 拼出 $\prod_{j\neq i}(k-j)$；分母 $\prod_{j\neq i}(i-j) = i!\,(n-1-i)!\,(-1)^{n-1-i}$，只用一次快速幂求 $(n-1)!$ 的逆，再递推出逆阶乘表 `ifac`，按 $n-1-i$ 的奇偶决定加或减该项。分子不做除法，因此 k 落在插值点上无需特判。

接口契约：lagrange_consecutive(ys,k,p) 已知 ys[i]=f(i)（i=0..n-1，deg f<n），返回 f(k) mod p。p 为素数，2≤p≤INT_MAX，n≤p；ys、k 可为任意 long long，内部取模；空 ys 返回 0。用前缀积、后缀积得所有 j≠i 的 (k-j) 之积，分母为 i!(n-1-i)! 并带符号 (-1)^(n-1-i)，只求一次逆，k 落在插值点上也正确。O(n+log p) 时间、O(n) 空间。横坐标为 a,a+1,…时把 k 换成 k-a；求 1^d+…+k^d 时取 0..d+1 的前缀和共 d+2 个样本。需要对同一组样本多次查询时用 Lagrange(ys,p)。

```cpp
// ys[i] = f(i) for i = 0..n-1 (deg f < n); returns f(k) mod prime p, n <= p. O(n + log p).
long long lagrange_consecutive(const vector<long long> &ys, long long k, int p)
{
    int n = ys.size();
    assert(p >= 2 && n <= p);
    if (!n) return 0;
    auto norm = [&](long long a) { return (a % p + p) % p; };
    auto power = [&](long long a, long long e)
    {
        long long r = 1;
        for (a = norm(a); e; e >>= 1, a = a * a % p)
            if (e & 1) r = r * a % p;
        return r;
    };
    k = norm(k);
    vector<long long> pre(n + 1, 1), suf(n + 1, 1), ifac(n, 1);
    for (int i = 0; i < n; i++) pre[i + 1] = pre[i] * norm(k - i) % p;
    for (int i = n - 1; i >= 0; i--) suf[i] = suf[i + 1] * norm(k - i) % p;
    long long fact = 1;
    for (int i = 1; i < n; i++) fact = fact * i % p;
    ifac[n - 1] = power(fact, p - 2);
    for (int i = n - 1; i > 0; i--) ifac[i - 1] = ifac[i] * i % p;
    long long answer = 0;
    for (int i = 0; i < n; i++)
    {
        // prod_{j != i} (k - j) / prod_{j != i} (i - j)
        long long term = norm(ys[i]) * pre[i] % p * suf[i + 1] % p * ifac[i] % p * ifac[n - 1 - i] % p;
        answer = ((n - 1 - i) & 1 ? answer - term + p : answer + term) % p;
    }
    return answer;
}
```

25.2 数值积分

25.2.1 有工作上限的自适应 Simpson 积分

问题描述：数值计算定积分 $\int_a^b f(x)\,\mathrm{d}x$：在绝对误差目标 `eps` 下自适应地加密采样，同时限制递归深度和总求值次数；返回近似值、误差估计、求值次数，以及是否满足局部误差条件。

核心思路：Simpson 公式 $S(l,r)=\frac{r-l}{6}\big(f(l)+4f(m)+f(r)\big)$。递归函数 `rec` 拿到区间、已算好的三个函数值与整体估计 s，只新算两个四分点，得到左右两半的估计 s_l, s_r；令 $\delta = s_l + s_r - s$，若 $|\delta|/15$ 不超过容差，就接受修正值 $s_l + s_r + \delta/15$；否则两半各分一半容差、深度减一继续递归。中点无法再分、求值预算将用完或出现非有限值时立即停止并标记未满足。反向区间先交换端点，最后取负。

思路中的情况	代码中的处理	行
零长度区间	不调用 f，直接返回 0	20
整体初值	求端点与中点的值和整区间估计	30
无法细分或预算不足	停止，返回粗估计且标记未满足	38
局部误差达标	接受修正值	48
深度用尽或容差下溢	返回当前修正值但标记未满足	49
继续细分	左右两半各用一半容差递归后相加	50

接口契约：integrate(f,a,b,eps,depth=20,limit=100000)，long double 绝对容差 eps>0，有限端点，0≤depth≤60、limit≥0。返回 value、error 估计、evaluations 与 met；met 仅指局部估计条件满足，不是实际精度保证。反向区间变号，零长度不调用 f；预算/深度/中点停滞/非有限计算会停止且 met=false。正常有限样本下保留粗或修正估计；无初始估计时 value 为 NaN；不可把失败值当答案。f 须返回确定值，异常原样传播，不能启用 fast-math。复用样本，O(求值数) 次固定字长运算、O(depth+1) 栈；需 numeric 的 midpoint、cmath、limits、utility。

```
struct AdaptiveSimpson
{
    using ld = long double;
    struct Result
    {
        ld value, error;
        int evaluations;
        bool met;
    };

    template<class F>
    static Result integrate(F &&f, ld a, ld b, ld eps,
                            int depth = 20, int limit = 100000)
    {
        const ld inf = numeric_limits<ld>::infinity();
        const ld nan = numeric_limits<ld>::quiet_NaN();
        if (!isfinite(a) || !isfinite(b) || !isfinite(eps) || eps <= 0 ||
            depth < 0 || depth > 60 || limit < 0)
            return {nan, inf, 0, false};
        if (a == b) return {0, 0, 0, true};
        if (limit < 3) return {nan, inf, 0, false};
        bool reverse = a > b;
        if (reverse) swap(a, b);
        auto area = [](ld l, ld r, ld u, ld v, ld w)
        {
            return (r - l) / 6 * (u + 4 * v + w);
        };
        ld fa = f(a), fm = f(midpoint(a, b)), fb = f(b);
        int used = 3;
        ld whole = area(a, b, fa, fm, fb);
        if (!isfinite(fa) || !isfinite(fm) || !isfinite(fb) || !isfinite(whole))
            return {nan, inf, used, false};
```

采样细分与预算合并（接上页同一 integrate 函数）：

```
        auto rec = [&](auto &&self, ld l, ld r, ld u, ld v, ld w,
                       ld s, ld tol, int left) -> Result
        {
            ld m = midpoint(l, r), x = midpoint(l, m), y = midpoint(m, r);
            if (m == l || m == r || x == l || x == m || y == m || y == r ||
                used > limit - 2)
                return {s, inf, 0, false};
            ld fx = f(x), fy = f(y);
            used += 2;
            ld sl = area(l, m, u, fx, v), sr = area(m, r, v, fy, w);
            ld sum = sl + sr, delta = sum - s;
            ld value = sum + delta / 15, error = abs(delta) / 15;
            if (!isfinite(fx) || !isfinite(fy) || !isfinite(sl) ||
                !isfinite(sr) || !isfinite(value) || !isfinite(error))
                return {s, inf, 0, false};
            if (error <= tol) return {value, error, 0, true};
            if (!left || tol / 2 == 0) return {value, error, 0, false};
            auto p = self(self, l, m, u, fx, v, sl, tol / 2, left - 1);
            auto q = self(self, m, r, v, fy, w, sr, tol / 2, left - 1);
            value = p.value + q.value;
            error = p.error + q.error;
            if (!isfinite(value)) return {s, inf, 0, false};
            return {value, error, 0, p.met && q.met && error <= tol};
        };
        auto answer = rec(rec, a, b, fa, fm, fb, whole, eps, depth);
        if (reverse) answer.value = -answer.value;
        answer.evaluations = used;
        return answer;
    }
};
```

25.2.2 数值积分：局部估计、误差预算与采样盲区

接口见第 25.2.1 节（第 247 页）。前提：定积分存在，采样点有定义；先按间断、尖峰、振荡分段，端点奇异须变量变换，不能只减小 eps。

$$S(a,b)=\frac{b-a}{6}\big(f(a)+4f(m)+f(b)\big),\qquad \int_a^b f-S(a,b)=-\frac{(b-a)^5}{2880}f^{(4)}(\xi),$$

对至多三次多项式精确。$S_2=S(a,m)+S(m,b)$，$\delta=S_2-S(a,b)$，误差指标 $|\delta|/15$，修正值 $S_2+\delta/15$；递归时误差预算对半分。error 是指标之和，`met` 不是实际误差证书；光滑函数也可能被初始五点漏采样。

eps 为正的绝对容差；depth=0 仍做根的五点比较；limit 为回调次数上限。深度耗尽、容差下溢、预算不足、四分点与端点重合或非有限值都令 `met=false`，此时 value 不可直接输出（可能为 NaN）。零长度不调用函数，反向区间取负。

25.2.3 数值算法与几何

排序必须是严格弱序，不能以 “差小于 eps 则相等” 作比较器。叉积与行列式易抵消；整数题优先精确整数判定。自适应积分要求函数足够规则，奇点、间断、强振荡先分段。三分要求单峰（整数域最后枚举剩余区间）；二分答案要求判定单调。

25.3 高斯消元

25.3.1 压位模 2 高斯消元与全部解

问题描述：给定 $\mathbb{F}_2$ 上 m 个方程、n 个未知数的线性方程组（每行一个 01 串，末位为右端项），判断是否有解；有解时求出一个特解和齐次方程组解空间的一组基，全部解就是特解异或任意一组基向量。

核心思路：每行按 64 位压成 `unsigned long long` 数组后做 Gauss–Jordan 消元：逐列寻找主元行换到当前行 `row`，并用它消去**所有**其他行在该列的 1，`pos[col]` 记下主元所在行。消元后每个主元变量只出现在自己那一行。秩以下的行若右端为 1 则无解。特解令自由变量全为 0，主元变量取所在行的右端值；对每个自由列 j，令 $x_j=1$、其余自由变量为 0，主元变量取所在行第 j 列的值，得到一个零空间基向量。

思路中的情况	代码中的处理	行
压位读入	第 j 个字符写入第 `j/64` 个字的第 `j%64` 位	23
找主元	在当前行及以下找该列为 1 的行，找不到则该列是自由列	34
消去其他行	主元行异或进所有该列为 1 的其他行，只需从第 `col/64` 个字开始	38
判无解	零行的右端为 1	43
特解	主元变量取其行右端值	46
零空间基	每个自由列生成一个基向量	51

接口契约：GaussXor::solve(a,n)，a 为 m 行 n+1 列的 01 字符串增广矩阵，0≤n<INT_MAX，最末字符为右端项。返回 consistent、系数秩 rank、01 字符串特解 particular 与零空间基 kernel；所有解为特解异或任意基子集。无解时解数组为空；rank 仍为系数秩。有解时 kernel.size()=n-rank，唯一解的 kernel 为空。允许空方程组或零变量，原输入不变。自由列按原列号递增，基在各自自由列为 1、其他自由列为 0；没有最少 1 个数保证。时间 O(mn+mn min(m,n)/64+n^2)，除输入外空间 O(m ceil((n+1)/64)+n^2)；输出本身可能有 n^2 字符。全程精确异或，不用 eps。

```
struct GaussXor
{
    struct Solution
    {
        bool consistent;
        int rank;
        string particular;
        vector<string> kernel;
    };

    // m rows of n+1 characters: coefficients followed by the right-hand side.
    static Solution solve(const vector<string> &a, int n)
    {
        assert(n >= 0);
        int m = a.size(), w = n / 64 + 1, row = 0;
        vector<vector<unsigned long long>> b(m, vector<unsigned long long>(w));
        for (int i = 0; i < m; i++)
        {
            assert(a[i].size() == (size_t)n + 1);
            for (int j = 0; j <= n; j++)
            {
                assert(a[i][j] == '0' || a[i][j] == '1');
                if (a[i][j] == '1') b[i][j / 64] |= 1ULL << (j % 64);
            }
        }
        auto get = [&](int i, int j)
        {
            return b[i][j / 64] >> (j % 64) & 1;
        };
        vector<int> pos(n, -1);
        for (int col = 0; col < n && row < m; col++)
        {
            int p = row;
            while (p < m && !get(p, col)) p++;
            if (p == m) continue;
            swap(b[p], b[row]);
            for (int i = 0; i < m; i++)
                if (i != row && get(i, col))
                    for (int j = col / 64; j < w; j++) b[i][j] ^= b[row][j];
            pos[col] = row++;
        }
        for (int i = row; i < m; i++)
            if (get(i, n)) return {false, row, {}, {}};
```

构造特解与零空间基（接上页 solve）（接上页同一结构体）：

```
        Solution ans{true, row, string(n, '0'), {}};
        for (int j = 0; j < n; j++)
            if (pos[j] != -1) ans.particular[j] += get(pos[j], n);
        for (int j = 0; j < n; j++)
            if (pos[j] == -1)
            {
                string v(n, '0');
                v[j] = '1';
                for (int k = 0; k < n; k++)
                    if (pos[k] != -1) v[k] += get(pos[k], j);
                ans.kernel.push_back(v);
            }
        return ans;
    }
};
```

模板题使用：Library Checker system_of_linear_equations_mod_2

题意：给定模 2 矩阵 A 和右端 b，输出无解-1，或解空间维数、一个特解和全部零空间基。输出维数是变量数减系数秩，不是 rank。输入每行是连续 01 字符，先把 b 追加到对应行；行列数各至 4096。

所需模板：GaussXor（第 250 页，25.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/system_of_linear_equations_mod_2

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int m, n;
    cin >> m >> n;
    vector<string> a(m);
    for (auto &s : a) cin >> s;
    string b;
    cin >> b;
    for (int i = 0; i < m; i++) a[i] += b[i];
    auto sol = GaussXor::solve(a, n);
    if (!sol.consistent)
    {
        cout << -1 << '\n';
        return 0;
    }
    cout << sol.kernel.size() << '\n';
    cout << sol.particular << '\n';
    for (auto &v : sol.kernel) cout << v << '\n';
}
```

25.3.2 素数模高斯消元与全部解

问题描述：求解模素数 p 的线性方程组 $Ax = b$（给出 m 行增广矩阵、n 个未知数）：判断是否有解，有解时给出系数矩阵的秩、一个特解和齐次方程组解空间的一组基，从而描述全部解。

核心思路：高斯—若尔当消元化为简化行阶梯形：逐列找本列非零的行作主元，换到当前行 `row`，整行乘主元的逆归一，再消去其他所有行的该列。`where[col]` 记录主元列所在的行，没有主元的列是自由变量。消元后秩以

下各行若右端非零则无解。特解令自由变量为 0，主元变量取所在行的右端；每个自由变量 j 生成一个基向量：第 j 维为 1，各主元变量取 $-a_{\text{where}[k],j}$。

代码依赖：ModInt（第 90 页，18.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约： GaussMod<p>::solve(a,n) 接收 m 行 n+1 列增广矩阵，p 必须是素数，n≥0。返回 consistent、系数矩阵秩 rank、特解 particular 和零空间基 kernel；解为特解加基向量的任意线性组合。无解时解数组为空；空方程组需显式给出变量数。O(m n min(m,n)+n^2) 时间，矩阵及解空间输出 O(mn+n^2) 空间。只依赖 ModInt，不含行列式或矩阵树。

```
template <int mod> struct GaussMod
{
    using Z = ModInt<mod>;
    using Matrix = vector<vector<Z>>;

    struct Solution
    {
        bool consistent;
        int rank;
        vector<Z> particular;
        Matrix kernel;
    };

    // Augmented m x (n+1), prime modulus. Empty system needs explicit n.
    static Solution solve(Matrix a, int n)
    {
        int m = (int)a.size(), row = 0;
        vector<int> where(n, -1);
        for (const auto &v : a) assert((int)v.size() == n + 1);
        for (int col = 0; col < n && row < m; col++)
        {
            int p = row;
            while (p < m && !a[p][col].v) ++p;
            if (p == m) continue;
            swap(a[p], a[row]);
            Z inv = a[row][col].inv();
            for (int j = col; j <= n; j++) a[row][j] = a[row][j] * inv;
            for (int i = 0; i < m; i++)
                if (i != row && a[i][col].v)
                {
                    Z f = a[i][col];
                    for (int j = col; j <= n; j++) a[i][j] = a[i][j] - f * a[row][j];
                }
            where[col] = row++;
        }
        for (int i = row; i < m; i++)
            if (a[i][n].v) return {false, row, {}, {}};
```

```
        Solution ans{true, row, vector<Z>(n), {}};
        for (int j = 0; j < n; j++)
            if (where[j] != -1) ans.particular[j] = a[where[j]][n];
        for (int j = 0; j < n; j++)
            if (where[j] == -1)
            {
                vector<Z> v(n);
                v[j] = 1;
                for (int k = 0; k < n; k++)
                    if (where[k] != -1) v[k] = Z(0) - a[where[k]][j];
                ans.kernel.push_back(v);
            }
        return ans;
    }
};
```

模板题使用：Library Checker system_of_linear_equations

题意：模 998244353 下解 n 个方程、m 个未知数的线性方程组。输入先给系数矩阵，再给常数列。无解输出 -1；否则输出解空间维数、一个特解和齐次解基。

所需模板：ModInt（第 90 页，18.1.2 节）、GaussMod（第 252 页，25.3.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/system_of_linear_equations

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using G = GaussMod<998244353>;
    int n, m;
    cin >> n >> m;
    G::Matrix a(n, vector<G::Z>(m + 1));
    for (auto &row : a)
        for (int j = 0; j < m; j++) cin >> row[j].v;
    for (auto &row : a) cin >> row[m].v;
    auto s = G::solve(move(a), m);
    if (!s.consistent)
    {
        cout << -1 << '\n';
        return 0;
    }
    cout << s.kernel.size() << '\n';
    auto print = [&](const vector<G::Z> &row)
    {
        for (int j = 0; j < m; j++) cout << row[j].v << (j + 1 == m ? '\n' : ' ');
    };
    print(s.particular);
    for (auto &row : s.kernel) print(row);
}
```

25.3.3 实数高斯消元与数值解空间

问题描述：用 `long double` 浮点数求解实系数线性方程组 $Ax = b$：在容差 `eps` 下判断是否相容，给出数值秩、一个特解和齐次解空间的一组基。

核心思路：先把每行按其系数最大绝对值缩放，使各方程量级一致。然后逐列做部分选主元的高斯—若尔当消元：在未用行中取本列绝对值最大者为主元；若它不超过 `eps`，视该列为零，把活动行的这一列清零后跳过（成为自由变量）；否则换行、主元行归一，再消去其他所有行的该列，`where` 记录主元所在行。剩余行右端绝对值超过 `eps` 判为不相容；特解令自由变量为 0，每个自由变量生成一个基向量，主元变量从对应行读出。

接口契约：solve(a, n, eps) 接收 m×(n+1) 增广矩阵，0≤n<INT_MAX、m≤INT_MAX，输入/中间值须有限可表示，0<eps<1。long double 精度依平台；先按每行系数最大绝对值归一化（不含右端项），再部分选主元；绝对值≤eps 的候选列在活动行中明确置零。对所有非零消元因子更新，不以 eps 跳过。返回 consistent、rank、particular 和 kernel：均描述最终数值消元模型，不是原系统精确可解性/秩的证书；剩余右端项绝对值 ≤eps 按零处理。原全零系数行保留原始右端尺度，n=0 也用此约定。相容时自由变量取零给特解，kernel 列出自由变量方向；任意大线性组合的原方程误差不受 eps 统一保证。非相容时两个解数组为空；输入按值传入不变。主消元通常 O(mn min(m,n)) 时间，另有初始化；O(m(n+1)+n) 工作空间，另有 O(n(n−rank)) 解基输出。近阈值、病态或非有限过程不能当成数学无解；须结合原方程残差与问题条件判断。正常浮点语义，禁 fast-math；assert 仅调试诊断，非发布模式运行时校验。

```
struct GaussReal
{
    using R = long double;
    using Matrix = vector<vector<R>>;

    struct Solution
    {
        bool consistent;
        int rank;
        vector<R> particular;
        Matrix kernel;
    };

    // Numerical classification after coefficient-row scaling, not exact rank.
    static Solution solve(Matrix a, int n, R eps)
    {
        assert(0 <= n && n < INT_MAX && a.size() <= (size_t)INT_MAX);
        assert(isfinite(eps) && 0 < eps && eps < 1);
        int m = a.size(), row = 0;
        vector<int> where(n, -1);
        for (auto &v : a)
        {
            assert(v.size() == (size_t)n + 1);
            R s = 0;
            for (int j = 0; j <= n; j++)
            {
                assert(isfinite(v[j]));
                if (j < n) s = max(s, abs(v[j]));
            }
            if (s > 0)
                for (R &x : v)
                {
                    x /= s;
                    assert(isfinite(x));
                }
        }
```

主元消元与数值解空间（接上页同一 solve 函数）：

```
        for (int col = 0; col < n && row < m; col++)
        {
            int p = row;
            for (int i = row + 1; i < m; i++)
                if (abs(a[i][col]) > abs(a[p][col])) p = i;
            if (abs(a[p][col]) <= eps)
            {
                for (int i = row; i < m; i++) a[i][col] = 0;
                continue;
            }
            swap(a[p], a[row]);
            R pivot = a[row][col];
            for (int j = col + 1; j <= n; j++)
            {
                a[row][j] /= pivot;
                assert(isfinite(a[row][j]));
            }
            a[row][col] = 1;
            for (int i = 0; i < m; i++)
                if (i != row && a[i][col] != 0)
                {
                    R f = a[i][col];
                    for (int j = col + 1; j <= n; j++)
                    {
                        a[i][j] -= f * a[row][j];
                        assert(isfinite(a[i][j]));
                    }
                    a[i][col] = 0;
                }
            where[col] = row++;
        }
        for (int i = row; i < m; i++)
            if (abs(a[i][n]) > eps) return {false, row, {}, {}};
        Solution ans{true, row, vector<R>(n), {}};
        for (int j = 0; j < n; j++)
            if (where[j] != -1) ans.particular[j] = a[where[j]][n];
        for (int j = 0; j < n; j++)
            if (where[j] == -1)
            {
                vector<R> v(n);
                v[j] = 1;
                for (int k = 0; k < n; k++)
                    if (where[k] != -1) v[k] = -a[where[k]][j];
                ans.kernel.push_back(v);
            }
        return ans;
    }
};
```

25.3.4 模线性方程组：工厂工期

POJ2947：星期一为 0、星期日为 6，产品 j 在记录 i 中出现 a_{ij} 次（重复累加），

$$\sum_j a_{ij}x_j \equiv \text{finish}_i - \text{start}_i + 1 \pmod 7.$$

用第 252 页的模高斯消元，模 7，增广矩阵 $m \times (n+1)$。先判 `consistent`（假则无解），再看秩小于 n 为多解，否则特解中小于 3 的分量加 7。合数模数不能直接套用此消元。

25.3.5 模 2 压位消元与完整解空间

`GaussXor::solve(a,n)` 输入 m 行 $n+1$ 列 01 串，末列为右端项。自由变量全设零得特解；每次一个自由变量设一得零空间基。全部解为特解异或任意基子集，维数 $n - rank$。每行 $\lfloor n/64 \rfloor + 1$ 个字；无解与零变量有解都可能返回空特解，须查 `consistent`。输出完整基最坏 n^2 个字符；特解不保证一的个数最少。

25.3.6 模 2 消元后的最少开关用法

每个方格一个变量，系数为自身及四邻，右端项为初始颜色。解集为特解异或零空间基任意子集，须枚举取一最少者；无解输出 `inf`。自由维数 d 时代价 $O(n^2 2^d)$；对此四邻棋盘 $d \le n$，不可推广到任意方程组。

第二十六章　Berlekamp–Massey 算法

26.1　Berlekamp–Massey：最短线性递推

问题描述：给定模素数的有限序列 $s_0, \ldots, s_{N-1}$，求阶数最小的线性递推 $s_i = \sum_{j=1}^{k} c_{j-1} s_{i-j}$（对所有 $i \geq k$ 成立）。常与求线性递推第 n 项配合，用于“打表找递推”。

核心思路：维护当前连接多项式 `c`（$c_0 = 1$，满足 $\sum_{j=0}^{L} c_j s_{i-j} = 0$，$L$ 即 `len`），以及上次长度变化前的多项式 `b`、它的失配值 `last` 和相对位移 `shift`。读入 s_i 时算失配 d；$d = 0$ 只增加位移。否则令 $c \leftarrow c - \frac{d}{\text{last}} x^{\text{shift}} b$ 消去本次失配；若 $2L \leq i$，长度必须增长为 $i + 1 - L$，修改前的 `c` 成为新的 `b`。最后把 $c_1, \ldots, c_L$ 取负即为递推系数。

接口契约：berlekamp_massey(s,p) 要求 p 为素数，2≤p≤INT_MAX，输入均在 [0,p)；返回 c，使 s[i]=∑ c[j-1]s[i-j] (j=1..k，i≥k)。空序列或全零序列返回空系数。有限前缀可能对应多种未来，不能无条件外推；若已知原序列满足阶数至多 d 的齐次线性递推，提供至少 2d 项才能恢复可外推的递推。模运算次数 O(Nk+N log p)，后项来自逐次求逆；空间 O(N)。不适用于合数模数或浮点拟合。

```
// Prime p; normalized input. s[i] = sum(c[j-1] * s[i-j]), i >= c.size().
vector<int> berlekamp_massey(const vector<int> &s, int p = 998244353)
{
    assert(p >= 2);
    vector<int> c{1}, b{1};
    int len = 0, shift = 1, last = 1;
    for (int i = 0; i < (int)s.size(); i++)
    {
        assert(0 <= s[i] && s[i] < p);
        long long d = s[i];
        for (int j = 1; j <= len; j++) d = (d + 1LL * c[j] * s[i - j]) % p;
        if (d == 0)
        {
            shift++;
            continue;
        }
        long long inv = 1, x = last;
        for (int e = p - 2; e; e >>= 1)
        {
            if (e & 1) inv = inv * x % p;
            x = x * x % p;
        }
        long long factor = d * inv % p;
        auto old = c;
        if (c.size() < b.size() + shift) c.resize(b.size() + shift);
        for (int j = 0; j < (int)b.size(); j++)
            c[j + shift] = (c[j + shift] - factor * b[j] % p + p) % p;
        if (2 * len <= i)
        {
            len = i + 1 - len;
            b = old;
            last = d;
            shift = 1;
```

```
        }
        else
            shift++;
    }
    c.resize(len + 1);
    c.erase(c.begin());
    for (int &x : c) x = x == 0 ? 0 : p - x;
    return c;
}
```

模板题使用：Library Checker find_linear_recurrence

题意：给模 998244353 下的有限序列，求最短线性递推。c[0] 乘前一项，输出阶数及系数；空序列或全零序列可以是 0 阶。递推只保证对已给序列成立，不能仅凭前缀推定未知后续。

所需模板：berlekamp_massey（第 259 页，26.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/find_linear_recurrence

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<int> a(n);
    for (int &x : a) cin >> x;
    auto c = berlekamp_massey(a);
    cout << c.size() << '\n';
    for (int x : c) cout << x << ' ';
    cout << '\n';
}
```

联合使用示例见第 180 页（Luogu P5487），其中直接调用本模板。

第五部分

数据结构

第二十七章　哈希表

27.1　GNU gp_hash_table 与 cc_hash_table

功能：整数键的哈希表：gp_map 是 GNU pb_ds 的开放寻址表，cc_map 是链式表，用法与 unordered_map 基本相同（下标访问、查找、插入、删除）。

核心思路：两者只是别名，关键在于自定义哈希 hash64：先给键加上程序启动时由时钟产生的随机种子，再做 splitmix64 的三轮异或移位与乘法混合，使高低位充分扩散，避免针对固定哈希函数构造的冲突数据卡掉表。

接口契约：GNU libstdc++ 扩展，包含 ext/pb_ds/assoc_container.hpp。gp_map<K,V> 为探测哈希表，cc_map<K,V> 为碰撞链接哈希表；K 为不超过 64 位的整数，复用下方 hash64。两者支持 operator[]、find、insert、erase(key)、clear、size、empty；insert 不覆盖已有键，operator[] 对不存在的键插入默认值。只查询时用 find 判断 end。通常查改摊还 O(1)，最坏 O(n)，空间 O(n)，不支持排名或有序前驱。纯键值查改可先试 gp_map；需要在插入/删除其他键后保留找到的元素引用时可考虑 cc_map，其节点分配和链接有成本，速度仍须按题实测。gp_map 修改后重新 find；cc_map 的点迭代器不等于可递增的遍历迭代器，删除该元素、clear 或销毁后不能保留引用。默认策略不直接调用 resize，也不照搬 unordered_map::reserve。hash64 使用进程内固定随机种子，无符号回绕；只针对整数键，不保证已知种子下的最坏碰撞。历史 associative_array AC 仅覆盖 gp_map，cc_map 另行验证。

需要排名与有序前驱时，PBDS 平衡树见第 326 页。

```
struct hash64
{
    static uint64_t mix(uint64_t x)
    {
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }

    size_t operator()(uint64_t x) const
    {
        static const uint64_t seed =
            chrono::steady_clock::now().time_since_epoch().count();
        return mix(x + seed);
    }
};

template <class K, class V> using gp_map = __gnu_pbds::gp_hash_table<K, V, hash64>;
template <class K, class V> using cc_map = __gnu_pbds::cc_hash_table<K, V, hash64>;
```

模板题使用：Library Checker associative_array

题意：维护整数键值映射，修改为覆盖赋值，查询不存在的键输出 0。gp_map 用 operator[] 写入，用 find 查询，避免查询插入新键。所需条目中包含 hash64。

所需模板：gp_map（第 262 页，27.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/associative_array

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    gp_map<uint64_t, uint64_t> a;
    while (q--)
    {
        int op;
        uint64_t k, v;
        cin >> op >> k;
        if (op == 0)
        {
            cin >> v;
            a[k] = v;
        }
        else
        {
            auto it = a.find(k);
            cout << (it == a.end() ? 0 : it->second) << '\n';
        }
    }
}
```

第二十八章　并查集

28.1　并查集

28.1.1　并查集

功能：维护 n 个点的不相交集合，支持合并两个集合、判断两点是否同集合、查询集合大小以及列出所有集合。

核心思路：每个集合是一棵以根为代表的树，`fa` 存父亲，`sz` 在根处存集合大小。`find` 用路径减半：沿父链上走时把每个点的父亲改成祖父。`merge` 找到两根后把 y 的根接到 x 的根下并累加大小。`groups` 按根把点分桶，再删去空桶。

接口契约：dsu(n)，点号 0..n-1，n≥0。find(x) 路径减半；same(x,y) 判断同集合；merge(x,y) 把 y 的根接到 x 的根，已同集合返回 false。size(x) 返回所在集合大小。groups() 返回所有非空分组，组内点号递增、组间按当前根编号排列；可能压缩路径，但不改变分组。fa、sz 可读，sz 仅根处有效；不要外部改写。本版只做路径减半，不按大小或秩合并；单次 find 最坏 O(n)。空间 O(n)，groups 另需 O(n) 输出与工作空间，并执行 n 次 find。空结构可调用 groups，不能查询顶点。

```
struct dsu
{
    vector<int> fa, sz;

    dsu(int n) : fa(n), sz(n, 1) { iota(fa.begin(), fa.end(), 0); }

    int find(int x)
    {
        while (x != fa[x]) x = fa[x] = fa[fa[x]];
        return x;
    }

    bool same(int x, int y) { return find(x) == find(y); }

    bool merge(int x, int y)
    {
        x = find(x), y = find(y);
        if (x == y) return false;
        sz[x] += sz[y], fa[y] = x;
        return true;
    }

    int size(int x) { return sz[find(x)]; }

    vector<vector<int>> groups()
    {
        vector<vector<int>> res(fa.size());
        for (int i = 0; i < int(fa.size()); i++) res[find(i)].push_back(i);
        res.erase(
            remove_if(res.begin(), res.end(), [&](const auto &v) { return v.empty(); }),
            res.end());
        return res;
    }
};
```

模板题使用：Luogu P3367

题意：维护集合合并与两点是否同集合。题目点号 1..n，传给 dsu 前减 1；查询输出 Y 或 N。

所需模板：dsu（第 264 页，28.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3367

```
int main()
{
    int n, m;
    scanf("%d%d", &n, &m);
    dsu d(n);
    while (m--)
    {
        int op, x, y;
        scanf("%d%d%d", &op, &x, &y);
        --x, --y;
        if (op == 1)
            d.merge(x, y);
        else
            puts(d.same(x, y) ? "Y" : "N");
    }
}
```

28.1.2 可撤销并查集

功能：支持撤销的并查集：合并两个集合、查询代表元，并能把结构恢复到之前某个快照时的状态，常用于离线版本树 DFS 或线段树分治。

核心思路：不做路径压缩，只按集合大小把小根接到大根下，树高为对数级，因此每次成功合并只改动一个 `fa[y]` 和一个 `siz[x]`。合并时把被接上的根 y 与 x 合并前的大小压入栈 `his`；快照就是栈长，`rollback(t)` 不断弹栈，恢复 `siz[fa[y]]` 并令 y 重新成为根。

接口契约：RollbackDSU(n) 可使用 0..n（若题目 0-based 则不用 n）。find 不压缩路径，merge 按集合大小合并，已同集合返回 false 且不记历史；siz 仅根处有效。snapshot() 返回历史长度，退出递归分支时 rollback(t) 撤销此后成功合并。t 必须是当前历史的祖先快照，不能用已回滚分支的旧快照恢复版本。find/merge 为 O(log(n+1))，每次撤销成功合并 O(1)，空间 O(n)；适合版本树或时间分治。它按大小合并以保证回滚复杂度，接口与普通 dsu 的根连接方向不同。

```
struct RollbackDSU
{
    vector<int> fa, siz;
    vector<pair<int, int>> his;

    RollbackDSU(int n) : fa(n + 1), siz(n + 1, 1) { iota(fa.begin(), fa.end(), 0); }

    int find(int x) const
    {
        while (x != fa[x]) x = fa[x];
        return x;
    }

    int snapshot() const { return (int)his.size(); }

    bool merge(int x, int y)
    {
        x = find(x);
        y = find(y);
        if (x == y) return false;
        if (siz[x] < siz[y]) swap(x, y);
        his.push_back({y, siz[x]});
        fa[y] = x;
        siz[x] += siz[y];
        return true;
    }

    void rollback(int t)
    {
        assert(0 <= t && t <= snapshot());
        while (snapshot() > t)
        {
            auto [y, s] = his.back();
            his.pop_back();
            siz[fa[y]] = s;
            fa[y] = y;
        }
    }
};
```

模板题使用：Library Checker persistent_unionfind

题意：离线读入所有操作，按所依赖的版本建立树。版本 -1 映射到树根 0，操作 i 映射到 i+1；顶点仍是 0-based。递归进入版本前保存 snapshot，退出时 rollback，答案按原查询顺序输出。只有合并产生可引用版本；本例是离线解法，不提供在线任意版本访问。总时间 O(n+q log(n+1))，空间 O(n+q)，版本链递归深度可为 q。

所需模板：RollbackDSU（第 266 页，28.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/persistent_unionfind

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<int> type(q + 1), u(q + 1), v(q + 1), ans(q + 1);
    vector<vector<int>> g(q + 1);
    for (int i = 1; i <= q; i++)
    {
        int k;
        cin >> type[i] >> k >> u[i] >> v[i];
        g[k + 1].push_back(i);
    }
    RollbackDSU d(n);
    auto dfs = [&](auto &&self, int x) -> void
    {
        int t = d.snapshot();
        if (x)
        {
            if (type[x] == 0)
                d.merge(u[x], v[x]);
            else
                ans[x] = d.find(u[x]) == d.find(v[x]);
        }
        for (int y : g[x]) self(self, y);
        d.rollback(t);
    };
    dfs(dfs, 0);
    for (int i = 1; i <= q; i++)
        if (type[i] == 1) cout << ans[i] << '\n';
}
```

28.1.3 带势并查集与差值约束

功能：带权（带势）并查集：逐条加入差值约束 $p_u - p_v = w$，判断它与已接受的约束是否矛盾；并查询两点的差值 $p_u - p_v$（不连通时未知）。

核心思路：`d[u]` 存 $p_u - p_{\texttt{fa[u]}}$。`find` 递归到根后路径压缩，同时把父亲到根的差值累加进 `d[u]`，于是 `find` 后 `d[u]` 就是 $p_u - p_{\text{根}}$。加入约束时若 u, v 同根，只需检查 `d[u]` − `d[v]` $= w$；否则设两根为 a, b，由 $p_u - p_v = w$ 得 $p_b - p_a =$ `d[u]` − `d[v]` $- w$，把 b 挂到 a 下并存这个差（按大小交换方向时取相反数）。查询答案即 `d[u]` − `d[v]`。**接口契约：**PotentialDSU<T>(n)，n≥0，点号 0..n−1。merge(u,v,w) 尝试添加 potential[u]−potential[v]=w，返回是否与已接受信息一致；重复一致约束也返回 true，矛盾返回 false，已有逻辑关系不变（仍可能路径压缩）。diff(u,v) 返回 optional<T>，未连通为 nullopt，连通时为 u−v；合法零/负值不是失败标志。find 求根并压缩路径，size 返回连通块大小。T 须提供精确加法交换群的 +、−、== 及零 T(0)，不是非交换群或浮点近似比较；整数须保证所有中间运算不溢出，动态模数不可在实例存活时改变。n 点、每条输入差值绝对值 ≤W 时，(2n+1)W≤LLONG_MAX 是 long long 的一个保守充分条件。按大小合并，摊还 O(α(n)) 次 T 运算、O(n) 空间；递归 find 未压缩前深度 O(log n)。fa/sz/d 为内部状态，不直接改写。原普通并查集的指定根合并方向与 bool 语义不变。

```
template <class T = long long> struct PotentialDSU
{
    vector<int> fa, sz;
    vector<T> d;

    PotentialDSU(int n) : fa(n), sz(n, 1), d(n, T(0))
    {
        iota(fa.begin(), fa.end(), 0);
    }

    int find(int u)
    {
        assert(0 <= u && u < (int)fa.size());
        int p = fa[u];
        if (p != u)
        {
            fa[u] = find(p);
            d[u] = d[u] + d[p];
        }
        return fa[u];
    }
```

约束合并、差值查询与分量大小（接上页同一结构体）：

```
    // Accept potential[u] - potential[v] = w iff it is consistent.
    bool merge(int u, int v, T w)
    {
        int a = find(u), b = find(v);
        if (a == b) return d[u] - d[v] == w;
        T t = d[u] - d[v] - w;
        if (sz[a] < sz[b])
        {
            swap(a, b);
            t = T(0) - t;
        }
        fa[b] = a;
        d[b] = t;
        sz[a] += sz[b];
        return true;
    }

    optional<T> diff(int u, int v)
    {
        int a = find(u), b = find(v);
        if (a != b) return nullopt;
        return d[u] - d[v];
    }

    int size(int u) { return sz[find(u)]; }
};
```

模板题使用：Library Checker Unionfind with Potential

题意：未知序列 a，1≤N,Q≤200000，模 998244353。0 u v x 尝试加入 a[u]−a[v]=x；与此前已接受信息一致输出 1，否则输出 0 并拒绝，不把矛盾边加入状态。1 u v 查询 a[u]−a[v] 的规范模值，未连通输出 −1；点号 0-based，0≤x<mod。重复一致、自约束及零差值均有效。先设置模整数的模数再构造结构，存活期间不改变它；复用已有加减与相等比较，不需要乘法逆元。

所需模板：PotentialDSU（第 268 页，28.1.3 节）、mint（第 86 页，18.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/unionfind_with_potential

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    using Z = mint<>;
    Z::set_mod(998244353);
    PotentialDSU<Z> d(n);
    while (q--)
    {
        int t, u, v;
        cin >> t >> u >> v;
        if (t == 0)
        {
            int x;
            cin >> x;
            cout << d.merge(u, v, Z(x)) << '\n';
        }
        else
        {
            auto x = d.diff(u, v);
            cout << (x ? x->v : -1) << '\n';
        }
    }
}
```

28.1.4 时间线段树分治：离线动态连通性

问题描述：离线动态图连通性：已知每条边在哪段时间区间内存在（加边、删边已配成区间），对每个时刻 t 回调一次，回调中可查询此刻的连通性和连通块大小。

核心思路：线段树分治：在时间轴 $[0,q)$ 上建线段树，每条边的存在区间拆成 $O(\log q)$ 个节点挂入 `edges[p]`。DFS 遍历线段树，进入节点时把挂在上面的边并入可撤销并查集 `RollbackDSU`，到达叶子 t 时并查集恰好包含此刻存在的全部边，调用 `visit`；离开节点时回滚到进入前的快照。

代码依赖：RollbackDSU（第 266 页，28.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：TimeConnectivity(n,q)，点号 0..n（1-based 图不使用 0），时间 0..q-1。add(l,r,u,v) 登记边在半开区间 [l,r) 有效，允许空区间、重叠区间、重边和自环。run(visit) 按时间递增调用 visit(t,const RollbackDSU &d)，可用 find 判断连通、siz[find(u)] 取分量大小；只在回调内读取，不保留引用，不在遍历中改图。每次 run 重新建并查集，可重复调用，也可在两次 run 间加区间。q=0 不调用回调。n>=0 且 n<INT_MAX，0<=q<=INT_MAX/4，并须满足内存限制。K 个非空区间：存储 O(n+q+K log(q+1))；建区间 O(K log(q+1))，遍历 O(n+q+K log(q+1) log(n+2))，另加回调代价。依赖 RollbackDSU，不做路径压缩；DFS 保持递归。

```
struct TimeConnectivity
{
```

```
    int n, q;
    vector<vector<pair<int, int>>> edges;

    TimeConnectivity(int n, int q) : n(n), q(q), edges(4 * size_t(q) + 1)
    {
        assert(n >= 0 && n < INT_MAX && q >= 0 && q <= INT_MAX / 4);
    }

    // Edge (u,v) exists at times l <= t < r. Empty intervals are allowed.
    void add(int l, int r, int u, int v)
    {
        assert(0 <= l && l <= r && r <= q);
        assert(0 <= u && u <= n && 0 <= v && v <= n);
        if (l < r)
            insert(1, 0, q, l, r, {u, v});
    }

    template <class F> void run(F visit) const
    {
        RollbackDSU d(n);
        if (q > 0)
            dfs(1, 0, q, d, visit);
    }
```

时间区间插入与回滚递归（接上页同一结构体）：

```
private:
    void insert(int p, int l, int r, int a, int b, pair<int, int> e)
    {
        if (a <= l && r <= b)
        {
            edges[p].push_back(e);
            return;
        }
        int m = l + (r - l) / 2;
        if (a < m)
            insert(p * 2, l, m, a, b, e);
        if (m < b)
            insert(p * 2 + 1, m, r, a, b, e);
    }

    template <class F>
    void dfs(int p, int l, int r, RollbackDSU &d, F &visit) const
    {
        int saved = d.snapshot();
        for (auto [u, v] : edges[p])
            d.merge(u, v);
        if (r - l == 1)
            visit(l, static_cast<const RollbackDSU &>(d));
        else
        {
            int m = l + (r - l) / 2;
            dfs(p * 2, l, m, d, visit);
            dfs(p * 2 + 1, m, r, d, visit);
        }
        d.rollback(saved);
    }
};
```

模板题使用：LOJ121 动态图连通性

题意：初始空的无向简单图，点号 1..n；0 加不存在的边、1 删存在的边、2 查连通性，输出 Y/N。n≤5000，q≤500000；端点统一排序，加删配成半开生存区间，剩余边持续到 q。

所需模板：RollbackDSU（第 266 页，28.1.2 节）、TimeConnectivity（第 270 页，28.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://loj.ac/p/121

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    TimeConnectivity tree(n, q);
    map<pair<int, int>, int> start;
    vector<pair<int, int>> query(q, {-1, -1});
    for (int t = 0; t < q; t++)
    {
        int op, u, v;
        cin >> op >> u >> v;
        if (u > v) swap(u, v);
        pair<int, int> edge{u, v};
        if (op == 0)
            start[edge] = t;
        else if (op == 1)
        {
            auto it = start.find(edge);
            tree.add(it->second, t, u, v);
            start.erase(it);
        }
        else
            query[t] = edge;
    }
    for (auto [edge, l] : start)
        tree.add(l, q, edge.first, edge.second);
    tree.run([&](int t, const RollbackDSU &d)
    {
        auto [u, v] = query[t];
        if (u != -1)
            cout << (d.find(u) == d.find(v) ? 'Y' : 'N') << '\n';
    });
}
```

28.1.5 并查集模板题的接口范围

P3367 输入点号为 1-based，统一减一后调用第 264 页的 `dsu`（merge、same）。

第二十九章　堆

29.1　配对堆

29.1.1　PBDS 可修改、可合并优先队列

功能：GNU pb_ds 的配对堆优先队列（默认大根堆）：除普通的入队、取堆顶、出队外，`push` 返回元素句柄，可通过句柄修改或删除任意元素，并能把一个堆整体合并进另一个堆。

核心思路：配对堆是一棵多叉堆序树：插入和合并只把两个根中较劣的挂到较优者下面，均为常数时间；弹出堆顶时把根的所有孩子两两配对合并、再从右往左依次合并成新树，摊还对数。句柄 `point_iterator` 直接指向树节点，修改或删除时把该节点从树中摘下，调整后再与根合并。

接口契约：pheap<T,C> 默认比较器为 less<T>，固定 pairing_heap_tag，形成大根堆；小根堆用 greater<T>。

包含 ext/pb_ds/priority_queue.hpp。push(x) 返回 point_iterator 句柄；modify(h,x) 改值，erase(h) 删除指定元素。top/pop 要求非空，size/empty/clear 沿用容器接口。a.join(b) 把 b 全部移入 a，b 变空；原 b 元素句柄仍指向同一元素，此后应交给 a。a.swap(b) 后句柄随元素交换归属。split(pred,b) 将满足谓词的元素移入 b，本模板约定 b 原先为空。join/split 两个堆须不同，比较规则一致。删除元素、pop 对应堆顶、clear 或销毁后，相应句柄失效；不要把原堆句柄用于深复制的副本。top、push、join 为 O(1)；pop 摊还 O(log n)，modify/erase 的单次最坏可达 O(n)，不当作最坏常数。split/复制/clear 可能扫描整个堆。适合需要修改已入堆元素或合并的任务；只要普通入队出队时可用 std::priority_queue。复杂度按比较、复制标量或编号对为常数计，T 须满足容器的复制要求。

```
template <class T, class C = less<T>>
using pheap = __gnu_pbds::priority_queue<T, C, __gnu_pbds::pairing_heap_tag>;
```

```
pheap<int> a, b;
auto h = a.push(7);
a.modify(h, 2);
b.join(a);
b.erase(h);
```

模板题使用：Library Checker Double-Ended Priority Queue

题意：多重集合支持插入和删除一个最小/最大值；$0\le n\le 500000$、$1\le q\le 500000$，值在 $[-10^9,10^9]$，删除时保证非空。用两个大根配对堆分别保存 (-值,id)/(值,id)，同次插入保存两份句柄；删除时先 erase 另一堆的句柄，再 pop 当前堆。id 每次新插入都递增，重复值也可区分。删除后两份句柄失效，禁止再次使用。本题取负在 int 范围内；全 int64 值不能直接照搬取负。

所需模板：pheap（第 274 页，29.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/double_ended_priority_queue

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    using H = pheap<pair<int, int>>;
    array<H, 2> q;
    vector<array<H::point_iterator, 2>> h;
    h.reserve(n + m);
    auto add = [&](int x)
    {
        int id = h.size();
        h.push_back({q[0].push({-x, id}), q[1].push({x, id})});
    };
    for (int i = 0; i < n; i++)
    {
        int x;
        cin >> x;
        add(x);
    }
    while (m--)
    {
        int t;
        cin >> t;
        if (t == 0)
        {
            int x;
            cin >> x;
            add(x);
        }
        else
        {
            int k = t - 1;
            auto [x, id] = q[k].top();
            cout << (k ? x : -x) << '\n';
            q[k ^ 1].erase(h[id][k ^ 1]);
            q[k].pop();
        }
    }
}
```

29.2 左偏树

29.2.1 左偏树可并小根堆

功能：维护多个小根堆，支持新建单元素堆、合并两个堆、删除堆顶，堆以根节点编号表示。

核心思路：左偏树：节点的 `dist` 为到最近空孩子的距离，始终保持左孩子 `dist` 不小于右孩子，故右链长度为对数级。`meld(x,y)` 取 (key, 编号) 较小者为根，把另一堆与它的右子树递归合并，回溯时若左 `dist` 更小就交换左右孩子，并令 `dist` = `dist`(右) + 1。`pop` 合并根的左右子树并把根重置为孤立点。

接口契约：默认构造为空节点池；singleton(key) 创建单点堆并返回节点编号，编号从 1 开始，0 表示空堆。meld(x,y) 原地合并两个不相交堆并返回新根，调用后旧根不能再当独立堆使用；pop(root) 删除非空堆的最小节点并返回新根，必须保存返回值。相同 key 按节点编号小者优先，t[root].key 为最小值，root 本身就是最小节点编号。key 支持完整 long long；仅比较而不做减法。dist[空]=0，叶子 dist=1，每点左子树 dist 不小于右子树；递归沿右链合并，meld/pop 为 O(log(n+1))，其中 n 为参与堆节点数。singleton 均摊 O(1)，节点池空间 O(累计创建数)，节点不回收，不保留版本；pop 后删除节点已脱离堆，不应继续作为根操作。只保证根操作，不提供任意点所属堆查询，P3377 驱动另用并查集和删除标记维护。不要改写内部 l/r/dist，也不要合并有交集的堆；assert(x!=y) 不能检测所有非法重叠。

```
struct LeftistHeap
{
    struct Node
    {
        long long key;
        int l, r, dist;
    };

    vector<Node> t{{0, 0, 0, 0}};

    int singleton(long long key)
    {
        t.push_back({key, 0, 0, 1});
        return int(t.size()) - 1;
    }

    int meld(int x, int y)
    {
        if (!x || !y) return x ? x : y;
        assert(x != y);
        if (pair(t[x].key, x) > pair(t[y].key, y)) swap(x, y);
        t[x].r = meld(t[x].r, y);
        if (t[t[x].l].dist < t[t[x].r].dist) swap(t[x].l, t[x].r);
        t[x].dist = t[t[x].r].dist + 1;
        return x;
    }

    int pop(int x)
    {
        assert(x > 0);
        int root = meld(t[x].l, t[x].r);
        t[x].l = t[x].r = 0;
        t[x].dist = 1;
        return root;
    }
};
```

模板题使用：Luogu P3377

题意：合并指定元素所在的堆，或弹出其堆最小值；已删除元素的操作无效。同值先删先输入者。并查集代表不一定是堆根，root 单独保存；节点编号比输入数组下标大 1。

所需模板：LeftistHeap（第 276 页，29.2.1 节）、dsu（第 264 页，28.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3377

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    LeftistHeap heap;
    dsu sets(n);
    vector<int> root(n);
    vector<bool> removed(n);
    for (int i = 0; i < n; i++)
    {
        long long key;
        cin >> key;
        root[i] = heap.singleton(key);
    }
    while (m--)
    {
        int op, x;
        cin >> op >> x;
        x--;
        if (op == 1)
        {
            int y;
            cin >> y;
            y--;
            if (removed[x] || removed[y]) continue;
            int a = sets.find(x), b = sets.find(y);
            if (a == b) continue;
            root[a] = heap.meld(root[a], root[b]);
            sets.merge(a, b);
        }
        else if (removed[x])
            cout << -1 << '\n';
        else
        {
            int a = sets.find(x), id = root[a];
            cout << heap.t[id].key << '\n';
            removed[id - 1] = true;
            root[a] = heap.pop(id);
        }
    }
}
```

第三十章　ST 表

30.1　位掩码静态区间最值与位置

功能：静态数组上的区间最值查询：给定 $[l,r)$，返回区间内最优（默认最小）元素的位置，并列取最左。可传入比较器改为最大值或其他严格弱序。

核心思路：按长度 b（约 $\log_2 n$）分块。块内从左到右维护单调栈，并用位掩码表示：`mask[i]` 的第 j 位为 1 表示块内位置 j 在“块首到 i”的单调栈中，即它之后到 i 没有严格更优的元素。于是块内区间 $[l,r)$ 的答案是 `mask[r-1]` 中不低于 l 的最低位。每块的最优位置组成数组，再建 ST 表。跨块询问拆成左块后缀、右块前缀和中间整块三部分，用 `better` 取优（相等取下标小者）。

思路中的情况	代码中的处理	行
块内单调栈	新元素严格更优时弹掉掩码最高位（栈顶）	23
块最优值	块末掩码的最低位即整块最左最优位置	31
块间 ST 表	倍增合并相邻两段的最优位置	40
块内询问	掩码右移掉 l 之前的位后取最低位	54
跨块询问	左块后缀与右块前缀先比较，中间整块查 ST 表	62

接口契约：StaticRMQ<T,Cmp>(a,cmp) 复制数组；默认 T=long long、Cmp=less<T> 求最小，greater<T> 求最大，也可传有状态比较器。query(l,r) 返回 0-based 半开区间 [l,r) 内最优元素的原编号，并列总取最左；值读 rmq.a[编号]。要求 0≤l<r≤n；空数组可构造但不可查询；重新赋值新对象重建，不支持修改。比较器必须是稳定的严格弱序、const 可调用，元素类型可复制；浮点数据不能含 NaN。

单调栈位掩码处理块内区间，块间用 ST 表；块长为 n 的二进制位数（空数组取 1），n≤INT_MAX/2 且内存可分配，32 位掩码足够。在复制/比较/位操作 O(1) 的模型下，预处理 O(n)、查询 O(1)、空间 O(n)。所有成员约定只读，better/small 是内部辅助；small 仅供非空同块区间。只比较元素，不做元素算术，signed64 极值合法。

```cpp
template <class T = long long, class Cmp = less<T>>
struct StaticRMQ
{
    int n, b;
    vector<T> a;
    vector<uint32_t> mask;
    vector<vector<int>> st;
    Cmp cmp;

    StaticRMQ(const vector<T> &v, Cmp cmp = Cmp()) : a(v), cmp(cmp)
    {
        assert(v.size() <= INT_MAX / 2);
        n = v.size();
        b = max(1, (int)bit_width((uint32_t)n));
        mask.resize(n);
        if (!n) return;
        st.push_back({});
        uint32_t s = 0;
        for (int i = 0; i < n; i++)
        {
            int start = i - i % b;
            if (i % b == 0) s = 0;
            while (s && cmp(a[i], a[start + bit_width(s) - 1]))
            {
                s ^= uint32_t(1) << (bit_width(s) - 1);
            }
            s |= uint32_t(1) << (i % b);
            mask[i] = s;
            if (i % b == b - 1 || i == n - 1)
            {
                st[0].push_back(start + countr_zero(s));
            }
        }
```

块间表构建、最优位置合并与查询（接上页同一结构体）：

```
        int m = st[0].size();
        for (int k = 1; (1 << k) <= m; k++)
        {
            st.push_back(vector<int>(m - (1 << k) + 1));
            for (int i = 0; i + (1 << k) <= m; i++)
            {
                st[k][i] = better(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
            }
        }
    }

    int better(int x, int y) const
    {
        if (cmp(a[x], a[y])) return x;
        if (cmp(a[y], a[x])) return y;
        return min(x, y);
    }

    int small(int l, int r) const
    {
        return l + countr_zero(mask[r - 1] >> (l % b));
    }

    int query(int l, int r) const
    {
        assert(0 <= l && l < r && r <= n);
        int x = l / b, y = (r - 1) / b;
        if (x == y) return small(l, r);
        int ans = better(small(l, (x + 1) * b), small(y * b, r));
        x++;
        if (x < y)
        {
            int k = bit_width((uint32_t)(y - x)) - 1;
            ans = better(ans, better(st[k][x], st[k][y - (1 << k)]));
        }
        return ans;
    }
};
```

模板题使用：Library Checker Static RMQ

题意：静态数组上查询 0-based 半开区间 [l,r) 最小值；n,q≤500000，值为 0..10^9。模板返回最左最小值位置，通过 rmq.a[位置] 输出值。

所需模板：StaticRMQ（第 278 页，30.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/staticrmq

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<int> a(n);
    for (int &x : a) cin >> x;
    StaticRMQ<int> rmq(a);
    while (q--)
    {
        int l, r;
        cin >> l >> r;
        cout << rmq.a[rmq.query(l, r)] << '\n';
    }
}
```

第三十一章　树状数组

31.1　四矩二维树状数组：矩形加与矩形和

功能：维护 $n \times m$ 的整数矩阵（初值全 0），支持给一个子矩形整体加 v，以及查询一个子矩形的元素和。

核心思路：矩形加在二维差分数组 D 上只改四个角（`corner`）。前缀和 $S(x,y)=\sum_{a<x,b<y} D(a,b)(x-a)(y-b) = xy\sum D - y\sum aD - x\sum bD + \sum abD$，所以用一棵二维树状数组同时维护 D, aD, bD, abD 四个量（`bit[i][j]` 的四个分量），`prefix` 求出四个和后按上式组合，矩形和再由四个前缀容斥。落在右边界或上边界的角不影响网格，直接跳过。

接口契约：RectangleFenwick<T>(n,m)，T 默认 long long，0≤n,m<2^30，初始化全 0；稠密空间与构造 O((n+1)(m+1))，需先确认可分配。add(l,d,r,u,v) 给 0-based 半开矩形 [l,r)×[d,u) 加 v；sum(l,d,r,u) 查询同一区间，prefix(x,y) 查询 [0,x)×[0,y)。端点须在 [0,n]×[0,m]，允许空矩形及零维矩阵。差分四角维护 D、xD、yD、xyD 四个矩，前缀和为 xyS0-ySx-xSy+Sxy；每次操作 O(log n log m)，实际存储每格 4 个 T。单点操作令右上界各加 1。T 支持零值、加减、取负和乘 int；包括-v、各坐标乘积及容斥在内的中间值均须可表示，仅最终答案可表示不够。内部 bit 与 corner 不供直接修改。重新赋值 RectangleFenwick(n,m) 清空；不支持稀疏大坐标自动压缩，因为区间加仍取决于真实格子数量。

```
template <class T = long long> struct RectangleFenwick
{
    int n, m;
    vector<vector<array<T, 4>>> bit;

    RectangleFenwick(int n, int m) : n(n), m(m)
    {
        assert(0 <= n && n < (1 << 30) && 0 <= m && m < (1 << 30));
        bit.assign(n + 1, vector<array<T, 4>>(m + 1));
    }

    // Difference impulse at (x,y); boundary impulses do not affect the grid.
    void corner(int x, int y, T v)
    {
        if (x == n || y == m) return;
        array<T, 4> a{v, v * x, v * y, v * x * y};
        for (int i = x + 1; i <= n; i += i & -i)
            for (int j = y + 1; j <= m; j += j & -j)
                for (int k = 0; k < 4; k++) bit[i][j][k] += a[k];
    }

    // Add to [l,r) x [d,u), 0-based. Empty rectangles are allowed.
    void add(int l, int d, int r, int u, T v)
    {
        assert(0 <= l && l <= r && r <= n && 0 <= d && d <= u && u <= m);
        if (l == r || d == u) return;
        corner(l, d, v);
        corner(r, d, -v);
        corner(l, u, -v);
        corner(r, u, v);
    }
```

更新与半开矩形求和（接上页同一结构体）：

```
    T prefix(int x, int y) const
    {
        assert(0 <= x && x <= n && 0 <= y && y <= m);
        array<T, 4> a{};
        for (int i = x; i; i -= i & -i)
            for (int j = y; j; j -= j & -j)
                for (int k = 0; k < 4; k++) a[k] += bit[i][j][k];
        return a[0] * x * y - a[1] * y - a[2] * x + a[3];
    }

    T sum(int l, int d, int r, int u) const
    {
        assert(0 <= l && l <= r && r <= n && 0 <= d && d <= u && u <= m);
        if (l == r || d == u) return T{};
        return prefix(r, u) - prefix(l, u) - prefix(r, d) + prefix(l, d);
    }
};
```

模板题使用：Luogu P4514 / 上帝造题的七分钟

题意：输入 X n m 初始化零矩阵，n,m≤2048；读到 EOF，L a b c d delta 给 1-based 闭矩形加 delta，k a b c d 查询区域和（小写 k）。至多 200000 操作，|delta|≤500；转换为 [a-1,c)×[b-1,d)。最终答案在 32 位内，中间值未保证，使用 long long。

所需模板：RectangleFenwick（第 282 页，31.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4514

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    char op;
    int n, m;
    std::cin >> op >> n >> m;
    RectangleFenwick tree(n, m);
    int a, b, c, d;
    while (std::cin >> op >> a >> b >> c >> d)
    {
        if (op == 'L')
        {
            long long v;
            std::cin >> v;
            tree.add(a - 1, b - 1, c, d, v);
        }
        else
            std::cout << tree.sum(a - 1, b - 1, c, d) << '\n';
    }
}
```

31.2 树状数组第 k 小与组合依赖

功能：维护长度为 n 的数组，支持单点加、前缀和与区间和；元素表示频数（均非负）时还能求“前缀和首次达到 k 的位置”，即可重集合的第 k 小。

核心思路：`a[x]` 存原数组在 $(x-\text{lowbit}(x), x]$ 上的和。前缀和沿 $x \leftarrow x-\text{lowbit}(x)$ 累加，单点修改沿 $x \leftarrow x+\text{lowbit}(x)$ 更新所有覆盖它的节点，各 $O(\log n)$。第 k 小从高位到低位倍增：若跳过下一段后累计仍小于 k 就跳过去。

接口契约：保留频数第 k 小和泛型组合用途；P3374 基础单点加/区间和题不再独立收录或安排提交。add(x,v)、sum(x)、query(l,r)，下标 1..n，闭区间；kth(k) 要求所有频率非负且 k>0，超过总和返回 n+1。单次 O(log n)。T 须能从 0 构造并支持 += 与减法；可直接使用 ModInt 维护模数下的区间和。kth 只适用于有序累计频数，不适用于取模后的区间和。

```
template <class T = long long> struct Fenwick
{
    int n;
    vector<T> a;

    Fenwick(int n = 0) : n(n), a(n + 1) {}

    void add(int x, T v)
    {
        assert(x >= 1);
        for (; x <= n; x += x & -x) a[x] += v;
    }

    T sum(int x) const
    {
        T ans = 0;
        for (; x; x -= x & -x) ans += a[x];
        return ans;
    }

    T query(int l, int r) const { return sum(r) - sum(l - 1); }

    // Nonnegative frequencies; returns n+1 if k exceeds total.
    int kth(T k) const
    {
        assert(k > 0);
        int x = 0, b = 1;
        while (b <= n / 2) b *= 2;
        for (; b; b >>= 1)
            if (x + b <= n && a[x + b] < k)
            {
                k -= a[x + b];
                x += b;
            }
        return x + 1;
    }
};
```

模板题使用：Luogu P3369

题意：维护可重集合的插入、删除一个、排名、第 k 小和严格前驱/后继。预读全部操作并离散化非第 k 小查询的值域；频数下标为 1-based，kth 的返回值减 1 后映射回原值。排名查询允许 x 不在集合中；第 k 小与前驱/后继按题目保证存在。n≤100000，|x|≤10000000；总复杂度 O(n log n)，空间 O(n)。

所需模板：Fenwick（第 284 页，31.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3369

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    vector<pair<int, int>> q(n);
    vector<int> a;
    for (auto &[op, x] : q)
    {
        cin >> op >> x;
        if (op != 4) a.push_back(x);
    }
    sort(a.begin(), a.end());
    a.erase(unique(a.begin(), a.end()), a.end());
    Fenwick<int> f(a.size());
    for (auto [op, x] : q)
    {
        int p = lower_bound(a.begin(), a.end(), x) - a.begin();
        if (op == 1)
            f.add(p + 1, 1);
        else if (op == 2)
        {
            if (f.query(p + 1, p + 1) > 0) f.add(p + 1, -1);
        }
        else if (op == 3)
            cout << f.sum(p) + 1 << '\n';
        else if (op == 4)
            cout << a[f.kth(x) - 1] << '\n';
        else if (op == 5)
            cout << a[f.kth(f.sum(p)) - 1] << '\n';
        else
        {
            p = upper_bound(a.begin(), a.end(), x) - a.begin();
            cout << a[f.kth(f.sum(p) + 1) - 1] << '\n';
        }
    }
}
```

相关应用：离线二维扫描线应用，不代表整页覆盖

31.3 离线静态矩形权值和

问题描述：给定平面上若干带权点（不修改），多次询问左闭右开矩形 $[l,r)\times[d,u)$ 内点的权值和，所有询问一次读入后统一回答。

核心思路：把询问拆成两条竖直扫描线：答案 $= F(r) - F(l)$，其中 $F(X)$ 是横坐标小于 X、纵坐标在 $[d, u)$ 内的点权和。点按横坐标排序，事件按扫描位置排序；处理位置为 X 的事件前，把所有横坐标严格小于 X 的点加入按纵坐标离散化的树状数组，再用两次前缀和求纵坐标落在 $[d, u)$ 的权值和，乘上事件的符号 ± 1 累加到对应询问。

代码依赖：Fenwick（第 284 页，31.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：offline_rectangle_sum(points,queries)：points 每项为 {x,y,w}，queries 每项为 {l,d,r,u}，返回按询问原顺序排列的 [l,r)×[d,u) 内权值和。坐标、端点与权值均为 long long；要求 l<=r、d<=u，允许空矩形、空点集、空询问、重合点和负权。传入点按值复制，两个输入均不改变，多次调用互不影响。坐标仅比较，不用端点 ±1，支持 LLONG_MIN/MAX；半开矩形无法包含位于 LLONG_MAX 的上边界坐标。

一次读完所有询问，按横坐标扫描，在纵轴离散化树状数组上加点。同横坐标事件只包含 x< 事件横坐标的点，保证左右开闭正确。时间 O((N+Q)log(N+Q+1))，空间 O(N+Q)，N,Q 均小于 2^30 且内存可分配。要求所有权值绝对值之和 <=LLONG_MAX，保证树状数组、容斥与符号乘法不溢出。依赖 Fenwick；若需逐条在线回答，改用持久化前缀版本。

```
vector<long long> offline_rectangle_sum(vector<array<long long, 3>> points,
                                        const vector<array<long long, 4>> &queries)
{
    using ll = long long;
    assert(points.size() < (1U << 30) && queries.size() < (1U << 30));
    vector<ll> ys;
    for (auto [x, y, w] : points)
        ys.push_back(y);
    sort(ys.begin(), ys.end());
    ys.erase(unique(ys.begin(), ys.end()), ys.end());
    auto rank = [&](ll y)
    {
        return int(lower_bound(ys.begin(), ys.end(), y) - ys.begin());
    };
    struct Event
    {
        ll x;
        int d, u, id, sign;
    };
    vector<Event> events;
    events.reserve(2 * queries.size());
    for (int i = 0; i < (int)queries.size(); i++)
    {
        auto [l, d, r, u] = queries[i];
        assert(l <= r && d <= u);
        if (l == r || d == u)
            continue;
        events.push_back({l, rank(d), rank(u), i, -1});
        events.push_back({r, rank(d), rank(u), i, 1});
    }
    sort(points.begin(), points.end());
    sort(events.begin(), events.end(), [](const Event &a, const Event &b)
    {
        return a.x < b.x;
    });
    Fenwick<ll> bit(ys.size());
    vector<ll> answer(queries.size());
    int p = 0;
    for (auto e : events)
    {
```

```
        while (p < (int)points.size() && points[p][0] < e.x)
        {
            auto [x, y, w] = points[p];
            bit.add(rank(y) + 1, w);
            p++;
        }
        ll sum = bit.sum(e.u) - bit.sum(e.d);
        answer[e.id] += e.sign * sum;
    }
    return answer;
}
```

模板题使用：Library Checker Rectangle Sum（离线）

题意：N,Q<=200000，点坐标与权值在 [0,1e9]；读取全部询问后用扫描线返回半开矩形权值和。此例可离线处理，节省持久化节点的内存；需要在线查询时使用持久化版本（用法 432）。Library Checker C++20 线上 AC 410016，258ms、29.98MiB。

所需模板：Fenwick（第 284 页，31.2 节）、offline_rectangle_sum（第 285 页，31.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/rectangle_sum

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<array<long long, 3>> points(n);
    vector<array<long long, 4>> queries(q);
    for (auto &p : points)
        cin >> p[0] >> p[1] >> p[2];
    for (auto &r : queries)
        cin >> r[0] >> r[1] >> r[2] >> r[3];
    for (auto answer : offline_rectangle_sum(points, queries))
        cout << answer << '\n';
}
```

第三十二章　线段树

32.1　线段树基础

32.1.1　通用懒标记线段树与边界搜索

功能：对由幺半群 $(S, \mathtt{op}, \mathtt{e})$ 元素组成的数组，支持区间作用一个映射 $f \in F$、区间按序合并、单点读写，以及从某端点出发二分求满足谓词的最远边界。具体问题只需给出 S、F 和五个函数。

核心思路：递归线段树，`d[p]` 是节点区间的合并结果（已包含本节点标记的作用），`tag[p]` 是尚未下传给孩子的映射。区间作用在完全覆盖的节点上调用 `act`：用 `mapping` 更新 `d`，用 `composition` 把新映射复合到旧标记外层；进入孩子前 `push`，回溯时 `pull`。`max_right` 从左往右累积 s，整块合并后仍满足谓词就整块跳过，否则下传并进入该块，先左后右找到第一个使谓词变假的叶子；`min_left` 方向相反、合并顺序为 $\mathtt{op}(\mathtt{d[p]}, s)$。

接口契约：模板参数依次为 S、F、op、e、mapping、composition、id；以数组 a 构造，0-based 半开区间；n 构造将 n 个元素置为 e。0≤n≤2^28-1，受内存限制。op 满足结合律、e 为双侧单位元；单位标记不改变节点，mapping 必须保持 op 的合并结果；composition(f,g) 表示先 g 后 f，须满足作用复合一致性，id 为标记单位元。无需比较标记是否等于 id，每次下传都应用标记。set/get 单点，apply(l,r,f) 区间作用，prod(l,r) 保序合并，all() 读取根；空区间不更新且查询返回 e。区间长度若参与修改，必须存入 S，不能从 e 推断长度：区间和节点为 {sum,len}，真实叶 len=1，e={0,0}。max_right/min_left 的谓词须满足 ok(e())、无副作用、扩展时真变假后不再变真；两者查询会下传标记，非 const。递归深度 O(log n)，构造 O(n)、操作/搜索 O(log n)、all O(1)，空间 O(n)，假设运算和类型复制 O(1)。d/tag 及其他递归辅助方法仅供内部使用。

```
template <class S, class F, auto op, auto e, auto mapping, auto composition, auto id>
struct lazy_segtree
{
    int n;
    vector<S> d;
    vector<F> tag;

    lazy_segtree(int n = 0) : lazy_segtree(vector<S>(n, e())) {}

    lazy_segtree(const vector<S> &a)
        : n(a.size()), d(4 * n + 4, e()), tag(4 * n + 4, id())
    {
        if (n) build(1, 0, n, a);
    }

    void pull(int p) { d[p] = op(d[p * 2], d[p * 2 + 1]); }

    void build(int p, int l, int r, const vector<S> &a)
    {
        if (r - l == 1)
        {
            d[p] = a[l];
            return;
        }
        int m = (l + r) / 2;
        build(p * 2, l, m, a);
        build(p * 2 + 1, m, r, a);
        pull(p);
    }
```

```
    void act(int p, F f)
    {
        d[p] = mapping(f, d[p]);
        tag[p] = composition(f, tag[p]);
    }

    void push(int p)
    {
        act(p * 2, tag[p]);
        act(p * 2 + 1, tag[p]);
        tag[p] = id();
    }

    void set(int x, S v)
    {
        assert(0 <= x && x < n);
        put(1, 0, n, x, v);
    }

    void put(int p, int l, int r, int x, S v)
    {
        if (r - l == 1)
        {
            d[p] = v;
            tag[p] = id();
            return;
        }
        push(p);
        int m = (l + r) / 2;
        if (x < m)
            put(p * 2, l, m, x, v);
        else
            put(p * 2 + 1, m, r, x, v);
        pull(p);
    }

    void apply(int l, int r, F f)
    {
        assert(0 <= l && l <= r && r <= n);
        if (l < r) update(1, 0, n, l, r, f);
    }

    void update(int p, int l, int r, int a, int b, F f)
    {
        if (a <= l && r <= b)
        {
            act(p, f);
            return;
        }
        push(p);
        int m = (l + r) / 2;
        if (a < m) update(p * 2, l, m, a, b, f);
        if (m < b) update(p * 2 + 1, m, r, a, b, f);
        pull(p);
    }
```

```
87      S all() const { return d[1]; }
88
89      S get(int x)
90      {
91          assert(0 <= x && x < n);
92          return prod(x, x + 1);
93      }
94
95      S prod(int l, int r)
96      {
97          assert(0 <= l && l <= r && r <= n);
98          return l == r ? e() : query(1, 0, n, l, r);
99      }
100
101     S query(int p, int l, int r, int a, int b)
102     {
103         if (a <= l && r <= b) return d[p];
104         push(p);
105         int m = (l + r) / 2;
106         if (b <= m) return query(p * 2, l, m, a, b);
107         if (m <= a) return query(p * 2 + 1, m, r, a, b);
108         return op(query(p * 2, l, m, a, b), query(p * 2 + 1, m, r, a, b));
109     }
110
111     template <class G> int max_right(int l, G ok)
112     {
113         assert(0 <= l && l <= n && ok(e()));
114         S s = e();
115         return l == n ? n : right(1, 0, n, l, s, ok);
116     }
117
118     template <class G> int right(int p, int l, int r, int a, S &s, G &ok)
119     {
120         if (r <= a) return n;
121         if (a <= l && ok(op(s, d[p])))
122         {
123             s = op(s, d[p]);
124             return n;
125         }
126         if (r - l == 1) return l;
127         push(p);
128         int m = (l + r) / 2;
129         int x = right(p * 2, l, m, a, s, ok);
130         return x != n ? x : right(p * 2 + 1, m, r, a, s, ok);
131     }
```

```
133     template <class G> int min_left(int r, G ok)
134     {
135         assert(0 <= r && r <= n && ok(e()));
136         S s = e();
137         return r == 0 ? 0 : left(1, 0, n, r, s, ok);
138     }
139 
140     template <class G> int left(int p, int l, int r, int b, S &s, G &ok)
141     {
142         if (b <= l) return 0;
143         if (r <= b && ok(op(d[p], s)))
144         {
145             s = op(d[p], s);
146             return 0;
147         }
148         if (r - l == 1) return r;
149         push(p);
150         int m = (l + r) / 2;
151         int x = left(p * 2 + 1, m, r, b, s, ok);
152         return x != 0 ? x : left(p * 2, l, m, b, s, ok);
153     }
154 };
```

模板题使用：Library Checker range_affine_range_sum

题意：对数组区间施加 x→b*x+c，并查询区间和。S 保存和与长度；空单位元长度 0，单个元素长度 1。composition(f,g) 表示先 g 后 f，mapping 按长度累加常数项。区间为 0-based 半开，模 998244353。

所需模板：lazy_segtree（第 289 页，32.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/range_affine_range_sum

```
using S = pair<long long, int>;
using F = pair<long long, long long>;
const int mod = 998244353;

S op(S a, S b)
{
    return {(a.first + b.first) % mod, a.second + b.second};
}

S e()
{
    return {0, 0};
}

S mapping(F f, S a)
{
    return {(f.first * a.first + f.second * a.second) % mod, a.second};
}

F composition(F f, F g)
{
    return {f.first * g.first % mod, (f.first * g.second + f.second) % mod};
}

F id()
{
    return {1, 0};
}
```

使用入口（接上页配置函数）：

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<S> a(n);
    for (auto &[x, len] : a)
    {
        cin >> x;
        len = 1;
    }
    lazy_segtree<S, F, op, e, mapping, composition, id> s(a);
    while (q--)
    {
        int t, l, r;
        cin >> t >> l >> r;
        if (t == 0)
        {
            long long b, c;
            cin >> b >> c;
            s.apply(l, r, {b, c});
        }
        else
            cout << s.prod(l, r).first << '\n';
    }
}
```

32.1.2 通用线段树与左右边界搜索

功能：对由幺半群 $(S, \mathtt{op}, \mathtt{e})$ 元素组成的数组，支持单点赋值、区间按从左到右的顺序合并，以及从一个端点出发求满足单调谓词的最远另一端点。

核心思路：非递归线段树：长度补到 2 的幂 `sz`，叶子在 `d[sz+i]`，内部节点 p 合并 $2p, 2p+1$。`prod` 从两端向上爬，左侧结果 `a` 在右边追加、右侧结果 `b` 在左边追加，保证不交换律也正确。`max_right` 从 l 起每次跳到当前位置所在的最大完整块，若并入后谓词变假，就在该块内向下二分（能并入左孩子则并入并转向右孩子）；`min_left` 对称地从右往左，并始终以 $\mathtt{op}(\mathtt{d[r]}, a)$ 的顺序合并。

接口契约：segtree<S,op,e>(a) 从 0-based vector 构建，或 segtree(n) 初始化为单位元。0≤n≤2^29，实际受内存限制。op(S,S) 须满足结合律，e() 是双侧单位元；无需交换律。set(p,x)、get(p) 操作单点，prod(l,r) 返回半开区间 [l,r) 从左到右的合并，空区间返回 e；all() 返回全区间结果。max_right(l,ok) 返回最大的 r 使 ok(prod(l,r)) 为真；min_left(r,ok) 返回最小的 l。要求 ok(e()) 为真，ok 无副作用，并随对应方向扩展从真变假后不再变真；例如非负和不超过阈值，不能对任意负数和这样搜索。l=n、r=0 支持空端点。构建 O(n)，单点/区间/边界搜索 O(log n)，all O(1)，空间 O(n)，假设 op、e、ok 及 S 复制为 O(1)。d、sz、pull 为内部实现，不直接修改。函数复合时 op(f,g) 表示 g(f(x))，对应先执行左区间再执行右区间；min_left 不能照抄 max_right 的合并顺序。

```
template <class S, auto op, auto e> struct segtree
{
    int n, sz = 1;
    vector<S> d;

    segtree(int n = 0) : segtree(vector<S>(n, e())) {}

    segtree(const vector<S> &a) : n(a.size())
    {
        while (sz < n) sz <<= 1;
        d.assign(2 * sz, e());
        copy(a.begin(), a.end(), d.begin() + sz);
        for (int i = sz - 1; i; i--) pull(i);
    }

    void pull(int p) { d[p] = op(d[p * 2], d[p * 2 + 1]); }

    void set(int p, S x)
    {
        assert(0 <= p && p < n);
        d[p += sz] = x;
        while (p >>= 1) pull(p);
    }

    S get(int p) const
    {
        assert(0 <= p && p < n);
        return d[p + sz];
    }

    S all() const { return d[1]; }
```

```
    S prod(int l, int r) const
    {
        assert(0 <= l && l <= r && r <= n);
        S a = e(), b = e();
        for (l += sz, r += sz; l < r; l >>= 1, r >>= 1)
        {
            if (l & 1) a = op(a, d[l++]);
            if (r & 1) b = op(d[--r], b);
        }
        return op(a, b);
    }

    template <class F> int max_right(int l, F ok) const
    {
        assert(0 <= l && l <= n && ok(e()));
        if (l == n) return n;
        S a = e();
        l += sz;
        do
        {
            while (!(l & 1)) l >>= 1;
            if (!ok(op(a, d[l])))
            {
                while (l < sz)
                {
                    l *= 2;
                    if (ok(op(a, d[l]))) a = op(a, d[l++]);
                }
                return l - sz;
            }
            a = op(a, d[l++]);
        } while ((l & -l) != l);
        return n;
    }
```

```
    template <class F> int min_left(int r, F ok) const
    {
        assert(0 <= r && r <= n && ok(e()));
        if (r == 0) return 0;
        S a = e();
        r += sz;
        do
        {
            --r;
            while (r > 1 && (r & 1)) r >>= 1;
            if (!ok(op(d[r], a)))
            {
                while (r < sz)
                {
                    r = r * 2 + 1;
                    if (ok(op(d[r], a))) a = op(d[r--], a);
                }
                return r + 1 - sz;
            }
            a = op(d[r], a);
        } while ((r & -r) != r);
        return 0;
    }
};
```

模板题使用：Library Checker point_set_range_composite

题意：单点修改一次函数，查询区间内从左到右依次作用于 x 的结果。op(f,g) 表示先 f 后 g，单位元为 x；prod(l,r) 使用 0-based 半开区间，空区间返回单位元。配置函数须和 main 一起抄写。

所需模板：segtree（第 295 页，32.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/point_set_range_composite

```
using S = pair<long long, long long>;
const int mod = 998244353;

S op(S f, S g)
{
    return {f.first * g.first % mod, (f.second * g.first + g.second) % mod};
}

S e()
{
    return {1, 0};
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<S> a(n);
    for (auto &[x, y] : a) cin >> x >> y;
    segtree<S, op, e> s(a);
    while (q--)
    {
        int t;
        cin >> t;
        if (t == 0)
        {
            int p;
            long long c, d;
            cin >> p >> c >> d;
            s.set(p, {c, d});
        }
        else
        {
            int l, r;
            long long x;
            cin >> l >> r >> x;
            auto [c, d] = s.prod(l, r);
            cout << (c * x + d) % mod << '\n';
        }
    }
}
```

相关应用：线段树维护单调栈的双向极大值贡献；区间加接雨水应用，不将普通单调栈直接视为覆盖。

32.1.3 线段树维护单调栈：区间加与接雨水

功能：维护高度数组，支持区间加；查询整个数组的接雨水体积 $\sum_i(\min(\text{前缀最大}, \text{后缀最大}) - a_i)$，以及从给定初始高度 h 出发向左或向右扫描一段时 $\sum(\text{当前最大值} - a_i)$。

核心思路：“线段树维护单调栈”：节点 `fill[0]`、`fill[1]` 缓存本区间自左（自右）扫描时 $\sum(\text{运行最大值} - a_i)$。`extra(p,h,dir)` 求以初值 h 扫过节点的同一量：h 不小于整段最大值时为 $h \cdot$ `len` $-$ `sum`；不小于先扫孩子的最大值时先扫孩子全为 h，递归另一孩子；否则递归先扫孩子，另一孩子的贡献只取决于先扫孩子的最大值，等于 `fill`$(p)-$`fill`(先扫孩子)。故每次 `extra` 只走一条链。由 $\min(P,S) = P+S-\max$，`water` 为 `fill[0]`$+$`fill[1]`$-(n \cdot \max - \sum a)$。

接口契约：MonotoneStackSeg(a) 要求非空 long long 数组；add(l,r,delta) 对 0-based 半开区间加值，空区间不变。water() 返回整个数组的接雨水体积。scan(l,r,height,reverse=false) 按左到右（reverse=true 则右到左）扫描，将 height 与当前元素取 max 并累加 height-a[i]，返回 (累加值, 最后的 height)；空区间返回 (0,height)。支持负高度与负增量，无固定零边界。节点缓存双向前缀极大值减原值的和；区间平移不改变两者差。建树 O(n)，add/scan 最坏 O($\log^2$(n+1))，water 为 O(1)，空间 O(n)；递归深度 O(log n)。4n 须可由 int 表示；高度、和、懒标记、外部 height 与长度的乘积以及所有中间运算均须在 long long 范围内。t 与 extra/pull/push 及带节点编号的函数属内部实现。

```
struct MonotoneStackSeg
{
    using ll = long long;

    struct Node
    {
        int len = 0;
        ll mx = 0, sum = 0, lazy = 0;
        array<ll, 2> fill{};
    };

    int n;
    vector<Node> t;

    MonotoneStackSeg(const vector<ll> &a) : n(a.size()), t(4 * n)
    {
        assert(n > 0);
        build(1, 0, n, a);
    }

    void apply(int p, ll delta)
    {
        t[p].mx += delta;
        t[p].sum += delta * t[p].len;
        t[p].lazy += delta;
    }

    void push(int p)
    {
        if (!t[p].lazy) return;
        apply(p * 2, t[p].lazy);
        apply(p * 2 + 1, t[p].lazy);
        t[p].lazy = 0;
    }
```

单路扫描、合并与建树（接上页同一结构体）：

```
ll extra(int p, ll height, int dir)
{
    if (height >= t[p].mx) return height * t[p].len - t[p].sum;
    if (t[p].len == 1) return 0;
    push(p);
    int first = p * 2 + dir, second = p * 2 + 1 - dir;
    if (height >= t[first].mx)
        return height * t[first].len - t[first].sum + extra(second, height, dir);
    return extra(first, height, dir) + (t[p].fill[dir] - t[first].fill[dir]);
}

void pull(int p)
{
    int l = p * 2, r = l + 1;
    t[p].len = t[l].len + t[r].len;
    t[p].mx = max(t[l].mx, t[r].mx);
    t[p].sum = t[l].sum + t[r].sum;
    t[p].fill[0] = t[l].fill[0] + extra(r, t[l].mx, 0);
    t[p].fill[1] = t[r].fill[1] + extra(l, t[r].mx, 1);
}

void build(int p, int l, int r, const vector<ll> &a)
{
    if (r - l == 1)
    {
        t[p].len = 1;
        t[p].mx = a[l];
        t[p].sum = a[l];
        return;
    }
    int m = (l + r) / 2;
    build(p * 2, l, m, a);
    build(p * 2 + 1, m, r, a);
    pull(p);
}
```

区间加、双向查询与雨水体积（接上页同一结构体）：

```
    void add(int p, int l, int r, int ql, int qr, ll delta)
    {
        if (ql <= l && r <= qr) return apply(p, delta);
        push(p);
        int m = (l + r) / 2;
        if (ql < m) add(p * 2, l, m, ql, qr, delta);
        if (m < qr) add(p * 2 + 1, m, r, ql, qr, delta);
        pull(p);
    }

    void add(int l, int r, ll delta)
    {
        assert(0 <= l && l <= r && r <= n);
        if (l < r) add(1, 0, n, l, r, delta);
    }

    ll scan(int p, int l, int r, int ql, int qr, ll &height, int dir)
    {
        if (ql <= l && r <= qr)
        {
            ll answer = extra(p, height, dir);
            height = max(height, t[p].mx);
            return answer;
        }
        push(p);
        int m = (l + r) / 2;
        ll answer = 0;
        if (dir && m < qr) answer += scan(p * 2 + 1, m, r, ql, qr, height, dir);
        if (ql < m) answer += scan(p * 2, l, m, ql, qr, height, dir);
        if (!dir && m < qr) answer += scan(p * 2 + 1, m, r, ql, qr, height, dir);
        return answer;
    }

    // Return (sum of running maximum - a[i], final running maximum).
    pair<ll, ll> scan(int l, int r, ll height, bool reverse = false)
    {
        assert(0 <= l && l <= r && r <= n);
        ll answer = 0;
        if (l < r) answer = scan(1, 0, n, l, r, height, reverse);
        return {answer, height};
    }

    ll water() const
    {
        ll gap = t[1].mx * n - t[1].sum;
        return t[1].fill[0] - (gap - t[1].fill[1]);
    }
};
```

32.1.4 ACL 的组合约定

懒标记 composition(f,g) 表示先作用 g、再作用 f；SCC 分量方向与 2-SAT 的真假编码须配套，不能分别替换。Fenwick 的取模区间和不适用于 kth。

32.1.5 线段树维护单调栈：双向扫描与接雨水

`MonotoneStackSeg` 每段存 F_0, F_1：从左、从右扫描的运行最大值减原值之和。外部高度 $x \geq$ 段最大值时贡献 $x \cdot len - sum$；否则按扫描顺序孩子 A, B：$x \geq \max A$ 递归 B，否则递归 A 并加 $F_{parent} - F_A$。单段扫描 $O(\log n)$，区间修改及任意子区间扫描最坏 $O(\log^2(n+1))$，建树 $O(n)$。雨水体积（M 为全局最大值）根上 $O(1)$：

$$\sum_i \big(\min(L_i, R_i) - a_i\big) = F_0 + F_1 - \big(nM - \sum_i a_i\big).$$

负高度可用；所有中间量须在 long long 内。

32.2 线段树合并 & 分裂

32.2.1 线段树合并与分裂

功能：维护多个可重集合，元素是值域 $[0, n)$ 内的整数：支持增减某个值的个数、把一个集合并入另一个、按值域区间或按排名（最小或最大的 k 个）分出一个新集合，以及查询值域区间计数与第 k 小。

核心思路：每个集合是一棵动态开点的权值线段树，节点 `sum` 为该值域的元素个数，个数变为 0 的节点回收到 `free` 复用。合并时两树同步递归，只在双方都有节点处继续，把 q 的计数并入 p 后回收 q。分裂时同步构造“留下”与“取走”两棵树：完全落入目标区间的子树整体移交，部分相交的节点为取走部分新建一个节点。按排名分裂时先算出左子树应取走多少个，叶子处可以拆分重数。

思路中的情况	代码中的处理	行
合并	一方为空就直接用另一方，叶子处累加计数，回收 q	107
值域分裂：子树完全在区间内	整棵子树移给新集合	136
值域分裂：部分相交	递归两侧后，原节点空了就回收；取走部分非空才新建节点	149
排名分裂：分配个数	取最小 k 个时左侧取 $\min(k, \text{左计数})$，取最大时左侧只取超出右计数的部分	188
排名分裂：叶子	同一值的重数拆成留下与取走两份	179
第 k 小	左子树计数不少于 k 走左，否则减去后走右	231

接口契约：MergeSplitTree(n) 建空集合 0，或传 vector<long long> 建初始重数；n>0，值域 [0,n)。add(s,x,delta) 点增减，结果重数非负；sum(s,l,r) 查询半开值域；kth(s,k) 按重数返回第 k 小的 0-based 值，不存在返回-1。split(s,l,r) 移走区间并返回新集合号，空区间也创建集合；split_rank(s,k,reverse=false) 移走最小 k 个元素到新集合；reverse=true 移走最大 k 个，k 在 [0, 集合总重数] 内，同值的重数可拆开，k=0 也创建空集合。merge(dst,src) 消耗不同的 src，src 变空后可重用。所有集合总量分别须 ≤LLONG_MAX，节点/集合索引须适合 int。仅通过集合号操作，root/t/free 只读，其余递归函数内部使用，禁止浅拷贝根。初始化 O(n)，增减/分裂/查询 O(log(n+1))；单次合并非 O(log n)，遍历重叠节点并回收，总合并开销可摊到节点创建。完整对象值复制独立。

```
struct MergeSplitTree
{
    using ll = long long;
    struct Node
    {
        int l = 0, r = 0;
        ll sum = 0;
    };

    int n;
    vector<Node> t{Node{}};
    vector<int> root{0}, free;

    MergeSplitTree(int n) : n(n)
    {
        assert(n > 0);
    }

    MergeSplitTree(const vector<ll> &a) : MergeSplitTree((int)a.size())
    {
        root[0] = build(0, n, a);
    }

    int make()
    {
        if (free.empty())
        {
            t.push_back(Node{});
            return (int)t.size() - 1;
        }
        int p = free.back();
        free.pop_back();
        return p;
    }
```

线段树合并与分裂（接上页同一结构体）：

```
    void release(int p)
    {
        t[p] = Node{};
        free.push_back(p);
    }

    void pull(int p)
    {
        assert(t[t[p].l].sum <= LLONG_MAX - t[t[p].r].sum);
        t[p].sum = t[t[p].l].sum + t[t[p].r].sum;
    }

    int build(int l, int r, const vector<ll> &a)
    {
        if (r - l == 1)
        {
            assert(a[l] >= 0);
            if (!a[l]) return 0;
            int p = make();
            t[p].sum = a[l];
            return p;
        }
        int m = l + (r - l) / 2;
        int x = build(l, m, a);
        int y = build(m, r, a);
        if (!x && !y) return 0;
        int p = make();
        t[p].l = x;
        t[p].r = y;
        pull(p);
        return p;
    }
```

线段树合并与分裂（接上页同一结构体）：

```
    int add_node(int p, int l, int r, int x, ll v)
    {
        if (!p) p = make();
        if (r - l == 1)
        {
            assert(v >= -t[p].sum);
            t[p].sum += v;
        }
        else
        {
            int m = l + (r - l) / 2;
            if (x < m)
            {
                int q = add_node(t[p].l, l, m, x, v);
                t[p].l = q;
            }
            else
            {
                int q = add_node(t[p].r, m, r, x, v);
                t[p].r = q;
            }
            pull(p);
        }
        if (t[p].sum) return p;
        release(p);
        return 0;
    }

    // Sets and coordinates are 0-based; resulting multiplicities are nonnegative.
    void add(int s, int x, ll v)
    {
        assert(0 <= s && s < (int)root.size());
        assert(0 <= x && x < n);
        assert(v <= 0 || t[root[s]].sum <= LLONG_MAX - v);
        if (!v) return;
        root[s] = add_node(root[s], 0, n, x, v);
    }
```

线段树合并与分裂（接上页同一结构体）：

```
    int meld(int p, int q, int l, int r)
    {
        if (!p || !q) return p ? p : q;
        if (r - l == 1)
            t[p].sum += t[q].sum;
        else
        {
            int m = l + (r - l) / 2;
            t[p].l = meld(t[p].l, t[q].l, l, m);
            t[p].r = meld(t[p].r, t[q].r, m, r);
            pull(p);
        }
        release(q);
        return p;
    }

    // Consume src; its set ID remains valid and becomes empty.
    void merge(int dst, int src)
    {
        assert(0 <= dst && dst < (int)root.size());
        assert(0 <= src && src < (int)root.size() && dst != src);
        assert(t[root[dst]].sum <= LLONG_MAX - t[root[src]].sum);
        root[dst] = meld(root[dst], root[src], 0, n);
        root[src] = 0;
    }
```

线段树合并与分裂（接上页同一结构体）：

```
    pair<int, int> cut(int p, int l, int r, int a, int b)
    {
        if (!p || b <= l || r <= a) return {p, 0};
        if (a <= l && r <= b) return {0, p};
        int m = l + (r - l) / 2;
        auto [x, u] = cut(t[p].l, l, m, a, b);
        auto [y, v] = cut(t[p].r, m, r, a, b);
        t[p].l = x;
        t[p].r = y;
        pull(p);
        if (!t[p].sum)
        {
            release(p);
            p = 0;
        }
        int q = 0;
        if (u || v)
        {
            q = make();
            t[q].l = u;
            t[q].r = v;
            pull(q);
        }
        return {p, q};
    }

    // Move [l,r) into a new set, even when the extracted set is empty.
    int split(int s, int l, int r)
    {
        assert(0 <= s && s < (int)root.size());
        assert(0 <= l && l <= r && r <= n);
        auto [p, q] = cut(root[s], 0, n, l, r);
        root[s] = p;
        root.push_back(q);
        return (int)root.size() - 1;
    }

    pair<int, int> cut_rank(int p, int l, int r, ll k, bool reverse)
    {
        if (!p || k == 0)
            return {p, 0};
        if (k == t[p].sum)
            return {0, p};
        int q = make();
        if (r - l == 1)
        {
            t[p].sum -= k;
            t[q].sum = k;
            return {p, q};
        }
        int m = l + (r - l) / 2;
        ll left = t[t[p].l].sum;
        ll right = t[t[p].r].sum;
        ll take_left = k < left ? k : left;
```

```
187         if (reverse)
188             take_left = k > right ? k - right : 0;
189         auto [a, b] = cut_rank(t[p].l, l, m, take_left, reverse);
190         auto [c, d] = cut_rank(t[p].r, m, r, k - take_left, reverse);
191         t[p].l = a;
192         t[p].r = c;
193         t[q].l = b;
194         t[q].r = d;
195         pull(p);
196         pull(q);
197         return {p, q};
198     }
199 
200     // Move the k smallest (or largest) occurrences into a new set.
201     int split_rank(int s, ll k, bool reverse = false)
202     {
203         assert(0 <= s && s < (int)root.size());
204         assert(0 <= k && k <= t[root[s]].sum);
205         auto [p, q] = cut_rank(root[s], 0, n, k, reverse);
206         root[s] = p;
207         root.push_back(q);
208         return (int)root.size() - 1;
209     }
```

线段树合并与分裂（接上页同一结构体）：

```
    ll query(int p, int l, int r, int a, int b) const
    {
        if (!p || b <= l || r <= a) return 0;
        if (a <= l && r <= b) return t[p].sum;
        int m = l + (r - l) / 2;
        return query(t[p].l, l, m, a, b) + query(t[p].r, m, r, a, b);
    }

    ll sum(int s, int l, int r) const
    {
        assert(0 <= s && s < (int)root.size());
        assert(0 <= l && l <= r && r <= n);
        return query(root[s], 0, n, l, r);
    }

    int select(int p, int l, int r, ll k) const
    {
        if (r - l == 1) return l;
        int m = l + (r - l) / 2;
        ll left = t[t[p].l].sum;
        if (k <= left) return select(t[p].l, l, m, k);
        return select(t[p].r, m, r, k - left);
    }

    int kth(int s, ll k) const
    {
        assert(0 <= s && s < (int)root.size());
        if (k <= 0 || k > t[root[s]].sum) return -1;
        return select(root[s], 0, n, k);
    }
};
```

模板题使用：Luogu P5494 / 线段树分裂

题意：n,m≤200000，初始集合 1 含每种值 1..n 的重数 ai≤m。0 p x y 移走闭区间到新集合；1 p t 合并并清空 t，之后不再使用 t；2 p x q 增加 x 个值 q；3 p x y 查区间重数；4 p k 查第 k 小，不存在-1。集合和值均转 0-based，闭区间转 [x-1,y)，重数用 long long。

所需模板：MergeSplitTree（第 303 页，32.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5494

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    MergeSplitTree seg(a);
    for (int i = 0; i < m; i++)
    {
        int op, p;
        cin >> op >> p;
        p--;
        if (op == 0 || op == 3)
        {
            int x, y;
            cin >> x >> y;
            if (op == 0)
                seg.split(p, x - 1, y);
            else
                cout << seg.sum(p, x - 1, y) << '\n';
        }
        else if (op == 1)
        {
            int t;
            cin >> t;
            seg.merge(p, t - 1);
        }
        else if (op == 2)
        {
            long long x;
            int q;
            cin >> x >> q;
            seg.add(p, q - 1, x);
        }
        else
        {
            long long k;
            cin >> k;
            int x = seg.kth(p, k);
            cout << (x == -1 ? -1 : x + 1) << '\n';
        }
    }
}
```

32.3 李超线段树

32.3.1 精确线段李超树：最大值编号

功能：离线给出所有可能查询的横坐标，在线插入若干条线段（仅在其横坐标范围内有效），查询某个横坐标处纵坐标最大的线段编号；也可取出该线段在此处的精确值。

核心思路：李超线段树建在排序去重的查询横坐标 `xs` 上。插入时先把线段的横坐标范围拆成 $O(\log)$ 个完整节点；在每个节点比较新旧线段在中点 `xs[m]` 的值，较优者留在节点，较劣者只可能在一侧更优：左端点处更优就递归左孩子，右端点处更优就递归右孩子。查询从根走到叶，取路径上各节点线段的最优者。线段在 x 处的值写成分数 $\frac{y_1(x_2-x)+y_2(x-x_1)}{x_2-x_1}$，用 `__int128` 交叉相乘比较，相等时取编号小者。

接口契约：构造传入非空的所有查询横坐标，排序去重后数量 M。add(x1,y1,x2,y2) 插入闭线段并返回从 1 开始的编号；可交换端点，竖直线段在该横坐标取较高端点。query(x) 返回交点最高的线段编号，同值取较早插入者；无交点返回 0，x 必须在预登记坐标内。value(id,x) 返回未约分的 (分子, 正分母)，要求 id>0 且 x 在线段内；不直接相除即可保留精度。横坐标绝对值 ≤10^9，端点纵坐标绝对值 ≤2×10^18，int128 交叉乘积在这些界内安全；不使用浮点或 epsilon。最小值问题可先把纵坐标取负，得到编号后再还原。整条直线用原 LiChao 更短；这里插入仅作用于端点范围。构造 O(M log M)，插入 O($\log^2$(M+1))，查询 O(log(M+1))，空间 O(M+ 线段数)；要求 M<INT_MAX/4，累计插入线段数 <INT_MAX。

```
struct SegmentLiChao
{
    using ll = long long;
    using I = __int128_t;

    struct Line
    {
        ll x1, y1, x2, y2;
    };

    vector<ll> xs;
    vector<Line> lines{{0, 0, 0, 0}};
    vector<int> t;

    SegmentLiChao(vector<ll> x) : xs(move(x))
    {
        sort(xs.begin(), xs.end());
        xs.erase(unique(xs.begin(), xs.end()), xs.end());
        assert(!xs.empty() && xs.front() >= -1000000000 && xs.back() <= 1000000000);
        t.resize(4 * xs.size());
    }

    pair<I, I> value(int id, ll x) const
    {
        auto [a, b, c, d] = lines[id];
        if (a == c) return {max(b, d), 1};
        return {I(b) * (c - x) + I(d) * (x - a), c - a};
    }
```

精确比较与线段插入（接上页同一结构体）：

```
    bool better(int u, int v, ll x) const
    {
        if (!u || !v) return u != 0;
        auto [a, b] = value(u, x);
        auto [c, d] = value(v, x);
        return a * d == c * b ? u < v : a * d > c * b;
    }

    void insert(int p, int l, int r, int ql, int qr, int id)
    {
        int m = (l + r) / 2;
        if (ql <= l && r <= qr)
        {
            if (better(id, t[p], xs[m])) swap(id, t[p]);
            if (!id || l == r) return;
            if (better(id, t[p], xs[l]))
                insert(p * 2, l, m, l, m, id);
            else if (better(id, t[p], xs[r]))
                insert(p * 2 + 1, m + 1, r, m + 1, r, id);
            return;
        }
        if (ql <= m) insert(p * 2, l, m, ql, qr, id);
        if (qr > m) insert(p * 2 + 1, m + 1, r, ql, qr, id);
    }

    int add(ll x1, ll y1, ll x2, ll y2)
    {
        assert(-1000000000 <= x1 && x1 <= 1000000000);
        assert(-1000000000 <= x2 && x2 <= 1000000000);
        assert(-2000000000000000000LL <= y1 && y1 <= 2000000000000000000LL);
        assert(-2000000000000000000LL <= y2 && y2 <= 2000000000000000000LL);
        if (x1 > x2)
        {
            swap(x1, x2);
            swap(y1, y2);
        }
        int id = lines.size();
        lines.push_back({x1, y1, x2, y2});
        int l = lower_bound(xs.begin(), xs.end(), x1) - xs.begin();
        int r = upper_bound(xs.begin(), xs.end(), x2) - xs.begin() - 1;
        if (l <= r) insert(1, 0, (int)xs.size() - 1, l, r, id);
        return id;
    }
```

查询最优线段编号（接上页同一结构体）：

```
    int query(ll x) const
    {
        int i = lower_bound(xs.begin(), xs.end(), x) - xs.begin();
        assert(i < (int)xs.size() && xs[i] == x);
        int p = 1, l = 0, r = (int)xs.size() - 1, ans = 0;
        while (true)
        {
            if (better(t[p], ans, x)) ans = t[p];
            if (l == r) return ans;
            int m = (l + r) / 2;
            if (i <= m)
            {
                p *= 2;
                r = m;
            }
            else
            {
                p = p * 2 + 1;
                l = m + 1;
            }
        }
    }
};
```

模板题使用：Library Checker segment_add_get_min

题意：在线段有效区间内查询最小值。预读所有查询横坐标以离散化，再按原序处理；没有查询时直接退出。题目横坐标都是整数，因此半开区间 [l,r) 换成闭区间 [l,r-1]。端点值取负，把最小值转为最大值；无结果输出 INFINITY，返回分数在本题必为整数。原线性式的端点计算在题目范围内可用 long long。

所需模板：SegmentLiChao（第 312 页，32.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/segment_add_get_min

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;

    struct Query
    {
        int type;
        long long l, r, a, b;
    };

    vector<Query> ops(n + q);
    vector<long long> xs;
    for (int i = 0; i < n + q; i++)
    {
        auto &o = ops[i];
        o.type = 0;
        if (i >= n) cin >> o.type;
        if (o.type == 0)
            cin >> o.l >> o.r >> o.a >> o.b;
        else
        {
            cin >> o.l;
            xs.push_back(o.l);
        }
    }
```

继续处理操作（接上页同一 main）：

```
    if (xs.empty()) return 0;
    SegmentLiChao t(xs);
    for (auto o : ops)
        if (o.type == 0)
            t.add(o.l, -o.a * o.l - o.b, o.r - 1, -o.a * (o.r - 1) - o.b);
        else
        {
            int id = t.query(o.l);
            if (!id)
                cout << "INFINITY\n";
            else
            {
                auto [num, den] = t.value(id, o.l);
                cout << (long long)(-num / den) << '\n';
            }
        }
}
```

32.3.2 离散李超树与最优直线编号

功能：离线给出所有查询横坐标，在线插入整条直线 $y = kx + b$（各带调用者指定的编号），查询某个横坐标处的最小函数值及取到它的直线编号（同值取编号小者）。

核心思路：李超线段树建在排序去重的横坐标 `xs` 上，节点按需动态创建。每个节点保存一条在其区间中点 `xs[m]` 处最优的直线。插入时若新线在中点更优就与节点的线交换；两条直线至多相交一次，落败的那条只可能在一侧端点处更优，于是只递归进入那一侧（两侧都不优则丢弃）。查询沿根到对应叶子的路径，取路径上所有直线在 x 处的最优者。比较用 `__int128` 计算 $kx + b$，以 (值, 编号) 字典序决胜。

接口契约：构造传入所有可能查询的横坐标，add({k,b,id}) 插入直线，query(x) 返回最小值与最小编号，要求 x 在预登记坐标内且至少有一条线。整数求值用 int128，k、b、x 为有符号 64 位且中间值可表示。单次 O(log n)。

```
struct LiChao
{
    using ll = long long;
    using I = __int128_t;

    struct Line
    {
        ll k = 0, b = 0;
        int id = -1;

        I value(ll x) const { return I(k) * x + b; }
    };

    struct Node
    {
        Line line;
        int left = -1, right = -1;
    };

    vector<ll> xs;
    vector<Node> a{Node{}};

    LiChao(vector<ll> x) : xs(x)
    {
        sort(xs.begin(), xs.end());
        xs.erase(unique(xs.begin(), xs.end()), xs.end());
        assert(!xs.empty());
    }

    bool better(Line u, Line v, ll x) const
    {
        return v.id == -1 ||
               (u.id != -1 && pair{u.value(x), u.id} < pair{v.value(x), v.id});
    }

    void add(Line line)
    {
        assert(line.id >= 0);
        add(0, 0, (int)xs.size() - 1, line);
    }
```

插入与查询（接上页同一结构体）：

```
    void add(int p, int l, int r, Line line)
    {
        int m = (l + r) / 2;
        if (better(line, a[p].line, xs[m])) swap(line, a[p].line);
        if (l == r || line.id == -1) return;
        bool left = better(line, a[p].line, xs[l]);
        bool right = better(line, a[p].line, xs[r]);
        if (!left && !right) return;
        int next = left ? a[p].left : a[p].right;
        if (next == -1)
        {
            next = (int)a.size();
            a.push_back(Node{});
            if (left)
                a[p].left = next;
            else
                a[p].right = next;
        }
        if (left)
            add(next, l, m, line);
        else
            add(next, m + 1, r, line);
    }

    pair<I, int> query(ll x) const
    {
        int idx = lower_bound(xs.begin(), xs.end(), x) - xs.begin();
        assert(idx < (int)xs.size() && xs[idx] == x);
        int p = 0, l = 0, r = (int)xs.size() - 1;
        Line best;
        while (p != -1)
        {
            if (better(a[p].line, best, x)) best = a[p].line;
            if (l == r) break;
            int m = (l + r) / 2;
            if (idx <= m)
            {
                p = a[p].left;
                r = m;
            }
            else
            {
                p = a[p].right;
                l = m + 1;
            }
        }
        assert(best.id != -1);
        return {best.value(x), best.id};
    }
};
```

模板题使用：Library Checker line_add_get_min

题意：插入整条直线并查询最小值。先收集查询横坐标，再按输入顺序处理；初始至少一条线，查询必有结果。编号由调用者递增指定，本题只输出值。题目系数与横坐标范围保证最终值可转为 long long；模板内部用 int128 比较。

所需模板：LiChao（第 317 页，32.3.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/line_add_get_min

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;

    struct Query
    {
        int type;
        long long a, b;
    };

    vector<Query> ops(n + q);
    vector<long long> xs;
    for (int i = 0; i < n + q; i++)
    {
        auto &o = ops[i];
        o.type = 0;
        if (i >= n) cin >> o.type;
        cin >> o.a;
        if (o.type == 0)
            cin >> o.b;
        else
            xs.push_back(o.a);
    }
    if (xs.empty()) return 0;
    LiChao t(xs);
    int id = 0;
    for (auto o : ops)
        if (o.type == 0)
            t.add({o.a, o.b, id++});
        else
            cout << (long long)t.query(o.a).first << '\n';
}
```

32.4 区间最值操作 & 区间历史最值

32.4.1 Segment Tree Beats：区间截断、加法与和

功能：维护整数数组，支持区间对 x 取 min（$a_i \leftarrow \min(a_i, x)$）、区间取 max、区间加，以及区间求和。

核心思路：每个节点存和、最大值 mx、严格次大值 mx2、最大值个数 cmx，最小值一侧对称，另有加法标记。区间取 min 时：若 mx $\le x$ 无影响；若 mx2 $< x <$ mx，只有最大值们改变，和减去 $(\texttt{mx} - x) \cdot \texttt{cmx}$ 后打在本节点；否则继续递归（势能分析保证总复杂度）。取 max 对称。下传时先下传加法，再用父节点的 mx/mn 截断孩子，即父节点的最值本身充当截断标记。只有一两种值时最大值与最小值会互相影响，需要同步修改。

思路中的情况	代码中的处理	行
合并孩子	求最大、严格次大及个数，最小一侧对称	28
节点只需改最大值	和加 $(x - \texttt{mx}) \cdot \texttt{cmx}$；最小值或次小值等于原最大值时一起改	75
下传	先给两孩子加父标记，再按父 mx、mn 截断孩子	102
取 min 不影响该节点	mx $\le x$ 时直接返回（剪枝）	111
完全覆盖且 mx2 $< x$	直接打在本节点；否则下传并递归两侧	115

接口契约：SegmentBeats(a) 接受 0-based 数组，chmin/chmax/add(l,r,x)、sum(l,r) 均用半开区间。空数组/空区间允许，空和为 0，重新构造用于多测。维护最大/严格第二大/次数及最小对称项；阈值等于第二极值必须递归，先下传加法再按父极值截断。n≤(INT_MAX-4)/4 且内存可用；真实值、阈值、有限延迟极值、加法标记及先加后截断的中间值须严格在 ±2^60 内，返回和须可用 long long 表示。内部和与差值乘次数用 int128，避免尚未同步的孩子临时总和溢出；不能由最终值有界推断中间和安全。建树/空间 O(n)，sum/add 为 O(log n)，单次 chmin/chmax 最坏 O(n)，混合操作总摊还 $O((n+q)\log^2(n+1))$。sum 会下传，不是 const。内部辅助函数与 t 不得直接调用/改写；不维护历史最值。

```
struct SegmentBeats
{
    using ll = long long;
    static constexpr ll inf = 1LL << 60;

    struct Node
    {
        __int128 sum = 0;
        ll mx = -inf, mx2 = -inf, mn = inf, mn2 = inf, tag = 0;
        int cmx = 0, cmn = 0;
    };

    int n;
    vector<Node> t;

    SegmentBeats(const vector<ll> &a) : n(a.size()), t(4 * a.size() + 4)
    {
        assert(a.size() <= (INT_MAX - 4) / 4);
        if (n) build(1, 0, n, a);
    }
```

极值合并与递归建树（接上页同一结构体）：

```
    void pull(int p)
    {
        auto &u = t[p];
        const auto &a = t[p * 2], &b = t[p * 2 + 1];
        u.sum = a.sum + b.sum;
        u.mx = max(a.mx, b.mx);
        u.mx2 = max(a.mx == u.mx ? a.mx2 : a.mx,
                    b.mx == u.mx ? b.mx2 : b.mx);
        u.cmx = (a.mx == u.mx ? a.cmx : 0) + (b.mx == u.mx ? b.cmx : 0);
        u.mn = min(a.mn, b.mn);
        u.mn2 = min(a.mn == u.mn ? a.mn2 : a.mn,
                    b.mn == u.mn ? b.mn2 : b.mn);
        u.cmn = (a.mn == u.mn ? a.cmn : 0) + (b.mn == u.mn ? b.cmn : 0);
        u.tag = 0;
    }

    void build(int p, int l, int r, const vector<ll> &a)
    {
        if (r - l == 1)
        {
            assert(-inf < a[l] && a[l] < inf);
            t[p].sum = t[p].mx = t[p].mn = a[l];
            t[p].cmx = t[p].cmn = 1;
            return;
        }
        int m = (l + r) / 2;
        build(p * 2, l, m, a);
        build(p * 2 + 1, m, r, a);
        pull(p);
    }
```

整段加法和单极值组截断（接上页同一结构体）：

```
    void apply_add(int p, int len, ll x)
    {
        auto &u = t[p];
        assert(-(__int128)inf < (__int128)u.mn + x &&
               (__int128)u.mx + x < inf);
        assert(-(__int128)inf < (__int128)u.tag + x &&
               (__int128)u.tag + x < inf);
        u.sum += (__int128)x * len;
        u.mx += x;
        u.mn += x;
        if (u.mx2 != -inf) u.mx2 += x;
        if (u.mn2 != inf) u.mn2 += x;
        u.tag += x;
    }

    // Only the maximum changes; equality with mx2 must descend instead.
    void apply_min(int p, ll x)
    {
        auto &u = t[p];
        if (u.mx <= x) return;
        assert(u.mx2 < x);
        u.sum += ((__int128)x - u.mx) * u.cmx;
        if (u.mn == u.mx) u.mn = x;
        else if (u.mn2 == u.mx) u.mn2 = x;
        u.mx = x;
    }

    void apply_max(int p, ll x)
    {
        auto &u = t[p];
        if (u.mn >= x) return;
        assert(x < u.mn2);
        u.sum += ((__int128)x - u.mn) * u.cmn;
        if (u.mx == u.mn) u.mx = x;
        else if (u.mx2 == u.mn) u.mx2 = x;
        u.mn = x;
    }
```

标记下传与有条件递归更新（接上页同一结构体）：

```
    void push(int p, int l, int r)
    {
        int m = (l + r) / 2;
        if (t[p].tag)
        {
            apply_add(p * 2, m - l, t[p].tag);
            apply_add(p * 2 + 1, r - m, t[p].tag);
            t[p].tag = 0;
        }
        for (int c : {p * 2, p * 2 + 1})
        {
            apply_min(c, t[p].mx);
            apply_max(c, t[p].mn);
        }
    }

    // type: 0 = chmin, 1 = chmax, 2 = add.
    void update(int p, int l, int r, int ql, int qr, ll x, int type)
    {
        if (qr <= l || r <= ql) return;
        if (type == 0 && t[p].mx <= x) return;
        if (type == 1 && t[p].mn >= x) return;
        if (ql <= l && r <= qr)
        {
            if (type == 0 && t[p].mx2 < x)
            {
                apply_min(p, x);
                return;
            }
            if (type == 1 && x < t[p].mn2)
            {
                apply_max(p, x);
                return;
            }
            if (type == 2)
            {
                apply_add(p, r - l, x);
                return;
            }
        }
        push(p, l, r);
        int m = (l + r) / 2;
        update(p * 2, l, m, ql, qr, x, type);
        update(p * 2 + 1, m, r, ql, qr, x, type);
        pull(p);
    }
```

公开修改与区间和查询（接上页同一结构体）：

```
    void chmin(int l, int r, ll x)
    {
        assert(0 <= l && l <= r && r <= n && -inf < x && x < inf);
        if (l < r) update(1, 0, n, l, r, x, 0);
    }

    void chmax(int l, int r, ll x)
    {
        assert(0 <= l && l <= r && r <= n && -inf < x && x < inf);
        if (l < r) update(1, 0, n, l, r, x, 1);
    }

    void add(int l, int r, ll x)
    {
        assert(0 <= l && l <= r && r <= n && -inf < x && x < inf);
        if (l < r) update(1, 0, n, l, r, x, 2);
    }

    __int128 query(int p, int l, int r, int ql, int qr)
    {
        if (qr <= l || r <= ql) return 0;
        if (ql <= l && r <= qr) return t[p].sum;
        push(p, l, r);
        int m = (l + r) / 2;
        return query(p * 2, l, m, ql, qr) + query(p * 2 + 1, m, r, ql, qr);
    }

    ll sum(int l, int r)
    {
        assert(0 <= l && l <= r && r <= n);
        __int128 ans = l < r ? query(1, 0, n, l, r) : 0;
        assert(LLONG_MIN <= ans && ans <= LLONG_MAX);
        return (ll)ans;
    }
};
```

模板题使用：Library Checker Range Chmin Chmax Add Range Sum

题意：维护数组，操作 0/1/2/3 分别为区间 chmin、chmax、加法、求和。全部使用 0-based 非空半开区间。N,Q≤200000；所有时刻元素绝对值 ≤10^12，修改参数绝对值 ≤2×10^12。因此延迟值满足本模板的范围要求，公开区间和也可用 long long 表示。输入区间不需要加减 1，按操作号调用对应方法。

所需模板：SegmentBeats（第 320 页，32.4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/range_chmin_chmax_add_range_sum

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    SegmentBeats t(a);
    while (q--)
    {
        int type, l, r;
        cin >> type >> l >> r;
        if (type == 3)
        {
            cout << t.sum(l, r) << '\n';
            continue;
        }
        long long x;
        cin >> x;
        if (type == 0) t.chmin(l, r, x);
        else if (type == 1) t.chmax(l, r, x);
        else t.add(l, r, x);
    }
}
// END USAGE
```

第三十三章　二叉搜索树 & 平衡树

33.1　二叉搜索树 & 平衡树

33.1.1　PBDS 有序树与顺序统计

功能： GNU pb_ds 红黑树实现的有序集合，在 `set` 的接口之外支持按排名访问：`find_by_order(k)` 取第 k 个（从 0 计）元素，`order_of_key(x)` 求严格小于 x 的元素个数。

核心思路： `tree_order_statistics_node_update` 让每个节点额外维护子树大小，排名查询沿根向下按左子树大小决定走向。集合不允许重复元素，需要可重集合时存 (值, 全局唯一编号) 二元组。

接口契约： GNU libstdc++ 扩展，包含 ext/pb_ds/assoc_container.hpp 和 ext/pb_ds/tree_policy.hpp。ost<T> 为去重有序集合；insert(x)、erase(x)、find(x)、lower_bound(x)、upper_bound(x) 沿用集合接口。order_of_key(x) 为严格小于 x 的元素数，find_by_order(k) 返回 0-based 第 k 个的迭代器，越界返回 end。默认 less<T>；改用其他严格弱序比较器时，排名和边界均按该比较器解释。单次 O(log n)，空间 O(n)，假设比较为 O(1)。重复值使用 ost<pair<T,int»，插入 {x,id} 且 id 全局唯一、1≤id<INT_MAX。order_of_key({x,0}) 统计小于 x，order_of_key({x,INT_MAX}) 统计不大于 x；find_by_order(k)->first 取值，须先检查 end。删除一个 x：it=lower_bound({x,0})，检查 it!=end 且 it->first==x 后 erase(it)。不可用 less_equal 模拟多重集合。前驱先检查迭代器是否为 begin，再递减；空集合和无解不得解引用 end。

只需键值查改而不需要有序排名时，gp_hash_table / cc_hash_table 及选型建议见第 262 页。

序列中间插删、截取与版本共享的 GNU rope 见第 328 页。

```
template <class T, class C = less<T>>
using ost = __gnu_pbds::tree<T,
                             __gnu_pbds::null_type,
                             C,
                             __gnu_pbds::rb_tree_tag,
                             __gnu_pbds::tree_order_statistics_node_update>;
```

模板题使用：Library Checker ordered_set

题意：维护无重复整数集合，支持第 k 小、计数及含等号的前驱/后继。find_by_order 使用 0-based 排名；order_of_key 只计严格小于 x，故不大于 x 的数量需补上相等元素。越界查询按题意输出 -1。

所需模板：ost（第 326 页，33.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/ordered_set

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q, x;
    cin >> n >> q;
    ost<int> s;
    while (n--)
    {
        cin >> x;
        s.insert(x);
    }
    while (q--)
    {
        int op;
        cin >> op >> x;
        if (op == 0) s.insert(x);
        if (op == 1) s.erase(x);
        if (op == 2)
        {
            auto it = s.find_by_order(x - 1);
            cout << (it == s.end() ? -1 : *it) << '\n';
        }
        if (op == 3) cout << s.order_of_key(x) + (s.find(x) != s.end()) << '\n';
        if (op == 4)
        {
            auto it = s.upper_bound(x);
            cout << (it == s.begin() ? -1 : *--it) << '\n';
        }
        if (op == 5)
        {
            auto it = s.lower_bound(x);
            cout << (it == s.end() ? -1 : *it) << '\n';
        }
    }
}
```

33.1.2 GNU rope 与版本共享

功能：GNU 扩展的 `rope`：支持在任意位置插入、删除、截取子串和按下标访问的序列；复制一个 rope 只共享根，修改副本不影响原序列，可用来保存多个历史版本。

核心思路：rope 用平衡二叉树把序列拼接成若干叶子块，节点带引用计数并共享子树。复制只增加根的引用；修改时复制受影响的路径与叶块（写时复制），其余部分继续共享，所以各版本互不干扰。

接口契约：实际类型为 __gnu_cxx::rope<T>，不是 std::rope；它是 GNU 扩展序列，包含 ext/rope。本工具链随后重新包含 cassert，保留断言接口。rp<int>、rp<char> 表示序列，默认空。size()、empty()、push_back(x)、insert(p,x)、erase(p,len)、replace(p,x)、substr(p,len) 直接沿用 rope；位置为 0-based，len 是长度而非右端点。约定 0≤p≤size，删除/截取要求 len≤size-p，读写单点要求 p<size。r[i] 读取第 i 项；substr 返回新序列。rp<T> b=a 共享底层树，随后通过容器接口修改 b 不影响 a，可用 vector<rp<int» 保存分支版本。复制构造共享根为 O(1)；随机访问沿树路径，正常平衡时 O(log n)。插入/删除/截取涉及路径、叶块复制和可能的再平衡，不能把每次操作都写成严格最坏 O(log n)；释放最后引用可能递归回收整棵子树。长期保留所有版本的内存不能按单个序列 O(n) 估计。不可缓存修改前的迭代器或使用可变代理跨版本写入。仅使用默认内存分配器及普通整数/字符值；函数生成型 rope 不在本模板范围。可持久化队列已通过定长构造加头尾下标版本：Q 个零单元，复制根与 l/r，入队 replace(r++,x)，出队读取 [l++]；直接 push_back/erase 版本在针对数据上耗尽内存。线上 942 ms、591.26 MiB，具体失败与通过范围见 docs/ROPE-NOTES.md。适合需要序列中间插删、截取和少量分支快照的场景；只按下标读写通常使用 vector 更直接。需要区间和、翻转等懒标记聚合时使用隐式 Treap，rope 本身不提供这些接口。下方示例通过修改 b 演示 a 保持原版本。

只需键值查改而不需要有序排名时，gp_hash_table / cc_hash_table 及选型建议见第 262 页。

```
template <class T> using rp = __gnu_cxx::rope<T>;
```

```
rp<int> a;
a.push_back(3);
a.insert(0, 7);
rp<int> b = a;
b.replace(1, 9);
b.erase(0, 1);
auto c = a.substr(0, 1);
int x = a[1];
```

模板题使用：Library Checker persistent_queue

题意：维护可持久化队列，每次从指定版本入队或出队并建立新版本。题目的 -1 号空版本映射为下标 0；rope 副本和头尾指针一起复制，replace 写入不会改变历史版本，出队只推进该版本队头。

所需模板：rp（第 328 页，33.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/persistent_queue

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    vector<rp<int>> s(q + 1);
    vector<int> l(q + 1), r(q + 1);
    s[0] = rp<int>(q, 0);
    for (int i = 1; i <= q; i++)
    {
        int op, t, x;
        cin >> op >> t;
        ++t;
        s[i] = s[t];
        l[i] = l[t];
        r[i] = r[t];
        if (op == 0)
        {
            cin >> x;
            s[i].replace(r[i]++, x);
        }
        else
        {
            cout << s[i][l[i]++] << '\n';
        }
    }
}
```

33.2 Treap

33.2.1 FHQ Treap：可重集合与顺序统计

功能：维护可重整数集合，支持插入、删除一个值、求排名、求第 k 小、求严格前驱和严格后继。

核心思路：无旋 Treap：节点按值有序、按随机优先级成堆。`split(p,x)` 把树分成值小于 x（`equal` 为真时为不超过 x）和其余两部分，`merge` 按优先级合并两棵值域有序的树。插入时按 x 分裂后把新节点夹在中间合并；删除时分成小于、等于、大于 x 三段，等于段去掉其根节点（合并它的左右子树）再拼回。排名、第 k 小只沿树下走并用子树大小计数，前驱、后继转化为按排名求第 k 小。

接口契约：insert(x) 插入一个副本，erase(x) 删除一个副本并返回是否存在；rank(x) 为严格小于 x 的元素数加一，x 不必存在；kth(k) 要求 1≤k≤size()。prev/next 为严格前驱/后继，不存在时返回 nullopt。支持重复值、LLONG_MIN 和 LLONG_MAX，分裂不使用 x-1。随机优先级下单次期望 O(log n)，并非确定性最坏界。节点池不回收已删除节点，空间按累计插入次数计算；可在构造时指定随机种子。可合并平衡树：合并两棵树时把较小的一棵逐个 insert 进较大的一棵（启发式合并），每个元素至多移动 O(log n) 次，总 $O(n \log^2 n)$。

```
struct OrderedTreap
{
```

```
    using ll = long long;

    struct Node
    {
        int l = 0, r = 0, siz = 0;
        ll val = 0;
        unsigned long long pri = 0;
    };

    vector<Node> a{Node{}};
    mt19937_64 rng;
    int root = 0;

    OrderedTreap(unsigned long long seed = 712367821) : rng(seed) {}

    void pull(int p) { a[p].siz = a[a[p].l].siz + a[a[p].r].siz + 1; }

    int node(ll x)
    {
        int p = a.size();
        a.push_back({0, 0, 1, x, rng()});
        return p;
    }

    // left: val < x, or val <= x when equal is true. No x-1 overflow.
    void split(int p, ll x, int &l, int &r, bool equal = false)
    {
        if (!p)
        {
            l = r = 0;
            return;
        }
        if (a[p].val < x || (equal && a[p].val == x))
        {
            l = p;
            split(a[p].r, x, a[p].r, r, equal);
        }
        else
        {
            r = p;
            split(a[p].l, x, l, a[p].l, equal);
        }
        pull(p);
    }

    int merge(int l, int r)
    {
        if (!l || !r) return l ? l : r;
        if (a[l].pri > a[r].pri)
        {
            a[l].r = merge(a[l].r, r);
            pull(l);
            return l;
        }
        a[r].l = merge(l, a[r].l);
        pull(r);
```

```
        return r;
    }

    void insert(ll x)
    {
        int p = node(x), l, r;
        split(root, x, l, r);
        root = merge(merge(l, p), r);
    }

    bool erase(ll x)
    {
        int l, mid, r;
        split(root, x, l, mid);
        split(mid, x, mid, r, true);
        bool found = mid != 0;
        if (mid) mid = merge(a[mid].l, a[mid].r);
        root = merge(l, merge(mid, r));
        return found;
    }

    int size() const { return a[root].siz; }

    int less(ll x, bool equal = false) const
    {
        int p = root, ans = 0;
        while (p)
        {
            if (a[p].val < x || (equal && a[p].val == x))
            {
                ans += a[a[p].l].siz + 1;
                p = a[p].r;
            }
            else
                p = a[p].l;
        }
        return ans;
    }

    int rank(ll x) const { return less(x) + 1; }

    ll kth(int k) const
    {
        assert(1 <= k && k <= size());
        int p = root;
        while (true)
        {
            int left = a[a[p].l].siz;
            if (k == left + 1) return a[p].val;
            if (k <= left)
                p = a[p].l;
            else
            {
                k -= left + 1;
                p = a[p].r;
            }
```

```
        }
    }

    optional<ll> prev(ll x) const
    {
        int k = less(x);
        if (!k) return nullopt;
        return kth(k);
    }

    optional<ll> next(ll x) const
    {
        int k = less(x, true);
        if (k == size()) return nullopt;
        return kth(k + 1);
    }
};
```

模板题使用：Luogu P3369

题意：维护可重复整数集合，支持插入、删除一个值、排名、第 k 小、严格前驱和严格后继。题目保证前驱/后继存在，因此可解引用返回值。

所需模板：OrderedTreap（第 329 页，33.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3369

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    OrderedTreap g;
    while (n--)
    {
        int op;
        long long x;
        cin >> op >> x;
        if (op == 1)
            g.insert(x);
        else if (op == 2)
            g.erase(x);
        else if (op == 3)
            cout << g.rank(x) << '\n';
        else if (op == 4)
            cout << g.kth(x) << '\n';
        else if (op == 5)
            cout << *g.prev(x) << '\n';
        else
            cout << *g.next(x) << '\n';
    }
}
```

33.2.2 隐式 Treap：顺序反转、位翻转、加值与区间和

功能：维护一个序列，支持在指定位置插入、删除一段、翻转一段的顺序、对一段加值、把一段 0/1 值逐位取反，以及查询一段的和。

核心思路：隐式 Treap：以子树大小作为下标，`split(p,k)` 分出前 k 个元素。每个区间操作都先分成前 $l-1$ 个、$[l,r]$、其余三段，对中段的根打标记后再合并回去。标记有三种：顺序翻转 `rev`（交换左右孩子）；值变换写成 $x \mapsto \pm x + \texttt{tag}$，取负时连同已有的 `tag` 一起取负，下传时先取负再加。位翻转即 $x \mapsto 1-x$，用一次取负加一次加 1 实现。`split/merge` 访问节点前先 `push`。

接口契约：insert(k,value) 在前 k 个元素后插入，0≤k≤size()；区间接口 erase、reverse、flip_bits、add、query 均使用 1-based 非空闭区间。reverse 反转元素顺序；flip_bits 要求区间内全为 0/1，逐位取反并保持顺序，不检查这一前提。query 返回和，二进制区间即 1 的个数。values() 递归中序返回完整序列，O(n)。其余操作期望 O(log n)，不是最坏界。符号与加法标记表示 s*x+b，位翻转通过取负后加 1 实现，下传必须先符号再加法；顺序反转另行维护。所有值、和、标记、乘法及取负中间结果须在 long long 范围内。删除不回收节点，空间按累计插入次数计算；辅助函数和节点数组只供内部使用。

```
struct SequenceTreap
{
    using ll = long long;

    struct Node
    {
        int l = 0, r = 0, siz = 0;
        ll val = 0, sum = 0, tag = 0;
        unsigned long long pri = 0;
        bool rev = false, neg = false;
    };

    vector<Node> a{Node{}};
    mt19937_64 rng;
    int root = 0;

    SequenceTreap(unsigned long long seed = 712367821) : rng(seed) {}

    int size() const { return a[root].siz; }

    void pull(int p)
    {
        a[p].siz = a[a[p].l].siz + a[a[p].r].siz + 1;
        a[p].sum = a[a[p].l].sum + a[p].val + a[a[p].r].sum;
    }

    void apply_add(int p, ll x)
    {
        if (!p) return;
        a[p].val += x;
        a[p].sum += x * a[p].siz;
        a[p].tag += x;
    }

    void apply_negate(int p)
    {
        if (!p) return;
        a[p].val = -a[p].val;
        a[p].sum = -a[p].sum;
        a[p].tag = -a[p].tag;
        a[p].neg = !a[p].neg;
    }
```

顺序反转、懒标记下传与分裂（接上页同一结构体）：

```
    void apply_reverse(int p)
    {
        if (!p) return;
        swap(a[p].l, a[p].r);
        a[p].rev = !a[p].rev;
    }

    void push(int p)
    {
        if (a[p].rev)
        {
            apply_reverse(a[p].l);
            apply_reverse(a[p].r);
            a[p].rev = false;
        }
        if (a[p].neg)
        {
            apply_negate(a[p].l);
            apply_negate(a[p].r);
            a[p].neg = false;
        }
        if (a[p].tag)
        {
            apply_add(a[p].l, a[p].tag);
            apply_add(a[p].r, a[p].tag);
            a[p].tag = 0;
        }
    }

    // Split into the first k elements and the rest.
    void split(int p, int k, int &l, int &r)
    {
        if (!p)
        {
            l = r = 0;
            return;
        }
        push(p);
        int left = a[a[p].l].siz;
        if (k > left)
        {
            l = p;
            split(a[p].r, k - left - 1, a[p].r, r);
        }
        else
        {
            r = p;
            split(a[p].l, k, l, a[p].l);
        }
        pull(p);
    }
```

合并、插入、删除与顺序反转（接上页同一结构体）：

```
    int merge(int l, int r)
    {
        if (!l || !r) return l ? l : r;
        if (a[l].pri > a[r].pri)
        {
            push(l);
            a[l].r = merge(a[l].r, r);
            pull(l);
            return l;
        }
        push(r);
        a[r].l = merge(l, a[r].l);
        pull(r);
        return r;
    }

    // Insert after the first k elements, 0 <= k <= size().
    void insert(int k, ll value)
    {
        assert(0 <= k && k <= size());
        int p = a.size(), l, r;
        a.push_back({0, 0, 1, value, value, 0, rng(), false, false});
        split(root, k, l, r);
        root = merge(merge(l, p), r);
    }

    // All range operations use 1-based inclusive intervals.
    void erase(int l, int r)
    {
        assert(1 <= l && l <= r && r <= size());
        int x, y, z;
        split(root, r, y, z);
        split(y, l - 1, x, y);
        root = merge(x, z);
    }

    void reverse(int l, int r)
    {
        assert(1 <= l && l <= r && r <= size());
        int x, y, z;
        split(root, r, y, z);
        split(y, l - 1, x, y);
        apply_reverse(y);
        root = merge(merge(x, y), z);
    }
```

位翻转、加法、查询与输出（接上页同一结构体）：

```
    // Requires every value in [l,r] to be 0 or 1; preserves their order.
    void flip_bits(int l, int r)
    {
        assert(1 <= l && l <= r && r <= size());
        int x, y, z;
        split(root, r, y, z);
        split(y, l - 1, x, y);
        apply_negate(y);
        apply_add(y, 1);
        root = merge(merge(x, y), z);
    }

    void add(int l, int r, ll value)
    {
        assert(1 <= l && l <= r && r <= size());
        int x, y, z;
        split(root, r, y, z);
        split(y, l - 1, x, y);
        apply_add(y, value);
        root = merge(merge(x, y), z);
    }

    ll query(int l, int r)
    {
        assert(1 <= l && l <= r && r <= size());
        int x, y, z;
        split(root, r, y, z);
        split(y, l - 1, x, y);
        ll ans = a[y].sum;
        root = merge(merge(x, y), z);
        return ans;
    }

    void collect(int p, vector<ll> &out)
    {
        if (!p) return;
        push(p);
        collect(a[p].l, out);
        out.push_back(a[p].val);
        collect(a[p].r, out);
    }

    vector<ll> values()
    {
        vector<ll> answer;
        collect(root, answer);
        return answer;
    }
};
```

模板题使用：Luogu P3391

题意：初始序列为 1..n，多次翻转闭区间 [l,r]，最后输出序列。插入参数为前面已有元素数，翻转区间为 1-based 闭区间。

所需模板：SequenceTreap（第 333 页，33.2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3391

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    SequenceTreap g;
    for (int i = 1; i <= n; i++) g.insert(i - 1, i);
    while (m--)
    {
        int l, r;
        cin >> l >> r;
        g.reverse(l, r);
    }
    for (long long x : g.values()) cout << x << ' ';
    cout << '\n';
}
```

33.2.3 隐式 Treap 位翻转与顺序反转

第 333 页 `flip_bits(l,r)` 逐位取反（要求区间元素均为 0/1），`reverse(l,r)` 反转顺序；均为 1-based 闭区间。二进制区间的 `query` 即 1 的个数。取反为 $x \mapsto 1-x$，用 $x \mapsto sx+b\,(s \in \{1,-1\})$ 表示：取负时 $s \leftarrow -s, b \leftarrow -b$ 再加 1，下传**先符号、再加法**。顺序反转只交换儿子，与取负、加法可交换。所有标记与中间量须在 `long long` 内，不能对 `LLONG_MIN` 取负。逐次 `insert` 构建期望 $O(n \log n)$，单次操作期望 $O(\log n)$，不是最坏界。

33.3 Splay 树

33.3.1 Splay：可重集合与顺序统计

功能：维护可重整数集合，支持插入、删除一个值、求排名、求第 k 小、求严格前驱和严格后继。

核心思路：Splay 树，相同值存成一个节点并计数 `cnt`。每次操作沿二叉搜索树走到目标或最后访问的节点，再把它伸展到根，保证均摊对数。插入时值已存在就增计数，否则新建叶子。删除先 `find` 把目标伸展到根；计数多于 1 就减计数，否则断开左右子树，在左子树里把最大节点伸展为根，再把右子树接成它的右孩子。排名、第 k 小、前驱、后继都是一次带计数的下行查找。

接口契约：insert/erase/rank/kth/prev/next 与有序 Treap 同义，严格前驱后继不存在返回 nullopt。重复键计数存储；失败查询也伸展最后访问节点，单次均摊 O(log n)。不回收节点，空间按累计新建节点数计算。支持 long long 全范围；不包含序列 Splay、虚子树信息或矩阵动态 DP。

```
struct OrderedSplay
{
    using ll = long long;

    struct Node
    {
        int ch[2]{}, fa = 0, cnt = 0, siz = 0;
        ll val = 0;
    };

    vector<Node> a{Node{}};
    int root = 0;

    int size() const { return a[root].siz; }

    void pull(int x) { a[x].siz = a[a[x].ch[0]].siz + a[x].cnt + a[a[x].ch[1]].siz; }

    int node(ll value, int parent)
    {
        int x = a.size();
        a.push_back({{0, 0}, parent, 1, 1, value});
        return x;
    }

    void rotate(int x)
    {
        int y = a[x].fa, z = a[y].fa;
        int k = a[y].ch[1] == x;
        int w = a[x].ch[k ^ 1];
        if (z)
            a[z].ch[a[z].ch[1] == y] = x;
        else
            root = x;
        a[x].fa = z;
        a[y].ch[k] = w;
        if (w) a[w].fa = y;
        a[x].ch[k ^ 1] = y;
        a[y].fa = x;
        pull(y);
        pull(x);
    }
```

伸展与查找（接上页同一结构体）：

```
    void splay(int x)
    {
        while (a[x].fa)
        {
            int y = a[x].fa, z = a[y].fa;
            if (z)
            {
                bool same = (a[y].ch[1] == x) == (a[z].ch[1] == y);
                rotate(same ? y : x);
            }
            rotate(x);
        }
    }

    // Splay the last visited node even when the value does not exist.
    int find(ll value)
    {
        int x = root, last = 0;
        while (x && a[x].val != value)
        {
            last = x;
            x = a[x].ch[value > a[x].val];
        }
        if (x)
            splay(x);
        else if (last)
            splay(last);
        return x;
    }
```

插入与删除（接上页同一结构体）：

```
    void insert(ll value)
    {
        int x = root, parent = 0;
        while (x && a[x].val != value)
        {
            parent = x;
            x = a[x].ch[value > a[x].val];
        }
        if (x)
        {
            ++a[x].cnt;
            pull(x);
        }
        else
        {
            x = node(value, parent);
            if (parent)
                a[parent].ch[value > a[parent].val] = x;
            else
                root = x;
        }
        splay(x);
    }

    bool erase(ll value)
    {
        int x = find(value);
        if (!x) return false;
        if (a[x].cnt > 1)
        {
            --a[x].cnt;
            pull(x);
            return true;
        }
        int l = a[x].ch[0], r = a[x].ch[1];
        if (l) a[l].fa = 0;
        if (r) a[r].fa = 0;
        if (!l)
        {
            root = r;
            return true;
        }
        root = l;
        x = l;
        while (a[x].ch[1]) x = a[x].ch[1];
        splay(x);
        a[x].ch[1] = r;
        if (r) a[r].fa = x;
        pull(x);
        return true;
    }
```

排名与第 k 小（接上页同一结构体）：

```
    int rank(ll value)
    {
        int x = root, last = 0, ans = 1;
        while (x)
        {
            last = x;
            if (value <= a[x].val)
                x = a[x].ch[0];
            else
            {
                ans += a[a[x].ch[0]].siz + a[x].cnt;
                x = a[x].ch[1];
            }
        }
        if (last) splay(last);
        return ans;
    }

    ll kth(int k)
    {
        assert(1 <= k && k <= size());
        int x = root;
        while (true)
        {
            int left = a[a[x].ch[0]].siz;
            if (k <= left)
                x = a[x].ch[0];
            else if (k <= left + a[x].cnt)
            {
                splay(x);
                return a[x].val;
            }
            else
            {
                k -= left + a[x].cnt;
                x = a[x].ch[1];
            }
        }
    }
```

严格前驱与后继（接上页同一结构体）：

```
    optional<ll> prev(ll value)
    {
        int x = root, last = 0, answer = 0;
        while (x)
        {
            last = x;
            if (a[x].val < value)
            {
                answer = x;
                x = a[x].ch[1];
            }
            else
                x = a[x].ch[0];
        }
        if (last) splay(last);
        if (!answer) return nullopt;
        splay(answer);
        return a[answer].val;
    }

    optional<ll> next(ll value)
    {
        int x = root, last = 0, answer = 0;
        while (x)
        {
            last = x;
            if (a[x].val > value)
            {
                answer = x;
                x = a[x].ch[0];
            }
            else
                x = a[x].ch[1];
        }
        if (last) splay(last);
        if (!answer) return nullopt;
        splay(answer);
        return a[answer].val;
    }
};
```

模板题使用：Luogu P3369

题意：维护可重复整数集合，支持插入、删除一个值、排名、第 k 小、严格前驱和严格后继。题目保证前驱/后继存在，因此可解引用返回值。

所需模板：OrderedSplay（第 339 页，33.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3369

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    OrderedSplay g;
    while (n--)
    {
        int op;
        long long x;
        cin >> op >> x;
        if (op == 1)
            g.insert(x);
        else if (op == 2)
            g.erase(x);
        else if (op == 3)
            cout << g.rank(x) << '\n';
        else if (op == 4)
            cout << g.kth(x) << '\n';
        else if (op == 5)
            cout << *g.prev(x) << '\n';
        else
            cout << *g.next(x) << '\n';
    }
}
```

33.4 替罪羊树

33.4.1 替罪羊树：重复计数与墓碑清理

区分 `size`（副本总数）、`nodes`（物理节点数）、`live`（仍有副本的不同键数）；重复值增加计数，删空后保留墓碑。排名只累计副本数。回溯时若某孩子节点数占比超过 3/4，或有效键不足物理节点一半，就中序收集活节点按中点重建。树高 $O(\log(d+2))$（d 为不同键数），查询最坏对数，更新摊还对数，单次重建可能线性；空间按累计分配量计。排名定义为严格较小副本数加一（空树为 1）；严格前驱后继用 `optional` 表示缺失。

33.5 笛卡尔树

33.5.1 笛卡尔树与相等值优先级

功能：给定数组 a，构造它的笛卡尔树：按下标是二叉搜索树（中序遍历即原顺序），按值是小根堆，因此子树恰好对应一段区间，根是该区间的最小值。常用于把“区间最小值”问题转成树上问题。

核心思路：从左到右加入元素，单调栈 `st` 自底向上保存当前树的最右链，栈中值单调不降。加入 a_i 时弹出所有严格大于 a_i 的栈顶，最后弹出的那个（整条被弹链的顶端）成为 i 的左孩子；若栈非空，则 i 接为新栈顶的右孩子，替换其原右孩子，然后 i 入栈。相等值不弹出，故较早的相等元素留在上方。处理完后栈底即根。

接口契约：cartesian_tree(a) 返回 CartesianTree {root,left,right}，下标 0..n-1，缺失孩子及空树根为-1。中序遍历为原下标顺序，按值为小根堆；相等值保留较小下标在上方，即区间最左最小值为根。输入不修改，n 须能用 int 表示，元素比较须构成严格弱序，浮点 NaN 不适用。

比较 O(1) 时，构造 O(n)，结果数组及辅助单调栈均 O(n)。每点入栈一次、出栈至多一次；单调栈维护树的最右路径，不模拟 DFS。弹出节点中最后一个成为新节点的左孩子，剩余栈顶的右孩子改成新节点。P5854 用 1..n 排列，n≤10^7；P5854 已用 C++14 GCC9 满分通过，排名未核验。

```
struct CartesianTree
{
    int root;
    vector<int> left, right;
};

template <class T>
CartesianTree cartesian_tree(const vector<T> &a)
{
    int n = a.size();
    CartesianTree tree{-1, vector<int>(n, -1), vector<int>(n, -1)};
    vector<int> st;
    st.reserve(n);
    for (int i = 0; i < n; i++)
    {
        int last = -1;
        while (!st.empty() && a[i] < a[st.back()])
        {
            last = st.back();
            st.pop_back();
        }
        tree.left[i] = last;
        if (!st.empty()) tree.right[st.back()] = i;
        st.push_back(i);
    }
    if (n) tree.root = st.front();
    return tree;
}
```

模板题使用：P5854 笛卡尔树

题意：输入 1..n 的排列，构造小根笛卡尔树并输出左右孩子的异或校验和。孩子由 0-based 转为 1-based，缺失孩子为 0；乘法使用 long long。

所需模板：CartesianTree（第 346 页，33.5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5854

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<int> a(n);
    for (int &x : a) cin >> x;
    auto tree = cartesian_tree(a);
    long long left = 0;
    long long right = 0;
    for (int i = 0; i < n; i++)
    {
        left ^= 1LL * (i + 1) * (tree.left[i] + 2);
        right ^= 1LL * (i + 1) * (tree.right[i] + 2);
    }
    cout << left << ' ' << right << '\n';
}
```

第三十四章　可持久化数据结构

34.1　可持久化线段树

34.1.1　主席树：静态区间第 k 小

问题描述：给定不再修改的数组，多次询问区间 $[l,r]$ 内第 k 小的值（重复值按次数计）。

核心思路：值离散化后，对每个前缀 $a_1..a_i$ 建一棵权值线段树版本 `root[i]`，节点 `sum` 记该值域内的元素个数；版本 i 由版本 $i-1$ 复制一条根到叶的路径得到，其余节点共享。区间 $[l,r]$ 的值域计数等于版本 r 减版本 $l-1$。查询时两版本同步从根下走：左孩子计数差 $c \geq k$ 就进左子树，否则 $k \leftarrow k-c$ 进右子树，到叶即答案。

接口契约：构造传入 0-based 数组，kth(l,r,k) 的区间使用 1-based 闭区间；要求 1≤k≤r-l+1。离散化后每个前缀一个版本，单次 O(log n)，总空间 O(n log n)。树上路径第 k 小：每个点在父版本上插入点权，路径 (u,v) 用 root[u]+root[v]-root[lca]-root[fa[lca]] 四个版本同步二分。区间不同数个数：位置 i 在 pre[i]（上一次出现位置）处插入，查询 [l,r] 统计版本 r 减版本 l-1 中 pre<l 的个数。

```
struct PersistentKth
{
    struct Node
    {
        int l = 0, r = 0, sum = 0;
    };

    vector<Node> t{Node{}};
    vector<int> root{0};
    vector<long long> vals;

    PersistentKth(const vector<long long> &a) : vals(a)
    {
        sort(vals.begin(), vals.end());
        vals.erase(unique(vals.begin(), vals.end()), vals.end());
        for (long long x : a)
        {
            int p = lower_bound(vals.begin(), vals.end(), x) - vals.begin();
            root.push_back(add(root.back(), 0, (int)vals.size() - 1, p));
        }
    }

    int add(int old, int l, int r, int x)
    {
        int p = t.size();
        Node copy = t[old];
        t.push_back(copy);
        ++t[p].sum;
        if (l < r)
        {
            int m = (l + r) / 2;
            if (x <= m)
            {
                int v = add(t[old].l, l, m, x);
                t[p].l = v;
            }
```

```
            else
            {
                int v = add(t[old].r, m + 1, r, x);
                t[p].r = v;
            }
        }
        return p;
    }

    long long kth(int l, int r, int k) const
    {
        assert(1 <= l && l <= r && r < (int)root.size() && 1 <= k && k <= r - l + 1);
        int u = root[l - 1], v = root[r], L = 0, R = (int)vals.size() - 1;
        while (L < R)
        {
            int m = (L + R) / 2, c = t[t[v].l].sum - t[t[u].l].sum;
            if (k <= c)
            {
                u = t[u].l;
                v = t[v].l;
                R = m;
            }
            else
            {
                k -= c;
                u = t[u].r;
                v = t[v].r;
                L = m + 1;
            }
        }
        return vals[L];
    }
};
```

模板题使用：Luogu P3834

题意：给定不再修改的数组，查询闭区间 [l,r] 内第 k 小的值，重复值按出现次数计数。构造传入 0-based vector，查询 l、r、k 均从 1 开始。

所需模板：PersistentKth（第 348 页，34.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3834

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    std::vector<long long> a(n);
    for (auto &x : a) std::cin >> x;
    PersistentKth tree(a);
    while (m--)
    {
        int l, r, k;
        std::cin >> l >> r >> k;
        std::cout << tree.kth(l, r, k) << '\n';
    }
}
```

34.1.2 可持久化区间加与版本拼接

功能：维护长度为 n 的数组的多个版本（初始全 0），支持在某版本上区间加得到新版本、把一个版本的某段替换成另一个版本同位置的值得到新版本，以及查询任一版本的区间和。

核心思路：可持久化线段树，按需复制节点，标记永久化：区间加只在完全覆盖的节点上累加 `lazy` 并修正 `sum`，标记从不下传；节点的真实区间和等于其 `sum` 加上所有祖先 `lazy` 之和乘区间长度，查询时用 `carry` 累计祖先标记。拼接 `replace` 同时走目标树 p 与源树 q，`offset` 累计“源祖先标记和减目标祖先标记和”；到完全覆盖处直接借用源子树 q，若 `offset` $\neq 0$ 则复制一份把 `offset` 补进它的 `lazy`，使挂到目标祖先下后数值不变。

接口契约：PersistentRange(n) 建立长度 n 的全零版本 0，n≥1；使用隐式节点，不预建整棵树。add(v,l,r,x) 在版本 v 的 [l,r) 加 x；splice(dst,src,l,r) 将 dst 的 [l,r) 替换为 src 同一位置的值，其他位置保留 dst；二者均返回新版本编号。query(v,l,r) 返回区间和，const 且不分配节点。下标 0-based、区间半开，允许空区间；空修改及零增量仍创建版本但复用根，整段拼接直接共享源根。全体版本属于同一对象、同一坐标域。懒标记保留在祖先，用 offset 修正跨版本拼接；每次修改/拼接/查询 O(log n)，版本别名 O(1)，总节点 O(q log n)。sum、lazy、delta、carry、offset 及其中间和差和长度乘积均须在 long long 内；节点总数和版本数须小于 INT_MAX。t、root 可读以统计空间或查看根，不要外部改写。

```
struct PersistentRange
{
    using ll = long long;

    struct Node
    {
        int left = 0, right = 0;
        ll sum = 0, lazy = 0;
    };

    int n;
    vector<Node> t{Node{}};
    vector<int> root{0};

    explicit PersistentRange(int size) : n(size) { assert(n > 0); }

    int clone(int p)
    {
        Node node = t[p];
        t.push_back(node);
        return int(t.size()) - 1;
    }

    void pull(int p, int len)
    {
        t[p].sum = t[t[p].left].sum + t[t[p].right].sum + t[p].lazy * len;
    }
```

区间加与版本拼接（接上页同一结构体）：

```
    int change(int p, int l, int r, int ql, int qr, ll value)
    {
        if (qr <= l || r <= ql) return p;
        int u = clone(p);
        if (ql <= l && r <= qr)
        {
            t[u].lazy += value;
            t[u].sum += value * (r - l);
            return u;
        }
        int m = l + (r - l) / 2;
        int a = change(t[p].left, l, m, ql, qr, value);
        int b = change(t[p].right, m, r, ql, qr, value);
        t[u].left = a;
        t[u].right = b;
        pull(u, r - l);
        return u;
    }

    int replace(int p, int q, int l, int r, int ql, int qr, ll offset)
    {
        if (qr <= l || r <= ql) return p;
        if (ql <= l && r <= qr)
        {
            if (offset == 0) return q;
            int u = clone(q);
            t[u].lazy += offset;
            t[u].sum += offset * (r - l);
            return u;
        }
        int u = clone(p), m = l + (r - l) / 2;
        offset += t[q].lazy - t[p].lazy;
        int a = replace(t[p].left, t[q].left, l, m, ql, qr, offset);
        int b = replace(t[p].right, t[q].right, m, r, ql, qr, offset);
        t[u].left = a;
        t[u].right = b;
        pull(u, r - l);
        return u;
    }
```

只读查询与对外接口（接上页同一结构体）：

```
    ll get(int p, int l, int r, int ql, int qr, ll carry) const
    {
        if (qr <= l || r <= ql) return 0;
        if (ql <= l && r <= qr) return t[p].sum + carry * (r - l);
        carry += t[p].lazy;
        int m = l + (r - l) / 2;
        return get(t[p].left, l, m, ql, qr, carry) +
               get(t[p].right, m, r, ql, qr, carry);
    }

    void check(int version, int l, int r) const
    {
        assert(0 <= version && version < int(root.size()));
        assert(0 <= l && l <= r && r <= n);
    }

    int add(int version, int l, int r, ll value)
    {
        check(version, l, r);
        int p = l == r || value == 0 ? root[version]
                                     : change(root[version], 0, n, l, r, value);
        root.push_back(p);
        return int(root.size()) - 1;
    }

    int splice(int destination, int source, int l, int r)
    {
        check(destination, l, r);
        check(source, l, r);
        int p = l == r ? root[destination]
                       : replace(root[destination], root[source], 0, n, l, r, 0);
        root.push_back(p);
        return int(root.size()) - 1;
    }

    ll query(int version, int l, int r) const
    {
        check(version, l, r);
        return l == r ? 0 : get(root[version], 0, n, l, r, 0);
    }
};
```

34.1.3 可持久化并查集

功能：可持久化并查集：在任一历史版本上合并两个集合得到新版本，并能查询任一版本中两点是否连通、所在集合大小。

核心思路：并查集数组用可持久化线段树保存：叶子值非负表示父亲，负数表示该点是根且值为负的集合大小；空节点读出 -1，即初始全是大小为 1 的单点。因不能路径压缩，`merge` 按大小合并，把较小集合的根挂到较大集合的根下，树高为对数级；`find` 从 x 反复读父亲直到读到负数。一次合并只修改两个叶子（被挂根的父亲、新根的大小），各复制一条路径，作为新版本的根。

接口契约：PersistentDSU(n)，n>0，点号 0..n-1，版本 0 为单点集合。merge(ver,x,y) 从任意版本合并并返回新版本号，已连通也创建版本；copy(ver) 复制版本。find、same、size 只读，不压缩路径；按大小合并，等大时第一根

挂到第二根。区别于普通 dsu 的固定方向合并。稀疏可持久化线段树的叶子存父亲或负大小，空节点表示-1。初始化 O(1)，copy 均摊 O(1)，其余 $O(\log^2(n+1))$；成功合并增加 O(log(n+1)) 节点。节点数/版本数须可用 int 索引。root、t 只读，get/set 内部函数勿直接调用；旧版本永不被改写。

```
struct PersistentDSU
{
    struct Node
    {
        int l = 0, r = 0, v = -1;
    };

    int n;
    vector<Node> t{Node{}};
    vector<int> root{0};

    PersistentDSU(int n) : n(n)
    {
        assert(n > 0);
    }

    int get(int p, int l, int r, int x) const
    {
        if (!p) return -1;
        if (r - l == 1) return t[p].v;
        int m = l + (r - l) / 2;
        if (x < m) return get(t[p].l, l, m, x);
        return get(t[p].r, m, r, x);
    }

    int set(int old, int l, int r, int x, int v)
    {
        Node node = t[old];
        int p = t.size();
        t.push_back(node);
        if (r - l == 1)
            t[p].v = v;
        else
        {
            int m = l + (r - l) / 2;
            if (x < m)
            {
                int q = set(node.l, l, m, x, v);
                t[p].l = q;
            }
            else
            {
                int q = set(node.r, m, r, x, v);
                t[p].r = q;
            }
        }
        return p;
    }
```

可持久化并查集（接上页同一结构体）：

```
    // Queries never compress paths or allocate nodes. Vertices are 0-based.
    int find(int ver, int x) const
    {
        assert(0 <= ver && ver < (int)root.size());
        assert(0 <= x && x < n);
        int p = get(root[ver], 0, n, x);
        while (p >= 0)
        {
            x = p;
            p = get(root[ver], 0, n, x);
        }
        return x;
    }

    int size(int ver, int x) const
    {
        return -get(root[ver], 0, n, find(ver, x));
    }

    bool same(int ver, int x, int y) const
    {
        return find(ver, x) == find(ver, y);
    }

    int copy(int ver)
    {
        assert(0 <= ver && ver < (int)root.size());
        root.push_back(root[ver]);
        return (int)root.size() - 1;
    }

    // Always create one version; ties attach the first root to the second.
    int merge(int ver, int x, int y)
    {
        x = find(ver, x);
        y = find(ver, y);
        if (x == y) return copy(ver);
        int a = get(root[ver], 0, n, x);
        int b = get(root[ver], 0, n, y);
        if (a < b)
        {
            swap(x, y);
            swap(a, b);
        }
        int p = set(root[ver], 0, n, x, y);
        p = set(p, 0, n, y, a + b);
        root.push_back(p);
        return (int)root.size() - 1;
    }
};
```

模板题使用：Luogu P3402 / 可持久化并查集

题意：n≤100000，m≤200000。1 a b 合并；2 k 回到第 k 次操作后的状态，可回到 0；3 a b 输出是否连通。每种操作均产生版本，操作号等于版本号，点号转 0-based，无异或。

所需模板：PersistentDSU（第 353 页，34.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3402

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    PersistentDSU d(n);
    for (int i = 1; i <= m; i++)
    {
        int op, x, y;
        cin >> op >> x;
        if (op == 2)
            d.copy(x);
        else
        {
            cin >> y;
            if (op == 1)
                d.merge(i - 1, x - 1, y - 1);
            else
            {
                cout << d.same(i - 1, x - 1, y - 1) << '\n';
                d.copy(i - 1);
            }
        }
    }
}
```

34.1.4 可持久化区间拼接的懒标记

代码见第 350 页。从全零版本开始，同一对象内版本共享坐标域；标记永久化：sum = left.sum + right.sum + lazy · len，查询只读。拼接把目标版本某半开区间替换为源版本同一区间；递归参数 offset 下降时更新为 offset + source.lazy − destination.lazy，完整覆盖时复制源节点并把 offset 加到 lazy 与 sum（offset 为零直接共享）。修改、拼接 $O(\log n)$ 新节点，查询 $O(\log n)$ 零新节点。不支持错位平移粘贴，不回收节点。和、差、标记与长度乘积须在 long long 内；节点数与版本数小于 `INT_MAX`。题目时间戳可回退时，用 ver[t] 映射时间戳到模板版本，不能把 add 返回的追加编号当时间戳。

34.1.5 杭州 2023 K：跨版本拼接应用

0-based，$F_i(r)$ 为 $[i,r]$ 剩余牌数，p 为 i 之后第一张与 a_i 相同的牌（无则 $p=n$）：

$$F_i(r)=\begin{cases}1+F_{i+1}(r), & r<p,\\ F_{p+1}(r), & r\ge p.\end{cases}$$

从 $i=n-1$ 向前：复制版本 $i+1$，$[i,p)$ 加 1，$[p,n)$ 拼接自版本 $p+1$。询问 $[l,r]$ 读版本 $l-1$ 的坐标 $r-1$；强制在线。构建 $O(n\log n)$，查询 $O(q\log n)$。

34.2 可持久化平衡树

34.2.1 可持久化有序 Treap：分叉版本与顺序统计

功能：维护可重整数集合的多个历史版本：在任一旧版本上插入或删除一个值得到新版本；在指定版本上查询元素个数、排名、第 k 小、严格前驱与后继。

核心思路：Treap 节点存值、重复次数 `cnt` 与子树大小 `siz`，修改时沿查找路径复制节点（`clone`），未改动的子树在版本间共享。插入递归到位置后，若新孩子优先级更高就把它旋转上来，只旋转新复制的节点。删除时若 `cnt` >1 只减计数，否则用 `merge` 合并左右子树代替该节点；值不存在时子树原样返回，不产生新节点。查询从 `root[v]` 沿树下走：`less` 统计小于（或不超过）x 的个数，前驱、后继转成按排名求 `kth`。

接口契约：版本 0 为空。insert(v,x)、erase(v,x)、copy(v) 追加并返回新版本；删除不存在的值也追加相同集合的版本。size/less/rank/kth/prev/next 均显式传版本且只读，不创建版本或树节点。重复值计数，rank 为严格小于 x 的副本数加一；kth 为 1-based 且要求合法，严格前后继缺失返回 nullopt。支持 long long 全范围，不用 x±1。随机优先级下期望 O(log(d+1))，d 为版本不同键数；copy 摊还 O(1)，没有最坏对数保证。递归复制更新路径，节点不回收，空间按全部历史分配节点与版本数计。节点/版本/各版副本数须小于 INT_MAX；t、root 及带节点编号的辅助函数属内部状态。当前常见 64 位布局每节点 32 字节；500000 次随机不同键插入实测约 1200 万节点，不预留的 vector 扩容可能超过 512MiB 峰值。预留 26*n+1 仅是明确预算而非上界，先核对内存限制；重新构造清空全部版本。

```
struct PersistentOrderedTreap
{
    using ll = long long;

    struct Node
    {
        ll val = 0;
        unsigned long long pri = 0;
        int l = 0, r = 0, siz = 0, cnt = 0;
    };

    vector<Node> t{Node{}};
    vector<int> root{0};
    mt19937_64 rng;

    PersistentOrderedTreap(unsigned long long seed = 712367821) : rng(seed) {}

    void check(int v) const { assert(0 <= v && v < (int)root.size()); }

    int clone(int p)
    {
        Node node = t[p];
        t.push_back(node);
        return (int)t.size() - 1;
    }

    void pull(int p) { t[p].siz = t[t[p].l].siz + t[t[p].r].siz + t[p].cnt; }
```

复制路径插入与旋转（接上页同一结构体）：

```
    // Only rotate freshly copied nodes; the promoted child was copied by add.
    int add(int p, ll x)
    {
        if (!p)
        {
            t.push_back({x, rng(), 0, 0, 1, 1});
            return (int)t.size() - 1;
        }
        int u = clone(p);
        if (x == t[u].val)
            t[u].cnt++;
        else if (x < t[u].val)
        {
            int q = add(t[u].l, x);
            t[u].l = q;
            if (t[q].pri > t[u].pri)
            {
                t[u].l = t[q].r;
                t[q].r = u;
                pull(u);
                u = q;
            }
        }
        else
        {
            int q = add(t[u].r, x);
            t[u].r = q;
            if (t[q].pri > t[u].pri)
            {
                t[u].r = t[q].l;
                t[q].l = u;
                pull(u);
                u = q;
            }
        }
        pull(u);
        return u;
    }
```

复制路径合并与删除（接上页同一结构体）：

```
68      int merge(int l, int r)
69      {
70          if (!l || !r) return l ? l : r;
71          if (t[l].pri > t[r].pri)
72          {
73              int u = clone(l);
74              int q = merge(t[u].r, r);
75              t[u].r = q;
76              pull(u);
77              return u;
78          }
79          int u = clone(r);
80          int q = merge(l, t[u].l);
81          t[u].l = q;
82          pull(u);
83          return u;
84      }
85
86      int remove(int p, ll x)
87      {
88          if (!p) return 0;
89          if (x == t[p].val)
90          {
91              if (t[p].cnt == 1) return merge(t[p].l, t[p].r);
92              int u = clone(p);
93              t[u].cnt--;
94              pull(u);
95              return u;
96          }
97          bool left = x < t[p].val;
98          int old = left ? t[p].l : t[p].r;
99          int q = remove(old, x);
100         if (q == old) return p;
101         int u = clone(p);
102         if (left)
103             t[u].l = q;
104         else
105             t[u].r = q;
106         pull(u);
107         return u;
108     }
```

版本接口、大小与排名（接上页同一结构体）：

```
    // Updates append versions; erase removes one occurrence, or changes nothing.
    int insert(int v, ll x)
    {
        check(v);
        int p = add(root[v], x);
        root.push_back(p);
        return (int)root.size() - 1;
    }

    int erase(int v, ll x)
    {
        check(v);
        int p = remove(root[v], x);
        root.push_back(p);
        return (int)root.size() - 1;
    }

    int copy(int v)
    {
        check(v);
        root.push_back(root[v]);
        return (int)root.size() - 1;
    }

    int size(int v) const
    {
        check(v);
        return t[root[v]].siz;
    }

    int less(int v, ll x, bool equal = false) const
    {
        check(v);
        int p = root[v], ans = 0;
        while (p)
        {
            if (t[p].val < x || (equal && t[p].val == x))
            {
                ans += t[t[p].l].siz + t[p].cnt;
                p = t[p].r;
            }
            else
                p = t[p].l;
        }
        return ans;
    }

    int rank(int v, ll x) const { return less(v, x) + 1; }
```

第 k 小与严格前驱后继（接上页同一结构体）：

```
    // Ranks and k are 1-based. No sentinel values are reserved.
    ll kth(int v, int k) const
    {
        assert(1 <= k && k <= size(v));
        int p = root[v];
        while (true)
        {
            int left = t[t[p].l].siz;
            if (k <= left)
                p = t[p].l;
            else if (k <= left + t[p].cnt)
                return t[p].val;
            else
            {
                k -= left + t[p].cnt;
                p = t[p].r;
            }
        }
    }

    optional<ll> prev(int v, ll x) const
    {
        int k = less(v, x);
        if (!k) return nullopt;
        return kth(v, k);
    }

    optional<ll> next(int v, ll x) const
    {
        int k = less(v, x, true);
        if (k == size(v)) return nullopt;
        return kth(v, k + 1);
    }
};
```

模板题使用：Luogu P3835 【模板】可持久化平衡树

题意：维护可重整数集合的历史版本；每条插入、删除或排名/第 k 小/严格前后继查询都从指定旧版本生成版本 i。删除缺失值忽略但仍产生版本；查询输出后 copy(v)，避免版本编号错位。第 k 小保证合法；缺失前后继分别输出-2147483647、2147483647，仅由驱动映射 optional。n≤500000、键绝对值 ≤10^9。reserve(26*n+1) 是约 32 字节节点的预留预算，不是分配上界；最大 n 预留约 397MiB，需先核对内存限制，不能将本地通过当作线上 AC。

所需模板：PersistentOrderedTreap（第 357 页，34.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3835

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    PersistentOrderedTreap tree;
    tree.root.reserve(n + 1);
    tree.t.reserve(26ULL * n + 1);
    for (int i = 1; i <= n; i++)
    {
        int v, op;
        long long x;
        cin >> v >> op >> x;
        if (op == 1)
            tree.insert(v, x);
        else if (op == 2)
            tree.erase(v, x);
        else
        {
            if (op == 3) cout << tree.rank(v, x) << '\n';
            if (op == 4) cout << tree.kth(v, (int)x) << '\n';
            if (op == 5) cout << tree.prev(v, x).value_or(-2147483647LL) << '\n';
            if (op == 6) cout << tree.next(v, x).value_or(2147483647LL) << '\n';
            tree.copy(v);
        }
    }
}
```

34.3 可持久化字典树

34.3.1 前缀可持久化 01 字典树

功能：按顺序追加若干个 bits 位非负整数，形成序列 $a_0, a_1, \ldots$；查询时给定区间 $[l, r)$ 与整数 x，求区间内某个 a_i 与 x 的最大异或值。

核心思路：可持久化 01 字典树：root[k] 是前 k 个数构成的字典树，每个节点的 cnt 记经过它的数的个数。追加 x 时从高位到低位复制旧版本的一条路径（共 bits + 1 个新节点），其余子树共享；节点 0 是自环的空节点。查询时从 root[r] 与 root[l] 同步下走，每位优先走与 x 该位相反的孩子，两版本在该孩子上 cnt 之差为正才说明区间内存在这样的数，否则只能走相同位。

接口契约：PersistentXorTrie(bits,cap)，1≤bits≤64，0≤cap≤(INT_MAX−1)/(bits+1)，构造时预留最多 cap 个键。append(x) 追加一个 uint64_t 原值，重复值按出现次数保留；满容量或 x 不满足位宽时返回 false 且逻辑状态不变。size() 为键数；max_xor(l,r,x) 从 0-based 半开区间 [l,r) 选择一个键 y，返回最大 x xor y，空区间 nullopt。查询 x 允许完整 uint64；不是子集异或、任意分支版本或字符串 Trie。构造 O(cap*bits) 容量空间，追加/查询 O(bits)；复制后首次追加为恢复预留空间可花 O(size*bits)。append 分配失败传播异常，逻辑内容不变；构造参数非法抛异常，查询下标是调用者前置条件，assert 只作调试检查。内部成员只读；不要复用已移动对象。

```
struct PersistentXorTrie
{
    using U = uint64_t;
    struct Node
    {
        int ch[2] = {}, cnt = 0;
    };
    int bits, cap;
    vector<Node> t{Node{}};
    vector<int> root{0};

    PersistentXorTrie(int bits, int cap) : bits(bits), cap(cap)
    {
        if (bits < 1 || bits > 64 || cap < 0)
            throw invalid_argument("PersistentXorTrie domain");
        if (cap > (INT_MAX - 1) / (bits + 1))
            throw length_error("PersistentXorTrie capacity");
        reserve();
    }

    void reserve()
    {
        t.reserve(1 + size_t(cap) * (bits + 1));
        root.reserve(size_t(cap) + 1);
    }

    int size() const { return (int)root.size() - 1; }
```

前缀追加与区间单值最大异或（接上页同一结构体）：

```
    bool append(U x)
    {
        if (size() == cap || (bits < 64 && (x >> bits))) return false;
        reserve(); // A copied vector need not preserve its original capacity.
        int old = root.back(), p = t.size();
        Node copy = t[old];
        t.push_back(copy);
        root.push_back(p);
        t[p].cnt++;
        for (int d = bits - 1; d >= 0; d--)
        {
            int b = x >> d & 1, q = t.size();
            old = t[old].ch[b];
            copy = t[old];
            t.push_back(copy);
            t[p].ch[b] = q;
            p = q;
            t[p].cnt++;
        }
        return true;
    }

    optional<U> max_xor(int l, int r, U x) const
    {
        assert(0 <= l && l <= r && r <= size());
        if (l == r) return nullopt;
        int a = root[l], b = root[r];
        U y = 0;
        for (int d = bits - 1; d >= 0; d--)
        {
            int k = (x >> d & 1) ^ 1;
            if (t[t[b].ch[k]].cnt == t[t[a].ch[k]].cnt) k ^= 1;
            y |= U(k) << d;
            a = t[a].ch[k];
            b = t[b].ch[k];
        }
        return x ^ y;
    }
};
```

第三十五章　树套树

35.1　树状数组套权值线段树

35.1.1　动态区间第 k 小（树状数组套权值线段树）

问题描述：给定数组，支持单点修改为新值，以及询问区间 $[l,r]$ 内第 k 小的值。

核心思路：把所有初值和将来的新值一起离散化。按位置建树状数组，第 i 个节点挂一棵动态开点的权值线段树，记录树状数组覆盖的那段位置中每个值出现的次数。修改先在 $O(\log n)$ 棵树中删去旧值、再插入新值；计数变为 0 的节点挂到 `free_head` 空闲链上复用。查询时取出前缀 r 与前缀 $l-1$ 对应的两组树根，在值域上同步二分：左孩子计数之和（前一组减后一组）不少于 k 就整体往左走，否则减去后往右走。

接口契约：DynamicKth(a,future) 传入非空 0-based 初值，future 包含所有将来可能赋入的新值；内部自动合并初值并离散化。set(pos,value) 单点赋值，kth(l,r,k) 查询闭区间第 k 小，下标均为 1-based；1≤k≤r-l+1。允许重复值与完整 long long 权值；新值必须已在离散化集合内，操作可按原顺序执行，不保留历史版本。每个树状数组节点对应一棵可变权值线段树；删除旧值后插入新值。空节点回收到 free_head 链，重复修改不会按操作次数累计泄漏节点。设 D 为不同值数，S=n+future.size()，预处理 O(S log S+n log n log(D+1))，单次修改及查询 O(log n log(D+1))。节点数上界 B=(ceil(log2(D))+1) · $\sum$ lowbit(i)，求和 i=1..n；空间 O(S+n+B)。update、modify、id 和节点池均为内部接口；不能手动减去不存在的值。

```
struct DynamicKth
{
    using ll = long long;

    struct Node
    {
        int l = 0, r = 0, sum = 0;
    };

    int n, free_head = 0;
    vector<int> bit, rank;
    vector<ll> vals;
    vector<Node> t{Node{}};

    DynamicKth(const vector<ll> &a, vector<ll> future = {})
        : n(a.size()), bit(n + 1), rank(n + 1)
    {
        assert(n > 0);
        vals = future;
        vals.insert(vals.end(), a.begin(), a.end());
        sort(vals.begin(), vals.end());
        vals.erase(unique(vals.begin(), vals.end()), vals.end());
        for (int i = 1; i <= n; i++)
        {
            rank[i] = id(a[i - 1]);
            modify(i, rank[i], 1);
        }
    }

    int id(ll value) const
    {
        int p = lower_bound(vals.begin(), vals.end(), value) - vals.begin();
```

```
        assert(p < (int)vals.size() && vals[p] == value);
        return p;
    }

    int new_node()
    {
        if (!free_head)
        {
            t.push_back(Node{});
            return (int)t.size() - 1;
        }
        int p = free_head;
        free_head = t[p].l;
        t[p] = Node{};
        return p;
    }

    int update(int p, int l, int r, int x, int delta)
    {
        if (!p) p = new_node();
        t[p].sum += delta;
        if (l < r)
        {
            int m = (l + r) / 2;
            if (x <= m)
            {
                int child = update(t[p].l, l, m, x, delta);
                t[p].l = child;
            }
            else
            {
                int child = update(t[p].r, m + 1, r, x, delta);
                t[p].r = child;
            }
        }
        if (!t[p].sum)
        {
            t[p].l = free_head;
            free_head = p;
            return 0;
        }
        return p;
    }

    void modify(int pos, int x, int delta)
    {
        for (int i = pos; i <= n; i += i & -i)
            bit[i] = update(bit[i], 0, (int)vals.size() - 1, x, delta);
    }
```

```
    void set(int pos, ll value)
    {
        assert(1 <= pos && pos <= n);
        int x = id(value);
        if (rank[pos] == x) return;
        modify(pos, rank[pos], -1);
        modify(pos, x, 1);
        rank[pos] = x;
    }

    ll kth(int l, int r, int k) const
    {
        assert(1 <= l && l <= r && r <= n && 1 <= k && k <= r - l + 1);
        vector<int> a, b;
        for (int i = r; i; i -= i & -i) a.push_back(bit[i]);
        for (int i = l - 1; i; i -= i & -i) b.push_back(bit[i]);
        int L = 0, R = (int)vals.size() - 1;
        while (L < R)
        {
            int count = 0;
            for (int p : a) count += t[t[p].l].sum;
            for (int p : b) count -= t[t[p].l].sum;
            bool left = k <= count;
            if (!left) k -= count;
            for (int &p : a) p = left ? t[p].l : t[p].r;
            for (int &p : b) p = left ? t[p].l : t[p].r;
            int m = (L + R) / 2;
            if (left)
                R = m;
            else
                L = m + 1;
        }
        return vals[L];
    }
};
```

模板题使用：Luogu P2617

题意：支持单点赋值及闭区间第 k 小。先读完操作，把所有修改的新值收集到 future，再构造并按原顺序执行；初值 vector 为 0-based，位置、查询区间和 k 为 1-based。Query 仅保存输入字段。

所需模板：DynamicKth（第 365 页，35.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2617

```
int main()
{
    struct Query
    {
        char op;
        int x, y, k;
        long long value;
    };

    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    std::vector<long long> a(n), future;
    for (auto &x : a) std::cin >> x;
    std::vector<Query> queries(m);
    for (auto &q : queries)
    {
        std::cin >> q.op;
        if (q.op == 'Q')
            std::cin >> q.x >> q.y >> q.k;
        else
        {
            std::cin >> q.x >> q.value;
            future.push_back(q.value);
        }
    }
    DynamicKth tree(a, future);
    for (const auto &q : queries)
    {
        if (q.op == 'Q')
            std::cout << tree.kth(q.x, q.y, q.k) << '\n';
        else
            tree.set(q.x, q.value);
    }
}
```

第三十六章　K-D Tree

36.1　动态二维 K-D Tree：单点累加与矩形和

功能：在线维护平面上的带权点集，支持给某个坐标的点累加权值（不存在则新建），以及查询闭矩形内所有点的权值和。

核心思路：二维 K-D Tree：按深度交替以 x、y 为划分轴（坐标相同时比较另一维），每个节点存自身点与权值，以及子树的包围盒、点数和权值和。插入沿划分方向下行，相同坐标直接累加；回溯时若某个孩子的点数超过子树的 3/4，就中序收集该子树所有节点，用 `nth_element` 取中位数递归重建（替罪羊式重构），保持树高对数级。查询时包围盒与矩形不相交就剪掉，被矩形完全包含就直接返回子树和，否则计入本节点的点后递归两个孩子。

接口契约：KDTreeSum<Coord=int>(reserve_points=0) 初始为空；Coord 取 int 或 long long。add(x,y,delta) 累加有符号 long long 权值，相同坐标只占一个节点；零权节点仍保留，不支持删除坐标。query(x1,y1,x2,y2) 查询闭矩形和，左下端超过右上端则返回 0。坐标只比较，不相减，不用有限哨兵；可用所选整数类型极值。当前各点绝对权值总和及更新中间量须在 long long 内。不同点数 d、预留容量和节点编号须小于 INT_MAX。按轴交替分割，子树过重时递归收集编号并原位重建；树高 O(log(d+1))。nth_element 按平均线性选择计，插入摊还 $O(\log^2(d+1))$，单次大重建可 O(d log(d+1))；矩形查询最坏 O(d)，不能保证对数。空间 O(d)，预留容量另计，默认 int 节点常见为 56 字节；紧内存题预留最大不同点数，避免扩容峰值。a/root 及带节点编号的函数均为内部状态；可重新赋值空对象重置。

```
template <class Coord = int> struct KDTreeSum
{
    using ll = long long;
    using Point = array<Coord, 2>;

    struct Node
    {
        Point point{}, low{}, high{};
        array<int, 2> child{};
        int size = 1;
        ll value = 0, sum = 0;
    };

    vector<Node> a;
    int root = 0;

    KDTreeSum(int reserve_points = 0)
    {
        a.reserve(reserve_points + 1);
        a.push_back(Node{});
        a[0].size = 0;
    }

    bool less(Point x, Point y, int axis) const
    {
        if (x[axis] != y[axis]) return x[axis] < y[axis];
        return x[axis ^ 1] < y[axis ^ 1];
    }
```

包围盒维护、递归收集与重建（接上页同一结构体）：

```
    void pull(int p)
    {
        a[p].low = a[p].high = a[p].point;
        a[p].size = 1;
        a[p].sum = a[p].value;
        for (int q : a[p].child)
        {
            if (!q) continue;
            a[p].size += a[q].size;
            a[p].sum += a[q].sum;
            for (int d = 0; d < 2; d++)
            {
                a[p].low[d] = min(a[p].low[d], a[q].low[d]);
                a[p].high[d] = max(a[p].high[d], a[q].high[d]);
            }
        }
    }

    void collect(int p, vector<int> &v)
    {
        if (!p) return;
        collect(a[p].child[0], v);
        v.push_back(p);
        collect(a[p].child[1], v);
    }

    int build(vector<int> &v, int l, int r, int axis)
    {
        if (l == r) return 0;
        int m = l + (r - l) / 2;
        nth_element(v.begin() + l,
                    v.begin() + m,
                    v.begin() + r,
                    [&](int x, int y)
                    {
                        return less(a[x].point, a[y].point, axis);
                    });
        int p = v[m];
        a[p].child[0] = build(v, l, m, axis ^ 1);
        a[p].child[1] = build(v, m + 1, r, axis ^ 1);
        pull(p);
        return p;
    }
```

点权累加与闭矩形查询（接上页同一结构体）：

```
    int add(int p, Point point, ll delta, int axis)
    {
        if (!p)
        {
            p = a.size();
            a.push_back(Node{});
            a[p].point = a[p].low = a[p].high = point;
            a[p].value = a[p].sum = delta;
            return p;
        }
        if (a[p].point == point)
            a[p].value += delta;
        else
        {
            int side = !less(point, a[p].point, axis);
            a[p].child[side] = add(a[p].child[side], point, delta, axis ^ 1);
        }
        pull(p);
        int heavy = max(a[a[p].child[0]].size, a[a[p].child[1]].size);
        if (4LL * heavy <= 3LL * a[p].size) return p;
        vector<int> v;
        v.reserve(a[p].size);
        collect(p, v);
        return build(v, 0, v.size(), axis);
    }

    void add(Coord x, Coord y, ll delta)
    {
        root = add(root, {x, y}, delta, 0);
    }

    ll query(int p, Point low, Point high) const
    {
        if (!p) return 0;
        bool covered = true, inside = true;
        for (int d = 0; d < 2; d++)
        {
            if (a[p].high[d] < low[d] || high[d] < a[p].low[d]) return 0;
            covered &= low[d] <= a[p].low[d] && a[p].high[d] <= high[d];
            inside &= low[d] <= a[p].point[d] && a[p].point[d] <= high[d];
        }
        if (covered) return a[p].sum;
        ll answer = inside ? a[p].value : 0;
        for (int q : a[p].child) answer += query(q, low, high);
        return answer;
    }

    ll query(Coord x1, Coord y1, Coord x2, Coord y2) const
    {
        if (x1 > x2 || y1 > y2) return 0;
        return query(root, {x1, y1}, {x2, y2});
    }
};
```

36.2 静态 K-D Tree 最近与最远点

功能：给定静态点集，查询离给定点最近（或最远）的 k 个点的编号，按欧氏距离排序，等距时编号小者在前。

核心思路：按深度轮换坐标轴，以中位数递归建树，每个节点记录子树包围盒与子树中的最小编号 `first`。查询维护大小至多 k 的大根堆，键为 (平方距离, 编号)；求最远时用负的距离，统一成“越小越好”。`bound` 给出子树内可能达到的最好键：最近时是查询点到包围盒的距离，最远时是到包围盒最远角的距离取负，编号取 `first`。堆满且该下界不优于堆顶就整棵剪掉；否则用当前点更新堆，并先搜下界更好的孩子。最后把堆依次弹出再反转。

接口契约：KDNearest<D>(points)，D>0，Point=array<int,D>，坐标为有符号 32 位整数，输入编号 0..n-1 且允许重合点。query(q,k,far=false) 返回最近 k 个原编号；far=true 改为最远 k 个。结果按距离优先、等距编号小者优先排列，0≤k≤n；k=0 返回空，包含与查询重合的点，不自动排除自身。平方距离用 128 位整数精确计算。p 保留输入顺序；p/a/root 仅供读取，重建请重新赋值。递归中位数建树，平均 O(Dn log(n+1))，空间 O(Dn)；包围盒剪枝没有对任意数据的次线性保证，单次最坏 O(Dn+n log(k+1))，额外 O(k+log(n+1)) 空间。search/build/bound 为内部辅助，不直接调用。

```
template <int D = 2> struct KDNearest
{
    static_assert(D > 0);
    using Point = array<int, D>;
    using Wide = __int128_t;
    using Key = pair<Wide, int>;

    struct Node
    {
        Point low, high;
        array<int, 2> child{-1, -1};
        int first;
    };

    vector<Point> p;
    vector<Node> a;
    int root;

    KDNearest(const vector<Point> &points = {}) : p(points), a(p.size())
    {
        vector<int> order(p.size());
        iota(order.begin(), order.end(), 0);
        root = build(order, 0, order.size(), 0);
    }
```

中位数建树与距离界（接上页同一结构体）：

```
    int build(vector<int> &order, int l, int r, int axis)
    {
        if (l == r) return -1;
        int m = (l + r) / 2;
        nth_element(order.begin() + l, order.begin() + m, order.begin() + r,
            [&](int x, int y)
            {
                return pair(p[x][axis], x) < pair(p[y][axis], y);
            });
        int u = order[m];
        a[u].low = a[u].high = p[u];
        a[u].first = u;
        a[u].child[0] = build(order, l, m, (axis + 1) % D);
        a[u].child[1] = build(order, m + 1, r, (axis + 1) % D);
        for (int v : a[u].child)
        {
            if (v == -1) continue;
            a[u].first = min(a[u].first, a[v].first);
            for (int d = 0; d < D; d++)
            {
                a[u].low[d] = min(a[u].low[d], a[v].low[d]);
                a[u].high[d] = max(a[u].high[d], a[v].high[d]);
            }
        }
        return u;
    }

    Wide distance(const Point &x, const Point &y) const
    {
        Wide ans = 0;
        for (int d = 0; d < D; d++)
        {
            Wide t = Wide(x[d]) - y[d];
            ans += t * t;
        }
        return ans;
    }

    Key bound(int u, const Point &q, bool far) const
    {
        Wide ans = 0;
        for (int d = 0; d < D; d++)
        {
            Wide l = Wide(a[u].low[d]) - q[d];
            Wide r = Wide(a[u].high[d]) - q[d];
            if (far) ans -= max(l * l, r * r);
            else if (l > 0) ans += l * l;
            else if (r < 0) ans += r * r;
        }
        return {ans, a[u].first};
    }
```

剪枝搜索与查询入口（接上页同一结构体）：

```
    void search(int u, const Point &q, int k, bool far,
                priority_queue<Key> &heap) const
    {
        if (u == -1) return;
        if ((int)heap.size() == k && bound(u, q, far) >= heap.top()) return;
        Wide dist = distance(p[u], q);
        Key cur{far ? -dist : dist, u};
        if ((int)heap.size() < k || cur < heap.top())
        {
            heap.push(cur);
            if ((int)heap.size() > k) heap.pop();
        }
        int x = a[u].child[0];
        int y = a[u].child[1];
        if (x == -1 || (y != -1 && bound(y, q, far) < bound(x, q, far)))
        {
            swap(x, y);
        }
        search(x, q, k, far, heap);
        search(y, q, k, far, heap);
    }

    vector<int> query(const Point &q, int k, bool far = false) const
    {
        assert(0 <= k && k <= (int)p.size());
        if (k == 0) return {};
        priority_queue<Key> heap;
        search(root, q, k, far, heap);
        vector<int> ans;
        while (!heap.empty())
        {
            ans.push_back(heap.top().second);
            heap.pop();
        }
        reverse(ans.begin(), ans.end());
        return ans;
    }
};
```

36.3 动态 K-D Tree：矩形和与重建边界

KDTreeSum 按深度交替坐标轴，重复坐标累加权值，孩子过重（超过 3/4）时 nth_element 重建。树高 $O(\log(d+1))$，矩形查询最坏 $O(d)$；插入摊还 $O(\log^2(d+1))$。矩形闭边界；绝对权值和及中间量须在 long long 内。零权保留节点，负权不影响剪枝。

第三十七章　动态树

37.1　Link Cut Tree

37.1.1　Link-cut tree：动态森林与路径异或

功能：维护带点权的动态森林，支持连边、删边、修改点权、判断两点是否连通，以及查询两点间路径上点权的异或和。

核心思路：把每棵树剖成若干条实链，每条实链用一棵按深度为序的 Splay 维护，Splay 根的 `fa` 指向链顶在原树中的父亲（虚边），`is_root` 据此判断。`access(x)` 自下而上把根到 x 的路径变成一条实链；`make_root` 再整体打翻转标记使 x 成为原树根。路径查询先把 x 设为根再 `access(y)`，y 所在 Splay 的 `sum` 即答案。`splay` 前用显式栈自上而下下传翻转标记。

思路中的情况	代码中的处理	行
把根到 x 的路径变成实链	逐段 `splay`，把上一段接为右孩子，原右孩子变为虚边	77
换根	`access` 后翻转整条链，深度顺序颠倒	92
找所在树的根	`access` 后一路走左孩子找到深度最小点并伸展	98
连边	x 设为根，若 y 所在树根不是 x 就把 x 的父亲设为 y	116
删边	x 为根、`access(y)` 后，路径恰为 x, y 两点才是真实的边	125
路径查询	x 为根、`access(y)` 后读 y 的 `sum`	144

接口契约：构造 LinkCutTree(n)，点号 1..n，初始权值为零。set(x,value) 修改点权；link(x,y) 仅在不同连通块之间连边并返回是否成功；cut(x,y) 只删除真实存在的边，否则返回 false。connected 查询连通性；query(x,y) 返回路径点权异或，要求两点连通，支持 x=y。make_root/find_root 维护与查询当前表示树根。每次操作均摊 O(log n)，O(n) 空间，splay 下传使用显式栈，支持长链。权值为 unsigned long long。仅维护路径异或，不包含虚子树聚合或动态直径。路径最值或路径和：把异或聚合换成对应运算（如 max），在 pull 中合并左右子树与自身；路径加等标记须在 splay 前沿路径下传，与翻转标记一起在 push 中下传。

```
struct LinkCutTree
{
    using U = unsigned long long;

    struct Node
    {
        int ch[2]{}, fa = 0;
        U val = 0, sum = 0;
        bool rev = false;
    };

    vector<Node> a;
    vector<int> st;

    LinkCutTree(int n) : a(n + 1), st(n + 1) {}

    bool is_root(int x) const
    {
        int f = a[x].fa;
        return a[f].ch[0] != x && a[f].ch[1] != x;
    }

    void pull(int x) { a[x].sum = a[a[x].ch[0]].sum ^ a[x].val ^ a[a[x].ch[1]].sum; }

    void reverse(int x)
    {
        if (!x) return;
        swap(a[x].ch[0], a[x].ch[1]);
        a[x].rev = !a[x].rev;
    }

    void push(int x)
    {
        if (!a[x].rev) return;
        reverse(a[x].ch[0]);
        reverse(a[x].ch[1]);
        a[x].rev = false;
    }
```

```
40      void rotate(int x)
41      {
42          int y = a[x].fa, z = a[y].fa;
43          int k = a[y].ch[1] == x;
44          int w = a[x].ch[k ^ 1];
45          if (!is_root(y)) a[z].ch[a[z].ch[1] == y] = x;
46          a[x].fa = z;
47          a[y].ch[k] = w;
48          if (w) a[w].fa = y;
49          a[x].ch[k ^ 1] = y;
50          a[y].fa = x;
51          pull(y);
52          pull(x);
53      }
54
55      void splay(int x)
56      {
57          int top = 0, y = x;
58          st[top++] = y;
59          while (!is_root(y))
60          {
61              y = a[y].fa;
62              st[top++] = y;
63          }
64          while (top) push(st[--top]);
65          while (!is_root(x))
66          {
67              int y = a[x].fa, z = a[y].fa;
68              if (!is_root(y))
69              {
70                  bool same = (a[y].ch[1] == x) == (a[z].ch[1] == y);
71                  rotate(same ? y : x);
72              }
73              rotate(x);
74          }
75      }
```

```
77      void access(int x)
78      {
79          int last = 0;
80          for (int y = x; y; y = a[y].fa)
81          {
82              splay(y);
83              // The old right child's fa remains y: it becomes a path-parent.
84              a[y].ch[1] = last;
85              if (last) a[last].fa = y;
86              pull(y);
87              last = y;
88          }
89          splay(x);
90      }
91
92      void make_root(int x)
93      {
94          access(x);
95          reverse(x);
96      }
97
98      int find_root(int x)
99      {
100         access(x);
101         push(x);
102         while (a[x].ch[0])
103         {
104             x = a[x].ch[0];
105             push(x);
106         }
107         splay(x);
108         return x;
109     }
```

```
    bool connected(int x, int y) { return x == y || find_root(x) == find_root(y); }

    bool link(int x, int y)
    {
        make_root(x);
        if (find_root(y) == x) return false;
        a[x].fa = y;
        return true;
    }

    bool cut(int x, int y)
    {
        make_root(x);
        access(y);
        if (a[y].ch[0] != x || a[x].ch[1]) return false;
        a[y].ch[0] = 0;
        a[x].fa = 0;
        pull(y);
        return true;
    }

    void set(int x, U value)
    {
        access(x);
        a[x].val = value;
        pull(x);
    }

    // x and y must be connected. This is a path aggregate, not a subtree sum.
    U query(int x, int y)
    {
        make_root(x);
        access(y);
        return a[y].sum;
    }
};
```

模板题使用：Luogu P3690

题意：维护点权森林，支持路径异或、连边、删边和单点赋值。初始没有边；0 查询、1 连边、2 删边、3 赋值。题目保证查询两点连通，成环的连边和不存在的删边由接口拒绝。点号 1..n。

所需模板：LinkCutTree（第 375 页，37.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3690

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    LinkCutTree g(n);
    for (int i = 1; i <= n; i++)
    {
        unsigned long long x;
        cin >> x;
        g.set(i, x);
    }
    while (m--)
    {
        int op, x, y;
        cin >> op >> x >> y;
        if (op == 0)
            cout << g.query(x, y) << '\n';
        else if (op == 1)
            g.link(x, y);
        else if (op == 2)
            g.cut(x, y);
        else
            g.set(x, y);
    }
}
```

37.1.2 动态树路径乘积：虚子树与双向信息

$$F(u)=\sum_{v\in V}\prod_{x\in \mathrm{path}(u,v)} w_x,$$

含单点路径（孤立点答案为点权）。固定根时 $F(u)=w_u(1+\sum_{v\text{ 为子节点}}F(v))$。LCT 辅助树按实链维护链积 P、自链首的贡献 L、自链尾的贡献 R，每点另记虚儿子贡献和 G：

$$P=P_l w P_r,\qquad L=L_l+P_l w(1+G+L_r),\qquad R=R_r+P_r w(1+G+R_l).$$

空树 $P=1, L=R=0$。翻转须同时交换左右儿子与 L,R。access 时旧右链的 L 加入 G，新右链的 L 从 G 减去。修改点权前先换为根；换根后根处的 L 即答案。只用加减乘，零权、负权与合数模数均适用；固定树，不含连边删边。修改和查询摊还 $O(\log n)$，空间 $O(n)$；递归最坏深度 $O(n)$。

第六部分

图论

第三十八章　图论相关概念

38.1　函数图的入环链与到达查询

代码见第 471 页。接口 0-based：题目 1-based 时构造减 1，只有跳转返回的顶点加 1；最少步数的 0 与不可达的 -1 原样输出。

每个弱连通分量是一个环加若干指向环的树。入度剥离后剩下环点；反向处理剥离序列即可得到环编号、入环点和入环距离。跳转 k 步：k 小于入环距离时倍增，否则先到入环点，余步对环长取模；k 可达无符号 64 位上界，倍增表 $O(\log n)$ 层。u 到 v 最少步数：环编号不同则不可达；v 在树上时要求 $\mathrm{depth}(u) \geq \mathrm{depth}(v)$ 且跳深度差恰到 v；v 在环上时为 u 的入环步数加环上顺向距离。不可达返回 -1，自身为 0。position 只对环点有意义，树点为 -1。

38.2 杭州 2023 H：随机事件顺序与函数图

QOJ8237。事件 u 的目标 $v = b_u$：$a_u < a_v$ 必成功，$a_u \geq a_v + w_v$ 必失败，否则 u 成功当且仅当 v 更早执行且成功。确定事件的后继改为自身。到达确定失败终点或纯依赖环（含自环）则概率为零；到达确定成功终点、入环距离 d 时概率为 $1/(d+1)!$，期望 $a_u + w_u/(d+1)!$，模 10^9+7。依赖不可当作独立的 $1/2$。$\sum n \leq 500000$，阶乘表跨组只扩不缩；用通用函数图时 $O(n \log n)$ 时间与空间。

第三十九章　树上问题

39.1　树的直径

相关应用：固定拓扑下修改非负边权，递归欧拉序与区间加维护动态直径。

39.1.1　边权修改动态树直径

问题描述：给定一棵非负边权树，支持修改某条边的权值，并在每次修改后查询树的直径（最长路径的长度）。

核心思路：DFS 进入点时以及每个孩子返回后各记一次当前点的根距离，得到长 $2n-1$ 的序列 d。对 $i \le j \le k$，$d_i - 2d_j + d_k$ 的最大值就是直径：j 取在两点的 LCA 处时恰为路径长。线段树节点维护 `high`$=\max d$、`low`$=\max(-2d)$、`left`$=\max(d_i-2d_j)$、`right`$=\max(-2d_j+d_k)$ 和三项最大值 `best`，`merge` 枚举跨左右儿子的拼法。修改边权相当于把下端点 `child[id]` 的子树在序列中的区间 `first..last` 整体加 Δ，懒标记下这五个量分别变化 $+\Delta, -2\Delta, -\Delta, -\Delta, 0$。

接口契约：DynamicDiameter(n)，n≥1，点号 1..n。add(u,v,w) 返回从 0 开始的原边编号；输入须恰为一棵非负权无向树。build(root=1) 预处理；update(id,w) 设置原边权值；diameter() 返回最长路径长度，单点树为 0。仅返回长度，不支持 link/cut 和负权。可换根重建、复制；所有内部成员仅供只读。全部边权之和始终 ≤LLONG_MAX/4，保证中间值安全。

递归 DFS 记录进入顶点及每次孩子返回后的根距离，得到 2n-1 项欧拉序。修改父边等价于对子树首次至最后出现的闭区间加 Δ。维护 max(d[i]-2d[j]+d[k])，i≤j≤k；非负权下区间最小根距离等于 LCA 根距离，故最大值即直径。线段树五项 high、low、left、right、best 分别表示 d、-2d、前两项、后两项、完整三项最大值。统一加 Δ 后依次加 Δ、-2Δ、-Δ、-Δ、0。建树 O(n)，修改 O(log n)，查询 O(1)，空间 O(n)。DFS 保持递归，深链须给足原生栈。LOJ3163 已满分通过，排名未核验。

字段、加边与五项合并：

```
struct DynamicDiameter
{
    using ll = long long;
    struct Node
    {
        ll high, low, left, right, best, lazy;
    };
    struct Edge
    {
        int u, v;
        ll w;
    };
    int n, size = 0;
    ll total = 0;
    vector<Edge> edges;
    vector<vector<pair<int, int>>> g;
    vector<int> first, last, child;
    vector<Node> t;

    explicit DynamicDiameter(int n) : n(n), g(n + 1), first(n + 1), last(n + 1)
    {
        assert(n >= 1);
    }
    int add(int u, int v, ll w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(w >= 0 && w <= LLONG_MAX / 4 - total);
        int id = edges.size();
        edges.push_back({u, v, w});
        g[u].push_back({v, id});
        g[v].push_back({u, id});
        total += w;
        size = 0;
        return id;
    }
    static Node merge(const Node &a, const Node &b)
    {
        Node c;
        c.high = max(a.high, b.high);
        c.low = max(a.low, b.low);
        c.left = max({a.left, b.left, a.high + b.low});
        c.right = max({a.right, b.right, a.low + b.high});
        c.best = max({a.best, b.best, a.left + b.high, a.high + b.right});
        c.lazy = 0;
        return c;
    }
```

递归欧拉遍历与建树（接上页结构体）：

```
    void build(int root = 1)
    {
        assert(1 <= root && root <= n && (int)edges.size() == n - 1);
        vector<ll> d;
        child.resize(edges.size());
        auto dfs = [&](auto &&self, int u, int parent, ll depth) -> void
        {
            first[u] = d.size();
            d.push_back(depth);
            for (auto [v, id] : g[u])
            {
                if (v == parent) continue;
                child[id] = v;
                self(self, v, u, depth + edges[id].w);
                d.push_back(depth);
            }
            last[u] = d.size() - 1;
        };
        dfs(dfs, root, 0, 0);
        size = d.size();
        t.resize(4 * size);
        auto init = [&](auto &&self, int x, int l, int r) -> void
        {
            if (l == r)
            {
                t[x] = {d[l], -2 * d[l], -d[l], -d[l], 0, 0};
                return;
            }
            int mid = (l + r) / 2;
            self(self, x * 2, l, mid);
            self(self, x * 2 + 1, mid + 1, r);
            t[x] = merge(t[x * 2], t[x * 2 + 1]);
        };
        init(init, 1, 0, size - 1);
    }
```

区间加与原边修改（接上页结构体）：

```
    void apply(int x, ll delta)
    {
        t[x].high += delta;
        t[x].low -= 2 * delta;
        t[x].left -= delta;
        t[x].right -= delta;
        t[x].lazy += delta;
    }
    void change(int x, int l, int r, int ql, int qr, ll delta)
    {
        if (ql <= l && r <= qr)
        {
            apply(x, delta);
            return;
        }
        apply(x * 2, t[x].lazy);
        apply(x * 2 + 1, t[x].lazy);
        t[x].lazy = 0;
        int mid = (l + r) / 2;
        if (ql <= mid) change(x * 2, l, mid, ql, qr, delta);
        if (qr > mid) change(x * 2 + 1, mid + 1, r, ql, qr, delta);
        t[x] = merge(t[x * 2], t[x * 2 + 1]);
    }
    void update(int id, ll w)
    {
        assert(size && 0 <= id && id < (int)edges.size());
        assert(w >= 0 && w <= LLONG_MAX / 4 - (total - edges[id].w));
        ll delta = w - edges[id].w;
        total += delta;
        edges[id].w = w;
        int u = child[id];
        change(1, 0, size - 1, first[u], last[u], delta);
    }
    ll diameter() const
    {
        assert(size);
        return t[1].best;
    }
};
```

模板题使用：CEOI2019 Dynamic Diameter（LOJ3163 待复核提交）

题意：每次修改一条边后输出直径；用上次答案解码边号和新权，边编号 0-based。

所需模板：DynamicDiameter（第 384 页，39.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://loj.ac/p/3163

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    long long w;
    cin >> n >> q >> w;
    DynamicDiameter tree(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        long long c;
        cin >> u >> v >> c;
        tree.add(u, v, c);
    }
    tree.build();
    long long last = 0;
    while (q--)
    {
        long long d, e;
        cin >> d >> e;
        int id = (d + last) % (n - 1);
        tree.update(id, (e + last) % w);
        last = tree.diameter();
        cout << last << '\n';
    }
}
```

39.1.2 树上点集直径摘要

功能：维护一棵固定非负权树上某个点集的“直径摘要”：支持插入一个点、合并两个点集，并查询点集内距离最远的一对点（直径端点 a、b 与长度），以及点集中离给定点 v 最远的点。

核心思路：关键性质：非负权树上，点集 S 的直径端点为 a,b 时，$S\cup\{v\}$ 的直径端点可在 $\{a,b,v\}$ 中取到，任意点到 S 中最远点也可取 a 或 b。因此插入 v 只需算 $\mathrm{dist}(a,v)$、$\mathrm{dist}(b,v)$，若更长就用 v 替换对应端点；合并两个集合等于把对方的两个端点依次插入；最远点查询比较到 a、b 的距离。距离由调用方的回调给出。

接口契约：默认空点集，a=b=-1、length=0。insert(v,distance) 插入顶点，重复插入合法；merge(other,distance) 合并两个点集摘要，允许自合并；farthest(v,distance) 返回 optional<pair<int,long long»（最远顶点、距离），空集返回 nullopt，距离并列时可返回任意最远点。distance(u,v) 回调必须精确返回同一棵固定非负权树的距离，点号为非负整数；所有距离 ≤LLONG_MAX。每次插入或最远查询最多两次距离调用，合并最多四次，摘要只占 O(1) 空间。保存 a、b 为点集中某对直径端点，不保留全部点；字段只读，不能用一般图度量或负权树代替。复制摘要即可建立前缀、后缀或线段树聚合，重置用默认构造。

```cpp
struct TreeDiameter
{
    using ll = long long;
    int a = -1, b = -1;
    ll length = 0;

    // distance is the metric of one fixed tree with nonnegative edge weights.
    template <class Distance> void insert(int v, const Distance &distance)
    {
        assert(v >= 0);
        if (a == -1)
        {
            a = v;
            b = v;
            return;
        }
        ll x = distance(a, v), y = distance(b, v);
        if (x >= y && x > length)
        {
            b = v;
            length = x;
        }
        else if (y > length)
        {
            a = v;
            length = y;
        }
    }
```

合并与最远点查询（接上页同一结构体）：

```
    template <class Distance> void merge(TreeDiameter other, const Distance &distance)
    {
        if (other.a == -1) return;
        insert(other.a, distance);
        insert(other.b, distance);
    }

    template <class Distance>
    optional<pair<int, ll>> farthest(int v, const Distance &distance) const
    {
        if (a == -1) return nullopt;
        ll x = distance(a, v), y = distance(b, v);
        return x >= y ? make_pair(a, x) : make_pair(b, y);
    }
};
```

39.1.3 非负权树的点集直径摘要

代码见第 388 页。非空点集只需保存一对直径端点 a, b，任意点 x 到点集的最远距离为 $\max(d(x,a), d(x,b))$。要求边权非负（可为零），不能推广到一般图或负权树。插入新点只比较到两个旧端点的距离；合并点集只插入另一摘要的两个端点。并列直径任选，端点不固定。带权距离可用 $(D_u - D_z) + (D_v - D_z)$（$z$ 为 LCA）；所有路径权和须在 long long 内。

离线添叶仍是固定树度量。 CF379F：先建出最终树再预处理距离，按出现顺序向摘要插点（初始四点也插入）。$q \le 500000$ 时 $4 + 2q$ 个点，HLD $O(q)$ 空间、$O(q \log q)$ 时间。

QOJ8235（点权互异）：令 g 为全树点权的 mex，按点权 $0, \ldots, g-1$ 建前缀摘要；某前缀全在以 x 为中心、半径 k 的范围内当且仅当其最远距离 $\le k$，二分最长前缀；$g = 0$ 时为零。边带权，不能用边数距离。

39.2 最近公共祖先

39.2.1 两条树路径的交集

问题描述：给定一棵树和两条路径 a–b、c–d，求它们的公共部分：公共部分若非空必是一条路径，返回其两个端点和公共顶点数。

核心思路：把点 x 投影到路径 a–b 上（路径上离 x 最近的点），投影就是 $\mathrm{lca}(a,b)$、$\mathrm{lca}(a,x)$、$\mathrm{lca}(b,x)$ 中深度最大者。设 c,d 的投影为 p,q：若 $p\neq q$，则 c–d 必经过 p–q，这就是交集；若 $p=q$，交集至多是单点 p，用距离判断 p 是否在 c–d 上（$\mathrm{dist}(c,p)+\mathrm{dist}(p,d)=\mathrm{dist}(c,d)$），否则交集为空。距离由深度和 LCA 算出。

接口契约：path_intersection(a,b,c,d,depth,lca) 返回路径 a-b 与 c-d 的公共路径，字段 u、v 是交集端点，vertices 是公共顶点数；空交返回 {0,0,0}，单点交的 u=v。点号从 1 开始，depth 数组与 lca 回调必须来自同一棵非空树、同一个根，深度按边数计算，不能传带权根距离。支持数组或 vector 深度表，回调接收两个点并返回 LCA。将 c、d 分别投影到 a-b 得 p、q；投影是 lca(a,b)、lca(a,x)、lca(b,x) 中最深者。若 p≠q，c-d 必经过 p-q；若 p=q，检验 p 是否在 c-d 上即可区分单点交与空交。不依赖顶点编号大小。常数次 LCA 调用、O(1) 额外空间；使用 EulerLCA 时为 O(1)，HLD/倍增时为 O(log n)。公共边数为 max(0,vertices-1)。全小树根与四端点、随机换号树用路径点集独立验证；P3398 完整程序只检查是否相交，端点及数量由本地验证覆盖。

```
struct PathIntersection
{
    int u, v, vertices;
};

// Same nonempty tree and root for depth[] and lca(). Empty result is {0,0,0}.
template <class Depth, class Lca>
PathIntersection
path_intersection(int a, int b, int c, int d, const Depth &depth, Lca lca)
{
    int z = lca(a, b);
    auto project = [&](int x)
    {
        int p = z;
        for (int v : {lca(a, x), lca(b, x)})
            if (depth[v] > depth[p]) p = v;
        return p;
    };
    auto distance = [&](int u, int v)
    {
        return 1LL * depth[u] + depth[v] - 2LL * depth[lca(u, v)];
    };
    int p = project(c), q = project(d);
    if (p == q && distance(c, p) + distance(p, d) != distance(c, d)) return {0, 0, 0};
    return {p, q, int(distance(p, q) + 1)};
}
```

39.2.2 倍增 LCA、祖先与最大边权

功能：对一棵带边权的树做倍增预处理，支持：求 u 的 k 级祖先、两点的 LCA 与路径边数，以及两点之间路径上的最大边权。

核心思路：`up[k][u]` 是 u 向上 2^k 步的祖先，`mx[k][u]` 是这 2^k 条边的最大权，DFS 进入 u 时由 `up[k-1]` 两段拼出。跳 k 级祖先按 k 的二进制位逐位跳。求 LCA 时先把较深点跳到同一深度，若重合即为答案；否则从高位到低位，只要两点的 2^k 级祖先不同就同时上跳，最后父亲即 LCA。`max_edge` 沿完全相同的跳跃过程，把经过的各段 `mx` 取最大。0 号点作哨兵，表示“没有祖先”。

接口契约：构造 LiftingLCA(n)，add(u,v,w=1) 加边，build(root=1) 预处理非空无向树，点号 1..n。jump(u,k) 返回 k 级祖先，k 非负，超过根返回 0；lca(u,v) 求最近公共祖先；distance 返回路径边数，不是权值和；max_edge 返回路径最大边权，同点返回 nullopt，其他情况返回有值的 optional<long long>。支持负权和 LLONG_MIN，LOW 只是最大值合并的单位元，不表示空路径。DFS 递归，根深度为 0；up[k][u] 与 mx[k][u] 是 2 的 k 次方长度向上段的祖先与最大边权，0 号节点为哨兵。可重复 build 换根；修改图后必须重建。预处理 O(n log n) 时间与空间，各查询 O(log n)。不需要边权最值时，可删去 mx 和 max_edge 相关部分。全小树点对/根、极值权重及 50 万点负权链本地验证；P3379、POJ1330 驱动只验证 LCA，最大边权仍是本地证据。

```
struct LiftingLCA
{
    using ll = long long;
    static constexpr ll LOW = numeric_limits<ll>::lowest();
    int n, levels;
    bool ready = false;
    vector<vector<pair<int, ll>>> g;
    vector<vector<int>> up;
    vector<vector<ll>> mx;
    vector<int> depth;

    LiftingLCA(int n) : n(n), g(n + 1), depth(n + 1)
    {
        assert(n > 0);
        levels = 1;
        while ((1LL << levels) <= n) levels++;
        up.assign(levels, vector<int>(n + 1));
        mx.assign(levels, vector<ll>(n + 1, LOW));
    }

    void add(int u, int v, ll w = 1)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        g[u].push_back({v, w});
        g[v].push_back({u, w});
        ready = false;
    }
```

```
    void dfs(int u, int parent, ll weight)
    {
        up[0][u] = parent;
        mx[0][u] = weight;
        for (int k = 1; k < levels; k++)
        {
            int p = up[k - 1][u];
            up[k][u] = up[k - 1][p];
            mx[k][u] = max(mx[k - 1][u], mx[k - 1][p]);
        }
        for (auto [v, w] : g[u])
            if (v != parent)
            {
                depth[v] = depth[u] + 1;
                dfs(v, u, w);
            }
    }

    // Nonempty tree only. Node 0 means no ancestor.
    void build(int root = 1)
    {
        assert(1 <= root && root <= n);
        depth[root] = 0;
        dfs(root, 0, LOW);
        ready = true;
    }

    int jump(int u, long long steps) const
    {
        assert(ready && 1 <= u && u <= n && steps >= 0);
        if (steps > depth[u]) return 0;
        for (int k = 0; k < levels; k++)
            if (steps >> k & 1) u = up[k][u];
        return u;
    }
```

```
65      int lca(int u, int v) const
66      {
67          assert(ready && 1 <= u && u <= n && 1 <= v && v <= n);
68          if (depth[u] < depth[v]) swap(u, v);
69          u = jump(u, depth[u] - depth[v]);
70          if (u == v) return u;
71          for (int k = levels - 1; k >= 0; k--)
72              if (up[k][u] != up[k][v])
73              {
74                  u = up[k][u];
75                  v = up[k][v];
76              }
77          return up[0][u];
78      }
79
80      int distance(int u, int v) const
81      {
82          return depth[u] + depth[v] - 2 * depth[lca(u, v)];
83      }
84
85      optional<ll> max_edge(int u, int v) const
86      {
87          assert(ready && 1 <= u && u <= n && 1 <= v && v <= n);
88          if (u == v) return nullopt;
89          if (depth[u] < depth[v]) swap(u, v);
90          ll answer = LOW;
91          int delta = depth[u] - depth[v];
92          for (int k = 0; k < levels; k++)
93              if (delta >> k & 1)
94              {
95                  answer = max(answer, mx[k][u]);
96                  u = up[k][u];
97              }
98          if (u == v) return answer;
99          for (int k = levels - 1; k >= 0; k--)
100             if (up[k][u] != up[k][v])
101             {
102                 answer = max(answer, max(mx[k][u], mx[k][v]));
103                 u = up[k][u];
104                 v = up[k][v];
105             }
106         return max(answer, max(mx[0][u], mx[0][v]));
107     }
108 };
```

模板题使用：Luogu P3379

题意：给定以 root 为根的树，回答两点的最近公共祖先。加完所有边后 build(root)，再逐条调用 lca；点号为 1..n。此题不涉及跳祖先或路径最大边权。

所需模板：LiftingLCA（第 392 页，39.2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3379

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q, root;
    cin >> n >> q >> root;
    LiftingLCA graph(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        graph.add(u, v);
    }
    graph.build(root);
    while (q--)
    {
        int u, v;
        cin >> u >> v;
        cout << graph.lca(u, v) << '\n';
    }
    return 0;
}
```

39.2.3 树路径交：投影与单点交

接口见第 391 页。x 在路径 a–b 上的投影是 $\mathrm{lca}(a,b)$、$\mathrm{lca}(a,x)$、$\mathrm{lca}(b,x)$ 中深度最大者（同一根）。设 c,d 的投影为 p,q：$p\neq q$ 时交集为 p–q；$p=q$ 时若 $d(c,p)+d(p,d)=d(c,d)$ 则为单点交，否则为空。距离与 depth 按边数，不能换成带权根距离。返回交集端点和顶点数，空交为 $\{0,0,0\}$，公共边数 $\max(0,\text{vertices}-1)$。每次调用常数次 LCA；用 HLD 回调为 $O(\log n)$。

39.3 树链剖分

39.3.1 长链剖分与常数时间 K 级祖先

功能：对有根树做长链剖分，支持在线查询任意点 u 的 k 级祖先。

核心思路：`dfs1` 求深度、倍增表 `fa` 和子树高度 `len`，高度最大的儿子作为长儿子 `son`；`dfs2` 把树剖成长链，对每个长度为 L 的链顶 t 存下从 t 向下的整条链 `down[t]` 和从 t 向上的 L 个祖先 `up[t]`，总量 $O(n)$。查询时先用倍增跳 k 的最高二进制位 2^j 到 u'；u' 下方至少还有 2^j 层，所以它所在长链在链顶 t 之下至少延伸 $d+2^j$（d 为 u' 到 t 的深度差），而剩余步数小于 2^j。剩余 $k\le d$ 时在 `down[t]` 中取，否则在 `up[t]` 中取第 $k-d$ 个。

接口契约：LongChain(n)，1≤n<INT_MAX，点号 1..n；add(u,v) 加入无向树边，调用方保证全部边恰好构成一棵树，不支持环、重边、自环或森林。build(root=1) 以指定根重建，dep[root]=1；kth(u,k) 返回 k 级祖先，k≥0，k=0 返回自身，越过根返回 0。查询前必须 build，加边后必须重建；支持重复 build、换根与值复制。son 按子树最大高度选长儿子，不能换成子树大小；len 含自身。链顶 t 的 down[t] 从 t 向下，up[t][0]=t 且向上越根补 0；成

员只读，dfs1/dfs2 仅内部使用。预处理 O(n log(n+1))，空间 O(n log(n+1))，两次 DFS 均递归、栈深 O(n)。查询先跳最高 2 的幂，再在链顶上下表中查一次，32 位字长模型下 O(1)；k=0 必须先处理，不能求 0 的最高位。

```
struct LongChain
{
    int n;
    vector<int> dep, len, son, top;
    vector<vector<int>> g, fa, up, down;

    LongChain(int n) : n(n), g(n + 1)
    {
        assert(1 <= n && n < INT_MAX);
    }

    void add(int u, int v)
    {
        g[u].push_back(v);
        g[v].push_back(u);
    }

    void dfs1(int u, int p)
    {
        fa[0][u] = p;
        dep[u] = dep[p] + 1;
        for (int j = 1; j < (int)fa.size(); j++)
            fa[j][u] = fa[j - 1][fa[j - 1][u]];
        for (int v : g[u])
        {
            if (v == p) continue;
            dfs1(v, u);
            if (len[v] + 1 > len[u])
            {
                len[u] = len[v] + 1;
                son[u] = v;
            }
        }
    }
```

递归划分长链并填表（接上页同一结构体）：

```
    void dfs2(int u, int t)
    {
        top[u] = t;
        if (u == t)
        {
            down[t].reserve(len[u]);
            up[t].resize(len[u]);
            int x = u;
            for (int &v : up[t])
            {
                v = x;
                x = fa[0][x];
            }
        }
        down[t].push_back(u);
        if (son[u]) dfs2(son[u], t);
        for (int v : g[u])
        {
            if (v != fa[0][u] && v != son[u])
                dfs2(v, v);
        }
    }
```

重建与常数时间查询（接上页同一结构体）：

```
    void build(int root = 1)
    {
        assert(1 <= root && root <= n);
        dep.assign(n + 1, 0);
        len.assign(n + 1, 1);
        son.assign(n + 1, 0);
        top.assign(n + 1, 0);
        fa.assign(bit_width((unsigned)n), vector<int>(n + 1));
        up.assign(n + 1, {});
        down.assign(n + 1, {});
        dfs1(root, 0);
        dfs2(root, root);
    }

    int kth(int u, int k) const
    {
        assert(1 <= u && u <= n && k >= 0 && !fa.empty());
        if (k >= dep[u]) return 0;
        if (k == 0) return u;
        int j = bit_width((unsigned)k) - 1;
        u = fa[j][u];
        k -= 1 << j;
        int t = top[u];
        int d = dep[u] - dep[t];
        if (k <= d) return down[t][d - k];
        return up[t][k - d];
    }
};
```

为什么一次跳跃就够 先向上跳 p（不超过 k 的最大 2 的幂），到达 u，剩余 r 小于 p。原查询点是 u 下方距离 p 的后代，所以 u 的子树高度至少 p+1。u 所在长链顶 t 的链长至少 dep[u]-dep[t]+p+1，足以覆盖 r；若 r 不超过 u 到 t 的距离就查 down，否则查 up。选择长儿子必须按高度，不能按子树大小。

边界与空间 up[t][0] 和 down[t][0] 都等于 t；up 越过根填 0。先处理 k=0 和越根，避免对 0 求最高位。各长链长度之和为 n，上下表各存 n 个 int；倍增表为 O(n log n)。预处理两次 DFS 都保持递归，栈深可达 n。

模板题使用：P5903 【模板】树上 K 级祖先

题意：按题意用 32 位无符号 xorshift 顺序生成询问；根深度为 1，先生成 x 再生成 k，按 64 位累计 i* 答案的异或；n≤500000，q≤5000000。

所需模板：LongChain（第 395 页，39.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5903

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    uint32_t s;
    cin >> n >> q >> s;
    LongChain tree(n);
    int root = 0;
    for (int u = 1; u <= n; u++)
    {
        int p;
        cin >> p;
        if (p) tree.add(u, p);
        else root = u;
    }
    tree.build(root);
    auto get = [&]()
    {
        s ^= s << 13;
        s ^= s >> 17;
        s ^= s << 5;
        return s;
    };
    int last = 0;
    unsigned long long ans = 0;
    for (int i = 1; i <= q; i++)
    {
        int x = (get() ^ last) % n + 1;
        int k = (get() ^ last) % tree.dep[x];
        last = tree.kth(x, k);
        ans ^= 1ULL * i * last;
    }
    cout << ans << '\n';
}
```

39.3.2 重链剖分与 LCA

功能：对一棵树做重链剖分：给出 DFS 序 `dfn`，使每棵子树、每条重链都是连续区间；支持求 LCA，以及把 u–v 路径拆成 $O(\log n)$ 个 `dfn` 区间回调（可只取边对应的位置，也可按 u 到 v 的真实顺序给出），便于套线段树做路径修改与查询。

核心思路：`dfs1` 求父亲、深度、子树大小和重儿子 `son`（子树最大的儿子）；`dfs2` 先递归重儿子，使同一重链的 `dfn` 连续，`top` 为链顶。查询时每次让链顶更深的一端整段跳过本链（上报区间 $[\mathrm{dfn}[\mathrm{top}[u]], \mathrm{dfn}[u]]$）并跳到链顶的父亲，直到两点同链，再上报二者之间的一段；边模式下边权存在深的一端，最后一段跳过较浅点即 LCA。`path_ordered` 中 u 侧的段是自下而上走的，立即以 `rev=true` 回调；v 侧的段先存入 `down`，最后逆序以正向回调。

接口契约： add(u,v) 建无向树；build(root) 后读取 fa、dep、siz、son、top、dfn、rk。子树对应 [dfn[u],dfn[u]+siz[u]-1]。lca(u,v) 为 O(log n)，path(u,v,work,edge) 分解路径；work(l,r) 仅适用于可交换聚合，边权模式排除 LCA 位置。输入必须是连通无环树，建树 O(n)，两遍递归 DFS；支持改变根后重复 build，dfn 会重新编号。path_ordered(u, v, work, edge) 的 edge 默认为 false；按真实 u 到 v 顺序回调 work(l, r, rev)：1-based 闭区间 l≤r，rev=true 按 r 到 l 访问，否则按 l 到 r。支持非交换聚合，O(log n) 段与额外空间；结合线段树通常每次查询 $O(\log^2 n)$。edge=true 仅删除 LCA 对应位置，仍须自行处理边算子的方向；不自动求逆。u=v 时点模式一次单点回调，边模式无回调；改变根后须重建依赖 dfn 的结构。

```
struct HLD
{
    int n, timer = 0;
    vector<vector<int>> g;
    vector<int> fa, dep, siz, son, top, dfn, rk;

    HLD(int n)
        : n(n),
          g(n + 1),
          fa(n + 1),
          dep(n + 1),
          siz(n + 1),
          son(n + 1),
          top(n + 1),
          dfn(n + 1),
          rk(n + 1)
    {
    }

    void add(int u, int v)
    {
        g[u].push_back(v);
        g[v].push_back(u);
    }

    void dfs1(int u, int p)
    {
        fa[u] = p;
        siz[u] = 1;
        son[u] = 0;
        for (int v : g[u])
        {
            if (v == p) continue;
            dep[v] = dep[u] + 1;
            dfs1(v, u);
            siz[u] += siz[v];
            if (!son[u] || siz[v] > siz[son[u]]) son[u] = v;
        }
    }
```

重编号、LCA 与可交换路径（接上页同一结构体）：

```
    void dfs2(int u, int t)
    {
        top[u] = t;
        dfn[u] = ++timer;
        rk[timer] = u;
        if (son[u]) dfs2(son[u], t);
        for (int v : g[u])
        {
            if (v != fa[u] && v != son[u]) dfs2(v, v);
        }
    }

    // Connected tree only. Recursive DFS; subtree interval is [dfn, dfn+siz-1].
    void build(int root = 1)
    {
        assert(1 <= root && root <= n);
        timer = 0;
        dep[root] = 0;
        dfs1(root, root);
        dfs2(root, root);
        assert(timer == n);
    }

    int lca(int u, int v) const
    {
        while (top[u] != top[v])
        {
            if (dep[top[u]] < dep[top[v]]) swap(u, v);
            u = fa[top[u]];
        }
        return dep[u] < dep[v] ? u : v;
    }

    // Commutative operations only; edge=true excludes the LCA's position.
    template <class F> void path(int u, int v, F work, bool edge = false) const
    {
        while (top[u] != top[v])
        {
            if (dep[top[u]] < dep[top[v]]) swap(u, v);
            work(dfn[top[u]], dfn[u]);
            u = fa[top[u]];
        }
        if (dep[u] > dep[v]) swap(u, v);
        if (dfn[u] + edge <= dfn[v]) work(dfn[u] + edge, dfn[v]);
    }
```

按真实方向分解路径（接上页同一结构体）：

```
    // Emit u-to-v order; reverse=true traverses the closed interval from r to l.
    template <class F> void path_ordered(int u, int v, F work, bool edge = false) const
    {
        vector<pair<int, int>> down;
        while (top[u] != top[v])
        {
            if (dep[top[u]] >= dep[top[v]])
            {
                work(dfn[top[u]], dfn[u], true);
                u = fa[top[u]];
            }
            else
            {
                down.emplace_back(dfn[top[v]], dfn[v]);
                v = fa[top[v]];
            }
        }
        if (dep[u] >= dep[v])
        {
            if (dfn[v] + edge <= dfn[u]) work(dfn[v] + edge, dfn[u], true);
        }
        else if (dfn[u] + edge <= dfn[v]) work(dfn[u] + edge, dfn[v], false);
        for (auto it = down.rbegin(); it != down.rend(); ++it)
            work(it->first, it->second, false);
    }
};
```

模板题使用：Library Checker Vertex Set Path Composite

题意：静态无向树，1≤N,Q≤200000。点函数 f(x)=a*x+b，对 998244353 取模；题面 1≤a,c<mod，0≤b,d,x<mod。0 p c d 改单点函数，1 u v x 按 u 到 v（含两端点）依次代入。题目点号 0-based，传 HLD 时加 1；HLD 闭区间 [l,r] 对应 segtree 半开 [l−1,r)。S 存正向与反向复合，不能把组合顺序反过来或用逆函数代替反向序列。点改 O(log N)，路径 O($\log^2$ N)，空间 O(N)；保留递归 DFS，最大链须足够大栈。

所需模板：HLD（第 399 页，39.3.2 节）、segtree（第 295 页，32.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/vertex_set_path_composite

```
using S = array<long long, 4>;
const int mod = 998244353;

S op(S a, S b)
{
    return {b[0] * a[0] % mod,
            (b[0] * a[1] + b[1]) % mod,
            a[2] * b[2] % mod,
            (a[2] * b[3] + a[3]) % mod};
}

S e()
{
    return {1, 0, 1, 0};
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<S> a(n), val(n);
    for (auto &x : a)
    {
        cin >> x[0] >> x[1];
        x[2] = x[0];
        x[3] = x[1];
    }
    HLD h(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        h.add(u + 1, v + 1);
    }
    h.build();
    for (int u = 1; u <= n; u++) val[h.dfn[u] - 1] = a[u - 1];
    segtree<S, op, e> s(val);
```

继续处理操作（接上页同一 main）：

```
    while (q--)
    {
        int t;
        cin >> t;
        if (t == 0)
        {
            int p;
            long long c, d;
            cin >> p >> c >> d;
            s.set(h.dfn[p + 1] - 1, {c, d, c, d});
        }
        else
        {
            int u, v;
            long long x;
            cin >> u >> v >> x;
            h.path_ordered(u + 1, v + 1, [&](int l, int r, bool rev)
            {
                S z = s.prod(l - 1, r);
                int k = rev ? 2 : 0;
                x = (z[k] * x + z[k + 1]) % mod;
            });
            cout << x << '\n';
        }
    }
}
```

39.4 树上启发式合并

39.4.1 树上启发式合并：子树众数颜色和

问题描述：给定有根树，每个点有一种颜色。对每个点 u，求其子树中出现次数最多的颜色的出现次数，以及所有达到该次数的颜色编号之和。

核心思路：树上启发式合并（dsu on tree）。`prepare` 求子树大小、重儿子和 DFS 序，使子树对应 `order` 中一段区间。`solve(u,keep)` 先处理所有轻儿子并清空它们的统计，再处理重儿子并保留其统计；接着把轻儿子子树的点按区间逐个 `insert`，再插入 u，此时全局 `freq`、最大频次 `best` 及对应颜色和 `sum` 恰为 u 子树的答案。若 u 不需保留，就把整棵子树的频次减回去并将 `best`、`sum` 归零，而不是逐个删除时维护众数。每个点只在其到根路径上的轻边处被重新插入，次数为对数级。颜色先离散化。

接口契约：SubtreeColors(n)，点号 1..n，add(u,v) 建无向树；要求图连通且无环，n=0 可空运行。run(colors,root=1) 传入长度 n+1 的 int 颜色数组，colors[0] 忽略。answer[u] 为子树内所有最高频颜色的原始编号之和，max_freq[u] 为该频次；支持负颜色与稀疏编号，内部离散化，和用 long long。重复 run 可更换颜色或根。重儿子信息保留，轻儿子通过 DFS 序区间加入；清空整棵临时子树后再重置最大值，不尝试逐次删除时维护众数。O(n log n) 时间、O(n) 空间，全部 DFS 递归。insert 是内部增量统计，可按题目修改统计量，但相应清空规则也必须调整。

```
struct SubtreeColors
{
    int n, best = 0;
    long long sum = 0;
    vector<vector<int>> g;
    vector<int> siz, heavy, tin, tout, order, color, value, freq;
    vector<int> max_freq;
    vector<long long> answer;

    SubtreeColors(int n) : n(n), g(n + 1) {}

    void add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n && u != v);
        g[u].push_back(v);
        g[v].push_back(u);
    }

    void prepare(int u, int p)
    {
        siz[u] = 1;
        tin[u] = order.size();
        order.push_back(u);
        for (int v : g[u])
        {
            if (v == p) continue;
            prepare(v, u);
            siz[u] += siz[v];
            if (siz[v] > siz[heavy[u]]) heavy[u] = v;
        }
        tout[u] = (int)order.size() - 1;
    }

    void insert(int u)
    {
        int c = color[u];
        freq[c]++;
        if (freq[c] > best)
        {
            best = freq[c];
            sum = value[c];
        }
        else if (freq[c] == best)
            sum += value[c];
    }
```

启发式合并与运行入口（接上页同一结构体）：

```
    void solve(int u, int p, bool keep)
    {
        for (int v : g[u])
        {
            if (v != p && v != heavy[u]) solve(v, u, false);
        }
        if (heavy[u]) solve(heavy[u], u, true);
        for (int v : g[u])
        {
            if (v == p || v == heavy[u]) continue;
            for (int i = tin[v]; i <= tout[v]; i++) insert(order[i]);
        }
        insert(u);
        max_freq[u] = best;
        answer[u] = sum;
        if (!keep)
        {
            for (int i = tin[u]; i <= tout[u]; i++) freq[color[order[i]]]--;
            best = 0;
            sum = 0;
        }
    }

    // A connected tree, colors[1..n]. Sum all colors tied for maximum frequency.
    void run(const vector<int> &colors, int root = 1)
    {
        assert((int)colors.size() == n + 1);
        siz.assign(n + 1, 0);
        heavy = siz;
        tin = siz;
        tout = siz;
        color = siz;
        max_freq = siz;
        answer.assign(n + 1, 0);
        order.clear();
        value.assign(colors.begin() + 1, colors.end());
        sort(value.begin(), value.end());
        value.erase(unique(value.begin(), value.end()), value.end());
        freq.assign(value.size(), 0);
        best = 0;
        sum = 0;
        if (!n) return;
        assert(1 <= root && root <= n);
        for (int u = 1; u <= n; u++)
            color[u] =
                lower_bound(value.begin(), value.end(), colors[u]) - value.begin();
        prepare(root, 0);
        solve(root, 0, false);
    }
};
```

39.5 虚树

39.5.1 虚树：关键点压缩与路径信息

功能：给定一棵带非负边权的树，预处理后对每组关键点构建虚树：只保留关键点及它们两两的 LCA，原树中的一段路径压成一条虚边，并给出虚边对应的原路径长度和路径上最小边权，供在虚树上做 DP。

核心思路：预处理一遍 DFS 求入出时间 `tin`/`tout`（用于 $O(1)$ 判祖先）、根距 `dist`，以及倍增祖先 `up` 和跳 2^j 步的最小边权 `mn`。建树时把关键点按 `tin` 排序去重，补上 DFS 序中相邻两点的 LCA，再排序去重，这个点集对 LCA 封闭。然后按 DFS 序扫描，维护一条祖先链栈：弹出不是当前点祖先的栈顶，剩下的栈顶就是当前点在虚树中的父亲，连一条虚边。虚边的长度由 `dist` 相减得到，最小边权由 `path` 沿倍增上跳求出。

接口契约：VirtualTree(n)，n≥1，点号 1..n；add(u,v,w=1) 建非负权连通树，无环和重边。prepare(root=1) 递归预处理，可换根重建；根到任一点距离须在 long long 范围内。build(keys) 自动去重并补齐必要 LCA，返回 vertices 和 edges；全部编号沿用原树，vertices 按 DFS 序排列，非空时首项即虚树根。edges 为 (from,to,distance,minimum)，分别是祖先到后代的虚边、压缩路径长度和路径最小边权。空关键点返回空结果，单点返回单节点；原树根不会强制加入，需要时由调用者加入 keys。边按后代 DFS 序排列，可逆序做自底向上的 DP。不分配或清空 n 个查询邻接表；每次仅返回 O(k) 结果。预处理 O(n log n) 时间空间，单次 O(k log k+k log n)。lca、ancestor 也可直接使用，要求有效点号且已经 prepare。构造用的祖先链栈属于虚树算法，原树 DFS 保持递归。

```
struct VirtualTree
{
    using ll = long long;

    struct Edge
    {
        int from, to;
        ll distance, minimum;
    };

    struct Result
    {
        vector<int> vertices;
        vector<Edge> edges;
    };

    int n, log = 1, timer = 0;
    vector<vector<pair<int, ll>>> g;
    vector<vector<int>> up;
    vector<vector<ll>> mn;
    vector<int> depth, tin, tout;
    vector<ll> dist;

    VirtualTree(int n) : n(n), g(n + 1) {}

    void add(int u, int v, ll w = 1)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n && u != v && w >= 0);
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }
```

预处理与最近公共祖先（接上页同一结构体）：

```
    void dfs(int u, int p)
    {
        tin[u] = ++timer;
        up[0][u] = p;
        for (int j = 1; j < log; j++)
        {
            up[j][u] = up[j - 1][up[j - 1][u]];
            mn[j][u] = min(mn[j - 1][u], mn[j - 1][up[j - 1][u]]);
        }
        for (auto [v, w] : g[u])
        {
            if (v == p) continue;
            assert(dist[u] <= numeric_limits<ll>::max() - w);
            depth[v] = depth[u] + 1;
            dist[v] = dist[u] + w;
            mn[0][v] = w;
            dfs(v, u);
        }
        tout[u] = timer;
    }

    void prepare(int root = 1)
    {
        assert(1 <= root && root <= n);
        log = 1;
        while ((1LL << log) <= n) log++;
        up.assign(log, vector<int>(n + 1));
        mn.assign(log, vector<ll>(n + 1, numeric_limits<ll>::max()));
        depth.assign(n + 1, 0);
        tin = depth;
        tout = depth;
        dist.assign(n + 1, 0);
        timer = 0;
        dfs(root, root);
    }

    bool ancestor(int u, int v) const { return tin[u] <= tin[v] && tout[v] <= tout[u]; }

    int lca(int u, int v) const
    {
        if (ancestor(u, v)) return u;
        if (ancestor(v, u)) return v;
        for (int j = log - 1; j >= 0; j--)
        {
            if (!ancestor(up[j][u], v)) u = up[j][u];
        }
        return up[0][u];
    }
```

压缩路径与虚树构造（接上页同一结构体）：

```
    Edge path(int u, int v) const
    {
        assert(u != v && ancestor(u, v));
        ll minimum = numeric_limits<ll>::max();
        int x = v, steps = depth[v] - depth[u];
        for (int j = 0; j < log; j++)
        {
            if (steps >> j & 1)
            {
                minimum = min(minimum, mn[j][x]);
                x = up[j][x];
            }
        }
        return {u, v, dist[v] - dist[u], minimum};
    }

    // Uses original vertex IDs. Empty input gives an empty result.
    Result build(vector<int> keys) const
    {
        if (keys.empty()) return {};
        for (int u : keys) assert(1 <= u && u <= n);
        auto cmp = [&](int u, int v)
        {
            return tin[u] < tin[v];
        };
        sort(keys.begin(), keys.end(), cmp);
        keys.erase(unique(keys.begin(), keys.end()), keys.end());
        int k = keys.size();
        for (int i = 1; i < k; i++) keys.push_back(lca(keys[i - 1], keys[i]));
        sort(keys.begin(), keys.end(), cmp);
        keys.erase(unique(keys.begin(), keys.end()), keys.end());
        Result result{keys, {}};
        vector<int> st;
        for (int u : keys)
        {
            while (!st.empty() && !ancestor(st.back(), u)) st.pop_back();
            if (!st.empty()) result.edges.push_back(path(st.back(), u));
            st.push_back(u);
        }
        return result;
    }
};
```

39.6 树分治

39.6.1 点分治：树上距离点对计数

问题描述：给定非负边权的树（或森林）和阈值 k，统计距离不超过 k 的无序点对数，或距离恰好为 k 的点对数（等于“$\le k$”减“$\le k-1$”）。

核心思路：点分治：每次在当前连通块取重心 c，经过 c 的路径距离等于两端到 c 的距离之和。预处理时对每个重心存下块内所有点到 c 的距离 `all[c]`（含 c 自身的 0），以及删去 c 后各子块的距离 `part[c]`，都已排序；然后删去 c 递归处理各子块，`parent` 记点分树父亲。计数时 `pairs` 用双指针统计有序数组中和 $\le k$ 的对数：`all[c]` 中的对数减去每个 `part[c]` 内部的对数，就是经过 c 的路径数；对所有重心求和，每对点恰在第一个把它们分开的重心处计入一次。

接口契约：CentroidPairs(n)，点号 1..n；add(u,v,w) 建树，允许森林但禁止环、重边、自环，边权非负；所有简单路径长度须能用 long long 表示。build() 预处理，可重复调用；改变边后须重建。count_leq(k) 统计距离 ≤k 的无序不同顶点对，count_exact(k) 统计距离恰为 k 的点对；负 k 返回 0，零距离只统计不同顶点，跨连通块不配对。parent[c] 是点分树父节点，根为 0。all[c] 为该层分治块到重心的距离，含重心距离 0；part[c] 为去掉重心后各子块的距离。预处理 O(n $\log^2$ n)，空间 O(n log n)，每次计数 O(n log n)；固定树上多次查询可以复用排序结果。DFS 递归，距离和比较采用减法避免溢出。不是动态点分树更新模板。

```cpp
struct CentroidPairs
{
    using ll = long long;
    int n;
    vector<vector<pair<int, ll>>> g;
    vector<int> siz, removed, parent;
    vector<vector<ll>> all;
    vector<vector<vector<ll>>> part;

    CentroidPairs(int n)
        : n(n),
          g(n + 1),
          siz(n + 1),
          removed(n + 1),
          parent(n + 1),
          all(n + 1),
          part(n + 1)
    {
    }

    void add(int u, int v, ll w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n && u != v && w >= 0);
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }

    void size_dfs(int u, int p)
    {
        siz[u] = 1;
        for (auto [v, w] : g[u])
        {
            if (v == p || removed[v]) continue;
            size_dfs(v, u);
            siz[u] += siz[v];
        }
    }
```

重心与分解（接上页同一结构体）：

```
    int centroid(int u, int p, int total)
    {
        for (auto [v, w] : g[u])
        {
            if (v != p && !removed[v] && siz[v] > total / 2)
                return centroid(v, u, total);
        }
        return u;
    }

    void collect(int u, int p, ll d, vector<ll> &out)
    {
        out.push_back(d);
        for (auto [v, w] : g[u])
        {
            if (v == p || removed[v]) continue;
            assert(d <= numeric_limits<ll>::max() - w);
            collect(v, u, d + w, out);
        }
    }

    void decompose(int entry, int p)
    {
        size_dfs(entry, 0);
        int c = centroid(entry, 0, siz[entry]);
        removed[c] = 1;
        parent[c] = p;
        all[c] = {0};
        for (auto [v, w] : g[c])
        {
            if (removed[v]) continue;
            part[c].push_back({});
            auto &d = part[c].back();
            collect(v, c, w, d);
            sort(d.begin(), d.end());
            all[c].insert(all[c].end(), d.begin(), d.end());
        }
        sort(all[c].begin(), all[c].end());
        for (auto [v, w] : g[c])
        {
            if (!removed[v]) decompose(v, c);
        }
    }
```

重建与距离计数（接上页同一结构体）：

```
    // Input must be a forest. Rebuild after changing edges.
    void build()
    {
        fill(removed.begin(), removed.end(), 0);
        for (int u = 1; u <= n; u++)
        {
            all[u].clear();
            part[u].clear();
        }
        for (int u = 1; u <= n; u++)
        {
            if (!removed[u]) decompose(u, 0);
        }
    }

    ll pairs(const vector<ll> &d, ll k) const
    {
        int l = 0, r = (int)d.size() - 1;
        ll answer = 0;
        while (l < r)
        {
            if (d[l] <= k - d[r])
            {
                answer += r - l;
                l++;
            }
            else
                r--;
        }
        return answer;
    }

    // Unordered pairs of distinct connected vertices. Nonnegative edge weights.
    ll count_leq(ll k) const
    {
        if (k < 0) return 0;
        ll answer = 0;
        for (int c = 1; c <= n; c++)
        {
            answer += pairs(all[c], k);
            for (const auto &d : part[c]) answer -= pairs(d, k);
        }
        return answer;
    }

    ll count_exact(ll k) const
    {
        if (k < 0) return 0;
        return count_leq(k) - (k ? count_leq(k - 1) : 0);
    }
};
```

模板题使用：Luogu P3806

题意：正权树上查询距离为 k 的不同顶点对是否存在。先 build 一次，再用 count_exact 的结果是否非零输出 AYE/NAY。n 至 1 万、询问至 100、边权至 1 万；该程序查询存在性，底层计数另支持零权和森林。

所需模板：CentroidPairs（第 410 页，39.6.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3806

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    CentroidPairs tree(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        long long w;
        std::cin >> u >> v >> w;
        tree.add(u, v, w);
    }
    tree.build();
    while (m--)
    {
        long long k;
        std::cin >> k;
        std::cout << (tree.count_exact(k) ? "AYE" : "NAY") << '\n';
    }
}
```

39.6.2 点分治、启发式合并与虚树用法

点分治 count_exact 计无序不同顶点对，预处理 $O(n \log^2 n)$，每次查询 $O(n \log n)$；边修改后须重建。

启发式合并：轻子树先算后清空，重子树保留；并列最高频颜色编号之和须用 long long。

虚树：显式加入根后构造，只清零本次 vertices 的 dp；逆序遍历虚边，孩子为关键点取路径最小边权，否则取它与孩子 dp 的较小值（用 minimum，不用 distance）。

39.7 动态树分治

39.7.1 动态点分树：点权赋值与距离球求和

问题描述：给定单位边权的树和点权，支持单点赋值，以及询问到点 u 距离不超过 k 的所有点的点权和（动态点分治）。

核心思路：点分治时，对重心 c 所在连通块按到 c 的距离把点权放入桶，线性建出 Fenwick `all[c]`；对 c 的每个分支 b 另建 `part[c][b]`，同样以到 c 的距离为下标。每个点在 `path` 中记下各级重心祖先 c、到 c 的距离 d 和所在分支 b。查询 u 时对每个重心祖先，加上 `all[c]` 中距离不超过 $k-d$ 的和，再减去 `part[c][b]` 中同样范围的和：与 u 同分支的点到 u 的路径不经过 c，它们会在更深的重心处被统计。赋值时把差值加到 `path` 上的所有桶中。

代码依赖：Fenwick（第 284 页，31.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：构造 CentroidSum(n)，T 默认为 long long；n≥1，输入是固定连通单位边树。点号 0..n-1；先 add(u,v)，再 build(values)。set(u,x) 是点权赋值；query(u,k) 返回到 u 距离 ≤k 的全部点权和，含 u 自身；负半径返回 0，

半径用 long long。build 可在同一拓扑上重建并替换全部权值，建树后不再 add 边，不提供 link/cut。每点保存重心祖先、距离和分支，使用总距离桶减同支桶容斥；距离 d 对应 Fenwick 下标 d+1。线性建每层 Fenwick，构造/空间 O(n log n)，修改/查询 O($\log^2$ n)。T 支持零值及加减，所有桶和、赋值差与容斥中间值均须可表示；可用负权，不使用 kth。内部图、路径、桶和辅助函数不能外部改写。原树 DFS 递归深度最坏 O(n)，不等于重心层数；最大链测试提供 256MiB 栈，仍需核对目标评测栈限制。

```
template <class T = long long> struct CentroidSum
{
    struct Record
    {
        int centroid, distance, branch;
    };
    int n;
    vector<vector<int>> g;
    vector<int> siz, removed;
    vector<T> value;
    vector<vector<Record>> path;
    vector<Fenwick<T>> all;
    vector<vector<Fenwick<T>>> part;

    CentroidSum(int n)
        : n(n), g(n), siz(n), removed(n), value(n), path(n), all(n), part(n)
    {
        assert(n > 0);
    }

    void add(int u, int v)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v);
        g[u].push_back(v);
        g[v].push_back(u);
    }
```

子树大小、重心与距离记录（接上页同一结构体）：

```
    void size_dfs(int u, int p)
    {
        siz[u] = 1;
        for (int v : g[u])
        {
            if (v == p || removed[v]) continue;
            size_dfs(v, u);
            siz[u] += siz[v];
        }
    }

    int centroid(int u, int p, int total)
    {
        for (int v : g[u])
            if (v != p && !removed[v] && siz[v] > total / 2)
                return centroid(v, u, total);
        return u;
    }

    void collect(int u, int p, int d, int c, int b, vector<T> &bucket)
    {
        path[u].push_back({c, d, b});
        if ((int)bucket.size() <= d + 1) bucket.resize(d + 2);
        bucket[d + 1] += value[u];
        for (int v : g[u])
            if (v != p && !removed[v]) collect(v, u, d + 1, c, b, bucket);
    }
```

线性构建距离桶和递归分解（接上页同一结构体）：

```
    // Raw distance d is stored at 1-based Fenwick index d+1.
    void make_bit(Fenwick<T> &bit, vector<T> bucket)
    {
        bit.n = (int)bucket.size() - 1;
        bit.a = move(bucket);
        for (int i = 1; i <= bit.n; i++)
            if (i + (i & -i) <= bit.n) bit.a[i + (i & -i)] += bit.a[i];
    }

    void decompose(int entry)
    {
        size_dfs(entry, -1);
        int c = centroid(entry, -1, siz[entry]);
        removed[c] = 1;
        path[c].push_back({c, 0, -1});
        vector<T> total{T{}, value[c]};
        for (int v : g[c])
        {
            if (removed[v]) continue;
            vector<T> bucket(1);
            collect(v, c, 1, c, (int)part[c].size(), bucket);
            if (total.size() < bucket.size()) total.resize(bucket.size());
            for (int d = 1; d < (int)bucket.size(); d++) total[d] += bucket[d];
            part[c].emplace_back();
            make_bit(part[c].back(), move(bucket));
        }
        make_bit(all[c], move(total));
        for (int v : g[c])
            if (!removed[v]) decompose(v);
    }
```

重建、点权赋值与距离球求和（接上页同一结构体）：

```
    // Rebuild resets all values and indices; do not change the tree after building.
    void build(const vector<T> &values)
    {
        assert((int)values.size() == n);
        value = values;
        fill(removed.begin(), removed.end(), 0);
        for (int u = 0; u < n; u++)
        {
            path[u].clear();
            part[u].clear();
        }
        decompose(0);
    }

    void set(int u, T x)
    {
        assert(0 <= u && u < n);
        T delta = x - value[u];
        value[u] = x;
        for (auto [c, d, b] : path[u])
        {
            all[c].add(d + 1, delta);
            if (b != -1) part[c][b].add(d + 1, delta);
        }
    }

    T prefix(const Fenwick<T> &bit, long long radius) const
    {
        if (radius < 0) return T{};
        return bit.sum(radius >= bit.n ? bit.n : (int)radius + 1);
    }

    T query(int u, long long radius) const
    {
        assert(0 <= u && u < n);
        T answer{};
        if (radius < 0) return answer;
        for (auto [c, d, b] : path[u])
        {
            answer += prefix(all[c], radius - d);
            if (b != -1) answer -= prefix(part[c][b], radius - d);
        }
        return answer;
    }
};
```

模板题使用：Luogu P6329 【模板】点分树 / 震波

题意：固定单位边树上修改点权，查询到中心距离不超过 k 的权值和。操作 0 查询，操作 1 赋值；每次先将 x 及 y/k 与最近一次查询答案异或，再把 1-based 顶点 x 减一。更新不改变 last，操作类型不异或。n,m≤100000，点权/新值在 1..10000，解码后 k 在 0..n-1。构造后先 build；原树递归 DFS 需要足够栈空间。

所需模板：Fenwick（第 284 页，31.2 节）、CentroidSum（第 414 页，39.7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6329

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    std::vector<long long> value(n);
    for (auto &x : value) std::cin >> x;
    CentroidSum tree(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        std::cin >> u >> v;
        tree.add(u - 1, v - 1);
    }
    tree.build(value);
    long long last = 0;
    while (m--)
    {
        int op;
        long long x, y;
        std::cin >> op >> x >> y;
        x ^= last;
        y ^= last;
        if (op == 0)
        {
            last = tree.query((int)x - 1, y);
            std::cout << last << '\n';
        }
        else
            tree.set((int)x - 1, y);
    }
}
```

39.7.2 动态点分树：有符号边权活动点直径与端点

问题描述：固定的一棵树，边权可正可负；维护一个“活动点”集合，支持单点激活/取消，并随时查询活动点之间的最大路径权值（允许两端为同一点，此时为 0）及一对取到最大值的端点。

核心思路：建点分树。对重心 c，它自身和删去 c 后的每个分支各对应一个桶 `bag`，桶中存该部分活动点到 c 的距离；`path[u]` 记录 u 在每层重心下的（重心、桶号、距离）。`top[c]` 保存 c 的每个非空桶的最大值，取不同桶的前两名相加，就是经过 c 且两端位于不同分支的最长真实路径（`candidate`），所有重心的候选放在全局集合 `best` 中。修改点 u 时沿 `path[u]` 逐层：先删掉旧候选和旧桶顶，更新桶，再放回新桶顶和新候选。查询取 `best` 最大值，若不为正则返回任一活动点自身、距离 0。

思路中的情况	代码中的处理	行
重心自身的桶	重心 c 以距离 0 放入自己的桶	74
每个分支一个桶	DFS 记下分支内各点到 c 的距离与桶号	81
修改前撤销	从 `best` 删去 c 的旧候选，从 `top[c]` 删去旧桶顶	123
更新桶并恢复	插入或删除 (d, u)，再放回新桶顶和新候选	129
重心候选	`top[c]` 中来自不同桶的前两名距离相加	111
查询	最大候选不为正时，取单个活动点，答案 0	151

接口契约： CentroidDiameter(n)，固定连通无向树、0-based 点号，n>0 且 2n 不溢出 int。add(u,v,w) 仅在首次 build 前；允许负边权，所有简单路径权值和必须能用 long long 表示。build() 递归分解并清空活动集，可反复重建；set(u,on) 幂等启停点。query() 返回 optional<tuple<long long,int,int»，内容为最大距离与任意一对达到最优值的活动端点，允许两端相同，所以非空集答案至少 0；空集 nullopt。各重心按不同分支取最大距离，只合并跨分支真实路径，不能套用非负权直径端点摘要；并列不承诺最小编号。空集预处理 O(n log n)、set O($\log^2$ n)、query O(1)，空间 O(n log n)；逐点激活全树 O(n $\log^2$ n)。支持值拷贝；active 只读用于反色，其余字段与 sum/size_dfs/centroid/collect/decompose/candidate 为内部状态。DFS 保持递归，树输入合法性与足够栈空间由调用方保证。

```
struct CentroidDiameter
{
    using ll = long long;
    using Item = tuple<ll, int, int>;
    using Best = tuple<ll, int, int, int>;
    struct Entry
    {
        int c, b;
        ll d;
    };
    int n;
    bool built = false;
    vector<vector<pair<int, ll>>> g;
    vector<vector<Entry>> path;
    vector<int> siz, removed, active;
    vector<std::set<pair<ll, int>>> bag;
    vector<std::set<Item>> top;
    std::set<Best> best;
    std::set<int> live;

    CentroidDiameter(int n)
        : n(n), g(n), path(n), siz(n), removed(n), active(n), top(n)
    {
        assert(0 < n && n <= INT_MAX / 2);
    }

    ll sum(ll a, ll b) const
    {
        assert(b >= 0 ? a <= LLONG_MAX - b : a >= LLONG_MIN - b);
        return a + b;
    }

    void add(int u, int v, ll w = 1)
    {
        assert(!built && 0 <= u && u < n && 0 <= v && v < n && u != v);
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }
```

递归分解与不同分支的距离（接上页同一结构体）：

```
    void size_dfs(int u, int p)
    {
        siz[u] = 1;
        for (auto [v, w] : g[u])
        {
            if (v == p || removed[v]) continue;
            size_dfs(v, u);
            siz[u] += siz[v];
        }
    }

    int centroid(int u, int p, int total)
    {
        for (auto [v, w] : g[u])
            if (v != p && !removed[v] && siz[v] > total / 2)
                return centroid(v, u, total);
        return u;
    }

    void collect(int u, int p, ll d, int c, int b)
    {
        path[u].push_back({c, b, d});
        for (auto [v, w] : g[u])
        {
            if (v == p || removed[v]) continue;
            collect(v, u, sum(d, w), c, b);
        }
    }

    void decompose(int entry)
    {
        size_dfs(entry, -1);
        int c = centroid(entry, -1, siz[entry]);
        removed[c] = 1;
        path[c].push_back({c, (int)bag.size(), 0});
        bag.emplace_back();
        for (auto [v, w] : g[c])
        {
            if (removed[v]) continue;
            int b = bag.size();
            bag.emplace_back();
            collect(v, c, w, c, b);
        }
        for (auto [v, w] : g[c])
            if (!removed[v]) decompose(v);
    }
```

清空重建与重心候选（接上页同一结构体）：

```
    // Fixed tree, signed weights. Rebuild clears all active points.
    void build()
    {
        fill(removed.begin(), removed.end(), 0);
        fill(active.begin(), active.end(), 0);
        for (int u = 0; u < n; u++)
        {
            path[u].clear();
            top[u].clear();
        }
        bag.clear();
        best.clear();
        live.clear();
        decompose(0);
        built = true;
    }

    optional<Best> candidate(int c) const
    {
        if (top[c].size() < 2) return nullopt;
        auto it = top[c].rbegin();
        auto [x, u, a] = *it++;
        auto [y, v, b] = *it;
        if (u > v) swap(u, v);
        return Best{sum(x, y), u, v, c};
    }
```

活动点设置与端点查询（接上页同一结构体）：

```
    void set(int u, bool on)
    {
        assert(built && 0 <= u && u < n);
        if (active[u] == on) return;
        active[u] = on;
        if (on) live.insert(u);
        else live.erase(u);
        for (auto [c, b, d] : path[u])
        {
            if (auto old = candidate(c)) best.erase(*old);
            if (!bag[b].empty())
            {
                auto [x, v] = *bag[b].rbegin();
                top[c].erase({x, v, b});
            }
            if (on) bag[b].insert({d, u});
            else bag[b].erase({d, u});
            if (!bag[b].empty())
            {
                auto [x, v] = *bag[b].rbegin();
                top[c].insert({x, v, b});
            }
            if (auto now = candidate(c)) best.insert(*now);
        }
    }

    // Maximum distance and any attaining pair; equal endpoints are allowed.
    optional<Item> query() const
    {
        assert(built);
        if (live.empty()) return nullopt;
        if (!best.empty())
        {
            auto [d, u, v, c] = *best.rbegin();
            if (d > 0) return Item{d, u, v};
        }
        int u = *live.begin();
        return Item{0, u, u};
    }
};
```

39.8 树哈希

相关应用：Deterministic sorted-child interning and centroid rooting; exact alternative to randomized tree hashing.

39.8.1 确定性有根与无根树同构

问题描述：判断树同构：为有根树或无根树计算一个编号，两棵树同构当且仅当编号相同（同一对象内比较）；无根树返回一对编号。

核心思路：AHU 方法：有根子树的编号由“儿子编号的有序列表”唯一确定。DFS 求出所有儿子的编号后排序，在 `codes` 这个 `map` 中查找该列表，没出现过就分配新编号，因此编号相等等价于同构，没有哈希碰撞。无根

树先一次 DFS 找重心（最大子树不超过 $n/2$ 的点，至多两个）：只有一个重心时以它为根编码；有两个时分别以两者为根编码，排序后成对返回。

接口契约： TreeIsomorphism iso；rooted(g,root) 返回有根树编号，unrooted(g) 返回无根树 pair 编号。g 是 0-based 非空简单无向连通树，邻接表双向各存一次；root 在 0..n-1。递归 DFS，不验证非法图。所有待比较的树必须共用同一个 iso；编号依赖插入次序，不同对象或清空 codes 前后的编号不能直接比较。对子节点编号排序并用 map 精确驻留，无随机哈希碰撞。无根树取重心：单重心返回 (code,0)，双重心分别以两点为根编码并排序。支持单点；不考虑点/边标签，也不返回顶点映射。codes 只读，编号数须 <INT_MAX，n<=INT_MAX。S 为本次结束时驻留签名数，一次 n 点调用 O(n log(n+1)+n log(S+1))，临时 O(n)，跨调用累计存储至多 O(T)，T 为累计编码遍历的顶点数。

```
struct TreeIsomorphism
{
    map<vector<int>, int> codes;

    int rooted(const vector<vector<int>> &g, int root)
    {
        assert(0 <= root && root < int(g.size()));
        auto dfs = [&](auto &&self, int u, int p) -> int
        {
            vector<int> children;
            for (int v : g[u])
            {
                if (v == p) continue;
                children.push_back(self(self, v, u));
            }
            sort(children.begin(), children.end());
            int next = int(codes.size()) + 1;
            return codes.emplace(children, next).first->second;
        };
        return dfs(dfs, root, -1);
    }
```

继续求解（接上页同一结构体）：

```
    pair<int, int> unrooted(const vector<vector<int>> &g)
    {
        int n = g.size();
        assert(n > 0);
        vector<int> centers;
        auto dfs = [&](auto &&self, int u, int p) -> int
        {
            int size = 1, largest = 0;
            for (int v : g[u])
            {
                if (v == p) continue;
                int child = self(self, v, u);
                size += child;
                largest = max(largest, child);
            }
            largest = max(largest, n - size);
            if (largest <= n / 2) centers.push_back(u);
            return size;
        };
        dfs(dfs, 0, -1);
        int a = rooted(g, centers[0]);
        if (centers.size() == 1) return {a, 0};
        int b = rooted(g, centers[1]);
        if (a > b) swap(a, b);
        return {a, b};
    }
};
```

模板题使用：P5043 【模板】树同构 / [BJOI2015] 树的同构

题意：M 棵无根树，每棵 N 个点，1<=N,M<=50。输入 N 个父亲编号，0 表示根；转换无向树后忽略输入根。输出与本树同构的最早树编号；所有树共用同一驻留表。

所需模板：TreeIsomorphism（第 424 页，39.8.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5043

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int m;
    cin >> m;
    TreeIsomorphism iso;
    map<pair<int, int>, int> first;
    for (int id = 1; id <= m; id++)
    {
        int n;
        cin >> n;
        vector<vector<int>> g(n);
        for (int u = 0; u < n; u++)
        {
            int p;
            cin >> p;
            if (p == 0) continue;
            p--;
            g[u].push_back(p);
            g[p].push_back(u);
        }
        auto code = iso.unrooted(g);
        auto result = first.emplace(code, id);
        cout << result.first->second << '\n';
    }
}
```

相关应用：Deterministic sorted-child interning and centroid rooting; exact alternative to randomized tree hashing.

39.8.2 双 64 位随机树哈希（Mix13）

功能：对一棵有根树，求每个点子树的 128 位（两个 64 位）随机哈希值；两棵子树同构时哈希一定相等，不同构时以极高概率不等，可用于统计不同构子树的种类或比较多棵树。

核心思路：哈希定义为 $h(u) = s + \sum_{v\in \mathrm{son}(u)} \mathrm{mix13}(h(v))$，模 2^{64} 自然溢出，两个分量用不同种子 s_1, s_2 分别计算；求和与儿子顺序无关，因此不需要排序。`mix13` 是 64 位整数混淆函数，使不同子树的贡献难以相互抵消。代码先从根 BFS 得到父亲 `fa` 和层序 `q`，再逆层序把每个点的 `mix13` 值加到父亲上，非递归。

接口契约：tree_hash(g,root,s1,s2) 对 0-based 树 g（非空、连通、简单无向）以 root 为根，返回每个点 u 的有根子树哈希对 h[u]：h[u]=(s1,s2)+ 各儿子 v 的 (mix13(h[v].first),mix13(h[v].second)) 之和，全部模 2^64 自然溢出；叶子为 (s1,s2)。儿子顺序无关、不需要排序。种子为非零且互异的随机 64 位数，由调用方生成（如 mt19937_64 以时间为种子），需要互相比较的所有树必须共用同一对种子。h[u] 相等当且仅当两子树同构（概率意义，128 位哈希，非确定性证明）；统计不同子树种类时对 h 排序去重。BFS 序非递归，O(n) 时间、O(n) 空间，百万点长链不会爆栈。需要确定性结论时用 TreeIsomorphism。

```
// Double 64-bit random rooted-tree hash (Mix13); h[u] equal <=> subtrees isomorphic w.h.p.
// 0-based tree, any root. Trees compared together must share the same nonzero seeds s1 != s2.
unsigned long long mix13(unsigned long long x)
{
    x ^= x >> 30, x *= 0xbf58476d1ce4e5b9ULL;
    x ^= x >> 27, x *= 0x94d049bb133111ebULL;
    return x ^ (x >> 31);
}

vector<pair<unsigned long long, unsigned long long>>
tree_hash(const vector<vector<int>> &g, int root, unsigned long long s1, unsigned long long s2)
{
    int n = g.size();
    vector<int> fa(n, -1), q{root};
    for (int i = 0; i < (int)q.size(); i++)
        for (int v : g[q[i]])
            if (v != fa[q[i]]) fa[v] = q[i], q.push_back(v);
    vector<pair<unsigned long long, unsigned long long>> h(n, {s1, s2});
    for (int i = n - 1; i > 0; i--)
    {
        int u = q[i];
        h[fa[u]].first += mix13(h[u].first);
        h[fa[u]].second += mix13(h[u].second);
    }
    return h;
}
```

模板题使用：UOJ #763 树哈希

题意：n≤10^6 个点的树以 1 为根，求互不同构的有根子树种类数。点号减一转为 0-based，以时间为种子生成两枚 64 位种子，对所有 h[u] 排序去重计数。

所需模板：tree_hash（第 428 页，39.8.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://uoj.ac/problem/763

```
int main()
{
    int n;
    scanf("%d", &n);
    vector<vector<int>> g(n);
    for (int i = 1; i < n; i++)
    {
        int a, b;
        scanf("%d%d", &a, &b);
        a--, b--;
        g[a].push_back(b), g[b].push_back(a);
    }
    mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
    unsigned long long s1 = rng() | 1, s2 = rng() | 1;
    auto h = tree_hash(g, 0, s1, s2);
    sort(h.begin(), h.end());
    printf("%d\n", int(unique(h.begin(), h.end()) - h.begin()));
}
```

相关应用：Deterministic sorted-child interning and centroid rooting; exact alternative to randomized tree hashing.

39.8.3 树哈希 O(n) 换根

功能：在 tree_hash 的基础上，求以每个点 u 为根时整棵树的哈希 $f(u)$，用于无根树比较（取所有 $f(u)$ 的最小值等作代表）。

核心思路：换根 DP。根处 $f(\text{root}) = h(\text{root})$。按 BFS 序处理点 u（父亲 p 已算好）：以 p 为根的整树哈希 $f(p)$ 去掉儿子 u 的贡献 $\text{mix13}(h(u))$，就是“以 u 为根时 p 那一侧子树”的哈希；它作为 u 的新儿子，所以 $f(u) = h(u) + \text{mix13}(f(p) - \text{mix13}(h(u)))$，两个分量分别计算。

接口契约：tree_hash_reroot(g,root,h) 由同一根的 h=tree_hash(g,root,s1,s2) 求 f[u]：以 u 为根时整棵树的哈希。父方向子树的哈希为 f[p]-mix13(h[u])，故 f[u]=h[u]+mix13(f[p]-mix13(h[u]))，两分量分别计算。无根树比较取所有 f[u] 的最小值（或按重心）作为整树代表。O(n) 时间与空间，非递归。

```
// f[u] = hash of the whole tree rooted at u, from h = tree_hash(g, root, s1, s2). O(n).
vector<pair<unsigned long long, unsigned long long>>
tree_hash_reroot(const vector<vector<int>> &g, int root, const vector<pair<unsigned long long, unsigned long
    long>> &h)
{
    int n = g.size();
    vector<int> fa(n, -1), q{root};
    for (int i = 0; i < (int)q.size(); i++)
        for (int v : g[q[i]])
            if (v != fa[q[i]]) fa[v] = q[i], q.push_back(v);
    auto f = h;
    for (int i = 1; i < n; i++)
    {
        int u = q[i], p = fa[u];
        // the parent side seen from u is f[p] without u's own contribution
        f[u].first += mix13(f[p].first - mix13(h[u].first));
        f[u].second += mix13(f[p].second - mix13(h[u].second));
    }
    return f;
}
```

第四十章　最短路问题

40.1　最短路

40.1.1　非负权最短路

问题描述：给定非负边权有向图和源点 s，求 s 到所有点的最短距离，并可沿前驱 `pre` 恢复到某点的最短路径。

核心思路：小根堆按距离取点，懒删除：弹出的 (d,u) 若 d 不等于当前 `dis[u]` 说明已过期，直接跳过；否则 u 的距离已确定，用它松弛所有出边，更新 `dis` 和 `pre` 并把新距离入堆。`path(t)` 从 t 沿 `pre` 走回 s 后反转。

接口契约：add(u,v,w)、run(s)，dis 为距离，pre 为前驱，path(t) 返回点序列。不可达用 LLONG_MAX；要求所有有限最短距离严格小于该哨兵，w≥0。懒删除堆在多重边图中按 O(n+m log(m+1)) 计，空间 O(n+m)；简单图常写 O((n+m)log(n+1))。最短路树：run 后每个 v≠s 的 pre 入边构成一棵最短路树；要总边权最小，就在满足 dis[u]+w=dis[v] 的入边中选 w 最小者。Johnson 全源最短路：加超级源以 0 权连所有点，用 BellmanFord 求势 h，把边权改成 w+h[u]-h[v]≥0 后对每个起点跑 Dijkstra，真实距离为 d+h[t]-h[s]，O(nm log m)。

```
struct Dijkstra
{
    using ll = long long;
    static constexpr ll inf = LLONG_MAX;
    int n;
    vector<vector<pair<int, ll>>> g;
    vector<ll> dis;
    vector<int> pre;

    Dijkstra(int n) : n(n), g(n + 1) {}

    void add(int u, int v, ll w)
    {
        assert(w >= 0);
        g[u].push_back({v, w});
    }

    void run(int s)
    {
        dis.assign(n + 1, inf);
        pre.assign(n + 1, -1);
        dis[s] = 0;
        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> q;
        q.push({0, s});
        while (!q.empty())
        {
            auto [d, u] = q.top();
            q.pop();
            if (d != dis[u]) continue;
            for (auto [v, w] : g[u])
                if (w < inf - d && d + w < dis[v])
                {
                    dis[v] = d + w;
                    pre[v] = u;
                    q.push({dis[v], v});
                }
```

```
        }
    }

    vector<int> path(int t) const
    {
        if (dis[t] == inf) return {};
        vector<int> ans;
        for (; t != -1; t = pre[t]) ans.push_back(t);
        reverse(ans.begin(), ans.end());
        return ans;
    }
};
```

模板题使用：Luogu P4779

题意：给非负权有向图和源点 s，输出 s 到各点的最短距离。加完边后 run(s)，读取 dis[1..n]；本题保证源点可达所有点，一般输入的不可达值须按目标题目格式另行转换。

所需模板：Dijkstra（第 431 页，40.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4779

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m, s;
    std::cin >> n >> m >> s;
    Dijkstra graph(n);
    while (m--)
    {
        int u, v;
        long long w;
        std::cin >> u >> v >> w;
        graph.add(u, v, w);
    }
    graph.run(s);
    for (int i = 1; i <= n; i++) std::cout << graph.dis[i] << (i == n ? '\n' : ' ');
}
```

40.1.2 Bellman–Ford：负环影响与路径方案

问题描述：给定可含负权的有向图和起点 s，求 s 到各点的最短距离及一条最短路的原边序列；若某点可经由从 s 可达的负环到达，则其距离为 $-\infty$，并给出一个这样的负环。$s=0$ 时视为从所有点同时出发，用于判全图负环。

核心思路：做至多 n 轮“扫描全部边做松弛”，`pre[v]` 记最后一次更新 v 的边。无负环时 $n-1$ 轮内收敛；若第 n 轮仍有松弛，被更新的点必受负环影响，标记 `neg`。从第 n 轮最后被松弛的点沿 `pre` 回退 n 步必落在环上，再绕一圈收集环边。最后把 `neg` 沿边反复传播到不动点，凡能从负环到达的点都标为 $-\infty$。路径方案沿 `pre` 倒推后翻转。

思路中的情况	代码中的处理	行
最多 n 轮全边松弛	某轮无更新即提前结束；`last` 记本轮最后被松弛的点	31
第 n 轮仍可松弛	该点距离为 $-\infty$，置 `neg`	39
进入负环	从 `last` 沿 `pre` 回退 n 步	46
收集负环	从环上点 x 绕回自身，记录边号后翻转为正向	47
传播 $-\infty$	沿边扩散 `neg` 直到不再变化	54

接口契约： BellmanFord(n)，点号 1..n，0<n<INT_MAX。add(u,v,w) 加有向边并返回 0-based 原边号，允许重边、自环和完整 signed64 边权，总边数 ≤INT_MAX。run(s) 返回是否不存在从 s 可达的负环；run(0) 把全体点初值设 0，等价于超级源点，用于查全图负环。每次 run 重置状态；add 后旧结果失效，须重新 run。

运行后 dis[v]==inf 表示不可达；neg[v] 为真表示从起点可达的负环能到达 v，距离为负无穷（这时不可读取 dis 作为答案）；其他点的 dis 为有限 int128 最短距离。path(v) 返回有限最短路的正向原边号；不可达、负无穷或空路径都返回空 vector，须结合 dis/neg 判定。cycle 为空表示无相关负环，否则是一个相关负环的正向原边号，可通过 e[id].u/v/w 还原顶点和权值。run(0) 的有限距离是从任意起点出发的最小值，允许零边路径，不是某个固定单源距离。

O(nm+n+m) 时间、O(n+m) 空间；path 输出 O(路径长)。inf=2^126，全过程 int128 累加，不能无条件转为 long long。在给定 n,m 的 int 边界内，n 轮最多 nm 次更新，标签对应游走至多 nm 条边，绝对值 <2^125，不碰 inf 也不溢出。e/g/dis/pre/neg/cycle 和 ready 只读，不手改。

```
struct BellmanFord
{
    static constexpr long long inf = LLONG_MAX / 4;
    struct Edge
    {
        int u, v;
        long long w;
    };
    int n;
    vector<Edge> e;
    vector<long long> dis;
    vector<int> pre, neg, cycle; // neg[v]: distance is -inf; cycle: edge ids of one negative cycle

    BellmanFord(int n) : n(n) {}

    int add(int u, int v, long long w)
    {
        e.push_back({u, v, w});
        return int(e.size()) - 1;
    }

    // 1-based; s=0 starts every vertex at 0 to find any negative cycle. True iff none found.
    bool run(int s)
    {
        dis.assign(n + 1, s ? inf : 0);
        pre.assign(n + 1, -1);
        neg.assign(n + 1, 0);
        cycle.clear();
        dis[s] = 0;
        int last = -1;
        for (int step = 0; step < n; step++)
        {
            last = -1;
            for (int id = 0; id < (int)e.size(); id++)
            {
```

```
                auto [u, v, w] = e[id];
                if (dis[u] == inf || dis[u] + w >= dis[v]) continue;
                dis[v] = dis[u] + w, pre[v] = id, last = v;
                if (step == n - 1) neg[v] = 1;
            }
            if (last == -1) break;
        }
        if (last != -1)
        {
            int x = last;
            for (int i = 0; i < n; i++) x = e[pre[x]].u;
            for (int y = x; cycle.empty() || y != x; y = e[pre[y]].u) cycle.push_back(pre[y]);
            reverse(cycle.begin(), cycle.end());
        }
        for (bool changed = true; changed;) // everything reachable from round n is -inf
        {
            changed = false;
            for (auto [u, v, w] : e)
                if (neg[u] && !neg[v]) neg[v] = 1, changed = true;
        }
        return cycle.empty();
    }

    // Finite shortest path as edge ids; empty for unreachable, -inf or t=s.
    vector<int> path(int t) const
    {
        vector<int> ids;
        if (dis[t] == inf || neg[t]) return ids;
        for (; pre[t] != -1; t = e[pre[t]].u) ids.push_back(pre[t]);
        reverse(ids.begin(), ids.end());
        return ids;
    }
};
```

模板题使用：AOJ GRL_1_B / 含负边的单源最短路

题意：0-based 有向图，n≤1000、m≤2000、边权绝对值 ≤10000。从 r 可达负环时输出 NEGATIVE CYCLE，否则逐点输出距离，不可达输出 INF。转成模板 1-based 编号；本题有限距离在 signed64 内，故可显式转 long long 输出。不可达位置的负环不影响此题判断。

所需模板：BellmanFord（第 432 页，40.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=GRL_1_B

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s;
    cin >> n >> m >> s;
    BellmanFord t(n);
    while (m--)
    {
        int u, v, w;
        cin >> u >> v >> w;
        t.add(u + 1, v + 1, w);
    }
    if (!t.run(s + 1)) cout << "NEGATIVE CYCLE\n";
    else
        for (int u = 1; u <= n; u++)
        {
            if (t.dis[u] == BellmanFord::inf) cout << "INF\n";
            else cout << (long long)t.dis[u] << '\n';
        }
}
```

40.1.3 SPFA：队列松弛、负环判定与有限路径

问题描述：给定可含负权的有向图和起点 s，判断是否存在从 s 可达的负环；若不存在，求 s 到各点的最短距离与一条最短路的原边序列。$s=0$ 时所有点初始距离为 0，可判全图负环，也可直接求差分约束的一组可行解。

核心思路：队列优化的 Bellman–Ford：只有距离被改小的点才入队，再用它松弛出边；`in` 标记点是否已在队中以免重复入队。每次严格松弛时令 `len[v]=len[u]+1`，表示当前标签对应行走的边数；无负环时最短路是简单路径，边数不超过 $n-1$，因此一旦 `len[v]` 达到 n 就立即报告负环。`pre[v]` 记最后更新 v 的边，用于倒推路径。**接口契约：**Spfa(n)，点号 1..n，0<n<INT_MAX。add(u,v,w) 返回 0-based 原边号，允许重边、自环、完整 signed64 权值，边数 ≤INT_MAX；加边令 valid=false。run(s) 查从 s 可达的负环，s=0 同时初始化所有点为 0 以查全图。返回 false 时 valid=false，全部 dis/pre 均为未完成状态，不能读取为最短路或恢复路径；需要负环方案或逐点负无穷分类用 BellmanFord。

run 返回 true 时 valid=true，dis[v]==inf 为不可达，其余为 int128 最短距离；run(0) 距离为任意起点出发的最小值，且本身就是全部约束 x_v−x_u≤w 的一个可行解。path(v) 返回正向原边号；不可达与零边路径均为空，结合 dis 区分。每次 run 重置临时队列/标记/路径边数，支持反复改起点与对象复制。

FIFO 队列，最坏 O(nm+n+m) 时间，O(n+m) 空间，不保证近线性；队列去重不等于永久访问标记。严格松弛后维护标签所代表游走的边数，达到 n 立即判负环。停止前标签最多 n 条边，绝对值 ≤n · 2^63<2^94，int128 与 inf=2^126 足够。不要无条件转 long long；e/g/dis/pre/valid 只读，其余按接口使用。

```
struct Spfa
{
    static constexpr long long inf = LLONG_MAX / 4;
    struct Edge
```

```
    {
        int u, v;
        long long w;
    };
    int n;
    vector<Edge> e;
    vector<vector<int>> g;
    vector<long long> dis;
    vector<int> pre;

    Spfa(int n) : n(n), g(n + 1) {}

    int add(int u, int v, long long w)
    {
        e.push_back({u, v, w});
        g[u].push_back(int(e.size()) - 1);
        return int(e.size()) - 1;
    }

    // 1-based; s=0 starts from every vertex (difference constraints). False on a negative cycle.
    bool run(int s)
    {
        dis.assign(n + 1, inf);
        pre.assign(n + 1, -1);
        vector<int> in(n + 1), len(n + 1);
        queue<int> q;
        for (int u = 1; u <= n; u++)
            if (!s || u == s) dis[u] = 0, in[u] = 1, q.push(u);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            in[u] = 0;
            for (int id : g[u])
            {
                int v = e[id].v;
                if (dis[u] + e[id].w >= dis[v]) continue;
                dis[v] = dis[u] + e[id].w, pre[v] = id, len[v] = len[u] + 1;
                if (len[v] >= n) return false;
                if (!in[v]) in[v] = 1, q.push(v);
            }
        }
        return true;
    }

    // Edge ids of the shortest path to t; empty if unreachable or t is a start.
    vector<int> path(int t) const
    {
        vector<int> ids;
        if (dis[t] == inf) return ids;
        for (; pre[t] != -1; t = e[pre[t]].u) ids.push_back(pre[t]);
        reverse(ids.begin(), ids.end());
        return ids;
    }
};
```

模板题使用：Luogu P3385 / 从 1 可达的负环

题意：T 组图，每条输入边权非负时加双向边，负数仅加 u 到 v；run(1) 只查从 1 可达的负环，有则 YES，否则 NO。m 是输入记录数，不是最终边数；不可达部分的负环不影响本题。

所需模板：Spfa（第 435 页，40.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3385

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int T;
    cin >> T;
    while (T--)
    {
        int n, m;
        cin >> n >> m;
        Spfa t(n);
        while (m--)
        {
            int u, v, w;
            cin >> u >> v >> w;
            t.add(u, v, w);
            if (w >= 0) t.add(v, u, w);
        }
        cout << (t.run(1) ? "NO" : "YES") << '\n';
    }
}
```

40.1.4 Floyd 全源最短路与原边方案

问题描述：给定可含负权、重边的有向图，求所有点对之间的最短距离，并能输出任一点对的一条最短路（原边序列）；若图中有负环则报告失败。

核心思路：`dis[u][v]` 初值取 $u \to v$ 重边中的最小权，`nxt[u][v]` 记最短路的第一条原边。按 $k = 1..n$ 逐个允许 k 作中转点，用 $\mathrm{dis}[u][k] + \mathrm{dis}[k][v]$ 更新 $\mathrm{dis}[u][v]$，更新时第一条边继承 `nxt[u][k]`；只有两段都可达才相加。处理 k 前若 $\mathrm{dis}[k][k] < 0$ 说明存在经过 k 的负环，立即返回。输出路径时从 s 反复走 `nxt[s][t]` 这条边，直到到达 t。

接口契约：Floyd(n)，点号 1..n，0<n<INT_MAX；add(u,v,w) 返回 0-based 原边号，有向边，允许重边、自环及完整 signed64 权值，边数 ≤INT_MAX。无向图需分别加入双向边。run() 重新建立矩阵；存在任何负环返回 false，valid=false，全部结果不可读，不能只取未受负环影响的点对。

成功时 dis[u][v]==inf 为不可达，其他为 int128 最短距离，dis[u][u]=0。path(u,v) 返回正向原边号，并列任选；不可达和 u=v 均为空，结合 dis 区分。nxt 为恢复路径的首条原边号矩阵，通常只需调用 path；e/dis/nxt/valid 只读。add 清除 valid，支持重复运行、加边后重跑与独立值拷贝。

run 时间 $O(n^3+m)$，空间 $O(n^2+m)$，path 时间与输出空间 O(路径长)。每次处理 k 之前检查 dis[k][k]<0，立即拒绝，不能延后到所有松弛结束，否则负环下中间量可指数增长。通过的枢轴不含负环；有限标签取简单路径或闭合路径，拼接候选绝对值 $<2n \cdot 2^{63}<2^{95}$，int128 与 inf=2^{126} 足够。两段都有限才相加；不要无条件转 long long。矩阵占用约 $20(n+1)^2$ 字节，另计 vector 和原边表，n 应按时间/内存选择。

```
struct Floyd
{
    static constexpr long long inf = LLONG_MAX / 4;
    struct Edge
```

```
    {
        int u, v;
        long long w;
    };
    int n;
    vector<Edge> e;
    vector<vector<long long>> dis;
    vector<vector<int>> nxt;

    Floyd(int n) : n(n) {}

    int add(int u, int v, long long w)
    {
        e.push_back({u, v, w});
        return int(e.size()) - 1;
    }

    // 1-based. False on a negative cycle; then every result is invalid.
    bool run()
    {
        dis.assign(n + 1, vector<long long>(n + 1, inf));
        nxt.assign(n + 1, vector<int>(n + 1, -1));
        for (int u = 1; u <= n; u++) dis[u][u] = 0;
        for (int id = 0; id < (int)e.size(); id++)
        {
            auto [u, v, w] = e[id];
            if (w < dis[u][v]) dis[u][v] = w, nxt[u][v] = id;
        }
        for (int k = 1; k <= n; k++)
        {
            if (dis[k][k] < 0) return false;
            for (int u = 1; u <= n; u++)
                if (dis[u][k] != inf)
                    for (int v = 1; v <= n; v++)
                        if (dis[k][v] != inf && dis[u][k] + dis[k][v] < dis[u][v])
                            dis[u][v] = dis[u][k] + dis[k][v], nxt[u][v] = nxt[u][k];
        }
        return true;
    }

    // Edge ids from s to t; empty if unreachable or s=t.
    vector<int> path(int s, int t) const
    {
        vector<int> ids;
        if (dis[s][t] == inf) return ids;
        for (; s != t; s = e[ids.back()].v) ids.push_back(nxt[s][t]);
        return ids;
    }
};
```

模板题使用：AOJ GRL_1_C / All Pairs Shortest Path

题意：0-based 有向图输入转换为 1-based；整图有负环输出 NEGATIVE CYCLE，否则逐行输出距离矩阵，不可达为 INF。n≤100、m≤9900、|w|≤20000000，有限结果可转 long long。题面无自环重边，模板额外支持。

所需模板：Floyd（第 437 页，40.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=GRL_1_C&lang=en

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    Floyd t(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        t.add(u + 1, v + 1, w);
    }
    if (!t.run())
    {
        cout << "NEGATIVE CYCLE\n";
        return 0;
    }
    for (int u = 1; u <= n; u++)
    {
        for (int v = 1; v <= n; v++)
        {
            if (v > 1) cout << ' ';
            if (t.dis[u][v] == Floyd::inf) cout << "INF";
            else cout << (long long)t.dis[u][v];
        }
        cout << '\n';
    }
}
```

40.1.5 杭州 2023 G：开放时间的单位边最短路

QOJ8236。初始蛇身从头到尾编号 $0,\dots,k-1$，第 $i\ge 1$ 节所在点开放时间为 $k-i$（尾格为 1），其余为零；该限制必要且充分。转移 $d_v=\max(d_u+1,\mathrm{open}_v)$。按距离递增处理时首次发现即最优：候选等于 d_u+1 的入 FIFO，其余入开放时间桶，每次取两者较小者；桶指针只扫到下一个 FIFO 距离。$R=\max \mathrm{open}$，时间 $O(V+E+R)$，空间 $O(V+R)$。仅适用于所有边权为 1；一般非负权用优先队列。答案用 unsigned long long 累加距离平方（模 2^{64}）。

40.2 k 短路

40.2.1 非负权严格次短行走

问题描述：给定非负权有向图和起点 s，对每个点 v 求 s 到 v 的最短行走长度，以及严格大于它的最小行走长度（严格次短路）；行走可重复经过点和边。

核心思路：Dijkstra 的推广：每个点维护两个标签 `dis[v][0]`（最短）和 `dis[v][1]`（严格次短），堆中可同时有同一点的两种长度。弹出 (d,u) 后若 d 已大于 `dis[u][1]` 就丢弃，否则用 $d+w$ 松弛 v：若比最短还小，就与 `dis[v][0]` 交换，被替换下来的旧最短值接着作为次短候选；候选值严格介于最短和次短之间时更新次短。每次更新都把新值入堆。

接口契约：StrictSecondShortest(n)，点号 1..n，n<INT_MAX；add(u,v,w) 添加有向边，无向边添加两次。允许重边、自环及零权，0<=w<LLONG_MAX。run(s) 返回 dis[v][0/1]，分别为最短距离与严格大于它的最小距离；同长路线不重复计数，允许重复点边，s 到自身包含空路。不存在或长度 >=LLONG_MAX 返回 inf=LLONG_MAX；只保证此阈值以下的精确答案，超界扩展跳过。只读 g/n；可重复求解、加边后重算。O((n+m) log(n+m+1)) 时间，O(n+m) 空间，不返回行走方案。

```
struct StrictSecondShortest
{
    using ll = long long;
    static constexpr ll inf = LLONG_MAX;
    int n;
    vector<vector<pair<int, ll>>> g;

    StrictSecondShortest(int n) : n(n), g(n + 1)
    {
        assert(n > 0 && n < INT_MAX);
    }

    void add(int u, int v, ll w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(0 <= w && w < inf);
        g[u].push_back({v, w});
    }
```

继续求解（接上页同一结构体）：

```
    vector<array<ll, 2>> run(int s) const
    {
        assert(1 <= s && s <= n);
        vector<array<ll, 2>> dis(n + 1, {inf, inf});
        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> q;
        dis[s][0] = 0;
        q.push({0, s});
        while (!q.empty())
        {
            auto [d, u] = q.top();
            q.pop();
            if (d > dis[u][1]) continue;
            for (auto [v, w] : g[u])
            {
                if (w >= inf - d) continue;
                ll next = d + w;
                if (next < dis[v][0])
                {
                    swap(next, dis[v][0]);
                    q.push({dis[v][0], v});
                }
                if (dis[v][0] < next && next < dis[v][1])
                {
                    dis[v][1] = next;
                    q.push({next, v});
                }
            }
        }
        return dis;
    }
};
```

40.2.2 正权前 K 短行走

问题描述：给定正权有向图、起点 s、终点 t 和 k，按长度从小到大求 s 到 t 的前 k 短行走的长度；行走可重复经过点和边，不同边序列分别计数。

核心思路：对“到达状态”做 Dijkstra：堆中存 (d,u)，表示一条长度为 d、到达 u 的行走。点 u 第 i 次被弹出时，d 恰为到 u 的第 i 短行走长度，`count[u]` 记录已弹出次数。前 k 次弹出 t 即得答案；每个点最多扩展 k 次，超过的弹出直接丢弃（第 k 短以后的行走不会帮助产生前 k 短答案），已满 k 次的点也不再入堆。

接口契约：KShortestWalks(n)，点号 1..n，n<INT_MAX；add(u,v,w) 添加有向边，要求 0<w<LLONG_MAX，零权边/环不在接口范围内。允许自环与重边，每次 add 视为不同边。run(s,t,k) 按非降序返回至多 k 个行走长度，k>=0；不同边序列分别计数，即使等长。允许重复点边，不是简单路径算法。s=t 时第一个答案为空路 0；若排除空路，调用 k+1 后删除首项，需 k<INT_MAX。终点仍可继续走再回来。长度 >=LLONG_MAX 不返回；只有阈值以下答案有保证。每个顶点至多扩展 k 次，O(n+km log(km+2)) 时间、O(n+km) 空间；k 大时应改用更高效算法。只读 g/n，可重复求解和加边后重算，不返回方案。

```cpp
struct KShortestWalks
{
    using ll = long long;
    static constexpr ll inf = LLONG_MAX;
    int n;
    vector<vector<pair<int, ll>>> g;

    KShortestWalks(int n) : n(n), g(n + 1)
    {
        assert(n > 0 && n < INT_MAX);
    }

    void add(int u, int v, ll w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(0 < w && w < inf);
        g[u].push_back({v, w});
    }
```

继续求解（接上页同一结构体）：

```
    vector<ll> run(int s, int t, int k) const
    {
        assert(1 <= s && s <= n && 1 <= t && t <= n && k >= 0);
        vector<ll> answer;
        if (k == 0) return answer;
        vector<int> count(n + 1);
        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> q;
        q.push({0, s});
        while (!q.empty())
        {
            auto [d, u] = q.top();
            q.pop();
            if (count[u] == k) continue;
            count[u]++;
            if (u == t)
            {
                answer.push_back(d);
                if (int(answer.size()) == k) break;
            }
            for (auto [v, w] : g[u])
            {
                if (count[v] < k && w < inf - d)
                {
                    q.push({d + w, v});
                }
            }
        }
        return answer;
    }
};
```

第四十一章　生成树问题

41.1　最小生成树

41.1.1　Kruskal 重构森林与阈值连通块

功能：构建 Kruskal 重构树（森林）：按边权升序（或降序）做 Kruskal，每次成功合并新建一个父节点，权值为该边权。支持查询：只保留权不超过 w（降序时不小于 w，可改为严格）的边时，点 u 所在连通块对应的重构树节点，其子树叶子恰为该连通块。

核心思路：叶子是原点。排序后逐边用并查集合并，`top` 记每个并查集根当前对应的重构树节点；合并时新建节点 x，两个儿子是两侧的 `top`，`val[x]` 为边权。边按顺序处理，所以从叶到根 `val` 单调不减（降序时不增）；新节点编号总大于儿子，可按编号顺序自底向上聚合子树信息。查询时在倍增表 `up` 上从高位到低位跳：祖先的 `val` 满足阈值就跳上去，最终停在满足条件的最高祖先。

代码依赖：dsu（第 264 页，28.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：KruskalTree(n)，0≤n≤INT_MAX/2，原点 0..n-1；add(u,v,w) 加入无向边，返回插入编号，边数≤INT_MAX，支持完整 signed64 权、重边、自环、负权和断连。build(descending=false) 重建并返回连通块数；默认升序，true 降序。每次成功合并创建父点：ch 为左右儿子，val 为合并边权；叶点 ch={-1,-1} 且 val 无意义。总点数 k=2n-连通块数，父编号大于子编号；可按 n..k-1 聚合叶点信息，不需要模拟 DFS。up[0] 为父点，根为-1。

component(u,w,strict=false) 返回阈值子图中 u 所属分量的最高重构节点；默认升序保留边权 ≤w，降序保留 ≥w，strict 改成 < 或 >。其子树叶点恰为分量，可从聚合数组查大小、最小值等。同权边必须连续爬完所有合法祖先；不以叶点 val 充当无穷哨兵。构建 O(m log(m+1)+n log(n+1))，空间 O(m+n log(n+1))，查询 O(log(n+1))。依赖 dsu，合并前按大小交换根。支持重复 build、切换方向与值复制；加边后必须重建再查询，构建后成员只读。空图只允许 build、不允许查询。

```
struct KruskalTree
{
    struct Edge
    {
        int u, v;
        long long w;
    };

    int n;
    bool down = false;
    vector<Edge> e;
    vector<long long> val;
    vector<array<int, 2>> ch;
    vector<vector<int>> up;

    KruskalTree(int n) : n(n)
    {
        assert(0 <= n && n <= INT_MAX / 2);
    }

    int add(int u, int v, long long w)
    {
        assert(0 <= u && u < n && 0 <= v && v < n);
        e.push_back({u, v, w});
        return (int)e.size() - 1;
    }
```

重建森林与倍增表（接上页同一结构体）：

```
    int build(bool descending = false)
    {
        down = descending;
        ch.assign(n, {-1, -1});
        val.assign(n, 0);
        dsu d(n);
        vector<int> top(n), ids(e.size());
        iota(top.begin(), top.end(), 0);
        iota(ids.begin(), ids.end(), 0);
        sort(ids.begin(), ids.end(), [&](int a, int b)
        {
            if (e[a].w != e[b].w)
                return down ? e[a].w > e[b].w : e[a].w < e[b].w;
            return a < b;
        });
        for (int id : ids)
        {
            auto [u, v, w] = e[id];
            u = d.find(u);
            v = d.find(v);
            if (u == v) continue;
            if (d.sz[u] < d.sz[v]) swap(u, v);
            int x = ch.size();
            ch.push_back({top[u], top[v]});
            val.push_back(w);
            d.merge(u, v);
            top[u] = x;
        }
        int k = ch.size();
        int h = max(1, (int)bit_width((unsigned)k));
        up.assign(h, vector<int>(k, -1));
        for (int u = n; u < k; u++)
        {
            for (int v : ch[u]) up[0][v] = u;
        }
        for (int j = 1; j < h; j++)
        {
            for (int u = 0; u < k; u++)
            {
                int v = up[j - 1][u];
                if (v != -1) up[j][u] = up[j - 1][v];
            }
        }
        return 2 * n - k;
    }
```

阈值分量查询（接上页同一结构体）：

```
    int component(int u, long long w, bool strict = false) const
    {
        assert(0 <= u && u < n && !up.empty());
        for (int j = (int)up.size() - 1; j >= 0; j--)
        {
            int v = up[j][u];
            if (v == -1) continue;
            bool ok = down ? val[v] >= w : val[v] <= w;
            if (strict && val[v] == w) ok = false;
            if (ok) u = v;
        }
        return u;
    }
};
```

为何阈值查询正确 升序处理完所有权不超过 w 的边后，并查集恰好是该阈值子图的连通分量。每次成功合并的两棵子树叶点集合不交，新父点记录其并集；沿叶点到根的内部点权单调不减。因此最高合法祖先的叶点集合就是答案。降序情形反向比较；严格阈值排除等权合并。同权内部点可能连续出现，倍增必须爬到最高合法点。

如何聚合分量 叶点先存原数据，随后按编号 n 到 k-1 合并两个儿子的结果，可求大小、最小值、最大值等。不必为聚合另跑 DFS；这利用的是建点时已经确定的父子拓扑顺序。断连图的各棵树独立，无须添加超级根。

归程建模 先用长度求每个点到 1 号点的最短步行距离。汽车只能走海拔严格大于水位的边，故选降序构建和 strict=true；可在分量内任意点下车，答案为该重构子树的叶点距离最小值。普通最小生成树的总权不能代替这个分量查询。

41.1.2 曼哈顿最小生成树

问题描述：给定平面上 n 个点，两点间边权为曼哈顿距离 $|x_i - x_j| + |y_i - y_j|$，求完全图的最小生成树（总权与边集）。

核心思路：每个点只需与它周围 8 个 45° 区域中各自最近的点连边，MST 必在这些候选边中，对称区域合并后共 4 个方向。`candidates` 每次处理一个方向：按 $x + y$ 升序扫描点 i，`sweep` 以 $-y$ 为键保存还没找到最近点的点；取出满足 $y_j \le y_i$ 且 $x_i - x_j \ge y_i - y_j$ 的点 j（即 i 落在 j 的该区域内），由扫描顺序它就是 j 在该区域里最近的点，连边后删去 j。处理完一个方向后交替做交换 x, y 与取负 x 的坐标变换，四次覆盖所有方向，最后对候选边跑 Kruskal。

代码依赖：dsu（第 264 页，28.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：Point 为 pair<long long,long long>，坐标支持完整有符号 64 位，点数须 ≤INT_MAX 且内存可分配。candidates(points) 给出最多 4n 条 (weight,u,v) 候选，允许重复边；solve(points) 返回精确 128 位 weight 与原下标边对 edges。空/单点返回 0 且无边，重合点仍为不同顶点；同权最优树不唯一。先转 128 位再变换/加减，输入不变。四方向扫描与 Kruskal 为 O(n log(n+1))、O(n) 空间；调用处选较大 dsu 根在前，不改变普通 dsu 语义。候选保证只针对无约束完整 L1 图，不能直接删禁边后声称仍含受限 MST。

```
struct ManhattanMST
{
    using I = __int128;
    using Point = pair<long long, long long>;
    using Edge = tuple<I, int, int>;
    struct Result
    {
        I weight = 0;
        vector<pair<int, int>> edges;
    };

    static vector<Edge> candidates(const vector<Point> &points)
    {
        assert(points.size() <= INT_MAX);
        int n = int(points.size());
        vector<pair<I, I>> p(points.begin(), points.end());
        vector<int> id(n);
        iota(id.begin(), id.end(), 0);
        vector<Edge> e;
        e.reserve(size_t(n) * 4);
        for (int dir = 0; dir < 4; dir++)
        {
            sort(id.begin(), id.end(), [&](int i, int j)
            {
                return pair{p[i].first + p[i].second, i} <
                       pair{p[j].first + p[j].second, j};
            });
```

活动点扫描与 Kruskal（接上页同一结构体，仍在四轮循环中）：

```
            map<I, int> sweep;
            for (int i : id)
            {
                auto it = sweep.lower_bound(-p[i].second);
                while (it != sweep.end())
                {
                    int j = it->second;
                    I dx = p[i].first - p[j].first;
                    I dy = p[i].second - p[j].second;
                    if (dx < dy) break;
                    e.emplace_back(dx + dy, i, j);
                    it = sweep.erase(it);
                }
                sweep[-p[i].second] = i;
            }
            for (auto &[x, y] : p)
                if (dir & 1) x = -x;
                else swap(x, y);
        }
        return e;
    }

    static Result solve(const vector<Point> &points)
    {
        auto e = candidates(points);
        int n = int(points.size());
        Result r;
        if (n < 2) return r;
        sort(e.begin(), e.end());
        dsu uf(n);
        for (auto [w, u, v] : e)
        {
            int a = uf.find(u), b = uf.find(v);
            if (a == b) continue;
            if (uf.size(a) < uf.size(b)) swap(a, b);
            uf.merge(a, b);
            r.weight += w;
            r.edges.emplace_back(u, v);
            if (r.edges.size() == points.size() - 1) break;
        }
        assert(r.edges.size() == points.size() - 1);
        return r;
    }
};
```

模板题使用：Library Checker Manhattan MST

题意：1≤N≤200000，N 个点的坐标 0..10^9，可重合且每个输入位置仍是不同顶点。完整图边权为曼哈顿距离；输出最小总权及 N−1 条 0-based 原编号边，任意最优树均可。结果 weight 为 128 位；本题粗上界 2e9 乘以 N−1，严格小于 4e14，所以此驱动才可转 long long 打印。N=1 输出 0、不输出边；库的空输入和有符号 64 位坐标另属 API 扩展。

所需模板：dsu（第 264 页，28.1.1 节）、ManhattanMST（第 448 页，41.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/manhattanmst

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<ManhattanMST::Point> p(n);
    for (auto &[x, y] : p) cin >> x >> y;
    auto r = ManhattanMST::solve(p);
    cout << (long long)r.weight << '\n';
    for (auto [u, v] : r.edges) cout << u << ' ' << v << '\n';
}
```

41.1.3 严格与非严格次小生成树、换边方案

问题描述：给定带权无向图，求最小生成树，以及次小生成树的权值：严格模式要求权值严格大于 MST，非严格模式只要求边集不同。同时给出由 MST 换一条边得到次小树的方案（换入边 `in`、换出边 `out`）。

核心思路：次小生成树总能由 MST 加入一条非树边 (u, v, w)、删去树上 u 到 v 路径上的一条边得到。先 Kruskal 求出 MST 并建树，倍增表 `mx[j][u]` 保存向上 2^j 条边中权值互不相同的最大与次大边编号（`merge`）。对每条非树、非自环边，查询路径上的这两条边：非严格时删最大边；严格时若最大边与 w 等权就改删次大边，不存在则跳过。在所有候选中取 $\mathrm{MST} + w - w_{\mathrm{out}}$ 的最小值。

代码依赖：dsu（第 264 页，28.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：SecondMST(n)，n≥1，点号 0..n-1。add(u,v,w) 返回 0-based 边号；允许自环、重边、完整 signed64 位权。solve(strict=true) 返回 connected、128 位 MST 总权 weight、所选边号 tree，以及可选 128 位次小权 next 和换入/换出边号 in/out。strict=true 要求权值严格增大；false 要求边集与返回的 MST 不同，可同权。不连通返回默认空结果；无次小树时 next 为空且 in/out=-1。存在时将 tree 中的 out 删去并加入 in 即得方案。可加边后重算；每次 solve 重建。依赖 dsu，沿用路径减半及 y 根挂 x 根；不用按秩合并。时间 O((n+m)log(n+m))，空间 O(m+n log n)，DFS 递归深度 O(n)。merge/dfs/path 与工作数组为内部实现；原始 e 只读。

```
struct SecondMST
{
    using I = __int128_t;
    using Pair = array<int, 2>;

    struct Edge
    {
        int u, v;
        long long w;
    };

    struct Result
    {
        bool connected = false;
        I weight = 0;
        vector<int> tree;
        optional<I> next;
        int in = -1, out = -1;
    };

    int n, h;
    vector<Edge> e;
    vector<vector<pair<int, int>>> g;
    vector<int> dep;
    vector<vector<int>> up;
    vector<vector<Pair>> mx;

    SecondMST(int n) : n(n), h(bit_width((unsigned)n)) { assert(n > 0); }

    int add(int u, int v, long long w)
    {
        assert(0 <= u && u < n && 0 <= v && v < n);
        e.push_back({u, v, w});
        return (int)e.size() - 1;
    }
```

不同权摘要与递归预处理（接上页同一结构体）：

```
    // Keep edge IDs of the two largest DISTINCT weights; -1 means absent.
    Pair merge(Pair a, Pair b) const
    {
        for (int x : b)
        {
            if (x == -1) continue;
            if (a[0] == -1 || e[x].w > e[a[0]].w)
            {
                a[1] = a[0];
                a[0] = x;
            }
            else if (e[x].w != e[a[0]].w && (a[1] == -1 || e[x].w > e[a[1]].w))
                a[1] = x;
        }
        return a;
    }

    void dfs(int u, int p)
    {
        for (int j = 1; j < h; j++)
        {
            int v = up[j - 1][u];
            up[j][u] = up[j - 1][v];
            mx[j][u] = merge(mx[j - 1][u], mx[j - 1][v]);
        }
        for (auto [v, id] : g[u])
            if (v != p)
            {
                dep[v] = dep[u] + 1;
                up[0][v] = u;
                mx[0][v] = {id, -1};
                dfs(v, u);
            }
    }
```

路径查询（接上页同一结构体）：

```
    Pair path(int u, int v) const
    {
        Pair ans{-1, -1};
        if (dep[u] < dep[v]) swap(u, v);
        for (int j = h - 1; j >= 0; j--)
            if ((dep[u] - dep[v]) >> j & 1)
            {
                ans = merge(ans, mx[j][u]);
                u = up[j][u];
            }
        if (u == v) return ans;
        for (int j = h - 1; j >= 0; j--)
            if (up[j][u] != up[j][v])
            {
                ans = merge(ans, merge(mx[j][u], mx[j][v]));
                u = up[j][u];
                v = up[j][v];
            }
        return merge(ans, merge(mx[0][u], mx[0][v]));
    }
```

Kruskal 与逐边换入（接上页同一结构体）：

```
    Result solve(bool strict = true)
    {
        vector<int> ids(e.size());
        iota(ids.begin(), ids.end(), 0);
        sort(ids.begin(),
             ids.end(),
             [&](int a, int b) { return tie(e[a].w, a) < tie(e[b].w, b); });
        dsu ds(n);
        vector<bool> used(e.size());
        g.assign(n, {});
        Result ans;
        for (int id : ids)
        {
            auto [u, v, w] = e[id];
            if (!ds.merge(u, v)) continue;
            used[id] = true;
            ans.weight += (I)w;
            ans.tree.push_back(id);
            g[u].push_back({v, id});
            g[v].push_back({u, id});
        }
        if ((int)ans.tree.size() != n - 1) return {};
        ans.connected = true;
        dep.assign(n, 0);
        up.assign(h, vector<int>(n));
        mx.assign(h, vector<Pair>(n, Pair{-1, -1}));
        dfs(0, -1);
        for (int id : ids)
        {
            auto [u, v, w] = e[id];
            if (used[id] || u == v) continue;
            auto p = path(u, v);
            int out = p[0];
            if (strict && e[out].w == w) out = p[1];
            if (out == -1) continue;
            I cost = ans.weight + (I)w - (I)e[out].w;
            if (!ans.next || cost < *ans.next)
            {
                ans.next = cost;
                ans.in = id;
                ans.out = out;
            }
        }
        return ans;
    }
};
```

41.1.4 Kruskal 最小生成树与生成森林

问题描述：给定无向带权图，求最小生成树；图不连通时求最小生成森林（每个连通块一棵最小生成树），给出总权、所选边号和连通块数。

核心思路：把所有边按权值升序稳定排序（同权按编号），依次考察：若两端点在并查集中不连通，就选这条边并合并，否则跳过（会成环）。每选一条边连通块数减一，因此最终 `cnt` 就是连通块数，为 1 表示整图连通、得到的是最小生成树。

代码依赖：dsu（第 264 页，28.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：Kruskal(n)，0≤n≤INT_MAX，点号 0..n-1；add(u,v,w) 加入无向边并返回 0-based 原边号，边权为完整 signed64，支持重边、自环和负权，边数 ≤INT_MAX。run() 每次从全部原边重建最小生成森林，返回连通块数；n>0 时返回 1 表示存在 MST，空图返回 0。weight 为 int128 总权，ids 为所选原边号，按 (权值, 原编号) 排序；e 保持插入顺序。加边后必须重新 run，旧结果不代表新图。支持重复运行和独立值复制。

依赖普通 dsu，调用方先把较大根放在前，保留 dsu 的 fa[y]=x 语义。时间 O(n+m log(m+1)+m α(n+1))，空间 O(n+m)；add 均摊 O(1)。总权绝对值 <2^94，不能无条件转 long long。自环不入树，断开图仍返回各块最小树；不使用 −1 总权判断失败，合法 MST 总权也可为 −1。成员只读，n 和 e 不直接修改。

```
struct Kruskal
{
    struct Edge
    {
        int u, v;
        long long w;
    };
    int n;
    vector<Edge> e;
    vector<int> ids;
    long long weight = 0;

    Kruskal(int n) : n(n) {}

    int add(int u, int v, long long w)
    {
        e.push_back({u, v, w});
        return int(e.size()) - 1;
    }

    // Minimum spanning forest on 0..n-1; ties by edge id. Returns the component count.
    int run()
    {
        vector<int> order(e.size());
        iota(order.begin(), order.end(), 0);
        stable_sort(order.begin(), order.end(), [&](int a, int b) { return e[a].w < e[b].w; });
        dsu d(n);
        ids.clear();
        weight = 0;
        int cnt = n;
        for (int id : order)
```

```
32              if (d.merge(e[id].u, e[id].v))
33              {
34                  ids.push_back(id);
35                  weight += e[id].w;
36                  cnt--;
37              }
38          return cnt;
39      }
40  };
```

模板题使用：Luogu P3366 / 最小生成树

题意：无向图求最小生成树总权，不连通输出 orz；n≤5000、m≤200000、边权 1..10000，允许自环重边。输入 1-based 点号减一，run()!=1 表示断连；本题总权可安全转 long long。

所需模板：dsu（第 264 页，28.1.1 节）、Kruskal（第 455 页，41.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3366

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    Kruskal g(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        g.add(u - 1, v - 1, w);
    }
    if (g.run() != 1)
    {
        cout << "orz\n";
    }
    else
    {
        cout << (long long)g.weight << '\n';
    }
}
```

41.1.5 稠密图 Prim 最小生成森林

问题描述：给定稠密无向带权图（邻接矩阵），求最小生成树或最小生成森林的总权，以及每个点在树中的父点。

核心思路：`d[v]` 是未选点 v 到已选点集的最轻边权，`pre[v]` 为该边另一端。共 n 步，每步线性扫描选出 `d` 最小的未选点 u：若 $d[u]=\infty$，说明剩余点与已选点集不连通，u 作为新树的根，连通块数加一；否则把 $d[u]$ 计入总权。然后用 u 的邻接矩阵行更新其余未选点的 `d` 和 `pre`。

接口契约：Prim(n)，0≤n≤INT_MAX，点号 0..n-1；add(u,v,w) 自动加无向边，允许完整 signed64 权、零权、负权、自环，重边只保留最小权。不保存原边编号；cost 为 int128 矩阵，inf=2^126 表示缺边，−1 和 LLONG_MAX 均为合法边权。

run() 每次重建最小生成森林，返回原图连通块数。n>0 时返回 1 表示 MST，空图返回 0。weight 为 int128 总权；pre[v] 为父点，根为 −1，非根选边权为 cost[pre[v]][v]。父点森林无环，支持断连和孤立点；同权任选，不承诺字典序。加边后须重新 run，支持重复运行与独立复制；成员只读。

构造和存储 $O(n^2)$，add 为 $O(1)$，run 为 $O(n^2)$，矩阵约 $16n^2$ 字节，按实际内存选择 n；稀疏图优先 Kruskal。每步选到已选点集的最轻边；所有剩余点均无跨边时开启新树，不把 inf 计入总权。只向未选点严格更新父点，负自环不入树。总权绝对值 <2^94，可为 −1 或超出 long long，不能用权值判断断连。

```
struct Prim
{
    static constexpr long long inf = LLONG_MAX / 4;
    int n;
    vector<vector<long long>> cost;
    vector<int> pre;
    long long weight = 0;

    Prim(int n) : n(n), cost(n, vector<long long>(n, inf)) {}

    void add(int u, int v, long long w) { cost[u][v] = cost[v][u] = min(cost[u][v], w); }

    // O(n^2) minimum spanning forest on 0..n-1; returns the component count.
    int run()
    {
        vector<long long> d(n, inf);
        vector<bool> vis(n);
        pre.assign(n, -1);
        weight = 0;
        int cnt = 0;
        for (int step = 0; step < n; step++)
        {
            int u = -1;
            for (int v = 0; v < n; v++)
                if (!vis[v] && (u == -1 || d[v] < d[u])) u = v;
            if (d[u] == inf)
                cnt++;
            else
                weight += d[u];
            vis[u] = true;
            for (int v = 0; v < n; v++)
                if (!vis[v] && cost[u][v] < d[v]) d[v] = cost[u][v], pre[v] = u;
        }
        return cnt;
    }
};
```

模板题使用：AOJ ALDS1_12_A / Minimum Spanning Tree

题意：输入 n 及 n×n 对称邻接矩阵，−1 表示缺边，其余为 0..2000；n≤100 且保证连通。只读取上三角的有效边，输出 MST 总权；本题可安全转 long long。注意输入 −1 缺边是题目约定，Prim.add 的 −1 是合法负权。

所需模板：Prim（第 458 页，41.1.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=ALDS1_12_A&lang=en

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    Prim g(n);
    for (int u = 0; u < n; u++)
        for (int v = 0; v < n; v++)
        {
            int w;
            cin >> w;
            if (u < v && w != -1) g.add(u, v, w);
        }
    g.run();
    cout << (long long)g.weight << '\n';
}
```

41.1.6 曼哈顿最小生成树：闭锥扫描与平局割证明

接口见第 41.1.2 节（第 448 页）。边权 $d(p,q)=|p_x-q_x|+|p_y-q_y|$。每点只保留四个向右闭锥中的最近邻，再对候选边 Kruskal；候选图含至少一棵 MST，但不含所有最优边，禁边等额外约束不能套用。

单方向锥 $q_x-p_x\ge q_y-p_y\ge 0$：按 $(x+y,$原编号$)$ 排序，活动点以 $-y$ 为 map 键，首个满足条件者即最近邻，找到后删除。四轮坐标 $(x,y),(y,x),(-y,x),(x,-y)$。总时间 $O(n\log n)$，空间 $O(n)$。重复坐标是不同顶点，按编号生成零权链。

输入为 signed 64 位，先转 signed 128 位再做和差；单边最大 $2^{65}-2$。返回 0 基编号与 128 位总权；空集与单点返回 0 和空边集。

41.1.7 严格次小生成树与换边

`SecondMST` 先 Kruskal 求 MST，倍增维护树路径上两个不同的最大边权及边号。每条非树非自环边换入：非严格删路径最大边；严格且换入边等于最大值时删第二大不同权边，无则跳过。`tree` 删 `out`、加 `in` 得 `next`；无次小树时 `next` 为空，断图时 `connected=false`。总权用 int128，边号 -1 表示缺项。

41.2 最小树形图

41.2.1 有向最小树形图

问题描述：最小树形图：给定有向带权图和根 r，选出一些边，使从 r 出发能到达所有点且每个非根点恰有一条入边，求最小总权；有点不可达时无解。

核心思路：朱–刘算法。每轮为每个非根点选最小入边（忽略自环），先把这些边权全部计入答案；若某点没有入边则无解。沿 `pre` 从每个点往回走，若回到本轮已走过的点就找到一个环，给环上点分配同一编号 `id`；没有环时所选入边就是最优树形图。有环则把每个环缩成一个点，其余点各占一个新编号，每条边的权减去其原终点所选入边的权（选它进入环就要替换掉环上原来的那条入边），然后在缩小的图上重复。

思路中的情况	代码中的处理	行
每个非根点选最小入边	记录入边权 `in` 与起点 `pre`，自环不参与	21
计入答案或判无解	某非根点无入边则返回 `nullopt`，否则累加全部 `in`	29
沿入边回溯找环	用 `vis` 标记本次出发点，遇到已标记的点即在环上	36
无环	所选入边构成树形图，直接返回累计答案	48
缩环并修改边权	端点换成新编号，跨分量边减去原终点的 `in`	56

接口契约：solve(n,root,edges)，点号 0..n-1，n≥1、0≤root<n；返回以 root 为根向外可达的最小树形图费用，无解 nullopt。允许负权、重边和自环（自环不选），n=1 的费用为 0；费用 -1 与无解可区分。输入、缩环后边权及累计费用绝对值须小于 2^120，类型为 int128。每个非根点选最小入边；若成环则缩点，跨环入边减去原终点已选入边费用，以计算替换代价。O(VE)，edges 按值传入，原图保持不变。仅返回费用，不恢复边方案。双版本已做所有根的入边枚举与 800 点连续缩环核验，暂无专门 OJ AC。

```cpp
struct Arborescence
{
    using I = __int128_t;

    struct Edge
    {
        int u, v;
        I w;
    };

    // Directed minimum rooted OUT-arborescence cost, 0-based, or nullopt.
    static optional<I> solve(int n, int root, vector<Edge> e)
    {
        const I inf = I(1) << 120;
        I ans = 0;
        while (true)
        {
            vector<I> in(n, inf);
            vector<int> pre(n, -1), id(n, -1), vis(n, -1);
            for (auto a : e)
                if (a.u != a.v && a.w < in[a.v])
                {
                    in[a.v] = a.w;
                    pre[a.v] = a.u;
                }
            in[root] = 0;
            for (I x : in)
            {
                if (x == inf) return nullopt;
                ans += x;
            }
```

判环与缩点（接上页同一函数）：

```
            int cnt = 0;
            for (int i = 0; i < n; i++)
            {
                int u = i;
                while (vis[u] != i && id[u] == -1 && u != root)
                {
                    vis[u] = i;
                    u = pre[u];
                }
                if (u != root && id[u] == -1)
                {
                    id[u] = cnt;
                    for (int v = pre[u]; v != u; v = pre[v]) id[v] = cnt;
                    ++cnt;
                }
            }
            if (!cnt) return ans;
            for (int i = 0; i < n; i++)
                if (id[i] == -1) id[i] = cnt++;
            for (auto &a : e)
            {
                int v = a.v;
                a.u = id[a.u];
                a.v = id[a.v];
                if (a.u != a.v) a.w -= in[v];
            }
            root = id[root];
            n = cnt;
        }
    }
};
```

模板题使用：Luogu P4716

题意：给定根的有向最小树形图费用，无解输出-1。n 至 100、m 至 10^4、正权至 10^6；顶点和根从 1-based 转 0-based。optional 有值时才解引用，本题总费用可转 long long；核心只返回费用，不恢复选边。

所需模板：Arborescence（第 461 页，41.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4716

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, r;
    cin >> n >> m >> r;
    vector<Arborescence::Edge> e;
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        e.push_back({u - 1, v - 1, w});
    }
    auto ans = Arborescence::solve(n, r - 1, e);
    // This problem's answer fits in long long.
    cout << (ans ? (long long)*ans : -1) << '\n';
}
```

第四十二章　斯坦纳树

42.1　最小斯坦纳树与原边方案

问题描述：给定非负权无向图和若干关键点（终端），求连通全部关键点的边权和最小的边集（最小斯坦纳树），并给出所选原边编号。

核心思路：状态 $dp[S][v]$ 为连通终端子集 S、且包含点 v（视为根）的树的最小代价，初值为单个终端在自身处代价 0。按 `mask` 从小到大处理：先在同一点 v 合并 S 的两个互补非空子集 T 与 $S\setminus T$（每对只枚举一次）；再以全部 $dp[S][\cdot]$ 为初值做多源 Dijkstra，让根沿边移动。转移来源分别记在 `split`（合并用的子集）与 `last`（最后走的边）。答案为 $\min_v dp[\text{全集}][v]$，从该根递归回溯标记用到的边，`seen` 防止重复展开同一状态。

思路中的情况	代码中的处理	行
子集合并	枚举真子集 `sub`，只处理 `sub` 不大于补集的一半，逐点相加更新并记 `split`	52
移动根	多源 Dijkstra 沿边松弛同一 `mask` 层，记下最后一条边 `last`	79
选最优根	全集层取最小值，等于 `inf` 说明终端不连通	85
回溯合并状态	拆成两个子集分别在同一点继续回溯	96
回溯移动状态	标记边 `id`，走到边的另一端继续回溯	102

接口契约：SteinerTree(n)，点号 0..n-1；add(u,v,w) 添加无向边并返回 0-based 原边编号，允许重边、自环、零权。n≥1，w≥0，所有边权总和严格小于 LLONG_MAX/4。solve(terminals) 自动去重，返回 {cost,edges}，edges 为不重复、升序原边编号；空终端返回 0 及空方案，无法连通返回-1 及空方案。输入图不修改，可重复求解；恢复最优连通子图，不额外保证零权环树化，递归回溯。

令 k 为去重终端数。dp[mask][v] 先枚举互补非空子集在 v 合并，再多源 Dijkstra 移动根。时间 O(3^k n+2^k(m+n)log n)，空间 O(2^k n+n+m)。位掩码约束 k≤20，不代表此规模必然能在时限内完成；P6192 为 n≤100、m≤500、k≤10。负权不适用。当前已通过独立边子集枚举和方案校验，P6192 已 100 分通过，排名未核验。

正确性要点　非负权最优连通子图可删环成树。以 v 为根，分叉处将关键点分成两个非空集合，在同根合并；单分支路径由最短路扩展得到。真子集先处理，逐 mask 闭包即可。回溯合并严格缩小 mask，同 mask 前驱遵循 Dijkstra 的确定次序；严格改善避免零权前驱环。原边去重后的连通方案费用不大于 dp，最优性又排除严格更小，故边权和恰等于 cost。

图存储与加边：

```
struct SteinerTree
{
    using ll = long long;
    static constexpr ll inf = LLONG_MAX / 4;
    struct Edge
    {
        int u, v;
        ll w;
    };
    struct Result
    {
        ll cost;
        vector<int> edges;
    };
    int n;
    ll total = 0;
    vector<Edge> edges;
    vector<vector<pair<int, int>>> g;

    explicit SteinerTree(int n) : n(n), g(n)
    {
        assert(n >= 1);
    }
    int add(int u, int v, ll w)
    {
        assert(0 <= u && u < n && 0 <= v && v < n);
        assert(0 <= w && w < inf - total);
        int id = edges.size();
        edges.push_back({u, v, w});
        g[u].push_back({v, id});
        g[v].push_back({u, id});
        total += w;
        return id;
    }
```

状态初始化与同根合并（接上页结构体）：

```
    Result solve(vector<int> terminals) const
    {
        sort(terminals.begin(), terminals.end());
        terminals.erase(unique(terminals.begin(), terminals.end()), terminals.end());
        for (int v : terminals) assert(0 <= v && v < n);
        int k = terminals.size();
        assert(k <= 20);
        if (!k) return {0, {}};
        int count = 1 << k;
        vector<vector<ll>> dp(count, vector<ll>(n, inf));
        vector<vector<int>> split(count, vector<int>(n));
        vector<vector<int>> last(count, vector<int>(n, -1));
        for (int i = 0; i < k; i++) dp[1 << i][terminals[i]] = 0;
        for (int mask = 1; mask < count; mask++)
        {
            for (int sub = (mask - 1) & mask; sub; sub = (sub - 1) & mask)
            {
                int other = mask ^ sub;
                if (sub > other) continue;
                for (int v = 0; v < n; v++)
                {
                    ll value = dp[sub][v] + dp[other][v];
                    if (value < dp[mask][v])
                    {
                        dp[mask][v] = value;
                        split[mask][v] = sub;
                    }
                }
            }
```

多源最短路（接上页 mask 循环）：

```
            priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> pq;
            for (int v = 0; v < n; v++)
                if (dp[mask][v] < inf) pq.push({dp[mask][v], v});
            while (!pq.empty())
            {
                auto [distance, u] = pq.top();
                pq.pop();
                if (distance != dp[mask][u]) continue;
                for (auto [v, id] : g[u])
                {
                    ll value = distance + edges[id].w;
                    if (value < dp[mask][v])
                    {
                        dp[mask][v] = value;
                        split[mask][v] = 0;
                        last[mask][v] = id;
                        pq.push({value, v});
                    }
                }
            }
        }
```

最优根与递归方案恢复（接上页 solve 函数）：

```
        int root = min_element(dp.back().begin(), dp.back().end()) - dp.back().begin();
        if (dp.back()[root] == inf) return {-1, {}};
        vector<vector<char>> seen(count, vector<char>(n));
        vector<char> used(edges.size());
        auto trace = [&](auto &&self, int mask, int u) -> void
        {
            if (seen[mask][u]) return;
            seen[mask][u] = 1;
            int sub = split[mask][u];
            if (sub)
            {
                self(self, sub, u);
                self(self, mask ^ sub, u);
            }
            else if (last[mask][u] != -1)
            {
                int id = last[mask][u];
                used[id] = 1;
                self(self, mask, edges[id].u ^ edges[id].v ^ u);
            }
        };
        trace(trace, count - 1, root);
        Result result{dp.back()[root], {}};
        for (int id = 0; id < (int)edges.size(); id++)
            if (used[id]) result.edges.push_back(id);
        return result;
    }
};
```

模板题使用：P6192 最小斯坦纳树

题意：输入无向正权图及关键点，输出连接所有关键点的最小边权和。方案边编号可直接读取 solve 返回值的 edges。

所需模板：SteinerTree（第 465 页，42.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6192

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, k;
    cin >> n >> m >> k;
    SteinerTree tree(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        tree.add(u - 1, v - 1, w);
    }
    vector<int> terminals(k);
    for (int &v : terminals)
    {
        cin >> v;
        v--;
    }
    cout << tree.solve(terminals).cost << '\n';
}
```

第四十三章　连通性相关

43.1　强连通分量

相关应用：函数图包含尾链与环分解，导航无独立页；需正文定位，不应等同一般 SCC 算法。

43.1.1　函数图分解、跳转与到达

功能：给定每个点恰有一条出边的函数图（若干基环内向树），分解出所有环，支持查询从 u 出发走 k 步到达的点（k 可达 64 位），以及从 u 首次到达 v 的步数。

核心思路：先按入度剥叶得到 `order`，剩下入度非零的点都在环上，沿后继走一圈记下每个环 `cycles`、环编号 `component` 与环上位置 `position`。再逆序遍历 `order`，由后继推出树点的入环点 `entry` 与入环步数 `depth`。走 k 步时，若 k 不小于入环步数，先走到入环点，剩余步数对环长取模直接查环；否则 $k < n$，用倍增表 `up` 跳。`steps(u,v)`：v 不在环上时，检查 u 走深度差步是否恰好到 v；v 在环上时，答案为入环步数加上环上从 `entry[u]` 到 v 的距离。

接口契约：FunctionalGraph(to) 或 build(to) 重建，每个点恰有一条出边，to[u] 为后继，点号 0..n-1；空图允许，但不能查询。advance(u,k) 返回第 k 步到达的点，k 为完整 unsigned long long；steps(u,v) 返回首次到达 v 的最少步数，无法到达返回 -1，自身为 0。component[u] 为所在环的编号；cycles[id] 按后继方向保存环；entry[u] 为首次入环点；depth[u] 为入环步数；position 仅在环点上有效，非环点为 -1。order 为入度剥离顺序，逆序可用于从后继向入环树传播状态；up 是倍增表。所有字段只读。O(n log n) 预处理及空间，入环前查询 O(log n)，入环后取模 O(1)，不为 64 位步数分配 64 层表。默认构造为空图，支持自环、多个分量及重建。n≤INT_MAX，实际规模须满足内存。

```
struct FunctionalGraph
{
    using U = unsigned long long;
    int n;
    vector<int> component, position, entry, depth, order;
    vector<vector<int>> cycles, up;

    explicit FunctionalGraph(vector<int> to = {}) { build(move(to)); }

    void build(vector<int> to)
    {
        assert(to.size() <= INT_MAX);
        n = to.size();
        vector<int> degree(n);
        for (int v : to)
        {
            assert(0 <= v && v < n);
            degree[v]++;
        }
        order.clear();
        for (int u = 0; u < n; u++)
            if (degree[u] == 0) order.push_back(u);
        for (int i = 0; i < int(order.size()); i++)
        {
            int v = to[order[i]];
            if (--degree[v] == 0) order.push_back(v);
        }
```

环、入环树与倍增表（接上页同一结构体与函数）：

```
        component.assign(n, -1);
        position.assign(n, -1);
        entry.resize(n);
        depth.assign(n, 0);
        cycles.clear();
        for (int u = 0; u < n; u++)
            if (degree[u] && component[u] == -1)
            {
                vector<int> cycle;
                int v = u, id = cycles.size();
                do
                {
                    component[v] = id;
                    position[v] = cycle.size();
                    entry[v] = v;
                    cycle.push_back(v);
                    v = to[v];
                } while (v != u);
                cycles.push_back(move(cycle));
            }
        for (int i = int(order.size()) - 1; i >= 0; i--)
        {
            int u = order[i], v = to[u];
            component[u] = component[v];
            entry[u] = entry[v];
            depth[u] = depth[v] + 1;
        }
        up.assign(1, to);
        for (int k = 1; (1ULL << k) <= U(n); k++)
        {
            vector<int> row(n);
            for (int u = 0; u < n; u++) row[u] = up[k - 1][up[k - 1][u]];
            up.push_back(move(row));
        }
    }
```

跳转与最少步数接口（接上页同一结构体）：

```
    int advance(int u, U k) const
    {
        assert(0 <= u && u < n);
        if (k >= U(depth[u]))
        {
            k -= depth[u];
            u = entry[u];
            const auto &cycle = cycles[component[u]];
            return cycle[(U(position[u]) + k % cycle.size()) % cycle.size()];
        }
        for (int bit = 0; k; bit++, k >>= 1)
            if (k & 1) u = up[bit][u];
        return u;
    }

    int steps(int u, int v) const
    {
        assert(0 <= u && u < n && 0 <= v && v < n);
        if (component[u] != component[v] || depth[u] < depth[v]) return -1;
        if (depth[v] > 0)
        {
            int d = depth[u] - depth[v];
            return advance(u, d) == v ? d : -1;
        }
        int offset = position[v] - position[entry[u]];
        if (offset < 0) offset += int(cycles[component[u]].size());
        return depth[u] + offset;
    }
};
```

模板题使用：CSES 1750 Planets Queries I

题意：给每点唯一后继，查询从 x 走 k 步后的点，n,q≤200000，0≤k≤10^9。题目 1-based 点号读入减 1、输出加 1；k 用 unsigned long long，模板也支持完整 64 位步数。advance 先处理入环链再对环长取模，不能只建 30 层就宣称支持任意 64 位。预处理 O(n log n)，查询 O(log n)，入环后 O(1)。

所需模板：FunctionalGraph（第 471 页，43.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://cses.fi/problemset/task/1750

```
int main()
{
    int n, q;
    scanf("%d%d", &n, &q);
    vector<int> to(n);
    for (auto &v : to)
    {
        scanf("%d", &v);
        v--;
    }
    FunctionalGraph graph(to);
    while (q--)
    {
        int u;
        unsigned long long k;
        scanf("%d%llu", &u, &k);
        printf("%d\n", graph.advance(u - 1, k) + 1);
    }
}
```

43.1.2 Tarjan 强连通分量与缩点

问题描述：给定有向图，求每个点所属的强连通分量编号，并构造缩点后的 DAG（邻接表）。

核心思路：DFS 时记录时间戳 `dfn` 与 `low`（能回溯到的仍在栈中的最小时间戳），访问过的点压入栈 `st`。树边用子节点的 `low` 更新；指向已访问且尚未分配分量的点的边，用其 `dfn` 更新。回溯时若 $\mathrm{low}[u]=\mathrm{dfn}[u]$，$u$ 是分量的根，弹栈到 u 为止得到一个分量。分量按完成先后编号，先完成者无出边指向后完成者，因此跨分量边总从大编号指向小编号，编号即逆拓扑序。

接口契约：TarjanSCC(n)，点号 1..n；add(u,v) 加有向边，run() 从头计算。bel[u] 为 1..cnt 的强连通分量编号，跨分量边总从大编号指向小编号，可按 cnt..1 做拓扑 DP。dag() 返回缩点邻接表，保留重边；若 DP 需要对不同邻点去重，调用者应排序去重。支持自环、重边、不连通图和重复 run。时间及图存储 O(n+m)，递归 DFS 另需 O(n) 调用栈；本地验证应配置足够栈内存。注意已有 SCC 模板的编号方向与本模板相反。

```
struct TarjanSCC
{
    int n, timer = 0, cnt = 0;
    vector<vector<int>> g;
    vector<int> dfn, low, bel, st;

    TarjanSCC(int n) : n(n), g(n + 1) {}

    void add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        g[u].push_back(v);
    }
```

```
14
15      void dfs(int u)
16      {
17          dfn[u] = low[u] = ++timer;
18          st.push_back(u);
19          for (int v : g[u])
20          {
21              if (!dfn[v])
22              {
23                  dfs(v);
24                  low[u] = min(low[u], low[v]);
25              }
26              else if (!bel[v])
27                  low[u] = min(low[u], dfn[v]);
28          }
29          if (low[u] == dfn[u])
30          {
31              ++cnt;
32              while (true)
33              {
34                  int v = st.back();
35                  st.pop_back();
36                  bel[v] = cnt;
37                  if (v == u) break;
38              }
39          }
40      }
41
42      // IDs 1..cnt are in reverse topological order. Recursive DFS needs O(n) stack.
43      void run()
44      {
45          timer = cnt = 0;
46          dfn.assign(n + 1, 0);
47          low = dfn;
48          bel = dfn;
49          st.clear();
50          for (int u = 1; u <= n; u++)
51          {
52              if (!dfn[u]) dfs(u);
53          }
54      }
55
56      // Retains parallel edges. Use after run().
57      vector<vector<int>> dag() const
58      {
59          vector<vector<int>> result(cnt + 1);
60          for (int u = 1; u <= n; u++)
```

```
        {
            for (int v : g[u])
            {
                if (bel[u] != bel[v]) result[bel[u]].push_back(bel[v]);
            }
        }
        return result;
    }
};
```

模板题使用：Library Checker scc

题意：把有向图分成强连通分量，并按缩点图的拓扑顺序输出各分量。题目点号为 0..n-1，调用时加 1，输出时减 1。

所需模板：TarjanSCC（第 475 页，43.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/scc

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    TarjanSCC g(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        g.add(u + 1, v + 1);
    }
    g.run();
    vector<vector<int>> c(g.cnt + 1);
    for (int u = 1; u <= n; u++) c[g.bel[u]].push_back(u - 1);
    cout << g.cnt << '\n';
    for (int i = g.cnt; i >= 1; i--)
    {
        cout << c[i].size();
        for (int u : c[i]) cout << ' ' << u;
        cout << '\n';
    }
}
```

43.1.3 SCC 模板题与速度榜口径

QOJ906 要求按拓扑序输出强连通分量。第 475 页 `TarjanSCC` 的分量编号为逆拓扑序，按 cnt 到 1 输出；`bel[1..n]` 为 1-based，0-based 输入点号先加一、输出时减一。

43.2 双连通分量

43.2.1 点双与边双：共享 lowlink 核心

问题描述：对无向图（可不连通，可有重边、自环）一次求出点双连通分量（各块的顶点集合与割点）和边双连通分量（桥以及每个点所属的边双编号）。

核心思路：一次 Tarjan DFS 维护 `dfn` 与 `low`，按边编号而不是父点跳过父边，所以重边能正确地阻止成桥。树边 $u \to v$ 回溯后：若 $low_v > dfn_u$，该边是桥；若 $low_v \geq dfn_u$，u 是一个点双的最高点，从栈 `st` 弹出直到 v，连同 u 组成一个块，并且非根点、或有两个以上 DFS 儿子的根是割点。孤立点单独成块。之后用不经过桥的 `paint` 做连通染色，每个连通部分就是一个边双，编号存入 `bel`。

接口契约：BiconnectedCore(n)，点号 1..n；add(u,v) 返回原边的 0-based 编号，run() 从头计算，允许重复运行。

点双结果：blocks 为点双的顶点列表，桥两端成块，孤立点单独成块，割点可出现在多个块；cut[u] 标记割点。

边双结果：bridge[id] 标记桥，bel[u] 是 1..cnt 的边双编号；cnt 是全图边双个数。重边按边号处理；自环保留边号但不进 DFS，不是桥，不额外产生点双。一次共享 DFS 后递归染色，O(n+m) 时间和空间，DFS 保持递归。n=0 合法。run 后才可读取结果或构造森林；g、edges 为原图，不在 DFS 期间修改。

```
struct BiconnectedCore
{
    int n, timer = 0, cnt = 0;
    vector<vector<pair<int, int>>> g;
    vector<pair<int, int>> edges;
    vector<vector<int>> blocks;
    vector<int> dfn, low, cut, bridge, bel, st;

    BiconnectedCore(int n) : n(n), g(n + 1) {}

    int add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        int id = edges.size();
        edges.push_back({u, v});
        if (u != v)
        {
            g[u].push_back({v, id});
            g[v].push_back({u, id});
        }
        return id;
    }
```

```
    void dfs(int u, int parent_edge)
    {
        dfn[u] = low[u] = ++timer;
        st.push_back(u);
        int children = 0;
        for (auto [v, id] : g[u])
        {
            if (id == parent_edge) continue;
            if (!dfn[v])
            {
                children++;
                dfs(v, id);
                low[u] = min(low[u], low[v]);
                if (low[v] > dfn[u]) bridge[id] = 1;
                if (low[v] >= dfn[u])
                {
                    if (parent_edge != -1 || children > 1) cut[u] = 1;
                    blocks.push_back({u});
                    while (true)
                    {
                        int x = st.back();
                        st.pop_back();
                        blocks.back().push_back(x);
                        if (x == v) break;
                    }
                }
            }
            else
                low[u] = min(low[u], dfn[v]);
        }
        if (parent_edge == -1 && children == 0) blocks.push_back({u});
    }
```

```
    void paint(int u)
    {
        bel[u] = cnt;
        for (auto [v, id] : g[u])
        {
            if (!bridge[id] && !bel[v]) paint(v);
        }
    }
```

```
    void run()
    {
        timer = cnt = 0;
        dfn.assign(n + 1, 0);
        low = dfn;
        cut = dfn;
        bel = dfn;
        bridge.assign(edges.size(), 0);
        blocks.clear();
        st.clear();
        for (int u = 1; u <= n; u++)
        {
            if (!dfn[u])
            {
                dfs(u, -1);
                st.pop_back();
            }
        }
        for (int u = 1; u <= n; u++)
        {
            if (bel[u]) continue;
            ++cnt;
            paint(u);
        }
    }
};
```

模板题使用：Luogu P8435

题意：给定无向图，输出所有点双连通分量的顶点集合；割点可出现在多个集合中，孤立点单独成块。点号 1..n。

所需模板：BiconnectedCore（第 478 页，43.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P8435

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    BiconnectedCore graph(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        std::cin >> u >> v;
        graph.add(u, v);
    }
    graph.run();
    const auto &blocks = graph.blocks;
    std::cout << blocks.size() << '\n';
    for (const auto &block : blocks)
    {
        std::cout << block.size();
        for (int u : block) std::cout << ' ' << u;
        std::cout << '\n';
    }
}
```

43.2.2 边双缩点森林（保留桥编号）

功能：由已运行的 `BiconnectedCore` 构造边双缩点森林：每个边双连通分量缩成一个点，桥成为森林的边，邻接项附带原桥编号。

核心思路：遍历全部原边，只保留桥 `bridge[id]`，把两个端点换成所属边双编号 `bel`，在两个分量之间加一条双向边。删去桥后各边双互不相连，所以缩点后每个连通分量是一棵树。

代码依赖：BiconnectedCore（第 478 页，43.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：bridge_component_forest(graph) 接收已 run 的 BiconnectedCore，返回按 1..cnt 编号的边双森林，邻接项为 (相邻分量, 原桥编号)。每条桥变成一条森林边，重边和自环不会误作桥。0 号空置，不连通输入可产生多棵树，孤立边双保留空邻接表。O(n+m) 时间、O(n) 输出空间，不修改分解结果。

```
vector<vector<pair<int, int>>> bridge_component_forest(const BiconnectedCore &graph)
{
    int cnt = graph.cnt;
    const auto &edges = graph.edges;
    const auto &bridge = graph.bridge;
    const auto &bel = graph.bel;
    vector<vector<pair<int, int>>> tree(cnt + 1);
    for (int id = 0; id < (int)edges.size(); id++)
    {
        if (!bridge[id]) continue;
        auto [u, v] = edges[id];
        u = bel[u];
        v = bel[v];
        tree[u].push_back({v, id});
        tree[v].push_back({u, id});
    }
    return tree;
}
```

43.3 圆方树

43.3.1 静态仙人掌森林最短路

问题描述：给定边权非负的仙人掌森林（每条边至多属于一个简单环），多次询问两点间最短路的长度，不连通时返回 -1。

核心思路：建圆方树：DFS 中遇到返祖边 $u \to v$，就为这个环新建虚点 b，环长记为 `len[b]`；环的最高点 v 以权 0 连向 b，环上其余点 x 以到 v 的较短弧长连向 b，`pos[x]` 记它沿 DFS 树到 v 的距离。不在环上的树边原样保留。这样树上根距离 `dis` 就等于原图中到根的最短距离。询问用倍增求 LCA p：p 是原点时答案为 $dis_u + dis_v - 2dis_p$；p 是环虚点时，u, v 跳到的两个儿子 u', v' 在同一环上，答案为 $dis_u - dis_{u'} + dis_v - dis_{v'}$ 加上 u', v' 之间的较短弧。

思路中的情况	代码中的处理	行
普通树边	儿子 v 不属于以上方为顶的环时，直接作为带权树边	36
发现环	返祖边新建环虚点，环长为两端前缀距离差加该边权	40
环上点挂到虚点	沿 `par` 从 u 回走到 v，权取两条弧中较短者	47
LCA 是原点	根距离之和减去两倍 LCA 根距离	111
LCA 是环虚点	用两个儿子在环上的位置差取较短弧，替换经过虚点的部分	112

接口契约：Cactus(n)，0≤n<INT_MAX/2，原点 1..n。add(u,v,w) 加入无向边；输入须为无自环无重边的仙人掌森林（每条边最多属于一个简单环），不检查此结构前提。权非负，所有边权之和 ≤LLONG_MAX/4。build() 递归构造桥/环虚点森林并倍增；distance(u,v) 仅接受原点，断连返回-1，同点返回 0。查询前必须 build；加边后必

须重建，支持重复 build 和复制，无动态修改或路径方案接口。编号 n+1..cnt 为环虚点；t 是从根向下的带权邻接表，成员只读。环的顶点到最上方割点距离取较短弧，虚点到该割点权 0；LCA 是虚点时，须用两个儿子在环上的较短弧替换根距离贡献。dfs 与 prepare 均递归，环上沿 par 回走仅枚举边，不模拟 DFS 栈。原图 DFS 及环枚举 O(n+m)，总预处理/空间 O((n+m)log(n+1))，查询 O(log(n+1))，递归深度 O(n)。n=0 允许 build 但无合法查询；不支持负权、非法仙人掌或修改内部数组。

```
struct Cactus
{
    using ll = long long;
    int n, m = 0, cnt, timer = 0, lg;
    vector<vector<tuple<int, int, ll>>> g;
    vector<vector<pair<int, ll>>> t;
    vector<vector<int>> up;
    vector<int> dfn, par, bel, dep, root;
    vector<ll> pref, pos, len, dis;

    Cactus(int n) : n(n), g(n + 1)
    {
        assert(0 <= n && n < INT_MAX / 2);
    }

    void add(int u, int v, ll w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(u != v && w >= 0);
        g[u].push_back({v, m, w});
        g[v].push_back({u, m, w});
        m++;
    }
```

递归构造环虚点与根距离（接上页同一结构体）：

```
    void dfs(int u, int pe)
    {
        dfn[u] = ++timer;
        for (auto [v, id, w] : g[u])
        {
            if (id == pe) continue;
            if (!dfn[v])
            {
                par[v] = u;
                pref[v] = pref[u] + w;
                dfs(v, id);
                if (!bel[v]) t[u].push_back({v, w});
            }
            else if (dfn[v] < dfn[u])
            {
                int b = ++cnt;
                len[b] = pref[u] - pref[v] + w;
                t[v].push_back({b, 0});
                for (int x = u; x != v; x = par[x])
                {
                    bel[x] = b;
                    pos[x] = pref[x] - pref[v];
                    t[b].push_back({x, min(pos[x], len[b] - pos[x])});
                }
            }
        }
    }

    void prepare(int u)
    {
        for (int k = 1; k < lg; k++)
            up[k][u] = up[k - 1][up[k - 1][u]];
        for (auto [v, w] : t[u])
        {
            up[0][v] = u;
            dep[v] = dep[u] + 1;
            root[v] = root[u];
            dis[v] = dis[u] + w;
            prepare(v);
        }
    }
```

从零重建（接上页同一结构体）：

```
    void build()
    {
        cnt = n;
        timer = 0;
        int size = 2 * n + 1;
        t.assign(size, {});
        dfn.assign(size, 0);
        par = bel = dep = root = dfn;
        pref.assign(size, 0);
        pos = len = dis = pref;
        for (int u = 1; u <= n; u++)
            if (!dfn[u]) dfs(u, -1);
        lg = 1;
        while ((1LL << lg) <= cnt) lg++;
        up.assign(lg, vector<int>(size));
        for (int u = 1; u <= n; u++)
        {
            if (dfn[u] && !par[u])
            {
                root[u] = u;
                prepare(u);
            }
        }
    }
```

查询与环上距离修正（接上页同一结构体）：

```
    // Original vertices only; disconnected pairs return -1.
    ll distance(int u, int v) const
    {
        if (root[u] != root[v]) return -1;
        ll ans = dis[u] + dis[v];
        if (dep[u] < dep[v]) swap(u, v);
        int delta = dep[u] - dep[v];
        for (int k = 0; k < lg; k++)
            if (delta >> k & 1) u = up[k][u];
        if (u == v) return ans - 2 * dis[u];
        for (int k = lg - 1; k >= 0; k--)
        {
            if (up[k][u] != up[k][v])
            {
                u = up[k][u];
                v = up[k][v];
            }
        }
        int p = up[0][u];
        if (p <= n) return ans - 2 * dis[p];
        ll arc = abs(pos[u] - pos[v]);
        return ans - dis[u] - dis[v] + min(arc, len[p] - arc);
    }
};
```

为什么需要环上修正 桥和环块构成森林，块间路径唯一。若最近公共祖先是原点，两条根距离减去公共前缀即可；若是环虚点，u、v 现为该虚点的两个原点儿子，先扣掉这两点到根的距离，再加环上两段弧长的较小值。不能直接把虚点当普通 LCA，否则会强迫路径绕经环顶割点。

复杂度与前提 每条边只属于至多一个环，因此沿 par 回走的总次数为 O(m)，不是每个返祖边各走 O(n)。两次 DFS 均为递归。输入须为非负权简单仙人掌森林，全部边权和不超过 LLONG_MAX/4；不自动校验结构，非法图可能越界。加边后必须重建。

模板题使用：P5236 【模板】静态仙人掌

题意：正权无重边仙人掌，查询两点最短距离；n,q≤10000、m≤20000、w≤100000。

所需模板：Cactus（第 482 页，43.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5236

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, q;
    cin >> n >> m >> q;
    Cactus solver(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        solver.add(u, v, w);
    }
    solver.build();
    while (q--)
    {
        int u, v;
        cin >> u >> v;
        cout << solver.distance(u, v) << '\n';
    }
}
```

43.3.2 由点双构造圆方森林

功能：由已运行的 BiconnectedCore 的点双分解构造圆方树（森林）：原点为圆点，每个点双对应一个方点，返回邻接表。

核心思路：第 i 个点双的方点编号为 $n+i+1$，块内每个原点都与该方点连边。割点属于多个块，因此连向多个方点；原图两点间每条路径都必经的点，恰是它们在圆方树路径上的圆点。

代码依赖：BiconnectedCore（第 478 页，43.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：block_cut_forest(graph) 接收已经 run 的 BiconnectedCore，返回 1-based 邻接表。原点为 1..n，blocks[i] 的方点为 n+i+1；每个点双内的所有原点连向该方点。孤立点也连向自己的单点方块。不连通原图得到森林；0 号空置。O(n+m) 时间与输出空间，不修改分解结果。

```
vector<vector<int>> block_cut_forest(const BiconnectedCore &graph)
{
    int n = graph.n;
    const auto &blocks = graph.blocks;
    vector<vector<int>> tree(n + blocks.size() + 1);
    for (int i = 0; i < (int)blocks.size(); i++)
    {
```

```
        int b = n + i + 1;
        for (int u : blocks[i])
        {
            tree[u].push_back(b);
            tree[b].push_back(u);
        }
    }
    return tree;
}
```

第四十四章　环计数问题

44.1　简单无向图四元环计数

问题描述：给定简单无向图，统计四元环个数：四个不同点 a,b,c,d 满足 ab,bc,cd,da 都是边（不要求无弦）。例如 K_4 有 3 个四元环。

核心思路：按 (度数, 编号) 给所有点定一个全序，每个四元环只在其中最大的点 u 处计数。对每个 u，枚举二步路径 $u \to v \to w$，要求 v,w 都排在 u 之前；`cnt[w]` 记录已出现的从 u 到 w 的此类路径数，新路径与之前每条配对构成一个以 u,w 为对角的四元环，所以答案加 `cnt[w]`。处理完 u 后只清空 `touched` 中被访问过的 `cnt`。按度数定序保证总枚举量为 $O(m\sqrt{m})$。

接口契约：count_four_cycles(n,edges) 返回 long long 计数，n≥0、点 0..n-1、m=edges.size()≤INT_MAX。输入必须是简单无向图，每条边恰好给一次，不允许重边和自环；不自动去重。允许空图、孤立点、断连，输入不变，可重复调用。计四个不同点组成的四边环，不要求诱导子图无弦；K4 有 3 个四元环，旋转和反向不重复计算。按 (度数, 编号) 严格定序，以唯一最大点 u 为起点，统计 u-v-w 中 v,w 均小于 u 的二步路径；第二条路径与前面路径配对。每轮清空实际访问的 cnt。时间 O(n+m$\sqrt{\text{m}}$)，空间 O(n+m)，无需排序或递归；不可仅按编号定序后仍声称此复杂度。cnt 只需 int，答案必须 long long；在 m≤INT_MAX 约定下，环数 ≤m(m-1)/2<2^61，不溢出。

```
long long count_four_cycles(int n, const vector<pair<int, int>> &edges)
{
    assert(n >= 0 && edges.size() <= INT_MAX);
    vector<vector<int>> g(n);
    for (auto [u, v] : edges)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v);
        g[u].push_back(v);
        g[v].push_back(u);
    }
    auto less = [&](int u, int v)
    {
        return pair{g[u].size(), u} < pair{g[v].size(), v};
    };
    vector<int> cnt(n), touched;
    long long ans = 0;
    for (int u = 0; u < n; u++)
    {
        for (int v : g[u])
        {
            if (!less(v, u)) continue;
            for (int w : g[v])
            {
                if (!less(w, u)) continue;
```

```
                if (cnt[w] == 0) touched.push_back(w);
                ans += cnt[w];
                cnt[w]++;
            }
        }
        for (int w : touched) cnt[w] = 0;
        touched.clear();
    }
    return ans;
}
```

模板题使用：U367189 无向图四元环计数

题意：简单无向图，n≤100000，m≤200000；允许断连，不含重边和自环。输入 1-based 端点，输出非诱导四元环数。

所需模板：count_four_cycles（第 489 页，44.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/U367189

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<pair<int, int>> edges(m);
    for (auto &[u, v] : edges)
    {
        cin >> u >> v;
        u--;
        v--;
    }
    cout << count_four_cycles(n, edges) << '\n';
}
```

44.2 无向简单图三元环枚举

问题描述：给定简单无向图，对每个三元环（三个两两相邻的点）恰好调用一次回调 `visit(u,v,w)`，可用于计数、求权值和或收集方案。

核心思路：把每条边从（度数，编号）较小的端点指向较大的端点，得到无环有向图，每个三角形恰有一种 $u \to v \to w$ 且 $u \to w$ 的形态。枚举 u 时先把 u 的所有出邻点的 `mark` 设为 u，再枚举 $u \to v \to w$，若 w 带有标记就找到一个三角形。按度数定向保证每个点出度为 $O(\sqrt{m})$。

接口契约：enumerate_triangles(n,edges,visit) 使用 0..n-1 点号；edges 须为无向简单图，无自环、无重边，每条边只给一次，端点顺序任意。每个三角形恰调用一次 visit(u,v,w)，参数按度数和编号组成的次序排列，不保证编号递增。可在回调中计数、累计权值或收集方案；计数用 long long。按 (度数, 编号) 从小到大定向，标记 u 的出邻点后枚举 u 到 v 到 w 的有向路径。每个出度为 O($\sqrt{\text{m}}$)，总时间 O(n+m$\sqrt{\text{m}}$)，含常数时间回调，空间 O(n+m)；空图合法。调用期间不修改输入边集。

```
template <class F>
void enumerate_triangles(int n, const vector<pair<int, int>> &edges, F visit)
{
    assert(n >= 0);
    vector<int> deg(n), mark(n, -1);
    vector<vector<int>> g(n);
    for (auto [u, v] : edges)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v);
        deg[u]++, deg[v]++;
    }
    for (auto [u, v] : edges)
    {
        if (pair{deg[u], u} > pair{deg[v], v}) swap(u, v);
        g[u].push_back(v);
    }
    for (int u = 0; u < n; u++)
    {
        for (int v : g[u]) mark[v] = u;
        for (int v : g[u])
            for (int w : g[v])
                if (mark[w] == u) visit(u, v, w);
    }
}
```

模板题使用：Library Checker enumerate_triangles

题意：给简单无向图的每个点一个权值，求所有三角形三个点权乘积之和，模 998244353。点号 0..n-1。

所需模板：enumerate_triangles（第 491 页，44.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/enumerate_triangles

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<long long> x(n);
    for (auto &v : x) cin >> v;
    vector<pair<int, int>> edges(m);
    for (auto &[u, v] : edges) cin >> u >> v;
    const long long mod = 998244353;
    long long ans = 0;
    enumerate_triangles(n,
                        edges,
                        [&](int u, int v, int w)
                        { ans = (ans + x[u] * x[v] % mod * x[w]) % mod; });
    cout << ans << '\n';
}
```

第四十五章　最小环

相关应用：Karp 最小平均权有向环，目标不同于最小总权环；两遍 DP 线性空间。

45.1　有向图最小平均权环：线性空间 Karp

问题描述：给定带权有向图（权可负），求所有有向环中平均边权（环总权除以边数）的最小值，以未约分的分数形式返回；图中无环时返回空。

核心思路：Karp 定理：令 $F_k(v)$ 为恰好走 k 条边到达 v 的最短行走长度，起点任意（$F_0 \equiv 0$，相当于超级源），则最小平均权为 $\min_v \max_{0\le k<n} \frac{F_n(v)-F_k(v)}{n-k}$。为只用 $O(n)$ 空间，先用 `step` 滚动 n 轮求出 F_n 存入 `last`，再把 `d` 清零重新逐轮计算 $F_0,\dots,F_{n-1}$，同时对每个 v 维护分数最大值 `best[v]`，最后取各点中的最小者。分数比较用交叉相乘，整数权用 128 位保存。

接口契约：minimum_mean_cycle(n,edges)，0≤n<INT_MAX，点 0..n-1，edges 为 tuple<int,int,T> 有向边；支持断连、负权、自环和重边，输入不变。返回 optional<pair<W,int»，无环 nullopt；有环返回分子/正分母，未约分，不返回环方案。T 为有符号至多 64 位整数时 W=int128，全部比较精确，支持 LLONG_MIN/MAX；n<2^31 使交叉乘法绝对值 <2^126。实数 T 须支持值语义，W=T；禁止 NaN/Inf，要求 $2n^2\max|w|<2^{118}$ 避免溢出，仍有舍入误差，不能泛称精确。P3199 驱动用 GCC ___float128 进行 113 位核心运算，long double 仅用于输入输出；需 GCC 支持此扩展。F0 全部为 0，Fk 为恰好 k 条边的最短行走；先滚动到 Fn，再重算 F0..Fn-1，取 min_v max_k (Fn(v)-Fk(v))/(n-k)。时间 O(n(n+m))，额外 O(n)，输入 O(m)；非强连通也成立，无需 DFS、二分或负环启发式。k 为 0..n-1，不能只比较最后一层或允许至多 k 条边。

```
template <class T>
auto minimum_mean_cycle(int n, const vector<tuple<int, int, T>> &edges)
{
    static_assert(!is_integral_v<T> || (is_signed_v<T> && sizeof(T) <= 8));
    assert(0 <= n && n < INT_MAX);
    using W = conditional_t<is_integral_v<T>, __int128_t, T>;
    using Ratio = pair<W, int>;
    const W inf = W(1LL << 60) * W(1LL << 60);
    vector<W> d(n), next(n);
    auto step = [&]()
    {
        fill(next.begin(), next.end(), inf);
        for (auto [u, v, w] : edges)
        {
            if (d[u] == inf) continue;
            W value = d[u] + w;
            if (value < next[v]) next[v] = value;
        }
        d.swap(next);
    };
    for (int k = 0; k < n; k++) step();
    auto last = d;
    fill(d.begin(), d.end(), 0);
    auto less = [&](const Ratio &a, const Ratio &b)
    {
        return a.first * b.second < b.first * a.second;
    };
```

第二遍重算并取 min-max（接上页同一函数）：

```
    vector<Ratio> best(n, {0, 0});
    for (int k = 0; k < n; k++)
    {
        for (int v = 0; v < n; v++)
        {
            if (last[v] == inf || d[v] == inf) continue;
            Ratio now{last[v] - d[v], n - k};
            if (best[v].second == 0 || less(best[v], now))
                best[v] = now;
        }
        if (k + 1 < n) step();
    }
    optional<Ratio> ans;
    for (auto r : best)
    {
        if (r.second && (!ans || less(r, *ans))) ans = r;
    }
    return ans;
}
```

为什么全点初始化成立 把各边减去最小平均值后，所有环非负且存在零环。任意起点最短行走都能删环变成少于 n 条边的路径，所以每点的 max 比值非负。到零环的最短行走接无限重复零环，取长度 n 的最短前缀，其终点使 max 为 0；因此全局 min 为 0。恢复原权后所有比值增加同一常数，公式仍成立。无环图没有长度 n 行走，返回 nullopt。

返回分数与数值类型 分母为 1..n；不保证约分，也不保证分子本身就是某个环的权和。整数类型提升到 int128 进行运算和交叉比较；小数用调用者选择的 T，普通浮点不是精确计算。P3199 采用 GCC 的 113 位浮点进行核心运算，long double 仅用于输入输出。无需保存全部 DP 层，但第二遍必须重新从全 0 开始。

模板题使用：P3199 [HNOI2009] 最小圈

题意：实数有向边权，n≤3000、m≤10000、|w|≤10^7，保证有圈；GCC ___float128 核心运算，long double 读入/输出 8 位小数。

所需模板：minimum_mean_cycle（第 493 页，45.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3199

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using Real = __float128;
    int n, m;
    cin >> n >> m;
    vector<tuple<int, int, Real>> edges;
    while (m--)
    {
        int u, v;
        long double w;
        cin >> u >> v >> w;
        edges.emplace_back(u - 1, v - 1, w);
    }
    auto ans = minimum_mean_cycle(n, edges);
    assert(ans);
    cout << fixed << setprecision(8) << (long double)(ans->first / ans->second) << '\n';
}
```

45.2 非负权无向图最小环与原边方案

问题描述：给定非负权无向图，求至少含 3 个不同顶点的最小权简单环，并给出环上的顶点序列与对应的原边编号。

核心思路：Floyd 求最小环：外层按中间点 k 递增，在用 k 松弛之前，`d[i][j]` 是只经过编号小于 k 的点的最短路。此时枚举 $i<j<k$，若 i,j 都与 k 有边，候选环长为 $d_{ij}+w(i,k)+w(j,k)$；k 是环上编号最大的点，所以每个环都会在其最大点处被考虑到。每轮只记录本轮更优的一对 (i,j)，并立刻沿 `next` 恢复 i 到 j 的路径得到方案（之后 `next` 会被改写）。矩阵 `id` 只保存每对点之间最轻的一条边，自环不进入。

接口契约：MinCycle(n)，n≥0，点 0..n-1。add(u,v,w) 返回从 0 开始的原边号，边数 ≤INT_MAX，0≤w≤LLONG_MAX；允许自环和重边。solve() 从零求至少 3 个不同顶点的简单环，无解 nullopt；故自环忽略，平行边只保留最轻的一条（同权留最早），不计二元环。Result.weight 是 int128，vertices 不重复首点，edges[i] 连接 vertices[i] 与 vertices[(i+1)%size]，边号对应 e；并列返回任意一个，不保证字典序。支持空图、断连、零权、重复 solve 及加边后重算，旧结果独立；e/id 只读。Floyd 更新中间点 k 之前枚举 i<j<k，用 d[i][j]+ 边 (i,k)+ 边 (j,k) 更新最优值，每轮至多恢复一次路径；严格松弛避免零权环导致方案循环。solve 为 $O(n^3+n^2)$，构造 $O(n^2)$，add 摊还 O(1)，总空间 $O(n^2+m)$，恢复通过 next 逐边走路径，不是模拟 DFS。负边不支持；无穷大 2^120 只作内部哨兵，合法简单路径与候选和均小于 2^95。

```cpp
struct MinCycle
{
    using I = __int128_t;
    struct Edge
    {
        int u, v;
        long long w;
    };
    struct Result
    {
        I weight;
        vector<int> vertices, edges;
    };

    int n;
    vector<Edge> e;
    vector<vector<int>> id;

    MinCycle(int n) : n(n), id(n, vector<int>(n, -1))
    {
        assert(n >= 0);
    }

    int add(int u, int v, long long w)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && w >= 0);
        int k = e.size();
        e.push_back({u, v, w});
        if (u != v && (id[u][v] == -1 || w < e[id[u][v]].w))
            id[u][v] = id[v][u] = k;
        return k;
    }
```

初始化与本轮候选检查（接上页同一结构体）：

```
    optional<Result> solve() const
    {
        const I inf = I(1) << 120;
        vector<vector<I>> d(n, vector<I>(n, inf));
        vector<vector<int>> next(n, vector<int>(n, -1));
        for (int i = 0; i < n; i++)
        {
            d[i][i] = 0;
            for (int j = 0; j < n; j++)
            {
                if (id[i][j] == -1) continue;
                d[i][j] = e[id[i][j]].w;
                next[i][j] = j;
            }
        }
        Result ans{inf, {}, {}};
        for (int k = 0; k < n; k++)
        {
            int u = -1, v = -1;
            for (int i = 0; i < k; i++)
            {
                if (id[i][k] == -1) continue;
                for (int j = i + 1; j < k; j++)
                {
                    if (id[j][k] == -1 || d[i][j] == inf) continue;
                    I w = d[i][j] + e[id[i][k]].w + e[id[j][k]].w;
                    if (w >= ans.weight) continue;
                    ans.weight = w;
                    u = i;
                    v = j;
                }
            }
```

方案恢复与 Floyd 更新（接上页 solve 的 k 循环）：

```
            if (u != -1)
            {
                ans.vertices = {k, u};
                ans.edges = {id[k][u]};
                while (u != v)
                {
                    int x = next[u][v];
                    ans.edges.push_back(id[u][x]);
                    ans.vertices.push_back(x);
                    u = x;
                }
                ans.edges.push_back(id[v][k]);
            }
            for (int i = 0; i < n; i++)
            {
                for (int j = 0; j < n; j++)
                {
                    if (d[i][k] == inf || d[k][j] == inf) continue;
                    I w = d[i][k] + d[k][j];
                    if (w >= d[i][j]) continue;
                    d[i][j] = w;
                    next[i][j] = next[i][k];
                }
            }
        }
        if (ans.weight == inf) return nullopt;
        return ans;
    }
};
```

检查必须在松弛之前 固定环中最大编号点 k，其两侧邻点 i、j 都小于 k。此时 d[i][j] 只允许小于 k 的中间点，与两条原边共同构成合法简单环；先做本轮 Floyd 会使路径经过 k，错误接受重复点。

恢复不能每次改进都做 每轮只记最优 i、j，结束枚举后恢复一次，所有恢复共 $O(n^2)$。next 沿真实路径逐边走，不是模拟 DFS；更新必须严格变小，尤其不能把零权等长路径改成自指。

模板题使用：P6175 无向图的最小环问题

题意：求至少 3 个不同顶点的最小权环；n≤100，m≤5000，边权 1..100000，答案 ≤10000000，可转 long long 输出；无解精确输出 No solution.。

所需模板：MinCycle（第 495 页，45.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6175

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    MinCycle g(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        g.add(u - 1, v - 1, w);
    }
    auto ans = g.solve();
    if (ans)
        cout << (long long)ans->weight << '\n';
    else
        cout << "No solution.\n";
}
```

第四十六章　2-SAT

46.1　2-SAT 与任意可行方案

问题描述：给定 n 个布尔变量和若干形如 $(x = a) \vee (y = b)$ 的子句，判断能否同时满足所有子句，并给出一组可行赋值。

核心思路：每个变量拆成“取假” $2x-1$ 和“取真” $2x$ 两个文字节点。子句 $(x = a) \vee (y = b)$ 等价于两条蕴含 $\neg(x = a) \Rightarrow (y = b)$ 与 $\neg(y = b) \Rightarrow (x = a)$，各连一条边。用 `TarjanSCC` 缩点：若 x 的两个文字在同一 SCC 中则无解；否则取拓扑序靠后的文字为真。Tarjan 的分量编号是逆拓扑序，所以选 `bel` 编号较小的那个。

代码依赖：TarjanSCC（第 475 页，43.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：TwoSAT(n)，变量 1..n；add(x,a,y,b) 添加 (x=a) 或 (y=b)，强制值写 add(x,a,x,a)，蕴含 (x=a)=>(y=b) 写 add(x,!a,y,b)。solve() 返回可满足性，仅成功时读取 ans[1..n]；可重复求解和追加子句后重算，空公式可满足。不保证字典序最小。依赖同文件 TarjanSCC：分量编号为逆拓扑序，故真值分量编号较小时取真；换用拓扑序编号的实现须反转不等号。O(n+m) 时间和空间，DFS 递归。

```
struct TwoSAT
{
    int n;
    TarjanSCC g;
    vector<int> ans;

    TwoSAT(int n) : n(n), g(2 * n) {}

    int id(int x, bool value) const { return 2 * x - 1 + value; }

    void add(int x, bool a, int y, bool b)
    {
        g.add(id(x, !a), id(y, b));
        g.add(id(y, !b), id(x, a));
    }

    bool solve()
    {
        g.run();
        ans.assign(n + 1, 0);
        for (int x = 1; x <= n; x++)
        {
            if (g.bel[id(x, 0)] == g.bel[id(x, 1)]) return false;
            ans[x] = g.bel[id(x, 1)] < g.bel[id(x, 0)]; // Tarjan ids are reverse topological
        }
        return true;
    }
};
```

模板题使用：Library Checker two_sat

题意：每个子句是两个文字的或，判断是否可满足并输出一组赋值。输入为 DIMACS：首行 p cnf n m，正数表示变量，负数表示取反，每个子句末尾为 0。

所需模板：TarjanSCC（第 475 页，43.1.2 节）、TwoSAT（第 500 页，46.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/two_sat

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string p, cnf;
    int n, m;
    cin >> p >> cnf >> n >> m;
    TwoSAT g(n);
    for (int i = 0; i < m; i++)
    {
        int a, b, zero;
        cin >> a >> b >> zero;
        g.add(abs(a), a > 0, abs(b), b > 0);
    }
    if (!g.solve())
    {
        cout << "s UNSATISFIABLE\n";
        return 0;
    }
    cout << "s SATISFIABLE\nv";
    for (int i = 1; i <= n; i++) cout << ' ' << (g.ans[i] ? i : -i);
    cout << " 0\n";
}
```

46.2 2-SAT 编号方向与双连通定义

TwoSAT 的 add(x,a,y,b) 表示 $(x=a)\vee(y=b)$；强制值 add(x,a,x,a)，蕴含 $(x=a)\Rightarrow(y=b)$ 写 add(x,!a,y,b)。Tarjan 逆拓扑编号下真文字分量编号更小时取真；换 SCC 实现须检查不等号。solve 仅成功时读 ans，$O(n+m)$，不保证字典序最小。

点双：连通且无割点；桥两端成块，孤立点单独成块，割点可属多块；自环不产生点双、不标桥。每条无向边只 add 一次；lowlink 只跳过父边编号，不跳过重边。BiconnectedCore 点号 $1\dots n$、边号从零；blocks 为点双顶点集，bridge 为桥标记，bel 为 $1\dots cnt$ 的边双号（不承诺拓扑方向）。不连通时圆方图与桥缩点均为森林。

removal_components 返回原块数 before 与删点后总块数 after；割点即 after[u]>before。连通无向图能强连通定向当且仅当无桥：DFS 树边向下、非树边向祖先。

圆方森林原点 $1\dots n$，第 i 个点双（从零）对应方点 $n+i+1$。原点权 -1、方点权为块大小时，s,f 间圆方树路径权和为可作中间点的顶点数。所在树原点数 C，删 u 后各支原点数 a_i，有序端点对经过 u 的次数为 $2\sum_{i<j}a_ia_j+2(C-1)[u\le n]$；sz 仅计原点，用 long long，总 $O(n+m)$。

桥森林邻接项为（边双号，原桥号）。连通图有 L 个叶子边双时，补成边双连通至少且恰需 $\lceil L/2\rceil$ 条边：DFS 序收集叶子，奇数补首叶，前后半段配对；仅一个边双不加边；非连通森林不适用。

第四十七章　欧拉图

47.1　有向图字典序最小欧拉路

问题描述：给定有向图（可含重边、自环），求一条恰好经过每条边一次的路径（欧拉路或欧拉回路），默认要求顶点序列字典序最小；输出顶点序列和对应的边号序列，不存在时返回失败。

核心思路：先检查度数：至多一个点出度比入度大 1（必须作起点），至多一个点入度比出度大 1，其余点出入度相等，且前两者同时有或同时无。起点取该点，否则取指定点或最小的有出边的点。之后用 Hierholzer 算法：`next[u]` 是 u 下一条未走出边的下标，DFS 走完 u 的所有出边后才把 u 及进入它的边加入序列，最后整体翻转。邻接表按终点排序后每次走最小的出边，得到字典序最小的方案。若走过的边数少于 m，说明有边的部分不连通。**接口契约：**构造 DirectedEuler(n)，n≥1，点号 1..n；add(u,v) 返回从 0 开始的逻辑边编号，支持自环与重边。run() 返回是否存在欧拉路；参数 start 默认 0，lexicographic 默认 true。start=0 自动选择最小可行起点，否则固定起点；孤立点不影响有边部分的连通性。成功后 vertices 长度为 m+1，edge_ids 长度为 m，第 i 条原边从 vertices[i] 指向 vertices[i+1]；失败清空两数组。无边时返回单点路径，自动起点为 1。可重复 run，add 会清空旧方案。默认求顶点序列字典序最小方案；关闭排序后只保证任意方案，邻接顺序可能保留上次排序。开启排序总时间 O(n+m log(m+1))，否则 O(n+m)，空间 O(n+m)。DFS 递归深度可能达到 m+1，不是 n。度数检查后，以消费边数判定是否遍历全部非孤立部分，逆后序形成连续走法。小图全部走法对拍同时验证存在性、字典序及边编号，P7771 双风格完整程序本地验证，在线待补。

```
struct DirectedEuler
{
    int n;
    vector<vector<pair<int, int>>> g;
    vector<pair<int, int>> edges;
    vector<int> in, out, next, vertices, edge_ids;

    DirectedEuler(int n) : n(n), g(n + 1), in(n + 1), out(n + 1), next(n + 1)
    {
        assert(n > 0);
    }

    int add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        int id = edges.size();
        edges.push_back({u, v});
        g[u].push_back({v, id});
        out[u]++;
        in[v]++;
        vertices.clear();
        edge_ids.clear();
        return id;
    }

    void dfs(int u, int incoming)
    {
        while (next[u] < (int)g[u].size())
        {
            auto [v, id] = g[u][next[u]++];
            dfs(v, id);
        }
        vertices.push_back(u);
        if (incoming != -1) edge_ids.push_back(incoming);
    }
```

```
    // start=0 chooses the smallest feasible start. Isolated vertices are ignored.
    bool run(int start = 0, bool lexicographic = true)
    {
        assert(0 <= start && start <= n);
        vertices.clear();
        edge_ids.clear();
        int source = 0, sink = 0;
        for (int u = 1; u <= n; u++)
        {
            int delta = out[u] - in[u];
            if (delta == 1 && !source)
                source = u;
            else if (delta == -1 && !sink)
                sink = u;
            else if (delta != 0)
                return false;
        }
        if (bool(source) != bool(sink)) return false;
        if (source)
        {
            if (start && start != source) return false;
            start = source;
        }
        if (!start)
        {
            start = 1;
            for (int u = 1; u <= n; u++)
                if (out[u])
                {
                    start = u;
                    break;
                }
        }
        if (lexicographic)
            for (int u = 1; u <= n; u++) sort(g[u].begin(), g[u].end());
        fill(next.begin(), next.end(), 0);
        dfs(start, -1);
        if (edge_ids.size() != edges.size())
        {
            vertices.clear();
            edge_ids.clear();
            return false;
        }
        reverse(vertices.begin(), vertices.end());
        reverse(edge_ids.begin(), edge_ids.end());
        return true;
    }
};
```

模板题使用：Luogu P7771

题意：给有向图，求字典序最小的欧拉路径。run 默认选择可行起点并按字典序遍历；成功后 vertices 含 m+1 个顶点，失败输出 No。点号 1..n。

所需模板：DirectedEuler（第 502 页，47.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P7771

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    DirectedEuler graph(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        graph.add(u, v);
    }
    if (!graph.run())
        cout << "No\n";
    else
    {
        for (int u : graph.vertices) cout << u << ' ';
        cout << '\n';
    }
    return 0;
}
```

47.2 无向图欧拉路与边方向

问题描述：给定无向图（可含重边、自环），求一条恰好经过每条边一次的路径，默认要求顶点序列字典序最小；同时给出边号序列，可据此得到每条边被走的方向。

核心思路：奇度点只能有 0 个或 2 个；有 2 个时起点必须是其中之一（自动取较小者），否则取指定点或最小的非孤立点。每条无向边在两端各存一份，`used[id]` 按边号标记，保证两份只走一次。DFS 用 `next[u]` 跳过已检查的邻边，走完所有边后才把 u 加入序列（Hierholzer），最后翻转；邻接表排序后得到字典序最小方案。走过的边数不足 m 即不连通、无解。

接口契约：构造 UndirectedEuler(n)，n≥1，点号 1..n；add(u,v) 返回从 0 开始的逻辑边号，自环和重边均保留。run() 返回是否存在欧拉路，参数 start 默认 0（自动），lexicographic 默认 true。自动选最小可行起点；指定起点须满足奇度条件且位于有边部分。成功得到 m+1 个 vertices 和 m 个 edge_ids，失败清空两数组。第 i 条边方向是 vertices[i] 到 vertices[i+1]，与 edges[edge_ids[i]] 比较可判断是否反向，自环任选方向。used 按逻辑边号标记，使两条半边共同只消费一次；自环贡献度数 2。孤立点不影响有边部分；无边图返回指定点或点 1 的单点序列。默认求顶点序列字典序最小方案；关闭排序只保证任意方案，可能保留上次排序。可重复 run，加边清空旧方案。排序开启时 O(n+m log(m+1))，否则 O(n+m)，空间 O(n+m)，DFS 递归深度可能达到 m+1。全小图走法、原边号与方向独立验证，P2731 字典序顶点序列已线上 AC；边编号、失败输入及其他模式仍由本地验证。

```
struct UndirectedEuler
{
    int n;
    vector<vector<pair<int, int>>> g;
    vector<pair<int, int>> edges;
    vector<int> next, used, vertices, edge_ids;

    UndirectedEuler(int n) : n(n), g(n + 1), next(n + 1) { assert(n > 0); }

    int add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        int id = edges.size();
        edges.push_back({u, v});
        g[u].push_back({v, id});
        g[v].push_back({u, id});
        vertices.clear();
        edge_ids.clear();
        return id;
    }

    void dfs(int u, int incoming)
    {
        while (next[u] < (int)g[u].size())
        {
            auto [v, id] = g[u][next[u]++];
            if (used[id]) continue;
            used[id] = 1;
            dfs(v, id);
        }
        vertices.push_back(u);
        if (incoming != -1) edge_ids.push_back(incoming);
    }
```

```
    // start=0 chooses the smallest feasible start. Isolated vertices are ignored.
    bool run(int start = 0, bool lexicographic = true)
    {
        assert(0 <= start && start <= n);
        vertices.clear();
        edge_ids.clear();
        vector<int> odd;
        for (int u = 1; u <= n; u++)
            if (g[u].size() % 2) odd.push_back(u);
        if (!odd.empty() && odd.size() != 2) return false;
        if (!odd.empty())
        {
            if (start && start != odd[0] && start != odd[1]) return false;
            if (!start) start = odd[0];
        }
        if (!start)
        {
            start = 1;
            for (int u = 1; u <= n; u++)
                if (!g[u].empty())
                {
                    start = u;
                    break;
                }
        }
        if (lexicographic)
            for (int u = 1; u <= n; u++) sort(g[u].begin(), g[u].end());
        fill(next.begin(), next.end(), 0);
        used.assign(edges.size(), 0);
        dfs(start, -1);
        if (edge_ids.size() != edges.size())
        {
            vertices.clear();
            edge_ids.clear();
            return false;
        }
        reverse(vertices.begin(), vertices.end());
        reverse(edge_ids.begin(), edge_ids.end());
        return true;
    }
};
```

模板题使用：Luogu P2731

题意：USACO Training：无向多重图保证有欧拉路，输出字典序最小顶点序列。m≤1024，点号 1..500；直接构造 500 个点，不把孤立点 1 强设为起点。run() 自动选较小奇点，若全偶选最小非孤立点，并按邻点排序。输出 vertices，共 m+1 项；本题不输出 edge_ids，边编号与方向另验。DFS 递归，最深可达 m+1。

所需模板：UndirectedEuler（第 506 页，47.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2731

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int m;
    cin >> m;
    UndirectedEuler graph(500);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        graph.add(u, v);
    }
    if (!graph.run()) return 0;
    for (int u : graph.vertices) cout << u << '\n';
    return 0;
}
```

47.3 欧拉回路与指定端点定向

问题描述：混合图欧拉回路/路径：给定若干边，有的方向固定，有的可任选方向，要求给可选边定向，使整张有向图存在欧拉回路，或存在从指定 s 到指定 t 的欧拉路径；输出每条边的最终方向，无解返回空。

核心思路：先把每条边都暂定为 $u \to v$，`balance` 记出度减入度；欧拉回路要求每点为 0，指定端点时 s 应为 $+1$、t 应为 -1，代码先从中扣掉这部分目标。把一条可选边反向，会使两端的 `balance` 分别变化 -2 和 $+2$，所以每点差值必须为偶数。用最大流决定反转哪些边：每条可选边建容量 1 的 $u \to v$ 边，有流即表示反转；源点向差值为正的点连容量 balance/2 的边，差值为负的点连向汇点。满流即可调平，再用并查集检查有边部分连通，就得到合法定向。

代码依赖：Dinic（第 518 页，48.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：mixed_euler_orientation(n,edges,start,finish) 返回可空的定向边序列。每条输入为 {u,v,type}，type=1 固定 u 到 v，type=0 可反转；点号 1..n，n≥1。省略两个端点或均传 0 时请求任意起点的欧拉回路；两个正端点请求指定起终点的欧拉路径，相等时为指定起点回路。无解返回 nullopt，有解返回与原边顺序对应的端点对；无边图可返回有值空序列。忽略无关孤立点，但正端点须在有边部分。先任意定向可反转边，扣除目标出入度差；每条反转将两端差值分别改变 -2 和 +2，奇数差无解，否则用容量 1 的反转边流调整。检查底层无向连通性后，满流即得到所需方向。最大流图 n+2 点、至多 m+n 条正向边，总时间可按 Dinic 的 $O((n+2)^2(m+n))$ 上界计，空间 O(n+m)。依赖 flow.hpp；返回边交给 DirectedEuler 可生成连续走法，指定端点时保持指定起点。定向不保证字典序最小。

```cpp
// Edge = {u,v,type}: type 1 is fixed u->v; type 0 may be reversed.
// start=finish=0 requests a circuit; positive endpoints request a specified trail.
optional<vector<pair<int, int>>> mixed_euler_orientation(
    int n, const vector<array<int, 3>> &edges, int start = 0, int finish = 0)
{
    assert(n > 0 && 0 <= start && start <= n && 0 <= finish && finish <= n);
    assert(bool(start) == bool(finish));
    vector<int> parent(n + 1), id;
    vector<long long> degree(n + 1), balance(n + 1);
    iota(parent.begin(), parent.end(), 0);
    auto find = [&](auto &&self, int u) -> int
    {
        if (parent[u] != u) parent[u] = self(self, parent[u]);
        return parent[u];
    };
    Dinic flow(n + 2);
    vector<pair<int, int>> direction;
    for (auto [u, v, type] : edges)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(type == 0 || type == 1);
        parent[find(find, u)] = find(find, v);
        degree[u]++;
        degree[v]++;
        balance[u]++;
        balance[v]--;
        id.push_back(type == 0 ? flow.add(u, v, 1) : -1);
        direction.push_back({u, v});
    }
```

```
    if (!edges.empty())
    {
        int root = find(find, edges[0][0]);
        for (int u = 1; u <= n; u++)
            if (degree[u] && find(find, u) != root) return nullopt;
        if (start && (!degree[start] || !degree[finish])) return nullopt;
    }
    if (start)
    {
        balance[start]--;
        balance[finish]++;
    }
    int source = n + 1, sink = n + 2;
    long long need = 0;
    for (int u = 1; u <= n; u++)
    {
        if (balance[u] % 2) return nullopt;
        if (balance[u] > 0)
        {
            flow.add(source, u, balance[u] / 2);
            need += balance[u] / 2;
        }
        else if (balance[u] < 0)
            flow.add(u, sink, -balance[u] / 2);
    }
    if (flow.flow(source, sink) != need) return nullopt;
    for (int i = 0; i < (int)edges.size(); i++)
        if (id[i] != -1 && flow.used(id[i]))
            swap(direction[i].first, direction[i].second);
    return direction;
}
```

47.4 任意起终点的混合欧拉路径

问题描述：混合图（部分边方向固定、部分可任选方向）中，不指定起终点，求一种定向使整图存在欧拉路径或欧拉回路，输出每条边的方向。

核心思路：不论方向，欧拉路径的端点只能是无向度数为奇数的点。代码统计奇度点：没有奇度点就请求欧拉回路；恰有两个奇度点 a, b 时，路径方向未知，依次尝试 “a 到 b” 和 “b 到 a” 两种指定端点的定向；其他情况无解。每次尝试都交给 `mixed_euler_orientation` 求解。

代码依赖：mixed_euler_orientation（第 509 页，47.3 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：mixed_euler_trail(n,edges) 返回任意可行欧拉路径的定向边序列，格式同上一节。忽略边原方向统计度数，自环计 2；若奇度点为 0 个，请求回路；若恰有 2 个，尝试这两个端点的两种顺序；否则无解。最多两次定向求解。返回方向可交给 DirectedEuler 自动选择起点，恢复顶点与原边编号序列。枚举所有可反转边方向、全部固定端点及自由端点对拍；逐边检查固定边不变和连续走法，另有 10 万点长流量路径测试。POJ1637 双驱动本地验证，仅检查回路存在性；完整方向和开放路径依靠本地证据，在线待补。

```cpp
// Any endpoints: zero odd degrees requests a circuit; two odd degrees tries both
// orders.
optional<vector<pair<int, int>>> mixed_euler_trail(int n,
                                                   const vector<array<int, 3>> &edges)
{
    assert(n > 0);
    vector<long long> degree(n + 1);
    vector<int> odd;
    for (auto [u, v, type] : edges)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        degree[u]++;
        degree[v]++;
    }
    for (int u = 1; u <= n; u++)
        if (degree[u] % 2) odd.push_back(u);
    if (odd.empty()) return mixed_euler_orientation(n, edges);
    if (odd.size() != 2) return nullopt;
    auto answer = mixed_euler_orientation(n, edges, odd[0], odd[1]);
    if (answer) return answer;
    return mixed_euler_orientation(n, edges, odd[1], odd[0]);
}
```

第四十八章　网络流

48.1　最大流

48.1.1　HLPP 最高标号预流推进与流方案

问题描述：给定有向容量网络和源汇 s,t，求最大流，并能读取每条原边的流量和一个最小割的源侧点集。

核心思路：预流推进：允许点暂存超额流 `ex`，高度 `h` 保证每条残量边 $u\to v$ 有 $h_u\le h_v+1$，只沿 $h_u=h_v+1$ 的边推流。初始高度由汇点反向 BFS 得到，$h_s=n$，源点出边全部推满。每次取高度最高的活跃点，用当前弧不断推流，弧用完就重标号为 $1+\min h_v$。某个低于 n 的高度层被清空（gap）时，高于它且低于 n 的点都已无法到达汇点，直接抬到 $n+1$，它们的超额流最终退回源点。没有活跃点时 `ex[t]` 即最大流。

思路中的情况	代码中的处理	行
初始高度	从 t 沿反向残量边 BFS，不可达点保持 $n+1$，再令源点高度为 n	91
源点推满	源点所有出边按容量推出，终点变为活跃点入桶	111
选最高活跃点	桶链表 `head`/`next` 按高度分层，`top` 为最高非空层	119
推流	当前弧可推时推 $\min(ex_u,$残量$)$，否则当前弧后移	129
重标号	当前弧走到末尾时重新取最小邻居高度加一	125
gap 优化	原层仍有点或原高度 $\ge n$ 时返回；否则抬高断层以上的点并重建活跃桶	59

接口契约：HLPP(n)，2≤n<INT_MAX/2，点 1..n。add(u,v,cap) 返回偶数残量边号 id，允许自环、重边、反向边和零容量；原边数 <INT_MAX/2，全部原容量之和 ≤LLONG_MAX。flow(s,t) 要求 s≠t，每次从零重算并返回总最大流，支持重复调用、换源汇和加边后重算；与 Dinic 返回增量的约定不同。used(id) 读原边净流，min_cut() 返回源侧 0/1 数组（下标 0 不用），只在完整 flow 后有效，加边后须重算。保证非源汇流守恒，多余预流确实退回源点；不止计算最大预流值。e/g 只读，不支持外部改容量或限流。初始反向 BFS 标号，最高标号单链桶、当前弧和 gap；没有周期全局重标号。不可达点初高 n+1，仍须放流。head/next 是活跃点桶链表，不是 DFS 栈；本算法无需 DFS。给出保守 $O(n^2(n+m))$ 时间、O(n+m) 空间界；不据经验或 AC 声称线性。

```
struct HLPP
{
    using ll = long long;
    struct Edge
    {
        int from, to;
        ll cap, initial;
    };
    int n, s, t, top;
    vector<Edge> e;
    vector<vector<int>> g;
    vector<int> h, count, cur, head, next;
    vector<ll> ex;

    HLPP(int n) : n(n), g(n + 1)
    {
        assert(n >= 2 && n < INT_MAX / 2);
    }

    int add(int u, int v, ll cap)
```

```
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n && cap >= 0);
        int id = e.size();
        e.push_back({u, v, cap, cap});
        e.push_back({v, u, 0, 0});
        g[u].push_back(id);
        g[v].push_back(id ^ 1);
        return id;
    }
```

活跃点、推流与 gap 重标号（接上页同一结构体）：

```
    void activate(int u)
    {
        if (u == s || u == t || !ex[u]) return;
        next[u] = head[h[u]];
        head[h[u]] = u;
        top = max(top, h[u]);
    }

    void push(int id, ll f)
    {
        auto &a = e[id];
        e[id ^ 1].cap += f;
        a.cap -= f;
        ex[a.from] -= f;
        bool empty = ex[a.to] == 0;
        ex[a.to] += f;
        if (empty) activate(a.to);
    }

    void relabel(int u)
    {
        int old = h[u];
        count[old]--;
        h[u] = 2 * n;
        for (int id : g[u])
            if (e[id].cap) h[u] = min(h[u], h[e[id].to] + 1);
        count[h[u]]++;
        cur[u] = 0;
        if (old >= n || count[old]) return;
        for (int v = 1; v <= n; v++)
        {
            if (v != s && h[v] > old && h[v] < n)
            {
                count[h[v]]--;
                h[v] = n + 1;
                count[h[v]]++;
                cur[v] = 0;
            }
        }
        fill(head.begin(), head.end(), -1);
        top = -1;
        for (int v = 1; v <= n; v++)
            if (v != u) activate(v);
    }
```

从零初始化与反向 BFS（接上页同一结构体）：

```
    // Recompute from zero, including after adding edges or changing terminals.
    ll flow(int source, int sink)
    {
        assert(1 <= source && source <= n && 1 <= sink && sink <= n);
        assert(source != sink);
        s = source;
        t = sink;
        for (auto &a : e) a.cap = a.initial;
        h.assign(n + 1, n + 1);
        cur.assign(n + 1, 0);
        ex.assign(n + 1, 0);
        next.assign(n + 1, -1);
        head.assign(2 * n + 1, -1);
        count.assign(2 * n + 1, 0);
        queue<int> q;
        h[t] = 0;
        q.push(t);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            for (int id : g[u])
            {
                int v = e[id].to;
                if (e[id ^ 1].cap && h[v] == n + 1)
                {
                    h[v] = h[u] + 1;
                    q.push(v);
                }
            }
        }
        h[s] = n;
        for (int u = 1; u <= n; u++) count[h[u]]++;
        top = -1;
        for (int id : g[s])
            if (e[id].to != s && e[id].cap) push(id, e[id].cap);
```

最高标号放流（接上页同一结构体）：

```
        while (top >= 0)
        {
            if (head[top] == -1)
            {
                top--;
                continue;
            }
            int u = head[top];
            head[top] = next[u];
            while (ex[u])
            {
                if (cur[u] == int(g[u].size()))
                {
                    relabel(u);
                    continue;
                }
                int id = g[u][cur[u]];
                if (e[id].cap && h[u] == h[e[id].to] + 1)
                    push(id, min(ex[u], e[id].cap));
                else
                    cur[u]++;
            }
        }
        return ex[t];
    }
```

原边流量与最小割（接上页同一结构体）：

```
    ll used(int id) const
    {
        return e[id].initial - e[id].cap;
    }

    vector<int> min_cut() const
    {
        vector<int> seen(n + 1);
        queue<int> q;
        seen[s] = 1;
        q.push(s);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            for (int id : g[u])
            {
                int v = e[id].to;
                if (e[id].cap && !seen[v])
                {
                    seen[v] = 1;
                    q.push(v);
                }
            }
        }
        return seen;
    }
};
```

预流不能冒充流方案 不可达汇点的活跃点仍须继续放流，直到多余流退回源点；否则虽然 ex[t] 可能已是最大流值，used 仍可能违反中间点流守恒。完成后，每对残量容量之和等于原容量，各原边流量合法；源侧残量可达集给出同值最小割。

桶与 gap head[h] 和 next[u] 是活跃点单链桶，不是 DFS 栈。点仅在多余流从 0 变正时加入，取出后放流至 0；gap 改变多个点高度后重建桶，并排除当前正在处理的点，避免重复入桶或旧高度记录。初始 BFS 只做一次，反向遍历必须看反边残量；没有周期全局重标号。

调用约定 flow 每次清空旧流并重算，因此允许换源汇；不返回增量，也不支持限流。used 只接收 add 返回的偶数边号，min_cut 只在完整求流后有效。容量总和不超过 LLONG_MAX，不能只保证最大流值未溢出而忽略初始化预流。

模板题使用：P4722 【模板】最大流加强版 / 预流推进

题意：n≤1200、m≤120000，有向容量，输出最大流；允许重边/反向边。

所需模板：HLPP（第 512 页，48.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4722

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    HLPP g(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long c;
        cin >> u >> v >> c;
        g.add(u, v, c);
    }
    cout << g.flow(s, t) << '\n';
}
```

48.1.2 最大流与方案

问题描述：给定有向容量网络和源汇 s,t，求最大流（可限制本次最多增广的流量），并能读取每条原边的流量以及残量网络中源点可达的点集（最小割）。

核心思路：每条边与其反向残量边编号相邻（id 与 $id\oplus 1$）。每个阶段先 `bfs` 在残量网络上从 s 分层得到 `dep`；若 t 可达，就在分层图上反复 `dfs`，只沿层数加一的残量边前进，当前弧 `cur` 跳过已无用的边，每找到一条增广路就沿途扣减残量、加到反向边。直到 t 不可达或达到 `limit`。`used(id)` 为初始容量减剩余残量，`cut` 复用 BFS 求 s 可达的点。注释中的 Scaling 改法把“残量为正”换成“残量不小于阈值”，阈值从大到小减半。

接口契约：Dinic(n)，点号 1..n；add(u,v,c) 返回偶数原边编号；flow(s,t,limit) 返回本次新增流；used(id) 查询原边净流，cut(s) 返回残量可达点集。允许重边、自环，容量非负，s≠t；重复求流须使用相同源汇。只有求尽最大流后 cut 才是最小割。一般 $O(V^2E)$，递归增广深度 O(V)。总流不得超过 long long 上限。get_edge(id) 返回 tuple(起点, 终点, 总容量, 当前流)，change_edge(id,cap,f) 修改总容量与流，要求 id 为 add 返回的偶数原边编号且 0≤f≤cap；正向残量变为 cap-f，反向为 f。仅保证此残量对一致，不自动维持所有顶点的流量守恒；继续 flow 前必须由调用方保证全局可行流及同源汇。增容且保留 f 合法，缩容低于已有流时不能只截断该边。要从零重算须将所有原边一起重置为零流。两个新接口为 O(1)，尚只有本地验证。可选容量缩放：按源码各处 Scaling 注释同步修改 BFS、DFS、递归调用和外层循环，不能只取消一处注释。阈值从不超过当前最大残量 mx 的最大二次幂降到 1，mx=0 直接返回；用 lim≤mx/2 再翻倍避免溢出。bfs/dfs 的 lim 默认 1，cut 仍检查全部正残量；flow 仍返回新增流且支持限流续流。缩放一般 O(VE log(U+1))，U 为本次初始最大残量；不保证比普通版快，大 INF 可能增加空阶段。

```
struct Dinic
{
    using ll = long long;

    struct Edge
    {
        int from, to;
        ll cap, initial;
    };

    int n;
    vector<Edge> e;
    vector<vector<int>> g;
    vector<int> dep, cur;

    Dinic(int n) : n(n), g(n + 1), dep(n + 1), cur(n + 1) {}

    int add(int u, int v, ll c)
    {
        assert(c >= 0);
        int id = (int)e.size();
        e.push_back({u, v, c, c});
        e.push_back({v, u, 0, 0});
        g[u].push_back(id);
        g[v].push_back(id + 1);
        return id;
    }
```

```
29      bool bfs(int s, int t) // Scaling: add parameter ll lim = 1.
30      {
31          fill(dep.begin(), dep.end(), -1);
32          queue<int> q;
33          dep[s] = 0;
34          q.push(s);
35          while (!q.empty())
36          {
37              int u = q.front();
38              q.pop();
39              for (int id : g[u])
40                  // Scaling: replace e[id].cap with e[id].cap >= lim.
41                  if (e[id].cap && dep[e[id].to] < 0)
42                  {
43                      dep[e[id].to] = dep[u] + 1;
44                      q.push(e[id].to);
45                  }
46          }
47          return dep[t] >= 0;
48      }
49
50      ll dfs(int u, int t, ll f) // Scaling: add parameter ll lim = 1.
51      {
52          if (u == t) return f;
53          for (int &i = cur[u]; i < (int)g[u].size(); i++)
54          {
55              int id = g[u][i], v = e[id].to;
56              // Scaling: replace !e[id].cap with e[id].cap < lim.
57              if (!e[id].cap || dep[v] != dep[u] + 1) continue;
58              ll d = dfs(v, t, min(f, e[id].cap)); // Scaling: pass lim last.
59              if (d)
60              {
61                  e[id].cap -= d;
62                  e[id ^ 1].cap += d;
63                  return d;
64              }
65          }
66          return 0;
67      }
```

```
    // Returns additional flow; reuse only with the same source/sink.
    ll flow(int s, int t, ll limit = LLONG_MAX)
    {
        assert(s != t && limit >= 0);
        ll ans = 0;
        // Scaling: mx = maximum current residual cap over all e (both directions).
        // Scaling: if mx == 0, return 0.
        // Scaling: start ll lim = 1, double while lim <= mx / 2.
        // Scaling: wrap the whole while below in for (; lim; lim >>= 1).
        while (ans < limit && bfs(s, t)) // Scaling: pass lim last.
        {
            fill(cur.begin(), cur.end(), 0);
            while (ans < limit)
            {
                ll d = dfs(s, t, limit - ans); // Scaling: pass lim last.
                if (!d) break;
                ans += d;
            }
        }
        return ans;
    }

    ll used(int id) const { return e[id].initial - e[id].cap; }

    tuple<int, int, ll, ll> get_edge(int id) const
    {
        assert(0 <= id && id < (int)e.size() && !(id & 1));
        return {e[id].from, e[id].to, e[id].initial, used(id)};
    }

    // Changes one residual pair; caller must preserve global flow conservation.
    void change_edge(int id, ll cap, ll f)
    {
        assert(0 <= id && id < (int)e.size() && !(id & 1));
        assert(0 <= f && f <= cap);
        e[id].initial = cap;
        e[id].cap = cap - f;
        e[id ^ 1].initial = 0;
        e[id ^ 1].cap = f;
    }

    vector<int> cut(int s)
    {
        bfs(s, s);
        vector<int> ans;
        for (int u = 1; u <= n; u++)
            if (dep[u] >= 0) ans.push_back(u);
        return ans;
    }
};
```

模板题使用：Luogu P3376

题意：给定有向容量网络与源点、汇点，求最大流。输入点号为 1..n，平行边逐条加入。

所需模板：Dinic（第 518 页，48.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3376

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    Dinic g(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long c;
        cin >> u >> v >> c;
        g.add(u, v, c);
    }
    cout << g.flow(s, t) << '\n';
}
```

48.1.3 排列约束与最大流单位边分类

已知元素切开未知段：段内候选来自原排列对应区间，需求为未知位置数；已知元素单独成需求一的组，首尾设虚拟锚点，保留零需求组。源连左组、右组连汇，容量为需求，每个元素是左右组间一条单位边；流量达到 k 等价于存在合法集合。满流后在残量网络求 SCC，判断能否沿环改变某条单位边，得到必选/可选/不可选。本题增广求流 $O(n^2)$，SCC 为 $O(n)$。

48.2 最小割

48.2.1 最大权闭合子图与选点方案

问题描述：给定点权（可正可负）和依赖关系“选 u 则必须选 v”，选出满足所有依赖的点集（闭合子图）使点权和最大，返回最大值和一个选点方案。

核心思路：最小割建模：源点向正权点连容量为权值的边，负权点向汇点连容量为 $-w$ 的边，依赖 $u \to v$ 连容量“无穷”（正权总和加一）的边。割的源侧就是选中的点集：无穷边保证不会出现选 u 不选 v；被割的边恰是放弃的正权点和选中的负权点，所以最大权等于正权总和减最小割。用 `Dinic` 求最大流，残量网络中源点可达的原点即为方案。

代码依赖：Dinic（第 518 页，48.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：maximum_closure(weight,dependencies) 返回 (最大权值, 选中点列表)，weight 为 0-based vector<long long>，图中点号 1..n。依赖边 (u,v) 表示选 u 必须选 v，允许环、自环与重边；允许空选集，因此答案不小于 0。依赖 flow.hpp 中的 Dinic。正权连源、负权连汇，依赖边容量取正权总和加 1；要求所有正权之和严格小于 LLONG_MAX。负权先提升到 int128 再取负，并截断到依赖边容量，因此 LLONG_MIN 也可处理。返回残量图源点可达的原顶点，权值为正权总和减最小割；超大负权边不会出现在最优割中，截断不改变原问题最优值。每次调用重建网络。复杂度沿用 Dinic：一般 $O(V^2E)$，空间 O(V+E)，增广保持递归。返回一种最优闭合集，不承诺最大或最小选点个数。

```
pair<long long, vector<int>> maximum_closure(const vector<long long> &weight,
                                             const vector<pair<int, int>> &dependencies)
{
    using I = __int128_t;
    int n = weight.size(), s = n + 1, t = n + 2;
    I positive = 0;
    for (long long w : weight)
        if (w > 0) positive += w;
    assert(positive < LLONG_MAX);
    long long inf = (long long)positive + 1;
    Dinic graph(n + 2);
    for (int u = 1; u <= n; u++)
    {
        long long w = weight[u - 1];
        if (w > 0)
            graph.add(s, u, w);
        else if (w < 0)
            graph.add(u, t, (long long)min(-I(w), I(inf)));
    }
    for (auto [u, v] : dependencies)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        graph.add(u, v, inf);
    }
    long long answer = (long long)positive - graph.flow(s, t);
    vector<int> selected;
    for (int u : graph.cut(s))
        if (u <= n) selected.push_back(u);
    return {answer, selected};
}
```

48.2.2 正容量最小割的可行边与必选边

问题描述：在已求出最大流的网络上，判断每条原边是否可能出现在某个最小割中（可行边），以及是否出现在所有最小割中（必选边）。

核心思路：在最终残量网络上（只取残量为正的边）用 `TarjanSCC` 缩点。属于最小割的边必须满流，非满流边两项均为假。满流边 $u \to v$ 若 u, v 不在同一 SCC，就存在最小割把它们分开，是可行边；若进一步 u 与 s 同 SCC、v 与 t 同 SCC，每个最小割都必须割它，是必选边。

代码依赖：Dinic（第 518 页，48.1.2 节）、TarjanSCC（第 475 页，43.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：mincut_edges(g,s,t) 只读分析 Dinic 已经完整求出的普通 s→t 最大流；不得传入限流提前停止或人为修改过的残量网络。所有原边初始容量必须严格为正，最大流值须可由 long long 表示。返回逐条原边的 {possible,forced}，对应 Dinic 原始偶数边号 id 的结果是 ans[id/2]。只有饱和边可能属于最小割：两端残量 SCC 不同则可行；起点与 s 同 SCC、终点与 t 同 SCC 则必选。支持平行边和自环，源汇不同，点号 1..g.n。O(n+m) 时间与空间，Tarjan 递归栈 O(n)。零容量边没有正流对应的反向残量边，不能套用本函数的同 SCC 简化条件。每次修改图或重新增广后都需重新分析。

```
vector<array<bool, 2>> mincut_edges(const Dinic &g, int s, int t)
{
    TarjanSCC d(g.n);
    for (auto e : g.e)
        if (e.cap) d.add(e.from, e.to);
    d.run();
    vector<array<bool, 2>> ans(g.e.size() / 2);
    for (int id = 0; id < (int)g.e.size(); id += 2)
    {
        auto e = g.e[id];
        assert(e.initial > 0);
        if (e.cap) continue;
        ans[id / 2][0] = d.bel[e.from] != d.bel[e.to];
        ans[id / 2][1] = d.bel[e.from] == d.bel[s] && d.bel[e.to] == d.bel[t];
    }
    return ans;
}
```

48.2.3 Gomory–Hu 最小割树

问题描述：给定无向非负容量图，构造 Gomory–Hu 树（最小割树）：一棵 n 个点的带权树，使原图任意两点 s,t 的最小割等于树上 s–t 路径的最小边权。

核心思路：Gusfield 算法：初始所有点的树父亲都是 0。依次对 $s = 1..n-1$，取 $t = \text{parent}[s]$，在原图上用 Dinic 求 s–t 最小割，`side` 为残量网络中 s 一侧。之后，原本挂在 t 下且位于 s 侧的点改挂到 s 下；若 t 的父亲也在 s 侧，则让 s 取代 t 的位置（s 接到 t 原父亲下，t 改挂到 s，割值也相应交换），否则边 (s,t) 的权就是本次割值。最后输出每个点与其父亲的边。

代码依赖：Dinic（第 518 页，48.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：gomory_hu(n,edges) 返回 n-1 条 (u,v, 容量) 树边；n≥1，原图点号 0..n-1，输入为无向边 tuple 列表，重边保留、自环忽略、容量非负。非自环边总容量须严格小于 LLONG_MAX。两点最小割是树路径最小边权；删去树边得到的两侧也是原图该边端点的最小割。允许非连通图和零权树边，n=1 返回空树。运行 n-1 次 Dinic，一般上界 $O(n^3m+n^2)$，额外空间 O(n+m)，DFS 增广保持递归。P4897 输入 n 表示 n+1 个点，驱动传 n+1。双版本 P4897 已 AC，最长单点分别 252、185 ms。

```
vector<tuple<int, int, long long>>
gomory_hu(int n, const vector<tuple<int, int, long long>> &edges)
{
    assert(n >= 1);
    vector<int> parent(n);
    vector<long long> value(n);
    for (int s = 1; s < n; s++)
    {
        int t = parent[s];
        Dinic flow(n);
        for (auto [u, v, w] : edges)
            if (u != v)
            {
                flow.add(u + 1, v + 1, w);
                flow.add(v + 1, u + 1, w);
            }
        long long cut = flow.flow(s + 1, t + 1);
        vector<bool> side(n);
        for (int u : flow.cut(s + 1)) side[u - 1] = true;
        for (int v = 1; v < n; v++)
            if (v != s && parent[v] == t && side[v]) parent[v] = s;
        if (side[parent[t]])
        {
            parent[s] = parent[t];
            parent[t] = s;
            value[s] = value[t];
            value[t] = cut;
        }
        else
            value[s] = cut;
    }
    vector<tuple<int, int, long long>> tree;
    for (int v = 1; v < n; v++) tree.push_back({v, parent[v], value[v]});
    return tree;
}
```

模板题使用：Luogu P4897

题意：给无向图，多次询问不同两点的最小割。点号为 0..n，因此建树传 n+1。再预处理所有点对的答案，查询矩阵额外占 O(n^2) 时间和空间。

所需模板：Dinic（第 518 页，48.1.2 节）、gomory_hu（第 525 页，48.2.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4897

```
int main()
{
    int n, m;
    scanf("%d%d", &n, &m);
    vector<tuple<int, int, long long>> edges;
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long w;
        scanf("%d%d%lld", &u, &v, &w);
        edges.push_back({u, v, w});
    }
    auto tree = gomory_hu(n + 1, edges);
    vector<vector<pair<int, long long>>> g(n + 1);
    for (auto [u, v, w] : tree)
    {
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }
    int q;
    scanf("%d", &q);
    while (q--)
    {
        int u, v;
        scanf("%d%d", &u, &v);
        // Minimum cut = minimum edge on the tree path.
        long long answer = LLONG_MAX;
        auto dfs = [&](auto &&self, int x, int parent, long long value) -> void
        {
            if (x == v) answer = value;
            for (auto [y, w] : g[x])
                if (y != parent) self(self, y, x, min(value, w));
        };
        dfs(dfs, u, -1, LLONG_MAX);
        printf("%lld\n", answer);
    }
    return 0;
}
```

48.2.4 闭合集、树形图与全局割的完整用法

最大权闭合子图：收益为正权点、成本为负权点，依赖边由前者指向后者；maximum_closure 返回所选点号，共享成本只计一次，总额用 long long。

Arborescence::solve 点号与根 0-based，返回 optional 区分无解与负费用；只返回费用，不恢复边。StoerWagner::solve 要求至少两点、非负对称权，side 为一侧 0-based 原顶点。核心费用为 128 位类型。

48.3 费用流

48.3.1 势能最小费用最大流

问题描述：给定有向网络，每条边有容量和单位费用（可为负，但初始无负环），求 s 到 t 的最小费用最大流（可限制流量上限），并可给出最小费用关于流量的分段线性函数的折点。

核心思路：逐次最短路增广配合势能。先以全 0 为初值做 Bellman–Ford（等价于加超级源），得到满足 $h_v \le h_u + c$ 的势 `h`；之后用约化费用 $c + h_u - h_v \ge 0$ 跑 Dijkstra，并把距离加进 `h`，使下一轮约化费用仍非负。每轮沿 `pre` 取 s 到 t 的最短路，以路径瓶颈（不超过剩余限额）增广并累计费用。这一轮的单位费用为 $h_t - h_s$；`slope` 记录每轮后的 (流量, 费用)，单位费用与上一段相同时合并为一段。

接口契约：MinCostFlow(n)，点号 1..n；add(u,v,cap,cost) 返回边编号，flow(s,t,limit) 返回新增流量及 128 位费用，used(id) 返回原边流量。允许负费用，初始残量网络必须没有负环；检测到负环抛异常。Bellman-Ford 初始化 O(VE)，每次增广 O(E log V)；增广次数不能简单假设为 O(V)。另要求 $\sum(\text{cap}\times|\text{cost}|)<2^{120}$，防止总费用超过 int128。slope(s,t,limit) 返回从 (0,0) 开始的本次新增流量/费用折点，相邻同斜率段合并，费用为 int128。无增广时只返回起点；每个整数流量的费用可在线性段内插值。重复调用仍用同源汇，返回值不是历史累计值。flow 不存储曲线；slope 额外空间为折点数。内部 run 不是用户接口。新曲线接口仅本地验证，P3381 驱动只输出终点，不能据此验证全部中间折点。

```
struct MinCostFlow
{
    using ll = long long;
    using i128 = __int128_t;

    struct Edge
    {
        int from, to;
        ll cap, cost, initial;
    };

    int n;
    vector<Edge> e;
    vector<vector<int>> g;

    MinCostFlow(int n) : n(n), g(n + 1) {}

    int add(int u, int v, ll cap, ll cost)
    {
        assert(cap >= 0 && cost != LLONG_MIN);
        int id = (int)e.size();
        e.push_back({u, v, cap, cost, cap});
        e.push_back({v, u, 0, -cost, 0});
        g[u].push_back(id);
        g[v].push_back(id + 1);
        return id;
    }

    ll used(int id) const { return e[id].initial - e[id].cap; }
```

```
    // Negative costs allowed; initial residual graph must have no negative cycle.
    pair<ll, i128> run(int s, int t, ll limit, vector<pair<ll, i128>> *curve)
    {
        assert(s != t && limit >= 0);
        const i128 inf = i128(1) << 120;
        vector<i128> h(n + 1, 0), d(n + 1);
        vector<int> pre(n + 1);
        // Super-source Bellman-Ford: feasible potentials for every vertex.
        for (int it = 0; it < n; it++)
        {
            bool changed = false;
            for (auto &a : e)
                if (a.cap && h[a.to] > h[a.from] + a.cost)
                {
                    h[a.to] = h[a.from] + a.cost;
                    changed = true;
                }
            if (!changed) break;
            if (it == n - 1) throw invalid_argument("negative cycle");
        }
        ll f = 0;
        i128 cost = 0;
        optional<i128> last;
        while (f < limit)
        {
            fill(d.begin(), d.end(), inf);
            d[s] = 0;
            priority_queue<pair<i128, int>,
                           vector<pair<i128, int>>,
                           greater<pair<i128, int>>>
                q;
            q.push({0, s});
            while (!q.empty())
            {
                auto [du, u] = q.top();
                q.pop();
                if (du != d[u]) continue;
                for (int id : g[u])
                {
                    auto &a = e[id];
                    i128 nd = du + a.cost + h[u] - h[a.to];
                    if (a.cap && nd < d[a.to])
                    {
                        d[a.to] = nd;
                        pre[a.to] = id;
                        q.push({nd, a.to});
                    }
                }
            }
```

```
            if (d[t] == inf) break;
            for (int u = 1; u <= n; u++)
                if (d[u] != inf) h[u] += d[u];
            ll take = limit - f;
            for (int u = t; u != s; u = e[pre[u]].from) take = min(take, e[pre[u]].cap);
            for (int u = t; u != s; u = e[pre[u]].from)
            {
                int id = pre[u];
                e[id].cap -= take;
                e[id ^ 1].cap += take;
                cost += i128(take) * e[id].cost;
            }
            f += take;
            if (curve)
            {
                i128 unit = h[t] - h[s];
                if (last && *last == unit) curve->pop_back();
                curve->push_back({f, cost});
                last = unit;
            }
        }
        return {f, cost};
    }

    pair<ll, i128> flow(int s, int t, ll limit = LLONG_MAX)
    {
        return run(s, t, limit, nullptr);
    }

    vector<pair<ll, i128>> slope(int s, int t, ll limit = LLONG_MAX)
    {
        vector<pair<ll, i128>> curve{{0, 0}};
        run(s, t, limit, &curve);
        return curve;
    }
};
```

模板题使用：Luogu P3381

题意：给有向容量费用网络，求最大流及对应最小费用。点号 1..n，add 参数依次为起点、终点、容量、单位费用。题目保证费用答案不超过 2^31-1，因此这里可将返回的 int128 转为 long long；一般题目不能照搬转换。

所需模板：MinCostFlow（第 527 页，48.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3381

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    MinCostFlow graph(n);
    while (m--)
    {
        int u, v;
        long long capacity, cost;
        cin >> u >> v >> capacity >> cost;
        graph.add(u, v, capacity, cost);
    }
    auto [flow, cost] = graph.flow(s, t);
    cout << flow << ' ' << (long long)cost << '\n';
}
```

48.3.2 允许负环的最小费用最大流

问题描述：带负费用（可含负环）的最小费用最大流：在流量不超过上限的前提下求 s 到 t 的最大流，并在此流量下使总费用最小；与源汇无关的负环也可以被利用来降低费用。可得每条原边的流量。

核心思路：普通最短路增广不能处理负环，因此先把所有负费用边直接流满：总费用加上 $c\cdot w$，残量图中只留费用为 $-w>0$ 的反向“退流”边，这样所有边的费用都非负，但各点出现流量不平衡 `balance`。再加一条 $t\to s$、费用 0 的边，从超级源向盈余点、从亏空点向超级汇连边，用最小费用流修复平衡；其中经过 $t\to s$ 的流量就是已经得到的 s–t 流量 `base`。随后关闭所有辅助边，从 s 到 t 继续最短路增广剩余流量。负费用边的实际流量为容量减去退流边上的流量。

思路中的情况	代码中的处理	行
负费用边	预先流满，计入费用，只建正费用的反向退流边	24
允许已有流经过源汇	加 $t\to s$ 的零费用边	41
修复流量平衡	超级源汇间求满流最小费用流，`base` 为 $t\to s$ 上的流量	51
关闭辅助边	把 `first` 之后添加的边容量清零	55
继续增广	从 s 到 t 补足剩余流量	56
还原原边流量	负费用边的流量为容量减退流量	65

代码依赖：MinCostFlow（第 527 页，48.3.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：NegativeCostFlow(n)，点号 1..n；add(u,v,cap,cost) 返回从 0 开始的原边编号。容量非负，所有原边容量总和 ≤LLONG_MAX；费用为 long long 且不等于 LLONG_MIN。允许重边、自环及任意位置的负环。solve(s, t, limit) 只调用一次，limit 默认 LLONG_MAX，s≠t、limit≥0，返回 (不超过 limit 的最大流量, 该流量下的最小费用)，费用为 int128。limit=0 可求最小费用循环流；不足 limit 时返回实际最大流。成功后 used(id) 返回原边流量，自动还原负边退流。先满流负费用边并建立反向正费用退流边，再用超级源汇修复平衡；初始残量费用非负，修复后无负环。删除辅助边后继续最短增广，包含不经过源汇的负环收益。依赖 MinCostFlow，使用两次势能预处理；若总增广次数为 K，时间 O(VE + K(E log(E+1) + V))，空间 O(V+E)，整数容量下为伪多项式，不是费用缩放算法。节点和残量边数须可由 int 表示。

```
struct NegativeCostFlow
{
    using ll = long long;
    using I = __int128_t;
    int n;
    MinCostFlow g;
    vector<ll> balance, cap;
    vector<int> ids;
    vector<bool> reversed;
    ll total = 0;
    I cost = 0;
    bool solved = false;

    NegativeCostFlow(int n) : n(n), g(n + 2), balance(n + 1) {}

    int add(int u, int v, ll c, ll w)
    {
        assert(!solved && 1 <= u && u <= n && 1 <= v && v <= n);
        assert(0 <= c && c <= LLONG_MAX - total && w != LLONG_MIN);
        total += c;
        int id = ids.size();
        cap.push_back(c);
        reversed.push_back(w < 0);
        if (w < 0)
        {
            balance[u] -= c;
            balance[v] += c;
            cost += I(c) * w;
            ids.push_back(g.add(v, u, c, -w));
        }
        else
            ids.push_back(g.add(u, v, c, w));
        return id;
    }
```

需求修复、增广与原边流量还原（接上页同一结构体）：

```
    pair<ll, I> solve(int s, int t, ll limit = LLONG_MAX)
    {
        assert(!solved && 1 <= s && s <= n && 1 <= t && t <= n && s != t);
        assert(limit >= 0);
        solved = true;
        int first = g.e.size(), back = g.add(t, s, min(total, limit), 0);
        ll need = 0;
        for (int u = 1; u <= n; u++)
            if (balance[u] > 0)
            {
                g.add(n + 1, u, balance[u], 0);
                need += balance[u];
            }
            else if (balance[u] < 0)
                g.add(u, n + 2, -balance[u], 0);
        auto [sent, correction] = g.flow(n + 1, n + 2, need);
        assert(sent == need);
        ll base = g.used(back);
        cost += correction;
        for (int id = first; id < (int)g.e.size(); id++) g.e[id].cap = 0;
        auto [more, extra] = g.flow(s, t, limit - base);
        cost += extra;
        return {base + more, cost};
    }

    ll used(int id) const
    {
        assert(solved && 0 <= id && id < (int)ids.size());
        ll f = g.used(ids[id]);
        return reversed[id] ? cap[id] - f : f;
    }
};
```

模板题使用：Luogu P7173

题意：求允许负费用环的最小费用最大流，独立于源汇的负环也计入费用。无额外流量上限时调用 solve(s,t)。题目容量和费用界保证最终费用可转为 long long；一般模板返回 int128，不能任意照搬此转换。

所需模板：MinCostFlow（第 527 页，48.3.1 节）、NegativeCostFlow（第 530 页，48.3.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P7173

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    NegativeCostFlow g(n);
    while (m--)
    {
        int u, v;
        long long cap, cost;
        cin >> u >> v >> cap >> cost;
        g.add(u, v, cap, cost);
    }
    auto [flow, cost] = g.solve(s, t);
    cout << flow << ' ' << (long long)cost << '\n';
}
```

48.3.3 SPFA 费用流与残量方案

SPFA 版允许负边，**不允许负环**：每次调用先全点零距离入队检查（含不可达负环），发现即抛异常。最终方案按原边编号查询 used；flow 返回本次新增流量与费用，重复调用须同源汇。容量 long long，距离与费用 int128；费用不能为 LLONG_MIN，总流不超过 long long，且 $\sum cap\,|cost| < 2^{120}$。增广 A 次为 $O((A+1)VE)$；通常优先势能 Dijkstra。

48.4 上下界网络流

48.4.1 上下界可行循环流

问题描述：给定有向图，每条边的流量必须在 $[lo, hi]$ 内，求一个无源汇的可行循环流（每点流入等于流出），判断是否存在并给出每条边的流量。

核心思路：先让每条边强制流满下界 lo，余下的自由量作为容量 $hi - lo$ 的边加入 `Dinic`。强制流使每个点产生不平衡 `balance`（流入下界和减流出下界和）：为正的点由超级源 $n+1$ 补给，为负的点向超级汇 $n+2$ 送出。最大流把超级源出边全部流满时可行，此时每条边的流量为 lo 加上它在 Dinic 中的流量。

代码依赖：Dinic（第 518 页，48.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：BoundedCirculation(n)，点号 1..n。add(u,v,lo,hi) 返回原边编号，0≤lo≤hi≤LLONG_MAX，所有下界总和须 ≤LLONG_MAX。solve() 仅调用一次，返回可行性；成功后 used(id) 读取边流量。每点要求流量守恒，属于循环流；带源汇最大流用 BoundedMaxFlow。hi-lo 是残量容量，下界引起 balance；各边上界不必求和，只要求下界总和保证 balance 和超级源总需求不溢出。依赖 Dinic。含负费用环、要求费用最优时用 NegativeCostFlow。

```
struct BoundedCirculation
{
    using ll = long long;
    int n;
    Dinic g;
    vector<ll> balance, lower;
    vector<int> ids;
    bool solved = false, feasible = false;

    BoundedCirculation(int n) : n(n), g(n + 2), balance(n + 1) {}

    int add(int u, int v, ll lo, ll hi)
    {
        assert(!solved && 0 <= lo && lo <= hi);
        int id = ids.size();
        ids.push_back(g.add(u, v, hi - lo));
        lower.push_back(lo);
        balance[u] -= lo;
        balance[v] += lo;
        return id;
    }

    bool solve()
    {
        assert(!solved);
        solved = true;
        ll need = 0;
        for (int u = 1; u <= n; u++)
            if (balance[u] > 0)
            {
                g.add(n + 1, u, balance[u]);
                need += balance[u];
            }
            else if (balance[u] < 0)
                g.add(u, n + 2, -balance[u]);
```

```
        return feasible = (g.flow(n + 1, n + 2) == need);
    }

    ll used(int id) const
    {
        assert(feasible);
        return lower[id] + g.used(ids[id]);
    }
};
```

模板题使用：LibreOJ 115

题意：无源汇有上下界可行流：每个点流入等于流出，按输入顺序输出每条边的合法流量，无解输出 NO。点号 1..n，n 不超过 200、m 不超过 10200，0≤lo≤hi<3000。add 返回 0 起原始边编号，used(i) 不是 Dinic 残量边号；solve 只能调用一次，且只有可行时才可读取 used。自环、重边逐条保留，不添加源汇回边。

所需模板：Dinic（第 518 页，48.1.2 节）、BoundedCirculation（第 534 页，48.4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://loj.ac/p/115

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    BoundedCirculation graph(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long lo, hi;
        cin >> u >> v >> lo >> hi;
        graph.add(u, v, lo, hi);
    }
    if (!graph.solve())
    {
        cout << "NO\n";
        return 0;
    }
    cout << "YES\n";
    for (int i = 0; i < m; i++) cout << graph.used(i) << '\n';
}
```

48.4.2 有源汇上下界最值流与边流量

问题描述：给定有向图，每条边流量必须在 $[lo, hi]$ 内，除源汇 s, t 外流量守恒，求 s 净流出量的最大值（`minimum` 求最小值）；无可行流时返回空。

核心思路：加两条辅助边 $t \to s$ 与 $s \to t$（下界 0、上界为所有上界之和，后者允许净流为负），问题化为 `BoundedCirculation` 的可行循环流。可行时 s 的净流出等于 $t \to s$ 的流量减去 $s \to t$ 的流量。然后用 `change_edge` 把从 `first` 起新增的辅助边（两条辅助边和超级源汇边）的容量和流全部清零，在剩下的残量网络上从 s 到 t 继续增广，把增量加到答案上；下界已固定在残量中不会被破坏。最小值通过交换源汇求最大值再取反得到。

代码依赖：BoundedCirculation（第 534 页，48.4.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：BoundedMaxFlow(n)，点号 1..n；add(u,v,lo,hi) 返回从 0 开始的原边编号，0≤lo≤hi，所有原边上界总和 ≤LLONG_MAX。solve(s,t) 求最大净流，minimum(s,t) 求最小净流；同一对象只能任选其一调用一次，s≠t。

minimum 交换源汇求最大值后取反，原边方向及 used 不变；返回 optional<long long>，nullopt 表示无可行流。目标为源点流出减流入的净流量，两种查询都允许负答案，不能把 -1 直接当无解；minimum 不额外强制净流量非负。只在成功后读取 used(id) 的原边方案。先加 t→s 与 s→t 两条辅助边兼容负净流，再求可行循环流；保存净值后清空全部辅助残量边，最后在原图从 s 到 t 增广。最优值与增加量均受原边上界总和控制。允许自环、重边；构建和清理 O(n+m)，主体为两次 Dinic，额外空间 O(n+m)。依赖 BoundedCirculation 与 Dinic，二者内部字段只读，不应在求解后再加边或改流。

```
struct BoundedMaxFlow
{
    using ll = long long;
    BoundedCirculation b;
    ll total = 0;
    int m = 0;

    BoundedMaxFlow(int n) : b(n) {}

    int add(int u, int v, ll lo, ll hi)
    {
        assert(0 <= lo && lo <= hi && hi <= LLONG_MAX - total);
        total += hi;
        b.add(u, v, lo, hi);
        return m++;
    }

    optional<ll> solve(int s, int t)
    {
        assert(!b.solved && 1 <= s && s <= b.n && 1 <= t && t <= b.n && s != t);
        int first = b.g.e.size();
        int back = b.add(t, s, 0, total);
        int forward = b.add(s, t, 0, total);
        if (!b.solve()) return nullopt;
        __int128 ans = (__int128)b.used(back) - b.used(forward);
        for (int id = first; id < (int)b.g.e.size(); id += 2) b.g.change_edge(id, 0, 0);
        ans += b.g.flow(s, t);
        assert(-(__int128)total <= ans && ans <= total);
        return (ll)ans;
    }
```

最小净流与原边流量（接上页同一结构体）：

```
    optional<ll> minimum(int s, int t)
    {
        auto ans = solve(t, s);
        if (ans) *ans = -*ans;
        return ans;
    }

    ll used(int id) const
    {
        assert(0 <= id && id < m);
        return b.used(id);
    }
};
```

48.5 Stoer–Wagner 算法

48.5.1 无向全局最小割

问题描述：给定无向非负权图（邻接矩阵），求全局最小割：把点集分成两个非空部分，使跨越两部分的边权和最小，返回割值和其中一侧的点集。

核心思路：重复 $n-1$ 个阶段。每阶段做“最大邻接序”：从空集开始，每次选出与已选集合连边总权 `dis` 最大的点加入。设最后两个加入的点为 `prev`、`sel`，则最后一点的 `dis` 是把 `sel` 与其余点分开的一个割，且等于当前（已合并的）图中 `prev`、`sel` 之间的最小割；用它更新答案，割的一侧为 `sel` 代表的原点集 `group`。然后把 `sel` 合并进 `prev`（边权相加），因为之后只需考虑不分开这两点的割。全局最小割是所有阶段割值的最小值。

接口契约：solve(w)，n≥2；w 为 n×n 对称非负权矩阵，重边先累加，主对角置零。返回 {割权, 一侧顶点列表}，编号 0..n-1；两侧非空，不要求原图连通，不连通时割权为 0。并列最优割返回其中一种，不承诺点数或字典序。矩阵按值传入，原图不变。O(n^3) 时间、O(n^2) 空间；无向边总权（每条只计一次）严格小于 2^120，内部 int128。负权不适用。已逐条核验返回点集在原图上的跨割费用，暂无专门 OJ AC。

```
struct StoerWagner
{
    using I = __int128_t;

    // Symmetric nonnegative adjacency matrix; returns weight and one cut side.
    static pair<I, vector<int>> solve(vector<vector<I>> w)
    {
        int n = w.size();
        assert(n >= 2);
        vector<int> active(n);
        iota(active.begin(), active.end(), 0);
        vector<vector<int>> group(n);
        for (int i = 0; i < n; i++) group[i] = {i};
        I best = I(1) << 120;
        vector<int> side;
        while (active.size() > 1)
        {
            vector<I> dis(n);
            vector<bool> used(n);
            int prev = -1;
            for (int step = 0; step < (int)active.size(); step++)
            {
                int sel = -1;
                for (int v : active)
                    if (!used[v] && (sel == -1 || dis[v] > dis[sel])) sel = v;
```

本轮割、合并与继续扩张（接上页同一函数）：

```
                if (step + 1 == (int)active.size())
                {
                    if (dis[sel] < best)
                    {
                        best = dis[sel];
                        side = group[sel];
                    }
                    for (int v : active)
                        if (v != prev && v != sel)
                            w[prev][v] = w[v][prev] = w[prev][v] + w[sel][v];
                    group[prev].insert(
                        group[prev].end(), group[sel].begin(), group[sel].end());
                    active.erase(find(active.begin(), active.end(), sel));
                    break;
                }
                used[sel] = true;
                prev = sel;
                for (int v : active)
                    if (!used[v]) dis[v] += w[sel][v];
            }
        }
        return {best, side};
    }
};
```

模板题使用：Luogu P5632

题意：无向连通正权图的全局最小割，n 至 600、边权总和至 10^9；矩阵按无向边双向累加、自环忽略。核心返回费用与 0-based 割侧 side，本题只输出费用。核心要求 n≥2；单点没有非平凡割，驱动额外约定输出 0。

所需模板：StoerWagner（第 538 页，48.5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5632

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    using I = StoerWagner::I;
    vector<vector<I>> w(n, vector<I>(n));
    while (m--)
    {
        int u, v;
        long long c;
        cin >> u >> v >> c;
        u--;
        v--;
        if (u == v) continue;
        w[u][v] += c;
        w[v][u] += c;
    }
    // A singleton has no nontrivial cut; use zero by convention.
    if (n == 1)
    {
        cout << 0 << '\n';
        return 0;
    }
    auto [ans, side] = StoerWagner::solve(w);
    // The statement bounds the sum of edge weights by 1e9.
    cout << (long long)ans << '\n';
}
```

第四十九章　图的匹配

49.1　二分图最大匹配

相关应用：正则多重二分图的随机交替游走；概率只影响耗时，不影响返回方案正确性。

49.1.1　正则二分图完美匹配

问题描述：给定 d-正则二分图（两侧各 n 个点，每点度数都为 $d \geq 1$，可含重边），求一个完美匹配。由 Hall 定理，正则二分图一定存在完美匹配。

核心思路：随机游走找增广路。每轮随机取一个未匹配左点 u，反复：从 u 的邻接表均匀随机取一项得到右点 v；若 v 未匹配，则沿记录的路径倒序把每个左点改配到它这一步取到的右点，完成增广并结束本轮；否则走到 v 的匹配左点 `next` 继续。`path` 保存当前交替路径上的左点，`pos` 为其下标；再次走到路径上的左点时把它之后的部分弹出，消去环，保持路径简单。步数超过 `limit` 时放弃该路径重来，匹配不变。

接口契约：regular_matching(g, seed) 返回左侧各点的右侧匹配点，均为 0..n-1；g 为左侧邻接表，两侧每点度数均须等于同一正整数 d。重边按重数重复存储，不可去重；空图返回空。输入不修改。仅求一个完美匹配，不适用于一般非正则二分图或非空零度图。

从均匀随机未匹配左点开始，按邻接项均匀抽边，沿匹配边继续；path 和 pos 维护无环交替游走，遇旧左点时删去环。命中未匹配右点后倒序翻转；只在匹配全部建立后返回，不存在概率性错误答案。未匹配左点数为 k 时，每次尝试最多 4(floor(n/k)+2) 步，超限清空路径重试，不改变匹配。包含当前匹配边的采样项只形成自环，可正常消除。

理想独立均匀随机模型下，游走期望 O(n log(n+1))；加上全量度数检查，总期望 O(nd+n log(n+1))，额外空间 O(n)。固定 PRNG 种子没有确定性最坏耗时保证。seed 显式传入，测试用固定种子，赛时用时钟。随机性仅影响耗时与选择哪份正确方案；QOJ265 已以 CCF_NOI 满分通过，排名未核验。

期望界　将匹配点对收缩、原边定向左到右，加入源到未匹配左点和未匹配右点到汇各 d 条边，再识别源汇。所有点入出度平衡，源出度 kd，总出度 $(n+2k)d$；随机游走返回源的期望步数不超过 $n/k+2$。截断后重试的单次成功概率至少 $1/2$，故增广期望 $O(1+n/k)$，求和得 $O(n\log(n+1))$。路径数组只保存游走并消环，不是模拟 DFS 调用栈。

度数检查、未匹配集合与随机起步：

```
vector<int> regular_matching(const vector<vector<int>> &g, uint64_t seed)
{
    int n = g.size();
    if (!n) return {};
    int d = g[0].size();
    assert(d > 0);
    vector<int> degree(n);
    for (const auto &row : g)
    {
        assert((int)row.size() == d);
        for (int v : row)
        {
            assert(0 <= v && v < n);
            degree[v]++;
        }
    }
    for (int value : degree) assert(value == d);
    vector<int> left(n, -1), right(n, -1), free(n), pos(n, -1), path;
    iota(free.begin(), free.end(), 0);
    mt19937_64 rng(seed);
    uniform_int_distribution<int> edge(0, d - 1);
    while (!free.empty())
    {
        int index = uniform_int_distribution<int>(0, free.size() - 1)(rng);
        int u = free[index];
        path.push_back(u);
        pos[u] = 0;
        long long limit = 4LL * (n / free.size() + 2);
        while (limit--)
        {
            int v = g[u][edge(rng)];
```

交替游走消环、增广与重试（接上页函数）：

```
            int next = right[v];
            if (next == -1)
            {
                for (int i = (int)path.size() - 1; i >= 0; i--)
                {
                    int x = path[i];
                    int old = left[x];
                    left[x] = v;
                    right[v] = x;
                    v = old;
                }
                free[index] = free.back();
                free.pop_back();
                break;
            }
            if (pos[next] == -1)
            {
                pos[next] = path.size();
                path.push_back(next);
            }
            else
            {
                while ((int)path.size() > pos[next] + 1)
                {
                    pos[path.back()] = -1;
                    path.pop_back();
                }
            }
            u = next;
        }
        for (int x : path) pos[x] = -1;
        path.clear();
    }
    return left;
}
```

模板题使用：QOJ265 正则二分图匹配

题意：两侧各 n 点、每点度数 d，允许重边；输出任一完美匹配，输入输出 1-based，函数内部 0-based。

所需模板：regular_matching（第 541 页，49.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://qoj.ac/problem/265

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, d;
    cin >> n >> d;
    vector<vector<int>> g(n, vector<int>(d));
    for (auto &row : g)
        for (int &v : row)
        {
            cin >> v;
            v--;
        }
    uint64_t seed = chrono::steady_clock::now().time_since_epoch().count();
    auto match = regular_matching(g, seed);
    for (int v : match) cout << v + 1 << ' ';
    cout << '\n';
}
```

49.1.2 Hopcroft–Karp 最大匹配与最小点覆盖

问题描述：给定二分图，用 Hopcroft–Karp 求最大匹配并给出每个点的匹配对象；求完后还能构造最小点覆盖（其补集为最大独立集）。

核心思路：每个阶段 `bfs` 从所有未匹配左点出发，按“左点、右点、该右点的匹配左点”分层，未匹配的右点用虚点 0 表示，`dep[0]` 即最短增广路的层数，更深的层不再扩展。随后对每个未匹配左点 `dfs`，只走层数加一的方向并使用当前弧，找到增广路就沿途翻转匹配；搜索失败的点把 `dep` 设为无效，不再进入。由 König 定理，最后一次 BFS 后，未被交替路径到达的左点与已匹配且其匹配左点被到达的右点构成最小点覆盖。

接口契约：左右两侧分别编号 1..n、1..m，可有空侧及重边；add(u,v) 加边，solve() 返回最大匹配总数，l[u] 与 r[v] 为匹配端点，0 表示未匹配。保留已有合法匹配，可加边后再次 solve；不支持删边或手动更改匹配数组。BFS 用 dep[0] 记录最短终点层，递归 DFS 只沿最短层增广，并使用当前弧。V=n+m，单次 solve 最坏 $O((V+E)\sqrt{V})$，空间 O(V+E)。cover() 在 solve 后、下一次改图前调用，返回 (左侧点列表, 右侧点列表)，为最小点覆盖；其补集是最大独立集。覆盖与匹配方案均可直接输出。

```
struct BipartiteMatching
{
    int n, m;
    vector<vector<int>> g;
    vector<int> l, r, dep, cur;

    BipartiteMatching(int n, int m)
        : n(n), m(m), g(n + 1), l(n + 1), r(m + 1), dep(n + 1), cur(n + 1)
    {
    }

    void add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= m);
```

```
        g[u].push_back(v);
    }

    bool bfs()
    {
        queue<int> q;
        fill(dep.begin(), dep.end(), n + 1);
        fill(cur.begin(), cur.end(), 0);
        for (int u = 1; u <= n; u++)
        {
            if (!l[u])
            {
                dep[u] = 0;
                q.push(u);
            }
        }
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            if (dep[u] >= dep[0]) continue;
            for (int v : g[u])
            {
                int next = r[v];
                if (dep[next] != n + 1) continue;
                dep[next] = dep[u] + 1;
                if (next) q.push(next);
            }
        }
        return dep[0] != n + 1;
    }
```

增广、求解与最小点覆盖（接上页结构体）：

```
    bool dfs(int u)
    {
        if (!u) return true;
        for (int &i = cur[u]; i < (int)g[u].size(); i++)
        {
            int v = g[u][i];
            if (dep[r[v]] == dep[u] + 1 && dfs(r[v]))
            {
                l[u] = v;
                r[v] = u;
                return true;
            }
        }
        dep[u] = n + 1;
        return false;
    }

    // Retains an existing matching; edges may be added between calls.
    int solve()
    {
        int answer = 0;
        for (int u = 1; u <= n; u++) answer += (l[u] != 0);
        while (bfs())
        {
            for (int u = 1; u <= n; u++)
            {
                if (!l[u] && dfs(u)) answer++;
            }
        }
        return answer;
    }

    // Call after solve(), before adding more edges. Returns left/right vertex IDs.
    pair<vector<int>, vector<int>> cover() const
    {
        vector<int> left, right;
        for (int u = 1; u <= n; u++)
        {
            if (dep[u] == n + 1) left.push_back(u);
        }
        for (int v = 1; v <= m; v++)
        {
            if (r[v] && dep[r[v]] != n + 1) right.push_back(v);
        }
        return {left, right};
    }
};
```

模板题使用：Luogu P3386

题意：给定二分图，求最大匹配数。左右两侧分别编号 1..n、1..m，add(u,v) 的 u 属于左侧，v 属于右侧；solve 返回匹配数。此题不输出匹配或最小点覆盖方案。

所需模板：BipartiteMatching（第 544 页，49.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3386

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m, e;
    std::cin >> n >> m >> e;
    BipartiteMatching graph(n, m);
    while (e--)
    {
        int u, v;
        std::cin >> u >> v;
        graph.add(u, v);
    }
    std::cout << graph.solve() << '\n';
}
```

49.1.3 二分图最大匹配的可行边与必选边

问题描述：给定二分图，判断每条边是否可能属于某个最大匹配（可行边）、是否属于所有最大匹配（必选边），并返回最大匹配大小。

核心思路：先用 BipartiteMatching 求一个最大匹配，建残量图：非匹配边左到右，匹配边右到左；再加源汇处理非完备匹配：未匹配左点 $s \to u$、已匹配左点 $u \to s$，已匹配右点 $t \to v$、未匹配右点 $v \to t$。用 TarjanSCC 缩点后，一条边可行当且仅当它在当前匹配中，或两端在同一 SCC（可沿交替环或经源汇的等长交替路换入）；必选当且仅当它在匹配中且两端不在同一 SCC。平行边只把第一条与匹配一致的边当作匹配边。

代码依赖：BipartiteMatching（第 544 页，49.1.2 节）、TarjanSCC（第 475 页，43.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：matching_edges(n,m,edges) 接收左右两侧点号各为 1..n、1..m 的边表，返回 (最大匹配大小, 逐边分类)。每项为 {possible,forced}：是否属于至少一个/每一个最大匹配；分类保持输入顺序，平行边作为不同边处理。允许空侧、空图和非完备匹配。先用 BipartiteMatching 求解，再建含源汇的残量图跑 TarjanSCC；已匹配或两端同 SCC 为可行，已匹配且两端不同 SCC 为必选。不能在非完备匹配时省略源汇，也不能把可行条件写成已匹配且同 SCC。时间 $O(E\sqrt{(n+m)}+n+m)$，空间 O(n+m+E)，节点与残量边数须可用 int 表示。HK 与 SCC 都保留递归，栈深可为 O(n+m)。

```
pair<int, vector<array<bool, 2>>>
matching_edges(int n, int m, const vector<pair<int, int>> &edges)
{
    BipartiteMatching b(n, m);
    for (auto [u, v] : edges)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= m);
        b.add(u, v);
    }
    int size = b.solve(), s = n + m + 1, t = s + 1;
    TarjanSCC d(t);
    for (int u = 1; u <= n; u++)
        if (b.l[u])
```

```
            d.add(u, s);
        else
            d.add(s, u);
    for (int v = 1; v <= m; v++)
        if (b.r[v])
            d.add(t, n + v);
        else
            d.add(n + v, t);
    vector<bool> chosen(n + 1), used(edges.size());
    for (int i = 0; i < (int)edges.size(); i++)
    {
        auto [u, v] = edges[i];
        used[i] = b.l[u] == v && !chosen[u];
        if (used[i])
        {
            chosen[u] = true;
            d.add(n + v, u);
        }
        else
            d.add(u, n + v);
    }
    d.run();
    vector<array<bool, 2>> ans(edges.size());
    for (int i = 0; i < (int)edges.size(); i++)
    {
        auto [u, v] = edges[i];
        bool same = d.bel[u] == d.bel[n + v];
        ans[i] = {used[i] || same, used[i] && !same};
    }
    return {size, ans};
}
```

49.1.4 二分图最大独立集方案

问题描述：给定已知左右划分的二分图，求一个最大独立集（两两不相邻、点数最多的点集），分左右两侧输出点号。

核心思路：由 König 定理，二分图最小点覆盖大小等于最大匹配数，而独立集恰是点覆盖的补集。代码先调用 `solve` 求最大匹配，再用 `cover` 取一个最小点覆盖，把覆盖中的点在 `x`/`y` 中划掉，剩下的左右点就是最大独立集。

代码依赖：BipartiteMatching（第 544 页，49.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：independent_set(g) 接收已建好的 BipartiteMatching，内部调用 solve 再取最小点覆盖的补集，返回左右两侧的原点号列表，分别从 1 计、升序排列。允许空侧、孤立点、重边与断开图；返回一种最大基数独立集，不保证字典序或最大权。只适用于已知左右划分的二分图，不能直接输入一般图或同侧边。

调用会更新 g 的匹配与搜索状态，保留合法旧匹配并继续增广；调用后可立即读 g.cover()。允许继续 add 后再次调用，不可删除边或手改匹配。n,m 非负且小于 INT_MAX，内存足够。最大独立集大小等于 n+m 减最大匹配数；只求数量可直接使用该式，求和时先转 long long。

依赖 BipartiteMatching；总时间 $O((V+E)\sqrt{(V+1)})$，V=n+m，额外时间与空间 O(V)。HK 增广保持递归，深度最坏 O(V)。骑士攻击使格子黑白染色反转，故可应用此函数；障碍格不要建成孤立点。

```
// Maximum independent set; left/right IDs are separate and 1-based.
pair<vector<int>, vector<int>> independent_set(BipartiteMatching &g)
{
    g.solve();
    auto [a, b] = g.cover();
    vector<bool> x(g.n + 1, true), y(g.m + 1, true);
    for (int u : a) x[u] = false;
    for (int v : b) y[v] = false;
    vector<int> left, right;
    for (int u = 1; u <= g.n; u++)
    {
        if (x[u]) left.push_back(u);
    }
    for (int v = 1; v <= g.m; v++)
    {
        if (y[v]) right.push_back(v);
    }
    return {left, right};
}
```

49.2 二分图最大权匹配

49.2.1 矩形二分图最小权匹配

问题描述：给定 $n \times m$ 费用矩阵（$n \le m$），为每一行分配一个互不相同的列，使所选元素的费用和最小（即矩形二分图的最小权完美匹配/指派问题），返回最小费用和每行所选的列。

核心思路：逐行加入匹配，并维护行势 `u`、列势 `v`，使所有约化费用 $c_{ij}-u_i-v_j \ge 0$。加入第 i 行时，从虚拟列 0 出发（`p[0]=i`）做类似 Dijkstra 的搜索：`dis[j]` 是到列 j 的最小约化距离，`way[j]` 记前一列；每次取 `dis` 最小的未访问列 `j1`，把差值 `delta` 加到已访问行、从已访问列的势中减去，使新边变紧。到达未匹配列后沿 `way` 回退，逐个把匹配平移一位完成增广。全部加完后最小费用等于 $-v_0$。

接口契约：solve(cost) 输入 n×m 完整权值矩阵，0≤n≤m；每行匹配不同列，返回最小费用及列号（0-based）。允许负费用，使用 int128 势能；$O(n^2m)$。禁止边不能用未经证明的有限大常数随意替代，当前接口仅表示完整二分图。缺边建模：求最小费用完美匹配时，禁止边代价取 C>n · (最大代价 − 最小代价)（C 须在 long long 内），任何不含禁止边的方案都比含禁止边的便宜；若最优解仍用到禁止边，则不存在完美匹配。求非完美的最大权匹配时，把权取负、缺边与负权边记 0，并补零列使 n≤m，选中零边即未匹配。

```
struct Hungarian
{
    using I = __int128_t;

    // Minimum cost injection from n rows to m columns, n<=m. Zero-based matrix.
    static pair<I, vector<int>> solve(const vector<vector<long long>> &cost)
    {
        int n = cost.size();
        if (!n) return {0, {}};
        int m = cost[0].size();
        assert(n <= m);
        vector<I> u(n + 1), v(m + 1);
        vector<int> p(m + 1), way(m + 1);
        const I inf = I(1) << 120;
        for (int i = 1; i <= n; i++)
        {
            assert((int)cost[i - 1].size() == m);
            p[0] = i;
            int j0 = 0;
            vector<I> dis(m + 1, inf);
            vector<bool> used(m + 1);
            do
            {
                used[j0] = true;
                int i0 = p[j0], j1 = 0;
                I delta = inf;
                for (int j = 1; j <= m; j++)
                    if (!used[j])
                    {
                        I cur = I(cost[i0 - 1][j - 1]) - u[i0] - v[j];
                        if (cur < dis[j])
                        {
                            dis[j] = cur;
                            way[j] = j0;
                        }
                        if (dis[j] < delta)
                        {
                            delta = dis[j];
                            j1 = j;
                        }
                    }
                for (int j = 0; j <= m; j++)
                    if (used[j])
                    {
                        u[p[j]] += delta;
                        v[j] -= delta;
                    }
                    else
                        dis[j] -= delta;
                j0 = j1;
            } while (p[j0]);
            do
            {
                int j1 = way[j0];
                p[j0] = p[j1];
```

```
                j0 = j1;
            } while (j0);
        }
        vector<int> ans(n);
        for (int j = 1; j <= m; j++)
            if (p[j]) ans[p[j] - 1] = j - 1;
        return {-v[0], ans};
    }
};
```

模板题使用：Library Checker assignment

题意：给完整方阵，为每一行分配一个不同的列，使费用和最小。返回列号是 0-based，按行顺序输出。题目 n≤500、单项绝对值 ≤10^9，总费用可由 int128 转为 long long；缺边问题不能直接套本例。

所需模板：Hungarian（第 549 页，49.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/assignment

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<vector<long long>> cost(n, vector<long long>(n));
    for (auto &row : cost)
        for (auto &x : row) cin >> x;
    auto [value, columns] = Hungarian::solve(cost);
    cout << (long long)value << '\n';
    for (int j : columns) cout << j << ' ';
    cout << '\n';
}
```

49.3 一般图最大匹配

49.3.1 一般图最大基数匹配：带花树

问题描述：一般图（不要求二分）最大匹配：给定无向图，选出尽量多两两不共端点的边，返回匹配边数及每个点的配对点。

核心思路：Edmonds 带花树。依次以每个未匹配点为根做 BFS 建交错树：队列中的点是外点（偶层），`pre[v]` 记奇层点 v 是从哪个外点走到的，`base` 记点所在花的花根。从外点 u 看边 u–v：若 v 也是外点，两条到根的交错路径加这条边构成奇环，用 `lca` 找花根，`mark` 沿环补齐 `pre`，把环上各点的 `base` 改为花根，环上原内点变为外点入队；若 v 未访问且未匹配，找到增广路，沿 `pre`/`mate` 交替翻转；若 v 已匹配，v 成为内点，其配偶成为外点入队。某点一次找不到增广路，以后也不会再有，故每点只试一次。

思路中的情况	代码中的处理	行
逐点尝试增广	对每个仍未匹配的点做一次 BFS	108
无用的边	同一朵花内部的边或已是匹配边，跳过	59
外点连外点	出现奇环：求花根、补 `pre`、缩花，新外点入队	60
到达未匹配点	沿 `pre` 与 `mate` 回溯翻转整条增广路	80
到达已匹配点	v 作内点，其配偶作外点入队	92

接口契约： Blossom(n)，点号 0..n-1；add(u,v) 加无向边，忽略自环，允许重边。solve() 从空匹配重新计算，返回匹配边数；mate[u] 为配对点，未匹配为 -1。结果支持直接输出方案。缩花维护交错树，单次增广 $O(n^2+m)$，总计 $O(n^3+nm)$，简单图为 $O(n^3)$；空间 O(n+m)。重复调用 solve 或加边后重算均可。不处理边权，不能代替最大权匹配。

```
struct Blossom
{
    int n;
    vector<vector<int>> g;
    vector<int> mate, pre, base, q;
    vector<bool> outer, flower, seen;

    Blossom(int n)
        : n(n), g(n), mate(n, -1), pre(n), base(n), outer(n), flower(n), seen(n)
    {
    }

    void add(int u, int v)
    {
        assert(0 <= u && u < n && 0 <= v && v < n);
        if (u == v) return;
        g[u].push_back(v);
        g[v].push_back(u);
    }

    int lca(int u, int v)
    {
        fill(seen.begin(), seen.end(), false);
        while (true)
        {
            u = base[u];
            seen[u] = true;
            if (mate[u] == -1) break;
            u = pre[mate[u]];
        }
        while (!seen[base[v]]) v = pre[mate[base[v]]];
        return base[v];
    }

    void mark(int u, int b, int v)
    {
        while (base[u] != b)
        {
            flower[base[u]] = true;
            flower[base[mate[u]]] = true;
            pre[u] = v;
            v = mate[u];
            u = pre[v];
        }
    }

    bool augment(int root)
    {
        fill(outer.begin(), outer.end(), false);
        fill(pre.begin(), pre.end(), -1);
        iota(base.begin(), base.end(), 0);
```

```
        q.assign(1, root);
        outer[root] = true;
        for (int head = 0; head < (int)q.size(); head++)
        {
            int u = q[head];
            for (int v : g[u])
            {
                if (base[u] == base[v] || mate[u] == v) continue;
                if (v == root || (mate[v] != -1 && pre[mate[v]] != -1))
                {
                    int b = lca(u, v);
                    fill(flower.begin(), flower.end(), false);
                    mark(u, b, v);
                    mark(v, b, u);
                    for (int x = 0; x < n; x++)
                    {
                        if (!flower[base[x]]) continue;
                        base[x] = b;
                        if (!outer[x])
                        {
                            outer[x] = true;
                            q.push_back(x);
                        }
                    }
                }
                else if (pre[v] == -1)
                {
                    pre[v] = u;
                    if (mate[v] == -1)
                    {
                        while (v != -1)
                        {
                            int p = pre[v];
                            int next = mate[p];
                            mate[v] = p;
                            mate[p] = v;
                            v = next;
                        }
                        return true;
                    }
                    int x = mate[v];
                    outer[x] = true;
                    q.push_back(x);
                }
            }
        }
        return false;
    }

    // Rebuild the maximum matching; mate[u] is -1 for an unmatched vertex.
    int solve()
```

```
    {
        fill(mate.begin(), mate.end(), -1);
        int answer = 0;
        for (int u = 0; u < n; u++)
        {
            if (mate[u] == -1 && augment(u)) answer++;
        }
        return answer;
    }
};
```

模板题使用：Library Checker general_matching

题意：给简单无向图，输出最大匹配的边数及匹配边。点号 0..n-1；solve 后 mate[u]=-1 表示未匹配，只在 u<mate[u] 时输出，避免同一条边打印两次。

所需模板：Blossom（第 551 页，49.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/general_matching

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    Blossom graph(n);
    while (m--)
    {
        int u, v;
        cin >> u >> v;
        graph.add(u, v);
    }
    cout << graph.solve() << '\n';
    for (int u = 0; u < n; u++)
        if (u < graph.mate[u]) cout << u << ' ' << graph.mate[u] << '\n';
}
```

49.4 一般图最大权匹配

49.4.1 一般图最大权匹配：带权带花树

问题描述：一般图最大权匹配：给定无向带权图，选出两两不共端点的边集使边权和最大（可以不是完美匹配）；也可选择先使匹配边数最多、再使权和最大，用于求最大权完美匹配。返回总权、匹配边数和每个点的配对点。

核心思路：带权带花树（原始–对偶）。每个点和每朵花有顶标 `label`，边的松弛量 $\mathrm{gap} = \ell_u + \ell_v - 2w$，为 0 的是紧边；只沿紧边扩展交错森林。`state` 为 0、1、−1 分别表示外点、内点、未标记。外点经紧边连到未标记点就把它标为内点、其配偶标为外点；两外点相连时，在同一棵树中则缩花（`shrink`），在不同树中则沿两侧增广。无紧边可用时按 `slack` 求 `delta`，外点顶标减 δ、内点顶标加 δ，外花顶标加 2δ、内花减 2δ；外点顶标降为 0 时再增广不会增加权值，结束；内花顶标为 0 时展开。基数优先模式给每条边加足够大的 `bonus`；只保留正权边，初始顶标取最大边权并贪心匹配最大权紧边。

思路中的情况	代码中的处理	行
建图	基数优先时每条边加 `bonus`，只保留加后为正的边	286
新一轮搜索	所有未匹配的外层点（或花）作为交错森林的根入队	205
紧边连到未标记点	标为内点，其配偶标为外点入队	176
紧边连接两个外点	同树则缩花，不同树则两侧增广	189
没有可用紧边	取各类松弛量的最小值作为 `delta` 并调整顶标	229
外点顶标降为 0	当前匹配已最优，本轮返回失败	248
内花顶标为 0	展开该花，恢复内部点的标记	265

接口契约：WeightedBlossom(n)，点 1..n，0≤n<INT_MAX/2。add_edge(u,v,w) 添加 signed64 边权，自环忽略，重边取最大。solve() 只最大化总权，允许空匹配，不保证同权时边数最多；solve(true) 先最大化匹配边数，再最大化原权。返回 weight（__int128）、pairs、mate（未匹配为 0）。求最大权完美匹配用 solve(true)，并检查 2*pairs==n。

每次 solve 从头计算，支持重复、切换模式、加边后重算。$O(n^3)$ 时间、$O(n^2)$ 空间，稠密矩阵适合数百点；不因接口允许大 n 而保证能装入内存。内部成员仅供核验，不应直接修改。缩花、展开与增广使用递归；全范围 signed64 权值以 128 位对偶计算。

改编自 Shahjalal Shohag 的 MIT 实现，完整许可见后页。P6699 已 100 分通过；负权与基数优先模式由独立本地验证覆盖。

字段与加边（同一结构体内连续抄写）：

```
struct WeightedBlossom
{
    using I = __int128;
    using ll = long long;
    static constexpr I INF = I(1) << 126;
    struct Edge
    {
        int u = 0, v = 0;
        I w = 0;
    };
    struct Result
    {
        I weight = 0;
        int pairs = 0;
        vector<int> mate;
    };
    int n, top = 0;
    vector<vector<I>> weight;
    vector<vector<Edge>> g;
    vector<vector<int>> flower, from;
    vector<int> match, slack, base, parent, state, seen;
    vector<I> label;
    queue<int> q;

    explicit WeightedBlossom(int n) : n(n), weight(n + 1, vector<I>(n + 1, -INF))
    {
        assert(0 <= n && n < INT_MAX / 2);
    }
    void add_edge(int u, int v, ll w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        if (u == v) return;
        weight[u][v] = max(weight[u][v], I(w));
        weight[v][u] = weight[u][v];
    }
```

松弛与递归代表元（同一结构体内连续抄写）：

```
I gap(const Edge &e) const
{
    return label[e.u] + label[e.v] - 2 * e.w;
}
void relax(int u, int x)
{
    if (!slack[x] || gap(g[u][x]) < gap(g[slack[x]][x])) slack[x] = u;
}
void reset_slack(int x)
{
    slack[x] = 0;
    for (int u = 1; u <= n; u++)
        if (g[u][x].w > 0 && base[u] != x && state[base[u]] == 0)
            relax(u, x);
}
void enqueue(int x)
{
    if (x <= n) q.push(x);
    else for (int v : flower[x]) enqueue(v);
}
void set_base(int x, int b)
{
    base[x] = b;
    if (x > n)
        for (int v : flower[x]) set_base(v, b);
}
int position(int b, int x)
{
    int k = find(flower[b].begin(), flower[b].end(), x) - flower[b].begin();
    if (k % 2 == 0) return k;
    reverse(flower[b].begin() + 1, flower[b].end());
    return flower[b].size() - k;
}
```

花内配对、增广与公共祖先（同一结构体内连续抄写）：

```
    void pair_with(int u, int v)
    {
        Edge e = g[u][v];
        match[u] = e.v;
        if (u <= n) return;
        int x = from[u][e.u];
        int k = position(u, x);
        for (int i = 0; i < k; i++) pair_with(flower[u][i], flower[u][i ^ 1]);
        pair_with(x, v);
        rotate(flower[u].begin(), flower[u].begin() + k, flower[u].end());
    }
    void augment(int u, int v)
    {
        int next = base[match[u]];
        pair_with(u, v);
        if (!next) return;
        pair_with(next, base[parent[next]]);
        augment(base[parent[next]], next);
    }
    int lca(int u, int v)
    {
        fill(seen.begin(), seen.end(), 0);
        while (u || v)
        {
            if (u)
            {
                if (seen[u]) return u;
                seen[u] = 1;
                u = base[match[u]];
                if (u) u = base[parent[u]];
            }
            swap(u, v);
        }
        return 0;
    }
```

缩花及原始端点映射（同一结构体内连续抄写）：

```
void shrink(int u, int root, int v)
{
    int b = n + 1;
    while (b <= top && base[b]) b++;
    if (b > top) top++;
    label[b] = 0;
    state[b] = 0;
    match[b] = match[root];
    flower[b] = {root};
    for (int end : {u, v})
    {
        for (int x = end; x != root; )
        {
            int y = base[match[x]];
            flower[b].push_back(x);
            flower[b].push_back(y);
            enqueue(y);
            x = base[parent[y]];
        }
        if (end == u) reverse(flower[b].begin() + 1, flower[b].end());
    }
    set_base(b, b);
    for (int x = 1; x <= top; x++)
    {
        g[b][x] = {};
        g[x][b] = {};
    }
    fill(from[b].begin(), from[b].end(), 0);
    for (int child : flower[b])
    {
        for (int x = 1; x <= top; x++)
            if (g[child][x].w > 0 &&
                (!g[b][x].w || gap(g[child][x]) < gap(g[b][x])))
            {
                g[b][x] = g[child][x];
                g[x][b] = g[x][child];
            }
        for (int x = 1; x <= n; x++)
            if (from[child][x]) from[b][x] = child;
    }
    reset_slack(b);
}
```

展开与紧边处理（同一结构体内连续抄写）：

```
    void expand(int b)
    {
        for (int x : flower[b]) set_base(x, x);
        int entry = from[b][g[b][parent[b]].u];
        int k = position(b, entry);
        for (int i = 0; i < k; i += 2)
        {
            int x = flower[b][i];
            int y = flower[b][i + 1];
            parent[x] = g[y][x].u;
            state[x] = 1;
            state[y] = 0;
            slack[x] = 0;
            reset_slack(y);
            enqueue(y);
        }
        state[entry] = 1;
        parent[entry] = parent[b];
        for (int i = k + 1; i < (int)flower[b].size(); i++)
        {
            int x = flower[b][i];
            state[x] = -1;
            reset_slack(x);
        }
        base[b] = 0;
    }
    bool use_edge(Edge e)
    {
        int u = base[e.u];
        int v = base[e.v];
        if (state[v] == -1)
        {
            parent[v] = e.u;
            state[v] = 1;
            int next = base[match[v]];
            slack[v] = 0;
            slack[next] = 0;
            state[next] = 0;
            enqueue(next);
        }
        else if (state[v] == 0)
        {
            int root = lca(u, v);
            if (root) shrink(u, root, v);
            else
            {
                augment(u, v);
                augment(v, u);
                return true;
            }
        }
        return false;
    }
```

交替森林搜索（同一结构体内连续抄写）：

```
bool search()
{
    fill(state.begin(), state.end(), -1);
    fill(slack.begin(), slack.end(), 0);
    q = {};
    for (int x = 1; x <= top; x++)
        if (base[x] == x && !match[x])
        {
            parent[x] = 0;
            state[x] = 0;
            enqueue(x);
        }
    if (q.empty()) return false;
    while (true)
    {
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            if (state[base[u]] == 1) continue;
            for (int v = 1; v <= n; v++)
                if (g[u][v].w > 0 && base[u] != base[v])
                {
                    if (gap(g[u][v]) == 0)
                    {
                        if (use_edge(g[u][v])) return true;
                    }
                    else relax(u, base[v]);
                }
        }
```

对偶更新与继续搜索（同一结构体内连续抄写）：

```
            I delta = INF;
            for (int b = n + 1; b <= top; b++)
                if (base[b] == b && state[b] == 1)
                    delta = min(delta, label[b] / 2);
            for (int x = 1; x <= top; x++)
                if (base[x] == x && slack[x])
                {
                    I d = gap(g[slack[x]][x]);
                    if (state[x] == -1) delta = min(delta, d);
                    if (state[x] == 0) delta = min(delta, d / 2);
                }
            for (int u = 1; u <= n; u++)
                if (state[base[u]] == 0) delta = min(delta, label[u]);
            bool finished = false;
            for (int u = 1; u <= n; u++)
            {
                if (state[base[u]] == 0)
                {
                    label[u] -= delta;
                    if (label[u] == 0) finished = true;
                }
                else if (state[base[u]] == 1) label[u] += delta;
            }
            for (int b = n + 1; b <= top; b++)
                if (base[b] == b)
                {
                    if (state[b] == 0) label[b] += 2 * delta;
                    if (state[b] == 1) label[b] -= 2 * delta;
                }
            if (finished) return false;
            q = {};
            for (int x = 1; x <= top; x++)
                if (base[x] == x && slack[x] && base[slack[x]] != x &&
                    gap(g[slack[x]][x]) == 0)
                    if (use_edge(g[slack[x]][x])) return true;
            for (int b = n + 1; b <= top; b++)
                if (base[b] == b && state[b] == 1 && label[b] == 0) expand(b);
        }
    }
```

两种目标、初始化与方案返回（同一结构体内连续抄写）：

```
    Result solve(bool cardinality_first = false)
    {
        int size = 2 * n + 1;
        top = n;
        g.assign(size, vector<Edge>(size));
        flower.assign(size, {});
        from.assign(size, vector<int>(n + 1));
        match.assign(size, 0);
        slack.assign(size, 0);
        base.assign(size, 0);
        parent.assign(size, 0);
        state.assign(size, -1);
        seen.assign(size, 0);
        label.assign(size, 0);
        I bound = 0;
        for (int u = 1; u <= n; u++)
            for (int v = u + 1; v <= n; v++)
                if (weight[u][v] != -INF) bound = max(bound, max(weight[u][v], -weight[u][v]));
        I bonus = cardinality_first ? n * bound + 1 : 0;
        I maximum = 0;
        for (int u = 1; u <= n; u++)
        {
            base[u] = u;
            from[u][u] = u;
            for (int v = 1; v <= n; v++)
                if (weight[u][v] != -INF && weight[u][v] + bonus > 0)
                {
                    g[u][v] = {u, v, weight[u][v] + bonus};
                    maximum = max(maximum, g[u][v].w);
                }
        }
        fill(label.begin() + 1, label.begin() + n + 1, maximum);
        Result result;
        for (int u = 1; u <= n; u++)
            if (!match[u] && maximum > 0)
                for (int v = u + 1; v <= n; v++)
                    if (!match[v] && g[u][v].w == maximum)
                    {
                        match[u] = v;
                        match[v] = u;
                        result.pairs++;
                        break;
                    }
        while (search()) result.pairs++;
        result.mate.assign(match.begin(), match.begin() + n + 1);
        for (int u = 1; u <= n; u++)
            if (result.mate[u] > u) result.weight += weight[u][result.mate[u]];
        return result;
    }
};
```

目标与证书 基数优先模式给每条存在边加 $B = nW + 1$，W 为最大绝对原权；任意两个匹配原权差不超过 nW，所以多一条边严格占优。求完美匹配必须选择此模式再检查 $2 \cdot pairs = n$。顶点标签与奇集标签给出匹配 LP 的对偶上界；边约束可行、匹配边紧、未匹配顶点标签零以及正标签花饱和使上下界相等。所有标签应完成更新后再终止，不能留下部分更新。

验证范围 六种构建各 28546 次子集 DP、方案和对偶检查；400 点 16 次大图证书检查通过 O2 与 ASan/UBSan。实际触发嵌套缩花与展开。P6699 记录 302632540，20 点全 AC，最慢 209ms、最高 24.02MB；负权、基数优先及完美判断为本地证据，排名未核验。

来源及 MIT 许可 改编自 Shahjalal Shohag 的带权带花树，固定提交 0884be6ab81f（完整提交号见仓库来源记录）。以下许可随本实现保留：

Copyright (c) 2021 Shahjalal Shohag License: MIT

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the ”Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED ”AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

模板题使用：P6699 一般图最大权匹配

题意：n≤400、m≤79800、1≤w≤5×10^8。输出最大总权及每点的匹配点，未匹配为 0。答案 ≤10^{11}，可转 long long；方案不唯一。当前源码记录 302632540，20 点全 AC。

所需模板：WeightedBlossom（第 555 页，49.4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6699

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    WeightedBlossom g(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        g.add_edge(u, v, w);
    }
    auto result = g.solve();
    cout << (long long)result.weight << '\n';
    for (int u = 1; u <= n; u++) cout << result.mate[u] << ' ';
    cout << '\n';
}
```

第五十章　Prüfer 序列

50.1　Prüfer 双射与有标号树计数

顶点标号 $1,\dots,n$ 的有标号树，$n\ge 2$：反复删最小标号叶子并记录其邻点，直到剩两点，得长度 $n-2$ 的序列。$n=2$ 为空序列；$n=1$ 单独处理，不代入含 $n-2$ 的公式。

50.1.1　双射为何成立

解码：读下一项 u，取未删除且不在剩余序列中的最小标号 v，连 uv 并删 v；最后连剩下两点。

$$d_i = 1 + \#\{\text{序列中等于} i \text{ 的项}\}.$$

50.1.2　Cayley、指定度数与叶子数

树总数 n^{n-2}。度数 $d_1,\dots,d_n$：若某 $d_i<1$ 或 $\sum_i d_i \ne 2n-2$ 则为 0，否则

$$\frac{(n-2)!}{\prod_{i=1}^{n}(d_i-1)!}.$$

恰有 ℓ 个叶子：

$$\binom{n}{\ell}\,(n-\ell)!\,S(n-2,n-\ell),$$

$n\ge 3$ 时 $2\le \ell\le n-1$，其余为 0；$n=2,\ell=2$ 得 1。取模时阶乘分母须可逆，素数模见第 20.1.1 节（第 189 页）。

50.1.3 加权 Cayley 与连接已有连通块

完全图边权 $w_{ij} = x_i x_j$ 时

$$\sum_T \prod_{ij \in T} w_{ij} = \sum_T \prod_i x_i^{d(i)} = \left(\prod_i x_i \right) \left(\sum_i x_i \right)^{n-2},$$

对零权、负权及任意正模数成立；$n = 2$ 时零次幂为 1。一般边权或缺边图用矩阵树定理，见第 51.3 节（第 575 页）及第 51.2 节（第 573 页）。

$k \geq 2$ 个连通块大小 $s_1, \ldots, s_k$，$N = \sum_i s_i$，恰加 $k-1$ 条块间边使其连通的无序边集数为

$$N^{k-2} \prod_{i=1}^{k} s_i;$$

$k = 1$ 时为 1。禁止部分端点对、限制新增边或计执行顺序时不适用。有标号树数不能除以 $n!$ 得无标号树数。

第五十一章　矩阵树定理

相关应用：生成树计数的相关扩展：Tutte 多项式，T(1,1) 对应连通图生成树数；不将其冒称为矩阵树定理实现。

51.1　Tutte 多项式求值

问题描述：给定简单无向图 $G=(V,E)$ 与 x,y，求 Tutte 多项式的值 $T_G(x,y)=\sum_{A\subseteq E}(x-1)^{k(A)-k(E)}(y-1)^{k(A)+|A|-|V|}$ 模 998244353，其中 $k(A)$ 是生成子图 (V,A) 的连通块数。生成树（森林）计数、色多项式、可靠性多项式等都是它的特例。

核心思路：各连通分量的值相乘。对一个连通分量，令 $c[S]$ 为 S 上所有连通生成子图按 $(y-1)^{\text{环秩}}$ 计权之和。按 S 的最小顶点 r 从大到小计算：删去 r 后 $S\setminus\{r\}$ 分成若干连通块 T，块内贡献 $c[T]$；r 连向 T 的 h 条边至少选一条，权为 $\sum_{j\ge1}\binom{h}{j}(y-1)^{j-1}=1+y+\cdots+y^{h-1}$（`geometric`）。于是 $c[S]$ 是集合幂级数 $a[T]=c[T]\cdot\mathrm{geometric}[h]$ 的 exp。最后把顶点集划分成若干连通块，顶点 0 所在块以外每块乘 $(x-1)$，再做一次 exp 求和。

思路中的情况	代码中的处理	行
拆连通分量	DFS 找出分量并重编号为位掩码图，结果逐个相乘	103
按最小点 r 求 c	r 从大到小，只涉及编号大于 r 的子集，所需 c 已算好	63
r 与块 T 的连接权	$c[T]$ 乘上 $1+y+\cdots+y^{h-1}$，h 为 r 到 T 的边数	70
集合幂级数 exp	按 popcount 分层做子集和变换，每个位置做形式幂级数 exp（$g_i=\frac{1}{i}\sum_j j a_j g_{i-j}$），再反演	42
$x=1$	只有单块划分有贡献，直接返回整个分量的 c	75
划分顶点集	其余块乘 $x-1$ 后做 exp，与顶点 0 所在块 c 配对求和	82

接口契约：tutte(adj,x,y) 返回模 998244353 的 Tutte 多项式值。简单无向图，n≤21，点号 0..n-1；adj[u] 为邻居位掩码，须对称、无自环、无越界位。0≤x,y<998244353；支持断连、孤立点及空图（值 1），不支持重边或自环；输入不修改。连通分量 DFS 保持递归。

每个连通分量独立计算并相乘。c[S] 为 S 上连通生成子图按 (y-1) 的环秩计权之和；删去 S 的最小顶点 r 后，每个剩余连通块 T 的连接权为 1+y+...+y^(h-1)，h 为 r 连向 T 的边数，h=0 时为 0。用快速子集指数求集合划分。最后固定顶点 0 所在块，其余各块乘 (x-1)。不除 x-1 或 y-1，x=1、y=1 均直接支持；普通形式指数只需 1..20 的模逆元。时间 O(n^2 2^n)，峰值空间 O(n2^n)。21 点 Linux 九次连续求值进程峰值约 312MiB；QOJ45 已以 CCF_NOI 获得满分；排名尚未核验。

集合指数为何正确　按集合大小分次数层，zeta 后做普通形式指数，再 Mobius 并取集合大小次项。逆变换只保留并集恰为 S 的项，次数等于 $|S|$ 排除重叠；$1/k!$ 抵消 k 个块的排列。最小顶点从大到小处理时，余下块的 c 值已知。最终固定 0 所在块，其他块各乘 $x-1$，恰好给出定义中的连通分量指数。

输入约定、等比和与分层 zeta：

```
int tutte(const vector<unsigned> &adj, int x, int y)
{
    constexpr int mod = 998244353;
    int n = adj.size();
    assert(n <= 21 && 0 <= x && x < mod && 0 <= y && y < mod);
    for (int u = 0; u < n; u++)
    {
        assert((adj[u] >> n) == 0 && !(adj[u] >> u & 1));
        for (int v = 0; v < n; v++)
            assert((adj[u] >> v & 1) == (adj[v] >> u & 1));
    }
    vector<int> inv(n + 1, 1), geometric(n + 1);
    for (int i = 2; i <= n; i++)
        inv[i] = mod - (long long)(mod / i) * inv[mod % i] % mod;
    for (int i = 1; i <= n; i++)
        geometric[i] = ((long long)geometric[i - 1] * y + 1) % mod;
    auto exponential = [&](const vector<int> &a)
    {
        int size = a.size();
        int k = countr_zero((unsigned)size), width = k + 1;
        vector<int> f((size_t)size * width), g(width), answer(size);
        for (int s = 1; s < size; s++) f[(size_t)s * width + popcount((unsigned)s)] = a[s];
        for (int bit = 1; bit < size; bit <<= 1)
            for (int s = 0; s < size; s++)
                if (s & bit)
                    for (int j = 1; j <= k; j++)
                    {
                        int &value = f[(size_t)s * width + j];
                        value += f[(size_t)(s ^ bit) * width + j];
                        if (value >= mod) value -= mod;
                    }
```

形式指数与 Mobius（接上页函数）：

```
        for (int s = 0; s < size; s++)
        {
            int *row = f.data() + (size_t)s * width;
            g[0] = 1;
            for (int i = 1; i <= k; i++) row[i] = (long long)row[i] * i % mod;
            for (int i = 1; i <= k; i++)
            {
                long long sum = 0;
                for (int j = 1; j <= i; j++)
                    sum = (sum + (long long)row[j] * g[i - j]) % mod;
                g[i] = sum * inv[i] % mod;
            }
            copy(g.begin(), g.end(), row);
        }
        for (int bit = 1; bit < size; bit <<= 1)
            for (int s = 0; s < size; s++)
                if (s & bit)
                    for (int j = 0; j <= k; j++)
                    {
                        int &value = f[(size_t)s * width + j];
                        value -= f[(size_t)(s ^ bit) * width + j];
                        if (value < 0) value += mod;
                    }
        for (int s = 0; s < size; s++) answer[s] = f[(size_t)s * width + popcount((unsigned)s)];
        return answer;
    };
```

连通子集递推及分量乘积（接上页函数）：

```
    auto connected = [&](const vector<unsigned> &g)
    {
        int m = g.size();
        int size = 1 << m;
        vector<int> c(size);
        for (int root = m - 1; root >= 0; root--)
        {
            int count = 1 << (m - root - 1);
            vector<int> a(count);
            for (int s = 1; s < count; s++)
            {
                unsigned t = (unsigned)s << (root + 1);
                a[s] = (long long)c[t] * geometric[popcount(g[root] & t)] % mod;
            }
            auto e = exponential(a);
            for (int s = 0; s < count; s++) c[(s << (root + 1)) | (1 << root)] = e[s];
        }
        if (x == 1) return c.back();
        vector<int> a(size / 2);
        int z = x ? x - 1 : mod - 1;
        for (int s = 1; s < size / 2; s++) a[s] = (long long)c[s << 1] * z % mod;
        auto e = exponential(a);
        long long answer = 0;
        for (int s = 0; s < size / 2; s++)
            answer = (answer + (long long)c[(size - 1) ^ (s << 1)] * e[s]) % mod;
        return (int)answer;
    };
    vector<int> seen(n), vertices;
    auto dfs = [&](auto &&self, int u) -> void
    {
        seen[u] = 1;
        vertices.push_back(u);
        for (int v = 0; v < n; v++)
            if ((adj[u] >> v & 1) && !seen[v]) self(self, v);
    };
    long long answer = 1;
    for (int u = 0; u < n; u++)
        if (!seen[u])
        {
            vertices.clear();
            dfs(dfs, u);
            vector<unsigned> g(vertices.size());
            for (int i = 0; i < (int)vertices.size(); i++)
                for (int j = 0; j < (int)vertices.size(); j++)
                    if (adj[vertices[i]] >> vertices[j] & 1) g[i] |= 1U << j;
            answer = answer * connected(g) % mod;
        }
    return answer;
}
```

模板题使用：QOJ45 Tutte 多项式

题意：输入简单无向图邻接矩阵和 x,y，输出模 998244353 的 Tutte 多项式值；n≤21。

所需模板：tutte（第 568 页，51.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://qoj.ac/problem/45

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<unsigned> adj(n);
    for (int u = 0; u < n; u++)
        for (int v = 0; v < n; v++)
        {
            int edge;
            cin >> edge;
            if (edge) adj[u] |= 1U << v;
        }
    int x, y;
    cin >> x >> y;
    cout << tutte(adj, x, y) << '\n';
}
```

51.2 任意正模数带权矩阵树

问题描述：给定带权多重图，求所有生成树（无向）或以指定点为根的内向/外向生成树的边权乘积之和，对任意正整数取模。边权为 1 时即生成树个数。

核心思路：矩阵树定理：构造拉普拉斯矩阵 $L = D - A$（有向时 D 取入度或出度），删去根所在的行和列，余子式的行列式就是答案。合数模数下没有逆元，行列式交给按欧几里得辗转相减消元的 `determinant_mod`。

代码依赖：determinant_mod（第 220 页，21.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：count(n, edges, root, kind, mod) 返回树的边权乘积之和对 mod 的最小非负余数。mod 为任意正 signed 64 位数，边权支持完整 signed 64 位。Kind 的 undirected、toward_root、away_from_root 分别表示无向、内向和外向树；点号 0..n-1。自环忽略，重边按不同边计数，单点返回 1%mod，mod=1 返回零。先在 128 位中构造模拉普拉斯余子式，再调用 determinant_mod；$O(n^3 \log \mathrm{mod} + m)$ 时间、$O(n^2)$ 额外空间。不使用模逆元，素数与合数模数通用。

```
struct MatrixTreeMod
{
    using ll = long long;
    using Edge = tuple<int, int, ll>;
    enum class Kind
    {
        undirected,
        toward_root,
        away_from_root
    };

    static ll count(int n, const vector<Edge> &edges, int root, Kind kind, ll mod)
    {
        assert(n >= 1 && root >= 0 && root < n && mod >= 1);
        vector<vector<ll>> lap(n - 1, vector<ll>(n - 1));
        auto add = [&](int u, int v, ll w)
        {
            if (u == root) return;
            int i = u - (u > root), j = v - (v > root);
            lap[i][i] = ((__int128)lap[i][i] + w) % mod;
            if (v != root) lap[i][j] = ((__int128)lap[i][j] + mod - w) % mod;
        };
        for (auto [u, v, w] : edges)
        {
            assert(u >= 0 && u < n && v >= 0 && v < n);
            if (u == v) continue;
            w %= mod;
            if (w < 0) w += mod;
            if (kind == Kind::away_from_root) swap(u, v);
            add(u, v, w);
            if (kind == Kind::undirected) add(v, u, w);
        }
        return determinant_mod(move(lap), mod);
    }
};
```

模板题使用：Library Checker Counting Spanning Trees (Undirected)

题意：无向多重图生成树计数，模 998244353；点号 0..n-1，1≤n≤500、0≤m≤500000，允许自环和重边。每条原边权重设 1，自环不参与树，重边是不同选择。删去任意根的拉普拉斯行列，这里取根 0；n=1 的空树计 1，断连计 0。此接口也支持合数模数，复杂度 $O(n^3 \log \mathrm{mod})$，本题固定质数亦可选更快的 MatrixTree 版本。

所需模板：determinant_mod（第 220 页，21.2.1 节）、MatrixTreeMod（第 573 页，51.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/counting_spanning_tree_undirected

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<MatrixTreeMod::Edge> e;
    e.reserve(m);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        e.emplace_back(u, v, 1);
    }
    cout << MatrixTreeMod::count(n, e, 0, MatrixTreeMod::Kind::undirected, 998244353)
         << '\n';
}
```

51.3 带权矩阵树定理与方向约定

结果为所有生成树的边权乘积之和；无权计数令边权为 1。重边是不同选择，自环不属于任何生成树，单点为 1。零权、负权按代数运算处理，结果为零不能作为连通性判定。

无向边 uv 权 w：L_{uu}, L_{vv} 各加 w，L_{uv}, L_{vu} 各减 w；删除 root 行列求行列式，与 root 无关。内向树形图（每点沿边走到根）：边 $u \to v$ 贡献 $L_{uu} \mathrel{+}= w$、$L_{uv} \mathrel{-}= w$，删根行列。外向树形图先反转所有边再用同一构造。只构造 $(n-1) \times (n-1)$ 余子式，时间 $O(n^3 + m)$、额外空间 $O(n^2)$；恒等式不需要除法，模数须为素数只是模逆元消元的要求。

51.4 任意正模数的矩阵树计数

实现见第 51.2 节（第 573 页）。沿用第 51.3 节的三种构造，行列式改用欧几里得消元（第 21.2.1 节，第 220 页）。接口 `MatrixTreeMod::count`，边权和正模数均为 signed 64 位；时间 $O(n^3 \log(m+1) + |E|)$、空间 $O(n^2)$，m 为模数。边权先正规化到 $[0, m)$，矩阵加减先扩展到 128 位再取模；重边权和不能先累加到普通整数。单点返回 $1 \bmod m$，模 1 返回零；返回零不代表没有生成树。

第五十二章　LGV 引理

52.1　LGV：DAG 路径权值行列式

问题描述：给定带权 DAG 和等长的起点序列 $a_1,\dots,a_k$、终点序列 $b_1,\dots,b_k$，求矩阵 $M_{ij}=e(a_i,b_j)$ 的行列式模素数，其中 $e(a,b)$ 是 a 到 b 所有路径边权乘积之和。由 LGV 引理，它等于顶点不交路径组按配对排列的符号加权求和；若只有恒等配对能不交，它就是不交路径组的权值和。

核心思路：Kahn 拓扑排序，同时判断是否有环。对每个起点 a_i 做一次拓扑序 DP：$dp[a_i]=1$，按拓扑序把 $dp[u]\cdot w$ 累加到后继，读出 $dp[b_j]$ 得到矩阵第 i 行。最后用 `determinant_mod` 求 $k\times k$ 行列式。

代码依赖：ModInt（第 90 页，18.1.2 节）、determinant_mod（第 220 页，21.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：dag_path_determinant<mod>(g,sources,sinks) 返回路径权值矩阵的行列式，mod 必须为素数。g[u] 的项为 (v,ModInt<mod> 权值)，点号 0..n-1；两个端点序列等长 k，各自不能重复，但可相互重叠。允许重边、零权、负权；长度为 0 的路径权为 1。整张图有环则返回 nullopt，空端点族在 DAG 上返回 1。由 LGV，它是顶点不交路径族按汇点排列带符号的权值和；只有能证明非恒等排列均不可能时，才是对应配对的普通计数。顺序不能随意排序，模结果为 0 也不能据此断言不存在路径族。时间 O((k+1)(n+m)+k^3+k log mod)，额外空间 O(n+k^2)，依赖 det_prime 和 ModInt。

```
template <int mod>
optional<ModInt<mod>>
dag_path_determinant(const vector<vector<pair<int, ModInt<mod>>>> &g,
                     const vector<int> &sources,
                     const vector<int> &sinks)
{
    using Z = ModInt<mod>;
    int n = g.size(), k = sources.size();
    assert((int)sinks.size() == k);
    vector<int> in(n), order;
    vector<bool> seen_source(n), seen_sink(n);
    for (int i = 0; i < k; i++)
    {
        assert(0 <= sources[i] && sources[i] < n);
        assert(0 <= sinks[i] && sinks[i] < n);
        assert(!seen_source[sources[i]] && !seen_sink[sinks[i]]);
        seen_source[sources[i]] = seen_sink[sinks[i]] = true;
    }
    for (const auto &edges : g)
        for (auto [v, w] : edges)
        {
            assert(0 <= v && v < n);
            in[v]++;
        }
    for (int u = 0; u < n; u++)
        if (!in[u]) order.push_back(u);
    for (int i = 0; i < (int)order.size(); i++)
        for (auto [v, w] : g[order[i]])
            if (!--in[v]) order.push_back(v);
    if ((int)order.size() != n) return nullopt;
```

```
    vector<vector<long long>> a(k, vector<long long>(k));
    for (int i = 0; i < k; i++)
    {
        vector<Z> dp(n);
        dp[sources[i]] = 1;
        for (int u : order)
            for (auto [v, w] : g[u]) dp[v] = dp[v] + dp[u] * w;
        for (int j = 0; j < k; j++) a[i][j] = dp[sinks[j]].v;
    }
    return Z(determinant_mod(move(a), mod));
}
```

第五十三章　支配树

53.1　支配树：Lengauer–Tarjan

问题描述：给定有向图和起点 r，对每个从 r 可达的点 v 求直接支配点 idom(v)：r 到 v 的每条路径都必经的点中（v 除外）离 v 最近的那个。所有 idom 边构成以 r 为根的支配树。

核心思路：Lengauer–Tarjan（只带路径压缩的简单版），全部按 DFS 序号运算。半支配点 semi(w) 是能经过序号都大于 w 的中间点走到 w 的最小序号点。按序号逆序处理 w：由各前驱的 `eval` 求出 `semi[w]`，把 w 放进 `bucket[semi[w]]`，再把 w 挂到 DFS 父亲下；然后处理父亲桶中的点 v：$u = \mathrm{eval}(v)$ 是 semi(v) 之下到 v 的树路径上 semi 最小的点，若 $\mathrm{semi}(u) < \mathrm{semi}(v)$ 则 $\mathrm{idom}(v) = \mathrm{idom}(u)$，暂记 u，否则 $\mathrm{idom}(v) = \mathrm{semi}(v)$。最后正序修正暂记的值。

思路中的情况	代码中的处理	行
DFS 编号	记录序号 `dfn`、逆映射 `vertex` 与 DFS 树父亲（均用序号）	18
带权并查集	路径压缩时把祖先链上 semi 最小的点传给 `label`	34
求半支配点	对每个前驱取 `eval` 的 semi 最小值	66
连接	把 w 挂到 DFS 父亲下，供之后的 `eval` 使用	68
处理父亲的桶	由 `eval` 判断 idom = semi，或暂记需要继承的点	72
正序修正	暂记的点继承其 `dom`，再映射回原点号	78

接口契约：DominatorTree(n)，点号 1..n；add(u,v) 加有向边，允许重边、自环。build(root) 后 idom 为原点号直接支配点，根指向自身，不可达为 0。仅对 root 可达点定义支配关系：所有 root 到 v 的路径均经过 u，等价于 u 在支配树上是 v 的祖先。timer 是可达点数，vertex[1..timer] 为 DFS 顺序，dfn 为逆映射；逆序向 idom 累加可得支配点数，根不向自身累加。g 保留原点号；pred、bucket、semi 等辅助数组使用 DFS 编号。可更换根重建；加边使预处理失效，需重新 build。简单 LT 时间 O((n+m) log(n+1))，空间及递归栈 O(n+m) 与 O(n)，n+1、边数须可由 int 表示。一般图与 DAG 均适用；可由 idom 建树组合 LCA。

```
struct DominatorTree
{
    int n, timer = 0;
    vector<vector<int>> g, pred, bucket;
    vector<int> dfn, vertex, parent, semi, label, ancestor, dom, idom;

    DominatorTree(int n) : n(n), g(n + 1) { assert(n > 0); }

    void add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        g[u].push_back(v);
        idom.clear();
    }

    void dfs(int u)
    {
        dfn[u] = ++timer;
        vertex[timer] = u;
        for (int v : g[u])
            if (!dfn[v])
            {
                dfs(v);
                parent[dfn[v]] = dfn[u];
            }
    }

    void compress(int v)
    {
        int a = ancestor[v];
        if (ancestor[a])
        {
            compress(a);
            if (semi[label[a]] < semi[label[v]]) label[v] = label[a];
            ancestor[v] = ancestor[a];
        }
    }
```

路径查询、半支配点与直接支配点计算（接上页同一结构体）：

```
    int eval(int v)
    {
        compress(v);
        return label[v];
    }

    void build(int root)
    {
        assert(1 <= root && root <= n);
        timer = 0;
        dfn.assign(n + 1, 0);
        vertex.assign(n + 1, 0);
        parent.assign(n + 1, 0);
        ancestor.assign(n + 1, 0);
        dom.assign(n + 1, 0);
        idom.assign(n + 1, 0);
        pred.assign(n + 1, {});
        bucket.assign(n + 1, {});
        semi.resize(n + 1);
        iota(semi.begin(), semi.end(), 0);
        label = semi;
        dfs(root);
        for (int u = 1; u <= n; u++)
            if (dfn[u])
                for (int v : g[u]) pred[dfn[v]].push_back(dfn[u]);
        for (int i = timer; i >= 2; i--)
        {
            for (int p : pred[i]) semi[i] = min(semi[i], semi[eval(p)]);
            bucket[semi[i]].push_back(i);
            ancestor[i] = parent[i];
            for (int v : bucket[parent[i]])
            {
                int u = eval(v);
                dom[v] = semi[u] < semi[v] ? u : parent[i];
            }
            bucket[parent[i]].clear();
        }
        for (int i = 2; i <= timer; i++)
        {
            if (dom[i] != semi[i]) dom[i] = dom[dom[i]];
            idom[vertex[i]] = vertex[dom[i]];
        }
        idom[root] = root;
    }
};
```

模板题使用：Library Checker dominatortree

题意：从指定起点求每个点的直接支配点。输入输出是 0-based，调用模板时加 1；输出减 1 后，不可达点为 -1，根仍指向自身。

所需模板：DominatorTree（第 579 页，53.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/dominatortree

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s;
    cin >> n >> m >> s;
    DominatorTree t(n);
    while (m--)
    {
        int u, v;
        cin >> u >> v;
        t.add(u + 1, v + 1);
    }
    t.build(s + 1);
    for (int u = 1; u <= n; u++) cout << t.idom[u] - 1 << (u == n ? '\n' : ' ');
}
```

第七部分

计算几何

第五十四章　二维计算几何基础

54.1　浮点点向量、圆与交点结果

功能：浮点二维几何的公共基础：点/向量 `Point`（加减、数乘、数除），圆 `Circle`，以及各求交函数共用的结果类型 `Result`（交点类别 `Kind` 加有限个交点）。

核心思路：坐标统一用 `long double`。`dot`、`cross` 为点积与叉积，`cross(a,b)` > 0 表示 b 在 a 的逆时针方向；`norm` 用 `hypotl` 求长度，避免先平方溢出；`perp` 把向量逆时针旋转 $90°$。其余实数几何函数都建立在这些运算上，交点无穷多或输入退化等情况通过 `Kind` 区分。

接口契约：R 为 long double；Point 支持向量加减、数乘和数除（除数非零）。dot/cross/norm/perp 为基本运算，norm 使用 hypotl；Circle 保存圆心和非负半径。Kind 为 none、one、two、infinite、degenerate；Result 同时保存类别与有限交点。输入须有限，尺度须使后续平方、乘积不溢出或下溢；排序不使用 epsilon。具体容差与退化约定见各函数。

```
struct RealPlane
{
    using R = long double;

    struct Point
    {
        R x = 0, y = 0;

        Point operator+(Point b) const { return {x + b.x, y + b.y}; }

        Point operator-(Point b) const { return {x - b.x, y - b.y}; }

        Point operator*(R k) const { return {x * k, y * k}; }

        Point operator/(R k) const { return {x / k, y / k}; }
    };

    struct Circle
    {
        Point o;
        R r;
    };
```

```
    enum class Kind
    {
        none,
        one,
        two,
        infinite,
        degenerate
    };

    struct Result
    {
        Kind kind;
        vector<Point> p;
    };

    static R dot(Point a, Point b) { return a.x * b.x + a.y * b.y; }

    static R cross(Point a, Point b) { return a.x * b.y - a.y * b.x; }

    static R norm(Point a) { return hypotl(a.x, a.y); }

    static Point perp(Point a) { return {-a.y, a.x}; }
};
```

模板题使用：AOJ CGL_2_A

题意：每组给两条非退化直线，平行输出 2，垂直输出 1，否则输出 0。输入为绝对值不超过 10000 的整数，点积与叉积在此范围可由浮点精确表示，所以直接比较 0；这不适用于一般实数或更大整数。只演示点向量与 dot/cross，不涵盖圆及 Result 分类。

所需模板：RealPlane（第 584 页，54.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.u-aizu.ac.jp/problems/CGL_2_A

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    while (q--)
    {
        RealPlane::Point a, b, c, d;
        cin >> a.x >> a.y >> b.x >> b.y >> c.x >> c.y >> d.x >> d.y;
        auto u = b - a, v = d - c;
        if (RealPlane::cross(u, v) == 0)
            cout << 2 << '\n';
        else if (RealPlane::dot(u, v) == 0)
            cout << 1 << '\n';
        else
            cout << 0 << '\n';
    }
}
```

54.2 点在直线上的投影

功能： 求点 p 在过 a、b 两点的直线上的正交投影（垂足）。

核心思路： 设方向 $v = b - a$，投影为 $a + v \cdot \frac{(p-a)\cdot v}{v\cdot v}$，即从 a 出发沿 v 走 $p - a$ 在 v 上的投影长度。$v \cdot v = 0$（两点重合）时直接返回 a。

代码依赖：RealPlane（第 584 页，54.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约： line_projection(p,a,b) 返回 p 在过 a、b 直线上的正交投影；若 a=b，则约定返回 a。类型为 RealPlane::Point。输入有限且尺度适当，平方和与点积须能可靠表示；不使用 epsilon 把短线段自动视为零。O(1) 时间和空间。

```
RealPlane::Point
line_projection(RealPlane::Point p, RealPlane::Point a, RealPlane::Point b)
{
    using G = RealPlane;
    using R = G::R;
    using Point = G::Point;
    Point v = b - a;
    R d = G::dot(v, v);
    if (d == 0) return a;
    return a + v * (G::dot(p - a, v) / d);
}
```

模板题使用：AOJ CGL_1_A

题意：固定直线两端点，对每个点输出其在无限直线上的投影。题目保证两端点不同，整数坐标绝对值不超过 10000，q 不超过 1000；投影可在线段外。接口参数为 (p,a,b)，不截断到线段；a=b 时模板返回 a 属于额外边界。

所需模板：RealPlane（第 584 页，54.1 节）、line_projection（第 586 页，54.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.u-aizu.ac.jp/problems/CGL_1_A

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    RealPlane::Point a, b;
    int q;
    cin >> a.x >> a.y >> b.x >> b.y >> q;
    cout << fixed << setprecision(12);
    while (q--)
    {
        RealPlane::Point p;
        cin >> p.x >> p.y;
        auto h = line_projection(p, a, b);
        cout << h.x << ' ' << h.y << '\n';
    }
}
```

54.3 浮点两直线交点

功能：求直线 ab 与直线 cd 的交点，并区分唯一交点、平行不交、重合（无穷多交点）和直线退化四种情况。

核心思路：设 $u = b - a$，$v = d - c$，$\Delta = u \times v$。若 $|\Delta| \le \varepsilon |u||v|$（两方向夹角的正弦足够小）视为平行，再看 c 到直线 ab 的距离 $|(c-a) \times u|/|u|$ 是否在相对容差内，区分重合与平行不交；否则交点为 $a + u \cdot \frac{(c-a)\times v}{\Delta}$。

代码依赖：RealPlane（第 584 页，54.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：line_intersection_real(a,b,c,d,eps=1e-12L) 返回 RealPlane::Result；eps 必须为正。端点重合使任一直线退化时返回 degenerate。按方向叉积与方向长度乘积的相对阈值判断平行，再按位置阈值区分 none 与 infinite；唯一交点返回 one。输入有限且尺度适当；近似分类不等同于精确共线判定。O(1)。

```
RealPlane::Result line_intersection_real(RealPlane::Point a,
                                         RealPlane::Point b,
                                         RealPlane::Point c,
                                         RealPlane::Point d,
                                         long double eps = 1e-12L)
{
    using G = RealPlane;
    using R = G::R;
    using Point = G::Point;
    using Kind = G::Kind;
    assert(eps > 0);
    Point u = b - a, v = d - c;
    R nu = G::norm(u), nv = G::norm(v);
    if (nu == 0 || nv == 0) return {Kind::degenerate, {}};
    R det = G::cross(u, v);
    if (fabsl(det) <= eps * nu * nv)
        return {fabsl(G::cross(c - a, u)) <= eps * nu * max(R(1), G::norm(c - a))
                    ? Kind::infinite
                    : Kind::none,
                {}};
    return {Kind::one, {a + u * (G::cross(c - a, v) / det)}};
}
```

模板题使用：AOJ CGL_2_C

题意：输出两条线段的唯一交点；题目保证非平行且相交，q 不超过 1000，整数坐标绝对值不超过 10000，因此可直接取 one 结果。模板计算支撑直线交点，不自动检查交点在线段内；一般输入须先检查 kind。整数方向叉积非零时绝对值至少 1，大于本题范围下默认相对阈值。

所需模板：RealPlane（第 584 页，54.1 节）、line_intersection_real（第 587 页，54.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.u-aizu.ac.jp/problems/CGL_2_C

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    cout << fixed << setprecision(12);
    while (q--)
    {
        RealPlane::Point a, b, c, d;
        cin >> a.x >> a.y >> b.x >> b.y >> c.x >> c.y >> d.x >> d.y;
        auto ans = line_intersection_real(a, b, c, d);
        assert(ans.kind == RealPlane::Kind::one);
        cout << ans.p[0].x << ' ' << ans.p[0].y << '\n';
    }
}
```

54.4 浮点直线与圆交点

功能：求直线 ab 与圆的交点，结果分为相离、相切（一点）、相交（两点）以及直线退化。

核心思路：分两种模式。eps= 0 时只用平方量：$v = b - a$，$z = v \times (a - o)$，圆心到直线距离为 $|z|/|v|$，判别式 $D = r^2|v|^2 - z^2$ 的符号决定交点个数；垂足 $h = o + \mathrm{perp}(v)\, z/|v|^2$，两交点为 $h \mp v\sqrt{D}/|v|^2$。eps> 0 时用 line_projection 求圆心垂足 h 及距离 d，与半径按相对容差比较，近切时合并为一点，否则从 h 沿直线方向偏移半弦长 $\sqrt{r^2 - d^2}$。

代码依赖：RealPlane（第 584 页，54.1 节）、line_projection（第 586 页，54.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：line_circle_intersections(a,b,c,eps=1e-12L) 返回 Result；c 为 RealPlane::Circle，半径非负，eps 非负；以下相对阈值描述适用于 eps>0。a=b 返回 degenerate；按投影点到圆心的距离和半径的相对阈值区分 none、one、two。点圆允许；只有正半径同心圆才涉及无限交点，本函数不会返回 infinite。依赖 line_projection，O(1)。输入有限且尺度适当，接近相切时按容差归类。默认容差不能直接满足 AOJ CGL_7_D 的所有合法输入；近切时归并交点可能超出坐标误差限。eps=0 为显式无容差模式：直接计算平方判别式，不主动合并近切点；一般实数仍可能发生舍入误判、溢出或下溢。整数坐标绝对值 ≤10000、整数半径 0..10000，二进制 long double 尾数 ≥64 位且正常最近舍入、禁用 fast-math 时，中间整数精确。交点坐标仍为浮点近似；其他输入不继承这一保证。

```
RealPlane::Result line_circle_intersections(RealPlane::Point a,
                                            RealPlane::Point b,
                                            RealPlane::Circle c,
                                            long double eps = 1e-12L)
{
    using G = RealPlane;
    using R = G::R;
    using Point = G::Point;
    using Kind = G::Kind;
    assert(eps >= 0);
    assert(c.r >= 0);
    Point v = b - a;
    if (eps == 0)
    {
        R q = G::dot(v, v);
        if (q == 0) return {Kind::degenerate, {}};
        R z = G::cross(v, a - c.o);
        R d = c.r * c.r * q - z * z;
        if (d < 0) return {Kind::none, {}};
        Point h = c.o + G::perp(v) * (z / q);
        if (d == 0) return {Kind::one, {h}};
        Point w = v * (sqrtl(d) / q);
        return {Kind::two, {h - w, h + w}};
    }
    R len = G::norm(v);
    if (len == 0) return {Kind::degenerate, {}};
    Point h = line_projection(c.o, a, b);
    R d = G::norm(h - c.o), tol = eps * max(d, c.r);
    if (d > c.r + tol) return {Kind::none, {}};
    if (fabsl(d - c.r) <= tol) return {Kind::one, {h}};
    R t = sqrtl(max(R(0), (c.r - d) * (c.r + d)));
    Point w = v * (t / len);
    return {Kind::two, {h - w, h + w}};
}
```

模板题使用：AOJ CGL_7_D —浮点接口无容差模式

题意：整数坐标绝对值 ≤10000、整数半径 1..10000，保证交点存在；每条直线两端点不同，1≤q≤1000。直接调用浮点接口并显式传 eps=0，不合并近切交点；仅在二进制 long double 尾数至少 64 位、正常最近舍入且禁用 fast-math 的平台采用此用法。此整数域的平方判别式精确，坐标仍为浮点构造；一般实数不继承精确分类保证。相切重复同一点，按普通 (x,y) 字典序输出，不能用 epsilon 排序；默认 eps>0 接口保持原近似语义。

所需模板：RealPlane（第 584 页，54.1 节）、line_projection（第 586 页，54.2 节）、line_circle_intersections（第 589 页，54.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_7_D

```
int main()
{
    static_assert(numeric_limits<long double>::radix == 2 &&
                  numeric_limits<long double>::digits >= 64);
#ifdef __FAST_MATH__
#error This example requires normal floating arithmetic
#endif
    if (fegetround() != FE_TONEAREST) return 1;
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    RealPlane::Circle c;
    int q;
    cin >> c.o.x >> c.o.y >> c.r >> q;
    cout << fixed << setprecision(12);
    while (q--)
    {
        RealPlane::Point a, b;
        cin >> a.x >> a.y >> b.x >> b.y;
        auto ans = line_circle_intersections(a, b, c, 0);
        assert(ans.p.size() == 1 || ans.p.size() == 2);
        if (ans.p.size() == 1) ans.p.push_back(ans.p[0]);
        if (tie(ans.p[1].x, ans.p[1].y) < tie(ans.p[0].x, ans.p[0].y))
            swap(ans.p[0], ans.p[1]);
        cout << ans.p[0].x << ' ' << ans.p[0].y << ' ' << ans.p[1].x << ' '
             << ans.p[1].y << '\n';
    }
}
```

54.5 浮点两圆交点

功能：求两圆的交点，区分相离或内含（无交点）、相切（一点）、相交（两点）以及两圆重合（无穷多）；半径为 0 的圆视为点。

核心思路：设 $v = o_2 - o_1$。交点连线与连心线的交点（垂足）为 $h = o_1 + v \cdot \frac{x}{|v|}$，其中 $x = \frac{|v|^2 + r_1^2 - r_2^2}{2|v|}$，两交点为 h 沿 $\mathrm{perp}(v)$ 方向偏移 $\pm\sqrt{r_1^2 - x^2}$。eps= 0 时全用平方量：$k = |v|^2 + r_1^2 - r_2^2$，判别式 $D = 4r_1^2|v|^2 - k^2$ 的符号决定交点个数。eps> 0 时先单独处理同心与点圆，再把圆心距与 $r_1 + r_2$、$|r_1 - r_2|$，以及 $|x|$ 与 r_1 按相对容差比较来分类。

代码依赖：RealPlane（第 584 页，54.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：circle_intersections(a,b,eps=1e-12L) 返回 Result；半径非负，eps 非负；以下相对阈值描述适用于 eps>0。两个完全相同的正半径圆返回 infinite；同位置两个点圆返回 one；同心不同半径返回 none。其余用距离和半径的相对阈值分类，点圆单独处理，接近相切可返回 one。输入有限且尺度适当，O(1) 时间空间；返回的有限交点仍受浮点构造误差影响。默认容差不能直接满足 AOJ CGL_7_E 的所有合法输入；近切时归并交点可能超出坐标误差限。eps=0 为显式无容差模式：直接计算平方判别式，不主动合并近切点；一般实数仍可能发生舍入误判、溢出或下溢。整数坐标绝对值 ≤10000、整数半径 0..10000，二进制 long double 尾数 ≥64 位且正常最近舍入、禁用 fast-math 时，中间整数精确。交点坐标仍为浮点近似；其他输入不继承这一保证。

```
RealPlane::Result
circle_intersections(RealPlane::Circle a, RealPlane::Circle b, long double eps = 1e-12L)
{
    using G = RealPlane;
    using R = G::R;
    using Point = G::Point;
    using Kind = G::Kind;
    assert(eps >= 0);
    assert(a.r >= 0 && b.r >= 0);
    Point v = b.o - a.o;
    if (eps == 0)
    {
        R q = G::dot(v, v);
        if (q == 0)
        {
            if (a.r != b.r) return {Kind::none, {}};
            if (a.r == 0) return {Kind::one, {a.o}};
            return {Kind::infinite, {}};
        }
        R k = q + a.r * a.r - b.r * b.r;
        R d = 4 * a.r * a.r * q - k * k;
        if (d < 0) return {Kind::none, {}};
        Point h = a.o + v * (k / (2 * q));
        if (d == 0) return {Kind::one, {h}};
        Point w = G::perp(v) * (sqrtl(d) / (2 * q));
        return {Kind::two, {h - w, h + w}};
    }
```

默认正容差分支（接上页同一函数）：

```
    R d = G::norm(v);
    R tol = eps * max({d, a.r, b.r});
    // Infinite intersections require identical positive-radius circles.
    if (d == 0)
    {
        if (a.r != b.r) return {Kind::none, {}};
        if (a.r == 0) return {Kind::one, {a.o}};
        return {Kind::infinite, {}};
    }
    if (a.r == 0 && b.r == 0) return {Kind::none, {}};
    if (a.r == 0 || b.r == 0)
    {
        R radius = max(a.r, b.r);
        if (fabsl(d - radius) > tol) return {Kind::none, {}};
        return {Kind::one, {a.r == 0 ? a.o : b.o}};
    }
    if (d > a.r + b.r + tol || d < fabsl(a.r - b.r) - tol) return {Kind::none, {}};
    R x = (d * d + (a.r - b.r) * (a.r + b.r)) / (2 * d);
    Point h = a.o + v * (x / d);
    // Near-concentric circles must not be mistaken for internal tangency.
    if (fabsl(x) > a.r + tol) return {Kind::none, {}};
    if (fabsl(fabsl(x) - a.r) <= tol) return {Kind::one, {h}};
    R y = sqrtl(max(R(0), (a.r - x) * (a.r + x)));
    Point w = G::perp(v) * (y / d);
    return {Kind::two, {h - w, h + w}};
}
```

模板题使用：AOJ CGL_7_E —浮点接口无容差模式

题意：整数坐标绝对值 ≤10000、整数半径 1..10000，保证交点存在；两圆圆心不同。直接调用浮点接口并显式传 eps=0，不合并近切交点；仅在二进制 long double 尾数至少 64 位、正常最近舍入且禁用 fast-math 的平台采用此用法。此整数域的平方判别式精确，坐标仍为浮点构造；一般实数不继承精确分类保证。相切重复同一点，按普通 (x,y) 字典序输出，不能用 epsilon 排序；默认 eps>0 接口保持原近似语义。

所需模板：RealPlane（第 584 页，54.1 节）、circle_intersections（第 591 页，54.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_7_E

```
int main()
{
    static_assert(numeric_limits<long double>::radix == 2 &&
                  numeric_limits<long double>::digits >= 64);
#ifdef __FAST_MATH__
#error This example requires normal floating arithmetic
#endif
    if (fegetround() != FE_TONEAREST) return 1;
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    RealPlane::Circle a, b;
    cin >> a.o.x >> a.o.y >> a.r >> b.o.x >> b.o.y >> b.r;
    auto ans = circle_intersections(a, b, 0);
    assert(ans.p.size() == 1 || ans.p.size() == 2);
    if (ans.p.size() == 1) ans.p.push_back(ans.p[0]);
    if (tie(ans.p[1].x, ans.p[1].y) < tie(ans.p[0].x, ans.p[0].y))
        swap(ans.p[0], ans.p[1]);
    cout << fixed << setprecision(12) << ans.p[0].x << ' ' << ans.p[0].y << ' '
         << ans.p[1].x << ' ' << ans.p[1].y << '\n';
}
```

54.6 两圆交面积

问题描述：求两个圆公共部分（交集）的面积。

核心思路：圆心距 $d \geq r_1 + r_2$ 时不相交，面积为 0；$d \leq |r_1 - r_2|$ 时小圆被包含，面积为小圆面积。否则交集由两个弓形拼成：由余弦定理求出公共弦在每个圆中所对圆心角的一半 α，弓形面积为 $r^2(\alpha - \frac{1}{2}\sin 2\alpha)$，两者相加。$\alpha$ 很小时直接相减会严重相消，改用展开式 $\frac{2}{3}\alpha^3 - \frac{2}{15}\alpha^5 + \frac{4}{315}\alpha^7$。

代码依赖：RealPlane（第 584 页，54.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：circle_overlap_area(a,b) 返回 long double 面积，半径非负。分离或外切返回 0；内含时返回小圆面积，含点圆和同心圆。部分相交计算两个圆弓形面积，小角度使用展开式减轻相消；acos 入参截断至 [-1,1]。不使用交点分类容差；要求有限且尺度适当的输入。O(1) 时间空间，不把浮点输出视为精确面积。

```
RealPlane::R circle_overlap_area(RealPlane::Circle a, RealPlane::Circle b)
{
    using G = RealPlane;
    using R = G::R;
    assert(a.r >= 0 && b.r >= 0);
    R d = G::norm(a.o - b.o), pi = acosl(-1.L);
    if (d >= a.r + b.r) return 0;
    if (d <= fabsl(a.r - b.r)) return pi * min(a.r, b.r) * min(a.r, b.r);
    R difference = (a.r - b.r) * (a.r + b.r);
    R x = acosl(clamp((d * d + difference) / (2 * d * a.r), R(-1), R(1)));
    R y = acosl(clamp((d * d - difference) / (2 * d * b.r), R(-1), R(1)));
    auto segment = [](R angle)
    {
        if (angle < 1e-3L)
        {
            R z = angle * angle;
            return angle * z * (R(2) / 3 + z * (-R(2) / 15 + z * R(4) / 315));
        }
        return angle - sinl(2 * angle) / 2;
    };
    return a.r * a.r * segment(x) + b.r * b.r * segment(y);
}
```

模板题使用：AOJ CGL_7_I

题意：两个圆的公共面积，整数坐标绝对值 $\leq 10^4$、半径 $1..10^4$。覆盖相离、内含、同心与部分相交；输出面积的绝对误差须小于 1e-6。

所需模板：RealPlane（第 584 页，54.1 节）、circle_overlap_area（第 594 页，54.6 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_7_I

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    RealPlane::Circle a, b;
    cin >> a.o.x >> a.o.y >> a.r >> b.o.x >> b.o.y >> b.r;
    cout << fixed << setprecision(12) << circle_overlap_area(a, b) << '\n';
}
```

相关应用：凸多边形裁剪：保留有向直线左侧；几何基础上的独立补充接口。

54.7 凸多边形裁剪：保留有向直线左侧

功能：用有向直线 $a \to b$ 切割凸多边形，返回多边形在直线左侧（含直线上）的部分，顶点保持原边界顺序。

核心思路：单条直线的 Sutherland–Hodgman 裁剪：令 $e = b - a$，依次考察每条边 $u \to v$，用 $s = e \times (u - a)$、$t = e \times (v - a)$ 判断两端在直线哪一侧。$s \geq 0$ 时保留顶点 u；两端严格异侧时再加入边与直线的交点 $u + (v - u)\frac{s}{s-t}$。端点恰在直线上时不求交，避免重复点。

代码依赖：RealPlane（第 584 页，54.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：convex_cut(p,a,b) 返回凸多边形与有向直线 a→b 左侧闭半平面的交。a≠b；输入按边界顺序，顺/逆时针均可，允许共线中间点；空集、点和线段也可。原输入不变，输出保留原方向，但可能有共线点或重复边界点，不能直接当作严格凸包传入 minimum_rectangle。

逐边保留左侧顶点，仅在两端严格异侧时加入交点，过顶点及沿边切割不作重复求交。O(n) 时间和输出空间。n 须能用 int 表示，输入及中间运算有限可表示。不使用 eps；任意近退化实数的符号受舍入影响，不保证精确拓扑。AOJ 整数范围内输入侧向叉积可精确表示，交点与面积仍舍入。

```
vector<RealPlane::Point> convex_cut(const vector<RealPlane::Point> &p,
    RealPlane::Point a, RealPlane::Point b)
{
    using G = RealPlane;
    assert(a.x != b.x || a.y != b.y);
    int n = int(p.size());
    vector<G::Point> q;
    auto e = b - a;
    for (int i = 0; i < n; i++)
    {
        auto u = p[i];
        auto v = p[(i + 1) % n];
        auto s = G::cross(e, u - a);
        auto t = G::cross(e, v - a);
        if (s >= 0)
            q.push_back(u);
        if ((s < 0 && t > 0) || (s > 0 && t < 0))
            q.push_back(u + (v - u) * (s / (s - t)));
    }
    return q;
}
```

模板题使用：AOJ CGL_4_C Convex Cut

题意：n,q≤100，整数坐标绝对值 ≤10000，输入逆时针凸多边形可含共线点。每条查询均裁剪原多边形，输出左侧面积 12 位小数，要求绝对误差小于 1e-5。已核对官方题面与本地精确模型；AOJ 未登录且无已保存账号，尚无在线提交。

所需模板：RealPlane（第 584 页，54.1 节）、convex_cut（第 595 页，54.7 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_4_C&lang=en

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, queries;
    cin >> n;
    vector<RealPlane::Point> p(n);
    for (auto &v : p)
        cin >> v.x >> v.y;
    cin >> queries;
    while (queries--)
    {
        RealPlane::Point a, b;
        cin >> a.x >> a.y >> b.x >> b.y;
        auto q = convex_cut(p, a, b);
        long double area = 0;
        long double error = 0;
        for (int i = 1; i + 1 < (int)q.size(); i++)
        {
            auto value = RealPlane::cross(q[i] - q[0], q[i + 1] - q[0]);
            auto y = value - error;
            auto next = area + y;
            error = (next - area) - y;
            area = next;
        }
        cout << fixed << setprecision(12) << abs(area) / 2 << '\n';
    }
}
```

相关应用：简单多边形面积重心；几何基础上的独立补充接口。

54.8 简单多边形面积重心

问题描述：求均匀密度简单多边形（可凹）的面积重心。

核心思路：以 p_0 为公共顶点把多边形剖成三角扇 (p_0, p_i, p_{i+1})。设 a, b 为相对 p_0 的向量，三角形有向面积的两倍为 $s = a \times b$，重心为 $(a+b)/3$；凹多边形中各三角形面积有正有负，恰好抵消多算的部分。累加 $\sum s$ 与 $\sum (a+b)s$，重心为 $p_0 + \frac{\sum (a+b)s}{3\sum s}$。三个累加量都用 Kahan 补偿求和（add）减小舍入误差。

代码依赖：RealPlane（第 584 页，54.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：polygon_centroid(p) 返回 optional<RealPlane::Point>，表示均匀密度简单多边形的面积重心；可凹，顺/逆时针均可。少于三点或计算有向面积恰为 0 返回 nullopt，不把原点作为失败标记。不用于自交、多环带孔多边形，也不是顶点平均或边界线重心。

相对 p[0] 作三角扇，有向面积与一阶矩补偿求和；除以总有向面积恢复坐标。O(n) 时间、O(1) 额外空间，原输入不变。n 须能用 int 表示，输入与中间运算有限可表示；不使用 eps。实数近退化时面积符号和零判定仍可能舍入错误，巨大平移也限制最终坐标精度；long double 不保证比 double 精度高。

```
optional<RealPlane::Point> polygon_centroid(const vector<RealPlane::Point> &p)
{
    using G = RealPlane;
    using R = G::R;
    int n = int(p.size());
    if (n < 3)
        return nullopt;
    array<R, 3> sum{};
    array<R, 3> error{};
    auto add = [&](int i, R v)
    {
        R y = v - error[i];
        R next = sum[i] + y;
        error[i] = (next - sum[i]) - y;
        sum[i] = next;
    };
    for (int i = 1; i + 1 < n; i++)
    {
        auto a = p[i] - p[0];
        auto b = p[i + 1] - p[0];
        R s = G::cross(a, b);
        add(0, s);
        add(1, (a.x + b.x) * s);
        add(2, (a.y + b.y) * s);
    }
    if (sum[0] == 0)
        return nullopt;
    G::Point offset{sum[1] / sum[0] / 3, sum[2] / sum[0] / 3};
    return p[0] + offset;
}
```

相关应用：三角形内心与内切圆，非完整二维几何来源覆盖

54.9 三角形内切圆：边长加权内心

问题描述：给定三角形的三个顶点，求其内切圆的圆心（内心）和半径。

核心思路：内心是三个顶点以各自对边长为权的加权平均。相对 a 平移得 $u=b-a$，$v=c-a$，三顶点的对边长分别为 $|v-u|$、$|v|$、$|u|$，于是内心为 $a+\frac{|v|u+|u|v}{|u|+|v|+|v-u|}$；半径为面积除以半周长，即 $|u\times v|$ 除以周长。代码先用原坐标求叉积判断退化，再把 u,v 除以最大分量 `scale` 后求边长，最后乘回，避免溢出。

代码依赖：RealPlane（第 584 页，54.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：triangle_incircle(a,b,c) 返回 optional<RealPlane::Circle>，圆心为内心、半径为面积除以半周长。顺逆时针均可，不改变输入；计算叉积恰为 0 或三点全重合返回 nullopt，不把原点作为失败标志。用对边长加权顶点，避免三角函数；O(1) 时间和空间。

先相对 a 平移，边向量按最大分量归一化再求边长；面积叉积在归一化前计算，保留 AOJ 整数域内精确零判定。要求输入、相减、叉积及输出等中间值有限可表示且不发生影响结果的下溢；任意近共线实数的零判定仍可能舍入错误，不是精确拓扑接口。AOJ 整数坐标绝对值 <=10000、三点不共线；叉积精确，圆心半径仍为浮点近似。long double 不保证比 double 精度高。

```
optional<RealPlane::Circle> triangle_incircle(RealPlane::Point a,
                                              RealPlane::Point b,
                                              RealPlane::Point c)
{
    using G = RealPlane;
    using R = G::R;
    auto u = b - a, v = c - a;
    R scale = max({fabsl(u.x), fabsl(u.y), fabsl(v.x), fabsl(v.y)});
    if (scale == 0)
        return nullopt;
    R area = fabsl(G::cross(u, v));
    if (area == 0)
        return nullopt;
    u = u / scale;
    v = v / scale;
    R x = G::norm(v - u), y = G::norm(v), z = G::norm(u);
    R sum = x + y + z;
    auto o = a + (u * (y / sum) + v * (z / sum)) * scale;
    return G::Circle{o, area / scale / sum};
}
```

模板题使用：AOJ CGL_7_B 三角形内切圆

题意：输入三个非共线整点，坐标绝对值不超过 10000，输出内心坐标与半径；要求误差小于 1e-6。官方题面已核对，完整协议本地验证，尚无本实现在线记录。

所需模板：RealPlane（第 584 页，54.1 节）、triangle_incircle（第 598 页，54.9 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_7_B&lang=en

```
int main()
{
    RealPlane::Point a, b, c;
    cin >> a.x >> a.y >> b.x >> b.y >> c.x >> c.y;
    auto circle = triangle_incircle(a, b, c);
    if (circle)
        cout << fixed << setprecision(12) << circle->o.x << ' '
             << circle->o.y << ' ' << circle->r << '\n';
}
```

54.10 有限二进制浮点极角精确排序

功能：浮点点的极角比较器：从正 x 轴起按逆时针极角排序，同方向按长度递增，原点排在最前；对已存入的浮点数比较结果精确。

核心思路：先按半平面分组（`half`：上半平面及正 x 轴在前），同组内看叉积 $a_x b_y - a_y b_x$ 的符号。为避免浮点叉积舍入，`product` 用 `frexp` 把每个因子拆成 64 位整数尾数和指数，尾数相乘得 128 位整数并左移规格化，于是乘积可按（符号，指数，规格化尾数）精确比较大小；`cross_sign` 据此比较两个乘积。叉积为 0（同方向）时比较非零坐标分量的绝对值。

代码依赖：RealPlane（第 584 页，54.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：RealPolarLess{}(a,b) 沿用 RealPlane::Point 存储，输入为实际已存入的有限二进制 long double，尾数 ≤64 位且 uint128 可用；禁 fast-math 及 FTZ/DAZ。原点最前，正 x 轴起逆时针 $[0,2\pi)$，同向按长度递增，同点及 ± 0 等价，满足严格弱序。用 frexp/ldexp 拆尾数并以 uint128 精确比较乘积，不做浮点叉乘、不用 epsilon。每次 O(1)，无分配。十进制输入先舍入；不修复调用前的坐标差，不替代其他浮点几何谓词。product/cross_sign 是有限输入下的内部辅助。

```
struct RealPolarLess
{
    using R = RealPlane::R;
    using P = RealPlane::Point;
    using U = __uint128_t;
    static_assert(numeric_limits<R>::radix == 2 && numeric_limits<R>::digits <= 64);

    struct Product
    {
        int sign;
        long long exponent;
        U value;
    };

    static Product product(R a, R b)
    {
        if (a == 0 || b == 0) return {0, 0, 0};
        int x, y;
        constexpr int p = numeric_limits<R>::digits;
        uint64_t u = ldexp(frexp(abs(a), &x), p);
        uint64_t v = ldexp(frexp(abs(b), &y), p);
        U m = U(u) * v;
        uint64_t hi = m >> 64;
        int bits = hi ? 64 + bit_width(hi) : bit_width(uint64_t(m));
        return {signbit(a) == signbit(b) ? 1 : -1,
                (long long)x + y - 2 * p + bits, m << (128 - bits)};
    }

    static int cross_sign(P a, P b)
    {
        Product u = product(a.x, b.y), v = product(a.y, b.x);
        if (u.sign != v.sign) return u.sign < v.sign ? -1 : 1;
        if (!u.sign) return 0;
        int c = u.exponent != v.exponent ? (u.exponent < v.exponent ? -1 : 1)
              : u.value != v.value ? (u.value < v.value ? -1 : 1) : 0;
        return u.sign * c;
    }
```

接上页同一结构体：极角比较接口。

```
    bool operator()(P a, P b) const
    {
        assert(isfinite(a.x) && isfinite(a.y) && isfinite(b.x) && isfinite(b.y));
        bool az = a.x == 0 && a.y == 0, bz = b.x == 0 && b.y == 0;
        if (az || bz) return az && !bz;
        auto half = [](P p) { return p.y < 0 || (p.y == 0 && p.x < 0); };
        if (half(a) != half(b)) return half(a) < half(b);
        int c = cross_sign(a, b);
        if (c) return c > 0;
        return a.x != 0 ? abs(a.x) < abs(b.x) : abs(a.y) < abs(b.y);
    }
};
```

54.11 整数点向量与线段判定

功能：整数坐标的二维几何基础：点类型及字典序比较、叉积、点积、距离平方，判断点是否在线段上、两条闭线段是否相交，以及极角比较器 `PolarLess`。

核心思路：所有运算先把坐标差扩展到 `__int128_t` 再相乘，全程精确，无需 eps。`cross(a,b,c)` 为 $(b-a)\times(c-a)$，正值表示 c 在 ab 左侧。`on_segment` 要求三点共线且 $(a-p)\cdot(b-p)\le 0$。`intersect` 先处理某端点与另一线段共线并落在其上的接触情形，否则要求两线段的端点分别严格位于对方直线两侧。`PolarLess` 先按半平面分组，组内用叉积比较极角，同向比长度。

接口契约：坐标绝对值不超过 10^{12}，使用 long long 存点、___int128_t 计算叉积、点积与距离平方。Point 按 x、y 字典序比较；cross(a,b,c) 为 ab 与 ac 的叉积，dot 为点积。on_segment 包含端点及退化线段；intersect 判断闭线段有交点。PolarLess 从正 x 轴按 [0,2π) 逆时针极角排序，同向按长度平方递增；零向量排在最前。同点等价，比较器满足严格弱序。按其他起点排序应整体循环移动，LC 的 (-π,π] 顺序仅将 y<0 后缀移到前面。全部单次运算 O(1)，不使用 epsilon。

```
struct IntegerPlane
{
    using ll = long long;
    using I = __int128_t;

    // Contract: |coordinate|<=1e12. Differences/products widened before arithmetic.
    struct Point
    {
        ll x = 0, y = 0;

        bool operator<(Point b) const { return tie(x, y) < tie(b.x, b.y); }

        bool operator==(Point b) const { return x == b.x && y == b.y; }
    };

    static I cross(Point a, Point b, Point c)
    {
        return (I(b.x) - a.x) * (I(c.y) - a.y) - (I(b.y) - a.y) * (I(c.x) - a.x);
    }

    static I dot(Point a, Point b, Point c)
    {
        return (I(b.x) - a.x) * (I(c.x) - a.x) + (I(b.y) - a.y) * (I(c.y) - a.y);
    }

    static I dist2(Point a, Point b) { return dot(a, b, b); }

    static int sign(I x) { return (x > 0) - (x < 0); }

    static bool on_segment(Point p, Point a, Point b)
    {
        return cross(a, b, p) == 0 && dot(p, a, b) <= 0;
    }

    static bool intersect(Point a, Point b, Point c, Point d)
    {
        int x = sign(cross(a, b, c)), y = sign(cross(a, b, d));
        int z = sign(cross(c, d, a)), w = sign(cross(c, d, b));
        if (!x && on_segment(c, a, b)) return true;
        if (!y && on_segment(d, a, b)) return true;
        if (!z && on_segment(a, c, d)) return true;
        if (!w && on_segment(b, c, d)) return true;
        return x * y < 0 && z * w < 0;
    }
```

极角比较器（接上页同一结构体）：

```
    struct PolarLess
    {
        static int half(Point p) { return p.y < 0 || (p.y == 0 && p.x < 0); }

        bool operator()(Point a, Point b) const
        {
            if (half(a) != half(b)) return half(a) < half(b);
            I c = cross({0, 0}, a, b);
            if (c) return c > 0;
            return dist2({0, 0}, a) < dist2({0, 0}, b);
        }
    };
};
```

模板题使用：Library Checker sort_points_by_argument

题意：将最多 20 万个整数点按 $(-\pi,\pi]$ 的极角排序，坐标绝对值 $\le 10^9$，约定原点角度为 0，同角顺序任意。PolarLess 从正 x 轴按 $[0,2\pi)$ 排序，再把 y<0 的后缀整体移到前面；负 x 轴留在最后，不要把它和负 y 半平面一起前移。仅调用整数点与极角比较器。

所需模板：IntegerPlane（第 602 页，54.11 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/sort_points_by_argument

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<IntegerPlane::Point> p(n);
    for (auto &v : p) cin >> v.x >> v.y;
    sort(p.begin(), p.end(), IntegerPlane::PolarLess{});
    auto first = find_if(p.begin(), p.end(), [](auto v) { return v.y < 0; });
    rotate(p.begin(), first, p.end());
    for (auto v : p) cout << v.x << ' ' << v.y << '\n';
}
```

54.12 整数多边形有向面积

功能：求按边界顺序给出的整数多边形有向面积的两倍（逆时针为正）。

核心思路：鞋带公式：$2S = \sum_i p_i \times p_{i+1}$（下标循环）。每一项是原点与一条边构成的三角形的有向面积两倍，多边形外的部分正负抵消。乘积在 `__int128_t` 中精确计算。

代码依赖：IntegerPlane（第 602 页，54.11 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：polygon_area2(p) 返回 ___int128_t 的有向面积两倍；逆时针为正，反向为负。输入按边界顺序排列，隐式闭合；空集或不足三个点返回 0。单个多边形点数不超过 10^5、坐标绝对值不超过 10^{12}，确保求和在有符号 128 位内。时间 O(n)，额外空间 O(1)。

```
IntegerPlane::I polygon_area2(const vector<IntegerPlane::Point> &p)
{
    using G = IntegerPlane;
    using I = G::I;
    I ans = 0;
    int n = (int)p.size();
    for (int i = 0; i < n; i++)
        ans += I(p[i].x) * p[(i + 1) % n].y - I(p[i].y) * p[(i + 1) % n].x;
    return ans;
}
```

模板题使用：AOJ CGL_3_A

题意：给出逆时针简单多边形，3 至 100 点，整数坐标绝对值不超过 10000，输出面积并保留一位小数。无需凸包，凹多边形也适用。接口返回有向二倍面积，逆时针为正；取绝对值后按奇偶精确打印.0 或.5。本题范围允许将半面积转为 long long；一般 int128 结果不可任意缩窄。

所需模板：IntegerPlane（第 602 页，54.11 节）、polygon_area2（第 605 页，54.12 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.u-aizu.ac.jp/problems/CGL_3_A

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<IntegerPlane::Point> p(n);
    for (auto &a : p) cin >> a.x >> a.y;
    auto s = polygon_area2(p);
    if (s < 0) s = -s;
    cout << (long long)(s / 2) << (s % 2 ? ".5\n" : ".0\n");
}
```

54.13 点在整数简单多边形内

问题描述：判断点 q 在简单多边形（可凹，顺逆时针均可）的外部、边界上还是内部。

核心思路：计算绕数。逐边 $a \to b$：若 q 在边上直接返回边界。否则用半开纵坐标区间判断边是否穿过过 q 的水平线：向上穿过（$a_y \le q_y < b_y$）且 q 在边左侧时绕数加一，向下穿过（$b_y \le q_y < a_y$）且 q 在边右侧时减一。半开区间保证顶点恰在水平线上时只计一次；绕数非零即在内部。

代码依赖：IntegerPlane（第 602 页，54.11 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：polygon_contains(p,q) 返回 0 外部、1 边界、2 内部，输入简单多边形，可顺时针或逆时针，不重复首点。空多边形返回 0；单点与线段按闭集判断边界。绕数算法先判线段，再按半开纵坐标区间累计，避免顶点重复计数。坐标范围同 IntegerPlane，O(n) 时间、O(1) 空间。

```
int polygon_contains(const vector<IntegerPlane::Point> &p, IntegerPlane::Point q)
{
    using G = IntegerPlane;
    using Point = G::Point;
    using I = G::I;
    int winding = 0, n = (int)p.size();
    for (int i = 0; i < n; i++)
    {
        Point a = p[i], b = p[(i + 1) % n];
        if (G::on_segment(q, a, b)) return 1;
        I c = G::cross(a, b, q);
        if (a.y <= q.y && b.y > q.y && c > 0) ++winding;
        if (a.y > q.y && b.y <= q.y && c < 0) --winding;
    }
    return winding ? 2 : 0;
}
```

模板题使用：AOJ CGL_3_C

题意：给定简单多边形和查询点，逐个输出 0 外部、1 边界、2 内部；允许凹多边形，按题目给出的逆时针边界顺序读入，不要求重建凸包。n 至 100、q 至 1000，坐标绝对值至 10000；核心也接受反向边界。

所需模板：IntegerPlane（第 602 页，54.11 节）、polygon_contains（第 606 页，54.13 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.u-aizu.ac.jp/problems/CGL_3_C

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n;
    vector<IntegerPlane::Point> p(n);
    for (auto &v : p) cin >> v.x >> v.y;
    cin >> q;
    while (q--)
    {
        IntegerPlane::Point v;
        cin >> v.x >> v.y;
        cout << polygon_contains(p, v) << '\n';
    }
}
```

54.14 整数输入直线与圆交点

功能：整数输入的直线与圆求交：直线由两整点给出，圆心为整点、半径为整数，返回交点个数类别与交点坐标。

核心思路：先在 `__int128_t` 中精确分类：$v = b - a$，$c = v \times (a - o)$，圆心到直线距离为 $|c|/|v|$，判别式 $D = r^2|v|^2 - c^2$ 的符号精确决定相离、相切或相交。之后才转成 `long double` 构造坐标：垂足 $h = o + \mathrm{perp}(v)\, c/|v|^2$，交点为 $h \mp v\sqrt{D}/|v|^2$。

代码依赖：IntegerPlane（第 602 页，54.11 节）、RealPlane（第 584 页，54.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：line_circle_i64(a,b,o,r)：a、b 定直线，o 为圆心。依赖 IntegerPlane::Point 及 RealPlane::Result。所有输入坐标绝对值 $\le 10^9$，半径在 $0..10^9$；先用 int128 精确判定交点数，再用 long double 构造坐标。a=b 返回 degenerate；相离 none、相切 one、相交 two。结果 p 不排序，one 只含一个点，none/infinite/degenerate 为空。零半径按点圆处理。O(1) 时间和空间。精确的是分类，坐标仍有舍入误差；在 10^9 尺度不承诺绝对 1e-6。AOJ 整数范围 10^4 内的驱动按字典序输出，相切点重复一次。本地高精度与消毒器测试不等于线上 AC。

```
RealPlane::Result line_circle_i64(IntegerPlane::Point a,
                                  IntegerPlane::Point b,
                                  IntegerPlane::Point o,
                                  long long r)
{
    using I = __int128_t;
    using R = RealPlane::R;
    using K = RealPlane::Kind;
    for (auto p : {a, b, o})
        assert(-1000000000 <= p.x && p.x <= 1000000000 && -1000000000 <= p.y &&
               p.y <= 1000000000);
    assert(0 <= r && r <= 1000000000);
    I x = I(b.x) - a.x, y = I(b.y) - a.y;
    I q = x * x + y * y;
    if (q == 0) return {K::degenerate, {}};
    I c = x * (I(a.y) - o.y) - y * (I(a.x) - o.x);
    I d = I(r) * r * q - c * c;
    if (d < 0) return {K::none, {}};
    RealPlane::Point v{R(x), R(y)}, center{R(o.x), R(o.y)};
    auto h = center + RealPlane::perp(v) * (R(c) / R(q));
    if (d == 0) return {K::one, {h}};
    auto w = v * (sqrtl(R(d)) / R(q));
    return {K::two, {h - w, h + w}};
}
```

模板题使用：AOJ CGL_7_D

题意：整数圆心/半径及直线，坐标绝对值 $\le 10^4$、半径 $1..10^4$，q≤1000。题目保证直线非退化且至少一个交点；精确分类后构造坐标，按 x/y 字典序输出，相切点复制一次。绝对误差要求小于 1e-6。

所需模板：IntegerPlane（第 602 页，54.11 节）、RealPlane（第 584 页，54.1 节）、line_circle_i64（第 607 页，54.14 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_7_D

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    IntegerPlane::Point o;
    long long r;
    int q;
    cin >> o.x >> o.y >> r >> q;
    cout << fixed << setprecision(12);
    while (q--)
    {
        IntegerPlane::Point a, b;
        cin >> a.x >> a.y >> b.x >> b.y;
        auto ans = line_circle_i64(a, b, o, r);
        assert(ans.p.size() == 1 || ans.p.size() == 2);
        if (ans.p.size() == 1) ans.p.push_back(ans.p[0]);
        if (tie(ans.p[1].x, ans.p[1].y) < tie(ans.p[0].x, ans.p[0].y))
            swap(ans.p[0], ans.p[1]);
        cout << ans.p[0].x << ' ' << ans.p[0].y << ' ' << ans.p[1].x << ' '
             << ans.p[1].y << '\n';
    }
}
```

54.15 整数输入两圆交点

功能：整数输入的两圆求交：圆心为整点、半径为整数，返回交点个数类别与交点坐标。

核心思路：设 $v = b - a$，$q = |v|^2$。同心时按半径是否相等、是否为 0 区分无交点、一点和无穷多。否则在 `__int128_t` 中计算 $k = q + r^2 - s^2$ 与判别式 $D = 4r^2q - k^2$，其符号精确决定交点个数；再用浮点构造：连心线上的垂足 $h = a + v\,k/(2q)$，两交点为 $h \mp \mathrm{perp}(v)\sqrt{D}/(2q)$。

代码依赖：IntegerPlane（第 602 页，54.11 节）、RealPlane（第 584 页，54.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：circle_intersections_i64(a,r,b,s)：a、b 为圆心。依赖 IntegerPlane::Point 及 RealPlane::Result。所有输入坐标绝对值 $\le 10^9$，半径在 $0..10^9$；先用 int128 精确判定交点数，再用 long double 构造坐标。相离或严格内含返回 none、相切 one、相交 two；同心等正半径返回 infinite，同心零半径返回 one。结果 p 不排序，one 只含一个点，none/infinite/degenerate 为空。零半径按点圆处理。O(1) 时间和空间。精确的是分类，坐标仍有舍入误差；在 10^9 尺度不承诺绝对 1e-6。AOJ 整数范围 10^4 内的驱动按字典序输出，相切点重复一次。本地高精度与消毒器测试不等于线上 AC。

```
RealPlane::Result circle_intersections_i64(IntegerPlane::Point a,
                                           long long r,
                                           IntegerPlane::Point b,
                                           long long s)
{
    using I = __int128_t;
    using R = RealPlane::R;
    using K = RealPlane::Kind;
    for (auto p : {a, b})
        assert(-1000000000 <= p.x && p.x <= 1000000000 && -1000000000 <= p.y &&
               p.y <= 1000000000);
    assert(0 <= r && r <= 1000000000 && 0 <= s && s <= 1000000000);
    I x = I(b.x) - a.x, y = I(b.y) - a.y;
    I q = x * x + y * y;
    RealPlane::Point center{R(a.x), R(a.y)}, v{R(x), R(y)};
    if (q == 0)
    {
        if (r != s) return {K::none, {}};
        if (r == 0) return {K::one, {center}};
        return {K::infinite, {}};
    }
    I k = q + I(r) * r - I(s) * s;
    I d = 4 * I(r) * r * q - k * k;
    if (d < 0) return {K::none, {}};
    auto h = center + v * (R(k) / (2 * R(q)));
    if (d == 0) return {K::one, {h}};
    auto w = RealPlane::perp(v) * (sqrtl(R(d)) / (2 * R(q)));
    return {K::two, {h - w, h + w}};
}
```

模板题使用：AOJ CGL_7_E

题意：整数圆心/半径，坐标绝对值 $\le 10^4$、半径 $1..10^4$；不同圆心且至少一个交点。按 x/y 字典序输出，相切点复制一次；不使用 eps 排序。绝对误差要求小于 1e-6。

所需模板：IntegerPlane（第 602 页，54.11 节）、RealPlane（第 584 页，54.1 节）、circle_intersections_i64（第 609 页，54.15 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_7_E

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    IntegerPlane::Point a, b;
    long long r, s;
    cin >> a.x >> a.y >> r >> b.x >> b.y >> s;
    auto ans = circle_intersections_i64(a, r, b, s);
    assert(ans.p.size() == 1 || ans.p.size() == 2);
    if (ans.p.size() == 1) ans.p.push_back(ans.p[0]);
    if (tie(ans.p[1].x, ans.p[1].y) < tie(ans.p[0].x, ans.p[0].y))
        swap(ans.p[0], ans.p[1]);
    cout << fixed << setprecision(12) << ans.p[0].x << ' ' << ans.p[0].y << ' '
         << ans.p[1].x << ' ' << ans.p[1].y << '\n';
}
```

54.16 圆与简单多边形相交面积

问题描述：求圆与简单多边形（可凹，顺逆时针均可）公共部分的面积。

核心思路：把圆心平移到原点，对多边形每条有向边 $a \to b$ 计算“原点与该边所成三角形和圆的交”的有向面积，求和取绝对值即为答案（与鞋带公式同理，凹处正负抵消）。`edge` 先求边与圆的交点，把边切成至多三段；某段中点在圆内则整段在圆内，贡献三角形面积 $\frac{1}{2}\,p \times q$，否则贡献扇形面积 $\frac{1}{2}r^2\theta$，有向角 θ 由 `atan2l` 求出。各边贡献用 Kahan 补偿求和。

代码依赖：RealPlane（第 584 页，54.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：area(points,circle) 返回非负相交面积；多边形须简单，允许凹多边形、顺时针/逆时针和相邻重复顶点。少于三点或半径零返回零。edge(a,b,r) 返回圆心在原点时有向边 a 到 b 的面积贡献，r 必须非负。按交点分段，内部用三角形，外部用有向扇形；补偿求和，O(n) 时间、O(1) 额外空间。依赖浮点几何的 Point/Circle 和向量运算。输入须有限、尺度适当，平移到圆心后作差、乘积和面积均须可表示；极小交叠及接近相切时仍有浮点误差。

```
struct CirclePolygon
{
    using G = RealPlane;
    using R = G::R;
    using Point = G::Point;
    using Circle = G::Circle;

    // Signed contribution of edge a -> b; circle centered at the origin.
    static R edge(Point a, Point b, R r)
    {
        if (r == 0) return 0;
        Point v = b - a;
        R length = G::norm(v);
        if (length == 0) return 0;
        Point u = v / length;
        R h = G::cross(u, a);
        Point cuts[4] = {a};
        int count = 1;
        if (fabsl(h) <= r)
        {
            R delta = sqrtl(max(R(0), (r - fabsl(h)) * (r + fabsl(h))));
            R t = -G::dot(a, u);
            Point foot = G::perp(u) * h;
            for (R s : {-delta, delta})
                if (t + s > 0 && t + s < length) cuts[count++] = foot + u * s;
        }
        cuts[count++] = b;
```

分段面积与逐边累加（接上页同一结构体与函数）：

```
        R answer = 0;
        for (int i = 1; i < count; i++)
        {
            Point p = cuts[i - 1], q = cuts[i];
            R cross = G::cross(p, q);
            if (G::norm((p + q) / 2) <= r)
                answer += cross / 2;
            else
                answer += r * r * atan2l(cross, G::dot(p, q)) / 2;
        }
        return answer;
    }

    // Simple polygon, either orientation; duplicate consecutive vertices allowed.
    static R area(const vector<Point> &p, Circle c)
    {
        assert(c.r >= 0);
        if (p.size() < 3 || c.r == 0) return 0;
        R answer = 0, error = 0;
        for (int i = 0; i < (int)p.size(); i++)
        {
            R value = edge(p[i] - c.o, p[(i + 1) % p.size()] - c.o, c.r);
            R y = value - error;
            R sum = answer + y;
            error = (sum - answer) - y;
            answer = sum;
        }
        return fabsl(answer);
    }
};
```

模板题使用：AOJ CGL_7_H

题意：原点圆与逆时针简单多边形的公共面积，允许凹多边形。n 为 3..100，整数坐标绝对值 ≤100，半径 1..100。直接传入顶点序列，不取凸包；绝对误差小于 1e-5。

所需模板：RealPlane（第 584 页，54.1 节）、CirclePolygon（第 611 页，54.16 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_7_H

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    long double r;
    cin >> n >> r;
    vector<RealPlane::Point> p(n);
    for (auto &a : p) cin >> a.x >> a.y;
    cout << fixed << setprecision(12) << CirclePolygon::area(p, {{0, 0}, r}) << '\n';
}
```

54.17 两圆公切线与过点切线

问题描述：求两圆的全部公切线（外公切线与内公切线），或过一点作圆的切线；返回每条切线在两圆上的切点和单位法向量，两圆重合时报告无穷多条。

核心思路：设 u 为连心线单位向量，d 为圆心距。切线的单位法向 n 满足 $n \cdot u = c$：外公切线（`side`$=1$）$c = (r_1 - r_2)/d$，内公切线（`side`$=-1$）$c = (r_1 + r_2)/d$；$|c| > 1$ 时该类切线不存在。两条切线的法向为 $n = cu \pm \sqrt{1-c^2}\,\mathrm{perp}(u)$，切点为 $o_1 + r_1 n$ 与 $o_2 + s\,r_2 n$（s 为 `side`）。$|c|$ 接近 1 时两圆相切，只输出一条。点视为半径 0 的圆，`from_point` 直接复用；两点时返回过两点的直线。

代码依赖：RealPlane（第 584 页，54.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：solve(a,b,eps) 返回 Result，包含 infinite 和 lines；infinite 为真表示无穷多条，否则 lines 给出全部切线。每条 Tangent 保存切点 a、b，单位法向量 normal，以及 side。a、b 对应输入两圆，直线方程 dot(normal,x-a)=0；切点相同时仍有效。side=1 外公切线、-1 内公切线、0 两个不同点。半径零表示点，from_point(p,c,eps) 等价于第一圆取点，圆上切点读 b。输入有限，坐标及半径绝对值 ≤1e100，半径非负。eps 默认 1e-12，用于相对圆心距的近相切判断；近相切可能合并为一条，浮点坐标仍有舍入误差。严格同心等半径时 infinite=true，包括同一点；同心不等半径返回空。O(1)；依赖浮点几何的 Point/Circle、点运算及 norm/perp。

```
struct CircleTangents
{
    using G = RealPlane;
    using R = G::R;
    using Point = G::Point;
    using Circle = G::Circle;

    struct Tangent
    {
        Point a, b, normal;
        int side; // 1 outer, -1 inner, 0 two distinct point circles.
    };

    struct Result
    {
        bool infinite = false;
        vector<Tangent> lines;
    };

    // Line: dot(normal, x - a) = 0. Radius zero represents a point.
    static Result solve(Circle a, Circle b, R eps = 1e-12L)
    {
        assert(a.r >= 0 && b.r >= 0 && eps > 0 && eps < 1);
        Point v = b.o - a.o;
        R d = G::norm(v);
        if (d == 0) return {a.r == b.r, {}};
        Point u = v / d;
        if (a.r == 0 && b.r == 0) return {false, {{a.o, b.o, G::perp(u), 0}}};
```

内外公切线枚举（接上页同一函数）：

```
        Result answer;
        for (int side : {1, -1})
        {
            if (side == -1 && (a.r == 0 || b.r == 0)) continue;
            R gap = a.r - side * b.r;
            if (fabsl(gap) > d * (1 + eps)) continue;
            bool touch = fabsl(gap) >= d * (1 - eps);
            R c = touch ? (gap < 0 ? -1 : 1) : gap / d;
            R s = sqrtl(max(R(0), (1 - c) * (1 + c)));
            for (int sign : {-1, 1})
            {
                Point normal = u * c + G::perp(u) * (sign * s);
                Point p = a.o + normal * a.r;
                Point q = b.o + normal * (side * b.r);
                answer.lines.push_back({p, q, normal, side});
                if (touch) break;
            }
        }
        return answer;
    }

    static Result from_point(Point p, Circle c, R eps = 1e-12L)
    {
        return solve({p, 0}, c, eps);
    }
};
```

模板题使用：AOJ CGL_7_F

题意：过圆外整数点作切线，输出圆上的两个切点。坐标绝对值 ≤1000，半径 1..1000，保证点严格在圆外。from_point 把点作为第一圆，圆上切点在 line.b；按 x/y 字典序输出，每行一个点，绝对误差小于 1e-5。

所需模板：RealPlane（第 584 页，54.1 节）、CircleTangents（第 613 页，54.17 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_7_F

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    RealPlane::Point p;
    RealPlane::Circle c;
    cin >> p.x >> p.y >> c.o.x >> c.o.y >> c.r;
    auto ans = CircleTangents::from_point(p, c);
    vector<RealPlane::Point> points;
    for (auto line : ans.lines) points.push_back(line.b);
    sort(points.begin(),
         points.end(),
         [](auto a, auto b) { return tie(a.x, a.y) < tie(b.x, b.y); });
    cout << fixed << setprecision(12);
    for (auto a : points) cout << a.x << ' ' << a.y << '\n';
}
```

相关应用：任意简单整数多边形面积并；符号化右侧极限、共线边编号去重和有理参数排序，浮点面积输出。

54.18 一般多边形面积并：整数精确判定

问题描述：给定若干整数顶点的简单多边形（可凹，彼此可交叉、重叠），求它们并集的面积。

核心思路：并集边界由各多边形的边中不被其他多边形覆盖的部分组成，面积等于对这些部分用鞋带公式求和。先把每个多边形统一成逆时针。对每条边 pq，枚举其他多边形的每条边，求其穿过直线 pq 处的参数 t（分数精确表示，截到 $[0,1]$），按穿越方向记 ±1 覆盖事件；排序后扫描，覆盖数为 0 的参数区间按长度比例计入 $\frac{1}{2}(p-o)\times(q-o)$。两边共线同向时只让编号小的多边形计入，避免重合边界算两次。

思路中的情况	代码中的处理	行
规范化	删去连续重复点和末尾重复首点，零面积多边形丢弃，顺时针的翻转为逆时针	36
选局部原点	包围盒相交的多边形用并查集合并，同一块共用一个原点 o 以减小浮点误差	62
别的边穿过直线 pq	交点参数为 $\frac{(d-c)\times(p-c)}{y-x}$，从右向左穿越记 -1（离开），反之记 $+1$	91
共线重合边	同向重叠部分只在编号较小的多边形中计一次：对编号大者把重叠区间标为已覆盖	92
扫描参数	覆盖数为 0 的区间对应并集边界，按比例累加面积（Kahan 补偿）	118

代码依赖：IntegerPlane（第 602 页，54.11 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：polygon_union_area(polygons) 求任意简单多边形的面积并，允许凹多边形、不同多边形边界交叉/重合以及并集孔洞；每个输入多边形本身不含洞、不自交。整数顶点为 IntegerPlane::Point，坐标绝对值 $\le 10^9$（比基础点类型约定更严格）。自动转逆时针，删除连续重复点及末尾重复首点；零面积退化输入忽略，但不负责检查自交。

共 V 个顶点，时间 $O(V^2 \log V)$、空间 $O(V)$，下标/覆盖数须可用 int 表示。128 位叉积与有理参数排序精确处理拓扑，最终面积 long double 近似；不扰动顶点、不用 eps 排序。包围盒相交的多边形共用局部原点并补偿求和，仍须按题目精度检验最终面积，不声称浮点累加无条件精确。

```cpp
long double polygon_union_area(vector<vector<IntegerPlane::Point>> polygons)
{
    using G = IntegerPlane;
    using P = G::Point;
    using I = G::I;
    using R = long double;
    struct Fraction
    {
        I num, den;
        Fraction(I x = 0, I y = 1) : num(x), den(y)
        {
            assert(den != 0);
            if (den < 0)
            {
                num = -num;
                den = -den;
            }
        }
        bool operator<(const Fraction &b) const
        {
            return num * b.den < b.num * den;
        }
    };
    vector<vector<P>> a;
    for (auto p : polygons)
    {
        for (auto v : p)
            assert(-1000000000 <= v.x && v.x <= 1000000000 &&
                   -1000000000 <= v.y && v.y <= 1000000000);
        p.erase(unique(p.begin(), p.end()), p.end());
        if (p.size() > 1 && p.front() == p.back()) p.pop_back();
        I area = 0;
        int m = p.size();
        for (int i = 0; i < m; i++) area += G::cross({0, 0}, p[i], p[(i + 1) % m]);
        if (area == 0) continue;
        if (area < 0) reverse(p.begin(), p.end());
        a.push_back(move(p));
    }
```

局部原点与边事件初始化（接上页函数）：

```
    int n = a.size();
    vector<array<long long, 4>> box(n);
    vector<int> parent(n);
    iota(parent.begin(), parent.end(), 0);
    auto find = [&](auto &&self, int x) -> int
    {
        if (parent[x] == x) return x;
        return parent[x] = self(self, parent[x]);
    };
    for (int i = 0; i < n; i++)
    {
        auto p = a[i][0];
        box[i] = {p.x, p.y, p.x, p.y};
        for (auto v : a[i])
        {
            box[i][0] = min(box[i][0], v.x);
            box[i][1] = min(box[i][1], v.y);
            box[i][2] = max(box[i][2], v.x);
            box[i][3] = max(box[i][3], v.y);
        }
        for (int j = 0; j < i; j++)
            if (max(box[i][0], box[j][0]) <= min(box[i][2], box[j][2]) &&
                max(box[i][1], box[j][1]) <= min(box[i][3], box[j][3]))
                parent[find(find, i)] = find(find, j);
    }
    R answer = 0;
    R error = 0;
    for (int i = 0; i < n; i++)
    {
        int m = a[i].size();
        P origin = a[find(find, i)][0];
        for (int k = 0; k < m; k++)
        {
            P p = a[i][k];
            P q = a[i][(k + 1) % m];
            vector<pair<Fraction, int>> events;
            auto add = [&](Fraction t, int delta)
            {
                t = max(Fraction(0), min(Fraction(1), t));
                events.push_back({t, delta});
            };
```

右侧覆盖与共线边去重（接上页函数）：

```
        for (int j = 0; j < n; j++)
        {
            if (i == j) continue;
            int size = a[j].size();
            for (int h = 0; h < size; h++)
            {
                P c = a[j][h];
                P d = a[j][(h + 1) % size];
                I x = G::cross(p, q, c);
                I y = G::cross(p, q, d);
                if ((x >= 0) != (y >= 0))
                    add(Fraction(G::cross(c, d, p), y - x), x < 0 ? -1 : 1);
                if (x == 0 && y == 0 && j < i)
                {
                    auto project = [&](P v)
                    {
                        if (p.x != q.x) return Fraction(I(v.x) - p.x, I(q.x) - p.x);
                        return Fraction(I(v.y) - p.y, I(q.y) - p.y);
                    };
                    auto l = project(c);
                    auto r = project(d);
                    if (l < r)
                    {
                        add(l, 1);
                        add(r, -1);
                    }
                }
            }
        }
```

有理参数排序及未覆盖段积分（接上页函数）：

```
            add(Fraction(1), 0);
            sort(events.begin(), events.end(), [](const auto &x, const auto &y)
            {
                return x.first < y.first;
            });
            int cover = 0;
            Fraction last;
            for (auto [t, delta] : events)
            {
                if (cover == 0)
                {
                    I num = t.num * last.den - last.num * t.den;
                    I den = t.den * last.den;
                    R value = R(G::cross(origin, p, q)) * (R(num) / R(den)) / 2;
                    R y = value - error;
                    R next = answer + y;
                    error = (next - answer) - y;
                    answer = next;
                }
                cover += delta;
                last = t;
            }
        }
    }
    return answer;
}
```

精确范围 令坐标界 M=10 的 9 次方。差向量分量至多 2M，叉积、交点参数分子及分母绝对值至多 8M 平方。分式比较的单个交叉乘积至多 64M 四次方，小于 128 位有符号上界；裁到 [0,1] 后的参数差也安全。不能直接沿用 IntegerPlane 允许的更大坐标界。

为何看边的右侧 每个多边形先转成逆时针，当前边的左侧已在并集中。只有其他多边形占据右侧时，该边才成为并集内部，不能贡献边界。把测试线向右侧平移无穷小量，原线上顶点视作在线上方：cross 大于等于 0 对比小于 0，等价于符号化极限，没有实际扰动坐标。向下穿越时覆盖加 1，向上穿越时减 1。凹多边形会产生多个区间，仍由相同事件累计处理。

共享边与端点 两条共线反向边的内部位于两侧，右侧测试已将其剔除；同向共线边仅编号较小的多边形保留，避免重复积分。所有事件参数精确裁到 [0,1]，同参数事件的任意顺序只影响零长度段，不影响面积。退化顶点、相切和部分重叠边均不需要随机扰动。

积分与孔洞 对边 p 到 q 的未覆盖参数段 [l,u]，贡献是 cross(p-origin,q-origin) 乘 (u-l)/2。按逆时针原边方向累加，孔洞自动带负方向，不对单边取绝对值。包围盒相交关系的连通块包含真实并集连通块，按该分组平移不改变闭合积分。最终转 long double 和补偿求和仍有误差；输入内含洞或自交轮廓不在本接口范围。

独立验证 Fraction 精确枚举所有线段交点横坐标，分竖直条带合并各多边形截面，以线性上下包络的梯形公式求真值。包含凹多边形、重复/共线/反向边、孔洞、极值坐标、刚体变换和 1000 三角形压力例。线上状态见验证记录。

模板题使用：P13338 三角形面积并加强版

题意：n≤1000 个非退化三角形，整数坐标绝对值 $\leq 10^6$；误差要求 | 答案-真值 |/max(1,| 真值 |)≤1e-6。此驱动使用一般多边形模板；凹多边形能力另由本地验证覆盖。当前实现已通过 P13338，记录 302630852。

所需模板：IntegerPlane（第 602 页，54.11 节）、polygon_union_area（第 615 页，54.18 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P13338

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<vector<IntegerPlane::Point>> a(n, vector<IntegerPlane::Point>(3));
    for (auto &p : a)
        for (auto &v : p) cin >> v.x >> v.y;
    cout << fixed << setprecision(12) << polygon_union_area(a) << '\n';
}
```

相关应用：圆面积并的圆弧覆盖与格林公式；支持包含、同圆、相切及孔洞，明确浮点局限。

54.19 圆面积并：未覆盖圆弧积分

问题描述：求若干圆（闭圆盘）并集的面积。

核心思路：用格林公式：并集面积等于沿并集边界的积分 $\frac{1}{2}\oint(x\,dy - y\,dx)$，而边界由每个圆上不被其他圆覆盖的圆弧组成。对每个圆，若被另一个圆包含（相同的圆只保留编号最小者）就跳过；否则求出每个相交圆在它上面覆盖的角区间，跨越 0 或 2π 时拆成两段，排序合并后对未覆盖的弧段累加闭式积分。

思路中的情况	代码中的处理	行
删点圆、分块	零半径圆无贡献；相交的圆用并查集合并，每块以代表圆心为原点，减小积分中的平移误差	29
被包含	另一圆更大且包含它，或与它完全相同且编号更小，则该圆无边界贡献	39
覆盖角区间	由三边长求交点对应的半角 α（`atan2l` 配合海伦式），区间为 $[\theta-\alpha, \theta+\alpha]$	50
弧段积分	半角 h、中点角 m 的弧贡献 $r^2h + r\sin h\,(o_x\cos m + o_y\sin m)$	74
合并区间	排序后维护已覆盖右端 `right`，空隙即未覆盖弧	84

代码依赖：RealPlane（第 584 页，54.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：circle_union_area(a) 求闭圆盘的面积并，输入 vector<RealPlane::Circle>，圆心及半径用 long double，半径 ≥0；空集/点圆贡献 0。同圆只保留最早编号的边界，内含圆无贡献；不要求连通或无孔洞。相交连通块各自平移后积分，原输入不改变。

每个圆将其他圆覆盖的角区间切到 [0,2π]，严格排序合并，只积分未覆盖的圆弧。O(n^2 log n) 时间、O(n) 额外空间；并查集递归最深 O(n)。所有输入和中间运算须有限可表示。使用原始浮点比较，不用 eps 排序或随机扰动；近退化分类、极端尺度和最终误差仍受浮点限制。整数题域与本地对拍不构成任意实数的精确保证。

```
RealPlane::R circle_union_area(vector<RealPlane::Circle> a)
{
    using R = RealPlane::R;
    R pi = acosl(-1.L);
    R tau = 2 * pi;
    for (auto c : a)
        assert(isfinite(c.o.x) && isfinite(c.o.y) && isfinite(c.r) && c.r >= 0);
    a.erase(remove_if(a.begin(), a.end(), [](auto c)
    {
        return c.r == 0;
    }), a.end());
    int n = a.size();
    vector<int> parent(n);
    iota(parent.begin(), parent.end(), 0);
    auto find = [&](auto &&self, int x) -> int
    {
        if (parent[x] == x) return x;
        return parent[x] = self(self, parent[x]);
    };
    for (int i = 0; i < n; i++)
        for (int j = 0; j < i; j++)
            if (RealPlane::norm(a[i].o - a[j].o) < a[i].r + a[j].r)
                parent[find(find, i)] = find(find, j);
    R answer = 0;
    R error = 0;
    for (int i = 0; i < n; i++)
    {
        auto c = a[i];
        auto o = c.o - a[find(find, i)].o;
        vector<pair<R, R>> intervals;
        bool covered = false;
```

两圆关系及覆盖角区间（接上页函数）：

```
        for (int j = 0; j < n; j++)
        {
            if (i == j) continue;
            auto v = a[j].o - c.o;
            R d = RealPlane::norm(v);
            R r = c.r;
            R s = a[j].r;
            if (s >= r && d <= s - r && (s > r || j < i))
            {
                covered = true;
                break;
            }
            if (d >= r + s || d <= abs(r - s)) continue;
            R scale = max({d, r, s});
            d /= scale;
            r /= scale;
            s /= scale;
            R h = (r + s - d) * (d + r - s) * (d - r + s) * (d + r + s);
            R angle = atan2l(sqrtl(max(R(0), h)), d * d + (r - s) * (r + s));
            R middle = atan2l(v.y, v.x);
            if (middle < 0) middle += tau;
            R l = middle - angle;
            R u = middle + angle;
            if (l < 0)
            {
                intervals.push_back({0, u});
                intervals.push_back({l + tau, tau});
            }
            else if (u > tau)
            {
                intervals.push_back({l, tau});
                intervals.push_back({0, u - tau});
            }
            else intervals.push_back({l, u});
        }
        if (covered) continue;
```

未覆盖圆弧积分与补偿求和（接上页函数）：

```
        auto add = [&](R l, R r)
        {
            R h = (r - l) / 2;
            R middle = (r + l) / 2;
            R value = c.r * c.r * h;
            if (r - l != tau)
                value += c.r * sinl(h) * (o.x * cosl(middle) + o.y * sinl(middle));
            R y = value - error;
            R next = answer + y;
            error = (next - answer) - y;
            answer = next;
        };
        sort(intervals.begin(), intervals.end());
        R right = 0;
        for (auto [l, r] : intervals)
        {
            if (l > right) add(right, l);
            right = max(right, r);
        }
        if (right < tau) add(right, tau);
    }
    return answer;
}
```

为什么只算未覆盖弧 删去重合边界后，圆盘并边界恰是没有落在其他圆内部的圆弧。各圆逆时针方向的内部位于左侧，所以外边界和孔洞边界自动获得正确符号；不能对每条弧单独取绝对值。相切点面积为零，内含圆无需贡献。

积分公式 对相对连通块原点的圆心 (x, y)，半径 r，弧角从 l 到 u，令 $h = (u - l)/2$、$t = (u + l)/2$。格林公式给出贡献 $r^2 h + r \sin h(x \cos t + y \sin t)$。整圆直接用 πr^2，避免 $\sin \pi$ 舍入误差；补偿求和减少大量正负弧贡献相加的损失。连通块分别平移不改变闭合边界积分。

数值与验证 角度排序使用严格原值比较，同圆按编号去重，不以 eps 合并不同圆。半角通过缩放后的 Heron 乘积与 atan2 求出，避免 acos 靠近正负 1 时的敏感性；仍不是精确几何。独立 70 位高精度验证沿竖直截面、在全部交点横坐标处分段解析积分，另有孔洞、相切、远平移和千圆压力例。线上 AC 状态见验证记录，不能由本地通过推断。

模板题使用：SP8073 CIRU 圆面积并

题意：n≤1000，整数圆心坐标绝对值及半径 ≤1000，允许零半径。输出圆盘面积并，保留三位小数；SPOJ 转交记录 302630657 仍 Waiting。当前核心另已通过用户模板题 U106005，输出精度适配见下一用法。

所需模板：RealPlane（第 584 页，54.1 节）、circle_union_area（第 622 页，54.19 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/SP8073

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<RealPlane::Circle> circles(n);
    for (auto &c : circles) cin >> c.o.x >> c.o.y >> c.r;
    cout << fixed << setprecision(3) << circle_union_area(circles) << '\n';
}
```

相关应用：分层圆弧积分求每个精确覆盖重数的面积，重复圆按编号分层；有独立高精度截面积分验证。

54.20 圆覆盖重数面积：恰好覆盖 k 次

问题描述：给定若干圆，对每个 k 求平面上恰好被 k 个圆覆盖的区域面积。

核心思路：与圆面积并一样用格林公式，但按覆盖层数分别积分。设 S_k 为至少被 k 个圆覆盖的面积。圆上一段弧若恰被 c 个圆覆盖（含自身），它就是“至少 c 层”区域的边界，于是把这段弧的积分计入 `answer[c]`。对每个圆，包含它的圆让基础层数 `count` 加一，相交的圆在角度上产生 $+1/-1$ 事件，排序扫描即得每段弧的层数。最后作差 $S_k - S_{k+1}$ 得到恰好 k 层的面积。

思路中的情况	代码中的处理	行
基础层数	自身计 1；被更大的圆包含、或与编号更小的同圆重合时再加 1，同圆因此落在不同层	45
相交圆	覆盖角区间 $[\theta-\alpha,\theta+\alpha]$ 转成 $+1$、-1 事件，跨越 0 时拆为两段；预置 0、2π 两个哨兵	33
扫描事件	相邻事件之间的弧段按当前层数积分，然后更新层数	99
按层累加	弧段积分计入 `answer[count]`，每层独立做 Kahan 补偿	88
作差	由至少 k 层面积得到恰好 k 层面积	104

代码依赖：RealPlane（第 584 页，54.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：circle_coverage_area(a) 返回长 n+1 的数组，ans[k] 为恰被 k 个闭圆盘覆盖的面积（1≤k≤n）；ans[0]=0 仅为占位，不表示无界平面面积。输入 vector<RealPlane::Circle>，圆心和半径 long double，半径 ≥0，n≤INT_MAX-2；按值传参不改原数组。支持重复圆、零半径、包含、相切和孔洞。

圆弧按覆盖层数积分得到至少 k 次面积，再相邻作差；同圆按编号依次归入不同层，不能去重。O(n^2 log n) 时间、O(n) 额外空间，并查集递归最深 O(n)。输入和中间运算须有限可表示；严格原值排序、分块平移及补偿求和不能消除近退化与小面积差分的浮点误差。不对核心结果强制取绝对值或截零。

```
vector<RealPlane::R> circle_coverage_area(vector<RealPlane::Circle> a)
{
    using R = RealPlane::R;
    R pi = acosl(-1.L);
    R tau = 2 * pi;
    for (auto c : a)
        assert(isfinite(c.o.x) && isfinite(c.o.y) && isfinite(c.r) && c.r >= 0);
    assert(a.size() <= INT_MAX - 2);
    int size = int(a.size());
    a.erase(remove_if(a.begin(), a.end(), [](auto c)
    {
        return c.r == 0;
    }), a.end());
    int n = int(a.size());
    vector<int> parent(n);
    iota(parent.begin(), parent.end(), 0);
    auto find = [&](auto &&self, int x) -> int
    {
        if (parent[x] == x)
            return x;
        return parent[x] = self(self, parent[x]);
    };
    for (int i = 0; i < n; i++)
        for (int j = 0; j < i; j++)
            if (RealPlane::norm(a[i].o - a[j].o) < a[i].r + a[j].r)
                parent[find(find, i)] = find(find, j);
    vector<R> answer(size + 2);
    vector<R> error(size + 2);
    for (int i = 0; i < n; i++)
    {
        auto c = a[i];
        auto o = c.o - a[find(find, i)].o;
        vector<pair<R, int>> events = {{0, 0}, {tau, 0}};
        int count = 1;
```

两圆关系及覆盖角区间（接上页函数）：

```
        for (int j = 0; j < n; j++)
        {
            if (i == j)
                continue;
            auto v = a[j].o - c.o;
            R d = RealPlane::norm(v);
            R r = c.r;
            R s = a[j].r;
            if (s >= r && d <= s - r && (s > r || j < i))
            {
                count++;
                continue;
            }
            if (d >= r + s || d <= abs(r - s))
                continue;
            R scale = max({d, r, s});
            d /= scale;
            r /= scale;
            s /= scale;
            R h = (r + s - d) * (d + r - s) * (d - r + s) * (d + r + s);
            R angle = atan2l(sqrtl(max(R(0), h)), d * d + (r - s) * (r + s));
            R middle = atan2l(v.y, v.x);
            if (middle < 0)
                middle += tau;
            R l = middle - angle;
            R u = middle + angle;
            if (l < 0)
            {
                events.push_back({0, 1});
                events.push_back({u, -1});
                events.push_back({l + tau, 1});
                events.push_back({tau, -1});
            }
            else if (u > tau)
            {
                events.push_back({l, 1});
                events.push_back({tau, -1});
                events.push_back({0, 1});
                events.push_back({u - tau, -1});
            }
            else
            {
                events.push_back({l, 1});
                events.push_back({u, -1});
            }
        }
```

分层圆弧积分与补偿求和（接上页函数）：

```
        auto add = [&](R l, R r)
        {
            R h = (r - l) / 2;
            R middle = (r + l) / 2;
            R value = c.r * c.r * h;
            if (r - l != tau)
                value += c.r * sinl(h) * (o.x * cosl(middle) + o.y * sinl(middle));
            R y = value - error[count];
            R next = answer[count] + y;
            error[count] = (next - answer[count]) - y;
            answer[count] = next;
        };
        sort(events.begin(), events.end());
        R left = 0;
        for (auto [right, delta] : events)
        {
            if (right > left)
                add(left, right);
            count += delta;
            left = right;
        }
    }
    for (int k = 1; k <= size; k++)
        answer[k] -= answer[k + 1];
    answer.resize(size + 1);
    return answer;
}
```

覆盖分层 当前圆弧左侧包含自身，另有 count-1 个圆覆盖；它是至少 count 次覆盖区域的正向边界。分别积分到对应层后，按 k 递增作 answer[k]-answer[k+1] 得到恰好 k 次。重合的 m 个圆按编号让第 t 个额外计入前 t-1 个同圆，从而每个阈值恰贡献一次边界；不能去重。ans[0] 只是占位。

数值与验证 各连通块平移、严格角排序、补偿累加；仍有浮点分类和相邻层相减的消去误差，小面积可能有微小负误差。466 组 70 位竖条截面积分及解析压力例，在 O2、NDEBUG、抄写版与 ASan/UBSan 下通过；不据此承诺任意实数精确。

积分公式 对相对连通块原点的圆心 (x, y)，半径 r，弧角从 l 到 u，令 $h = (u - l)/2$、$t = (u + l)/2$。格林公式给出贡献 $r^2 h + r \sin h(x \cos t + y \sin t)$。整圆直接用 πr^2，避免 $\sin \pi$ 舍入误差；补偿求和减少大量正负弧贡献相加的损失。连通块分别平移不改变闭合边界积分。

模板题使用：SP8119 CIRUT 圆覆盖重数面积

题意：n≤1000，互异圆，整数圆心坐标绝对值 ≤1000、0<r≤1000；输出恰被 k 个圆覆盖的面积，保留三位小数。核心额外支持同圆及零半径，另由本地独立验证覆盖。末端 max 仅避免舍入负零，不改变核心结果。已提交记录 302660709，仍 Waiting，不记 AC。

所需模板：RealPlane（第 584 页，54.1 节）、circle_coverage_area（第 627 页，54.20 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/SP8119

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<RealPlane::Circle> circles(n);
    for (auto &c : circles)
        cin >> c.o.x >> c.o.y >> c.r;
    auto area = circle_coverage_area(circles);
    cout << fixed << setprecision(3);
    for (int k = 1; k <= n; k++)
        cout << '[' << k << "] = " << max(0.L, area[k]) << '\n';
}
```

第五十五章　三维计算几何基础

相关应用：不扰动增量凸包，精确方向符号、低维退化输出、面积及六倍体积。

55.1　三维凸包：精确方向与退化处理

问题描述：求三维点集的凸包：给出外向三角面片，可计算表面积和体积；点集退化为点、线段或平面时也给出相应的边界。坐标可为整数或浮点数，方向判定精确。

核心思路：点按字典序排序去重后，依次找与 p_0, p_1 不共线的 p_c、与三者不共面的 p_d，由此确定维数。共面时丢掉法向第一个非零分量对应的坐标，投影成二维点，用单调链求二维凸包作 `boundary`，再扇形三角化。三维时以这四点建初始四面体（各面朝外），再逐点增量：删去新点能看到（方向为正）的面，被删面中反向边未被删的有向边构成地平线，与新点连成新面。方向符号整数用 `__int128` 精确算；浮点先用误差界过滤，不确定时改用浮点展开式精确求符号。

思路中的情况	代码中的处理	行
排序去重	按坐标字典序排序，相同点只保留编号最小者，`id` 记原编号	154
找不共线的第三点	`normal_sign` 三个分量全为 0 表示共线；找不到则维数为 1	167
找不共面的第四点	找不到则所有点共面，进入平面情形	171
平面情形	丢掉法向第一个非零分量对应的坐标做投影，两遍单调链得逆时针边界，再扇形三角化成 `faces`	176
初始四面体	若第四点在面的正侧就交换两顶点，使各面法向朝外	221
增量加点	可见面被删除，其有向边若反向边也被删则抵消，剩下的就是地平线	240
浮点精确符号	行列式值超过误差界直接取符号，否则用 `grow/product` 的展开式精确计算	114

接口契约：Hull3D<T>(points)，T 只取 long long 或 long double（默认）。输入编号从 0 起，不修改原数组。p 为去重后字典序点，id 映射首次原编号；faces 是 p 下标组成的外向三角面。dimension 为空/点/线/平面/三维时分别为-1/0/1/2/3；低维 boundary 给出点、线端点或平面凸多边形边界。area() 在平面时只算单面，三维时算表面积；volume6() 返回六倍体积，低维为 0。

整数坐标绝对值 $\le 10^9$，方向与六倍体积用 128 位精确运算。实数用过滤后浮点展开式确定符号，不用 eps、不扰动坐标；按已读入的二进制数求凸包，面积和实数体积仍为近似。需最近舍入、禁用 fast-math，运算及残差不得上下溢；足够安全的坐标域和具体约定见后页。

时间 $O(n^2 \log n)$、额外空间 $O(n)$。外露字段用于读取结果，不应构造后自行改写。点数与面下标须能用 int 表示。

字段与面积运算：

```cpp
template <class T = long double>
struct Hull3D
{
    static_assert(is_same_v<T, long long> || is_same_v<T, long double>);
    using W = conditional_t<is_same_v<T, long long>, __int128, long double>;
    struct Point
    {
        T x, y, z;
    };
    vector<Point> p;
    vector<int> id, boundary;
    vector<array<int, 3>> faces;
    int dimension = -1;

    static array<W, 3> normal(Point a, Point b, Point c)
    {
        W x = W(b.x) - a.x;
        W y = W(b.y) - a.y;
        W z = W(b.z) - a.z;
        W u = W(c.x) - a.x;
        W v = W(c.y) - a.y;
        W w = W(c.z) - a.z;
        return {y * w - z * v, z * u - x * w, x * v - y * u};
    }
    static W orient(Point a, Point b, Point c, Point d)
    {
        auto v = normal(a, b, c);
        return v[0] * (W(d.x) - a.x) + v[1] * (W(d.y) - a.y) +
               v[2] * (W(d.z) - a.z);
    }
```

浮点展开式运算（接同一结构体）：

```
// Exact signs for represented floating inputs; nearest rounding, no fast-math.
// Products and residuals must neither overflow nor underflow.
using Expansion = vector<long double>;
static void grow(Expansion &a, long double q)
{
    Expansion out;
    for (long double x : a)
    {
        long double s = q + x;
        long double v = s - q;
        long double error = (q - (s - v)) + (x - v);
        if (error != 0) out.push_back(error);
        q = s;
    }
    if (q != 0) out.push_back(q);
    a.swap(out);
}
static Expansion difference(long double a, long double b)
{
    Expansion result{a};
    grow(result, -b);
    return result;
}
static Expansion product(const Expansion &a, const Expansion &b)
{
    Expansion result;
    for (long double x : a)
        for (long double y : b)
        {
            long double value = x * y;
            grow(result, fmal(x, y, -value));
            grow(result, value);
        }
    return result;
}
static int expansion_sign(const Expansion &a)
{
    if (a.empty()) return 0;
    return a.back() > 0 ? 1 : -1;
}
```

二维方向与共线检查（接同一结构体）：

```
    static int turn_sign(T x, T y, T u, T v, T s, T t)
    {
        W a = W(u) - x;
        W b = W(v) - y;
        W c = W(s) - x;
        W d = W(t) - y;
        W value = a * d - b * c;
        if constexpr (is_same_v<T, long long>)
            return (value > 0) - (value < 0);
        else
        {
            long double bound = 32 * numeric_limits<long double>::epsilon() *
                                (abs(a * d) + abs(b * c));
            if (abs(value) > bound) return value > 0 ? 1 : -1;
            auto result = product(difference(u, x), difference(t, y));
            auto subtract = product(difference(v, y), difference(s, x));
            for (long double z : subtract) grow(result, -z);
            return expansion_sign(result);
        }
    }
    static array<int, 3> normal_sign(Point a, Point b, Point c)
    {
        return {turn_sign(a.y, a.z, b.y, b.z, c.y, c.z),
                turn_sign(a.z, a.x, b.z, b.x, c.z, c.x),
                turn_sign(a.x, a.y, b.x, b.y, c.x, c.y)};
    }
```

三维方向的过滤与精确回退（接同一结构体）：

```
static int orient_sign(Point a, Point b, Point c, Point d)
{
    W value = orient(a, b, c, d);
    if constexpr (is_same_v<T, long long>)
        return (value > 0) - (value < 0);
    else
    {
        array<long double, 3> x{b.x - a.x, b.y - a.y, b.z - a.z};
        array<long double, 3> y{c.x - a.x, c.y - a.y, c.z - a.z};
        array<long double, 3> z{d.x - a.x, d.y - a.y, d.z - a.z};
        long double permanent = 0;
        for (int i = 0; i < 3; i++)
        {
            int j = (i + 1) % 3;
            int k = (i + 2) % 3;
            permanent += (abs(x[j] * y[k]) + abs(x[k] * y[j])) * abs(z[i]);
        }
        long double bound = 128 * numeric_limits<long double>::epsilon() * permanent;
        if (abs(value) > bound) return value > 0 ? 1 : -1;
        array<Expansion, 3> ex{difference(b.x, a.x), difference(b.y, a.y), difference(b.z, a.z)};
        array<Expansion, 3> ey{difference(c.x, a.x), difference(c.y, a.y), difference(c.z, a.z)};
        array<Expansion, 3> ez{difference(d.x, a.x), difference(d.y, a.y), difference(d.z, a.z)};
        Expansion result;
        for (int i = 0; i < 3; i++)
        {
            int j = (i + 1) % 3;
            int k = (i + 2) % 3;
            auto positive = product(product(ex[j], ey[k]), ez[i]);
            auto negative = product(product(ex[k], ey[j]), ez[i]);
            for (long double v : positive) grow(result, v);
            for (long double v : negative) grow(result, -v);
        }
        return expansion_sign(result);
    }
}
```

构造、排序与去重（接同一结构体）：

```
132     explicit Hull3D(const vector<Point> &input)
133     {
134         vector<int> order(input.size());
135         iota(order.begin(), order.end(), 0);
136         for (auto v : input)
137         {
138             if constexpr (is_same_v<T, long long>)
139                 assert(-1000000000 <= v.x && v.x <= 1000000000 &&
140                        -1000000000 <= v.y && v.y <= 1000000000 &&
141                        -1000000000 <= v.z && v.z <= 1000000000);
142             else assert(isfinite(v.x) && isfinite(v.y) && isfinite(v.z));
143         }
144         auto key = [](Point v)
145         {
146             return make_tuple(v.x, v.y, v.z);
147         };
148         sort(order.begin(), order.end(), [&](int a, int b)
149         {
150             return make_tuple(input[a].x, input[a].y, input[a].z, a) <
151                    make_tuple(input[b].x, input[b].y, input[b].z, b);
152         });
153         for (int i : order)
154             if (p.empty() || key(input[i]) != key(p.back()))
155             {
156                 p.push_back(input[i]);
157                 id.push_back(i);
158             }
```

低维识别与平面投影（接同一构造函数）：

```
    int n = p.size();
    if (n == 0) return;
    dimension = 0;
    boundary = {0};
    if (n == 1) return;
    dimension = 1;
    boundary = {0, n - 1};
    int c = 2;
    while (c < n && normal_sign(p[0], p[1], p[c]) == array<int, 3>{}) c++;
    if (c == n) return;
    auto axis = normal_sign(p[0], p[1], p[c]);
    int d = 2;
    while (d < n && orient_sign(p[0], p[1], p[c], p[d]) == 0) d++;
    if (d == n)
    {
        dimension = 2;
        int drop = 0;
        while (axis[drop] == 0) drop++;
        auto project = [&](Point v)
        {
            if (drop == 0) return pair<T, T>{v.y, v.z};
            if (drop == 1) return pair<T, T>{v.x, v.z};
            return pair<T, T>{v.x, v.y};
        };
        auto turn = [&](int a, int b, int c)
        {
            auto [x, y] = project(p[a]);
            auto [u, v] = project(p[b]);
            auto [s, t] = project(p[c]);
            return turn_sign(x, y, u, v, s, t);
        };
```

平面单调链与三角化（接同一构造函数）：

```
            order.resize(n);
            iota(order.begin(), order.end(), 0);
            sort(order.begin(), order.end(), [&](int a, int b)
            {
                return project(p[a]) < project(p[b]);
            });
            boundary.clear();
            for (int pass = 0; pass < 2; pass++)
            {
                int base = boundary.size();
                for (int v : order)
                {
                    while ((int)boundary.size() >= base + 2 &&
                           turn(boundary[boundary.size() - 2], boundary.back(), v) <= 0)
                        boundary.pop_back();
                    boundary.push_back(v);
                }
                boundary.pop_back();
                reverse(order.begin(), order.end());
            }
            for (int i = 1; i + 1 < (int)boundary.size(); i++)
                faces.push_back({boundary[0], boundary[i], boundary[i + 1]});
            return;
        }
```

初始四面体与增量可见面（接同一构造函数）：

```
        dimension = 3;
        boundary.clear();
        array<int, 4> seed{0, 1, c, d};
        for (int i = 0; i < 4; i++)
        {
            array<int, 3> f{seed[(i + 1) % 4], seed[(i + 2) % 4], seed[(i + 3) % 4]};
            if (orient_sign(p[f[0]], p[f[1]], p[f[2]], p[seed[i]]) > 0)
                swap(f[1], f[2]);
            faces.push_back(f);
        }
        for (int v = 0; v < n; v++)
        {
            if (find(seed.begin(), seed.end(), v) != seed.end()) continue;
            vector<array<int, 3>> keep;
            set<pair<int, int>> horizon;
            for (auto f : faces)
            {
                if (orient_sign(p[f[0]], p[f[1]], p[f[2]], p[v]) <= 0)
                {
                    keep.push_back(f);
                    continue;
                }
                for (int k = 0; k < 3; k++)
                {
                    int a = f[k];
                    int b = f[(k + 1) % 3];
                    if (!horizon.erase({b, a})) horizon.insert({a, b});
                }
            }
            for (auto [a, b] : horizon) keep.push_back({a, b, v});
            faces.swap(keep);
        }
    }
```

面积与六倍体积（接同一结构体）：

```
    long double area() const
    {
        long double answer = 0;
        for (auto f : faces)
        {
            auto v = normal(p[f[0]], p[f[1]], p[f[2]]);
            answer += hypotl(hypotl((long double)v[0], (long double)v[1]),
                             (long double)v[2]) / 2;
        }
        return answer;
    }
    W volume6() const
    {
        W answer = 0;
        if (dimension != 3) return answer;
        for (auto f : faces) answer += orient(p[0], p[f[0]], p[f[1]], p[f[2]]);
        return answer;
    }
};
```

不变量 三维初始四面体各面均背向内部。新增点只删除严格可见的面，删除面中的反向公共边两两消去，剩余有向边是轮廓。用轮廓边和新点组成新面即可恢复闭合外向凸包；零方向不算可见，不能扰动共面点。所有点先排序去重，平面情况投影到法向非零的轴，单独用二维凸包处理。

精确符号的范围 浮点展开式保存每次加法和乘法被舍弃的残差，末项符号即精确和的符号；乘法残差通过 fma 计算。先用保守舍入误差界过滤明显非零的方向，不能确定时回退。需二进制、最近舍入、禁用 fast-math 及运算重排，保证运算和残差不上下溢。对于有效位不超过 113 且指数范围至少覆盖普通 double 的实现，每个非零坐标绝对值在 2 的负 100 次方至 2 的 100 次方内是一个足够安全域。输入十进制读取误差不被恢复。

测量和退化约定 精确方向不等于精确面积。面积的平方根与求和、实数体积仍可能失去相对精度，极薄模型需额外检验。整数域内六倍体积精确且非负；平面核心只返回单面面积，点和线面积为 0。题目驱动选择平面双面包裹约定，题面本身没有明确低维语义；其他题应按题意适配。

独立验证 565 组输入分别验证整数和实数版，参考为精确支撑平面枚举及每个面的二维凸包；检查外向、无退化面、反向边配对、Euler 关系、面积和体积。另有 8000 组精确有理数方向参考，含共面、nextafter 近共面、不同指数和减法残差。原先放大整数点导致浮点行列式误判的反例保留。

模板题使用：P4724 三维凸包

题意：4≤n≤2000 个实数点，输出三维凸包表面积，保留三位小数。题面未给坐标界。驱动对纯平面输入采用正反双面包裹面积约定；核心 area() 仍返回平面单面面积。线上状态见验证记录。

所需模板：Hull3D（第 632 页，55.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4724

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<Hull3D<>::Point> p(n);
    for (auto &v : p) cin >> v.x >> v.y >> v.z;
    Hull3D hull(p);
    long double area = hull.area();
    if (hull.dimension == 2) area *= 2;
    cout << fixed << setprecision(3) << area << '\n';
}
```

55.2 整数三维共线、共面与方向

功能： 整数坐标的三维基础判定：四点有向体积（方向，0 表示共面）、三点共线、点是否在线段上。

核心思路： 坐标差扩展到 `__int128_t` 后计算。`orient(a,b,c,d)` 为混合积 $((b-a)\times(c-a))\cdot(d-a)$，即四面体有向体积的 6 倍，正值表示 d 在 $(b-a)\times(c-a)$ 所指的一侧。`collinear` 判断 $(b-a)\times(c-a)$ 是否为零向量；`on_segment` 在共线基础上要求 $(a-p)\cdot(b-p)\le 0$。

接口契约： 坐标绝对值 $\le 10^9$；orient(a,b,c,d) 为有向六倍四面体体积，0 表示共面；collinear 判三点共线，on_segment 支持退化线段。int128 精确判定，不包含三维凸包构造。

```
struct IntegerGeometry3D
{
    using I = __int128_t;

    // |coordinates|<=1e9: triple products fit signed int128.
    struct Point
    {
        long long x, y, z;
    };

    struct Vector
    {
        I x, y, z;
    };

    static Vector diff(Point a, Point b)
    {
        return {I(a.x) - b.x, I(a.y) - b.y, I(a.z) - b.z};
    }

    static Vector cross(Vector a, Vector b)
    {
        return {a.y * b.z - a.z * b.y, a.z * b.x - a.x * b.z, a.x * b.y - a.y * b.x};
    }

    static I dot(Vector a, Vector b) { return a.x * b.x + a.y * b.y + a.z * b.z; }

    static I orient(Point a, Point b, Point c, Point d)
    {
        return dot(cross(diff(b, a), diff(c, a)), diff(d, a));
    }

    static bool collinear(Point a, Point b, Point c)
    {
        auto v = cross(diff(b, a), diff(c, a));
        return !v.x && !v.y && !v.z;
    }

    static bool on_segment(Point p, Point a, Point b)
    {
        return collinear(p, a, b) && dot(diff(a, p), diff(b, p)) <= 0;
    }
};
```

55.3 实数三维向量与夹角

功能：浮点三维向量基础：加减、数乘数除、字典序比较，以及点积、叉积、长度、单位化和两向量夹角。

核心思路：`norm` 用三参数 `hypot` 求长度，避免先平方溢出；`unit` 除以长度得单位向量。`angle` 先把两向量单位化，再用 atan2($|a \times b|, a \cdot b$) 求夹角，比 arccos 在夹角接近 0 或 π 时更稳定。

接口契约：Point 为 long double 三坐标，支持加减、数乘/除和精确字典序；排序禁用 epsilon。dot、cross、norm 为点积、叉积、长度；unit 和 angle 要求非零向量，angle 返回 0..π 的有向向量夹角，弧度制。unit(a)*r 变换长度；点间距离为 norm(a-b)。有限输入且中间量不溢出/下溢，极端尺度须预缩放；浮点不是精确共面判定。O(1) 时间空间。

```
struct RealSpace
{
    using R = long double;

    struct Point
    {
        R x = 0, y = 0, z = 0;

        Point operator+(Point b) const
        {
            return {x + b.x, y + b.y, z + b.z};
        }

        Point operator-(Point b) const
        {
            return {x - b.x, y - b.y, z - b.z};
        }

        Point operator*(R k) const
        {
            return {x * k, y * k, z * k};
        }

        Point operator/(R k) const
        {
            return {x / k, y / k, z / k};
        }

        bool operator<(Point b) const
        {
            return tie(x, y, z) < tie(b.x, b.y, b.z);
        }
    };
```

继续三维运算（接上页同一结构体）：

```
    static R dot(Point a, Point b)
    {
        return a.x * b.x + a.y * b.y + a.z * b.z;
    }

    static Point cross(Point a, Point b)
    {
        return {a.y * b.z - a.z * b.y,
                a.z * b.x - a.x * b.z,
                a.x * b.y - a.y * b.x};
    }

    static R norm(Point a)
    {
        return hypot(a.x, a.y, a.z);
    }

    static Point unit(Point a)
    {
        R len = norm(a);
        assert(len > 0);
        return a / len;
    }

    // Directed vectors: angle in [0, pi], both must be nonzero.
    static R angle(Point a, Point b)
    {
        a = unit(a);
        b = unit(b);
        return atan2(norm(cross(a, b)), dot(a, b));
    }
};

struct IntegerGeometry3D
{
    using I = __int128_t;

    // |coordinates|<=1e9: triple products fit signed int128.
    struct Point
    {
        long long x, y, z;
    };

    struct Vector
    {
        I x, y, z;
    };

    static Vector diff(Point a, Point b)
    {
        return {I(a.x) - b.x, I(a.y) - b.y, I(a.z) - b.z};
    }

    static Vector cross(Vector a, Vector b)
```

```
    {
        return {a.y * b.z - a.z * b.y, a.z * b.x - a.x * b.z, a.x * b.y - a.y * b.x};
    }

    static I dot(Vector a, Vector b) { return a.x * b.x + a.y * b.y + a.z * b.z; }

    static I orient(Point a, Point b, Point c, Point d)
    {
        return dot(cross(diff(b, a), diff(c, a)), diff(d, a));
    }

    static bool collinear(Point a, Point b, Point c)
    {
        auto v = cross(diff(b, a), diff(c, a));
        return !v.x && !v.y && !v.z;
    }

    static bool on_segment(Point p, Point a, Point b)
    {
        return collinear(p, a, b) && dot(diff(a, p), diff(b, p)) <= 0;
    }
};

struct Line3 : RealSpace
{
    Point p, d; // p + t*d; segment uses 0 <= t <= 1.

    Line3(Point p = {}, Point d = {}) : p(p), d(d)
    {
    }

    static Line3 through(Point a, Point b)
    {
        return Line3(a, b - a);
    }

    Point projection(Point q) const
    {
        if (norm(d) == 0) return p;
        Point u = unit(d);
        return p + u * dot(q - p, u);
    }

    Point segment_projection(Point q) const
    {
        R len = norm(d);
        if (len == 0) return p;
        Point u = d / len;
        R t = clamp(dot(q - p, u), R(0), len);
        return p + u * t;
    }

    R distance(Point q) const
    {
        if (norm(d) == 0) return norm(q - p);
        return norm(cross(unit(d), q - p));
```

```
    }

    R segment_distance(Point q) const
    {
        return norm(q - segment_projection(q));
    }

    bool on_segment(Point q, R eps = 1e-10L) const
    {
        assert(eps >= 0);
        return segment_distance(q) <= eps;
    }

    // Right-hand rotation about p + t*d; d must be nonzero.
    Point rotate(Point q, R rad) const
    {
        Point u = unit(d), v = q - p;
        R c = cos(rad), s = sin(rad);
        return p + v * c + cross(u, v) * s + u * (dot(u, v) * (1 - c));
    }
};

struct Plane3 : RealSpace
{
    Point p, n; // n is a unit normal; dot(n, x - p) = 0.

    Plane3(Point p, Point normal) : p(p), n(unit(normal))
    {
    }

    static Plane3 through(Point a, Point b, Point c)
    {
        return Plane3(a, cross(b - a, c - a));
    }

    // dot(normal, x) = h; normal must be nonzero.
    static Plane3 equation(Point normal, R h)
    {
        Point u = unit(normal);
        return Plane3(u * (h / norm(normal)), u);
    }

    R signed_distance(Point q) const
    {
        return dot(n, q - p);
    }

    Point projection(Point q) const
    {
        return q - n * signed_distance(q);
    }

    // 1: one point; 0: none; -1: the nondegenerate line lies in plane.
    int intersect(Line3 l, Point &q, R eps = 1e-10L, R ang = 1e-12L) const
    {
        assert(eps >= 0 && 0 <= ang && ang < 1);
```

```
        R h = signed_distance(l.p);
        if (norm(l.d) == 0)
        {
            if (abs(h) > eps) return 0;
            q = l.p;
            return 1;
        }
        Point u = unit(l.d);
        R t = dot(n, u);
        if (abs(t) <= ang) return abs(h) <= eps ? -1 : 0;
        q = l.p - u * (h / t);
        return 1;
    }

    // 1: one line; 0: distinct parallel planes; -1: coincident planes.
    int intersect(Plane3 f, Line3 &l, R eps = 1e-10L, R ang = 1e-12L) const
    {
        assert(eps >= 0 && 0 <= ang && ang < 1);
        Point d = cross(n, f.n);
        R len = norm(d);
        if (len <= ang) return abs(signed_distance(f.p)) <= eps ? -1 : 0;
        d = d / len;
        Point v = cross(d, n);
        Point q = p + v * (dot(f.n, f.p - p) / dot(f.n, v));
        l = Line3(q, d);
        return 1;
    }
};
```

55.4 三维直线投影、线段距离与绕轴旋转

功能：三维直线 $p+td$（线段取 $0 \le t \le 1$）：求点到直线或线段的投影与距离，判断点是否在线段上，以及把点绕该直线旋转。

核心思路：投影先把方向单位化为 u，取 $p+u\,((q-p)\cdot u)$；线段版把投影长度截断到 $[0,|d|]$。点到直线距离为 $|u\times(q-p)|$，到线段距离为到截断后投影点的距离。`rotate` 用 Rodrigues 公式：令 $v=q-p$，旋转后为 $v\cos\theta+(u\times v)\sin\theta+u\,(u\cdot v)(1-\cos\theta)$，再加回 p。

代码依赖：RealSpace（第 644 页，55.3 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：Line3(p,d) 表示 p+t*d，注意第二参数是方向；through(a,b) 从两个端点构造。线段限制 0≤t≤1。projection/segment_projection 返回最近点，distance/segment_distance 返回距离，on_segment(q,eps) 使用绝对距离容差。d=0 时这些操作按点 p 处理；rotate(q,rad) 必须 d 非零，绕 p+t*d 按右手法则旋转，角度单位弧度。公开 p,d。依赖 RealSpace；O(1)。近退化浮点运算须按输入尺度分析误差。

```
struct Line3 : RealSpace
{
    Point p, d; // p + t*d; segment uses 0 <= t <= 1.

    Line3(Point p = {}, Point d = {}) : p(p), d(d)
    {
    }

    static Line3 through(Point a, Point b)
    {
        return Line3(a, b - a);
```

```
    }

    Point projection(Point q) const
    {
        if (norm(d) == 0) return p;
        Point u = unit(d);
        return p + u * dot(q - p, u);
    }

    Point segment_projection(Point q) const
    {
        R len = norm(d);
        if (len == 0) return p;
        Point u = d / len;
        R t = clamp(dot(q - p, u), R(0), len);
        return p + u * t;
    }
```

继续三维运算（接上页同一结构体）：

```
    R distance(Point q) const
    {
        if (norm(d) == 0) return norm(q - p);
        return norm(cross(unit(d), q - p));
    }

    R segment_distance(Point q) const
    {
        return norm(q - segment_projection(q));
    }

    bool on_segment(Point q, R eps = 1e-10L) const
    {
        assert(eps >= 0);
        return segment_distance(q) <= eps;
    }

    // Right-hand rotation about p + t*d; d must be nonzero.
    Point rotate(Point q, R rad) const
    {
        Point u = unit(d), v = q - p;
        R c = cos(rad), s = sin(rad);
        return p + v * c + cross(u, v) * s + u * (dot(u, v) * (1 - c));
    }
};

struct Plane3 : RealSpace
{
    Point p, n; // n is a unit normal; dot(n, x - p) = 0.

    Plane3(Point p, Point normal) : p(p), n(unit(normal))
    {
    }

    static Plane3 through(Point a, Point b, Point c)
    {
        return Plane3(a, cross(b - a, c - a));
    }

    // dot(normal, x) = h; normal must be nonzero.
    static Plane3 equation(Point normal, R h)
    {
        Point u = unit(normal);
        return Plane3(u * (h / norm(normal)), u);
    }

    R signed_distance(Point q) const
    {
        return dot(n, q - p);
    }

    Point projection(Point q) const
    {
        return q - n * signed_distance(q);
```

```
    }

    // 1: one point; 0: none; -1: the nondegenerate line lies in plane.
    int intersect(Line3 l, Point &q, R eps = 1e-10L, R ang = 1e-12L) const
    {
        assert(eps >= 0 && 0 <= ang && ang < 1);
        R h = signed_distance(l.p);
        if (norm(l.d) == 0)
        {
            if (abs(h) > eps) return 0;
            q = l.p;
            return 1;
        }
        Point u = unit(l.d);
        R t = dot(n, u);
        if (abs(t) <= ang) return abs(h) <= eps ? -1 : 0;
        q = l.p - u * (h / t);
        return 1;
    }

    // 1: one line; 0: distinct parallel planes; -1: coincident planes.
    int intersect(Plane3 f, Line3 &l, R eps = 1e-10L, R ang = 1e-12L) const
    {
        assert(eps >= 0 && 0 <= ang && ang < 1);
        Point d = cross(n, f.n);
        R len = norm(d);
        if (len <= ang) return abs(signed_distance(f.p)) <= eps ? -1 : 0;
        d = d / len;
        Point v = cross(d, n);
        Point q = p + v * (dot(f.n, f.p - p) / dot(f.n, v));
        l = Line3(q, d);
        return 1;
    }
};
```

55.5 三维平面投影与分类求交

功能：三维平面（点 p 与单位法向 n）：可由三点或方程 $n \cdot x = h$ 构造，求有符号距离与投影，以及平面与直线、平面与平面的求交和分类。

核心思路：有符号距离为 $n \cdot (q - p)$，投影为 q 减去该距离乘 n。直线求交：方向单位化为 u，$t = n \cdot u$ 是直线与平面夹角的正弦，$|t|$ 不超过 ang 时视为平行，再按起点到平面的距离区分在面上或无交；否则交点为起点减 $u\,h/t$（h 为起点的有符号距离）。两平面求交：交线方向 $d = n_1 \times n_2$，过短则平行或重合；否则在本平面内沿垂直于 d 的 $v = d \times n_1$ 走到另一平面，得到交线上一点。

代码依赖：Line3（第 648 页，55.4 节）、RealSpace（第 644 页，55.3 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：Plane3(p,normal) 要求非零法向量并单位化；through(a,b,c) 要求三点不共线；equation(n,h) 表示 n 点乘 x=h。signed_distance 保留法向符号，projection 返回垂足。intersect(line,q,eps,ang) 返回 1 单点、0 无交、-1 整条非退化线在面上，退化线按点处理。intersect(plane,line,eps,ang) 返回 1 交线、0 平行不同、-1 重合。仅返回 1 时写输出参数；交线 d 为单位向量。eps 为距离容差，ang 为单位方向的正弦容差；默认 1e-10 和 1e-12。近乎平行按容差分类，非精确拓扑判定；近乎平行求交误差会放大。公开 p 和单位 n，勿破坏单位不变量。依赖 RealSpace/Line3；O(1)。

```
struct Plane3 : RealSpace
```

```
{
    Point p, n; // n is a unit normal; dot(n, x - p) = 0.

    Plane3(Point p, Point normal) : p(p), n(unit(normal))
    {
    }

    static Plane3 through(Point a, Point b, Point c)
    {
        return Plane3(a, cross(b - a, c - a));
    }

    // dot(normal, x) = h; normal must be nonzero.
    static Plane3 equation(Point normal, R h)
    {
        Point u = unit(normal);
        return Plane3(u * (h / norm(normal)), u);
    }

    R signed_distance(Point q) const
    {
        return dot(n, q - p);
    }

    Point projection(Point q) const
    {
        return q - n * signed_distance(q);
    }
```

继续三维运算（接上页同一结构体）：

```
    // 1: one point; 0: none; -1: the nondegenerate line lies in plane.
    int intersect(Line3 l, Point &q, R eps = 1e-10L, R ang = 1e-12L) const
    {
        assert(eps >= 0 && 0 <= ang && ang < 1);
        R h = signed_distance(l.p);
        if (norm(l.d) == 0)
        {
            if (abs(h) > eps) return 0;
            q = l.p;
            return 1;
        }
        Point u = unit(l.d);
        R t = dot(n, u);
        if (abs(t) <= ang) return abs(h) <= eps ? -1 : 0;
        q = l.p - u * (h / t);
        return 1;
    }

    // 1: one line; 0: distinct parallel planes; -1: coincident planes.
    int intersect(Plane3 f, Line3 &l, R eps = 1e-10L, R ang = 1e-12L) const
    {
        assert(eps >= 0 && 0 <= ang && ang < 1);
        Point d = cross(n, f.n);
        R len = norm(d);
        if (len <= ang) return abs(signed_distance(f.p)) <= eps ? -1 : 0;
        d = d / len;
        Point v = cross(d, n);
        Point q = p + v * (dot(f.n, f.p - p) / dot(f.n, v));
        l = Line3(q, d);
        return 1;
    }
};
```

第五十六章　距离

56.1　点到闭线段的距离

功能：求点 p 到闭线段 ab 的最短距离。

核心思路：把 p 投影到直线 ab 上，得参数 $t = \frac{(p-a)\cdot v}{v\cdot v}$（$v = b - a$），再截断到 $[0,1]$：投影落在线段外时最近点就是较近的端点。返回 p 到 $a + tv$ 的距离；$a = b$ 时退化为到该点的距离。

代码依赖：RealPlane（第 584 页，54.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：segment_distance_real(p,a,b) 返回点 p 到闭线段 ab 的 long double 距离。a=b 时返回到端点的距离；否则把投影参数截断至 [0,1]。要求有限且尺度适当的输入；O(1) 时间和空间。

```
RealPlane::R
segment_distance_real(RealPlane::Point p, RealPlane::Point a, RealPlane::Point b)
{
    using G = RealPlane;
    using R = G::R;
    using Point = G::Point;
    Point v = b - a;
    R d = G::dot(v, v);
    if (d == 0) return G::norm(p - a);
    R t = clamp(G::dot(p - a, v) / d, R(0), R(1));
    return G::norm(p - (a + v * t));
}
```

模板题使用：AOJ CGL_2_D

题意：求两条闭线段间距离，q 不超过 1000，整数坐标绝对值不超过 10000。先用 IntegerPlane 精确判相交（含接触和共线重叠），相交输出 0；否则取四个端点到另一线段距离的最小值。单点距离接口参数为 (p,a,b)。题目线段非退化，点线段属于额外接口边界。

所需模板：IntegerPlane（第 602 页，54.11 节）、RealPlane（第 584 页，54.1 节）、segment_distance_real（第 654 页，56.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.u-aizu.ac.jp/problems/CGL_2_D

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    cout << fixed << setprecision(12);
    while (q--)
    {
        vector<IntegerPlane::Point> p(4);
        vector<RealPlane::Point> a(4);
        for (int i = 0; i < 4; i++)
        {
            cin >> p[i].x >> p[i].y;
            a[i] = {(long double)p[i].x, (long double)p[i].y};
        }
        long double ans = 0;
        if (!IntegerPlane::intersect(p[0], p[1], p[2], p[3]))
            ans = min({segment_distance_real(a[0], a[2], a[3]),
                       segment_distance_real(a[1], a[2], a[3]),
                       segment_distance_real(a[2], a[0], a[1]),
                       segment_distance_real(a[3], a[0], a[1])});
        cout << ans << '\n';
    }
}
```

第五十七章　Pick 定理

57.1　Pick 定理：格点、剖分与孔洞

简单、非退化、整数顶点、无孔多边形，A 为面积，I 为严格内部格点数，B 为边界格点数：

$$A = I + \frac{B}{2} - 1, \qquad I = \frac{|S| - B + 2}{2}, \qquad S = \sum_i (x_i y_{i+1} - y_i x_{i+1}),$$

$$B = \sum_i \gcd(|x_{i+1} - x_i|, |y_{i+1} - y_i|).$$

对总和 S 取绝对值，不对单项取。有向面积见第 605 页。有 h 个互不接触、不嵌套的简单孔洞时 $A = I + \frac{B}{2} - 1 + h$，$B$ 含所有内外环，I 不含孔洞内部。非整数顶点、自交、边界相接或退化均不适用；面积非负不能代替简单性检查。坐标 $|x|, |y| \le 10^{12}$、$n \le 10^5$ 时叉积、S 与分子用 128 位。

第五十八章　凸包

58.1　凸多边形外点的两条切线

问题描述：给定严格凸、逆时针的整数多边形和一点 q，若 q 在多边形外，求从 q 向多边形所作两条切线的切点下标（右切点、左切点）；q 在多边形内或边界上时报告无切线。

核心思路：称边 i（$p_i \to p_{i+1}$）可见，若 q 严格在它右侧（`visible`）。从 q 看，可见边在环上连成一段，两个切点正是可见与不可见交替处的顶点。代码先借助 p_0 的两条邻边和以 p_0 为中心的扇形二分（`sector`），各找出一条可见边 v 与一条不可见边 h；于是环上从 h 到 v、从 v 到 h 两段的可见性都只切换一次，各用一次二分（`boundary`）找到切换处的顶点，即右、左切点。

思路中的情况	代码中的处理	行
边 0 可见	取 $v=0$；边 $n-1$ 不可见则 $h=n-1$，否则 q 在 p_0 内角的对顶角内，射线 $q \to p_0$ 穿出多边形的那条边不可见，用 `sector(-1)` 二分找到它	27
只有边 $n-1$ 可见	$v=n-1$，$h=0$	32
两条邻边都不可见	q 在 p_0 的内角中，`sector(1)` 找到包含 q 方向的扇形，其外侧边若不可见则 q 在多边形内或边界上，返回空	39
二分切换点	从 `start` 沿环二分第一条可见性与 `start` 不同的边，返回该边起点	46
输出	$h \to v$ 段的切换点为右切点，$v \to h$ 段的为左切点	57

代码依赖：IntegerPlane（第 602 页，54.11 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：p 须为严格凸、逆时针的整数多边形，无重复末点或共线边界顶点；3≤n≤INT_MAX/2，起始下标任意。所有顶点及 q 坐标绝对值 ≤10^12。返回 optional<pair<int,int» 的原 0-based 下标 (right,left)：对任意顶点 v，cross(q,p[right],v)≤0，cross(q,p[left],v)≥0。若切线贴整条边，选离 q 最近的端点。q 在内部或边界时返回 nullopt。O(log n) 时间、O(1) 额外空间，无预处理且输入不变；先扇形定位异态边，再二分可见边链两端。凸性、方向及数值范围均为前置条件，assert 不验证整个多边形；n<3 不在接口范围。散点先构造 integer_hull，若需原编号须另留映射。

```
optional<pair<int, int>> convex_tangents_i64(const vector<IntegerPlane::Point> &p,
                                             IntegerPlane::Point q)
{
    using G = IntegerPlane;
    assert(p.size() >= 3 && p.size() <= INT_MAX / 2);
    int n = p.size();
    auto visible = [&](int i)
    {
        return G::cross(p[i], p[(i + 1) % n], q) < 0;
    };
    auto sector = [&](int dir)
    {
        int l = 1, r = n - 1;
        while (l + 1 < r)
        {
            int m = (l + r) / 2;
            if (dir * G::cross(p[0], p[m], q) >= 0)
                l = m;
            else
                r = m;
        }
        return l;
    };
```

选择异态种子边并二分可见边链（接上页同一函数）：

```
    int a = G::sign(G::cross(p[0], p[1], q));
    int b = G::sign(G::cross(p[0], p[n - 1], q));
    int v, h;
    if (a < 0)
    {
        v = 0;
        h = b <= 0 ? n - 1 : sector(-1);
    }
    else if (b > 0)
    {
        v = n - 1;
        h = 0;
    }
    else
    {
        v = sector(1);
        h = 0;
        if (!visible(v)) return nullopt;
    }
    auto boundary = [&](int start, int end)
    {
        int l = 0, r = (end - start + n) % n;
        bool first = visible(start);
        while (l + 1 < r)
        {
            int m = (l + r) / 2;
            if (visible((start + m) % n) == first)
                l = m;
            else
                r = m;
        }
        return (start + r) % n;
    };
    return pair{boundary(h, v), boundary(v, h)};
}
```

58.2 点在整数凸包内（二分）

问题描述：判断点 q 在严格逆时针凸多边形的外部、边界上还是内部，每次询问对数时间。

核心思路：以 p_0 为中心把凸多边形剖成扇形三角形。先看 q 是否在射线 p_0p_1 与 p_0p_{n-1} 夹成的角内：在角外即为外部；恰在其中一条边所在直线上时，按是否落在对应边上判边界或外部。否则二分出 q 所在扇区，即满足 $(p_l - p_0) \times (q - p_0) \geq 0$ 的最大 l，最后由 q 相对边 p_lp_{l+1} 的叉积符号给出外部、边界或内部。

代码依赖：IntegerPlane（第 602 页，54.11 节）、polygon_contains（第 606 页，54.13 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：convex_contains_i64(p,q) 返回 0 外部、1 边界、2 内部。要求严格逆时针凸包，无共线边内点、不重复首点，例如 integer_hull 的输出；允许循环旋转起点。空集、单点、线段转用 polygon_contains。坐标范围同 IntegerPlane，O(log n) 时间、O(1) 空间。

```
int convex_contains_i64(const vector<IntegerPlane::Point> &p, IntegerPlane::Point q)
{
    using G = IntegerPlane;
    using I = G::I;
    int n = (int)p.size();
    if (n < 3) return polygon_contains(p, q);
    I a = G::cross(p[0], p[1], q), b = G::cross(p[0], p[n - 1], q);
    if (a < 0 || b > 0) return 0;
    if (!a) return G::on_segment(q, p[0], p[1]) ? 1 : 0;
    if (!b) return G::on_segment(q, p[0], p[n - 1]) ? 1 : 0;
    int l = 1, r = n - 1;
    while (l + 1 < r)
    {
        int m = (l + r) / 2;
        if (G::cross(p[0], p[m], q) >= 0)
            l = m;
        else
            r = m;
    }
    I c = G::cross(p[l], p[r], q);
    return c < 0 ? 0 : c == 0 ? 1 : 2;
}
```

58.3 整数凸包：单调链

问题描述：求平面整数点集的凸包，按逆时针输出顶点，去掉重复点和边上的非顶点。

核心思路：Andrew 单调链。点按 (x, y) 排序去重后，从左到右扫描构造下凸壳：栈顶两点与新点不构成严格左转（叉积 ≤ 0）就弹栈；再从右到左扫描构造上凸壳，弹栈时不越过下凸壳的部分（`low`）。两条链首尾相接，最后弹出重复的起点。因为叉积为 0 也弹栈，共线点都被删去，全共线时只剩两个端点。

代码依赖：IntegerPlane（第 602 页，54.11 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：integer_hull(points) 按值接收 IntegerPlane::Point 序列，输出严格逆时针凸包，不重复首点、不保留共线边内点。自动排序去重；全共线时保留两个端点，空集及单点直接返回。坐标范围同 IntegerPlane。O(n log n) 时间、O(n) 空间；供凸包包含、直径与 Minkowski 和调用。

```
vector<IntegerPlane::Point> integer_hull(vector<IntegerPlane::Point> p)
{
    using G = IntegerPlane;
    using Point = G::Point;
    sort(p.begin(), p.end());
    p.erase(unique(p.begin(), p.end()), p.end());
    int n = (int)p.size();
    if (n < 3) return p;
    vector<Point> h;
    for (Point v : p)
    {
        while (h.size() > 1 && G::cross(h[h.size() - 2], h.back(), v) <= 0)
            h.pop_back();
        h.push_back(v);
    }
    int low = (int)h.size();
    for (int i = n - 2; i >= 0; i--)
    {
        while ((int)h.size() > low && G::cross(h[h.size() - 2], h.back(), p[i]) <= 0)
            h.pop_back();
        h.push_back(p[i]);
    }
    h.pop_back();
    return h; // CCW, no repeated endpoint, no collinear interior points.
}
```

模板题使用：Library Checker static_convex_hull

题意：多组求整数点集的凸包，去重并去掉边上非顶点。输出顶点数及逆时针顶点，不重复首点；空集输出 0，共线时仅保留两端。传 move 交出点集，避免按值传参时额外复制。

所需模板：IntegerPlane（第 602 页，54.11 节）、integer_hull（第 661 页，58.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/static_convex_hull

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    while (tests--)
    {
        int n;
        cin >> n;
        vector<IntegerPlane::Point> points(n);
        for (auto &p : points) cin >> p.x >> p.y;
        auto hull = integer_hull(move(points));
        cout << hull.size() << '\n';
        for (auto p : hull) cout << p.x << ' ' << p.y << '\n';
    }
}
```

相关应用：固定导航没有独立 Minkowski 和条目，按凸包的相关应用归档。

58.4 凸多边形 Minkowski 和

问题描述：给定两个凸多边形 A, B，求 Minkowski 和 $A + B = \{a + b \mid a \in A, b \in B\}$ 的顶点（逆时针）。

核心思路：$A + B$ 的边恰是 A、B 全部边按极角归并的结果。先把两个多边形各自旋转到以最低（y 最小，其次 x 最小）顶点开头，此时两者的边都从同一极角起逆时针排列；从 $a_0 + b_0$ 出发，每步输出当前和点，用叉积比较两条当前边，前进极角较小的一侧，平行时两侧同时前进。点或线段输入不走归并，直接枚举所有和点再调用 `integer_hull`。

代码依赖：IntegerPlane（第 602 页，54.11 节）、integer_hull（第 661 页，58.3 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：minkowski_sum(a,b) 接受不重复首点的严格逆时针凸包；空输入返回空集，点和线段允许作为退化凸包。

点类型 IntegerPlane::Point；输入坐标及两集合坐标和的绝对值不超过 10^{12}。结果不重复首点，非退化时沿逆时针边方向合并，平行同向边同时推进；O(n+m) 时间。

退化输入枚举与另一凸包各点的和再求凸包，O((n+m) log(n+m))。按值传入，内部旋转不修改调用方；依赖 IntegerPlane 的 integer_hull/cross。

```
vector<IntegerPlane::Point> minkowski_sum(vector<IntegerPlane::Point> a,
                                          vector<IntegerPlane::Point> b)
{
    using G = IntegerPlane;
    using P = G::Point;
    using I = G::I;
    if (a.empty() || b.empty()) return {};
    auto plus = [](P u, P v)
    {
        return P{u.x + v.x, u.y + v.y};
    };
    if (a.size() < 3 || b.size() < 3)
    {
        vector<P> p;
        for (P u : a)
            for (P v : b) p.push_back(plus(u, v));
        return integer_hull(p);
    }
    auto start = [](vector<P> &p)
    {
        auto it = min_element(
            p.begin(), p.end(), [](P u, P v) { return tie(u.y, u.x) < tie(v.y, v.x); });
        rotate(p.begin(), it, p.end());
    };
    start(a);
    start(b);
    int n = a.size(), m = b.size(), i = 0, j = 0;
    vector<P> result;
```

合并凸包边（接上页同一函数）：

```
    while (i < n || j < m)
    {
        result.push_back(plus(a[i % n], b[j % m]));
        if (i == n)
        {
            ++j;
            continue;
        }
        if (j == m)
        {
            ++i;
            continue;
        }
        P u{a[(i + 1) % n].x - a[i].x, a[(i + 1) % n].y - a[i].y};
        P v{b[(j + 1) % m].x - b[j].x, b[(j + 1) % m].y - b[j].y};
        I c = G::cross({0, 0}, u, v);
        if (c >= 0) ++i;
        if (c <= 0) ++j;
    }
    return result;
}
```

第五十九章　扫描线

59.1　整数轴对齐矩形面积并

问题描述：给定若干轴对齐矩形（按左、下、右、上给出整数坐标），求它们并集的面积，重叠部分只计一次。

核心思路：扫描线沿 x 方向推进。每个矩形产生进入事件 $(l, d, u, +1)$ 与离开事件 $(r, d, u, -1)$，按横坐标排序；纵坐标离散化后，线段树叶子代表相邻两坐标之间的区间。节点 `cover` 记录该区间被整段覆盖的次数（不下传），`len` 为区间内被覆盖的总长度：`cover` 非零时取整段长度，否则取两孩子之和。处理每个事件前，先把与上一事件的横坐标差乘以根的覆盖长度 `len[1]` 累加进面积。

接口契约：调用 rectangle_union_area(rect)，每项按左、下、右、上存储四个 long long。端点有序，各坐标绝对值不超过 10^{18}；零面积矩形忽略，空集返回 0。返回精确 ___int128 面积；边界接触不增加面积，输入不变，多次调用独立。扫描线处理进出事件，纵向叶子代表相邻坐标间的半开区间。cover 仅记录整段覆盖次数，不下传；非零时取整段长度，为零时合并孩子。同横坐标事件间宽度为 0。时间 O(n log(n+1))、空间 O(n)，递归深度 O(log(n+1))；矩形数至多 268435454，且内存可用。题目坐标在 0 到 10^9，故驱动可将结果转 long long；一般输入不要直接窄化。不含周长、浮点或动态更新。

```
__int128 rectangle_union_area(const vector<array<long long, 4>> &rect)
{
    using ll = long long;
    assert(rect.size() <= (INT_MAX - 8) / 8);
    vector<array<ll, 4>> events;
    vector<ll> y;
    for (auto [l, d, r, u] : rect)
    {
        for (ll x : {l, d, r, u})
            assert(-1000000000000000000LL <= x && x <= 1000000000000000000LL);
        assert(l <= r && d <= u);
        if (l == r || d == u) continue;
        events.push_back({l, d, u, 1});
        events.push_back({r, d, u, -1});
        y.push_back(d);
        y.push_back(u);
    }
    if (events.empty()) return 0;
    sort(events.begin(), events.end());
    sort(y.begin(), y.end());
    y.erase(unique(y.begin(), y.end()), y.end());
    int n = int(y.size()) - 1;
    vector<int> cover(4 * n + 4);
    vector<ll> len(4 * n + 4);
```

覆盖长度维护与横向面积累加（接上页同一函数）：

```
    auto add = [&](auto &&self, int p, int l, int r, int a, int b, int v) -> void
    {
        if (a <= l && r <= b) cover[p] += v;
        else
        {
            int m = (l + r) / 2;
            if (a < m) self(self, p * 2, l, m, a, b, v);
            if (b > m) self(self, p * 2 + 1, m, r, a, b, v);
        }
        if (cover[p]) len[p] = y[r] - y[l];
        else len[p] = r - l == 1 ? 0 : len[p * 2] + len[p * 2 + 1];
    };
    __int128 area = 0;
    ll previous = events.front()[0];
    for (auto [x, d, u, v] : events)
    {
        area += (__int128)(x - previous) * len[1];
        int l = lower_bound(y.begin(), y.end(), d) - y.begin();
        int r = lower_bound(y.begin(), y.end(), u) - y.begin();
        add(add, 1, 0, n, l, r, int(v));
        previous = x;
    }
    return area;
}
```

模板题使用：Library Checker Area of Union of Rectangles

题意：输入 N 个轴对齐矩形，每行依次为 l,d,r,u。1≤N≤500000，0≤l<r≤10^9，0≤d<u≤10^9。直接按左、下、右、上存储，计算并集面积；不是面积和。所有矩形都在边长 10^9 的正方形内，最终面积 ≤10^18，可把 __int128 结果转成 long long 输出。

所需模板：rectangle_union_area（第 665 页，59.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/area_of_union_of_rectangles

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<array<long long, 4>> rect(n);
    for (auto &a : rect) for (auto &x : a) cin >> x;
    // The statement bounds the whole union by [0,1e9]^2.
    cout << (long long)rectangle_union_area(rect) << '\n';
}
```

第六十章　旋转卡壳

60.1　整数凸包直径：旋转卡壳

问题描述：求凸多边形顶点间最远距离的平方（平面点集的直径，散点先求凸包），可同时给出一对最远点的下标。

核心思路：旋转卡壳。按逆时针枚举每条边 p_ip_{i+1}，指针 j 指向离这条边所在直线最远的顶点：只要 p_{j+1} 与该边构成的三角形面积（叉积）更大就前移 j。边逆时针转动时 j 也单调前移，总共只绕一圈。最远点对必出现在某条边的端点与其对踵点之间，故每次用 (p_i,p_j)、(p_{i+1},p_j) 更新答案。

代码依赖：IntegerPlane（第 602 页，54.11 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：convex_diameter2(p,endpoints=nullptr) 返回最远两点距离平方，类型 ___int128_t；不足两点返回 0。输入须为严格逆时针凸包，不重复首点，例如 integer_hull 的输出。坐标范围同 IntegerPlane；旋转卡壳 O(n) 时间、O(1) 额外空间。可传 pair<int,int> 的地址，接收凸包内的 0-based 端点下标；空集为 {-1,-1}，单点为 {0,0}，至少两点时端点不同。不保证并列最优端点的顺序。映射到原始输入编号由调用者完成。

```
IntegerPlane::I convex_diameter2(const vector<IntegerPlane::Point> &p,
                                 pair<int, int> *endpoints = nullptr)
{
    using G = IntegerPlane;
    using I = G::I;
    int n = (int)p.size();
    if (endpoints) *endpoints = n ? pair{0, n - 1} : pair{-1, -1};
    if (n < 2) return 0;
    if (n == 2) return G::dist2(p[0], p[1]);
    I ans = 0;
    auto update = [&](int a, int b)
    {
        I value = G::dist2(p[a], p[b]);
        if (value > ans)
        {
            ans = value;
            if (endpoints) *endpoints = {a, b};
        }
    };
    int j = 1;
    for (int i = 0; i < n; i++)
    {
        int k = (i + 1) % n;
        while (G::cross(p[i], p[k], p[(j + 1) % n]) > G::cross(p[i], p[k], p[j]))
            j = (j + 1) % n;
        update(i, j);
        update(k, j);
    }
    return ans;
}
```

模板题使用：Library Checker furthest_pair

题意：多组求距离最远的两个不同输入点编号。先建凸包，端点接口返回凸包下标，再按坐标映射回输入下标。题目保证至少两点；若凸包只剩一个点，所有坐标相同，输出 0、1。

所需模板：IntegerPlane（第 602 页，54.11 节）、integer_hull（第 661 页，58.3 节）、convex_diameter2（第 667 页，60.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/furthest_pair

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    while (tests--)
    {
        int n;
        cin >> n;
        vector<IntegerPlane::Point> points(n);
        for (auto &p : points) cin >> p.x >> p.y;
        auto hull = integer_hull(points);
        if (hull.size() == 1)
        {
            cout << "0 1\n";
            continue;
        }
        pair<int, int> ends;
        convex_diameter2(hull, &ends);
        int a = -1, b = -1;
        for (int i = 0; i < n; i++)
        {
            if (points[i] == hull[ends.first]) a = i;
            if (points[i] == hull[ends.second]) b = i;
        }
        cout << a << ' ' << b << '\n';
    }
}
```

相关应用：凸包的三组支撑点旋转卡壳，最小包围矩形面积与逆时针四角。

60.2 最小面积包围矩形：旋转卡壳与四角

问题描述：给定凸多边形（严格逆时针凸包），求面积最小的包围矩形，返回面积与四个角点。

核心思路：最优矩形必有一边与凸包某条边共线。枚举每条边 p_ip_{i+1} 作底边方向 e，维护三个随边单调前移的指针：沿 e 投影最大的 `hi`、最小的 `lo`，以及离底边最远的 `top`。设 l, r 为两端的投影、h 为高（都乘了 $|e|$），矩形面积为 $(r-l)h/|e|^2$。更新最优解时，两个底角为 $p_i + e\,l/|e|^2$、$p_i + e\,r/|e|^2$，再加上高向量 $\mathrm{perp}(e)\,h/|e|^2$ 得另两角。首条边的三个指针先线性扫描初始化。

代码依赖：RealPlane（第 584 页，54.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：minimum_rectangle(p) 返回 pair(area,box)，box 是 array<Point,4>，非退化时按逆时针列出最优矩形四角；并列时不指定首点或方向。输入 RealPlane::Point 严格逆时针凸包，不重复首点、不保留共线中间点，不能直接传散点或顺时针多边形。空集返回 0 和四个原点；单点重复四次，线段返回 a,b,b,a；原输入不变。

沿每条凸包边维护前后投影极值与最大高度，恢复四条支撑线的交点。精确支撑比较下 O(n) 时间、O(1) 额外空间；n≤INT_MAX，输入及所有中间运算须有限可表示。浮点版不使用 eps 或扰动，近退化符号及角点精度仍受舍入影响；steps<n 是异常平坦比较的终止保护，不能修复舍入错误（异常时最坏 $O(n^2)$）。有界整数/定点题优先精确构造凸包，再传实数坐标。

```
pair<RealPlane::R, array<RealPlane::Point, 4>> minimum_rectangle(
    const vector<RealPlane::Point> &p)
{
    using G = RealPlane;
    using R = G::R;
    using P = G::Point;
    int n = int(p.size());
    if (n == 0)
        return {0, {}};
    if (n == 1)
        return {0, {p[0], p[0], p[0], p[0]}};
    if (n == 2)
        return {0, {p[0], p[1], p[1], p[0]}};
    int hi = 0;
    int lo = 0;
    int top = 0;
    P e = p[1] - p[0];
    for (int j = 1; j < n; j++)
    {
        if (G::dot(e, p[j] - p[hi]) > 0)
            hi = j;
        if (G::dot(e, p[j] - p[lo]) < 0)
            lo = j;
        if (G::cross(e, p[j] - p[top]) > 0)
            top = j;
    }
    R best = numeric_limits<R>::infinity();
    array<P, 4> box;
```

三组支撑点推进与四角恢复（接上页函数）：

```
    for (int i = 0; i < n; i++)
    {
        e = p[(i + 1) % n] - p[i];
        auto advance = [&](int &j, auto better)
        {
            int steps = 0;
            while (steps < n && better(p[(j + 1) % n] - p[j]))
            {
                j = (j + 1) % n;
                steps++;
            }
        };
        advance(hi, [&](P v)
        {
            return G::dot(e, v) >= 0;
        });
        advance(lo, [&](P v)
        {
            return G::dot(e, v) <= 0;
        });
        advance(top, [&](P v)
        {
            return G::cross(e, v) >= 0;
        });
        R d = G::dot(e, e);
        R l = G::dot(e, p[lo] - p[i]);
        R r = G::dot(e, p[hi] - p[i]);
        R h = G::cross(e, p[top] - p[i]);
        R area = (r - l) * h / d;
        if (area < best)
        {
            best = area;
            P a = p[i] + e * (l / d);
            P b = p[i] + e * (r / d);
            P v = G::perp(e) * (h / d);
            box = {a, b, b + v, a + v};
        }
    }
    return {best, box};
}
```

正确性 最小面积包围矩形至少有一条边与凸包边共线。固定此方向，左右和上方支撑线由投影最小、投影最大和叉积最大三点决定。严格凸包边方向依次旋转，三个最优支撑点只向前移动；相同投影推进到末端，以跨过平行边。第一条边完整扫描初始化，避免从错误局部极值开始。

恢复公式与边界 设底边方向 e、长度平方 d，最小/最大投影 l,r，最大叉积 h。面积为 (r-l)h/d，底边两端为 p+el/d、p+er/d，上边统一加 perp(e)h/d。不要求首角纵坐标最小。输入必须是严格逆时针凸包；任意实数判向和极薄模型不能由通过有界整数题保证。

模板题使用：P3187 最小矩形覆盖

题意：3≤n≤50000，坐标 [0,10]、至多五位小数。乘 10^5 后四舍五入还原整数，integer_hull 精确去重与判向，再将凸包转 long double 作卡壳。输出面积及逆时针四角，10 位小数；面积误差/max(1,S)<1e-4、点到矩形距离<1e-4。已 100 分 AC，记录 302662813；最慢 26ms，总计 113ms。

所需模板：IntegerPlane（第 602 页，54.11 节）、integer_hull（第 661 页，58.3 节）、RealPlane（第 584 页，54.1 节）、minimum_rectangle（第 669 页，60.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3187

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<IntegerPlane::Point> points(n);
    for (auto &p : points)
    {
        long double x, y;
        cin >> x >> y;
        p = {llroundl(x * 100000), llroundl(y * 100000)};
    }
    auto hull = integer_hull(points);
    vector<RealPlane::Point> p;
    for (auto q : hull)
        p.push_back({(long double)q.x, (long double)q.y});
    auto [area, box] = minimum_rectangle(p);
    cout << fixed << setprecision(10) << area / 10000000000.L << '\n';
    for (auto q : box)
        cout << q.x / 100000 << ' ' << q.y / 100000 << '\n';
}
```

第六十一章　半平面交

61.1　整数半平面的有界多边形交

问题描述：给定若干半平面 $ax+by\le c$（整数系数），已知交集有界，求交集多边形的顶点（逆时针，有理坐标）；交集为空、只是点或线段时返回空。

核心思路：按法向量 (a,b) 的极角排序，法向相同的只保留最紧的一条。用双端队列维护当前交的边界线：加入新半平面前，若队尾（队首）两条线的交点不满足它，就弹出队尾（队首）。全部加入后再用队首、队尾互相清理，相邻两线依次求交得到顶点。交点用 (x,y,d) 表示 $(x/d,y/d)$，所有排序和判定都在 `__int128_t` 中精确完成，不用 eps。

思路中的情况	代码中的处理	行
$0x+0y\le c$	$c\ge 0$ 时恒成立直接丢弃，$c<0$ 时交集为空	53
排序	先按法向半平面 `half`，再按叉积比较极角；法向平行同向时比较归一化的 c，更紧的在前	69
同向平行线	只保留排在前面（最紧）的一条	75
加入新半平面	队尾、队首两线的交点在新半平面外（`value` >0）则弹出	76
与队尾反向平行	交集为空或退化，返回空	79
首尾清理	用队首检查队尾交点、用队尾检查队首交点，删去多余的线	82
输出顶点	相邻两线求交，用有理数相等判断去掉重合顶点	89

接口契约：bounded_polygon(lines) 求 a*x+b*y≤c 的交集闭包顶点，Line={a,b,c}，三个整数系数绝对值均 $\le 10^9$。要求交集有界，空集也允许；只返回正面积多边形，点或线段交集返回空向量。无界输入不在接口范围内。Point={x,y,d} 表示 (x/d,y/d)，d>0，分数不必约分；按逆时针返回，不重复首点，相邻重复点已去除。排序与所有构造、判定使用整数和 int128，不使用 epsilon。O(n log n) 时间、O(n) 空间。浮点转换与面积计算需单独处理。

```
struct IntegerHalfplanes
{
    using I = __int128_t;
    using ll = long long;

    struct Line
    {
        ll a, b, c;
    };

    struct Point
    {
        ll x, y, d;
    };

    static I cross(Line u, Line v) { return I(u.a) * v.b - I(u.b) * v.a; }

    static int half(Line u) { return u.b < 0 || (u.b == 0 && u.a < 0); }

    static Point meet(Line u, Line v)
    {
        I d = cross(u, v);
        assert(d != 0);
        I x = I(u.c) * v.b - I(u.b) * v.c;
        I y = I(u.a) * v.c - I(u.c) * v.a;
        if (d < 0)
        {
            d = -d;
            x = -x;
            y = -y;
        }
        return {(ll)x, (ll)y, (ll)d};
    }
```

接上页同一结构体与函数:

```
35      static I value(Line u, Point p)
36      {
37          return I(u.a) * p.x + I(u.b) * p.y - I(u.c) * p.d;
38      }
39
40      static bool same(Point p, Point q)
41      {
42          return I(p.x) * q.d == I(q.x) * p.d && I(p.y) * q.d == I(q.y) * p.d;
43      }
44
45      static vector<Point> bounded_polygon(vector<Line> lines)
46      {
47          vector<Line> h;
48          for (auto u : lines)
49          {
50              assert(-1000000000 <= u.a && u.a <= 1000000000);
51              assert(-1000000000 <= u.b && u.b <= 1000000000);
52              assert(-1000000000 <= u.c && u.c <= 1000000000);
53              if (!u.a && !u.b)
54              {
55                  if (u.c < 0) return {};
56              }
57              else
58                  h.push_back(u);
59          }
60          sort(h.begin(),
61               h.end(),
62               [](Line u, Line v)
63               {
64                   if (half(u) != half(v)) return half(u) < half(v);
65                   I d = cross(u, v);
66                   if (d) return d > 0;
67                   ll x = u.a ? (u.a > 0 ? u.a : -u.a) : (u.b > 0 ? u.b : -u.b);
68                   ll y = v.a ? (v.a > 0 ? v.a : -v.a) : (v.b > 0 ? v.b : -v.b);
69                   return I(u.c) * y < I(v.c) * x;
70               });
```

接上页同一结构体与函数:

```
        deque<Line> q;
        for (int i = 0; i < (int)h.size(); i++)
        {
            Line u = h[i];
            if (i && half(u) == half(h[i - 1]) && cross(u, h[i - 1]) == 0) continue;
            while (q.size() >= 2 && value(u, meet(q[q.size() - 2], q.back())) > 0)
                q.pop_back();
            while (q.size() >= 2 && value(u, meet(q[0], q[1])) > 0) q.pop_front();
            if (!q.empty() && cross(q.back(), u) == 0) return {};
            q.push_back(u);
        }
        while (q.size() >= 3 && value(q.front(), meet(q[q.size() - 2], q.back())) > 0)
            q.pop_back();
        while (q.size() >= 3 && value(q.back(), meet(q[0], q[1])) > 0) q.pop_front();
        if (q.size() < 3 || cross(q.front(), q.back()) == 0) return {};
        vector<Point> answer;
        for (int i = 0; i < (int)q.size(); i++)
        {
            Point p = meet(q[i], q[(i + 1) % q.size()]);
            if (answer.empty() || !same(answer.back(), p)) answer.push_back(p);
        }
        if (answer.size() > 1 && same(answer.front(), answer.back())) answer.pop_back();
        if (answer.size() < 3) return {};
        return answer;
    }
};
```

第六十二章　平面最近点对

62.1　浮点平面最近点对

问题描述：给定平面上若干点，求最近两点之间的距离。

核心思路：分治。先按 x 排序，若有重复点直接得 0。`dfs(l,r)` 返回区间内的最近距离，并把区间按 y 排好序：不超过 3 个点时暴力；否则记下中点横坐标，递归两半后按 y 归并。只有离中线不超过当前答案 `best` 的点才可能构成更近的跨中线点对；按 y 顺序把它们放入 `strip`，每个点只向前比较 y 差不超过 `best` 的点，这样的点只有常数个。

接口契约：solve(points) 返回 long double 最短距离（不是平方），少于两个点返回 nullopt，重复点返回零。Point={x,y}；输入须有限，坐标绝对值 ≤1e100。递归分治，按纵坐标归并，O(n log n) 时间、O(n) 额外空间。用 hypotl 求距离，不先平方；排序不使用 epsilon。浮点运算仍有舍入误差，输入前已丢失的精度无法恢复。

```cpp
struct ClosestPair
{
    using R = long double;

    struct Point
    {
        R x, y;
    };

    // Finite coordinates, |x|, |y| <= 1e100. Returns distance, not its square.
    static optional<R> solve(vector<Point> p)
    {
        int n = p.size();
        if (n < 2) return nullopt;
        sort(p.begin(),
             p.end(),
             [](Point a, Point b) { return tie(a.x, a.y) < tie(b.x, b.y); });
        for (int i = 1; i < n; i++)
            if (p[i].x == p[i - 1].x && p[i].y == p[i - 1].y) return R(0);
        auto by_y = [](Point a, Point b)
        {
            return tie(a.y, a.x) < tie(b.y, b.x);
        };
        auto distance = [](Point a, Point b)
        {
            return hypotl(a.x - b.x, a.y - b.y);
        };
        vector<Point> tmp(n);
        function<R(int, int)> dfs = [&](int l, int r) -> R
        {
            if (r - l <= 3)
            {
                R best = numeric_limits<R>::infinity();
                for (int i = l; i < r; i++)
                    for (int j = l; j < i; j++) best = min(best, distance(p[i], p[j]));
                sort(p.begin() + l, p.begin() + r, by_y);
                return best;
            }
```

分治、归并与跨分界线点对（接上页同一函数）：

```
            int m = (l + r) / 2;
            R x = p[m].x;
            R best = min(dfs(l, m), dfs(m, r));
            merge(p.begin() + l,
                  p.begin() + m,
                  p.begin() + m,
                  p.begin() + r,
                  tmp.begin() + l,
                  by_y);
            copy(tmp.begin() + l, tmp.begin() + r, p.begin() + l);
            vector<Point> strip;
            for (int i = l; i < r; i++)
                if (fabsl(p[i].x - x) <= best)
                {
                    for (int j = (int)strip.size() - 1; j >= 0; j--)
                    {
                        if (p[i].y - strip[j].y > best) break;
                        best = min(best, distance(p[i], strip[j]));
                    }
                    strip.push_back(p[i]);
                }
            return best;
        };
        return dfs(0, n);
    }
};
```

模板题使用：Luogu P1429

题意：求平面最近两点的距离，输出小数点后四位。solve 返回距离而非平方；题目保证至少两点，因此可解引用 optional。坐标按 long double 读取。

所需模板：ClosestPair（第 676 页，62.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1429

```
int main()
{
    int n;
    scanf("%d", &n);
    vector<ClosestPair::Point> p(n);
    for (auto &v : p) scanf("%Lf%Lf", &v.x, &v.y);
    printf("%.4Lf\n", *ClosestPair::solve(p));
}
```

第六十三章　随机增量法

63.1　随机增量最小覆盖圆

问题描述：求覆盖平面上所有给定点的最小圆（最小覆盖圆）。

核心思路：随机增量法。先随机打乱点序，依次加入 p_i：若它在当前圆外，则前 i 个点的最小圆边界必过 p_i，令圆为点圆 p_i 并重扫前面的点；遇到圆外的 p_j 时边界必过 p_i, p_j，先取以它们为直径的圆，再扫描更早的点，遇到圆外的 p_k 就取三点外接圆。随机顺序下每层重建的概率很小，期望线性。`through` 缩放后解外接圆心，三点共线时退化为最远两点的直径圆；判断点在圆外带相对容差。

代码依赖：RealPlane（第 584 页，54.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：solve(points,rng,eps) 返回 optional<Circle>，Circle={o,r}；空集返回 nullopt，单点或重复同一点返回半径零。rng 是调用者提供的 mt19937_64 随机数引擎。期望 O(n) 时间，最坏 $O(n^3)$，O(n) 空间；随机化不可省略。eps 默认 32 倍 long double 机器精度，用于相对半径的包含判断。diameter(a,b) 求直径圆；through(a,b,c) 求外接圆，共线时返回最远点对的直径圆。依赖浮点几何的 Point/Circle 与向量运算。输入及中间运算须有限且尺度适当；接近共线时圆心可能不稳定，无法恢复读入前的精度。

```
struct EnclosingCircle
{
    using G = RealPlane;
    using R = G::R;
    using Point = G::Point;
    using Circle = G::Circle;

    static Circle diameter(Point a, Point b)
    {
        Point o = a + (b - a) / 2;
        return {o, max(G::norm(o - a), G::norm(o - b))};
    }

    static Circle through(Point a, Point b, Point c)
    {
        Point u = b - a, v = c - a;
        R scale = max({fabsl(u.x), fabsl(u.y), fabsl(v.x), fabsl(v.y)});
        if (scale == 0) return {a, 0};
        u = u / scale;
        v = v / scale;
        R det = 2 * G::cross(u, v);
        if (det == 0)
        {
            Circle best = diameter(a, b);
            for (Circle q : {diameter(a, c), diameter(b, c)})
                if (q.r > best.r) best = q;
            return best;
        }
        R x = G::dot(u, u), y = G::dot(v, v);
        Point offset{(x * v.y - y * u.y) / det, (u.x * y - v.x * x) / det};
        Point o = a + offset * scale;
        return {o, max({G::norm(o - a), G::norm(o - b), G::norm(o - c)})};
    }
```

随机增量主过程（接上页同一结构体）：

```
    static optional<Circle>
    solve(vector<Point> p, mt19937_64 &rng, R eps = 32 * numeric_limits<R>::epsilon())
    {
        assert(eps >= 0 && eps < 1);
        if (p.empty()) return nullopt;
        shuffle(p.begin(), p.end(), rng);
        Circle c{p[0], 0};
        auto outside = [&](Point q)
        {
            return G::norm(q - c.o) > c.r * (1 + eps);
        };
        for (int i = 1; i < (int)p.size(); i++)
            if (outside(p[i]))
            {
                c = {p[i], 0};
                for (int j = 0; j < i; j++)
                    if (outside(p[j]))
                    {
                        c = diameter(p[i], p[j]);
                        for (int k = 0; k < j; k++)
                            if (outside(p[k])) c = through(p[i], p[j], p[k]);
                    }
            }
        return c;
    }
};
```

模板题使用：Luogu P1742

题意：求包含全部点的最小圆，先输出半径，再输出圆心坐标。题目保证非空，因此可解引用返回值；显式传随机引擎，不能省略随机打乱。用 long double 读取并输出十二位小数。

所需模板：RealPlane（第 584 页，54.1 节）、EnclosingCircle（第 679 页，63.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1742

```
int main()
{
    int n;
    scanf("%d", &n);
    vector<EnclosingCircle::Point> p(n);
    for (auto &v : p) scanf("%Lf%Lf", &v.x, &v.y);
    mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
    auto c = *EnclosingCircle::solve(p, rng);
    printf("%.12Lf\n%.12Lf %.12Lf\n", c.r, c.o.x, c.o.y);
}
```

第六十四章　计算几何：精确判定与适用范围

64.1　定点凸包、直径和极角排序

两位小数坐标按字符串读成放大 100 倍的整数再求整数凸包，不要读浮点后截断。

旋转卡壳输入须为严格逆时针凸包，不重复首点、不含共线边内点；用整数叉积比较面积，仅在面积严格增大时推进对踵点，空集、单点和线段单独处理。返回 128 位距离平方。

整数极角比较器从正 x 轴按 $[0,2\pi)$ 排序，同向按距离递增，零向量最前：先分半圆，再比精确叉积，最后比长度；坐标轴归属必须明确，不能用 epsilon 相等作排序等价关系。要求 $(-\pi,\pi]$ 时，先按库排序，再把 $y<0$ 后缀循环移到最前（负 x 轴为 π，仍在最后）。

64.2　整数系数的有界半平面交

实现见第 672 页。输入 $ax+by\le c$，系数绝对值不超过 10^9 的整数。要求交集有界，只输出正面积多边形；无解、点交和线段交都返回空向量，空返回不等于闭交集为空。无界输入不在约定内。

两条非平行边界交点用齐次坐标

$$D=a_1b_2-b_1a_2,\quad X=c_1b_2-b_1c_2,\quad Y=a_1c_2-c_1a_2,$$

若 $D<0$ 三者同取负；用 $aX+bY\le cD$ 判断在半平面内。$|X|,|Y|,D\le 2\times10^{18}$ 存 64 位，比较的交叉乘积至多 4×10^{36} 用 128 位。按法向半圆与叉积排序，同向平行约束保留更紧者；零法向约束先单独处理。排序后单调队列线性扫描，总时间 $O(n\log n)$。逆时针边 $(x,y)\to(u,v)$ 转为 $a=v-y,b=x-u,c=ax+by$（左侧为可行域）。鞋带项分子 $X_iY_j-Y_iX_j$、分母 D_iD_j 先以 128 位求值再转浮点。

64.3　最近点对、Minkowski 和与三维整数判定

Minkowski 和见第 662 页。输入为严格逆时针凸包，不重复首点、不含共线冗余点，允许任意起点及空集、单点、线段；输入坐标及任意两点坐标和须在 $[-10^{12},10^{12}]$ 内。非退化时按边方向归并，$O(n+m)$；退化时枚举点和再求凸包，$O((n+m)\log(n+m))$。返回同样约定的严格凸包。

三维判定见第 643 页，坐标绝对值不超过 10^9。`orient(a,b,c,d)` 返回有向六倍四面体体积，零表示共面，交换两点变号；`collinear` 判三点共线，`on_segment` 含端点并支持零长度线段。点差分量至多 2×10^9，叉积分量至多 8×10^{18}，三重积至多 4.8×10^{28}，在 int128 内；公开的叉积、点积辅助函数只对点差及其叉积保证不溢出。不含三维凸包构造。

64.4　浮点最近点对

实现见第 676 页。`ClosestPair::solve` 接收 `Point{x,y}` 数组，返回最短距离（不是平方）；少于两点返回空 optional，重复点返回零。输入为有限 `long double`，坐标绝对值不超过 10^{100}。分治按纵坐标归并，$O(n\log n)$ 时间、$O(n)$ 额外空间；用 `hypotl`，排序不用 epsilon。

64.5　公切线与过点切线

实现见第 613 页。令圆心差 v，$d=|v|$，$u=v/d$；外公切线 $s=1$，内公切线 $s=-1$：

$$n\cdot u=\frac{r_1-sr_2}{d}=c,\qquad n=cu\pm\sqrt{(1-c)(1+c)}\,u^{\perp}.$$

切点 $p = o_1 + r_1 n$、$q = o_2 + s r_2 n$，直线为 $n \cdot (x - p) = 0$。`side` 为 $1/-1$ 表示外/内公切线；非相切分支两条，相切只输出一次。半径零视为点：一个点圆时只枚举外分支；两不同点返回连线，`side=0`；同心等半径返回 `infinite=true`，同心不等半径无公切线。`from_point(p,c)` 把点设为第一圆，圆上切点读 `b`；点在圆上一条，圆内为空。输入有限，坐标和半径绝对值不超过 10^{100}，半径非负。容差 $0 < \varepsilon < 1$ 默认 10^{-12}：$|r_1 - s r_2|$ 与 d 相对差在容差内即合并为相切，近切输入不承诺恢复精确分类。

64.6 随机增量最小覆盖圆

实现见第 679 页。随机打乱后依次加入；圆外点必为新圆边界点，依次固定一、二、三个边界点重扫前缀。期望 $O(n)$，最坏 $O(n^3)$，无递归。必须传入初始化好的 `mt19937_64`，不可省略打乱。空集返回空 optional，单点或重复点半径为零；三点共线时取最远点对的直径圆。`through` 只求外接圆，不是最小覆盖圆。包含判断用相对半径容差，默认 32 倍机器精度；近共线三点外接圆数值敏感。

64.7 圆与简单多边形相交面积

实现见第 611 页。圆心移到原点，沿有向边累加，最后取绝对值。边被交点分成至多三段：小段中点在圆内贡献 $\operatorname{cross}(p,q)/2$，否则贡献有向扇形

$$\frac{r^2}{2}\operatorname{atan2}\big(\operatorname{cross}(p,q), p \cdot q\big).$$

必须保留符号，不能逐边取绝对值。交点取 $h = \operatorname{cross}(u, a)$（$u$ 为单位边向），

$$hu^{\perp} \pm u\sqrt{(r - |h|)(r + |h|)}.$$

$O(n)$ 时间、$O(1)$ 额外空间。多边形须简单，可凹、任意方向、允许相邻重复顶点；半径非负，零半径或少于三个顶点面积为零。极小相交面积不保证相对误差。

64.8 最近点对编号与多边形闭集查询

整数坐标至 10^{12} 时最近点对距离平方可达 8×10^{24}，需用 int128；返回原下标时内部排序下标，不改变输入。

一般简单多边形直接调用 `polygon_contains`，不要先变成凸包；返回 0/1/2 为外部、边界、内部。凸包二分查询要求严格逆时针、无共线边内点、不重复首点。

两组点凸包 A, B，第二组平移 d 后相交当且仅当 $d \in A + (-B)$：先对第二组取负再求 Minkowski 和，对原平移向量做闭集查询（查询不取负，边界算相交）。复杂度 $O(n \log n + m \log m + q \log(n + m))$。

64.9 整数圆交点：精确判定与浮点构造

整数坐标、半径绝对值不超过 10^9，半径可为零。直线：$v = b - a$，$q = v \cdot v$，$c = v \times (a - o)$，$D = r^2 q - c^2$；$q = 0$ 为退化直线，否则按 D 符号分类；垂足 $h = o + \operatorname{perp}(v) c/q$，交点 $h \pm v\sqrt{D}/q$。两圆：$v = b - a$，$q = v \cdot v$，$k = q + r^2 - s^2$，$D = 4r^2 q - k^2$，交点 $a + vk/(2q) \pm \operatorname{perp}(v)\sqrt{D}/(2q)$；$q = 0$ 同心单独处理：相同正半径无穷交点，相同零半径一点。$q, |c| \le 8 \times 10^{18}$，$|k| \le 9 \times 10^{18}$，平方项至多 8.1×10^{37}，须先提升到 int128 再运算；扩大范围需重新核算。只有分类精确，坐标仍有舍入误差，在 10^9 尺度不承诺绝对 10^{-6}。相切只返回一个点；结果按精确字典序排序，不用 eps 比较器。

64.10 公切线：切点次序也需要精确判定

`CircleTangents` 的 `line.a`、`line.b` 分别是第一、第二圆的切点；`from_point` 把给定点视为第一圆，切点读 `line.b`。按切点坐标排序时浮点会错排相等横坐标，应精确比较。

圆心差 $v = (x, y)$，$q = x^2 + y^2$，外、内公切线取 $g = r - s$、$g = r + s$，$D = q - g^2$；$D < 0$ 无此类线，$D = 0$ 一条，否则两条。单位法向 $n = (vg \pm \operatorname{perp}(v)\sqrt{D})/q$；第一圆半径为正时，比较切点等价于比

较法向分量。比较化为 $A+B\sqrt{C}+E\sqrt{F}$ 的符号：令 $u=A+B\sqrt{C}$、$w=E\sqrt{F}$，同号直接决定；异号时求 $u^2-w^2=(A^2+B^2C-E^2F)+2AB\sqrt{C}$ 的符号再乘 $\operatorname{sgn}(u)$。单根式也先判符号再平方。坐标、半径至 10^4 时中间量不超过 2.304×10^{35}，int128 可容纳；不能沿用圆交点的 10^9 范围。相切时用法向定义唯一切线，不用两个重合切点求方向。

64.11 圆交点：容差分类与坐标误差

浮点实现见第 589、591 页。默认按相对容差处理近切，距离误差小不推出交点坐标误差小。显式传 `eps=0` 使用无容差分支，但不等于任意实数上的精确几何。

避免先开方再相减。 直线与两圆分别用上节的 $q,c,\Delta=r^2q-c^2$ 和 $q,k,\Delta=4r^2q-k^2$；直线偏移 $w=v\sqrt{\Delta}/q$，两圆 $h=a.o+vk/(2q)$、$w=\operatorname{perp}(v)\sqrt{\Delta}/(2q)$，返回 $h\pm w$。

AOJ 整数输入域的额外保证。 坐标绝对值不超过 $B=10^4$、半径 $1..B$ 的整数时，所有中间整数小于 $81B^4<2^{60}$；`long double` 有效尾数至少 64 位、正常舍入且不启用 fast-math 时，判别式符号精确。53 位尾数不够。

构造与排序。 上述条件下交点每坐标误差小于 1.1×10^{-14}，不同交点非零横坐标间隔至少 1.25×10^{-9}，故可直接按 (x,y) 字典序排序，不用 epsilon 比较器。

范围之外。 其他实数和更大尺度上判别式仍可能消去、上溢或下溢；`RealGeometry` 构造仍要求正 eps。

64.12 三维投影、旋转与平面求交的数值约定

三维向量、直线与线段、平面分别见第 644、648、651 页。

`Line3(p,d)` 第二参数是方向，表示 $p+td$；端点构造用 `Line3::through(a,b)`。投影到直线不截断，到线段限制 $0\le t\le1$。零方向按点处理，不能作旋转轴。向量排序用精确字典序。单位轴 u、$v=q-p$ 的右手旋转（弧度）：

$$p+v\cos\theta+(u\times v)\sin\theta+u(u\cdot v)(1-\cos\theta).$$

`angle(a,b)` 返回 $[0,\pi]$；无向平面夹角取 $\min(\theta,\pi-\theta)$。

平面存点 p 与单位法向 n：$n\cdot(x-p)=0$。`equation(n,h)` 表示 $n\cdot x=h$，一般式 $Ax+By+Cz+D=0$ 传 $h=-D$；法向非零，三点不共线。有符号距离 $n\cdot(q-p)$，投影 $q-n[n\cdot(q-p)]$。求交返回 1 单点或交线、0 无交、-1 重合或线在面上；仅返回 1 时写输出参数。两平面非平行时取 $d=\operatorname{unit}(n\times m)$、$v=d\times n$，

$$q=p+v\frac{m\cdot(f.p-p)}{m\cdot v},\qquad \text{交线为}q+td,$$

分母不能取绝对值。默认距离容差 10^{-10}、角度正弦容差 10^{-12}；近平行分类依赖尺度与容差，需要精确共面谓词时用第 643 页的整数版本。

64.12.1 原三维片段中需要避开的错误

坐标须有限，中间量避免溢出和下溢；平面角为 $\arccos\big((n\cdot m)/(|n||m|)\big)$，闭线段判定用点积范围而非等于零。巨大平移与微小差值、近共线和近平行求交会放大误差，应先平移/缩放并按题意选容差；不能假定 `long double` 总比 `double` 精确。

第八部分

杂项

第六十五章　离线算法

65.1　CDQ 分治

65.1.1　静态三维偏序计数

问题描述：给定 n 个三维点，对每个点求有多少**其他**点的三个坐标都不大于它的对应坐标（三维偏序计数；坐标完全相同的点互相计入）。

核心思路：CDQ 分治。先按三维字典序排序，把完全相同的点合并成一组并记重数 `w`，第三维离散化。对组序列分治：左半的组在字典序上都排在右半之前，第一维已满足。两半递归返回后各自已按第二维 `y` 有序，归并时对右半每个组，把左半中 `y` 不超过它的组按第三维以重数加入树状数组，再查第三维前缀和累加到 `cnt`；归并完撤销左半贡献，同时完成按 `y` 的归并排序。最后每个点的答案为其组的 `cnt` 加同组其他点数 $w-1$。

代码依赖：Fenwick（第 284 页，31.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：count(points) 返回每个原输入位置被其他点逐坐标 ≤ 的次数，重复坐标仍是不同点，排除自身。输入 array<long long,3>，完整 signed64 坐标，空输入返回空；输入不变。内部按三维字典序合并重合点，压缩第三维，CDQ 归并与 Fenwick 回滚。O(n $\log^2$(n+1)) 时间、O(n) 空间、O(log(n+1)) 递归；n 须 ≤INT_MAX/2 且内存可分配，保证共享 Fenwick 的 int 步进不溢出。仅静态无权弱偏序，不含严格不等号、动态更新或 CDQ 递推。

```
struct Dominance3D
{
    static vector<int> count(const vector<array<long long, 3>> &p)
    {
        assert(p.size() <= INT_MAX / 2);
        int n = int(p.size());
        vector<int> order(n), group(n), w;
        iota(order.begin(), order.end(), 0);
        sort(order.begin(), order.end(), [&](int i, int j) { return p[i] < p[j]; });
        vector<array<long long, 3>> v;
        for (int i : order)
        {
            if (v.empty() || v.back() != p[i])
            {
                v.push_back(p[i]);
                w.push_back(0);
            }
            ++w.back();
            group[i] = int(v.size()) - 1;
        }
        int m = int(v.size());
        vector<long long> z;
        for (auto q : v) z.push_back(q[2]);
        sort(z.begin(), z.end());
        z.erase(unique(z.begin(), z.end()), z.end());
        struct Node
        {
            long long y;
            int z, w, id;
        };
        vector<Node> a(m), tmp(m);
        vector<int> cnt(m), ans(n);
        for (int i = 0; i < m; ++i)
            a[i] = {v[i][1], int(lower_bound(z.begin(), z.end(), v[i][2]) -
                                 z.begin()) + 1, w[i], i};
        Fenwick<int> bit(int(z.size()));
```

三维偏序的 CDQ 归并与原序返回（接上页同一 count 函数）：

```
        auto cdq = [&](auto &&self, int l, int r) -> void
        {
            if (r - l <= 1) return;
            int mid = l + (r - l) / 2;
            self(self, l, mid);
            self(self, mid, r);
            int i = l, k = l;
            for (int j = mid; j < r; ++j)
            {
                while (i < mid && a[i].y <= a[j].y)
                {
                    bit.add(a[i].z, a[i].w);
                    tmp[k++] = a[i++];
                }
                cnt[a[j].id] += bit.sum(a[j].z);
                tmp[k++] = a[j];
            }
            for (int t = l; t < i; ++t) bit.add(a[t].z, -a[t].w);
            while (i < mid) tmp[k++] = a[i++];
            copy(tmp.begin() + l, tmp.begin() + r, a.begin() + l);
        };
        cdq(cdq, 0, m);
        for (int i = 0; i < n; ++i)
            ans[i] = cnt[group[i]] + w[group[i]] - 1;
        return ans;
    }
};
```

模板题使用：Luogu P3810 【模板】三维偏序 / 陌上花开

题意：1≤n≤100000，1≤ 三个坐标 ≤k≤200000。对每个输入点统计其他点三个坐标均 ≤ 自身的次数，输出次数为 0..n−1 的点数直方图。重复位置分别计数且各自排除自己。k 只约束题面坐标，库内部压缩，无需外部初始化；空输入和 signed64 极值另属 API 扩展。

所需模板：Fenwick（第 284 页，31.2 节）、Dominance3D（第 686 页，65.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3810

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, k;
    cin >> n >> k;
    vector<array<long long, 3>> p(n);
    for (auto &[x, y, z] : p) cin >> x >> y >> z;
    vector<int> hist(n);
    for (int x : Dominance3D::count(p)) ++hist[x];
    for (int x : hist) cout << x << '\n';
}
```

65.1.2 CDQ 卷积递推：贡献只跨越一次

`cdq_convolution` 求

$$f_i = b_i + \sum_{j<i} f_j g_{i-j}, \qquad g_0 = 0.$$

半开区间 $[l, r)$：先递归左半 $[l, m)$，再与 $g_{0\ldots r-l-1}$ 卷积加入右半，最后递归右半；顺序不可改。根层卷积长度 $n + \lfloor n/2 \rfloor - 1$ 须不超过 NTT 最大二次幂容量。时间 $O(n \log^2(n+1))$，额外空间 $O(n)$。

65.2 整体二分

65.2.1 整体二分：动态区间第 k 小

问题描述：数组支持单点赋值，并多次询问某一时刻闭区间 $[l, r]$ 中第 k 小的值；所有操作先登记，再离线一次求出全部答案。

核心思路：整体二分。初值和修改都拆成按时间排列的事件：在位置 `l` 加入某值（`k` 为 $+1$）或删去旧值（`k` 为 -1），询问是另一类事件；值先离散化。`solve` 处理答案位于值域 $[lo, hi]$ 的所有事件：按时间顺序扫描，值不超过 mid 的修改在树状数组的对应位置加上 `k`，询问查区间内这类值的个数 `smaller`；k 不超过它则答案在左半，否则 k 减去它进入右半，修改按值分到两半。扫完撤销树状数组，稳定划分保持时间顺序，再分别递归；值域缩成一点时即为该组询问的答案。

代码依赖：Fenwick（第 284 页，31.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：OverallKth(a) 读取 0-based 非空初始数组；后续位置用 1..n。set(pos,value) 登记单点赋值；query(l,r,k) 登记闭区间第 k 小并返回 0-based 查询 ID，1<=k<=r-l+1。run() 按查询 ID 返回 vector<long long>，答案对应登记查询时的状态，不是最后数组；可重复 run，也可继续登记后重新 run。支持完整 long long 值域，不做值的算术；同值修改跳过。依赖 Fenwick<int>，递归值域分治，保持时间顺序并撤销贡献。n<=INT_MAX/2，事件数 E=n+2U+Q<=INT_MAX（U 为实际变值次数），另须满足内存限制。V 为出现过的不同值数，时间 O(E log(E+1)+E log(n+1) log(V+1))，空间 O(n+E)。只读 current/events/queries，不外部修改。

```
struct OverallKth
{
    using ll = long long;

    struct Event
    {
        int l, r, k, id;
        ll value;
    };

    int n, queries = 0;
    vector<ll> current;
    vector<Event> events;

    OverallKth(const vector<ll> &a) : n(a.size()), current(a)
    {
        assert(n > 0);
        for (int i = 1; i <= n; i++)
        {
            events.push_back({i, 0, 1, -1, a[i - 1]});
        }
    }

    void set(int pos, ll value)
    {
        assert(1 <= pos && pos <= n);
        if (current[pos - 1] == value) return;
        events.push_back({pos, 0, -1, -1, current[pos - 1]});
        events.push_back({pos, 0, 1, -1, value});
        current[pos - 1] = value;
    }

    int query(int l, int r, int k)
    {
        assert(1 <= l && l <= r && r <= n);
        assert(1 <= k && k <= r - l + 1);
        int id = queries++;
        events.push_back({l, r, k, id, 0});
        return id;
    }
```

继续求解（接上页同一结构体）：

```
    vector<ll> run() const
    {
        vector<ll> answer(queries), values;
        if (queries == 0) return answer;
        auto a = events;
        for (auto e : a)
        {
            if (e.id < 0) values.push_back(e.value);
        }
        sort(values.begin(), values.end());
        values.erase(unique(values.begin(), values.end()), values.end());
        for (auto &e : a)
        {
            if (e.id < 0)
            {
                e.value = lower_bound(values.begin(), values.end(), e.value) - values.begin();
            }
        }
        vector<Event> buffer(a.size());
        vector<bool> left(a.size());
        Fenwick<int> bit(n);
```

继续求解（接上页同一结构体）：

```
        auto solve = [&](auto &&self, int lo, int hi, int begin, int end) -> void
        {
            if (begin == end) return;
            if (lo == hi)
            {
                for (int i = begin; i < end; i++)
                {
                    if (a[i].id >= 0) answer[a[i].id] = values[lo];
                }
                return;
            }
            int mid = lo + (hi - lo) / 2, count = 0;
            for (int i = begin; i < end; i++)
            {
                auto &e = a[i];
                if (e.id < 0)
                {
                    left[i] = e.value <= mid;
                    if (left[i]) bit.add(e.l, e.k);
                }
                else
                {
                    int smaller = bit.query(e.l, e.r);
                    left[i] = e.k <= smaller;
                    if (!left[i]) e.k -= smaller;
                }
                count += left[i];
            }
            int p = begin, q = begin + count;
            for (int i = begin; i < end; i++)
            {
                auto e = a[i];
                if (e.id < 0 && left[i]) bit.add(e.l, -e.k);
                if (left[i]) buffer[p++] = e;
                else buffer[q++] = e;
            }
            copy(buffer.begin() + begin, buffer.begin() + end, a.begin() + begin);
            self(self, lo, mid, begin, begin + count);
            self(self, mid + 1, hi, begin + count, end);
        };
        solve(solve, 0, int(values.size()) - 1, 0, a.size());
        return answer;
    }
};
```

65.3 莫队算法

65.3.1 普通莫队算法

65.3.1.1 曼哈顿 MST 的递归 DFS 查询顺序

功能：给定平面整数点（例如把莫队询问 $[l,r)$ 看作点 (l,r)），返回一个访问顺序，使相邻两点的曼哈顿距离之和不超过曼哈顿最小生成树权值的两倍，可作为莫队处理询问的顺序。

核心思路：先用 `ManhattanMST::solve` 求曼哈顿最小生成树并建邻接表，再从 0 号点递归 DFS，按先序输出。DFS 沿每条树边恰好往返一次，由三角不等式，先序中相邻两点的距离不超过它们在树上路径的长度，总和至多为树权的两倍。

代码依赖：ManhattanMST（第 448 页，41.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：输入二维整数点，返回从原编号 0 开始的 DFS 先序排列；空输入返回空序列，重复点分别保留。可把静态区间 [l,r) 看作点 (l,r)，按返回编号移动莫队双指针。只负责排序，统计和答案由调用处维护。

复用 ManhattanMST 构造树，递归 DFS，每个点恰好一次。设 MST 权值为 T，由 DFS 往返及三角不等式，相邻访问点的 L1 距离之和 ≤2T。起始空区间到首个查询的移动另计；排序 O(q log(q+1)) 时间、O(q) 空间与最坏 O(q) 递归栈。不保证比普通分块更快，也不保证最短访问顺序。点坐标可为 signed64，距离界用 int128。查询数须 ≤INT_MAX；输入不变。P1494 当前实现已 100 分 AC，10 点单点最大 113ms、峰值 10.49MB；排名未核查。

```
vector<int> manhattan_order(const vector<ManhattanMST::Point> &p)
{
    assert(p.size() <= INT_MAX);
    int n = int(p.size());
    if (n == 0)
        return {};
    auto t = ManhattanMST::solve(p);
    vector<vector<int>> g(n);
    for (auto [u, v] : t.edges)
    {
        g[u].push_back(v);
        g[v].push_back(u);
    }
    vector<int> ord;
    ord.reserve(n);
    auto dfs = [&](auto &&self, int u, int fa) -> void
    {
        ord.push_back(u);
        for (int v : g[u])
        {
            if (v != fa)
                self(self, v, u);
        }
    };
    dfs(dfs, 0, -1);
    return ord;
}
```

65.3.1.2 普通莫队：静态区间统计

功能：普通莫队调度器：离线回答静态数组上的区间询问 $[l,r)$，调用方提供加入位置、删除位置、记录答案三个回调，适用于能快速增删一个元素的统计。

核心思路：按左端点分块，块号小者在前；同块内右端点按块号奇偶交替升序、降序（蛇形），避免右指针每换一块都从头回跳。依次处理询问时，当前区间 $[l,r)$ 先向外扩张两端再向内收缩，保证删除的位置一定仍在区间

内。全部结束后删除剩余元素，使统计回到空状态。

```
struct Mo
{
    struct Query
    {
        int l, r, id;
    };
    int n;
    vector<Query> q;

    Mo(int n) : n(n)
    {
        assert(n >= 0);
    }
    int add(int l, int r)
    {
        assert(0 <= l && l <= r && r <= n);
        int id = q.size();
        q.push_back({l, r, id});
        return id;
    }
    template <class Add, class Del, class Ans>
    void run(Add add, Del del, Ans ans, int block = 0) const
    {
        if (q.empty()) return;
        if (block == 0) block = max(1, int(n / sqrt(double(q.size()))));
        assert(block > 0);
        auto ord = q;
        sort(ord.begin(), ord.end(), [&](const Query &a, const Query &b)
        {
            int x = a.l / block;
            int y = b.l / block;
            if (x != y) return x < y;
            return x & 1 ? a.r > b.r : a.r < b.r;
        });
        int l = 0;
        int r = 0;
        for (auto a : ord)
        {
            while (l > a.l) add(--l);
            while (r < a.r) add(r++);
            while (l < a.l) del(l++);
            while (r > a.r) del(--r);
            ans(a.id);
        }
        while (l < r) del(l++);
    }
};
```

接口契约： Mo(n)，n≥0；add(l,r) 登记 0-based 半开区间并返回查询编号，允许空区间。run(add,del,ans,block=0) 中加删回调接收下标，答案回调接收原查询编号。开始时统计状态须为空，结束后删除剩余点恢复空状态；可重复 run，运行期间不能修改查询表。适用于加删任意位置语义相同的集合统计；逆序对等顺序敏感统计需另外区分左右端点。

查询按左端块号、右端蛇形排序；先扩张再收缩，保证删除位置仍在当前区间。回调 O(1) 时，正块长 B 的时间 O(q log(q+1)+qB+n^2/B+n)，额外空间 O(q)。默认 B=max(1,floor(n/sqrt(q)))，q=0 直接返回。n 和 q 须能用 int 表示。P2709 频次平方和须用 long long，最大 n^2=10^10。P2709 当前代码已 AC（30 个测试点，单点最大 69ms）；运行时间排名未核查。曼哈顿最小生成树排序：把查询 [l,r) 看作点 (l,r)，用 ManhattanMST 建树后按 DFS 先序处理查询，相邻查询的指针移动总和不超过 2 倍 MST 权；不保证比分块排序更快，一般题直接用默认分块。

模板题使用：P2709 莫队 / 小 B 的询问

题意：静态数组值域 1..k，询问闭区间内各值出现次数的平方和。n,m,k≤100000，输入 1-based 闭区间转成 0-based 半开区间；加删一个值由平方差 O(1) 更新，答案用 long long。

所需模板：Mo（第 693 页，65.3.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2709

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, k;
    cin >> n >> m >> k;
    vector<int> a(n);
    for (int &x : a) cin >> x;
    Mo mo(n);
    for (int i = 0; i < m; i++)
    {
        int l, r;
        cin >> l >> r;
        mo.add(l - 1, r);
    }
    vector<int> cnt(k + 1);
    vector<long long> ans(m);
    long long sum = 0;
    auto add = [&](int i)
    {
        sum += 2LL * cnt[a[i]] + 1;
        cnt[a[i]]++;
    };
    auto del = [&](int i)
    {
        cnt[a[i]]--;
        sum -= 2LL * cnt[a[i]] + 1;
    };
    mo.run(add, del, [&](int id)
    {
        ans[id] = sum;
    });
    for (long long x : ans) cout << x << '\n';
}
```

65.3.2 带修改莫队

65.3.2.1 带修改的莫队

功能：带修改莫队的调度器：离线登记“前 t 次修改完成后对区间 $[l,r)$ 的询问”，由调用方提供加点、删点、执行或撤销修改、记录答案四个回调，按优化顺序移动指针回答全部询问。

核心思路：把询问看作三维点 (l,r,t)：先按 l 所在块、再按 r 所在块、最后按 t 排序，第二、三关键字按奇偶交替升降（蛇形），减少指针来回跳动；块长默认取 $n^{2/3}$ 量级。处理每个询问时先移动时间指针：前进时执行第 t 次修改、后退时撤销，回调同时拿到当前区间，以便修正区间内的贡献；再先扩张后收缩地移动左右端点，最后调用 `ans`。

接口契约：ModifiedMo(n,m) 保存离线询问，n>0，m 为修改总数。add(l,r,t) 登记第 t 次修改完成后的 [l,r)，返回从 0 开始的询问编号；0≤l≤r≤n，0≤t≤m，询问数小于 INT_MAX。run(add,del,chg,ans,block=0) 排序并移动三个指针。add(p)/del(p) 按下标更新当前区间统计；chg(i,forward,l,r) 将编号 i 的修改前进或回退，并同步修正当前 [l,r) 内的贡献；ans(id) 记录结果。修改编号从 0 开始，前进第 i 次修改时原时间为 i，回退时原时间为 i+1。回调初始状态必须为原数组、空区间、时间 0；重复 run 前由调用方重置状态。状态不会自动恢复，原询问顺序不变。仅适用于可增删维护的区间统计，需区分左右端操作时另行拆分回调。默认块长约 n^(2/3)，也可显式指定正块长。n、询问数、修改数同阶且回调 O(1) 时总时间 O(n^(5/3))，调度器空间 O(q)。

```
struct ModifiedMo
{
    struct Query
    {
        int l, r, t, id;
    };

    int n, m;
    vector<Query> q;

    ModifiedMo(int n, int m) : n(n), m(m) { assert(n > 0 && m >= 0); }

    int add(int l, int r, int t)
    {
        assert(0 <= l && l <= r && r <= n);
        assert(0 <= t && t <= m);
        int id = q.size();
        q.push_back({l, r, t, id});
        return id;
    }
```

```
    template <class Add, class Del, class Chg, class Ans>
    void run(Add add, Del del, Chg chg, Ans ans, int block = 0) const
    {
        if (q.empty()) return;
        if (block == 0) block = max(1, int(pow(n, 2.0 / 3)));
        assert(block > 0);
        auto ord = q;
        auto cmp = [&](const Query &a, const Query &b)
        {
            int x = a.l / block, y = b.l / block;
            if (x != y) return x < y;
            int p = a.r / block, q = b.r / block;
            if (p != q) return (x & 1) ? p > q : p < q;
            return ((x ^ p) & 1) ? a.t > b.t : a.t < b.t;
        };
        sort(ord.begin(), ord.end(), cmp);
        int l = 0, r = 0, t = 0;
        for (auto q : ord)
        {
            while (t < q.t)
            {
                chg(t, true, l, r);
                t++;
            }
            while (t > q.t)
            {
                t--;
                chg(t, false, l, r);
            }
            while (l > q.l) add(--l);
            while (r < q.r) add(r++);
            while (l < q.l) del(l++);
            while (r > q.r) del(--r);
            ans(q.id);
        }
    }
};
```

模板题使用：Luogu P1903

题意：单点改色、区间不同颜色数，n,m≤133333，颜色 1..1000000。先读完操作，cur 仅用于记录每次修改前后的颜色，a 保持初始数组。题目闭区间 [L,R] 转为 [L-1,R)，询问时间是此前修改数。chg 必须先删旧贡献、改 a[p]、再加新贡献，回退使用 pre。按登记顺序输出。三维莫队，回调 O(1)，同阶规模时 O(n^(5/3))。

所需模板：ModifiedMo（第 696 页，65.3.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1903

Luogu P1903：使用代码

```
int main()
{
    int n, m;
    scanf("%d%d", &n, &m);
    vector<int> a(n);
    for (int &v : a) scanf("%d", &v);
    vector<int> cur = a;
    vector<array<int, 3>> upd;
    vector<array<int, 3>> qry;
    for (int i = 0; i < m; i++)
    {
        char op;
        int x, y;
        scanf(" %c%d%d", &op, &x, &y);
        x--;
        if (op == 'Q')
            qry.push_back({x, y, int(upd.size())});
        else
        {
            upd.push_back({x, cur[x], y});
            cur[x] = y;
        }
    }
    ModifiedMo mo(n, upd.size());
    for (auto [l, r, t] : qry) mo.add(l, r, t);
    vector<int> cnt(1000001), res(qry.size());
    int tot = 0;
    auto add = [&](int p)
    {
        if (cnt[a[p]]++ == 0) tot++;
    };
    auto del = [&](int p)
    {
        if (--cnt[a[p]] == 0) tot--;
    };
    auto chg = [&](int t, bool fwd, int l, int r)
    {
        auto [p, pre, nxt] = upd[t];
        bool in = l <= p && p < r;
        if (in) del(p);
        a[p] = fwd ? nxt : pre;
        if (in) add(p);
    };
    auto ans = [&](int id)
    {
        res[id] = tot;
    };
    mo.run(add, del, chg, ans);
    for (int v : res) printf("%d\n", v);
}
```

65.3.2.2 带修莫队：时间轴与区间状态

离线记录每个单点修改的下标、修改前值和修改后值；同一位置的旧值取上次修改后的数组。离散化须纳入全部初值与修改后值。询问时间 t 表示前 t 次修改已执行。维护 (l, r, t) 与半开区间 $[l, r)$；前进写 after、回退写 before，修改点在当前区间内时先删旧贡献、改值、再加新贡献。run 结束后状态停在最后一条询问，再次调用须重置。左块、右块、时间三关键字排序（蛇形），时间为真实修改次数。块长 B、询问 Q、修改 U 时移动量

$$O\left(QB + \frac{n^2}{B} + U\min\left(Q, (\lfloor n/B \rfloor + 1)^2\right)\right),$$

另加 $O(Q \log Q)$ 排序、$O(Q)$ 空间；同阶时 $B \approx n^{2/3}$，总 $O(n^{5/3})$。需区分左右端增删时应拆分回调。

65.3.3 树上莫队

65.3.3.1 树上莫队：点权与边权

功能： 树上莫队调度器：离线回答若干条树上路径 u–v 的统计询问（点权；或把边权记在子节点上的边权），调用方提供加点、删点、记录答案三个回调。

核心思路： DFS 生成括号序 `euler`（每个点进出各记一次，长 $2n$）。设 $\text{in}_u \le \text{in}_v$：若 u 是 v 的祖先，路径上的点恰是区间 $[\text{in}_u, \text{in}_v]$ 中出现奇数次的点；否则取 $[\text{out}_u, \text{in}_v]$，其中缺 LCA，记为 `extra`。边权询问正好相反：祖先情形要从区间中去掉 LCA，记为 `extra`，另一情形无需补。于是化为括号序上的普通莫队，移动端点时用 `toggle` 翻转点的在/不在状态，出现两次的点自动抵消；回答前后各翻转一次 `extra`。

代码依赖：HLD（第 399 页，39.3.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约： 先对非空连通树 HLD.build(root)，再 TreeMo(tree)；tree 须活到 run 结束且不能修改。点号 1..n，双欧拉序长 2n，n<=INT_MAX/2；DFS 递归。add(u,v,edge=false) 返回 0-based 查询 ID；默认点权，edge=true 把父边权记在子点并排除 LCA。run(add,del,ans,block=0) 的 add/del 参数是顶点，ans 参数是查询 ID；只适合与集合排列无关且可增删的统计。调用前统计为空，结束恢复空；不可在回调内修改树/查询，可重复运行。混用点边查询时回调须有一致的数据解释。默认块长 max(1,2n/sqrt(m))；自定 B>0。m>0 时调度 O(m log(m+1)+mB+n^2/B+n)，查询转换另需 O(m log n)，回调代价另计。空间 O(n+m)，复用 HLD；整数大小另须满足内存限制。

```
struct TreeMo
{
    struct Query
    {
        int l, r, extra, id;
    };

    const HLD &tree;
    vector<int> in, out, euler;
    vector<Query> q;

    TreeMo(const HLD &tree) : tree(tree), in(tree.n + 1), out(tree.n + 1)
    {
        assert(tree.n > 0 && tree.timer == tree.n);
        dfs(tree.rk[1], 0);
    }

    void dfs(int u, int p)
    {
        in[u] = euler.size();
        euler.push_back(u);
        for (int v : tree.g[u])
        {
            if (v == p) continue;
            dfs(v, u);
```

```
        }
        out[u] = euler.size();
        euler.push_back(u);
    }

    int add(int u, int v, bool edge = false)
    {
        assert(1 <= u && u <= tree.n && 1 <= v && v <= tree.n);
        if (in[u] > in[v]) swap(u, v);
        int w = tree.lca(u, v);
        int l = w == u ? in[u] : out[u];
        int extra = w == u ? 0 : w;
        if (edge) extra = w == u ? w : 0;
        int id = q.size();
        q.push_back({l, in[v] + 1, extra, id});
        return id;
    }
```

查询调度与状态恢复（接上页同一结构体）：

```
    template <class Add, class Del, class Ans>
    void run(Add add, Del del, Ans ans, int block = 0) const
    {
        if (q.empty()) return;
        int n = euler.size();
        if (block == 0) block = max(1, int(n / sqrt(double(q.size()))));
        assert(block > 0);
        auto ord = q;
        sort(ord.begin(), ord.end(), [&](const Query &a, const Query &b)
        {
            int x = a.l / block, y = b.l / block;
            if (x != y) return x < y;
            return (x & 1) ? a.r > b.r : a.r < b.r;
        });
        vector<bool> active(tree.n + 1);
        auto toggle = [&](int u)
        {
            if (active[u]) del(u);
            else add(u);
            active[u] = !active[u];
        };
        int l = 0, r = 0;
        for (auto a : ord)
        {
            while (l > a.l) toggle(euler[--l]);
            while (r < a.r) toggle(euler[r++]);
            while (l < a.l) toggle(euler[l++]);
            while (r > a.r) toggle(euler[--r]);
            if (a.extra) toggle(a.extra);
            ans(a.id);
            if (a.extra) toggle(a.extra);
        }
        for (int u = 1; u <= tree.n; u++)
        {
            if (active[u]) del(u);
        }
    }
};
```

65.3.4 回滚莫队

65.3.4.1 回滚莫队

功能：回滚莫队调度器：离线回答区间询问，统计信息只需支持插入和整体回滚到快照，不需要删除（例如区间内相同值的最远距离）。

核心思路：询问按左端点所在块分组，组内按右端点升序。对一块，令 `end` 为块的右边界，右指针 `r` 从 `end` 起只向右扩展，维护 $[\text{end}, r)$ 的统计。处理询问时先把 `r` 推到询问右端点，拍快照，再从 `end` 向左逐个插入到询问左端点，记录答案后回滚到快照，左侧插入因此不会累积。右端点不超出本块的短询问直接暴力插入再回滚；换块时回滚到最初的空快照。

思路中的情况	代码中的处理	行
进入新块	回滚到空状态，右指针重置为块右边界 `end`	43
询问在块内	拍快照，暴力插入 $[l, r)$，回答后回滚	49
右侧扩展	右指针单调右移到询问右端点，这部分保留	57
左侧临时插入	拍快照后从 `end` 向左插入到 l，回答后回滚	59

接口契约： RollbackMo(n)，n>=0；add(l,r) 登记 0-based 半开区间 [l,r)，允许空区间，返回 0-based 查询 ID。run(add, snapshot, rollback, ans, block=0)：add 接收位置，snapshot 返回当前快照，rollback 恢复该祖先快照，ans 接收查询 ID。开始时统计必须为空；结束恢复初始快照。每次 add 都须可撤销，包含长期右段；不保留失效快照、不修改查询。只支持与插入次序无关的区间统计。默认 B=max(1,n/sqrt(m))；m>0 时调度与 O(1) 增删总计 O(m log(m+1)+mB+n^2/B+n)，空间 O(m) 加回调状态/历史 O(n)。可重复 run，n=0 只允许空查询；无查询不调用回调。

```
struct RollbackMo
{
    struct Query
    {
        int l, r, id;
    };

    int n;
    vector<Query> q;

    RollbackMo(int n) : n(n)
    {
        assert(n >= 0);
    }

    int add(int l, int r)
    {
        assert(0 <= l && l <= r && r <= n);
        int id = q.size();
        q.push_back({l, r, id});
        return id;
    }

    template <class Add, class Snap, class Undo, class Ans>
    void run(Add add, Snap snapshot, Undo rollback, Ans ans, int block = 0) const
    {
        if (q.empty()) return;
        if (block == 0) block = max(1, int(n / sqrt(double(q.size()))));
        assert(block > 0);
        auto ord = q;
        sort(ord.begin(), ord.end(), [&](const Query &a, const Query &b)
        {
            int x = a.l / block, y = b.l / block;
            if (x != y) return x < y;
            return a.r < b.r;
        });
```

查询调度与状态恢复（接上页同一结构体）：

```
        auto empty = snapshot();
        int last = -1, r = 0;
        for (auto a : ord)
        {
            int b = a.l / block;
            int end = a.l + min(block - a.l % block, n - a.l);
            if (b != last)
            {
                rollback(empty);
                last = b;
                r = end;
            }
            if (a.r <= end)
            {
                auto saved = snapshot();
                for (int i = a.l; i < a.r; i++) add(i);
                ans(a.id);
                rollback(saved);
                continue;
            }
            while (r < a.r) add(r++);
            auto saved = snapshot();
            for (int i = end; i > a.l; ) add(--i);
            ans(a.id);
            rollback(saved);
        }
        rollback(empty);
    }
};

struct TreeMo
{
    struct Query
    {
        int l, r, extra, id;
    };

    const HLD &tree;
    vector<int> in, out, euler;
    vector<Query> q;

    TreeMo(const HLD &tree) : tree(tree), in(tree.n + 1), out(tree.n + 1)
    {
        assert(tree.n > 0 && tree.timer == tree.n);
        dfs(tree.rk[1], 0);
    }

    void dfs(int u, int p)
    {
        in[u] = euler.size();
        euler.push_back(u);
        for (int v : tree.g[u])
        {
            if (v == p) continue;
```

```
            dfs(v, u);
        }
        out[u] = euler.size();
        euler.push_back(u);
    }

    int add(int u, int v, bool edge = false)
    {
        assert(1 <= u && u <= tree.n && 1 <= v && v <= tree.n);
        if (in[u] > in[v]) swap(u, v);
        int w = tree.lca(u, v);
        int l = w == u ? in[u] : out[u];
        int extra = w == u ? 0 : w;
        if (edge) extra = w == u ? w : 0;
        int id = q.size();
        q.push_back({l, in[v] + 1, extra, id});
        return id;
    }

    template <class Add, class Del, class Ans>
    void run(Add add, Del del, Ans ans, int block = 0) const
    {
        if (q.empty()) return;
        int n = euler.size();
        if (block == 0) block = max(1, int(n / sqrt(double(q.size()))));
        assert(block > 0);
        auto ord = q;
        sort(ord.begin(), ord.end(), [&](const Query &a, const Query &b)
        {
            int x = a.l / block, y = b.l / block;
            if (x != y) return x < y;
            return (x & 1) ? a.r > b.r : a.r < b.r;
        });
        vector<bool> active(tree.n + 1);
        auto toggle = [&](int u)
        {
            if (active[u]) del(u);
            else add(u);
            active[u] = !active[u];
        };
        int l = 0, r = 0;
        for (auto a : ord)
        {
            while (l > a.l) toggle(euler[--l]);
            while (r < a.r) toggle(euler[r++]);
            while (l < a.l) toggle(euler[l++]);
            while (r > a.r) toggle(euler[--r]);
            if (a.extra) toggle(a.extra);
            ans(a.id);
            if (a.extra) toggle(a.extra);
        }
        for (int u = 1; u <= tree.n; u++)
        {
            if (active[u]) del(u);
        }
    }
```

```
};
```

模板题使用：P5906 【模板】回滚莫队 & 不删除莫队

题意：n,m<=200000，值 1..2e9；对每个 1-based 闭区间输出相同值的最远下标差，无重复输出 0。离散化，保存各值最左/最右位置和全局答案，历史逐次恢复。

所需模板：RollbackMo（第 701 页，65.3.4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5906

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<int> a(n), values;
    for (int &x : a) cin >> x;
    values = a;
    sort(values.begin(), values.end());
    values.erase(unique(values.begin(), values.end()), values.end());
    for (int &x : a)
    {
        x = lower_bound(values.begin(), values.end(), x) - values.begin();
    }
    int m;
    cin >> m;
    RollbackMo mo(n);
    for (int i = 0; i < m; i++)
    {
        int l, r;
        cin >> l >> r;
        mo.add(l - 1, r);
    }
```

继续处理操作（接上页同一 main）：

```
    struct Change
    {
        int color, first, last, best;
    };
    vector<Change> history;
    vector<int> first(values.size(), n), last(values.size(), -1), answer(m);
    int best = 0;
    auto add = [&](int i)
    {
        int c = a[i];
        history.push_back({c, first[c], last[c], best});
        first[c] = min(first[c], i);
        last[c] = max(last[c], i);
        best = max(best, last[c] - first[c]);
    };
    auto snapshot = [&]()
    {
        return history.size();
    };
    auto rollback = [&](size_t saved)
    {
        while (history.size() > saved)
        {
            auto h = history.back();
            history.pop_back();
            first[h.color] = h.first;
            last[h.color] = h.last;
            best = h.best;
        }
    };
    mo.run(add, snapshot, rollback, [&](int id)
    {
        answer[id] = best;
    });
    for (int x : answer) cout << x << '\n';
}
```

65.3.5 莫队二次离线

65.3.5.1 莫队二次离线：异或位数配对

问题描述：给定数组 a 和整数 k，多次询问区间 $[l,r)$ 内有多少对下标 $i<j$ 满足 $a_i \oplus a_j$ 的二进制恰有 k 个 1。

核心思路：莫队二次离线。设 $c(i,P)$ 为前缀 $[0,P)$ 中与 a_i 配对成功的下标数。莫队每把端点移过一段下标，答案变化都是若干项 $\pm\big(c(i,i)-c(i,P)\big)$ 之和，P 为另一端点（左端移动且 $k=0$ 时还要扣掉自身配对）。$c(i,i)$ 与询问无关，用前缀和 `prefix` 直接加减；$c(i,P)$ 则作为事件“下标段对前缀 P”挂到 `events[P]`。第二遍从左到右扫描前缀，`count[v]` 为已扫前缀中与 v 配对的个数（每加入一个数，枚举所有含 k 个 1 的掩码更新），逐个回答事件。得到的是莫队顺序下相邻询问的答案差，最后按该顺序求前缀和。

思路中的情况	代码中的处理	行
预处理 $c(i,i)$	按下标扫描，前缀和 `prefix` 累计每个数与前面数的配对数	21
莫队排序	按左端点分块，块内右端点奇偶交替升降	24
右端点右移	加上 `prefix` 差，减去事件“新增段对前缀 `l`”	47
左端点左移	减去 `prefix` 差，加上事件“新增段对前缀 `r`”	53
第二次离线扫描	`count` 加入 a_{p-1} 后，回答挂在前缀 p 上的事件	77
扣除自身配对	$k=0$ 时 a_i 与自身也满足条件，位于前缀内则减一	80
还原答案	按莫队顺序对答案差做前缀和	87

接口契约： xor_hamming_pairs(a,queries,k,bits=14) 按查询输入顺序返回 long long 答案，统计 l≤i<j<r 且 popcount(a[i] xor a[j])=k 的下标对。queries 为 0-based 半开区间，允许空区间、重复查询与空数组；0≤bits≤20，0≤a[i]<2^bits，n、m 和容器下标须可由 int 表示。k 超出 [0,bits] 直接返回全零；相等值来自不同下标才构成一对。莫队移动生成前缀事件，第二次离线按前缀扫描；k=0 显式扣掉自身。令 C=binom(bits,k)，时间 O(2^bits+nC+m log(m+1)+n$\sqrt{\text{m}}$+m)，空间 O(2^bits+n+m)，m=0 直接返回。无需维护可逆的在线窗口，也不支持修改。

```cpp
// Queries are 0-based half-open; count unordered pairs of distinct indices.
vector<long long> xor_hamming_pairs(const vector<int> &a,
                                    const vector<pair<int, int>> &queries,
                                    int k,
                                    int bits = 14)
{
    int n = a.size(), m = queries.size();
    vector<long long> answer(m);
    assert(0 <= bits && bits <= 20);
    int universe = 1 << bits;
    for (int x : a) assert(0 <= x && x < universe);
    for (auto [l, r] : queries) assert(0 <= l && l <= r && r <= n);
    if (!m || k < 0 || k > bits) return answer;
    vector<int> masks;
    for (int x = 0; x < universe; x++)
        if (popcount(unsigned(x)) == k) masks.push_back(x);
    vector<int> count(universe);
    vector<long long> prefix(n + 1);
    for (int i = 0; i < n; i++)
    {
        prefix[i + 1] = prefix[i] + count[a[i]];
        for (int mask : masks) count[a[i] ^ mask]++;
    }
    int block = max(1, int(n / sqrt(m)));
    vector<int> order(m);
    iota(order.begin(), order.end(), 0);
    sort(order.begin(),
         order.end(),
         [&](int x, int y)
         {
             int bx = queries[x].first / block, by = queries[y].first / block;
             if (bx != by) return bx < by;
             return bx & 1 ? queries[x].second > queries[y].second
                           : queries[x].second < queries[y].second;
         });

    struct Event
    {
        int id, l, r, sign;
    };

    vector<vector<Event>> events(n + 1);
```

莫队端点移动与前缀事件（接上页同一函数）：

```
    int l = 0, r = 0;
    for (int id : order)
    {
        auto [ql, qr] = queries[id];
        if (r < qr)
        {
            events[l].push_back({id, r, qr, -1});
            answer[id] += prefix[qr] - prefix[r];
            r = qr;
        }
        if (l > ql)
        {
            events[r].push_back({id, ql, l, 1});
            answer[id] -= prefix[l] - prefix[ql];
            l = ql;
        }
        if (r > qr)
        {
            events[l].push_back({id, qr, r, 1});
            answer[id] -= prefix[r] - prefix[qr];
            r = qr;
        }
        if (l < ql)
        {
            events[r].push_back({id, l, ql, -1});
            answer[id] += prefix[ql] - prefix[l];
            l = ql;
        }
    }
```

第二次离线与差分还原（接上页同一函数）：

```
    fill(count.begin(), count.end(), 0);
    for (int p = 0; p <= n; p++)
    {
        if (p)
            for (int mask : masks) count[a[p - 1] ^ mask]++;
        for (auto e : events[p])
            for (int i = e.l; i < e.r; i++)
            {
                int value = count[a[i]] - (k == 0 && i < p);
                answer[e.id] += 1LL * e.sign * value;
            }
    }
    long long total = 0;
    for (int id : order)
    {
        total += answer[id];
        answer[id] = total;
    }
    return answer;
}
```

模板题使用：Luogu P4887

题意：统计每个区间内异或值恰有 k 个 1 的无序下标对。输入 1-based 闭区间 [l,r]，只将 l 减一转为 0-based 半开区间；k 可以大于 14，此时全零。k=0 统计相等值的不同下标，不计自身。

所需模板：xor_hamming_pairs（第 706 页，65.3.5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4887

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, k;
    cin >> n >> m >> k;
    vector<int> a(n);
    for (int &x : a) cin >> x;
    vector<pair<int, int>> queries(m);
    for (auto &[l, r] : queries)
    {
        cin >> l >> r;
        l--;
    }
    for (long long x : xor_hamming_pairs(a, queries, k)) cout << x << '\n';
}
```

65.3.5.2 莫队二次离线：前缀事件与自身扣除

区间内 $H(i,j)=[\text{popcount}(a_i \text{xor} a_j)=k]$ 的无序对数，用 `xor_hamming_pairs`。$t(i)=\sum_{j<i} H(i,j)$，$T(p,i)=\sum_{j<p} H(i,j)-[k=0 \wedge i<p]$；右扩贡献 $t(i)-T(L,i)$，左扩 $T(R,i)-t(i)$，删除取反；t 用前缀和，其余按前缀长度存事件顺扫。0-based 半开区间，允许空区间；位宽默认 14，k 超位宽答案全零。$C=\binom{b}{k}$，时间 $O(2^b+nC+m\log(m+1)+n\sqrt{m}+m)$，空间 $O(2^b+n+m)$；答案用 long long。

第六十六章　分数规划

66.1　0/1 分数规划：Dinkelbach

问题描述： 0/1 分数规划：在所有可行方案中最大化 $\sum A/\sum B$（要求 $\sum B>0$）；方案的结构由调用方的回调 `best` 描述。

核心思路： 对给定 λ，$\max(A-\lambda B)>0$ 当且仅当最优比值大于 λ。Dinkelbach 迭代：用 `best` 求出使 $A-\lambda B$ 最大的方案，若该值已不超过（相对的）`eps`，当前 λ 就是最优比值；否则令 λ 为这个方案的比值 A/B，它严格大于旧的 λ，继续迭代。

接口契约： dinkelbach(best,lambda,eps=1e-10) 求可行方案中 $\sum$A/$\sum$B 的最大值，要求所有可行方案 $\sum$B>0。best(λ) 返回使 A-λB 最大的方案的 {A,B}（double）；初值 lambda 取任一可行方案的比值，不能随意取 0（所有比值为负时会出错）。每轮 $\lambda\leftarrow$A/B 单调上升，A-λB$\le$eps $\cdot$ max(1,|B|) 时返回。迭代次数一般远少于二分；最小化比值时把 a 取负或回调改为求最小并反转比较。原理、二分写法与常见组合见知识节。

```cpp
// Maximize (sum a)/(sum b) over feasible choices with sum b > 0.
// best(lambda) returns {A, B} of a choice maximizing A - lambda * B; lambda starts at the
// ratio of any feasible choice. Lambda increases every round and stops at the optimum.
template <class F> double dinkelbach(F best, double lambda, double eps = 1e-10)
{
    while (true)
    {
        auto [A, B] = best(lambda);
        if (A - lambda * B <= eps * max(1.0, fabs(B))) return lambda;
        lambda = A / B;
    }
}
```

66.2　0/1 分数规划

问题：在可行方案集合中最大化 $\dfrac{\sum a_i x_i}{\sum b_i x_i}$，$x_i\in\{0,1\}$，要求所有可行方案 $\sum b_i x_i>0$。

转化：令 $F(\lambda)=\max\left(\sum a_i x_i-\lambda\sum b_i x_i\right)$，它关于 λ 严格单调递减，最优比值 λ^* 恰为 $F(\lambda)=0$ 的根：$F(\lambda)\ge 0\iff\lambda\le\lambda^*$。求最小比值时把 max 换成 min、不等号反向。

二分：在答案范围内二分 λ，每次求一次 $F(\lambda)$ 判断正负；实数二分固定 60 到 100 轮，或按题目精度停止。总复杂度为轮数乘一次判定。

Dinkelbach：从任一可行方案的比值出发，取使 $F(\lambda)$ 达到最大的方案 (A,B)，令 $\lambda\leftarrow A/B$；λ 单调上升，$F(\lambda)\le$ 精度时即为最优，迭代次数通常远少于二分。模板见第 711 页，回调只需返回该方案的 (A,B)。

常见组合：

- 恰选 k 个：按 $a_i-\lambda b_i$ 排序取前 k 个，$O(n\log n)$ 判定。
- 最优比率生成树：边权取 $a_e-\lambda b_e$，求最大（或最小）生成树。
- 最优比率环（最小化环上 $\sum a/\sum b$）：边权取 $a_e-\lambda b_e$，存在负环 $\iff\lambda>$ 最优比值，用第 435 页 SPFA 判负环后二分；Dinkelbach 需要求出最负的环，通常直接二分。
- 带上界约束的选择（背包、树上连通块）：判定内部换成对应 DP，DP 的目标改成 $\sum(a-\lambda b)$。

注意：b_i 可正可负时单调性不成立，不能直接套用；判定必须求出“该 λ 下的最优方案”，不能用贪心近似。整数数据要精确答案时，可在最终方案上用分数 A/B 输出。

第六十七章　表达式求值

67.1　整数表达式求值

问题描述：给定含非负整数、加减乘除、取模、乘方、括号与一元正负号的表达式字符串，按通常的优先级和结合性计算其整数值。

核心思路：递归下降，每一级优先级对应一个函数：`expr` 处理加减、`term` 处理乘除模，都用循环实现左结合；`unary` 处理一元正负号；`power` 先读一个 `atom`（数字或括号内的 `expr`），遇到乘方号再以 `unary` 读指数，由此乘方右结合，且比一元负号结合更紧（$-2^2=-4$），指数用快速幂计算。`peek` 跳过空格看下一个字符，决定进入哪个分支。

接口契约：Expression(s).eval() 计算整数表达式 s 的值（long long）。支持非负整数字面量、+ - * / % ^、括号与一元正负号，空格忽略。优先级从高到低：^（右结合，指数须非负，0^0=1）、一元正负、* / %、+ -，同级左结合；/ 和% 按 C++ 截断，除数不能为 0。递归下降，O(|s|) 时间，递归深度等于括号与一元符号的嵌套深度。所有中间结果须在 long long 内；需要取模时把各运算改成模运算即可。非法输入触发 assert。

```
// Integer expression: + - * / % ^, parentheses, unary + -, spaces ignored.
// Precedence: ^ (right-assoc) > unary sign > * / % > + -; / and % truncate as in C++.
struct Expression
{
    using ll = long long;
    string s;
    size_t i = 0;

    explicit Expression(string text) : s(move(text)) {}

    char peek()
    {
        while (i < s.size() && s[i] == ' ') i++;
        return i < s.size() ? s[i] : 0;
    }

    ll atom()
    {
        if (peek() == '(')
        {
            i++;
            ll v = expr();
            assert(peek() == ')');
            i++;
            return v;
        }
        assert(isdigit(peek()));
        ll v = 0;
        while (i < s.size() && isdigit(s[i])) v = v * 10 + (s[i++] - '0');
        return v;
    }

    ll power()
    {
        ll a = atom();
        if (peek() != '^') return a;
```

```
        i++;
        ll e = unary(), r = 1; // right-assoc: a^b^c = a^(b^c)
        assert(e >= 0);
        while (e)
        {
            if (e & 1) r *= a;
            if (e >>= 1) a *= a; // no square after the last bit
        }
        return r;
    }

    ll unary()
    {
        char c = peek();
        if (c == '+' || c == '-')
        {
            i++;
            ll v = unary();
            return c == '-' ? -v : v;
        }
        return power();
    }

    ll term()
    {
        ll v = unary();
        for (char c; (c = peek()) == '*' || c == '/' || c == '%';)
        {
            i++;
            ll b = unary();
            assert(c == '*' || b != 0);
            v = c == '*' ? v * b : c == '/' ? v / b : v % b;
        }
        return v;
    }

    ll expr()
    {
        ll v = term();
        for (char c; (c = peek()) == '+' || c == '-';)
        {
            i++;
            ll b = term();
            v = c == '+' ? v + b : v - b;
        }
        return v;
    }
```

```

    // Value of the whole string; the caller keeps every intermediate result within long long.
    ll eval()
    {
        i = 0;
        ll v = expr();
        assert(peek() == 0);
        return v;
    }
};
```

第六十八章　珂朵莉树/颜色段均摊

68.1　珂朵莉树（颜色段均摊）

功能：珂朵莉树：维护分段常值的序列，支持把区间推平为同一值，以及按段遍历区间（在回调中修改段值、求和、求第 k 小等）。

核心思路：`mp` 以段起点为键、段值为值，键 hi 作哨兵结束最后一段。`split(x)` 找到包含 x 的段，若 x 不是段起点就插入一个以 x 为起点、值相同的新段，使 x 成为分界。`assign` 先切出 r 与 l，删除其间所有段后插入一个新段；`each` 切出区间后逐段调用回调。推平不断把多段合成一段，随机数据下段数保持很少。

接口契约：ODT<T>(lo,hi,v) 维护半开区间 [lo,hi) 上的分段常值序列，初值 v；mp 的键为段起点，键 hi 为哨兵。split(x) 使 x (lo≤x≤hi) 成为段起点并返回其迭代器；assign(l,r,v) 把 [l,r) 推平为 v (l≤r，空区间不变)；each(l,r,f) 依次对 [l,r) 内每段调用 f(段起点, 段终点, 值引用)，可在 f 中修改值、统计和、收集 (值, 长度) 求第 k 小或幂和。map 插入不使已有迭代器失效，故 split 的先后无关；经典 set 写法（删除旧段再插入两段）必须先 split(r) 再 split(l)。复杂度依赖推平操作：数据随机且推平占常数比例时段数期望 O(log n)，总复杂度约 O(m log n)；没有推平或被构造数据卡时会退化为 O(n) 每次。

```
// Chtholly tree on [lo, hi): map keys are segment starts, the key hi closes the last segment.
template <class T> struct ODT
{
    map<long long, T> mp;

    ODT(long long lo, long long hi, T v = T())
    {
        mp[lo] = v;
        mp[hi] = T();
    }

    // Make x a segment start (lo <= x <= hi) and return its iterator.
    typename map<long long, T>::iterator split(long long x)
    {
        auto it = prev(mp.upper_bound(x));
        if (it->first == x) return it;
        return mp.emplace_hint(next(it), x, it->second);
    }

    // Set [l, r) to v, l <= r.
    void assign(long long l, long long r, T v)
    {
        auto R = split(r), L = split(l);
        mp.erase(L, R);
        mp.emplace_hint(R, l, v);
    }

    // f(l, r, value&) for every segment inside [l, r); f may change the value.
    template <class F> void each(long long l, long long r, F f)
    {
        auto R = split(r), L = split(l);
        for (; L != R; ++L) f(L->first, next(L)->first, L->second);
    }
};
```

算法索引

AdaptiveSimpson, 247
AhoCorasick, 43
Arborescence, 461

ballot theorem, 193
basis_intersection, 227
basis_sum_intersection, 229
batch_inverse, 105
BellmanFord, 432
Berlekamp–Massey, 231
Berlekamp-Massey, 188
berlekamp_massey, 259
bernoulli, 204
Bezout identity, 102
BiconnectedCore, 478
BigInteger, 79
Binomial, 189
BipartiteMatching, 544
block_cut_forest, 487
Blossom, 551
BostanMori, 180
BoundedCirculation, 534
BoundedMaxFlow, 535
bridge_component_forest, 482
Burnside, 213
BurnsideAverage, 210

Cactus, 482
Carmichael, 104
CartesianTree, 346
Catalan, 193
Cayley formula, 566
Cayley–Hamilton, 231
cc_hash_table, 262
cdq_convolution, 149
CentroidDiameter, 419
CentroidPairs, 410
CentroidSum, 414
chirp_z, 160
Chomp, 242
Cipolla, 123
circle_coverage_area, 627
circle_intersections, 591
circle_intersections_i64, 609

circle_overlap_area, 594
circle_union_area, 622
CirclePolygon, 611
CircleTangents, 613
ClosestPair, 676
ComplexFFT, 144
conditional expectation, 239
convex_contains_i64, 660
convex_cut, 595
convex_diameter2, 667
convex_tangents_i64, 657
convolution_i64, 154
convolution_mod_fft, 146
coprime pairs, 132
count_four_cycles, 489
CRT, 93
crt_merge, 109

dag_path_determinant, 577
determinant_mod, 220
Dijkstra, 431
Dinic, 518
Dinkelbach, 711
dinkelbach, 711
DirectedEuler, 502
Dirichlet convolution, 131
DiscreteLog, 127
divisor sum, 131
Dominance3D, 686
DominatorTree, 579
dsu, 264
Du Jiao sieve, 133
DuJiao, 132
DynamicDiameter, 384
DynamicDP, 20
DynamicKth, 365

EGF, 190
EnclosingCircle, 679
enumerate_triangles, 491
Euler characteristic, 656
Euler theorem, 93
Euler totient, 96
euler_phi, 96
euler_power, 103
ExactCover, 2
exCRT, 113
exkmp, 41

ExLucas, 117
exLucas, 120
Expression, 712
extended_gcd, 101

Fenwick, 284
floor moments, 143
floor quotient grouping, 130
floor sum, 142
floor_moments, 140
floor_sum, 138
Floyd, 437
FpsFunctions, 174
FpsInverse, 162
FpsPower, 176
FpsSqrt, 171
FunctionalGraph, 471

Garner, 113
garner, 111
GaussMod, 252
GaussReal, 255
GaussXor, 250
GCD 加权求和, 131
GeneralSAM, 62
generating functions, 188
geometric series, 131
gomory_hu, 525
gp_hash_table, 262
gp_map, 262
grid_cycle, 11
grid_cycles, 16

HLD, 399
HLPP, 512
Hull3D, 632
Hungarian, 549

inclusion-exclusion, 193
independent_set, 549
indicator variable, 239
integer partitions, 209
integer_hull, 661
IntegerGeometry3D, 643
IntegerHalfplanes, 672
IntegerPlane, 602
inverse_table, 107

KDNearest, 372
KDTreeSum, 369

kmp_match, 39
Knuth 优化, 29
knuth_interval_dp, 31
Korselt criterion, 104
Kruskal, 455
KruskalTree, 444
KShortestWalks, 441
KthResidue, 129
Kummer theorem, 115

Lagrange, 243
Lagrange interpolation, 120
Lagrange inversion, 187
lagrange_consecutive, 246
lattice polygon, 656
lazy_segtree, 289
LCT
    虚子树, 380
LeftistHeap, 276
Legendre formula, 115
LiChao, 317
lifting the exponent, 114
LiftingLCA, 392
Line3, 648
line_circle_i64, 607
line_circle_intersections, 589
line_intersection_real, 587
line_projection, 586
linear congruence, 108
linear sieve, 98
linear_congruence, 108
LinearSieve, 98
LinkCutTree, 375
LongChain, 395
LTE, 114
Lucas, 116, 120
Lucas theorem, 93
lyndon, 76

manacher, 69
Manhattan MST, 460
manhattan_order, 693
ManhattanMST, 448
matching_edges, 547
Matrix Tree theorem, 219
matrix_inverse, 217
MatrixTreeMod, 573
maximum_closure, 523

MaxPlusMatrix, 219
MergeSplitTree, 303
mex, 240
Miller–Rabin, 95
Min25, 134
MinCostFlow, 527
mincut_edges, 524
MinCycle, 495
minimum spanning tree, 460
minimum_mean_cycle, 493
minimum_rectangle, 669
minimum_rotation, 74
MinimumCover, 6
minkowski_sum, 662
mint, 86
mixed_euler_orientation, 509
mixed_euler_trail, 511
Mo, 693
Mobius inversion, 131
Mod64, 85
mod_inverse, 104
mod_sqrt, 121
ModifiedMo, 696
ModInt, 90
ModMatrix, 215
Monge, 27
monotone_dp_layer, 28
MonotoneHull, 24
MonotoneStackSeg, 300
multiplicative order, 126
MultipointEvaluation, 166

necklace_colorings, 212
NegativeCostFlow, 530
Nim, 240
Nim-k, 241
NttConvolution, 151
numerical stability, 250

ODT, 715
offline_rectangle_sum, 285
OGF, 190
OrderedSplay, 339
OrderedTreap, 329
ost, 326
OverallKth, 689

p-adic valuation, 114
PalindromicTree, 71

Partitions, 206
parts_at_most, 207
parts_at_most_fps, 208
path_intersection, 391
pentagonal number theorem, 209
permutation_cycles, 211
PersistentDSU, 353
PersistentKth, 348
PersistentOrderedTreap, 357
PersistentRange, 350
PersistentXorTrie, 363
pheap, 274
Pick theorem, 656
Plane3, 651
Pollard–Rho, 95
PollardRho, 99
Polya, 213
polygon_area2, 605
polygon_centroid, 597
polygon_contains, 606
polygon_union_area, 615
polynomial_interpolation, 169
polynomial_shift, 184
PolynomialDivision, 164
PositionBasis, 224
PotentialDSU, 268
prefix_function, 38
Prim, 458
Prime64, 94
PrimitiveRoot, 123
Prüfer sequence, 566

quadrangle inequality, 27
quadratic residue, 123

rank, 219
RealPlane, 584
RealPolarLess, 599
RealSpace, 644
rectangle_union_area, 665
RectangleFenwick, 282
recurrence_nth, 179
reflection principle, 193
regular_matching, 541
Richardson extrapolation, 250
RollbackDSU, 266
RollbackMo, 701
rp, 328

SAIS, 47
sam_lcs, 61
SAMDocuments, 65
SAMLex, 58
SecondMST, 450
segment_distance_real, 654
SegmentBeats, 320
segmented sieve, 99
segmented_primes, 97
SegmentLiChao, 312
segtree, 295
SequenceTreap, 333
SetConvolution, 156
Simplex, 233
Simpson integration, 250
SJ 定理, 241
Spfa, 435
Sprague-Grundy, 240
StaticRMQ, 278
SteinerTree, 465
Stirling numbers, 193, 195
stirling_first_column, 198
stirling_first_row, 196
stirling_second_column, 201
stirling_second_row, 199
StoerWagner, 538
StrictSecondShortest, 440
StringHash, 35
subset_convolution, 158
SubtreeColors, 404
SuffixArray, 52
SuffixAutomaton, 56
SuffixLCP, 54

tail sum, 239
TarjanSCC, 475
TimeConnectivity, 270
Treap
    位翻转, 338
tree_hash, 428
tree_hash_reroot, 430
TreeDiameter, 388
TreeIsomorphism, 424
TreeMo, 699
triangle_incircle, 598
tutte, 568
TwoSAT, 500

UndirectedEuler, 506

VirtualTree, 407

WeightedBlossom, 555
Widget Factory, 257
wqs_independent_set, 33

xor_hamming_pairs, 706
XorBasis, 222

z_function, 40

上升阶乘, 195
下降阶乘, 195
二元一次不定方程, 102
二分图博弈, 242
二项式反演, 192
互质数对, 131
伯努利数, 204

决策单调性, 27
    分治, 27
    区间, 29
分数规划, 711
分治优化, 27
动态树 DP, 380
区间筛素数, 99
十二重计数, 190
升幂引理, 114
原根, 126
反 Nim, 241
四边形不等式, 27
复合逆, 187
威佐夫博弈, 240
容斥原理, 192
对称策略, 242
巴什博弈, 240
带修莫队, 699
开放时间最短路, 439
形式留数, 187

恰好计数, 192
扩展中国剩余定理, 113
批量逆元, 107
排列数, 115
斐波那契博弈, 241
斯特林反演, 203
斯特林数, 195
无序数对, 131
替罪羊树, 345
有限状态计数, 231

树上删边, 241
树上路径交, 395
模高斯消元, 257
欧拉数, 205
混合进制, 113
满射, 192

球盒问题, 190
矩形 GCD 计数, 132
策略窃取, 242
筛法选型, 130
类欧几里德, 142
线性同余方程, 108
线性逆元表, 107
线性递推, 182, 231
群作用, 213

自然数幂和, 204
范德蒙德卷积, 209
计数建模, 190
贝尔数, 204
贪吃蛇, 439

错位排列, 193
阶乘表, 115
阶梯博弈, 241
隐式生成函数, 187